THE EXPERT'S VOICE®

SEGUNDA EDIÇÃO

Pro Git

TUDO O QUE VOCÊ PRECISA SABER SOBRE GIT

Scott Chacon and Ben Straub

Apress®

Pro Git

Scott Chacon, Ben Straub

Version 2.1.50, 2026-08-20

Table of Contents

Licença . . . 1
Prefácio por Scott Chacon . . . 2
Prefácio por Ben Straub . . . 3
Dedicatórias . . . 4
Colaboradores . . . 5
Introdução . . . 7
Primeiros Passos . . . 9
- Sobre Controle de Versão . . . 9
- Uma Breve História do Git . . . 13
- O que é Git? . . . 13
- A Linha de Comando . . . 17
- Instalando o Git . . . 17
- Configuração inicial do Git . . . 21
- Obtendo Ajuda . . . 24
- Resumo . . . 25

Fundamentos do Git . . . 26
- Obtendo um Repositório Git . . . 26
- Gravando Alterações no Repositório . . . 28
- Visualizando o Histórico de Commits . . . 40
- Desfazendo Coisas . . . 47
- Trabalhando com Repositórios Remotos . . . 51
- Criando Tags . . . 57
- Aliases (Apelidos) no Git . . . 62
- Resumo . . . 63

Ramificação (Branching) no Git . . . 64
- Branches em Poucas Palavras . . . 64
- Ramificação (Branching) e Mesclagem (Merging) Básicas . . . 71
- Gerenciamento de Branches . . . 80
- Fluxos de Trabalho de Branches (Branching Workflows) . . . 84
- Branches Remotos (Remote Branches) . . . 87
- Rebase (Rebasing) . . . 97
- Resumo . . . 107

Git no Servidor . . . 108
- Os Protocolos . . . 108
- Colocando o Git em um Servidor . . . 113
- Gerando a sua Chave Pública SSH . . . 116
- Configurando o Servidor . . . 117
- Git Daemon . . . 120

Smart HTTP . . . 121
GitWeb . . . 123
GitLab . . . 125
Opções de Hospedagem de Terceiros . . . 129
Resumo . . . 129
Git Distribuído . . . 130
Fluxos de Trabalho Distribuídos . . . 130
Contribuindo com um Projeto . . . 133
Mantendo um Projeto . . . 157
Resumo . . . 171
GitHub . . . 172
Configuração e Ajustes de Conta . . . 172
Contribuindo para um Projeto . . . 178
Mantendo um Projeto . . . 198
Gerenciando uma Organização . . . 213
Criando Scripts para o GitHub . . . 216
Resumo . . . 226
Ferramentas do Git . . . 227
Seleção de Revisão . . . 227
Área de Stage Interativa . . . 235
Fazendo Stash e Limpando . . . 239
Assinando Seu Trabalho . . . 245
Pesquisando (Searching) . . . 249
Reescrevendo o Histórico . . . 253
Reset Desmistificado . . . 261
Merging Avançado . . . 283
Rerere . . . 300
Depurando com Git . . . 307
Submódulos . . . 310
Empacotando (Bundling) . . . 331
Substituir (Replace) . . . 335
Armazenamento de Credenciais . . . 343
Resumo . . . 348
Customizando o Git . . . 349
Configuração do Git . . . 349
Atributos do Git . . . 360
Hooks do Git . . . 368
Uma Política de Exemplo Imposta pelo Git . . . 372
Resumo . . . 381
Git e Outros Sistemas . . . 382
Git como Cliente . . . 382

Migrando para o Git . . . 416
Resumo . . . 431
Git Internals (Por Dentro do Git) . . . 432
Encanamento e Porcelana (Plumbing and Porcelain) . . . 432
Objetos do Git . . . 433
Referências do Git . . . 443
Arquivos de pacote (Packfiles) . . . 447
Refspec . . . 450
Protocolos de Transferência . . . 453
Manutenção e Recuperação de Dados . . . 459
Variáveis de Ambiente . . . 466
Resumo . . . 472
Appendix A: O Git em Outros Ambientes . . . 473
Interfaces Gráficas . . . 473
Git no Visual Studio . . . 479
Git no Visual Studio Code . . . 479
Git no IntelliJ / PyCharm / WebStorm / PhpStorm / RubyMine . . . 480
Git no Sublime Text . . . 480
Git no Bash . . . 481
Git no Zsh . . . 482
Git no PowerShell . . . 484
Resumo . . . 486
Appendix B: Incorporando o Git nos seus Aplicativos . . . 487
Linha de Comando do Git . . . 487
Libgit2 . . . 487
JGit . . . 492
go-git . . . 496
Dulwich . . . 498
Appendix C: Comandos do Git . . . 500
Configuração e Ajustes . . . 500
Obtendo e Criando Projetos . . . 502
Snapshots Básicos . . . 503
Branches e Merges . . . 506
Compartilhando e Atualizando Projetos . . . 508
Inspeção e Comparação . . . 510
Depuração . . . 511
Patches . . . 511
E-mail . . . 512
Sistemas Externos . . . 513
Administração . . . 514
Comandos de Encanamento . . . 515

Índice 516

Licença

Esta obra está licenciada sob a Licença Creative Commons Atribuição-NãoComercial-CompartilhaIgual 3.0 Não Adaptada. Para ver uma cópia desta licença, visite https://creativecommons.org/licenses/by-nc-sa/3.0 ou envie uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Prefácio por Scott Chacon

Bem-vindo à segunda edição do Pro Git. A primeira edição foi publicada há mais de quatro anos. Desde então, muita coisa mudou e, no entanto, muitas coisas importantes não mudaram. Embora a maioria dos comandos e conceitos centrais ainda sejam válidos hoje, já que a equipe principal do Git é fantástica em manter a compatibilidade com versões anteriores, houve algumas adições e mudanças significativas na comunidade ao redor do Git. A segunda edição deste livro tem como objetivo abordar essas mudanças e atualizar o livro para que possa ser mais útil ao novo usuário.

Quando escrevi a primeira edição, o Git ainda era uma ferramenta relativamente difícil de usar e pouco adotada, voltada para hackers mais experientes. Estava começando a ganhar força em certas comunidades, mas não havia alcançado nem de perto a ubiquidade que tem hoje. Desde então, quase todas as comunidades de código aberto o adotaram. O Git fez um progresso incrível no Windows, na explosão de interfaces gráficas de usuário para ele em todas as plataformas, no suporte de IDEs e no uso corporativo. O Pro Git de quatro anos atrás não sabe de nada disso. Um dos principais objetivos desta nova edição é abordar todas essas novas fronteiras na comunidade Git.

A comunidade de código aberto usando Git também explodiu. Quando me sentei originalmente para escrever o livro, há quase cinco anos (levei um tempo para lançar a primeira versão), eu tinha acabado de começar a trabalhar em uma empresa muito pouco conhecida que desenvolvia um site de hospedagem Git chamado GitHub. Na época da publicação, talvez houvesse alguns milhares de pessoas usando o site e apenas quatro de nós trabalhando nele. Enquanto escrevo esta introdução, o GitHub está anunciando nosso décimo milionésimo projeto hospedado, com quase 5 milhões de contas de desenvolvedores registradas e mais de 230 funcionários. Ame ou odeie, o GitHub mudou fortemente grandes áreas da comunidade de código aberto de uma maneira que mal era concebível quando me sentei para escrever a primeira edição.

Escrevi uma pequena seção na versão original do Pro Git sobre o GitHub como um exemplo de Git hospedado com o qual nunca me senti muito confortável. Eu não gostava muito do fato de estar escrevendo o que sentia ser essencialmente um recurso da comunidade e também falando sobre minha empresa nele. Embora eu ainda não ame esse conflito de interesses, a importância do GitHub na comunidade Git é inevitável. Em vez de um exemplo de hospedagem Git, decidi transformar essa parte do livro em uma descrição mais profunda do que é o GitHub e de como usá-lo efetivamente. Se você vai aprender como usar o Git, saber como usar o GitHub o ajudará a participar de uma enorme comunidade, o que é valioso não importa qual host Git você decida usar para seu próprio código.

A outra grande mudança no tempo desde a última publicação foi o desenvolvimento e a ascensão do protocolo HTTP para transações de rede do Git. A maioria dos exemplos no livro foi mudada para HTTP em vez de SSH porque é muito mais simples.

Tem sido incrível assistir o Git crescer nos últimos anos, de um sistema de controle de versão relativamente obscuro para basicamente dominar o controle de versão comercial e de código aberto. Estou feliz que o Pro Git tenha se saído tão bem e também tenha sido capaz de ser um dos poucos livros técnicos no mercado que é bastante bem-sucedido e totalmente de código aberto.

Espero que você aproveite esta edição atualizada do Pro Git.

Prefácio por Ben Straub

A primeira edição deste livro foi o que me deixou viciado no Git. Esta foi minha introdução a um estilo de fazer software que parecia mais natural do que qualquer coisa que eu já tinha visto antes. Eu já era desenvolvedor há vários anos na época, mas esta foi a curva certa que me enviou por um caminho muito mais interessante do que o em que eu estava.

Agora, anos depois, sou um contribuidor de uma grande implementação do Git, trabalhei para a maior empresa de hospedagem Git e viajei o mundo ensinando as pessoas sobre o Git. Quando Scott perguntou se eu estaria interessado em trabalhar na segunda edição, eu nem precisei pensar.

Foi um grande prazer e privilégio trabalhar neste livro. Espero que o ajude tanto quanto me ajudou.

Dedicatórias

Para minha esposa, Becky, sem a qual esta aventura nunca teria começado. — Ben

Esta edição é dedicada às minhas garotas. Para minha esposa Jessica que me apoiou por todos esses anos e para minha filha Josephine, que me apoiará quando eu for velho demais para saber o que está acontecendo. — Scott

Colaboradores

Como este é um livro de código aberto, recebemos ao longo dos anos diversas correções e mudanças de conteúdo. Estas são as pessoas que contribuíram para a tradução brasileira do Pro Git como projeto de código aberto. Obrigado a todas elas por ajudarem a tornar este livro melhor para todos.

```
Contributors as of ba91715:

Adrien Ollier                   Joris Valette                   Sean Head
Akrom K                         Joshua Webb                     Sebastian Krause
Aleh Suprunovich                Justin Clift                    Sergio Cabral
Alexandre Garnier               Kaartic Sivaraam                Sergio Cabral (AI
Agent)
Alexandre R. Saccol             Kausar Mehmood                  Shengbin Meng
Altemir Pacheco                 Kenneth Kin Lum                 Siarhei Krukau
Andrei Dascalu                  Krzysztof Szumny                Skyper
Andrew MacFie                   Kyrylo Yatsenko                 Song Li
Andrew Murphy                   Lars Vogel                      Stephan van Maris
André Girol                     Lazar95                         Steven Roddis
André Moreira                   Leonard Laszlo                  Sven Selberg
AndyGee                         Logan Hasson                    Tacini
AnneTheAgile                    Louise Corrigan                 Thanix
Anthony Loiseau                 Luc Morin                       Thomas Ackermann
Anton Trunov                    Lucas Barros                    Tom Schady
Antonino Ingargiola             Lucas Oshiro                    Tomoki Aonuma
Atul Varma                      Marcelo Criscuolo               Vadim Markovtsev
AugustoAMendes                  Marius Žilėnas                  Van Ribeiro
Ben Sima                        Markus KARG                     Vanessa Ribeiro
Benjamin Dopplinger             Marti Bolivar                   Vangelis Katsikaros
Brett Cannon                    Masood Fallahpoor               Vinicius Barbosa de
Aguiar
Cadel Watson                    Matheus Catarino França         Vitaly Kuznetsov
Carlos Martín Nieto             Matthew Miner                   William Gathoye
Carlos Schults                  Matthieu Moy                    Yann Soubeyrand
Changwoo Park                   Michael MacAskill               Yue Lin Ho
Christoph Prokop                Michael Sheaver                 Yusuke SATO
Christopher Wilson              Michael Welch                   allen joslin
Cory Donnelly                   Michiel van der Wulp            atalakam
Cyril                           Mike Charles                    axmbo
Damien Tournoud                 Mike Thibodeau                  cupnodes
Dan Schmidt                     Niels Widger                    devwebcl
Daniel Shahaf                   Nils Reuße                      douglasmoraes
Daniele Tricoli                 Owen                            dualsky
Danny Lin                       Pablo Schläpfer                 flaviovisnardifilho
David Rogers                    Pascal Berger                   flip111
Denis Costa                     Pascal Borreli                  flyingzumwalt
Denis Savitskiy                 Patrick Steinhardt              git-em
DiamondeX                       Pavel Janík                     goekboet
```

Dmitri Tikhonov
Douglas Moraes
Fabiano Leite
Felix Nehrke
Fillippe Prata
Flavio V Filho
Fornost461
Francilio Araújo
Francisco de Assis Boldt
Frederico Mazzone
George Tavares Lopes
Geraldo
Haruo Nakayama
Helmut K. C. Tessarek
Henrique Serraglia
Jan Groenewald
Jean-Noël Avila
Jeroen Oortwijn
Jim Hill
Johannes Schindelin
Jon Forrest
Jon Freed
Paweł Krupiński
Pedro 7úcio
Peter Kokot
Philippe Miossec
Rafael Rossi
Ramon Domingos
Ray Chen
Ricky Senft
Rintze M. Zelle
Rob Blanco
Robert P. Goldman
Rogerio Prado de Jesus
Rohan D'Souza
Roman Kosenko
Ronald Wampler
Rüdiger Herrmann
SATO Yusuke
Sam Ford
Sanders Kleinfeld
Sarah Schneider
Saurav Sachidanand
Scott Bronson
haripetrov
i-give-up
iprok
johnhar
mestihudson
mmikeww
mosdalsvsocld
patrick96
paveljanik
petsuter
rahrah
rmzelle
sanders@oreilly.com
spacewander
twekberg
un1versal
xJom
zwPapEr
□□□□□□□
□□

Introdução

Você está prestes a passar várias horas da sua vida lendo sobre o Git. Vamos tirar um minuto para explicar o que temos reservado para você. Aqui está um breve resumo dos dez capítulos e três apêndices deste livro.

No **Capítulo 1**, vamos abordar os Sistemas de Controle de Versão (VCSs) e o básico do Git — nada técnico, apenas o que o Git é, por que ele surgiu em uma terra cheia de VCSs, o que o diferencia e por que tantas pessoas o estão usando. Em seguida, explicaremos como baixar o Git e configurá-lo pela primeira vez, se você ainda não o tem no seu sistema.

No **Capítulo 2**, abordaremos o uso básico do Git — como usar o Git em 80% dos casos que você encontrará com mais frequência. Depois de ler este capítulo, você deve ser capaz de clonar um repositório, ver o que aconteceu no histórico do projeto, modificar arquivos e contribuir com alterações. Se o livro entrar em combustão espontânea neste ponto, você já deve ser bastante útil manejando o Git no tempo que levar para ir pegar outra cópia.

O **Capítulo 3** é sobre o modelo de ramificação (branching) no Git, frequentemente descrito como a funcionalidade matadora do Git. Aqui você aprenderá o que realmente diferencia o Git do resto do grupo. Quando você terminar, pode sentir a necessidade de passar um momento tranquilo ponderando como você viveu antes que as ramificações do Git fizessem parte da sua vida.

O **Capítulo 4** abordará o Git no servidor. Este capítulo é para aqueles de vocês que desejam configurar o Git dentro de sua organização ou em seu próprio servidor pessoal para colaboração. Também exploraremos várias opções hospedadas se você preferir deixar que outra pessoa cuide disso para você.

O **Capítulo 5** detalhará vários fluxos de trabalho distribuídos e como realizá-los com o Git. Quando você terminar este capítulo, deve ser capaz de trabalhar de forma experiente com vários repositórios remotos, usar o Git por e-mail e fazer malabarismos habilmente com vários branches remotos e patches contribuídos.

O **Capítulo 6** cobre o serviço de hospedagem GitHub e suas ferramentas em profundidade. Abordamos como se inscrever e gerenciar uma conta, criar e usar repositórios Git, fluxos de trabalho comuns para contribuir em projetos e aceitar contribuições no seu, a interface programática do GitHub e muitas pequenas dicas para tornar sua vida mais fácil em geral.

O **Capítulo 7** é sobre comandos avançados do Git. Aqui você aprenderá sobre tópicos como dominar o assustador comando 'reset', usar a busca binária para identificar bugs, editar o histórico, seleção de revisões em detalhes e muito mais. Este capítulo completará seu conhecimento sobre o Git para que você seja verdadeiramente um mestre.

O **Capítulo 8** é sobre a configuração do seu ambiente Git personalizado. Isso inclui configurar scripts de gancho (hooks) para impor ou encorajar políticas personalizadas e usar configurações de ambiente para que você possa trabalhar da maneira que desejar. Também abordaremos a criação do seu próprio conjunto de scripts para impor uma política de commit personalizada.

O **Capítulo 9** trata do Git e de outros VCSs. Isso inclui o uso do Git em um mundo Subversion (SVN) e a conversão de projetos de outros VCSs para o Git. Muitas organizações ainda usam o SVN e não

estão prestes a mudar, mas a essa altura você já terá aprendido o incrível poder do Git — e este capítulo mostra como lidar se você ainda precisar usar um servidor SVN. Também cobrimos como importar projetos de vários sistemas diferentes, caso você consiga convencer todos a darem o mergulho.

O **Capítulo 10** mergulha nas profundezas sombrias, porém belas, dos componentes internos do Git. Agora que você sabe tudo sobre o Git e pode manejá-lo com poder e graça, você pode seguir em frente para discutir como o Git armazena seus objetos, qual é o modelo de objeto, detalhes sobre packfiles, protocolos de servidor e mais. Ao longo do livro, nos referiremos a seções deste capítulo caso você sinta vontade de mergulhar fundo naquele momento; mas se você é como nós e quer mergulhar nos detalhes técnicos, pode querer ler o Capítulo 10 primeiro. Deixamos isso a seu critério.

No **Apêndice A**, olhamos para uma série de exemplos de uso do Git em vários ambientes específicos. Cobrimos uma série de diferentes GUIs e ambientes de programação de IDE nos quais você pode querer usar o Git e o que está disponível para você. Se você estiver interessado em uma visão geral do uso do Git em seu shell, seu IDE ou seu editor de texto, dê uma olhada aqui.

No **Apêndice B**, exploramos a criação de scripts e a extensão do Git por meio de ferramentas como libgit2 e JGit. Se você estiver interessado em escrever ferramentas personalizadas complexas e rápidas e precisar de acesso de baixo nível ao Git, é aqui que você pode ver como é essa paisagem.

Finalmente, no **Apêndice C**, examinamos todos os principais comandos do Git, um de cada vez, e revisamos onde no livro os abordamos e o que fizemos com eles. Se você quiser saber onde no livro usamos algum comando Git específico, pode procurar aqui.

Vamos começar.

Primeiros Passos

Este capítulo é sobre como dar os primeiros passos com o Git. Começaremos explicando alguns fundamentos sobre ferramentas de controle de versão, em seguida, passaremos para como fazer o Git rodar em seu sistema e, por fim, como configurá-lo para começar a trabalhar. Ao final deste capítulo, você deverá entender por que o Git existe, por que você deve usá-lo e deve estar com tudo configurado para fazê-lo.

Sobre Controle de Versão

O que é “controle de versão”, e por que você deveria se importar? O controle de versão é um sistema que registra as mudanças em um arquivo ou conjunto de arquivos ao longo do tempo, para que você possa recuperar versões específicas mais tarde. Para os exemplos neste livro, você usará o código-fonte de software como os arquivos sendo controlados, embora na realidade você possa fazer isso com quase qualquer tipo de arquivo em um computador.

Se você é um designer gráfico ou web e deseja manter cada versão de uma imagem ou layout (o que você certamente iria querer), usar um Sistema de Controle de Versão (VCS) é uma decisão muito sábia. Ele permite que você reverta arquivos selecionados para um estado anterior, reverta o projeto inteiro para um estado anterior, compare as alterações ao longo do tempo, veja quem modificou por último algo que pode estar causando um problema, quem introduziu um problema e quando, e mais. Usar um VCS também geralmente significa que se você estragar as coisas ou perder arquivos, pode se recuperar facilmente. Além disso, você obtém tudo isso com muito pouco esforço adicional.

Sistemas de Controle de Versão Locais

O método de controle de versão preferido de muitas pessoas é copiar os arquivos para outro diretório (talvez um diretório com carimbo de data/hora, se elas forem espertas). Essa abordagem é muito comum porque é muito simples, mas também é incrivelmente sujeita a erros. É fácil esquecer em qual diretório você está e acidentalmente escrever no arquivo errado ou sobrescrever arquivos que você não pretendia.

Para lidar com esse problema, os programadores desenvolveram, há muito tempo, VCSs locais que tinham um banco de dados simples que mantinha todas as alterações nos arquivos sob controle de revisão.

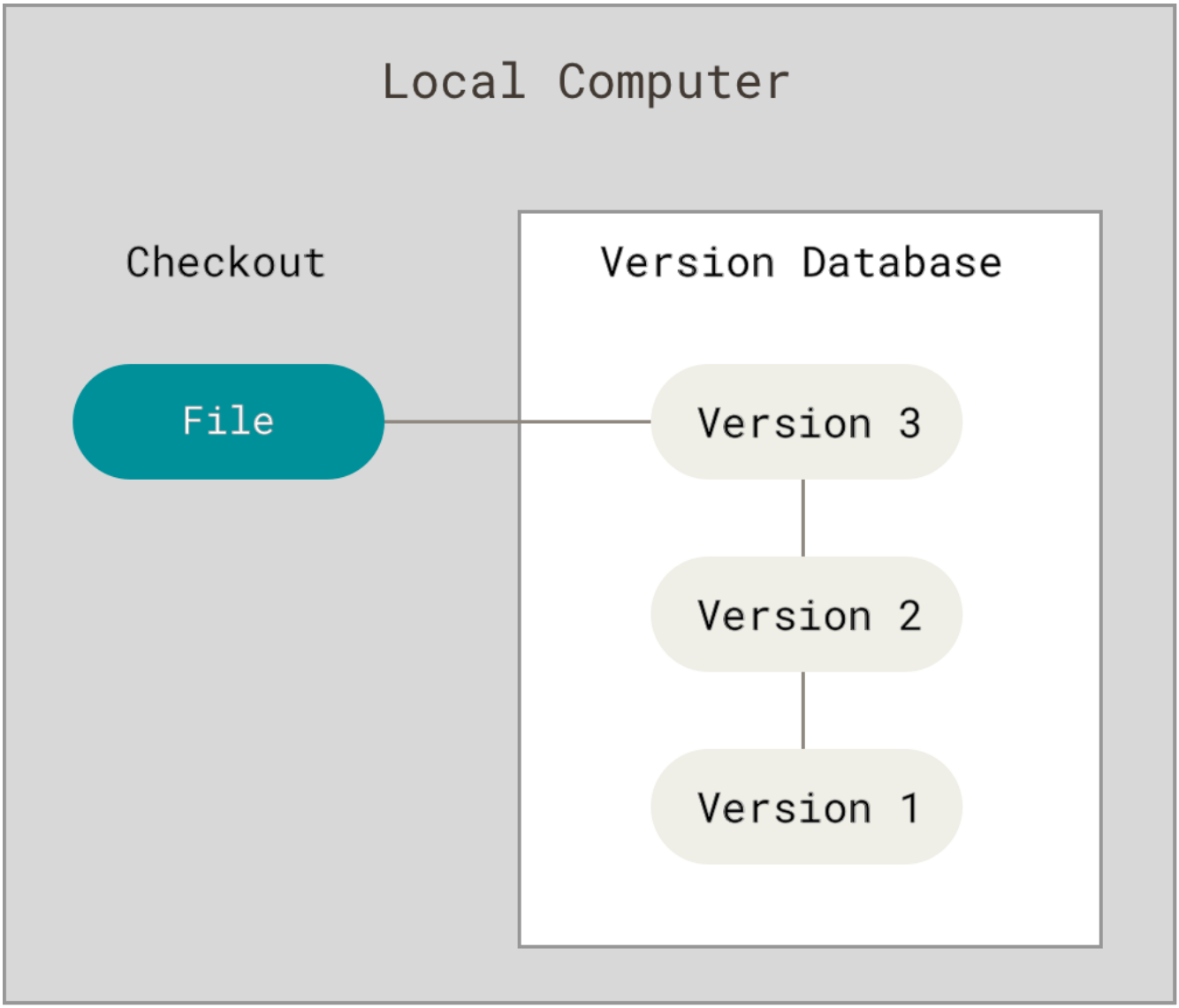


Figure 1. Diagrama de controle de versão local

Uma das ferramentas VCS mais populares foi um sistema chamado RCS, que ainda é distribuído com muitos computadores hoje. O RCS funciona mantendo conjuntos de patches (isto é, as diferenças entre arquivos) em um formato especial no disco; ele pode então recriar a aparência de qualquer arquivo em qualquer ponto no tempo, somando todos os patches.

Sistemas de Controle de Versão Centralizados

O próximo grande problema que as pessoas encontram é que precisam colaborar com desenvolvedores em outros sistemas. Para lidar com esse problema, os Sistemas de Controle de Versão Centralizados (CVCSs) foram desenvolvidos. Esses sistemas (como CVS, Subversion e Perforce) possuem um único servidor que contém todos os arquivos versionados, e vários clientes que fazem o checkout dos arquivos desse local central. Por muitos anos, este tem sido o padrão para controle de versão.

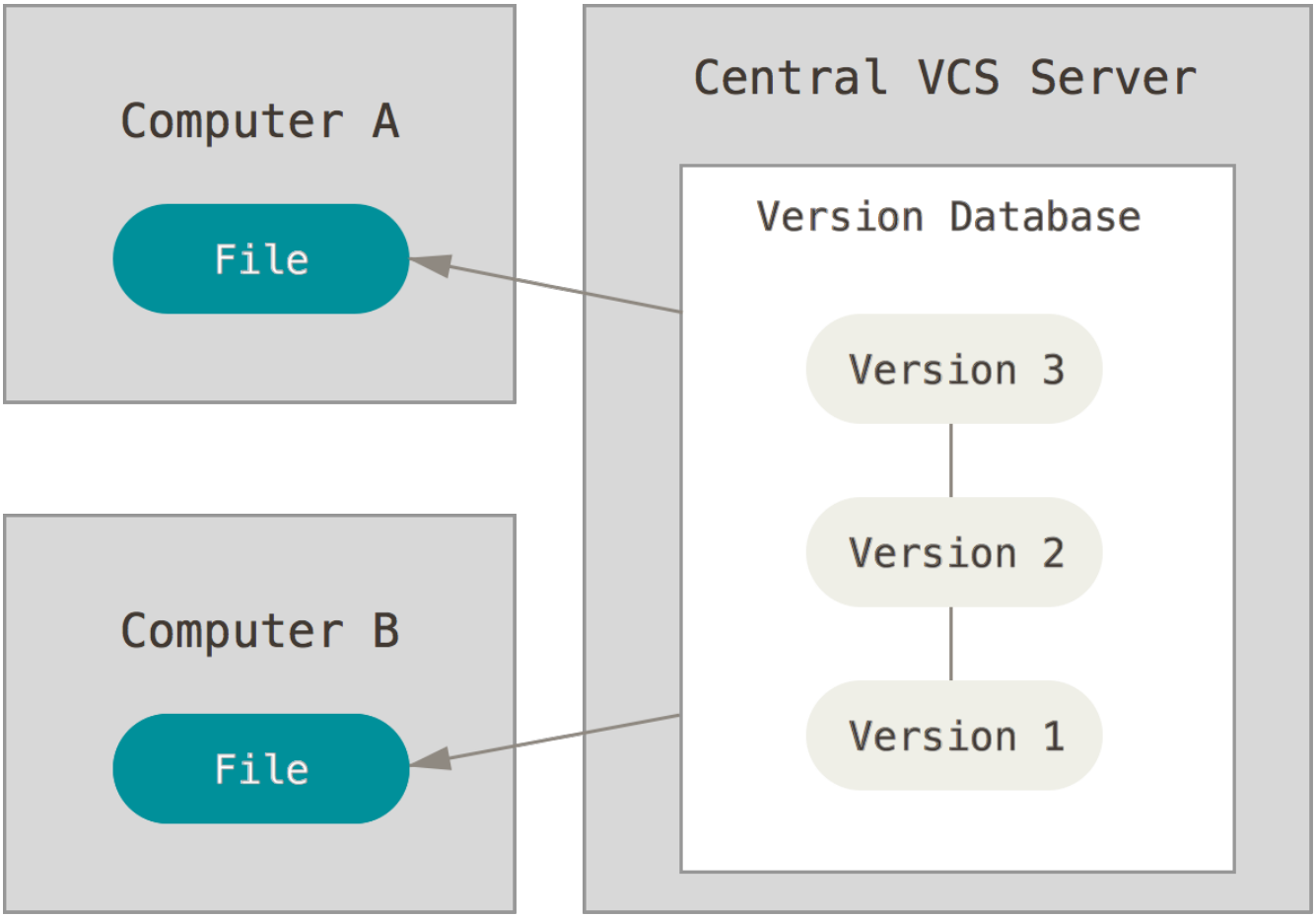


Figure 2. Diagrama de controle de versão centralizado

Essa configuração oferece muitas vantagens, especialmente sobre os VCSs locais. Por exemplo, todo mundo sabe, até certo ponto, o que todos os outros no projeto estão fazendo. Os administradores têm um controle refinado sobre quem pode fazer o quê, e é muito mais fácil administrar um CVCS do que lidar com bancos de dados locais em cada cliente.

No entanto, essa configuração também tem algumas desvantagens sérias. A mais óbvia é o ponto único de falha que o servidor centralizado representa. Se esse servidor ficar inativo por uma hora, durante essa hora ninguém poderá colaborar de forma alguma ou salvar alterações versionadas em nada no qual estejam trabalhando. Se o disco rígido onde o banco de dados central está for corrompido e não tiverem sido mantidos backups adequados, você perde absolutamente tudo — o histórico inteiro do projeto, exceto quaisquer snapshots únicos que as pessoas tenham em suas máquinas locais. Os VCSs locais sofrem do mesmo problema — sempre que você tem o histórico inteiro do projeto em um único lugar, corre o risco de perder tudo.

Sistemas de Controle de Versão Distribuídos

É aqui que os Sistemas de Controle de Versão Distribuídos (DVCSs) entram em cena. Em um DVCS (como Git, Mercurial ou Darcs), os clientes não fazem apenas o checkout do snapshot mais recente dos arquivos; em vez disso, eles espelham totalmente o repositório, incluindo todo o seu histórico. Assim, se qualquer servidor morrer, e esses sistemas estivessem colaborando através desse servidor, qualquer um dos repositórios dos clientes pode ser copiado de volta para o servidor para restaurá-lo. Cada clone é realmente um backup completo de todos os dados.

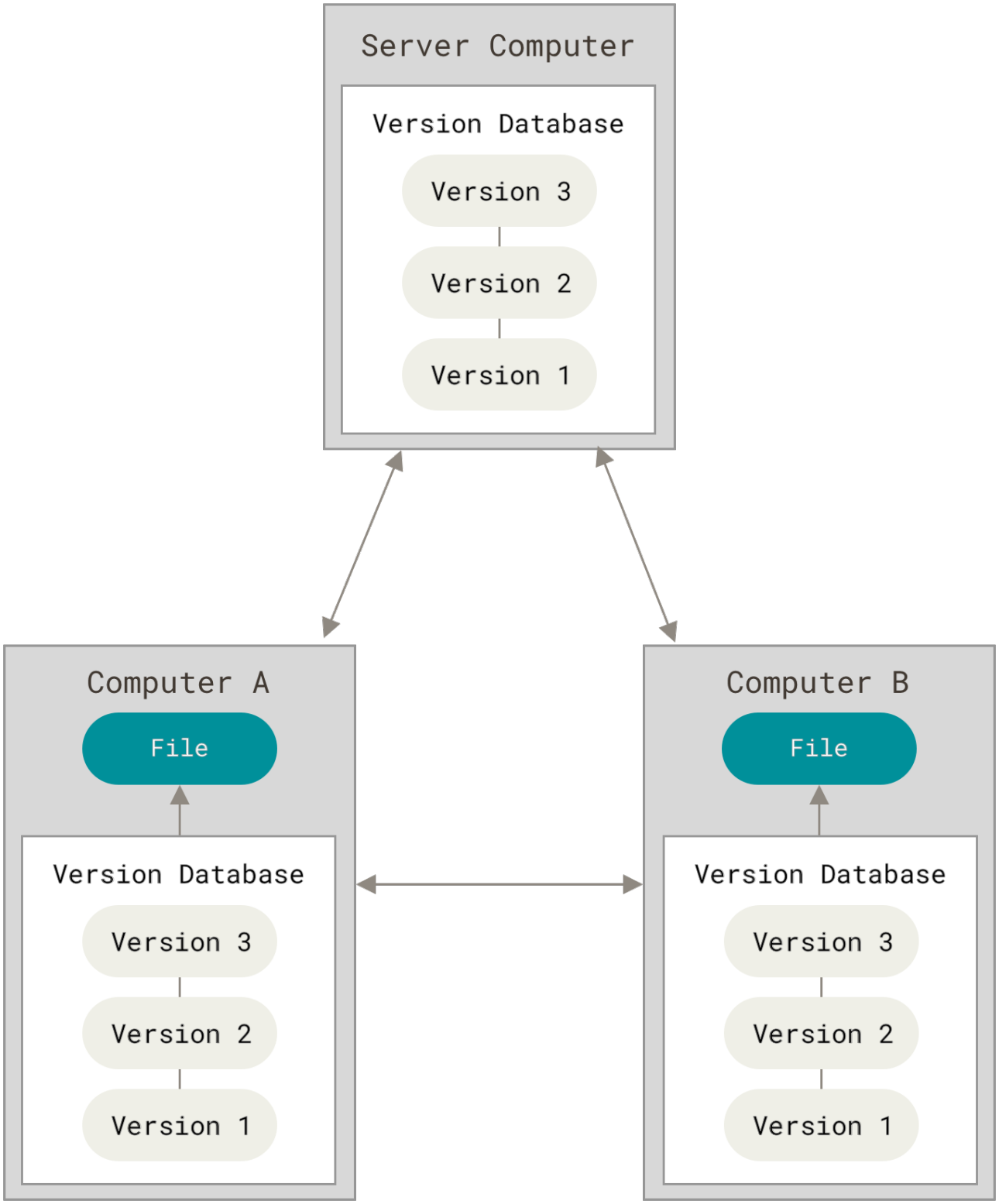


Figure 3. Diagrama de controle de versão distribuído

Além disso, muitos desses sistemas lidam muito bem com a existência de vários repositórios remotos com os quais podem trabalhar, de modo que você pode colaborar com diferentes grupos de pessoas de diferentes maneiras, simultaneamente, dentro do mesmo projeto. Isso permite que você configure vários tipos de fluxos de trabalho que não são possíveis em sistemas centralizados, como modelos hierárquicos.

Uma Breve História do Git

Como muitas coisas grandes na vida, o Git começou com um pouco de destruição criativa e uma controvérsia acalorada.

O kernel Linux é um projeto de software de código aberto de escopo bastante grande. Durante os primeiros anos de manutenção do kernel Linux (1991–2002), as alterações no software eram passadas como patches e arquivos compactados. Em 2002, o projeto do kernel Linux começou a usar um DVCS proprietário chamado BitKeeper.

Em 2005, o relacionamento entre a comunidade que desenvolveu o kernel Linux e a empresa comercial que desenvolveu o BitKeeper desmoronou, e o status gratuito da ferramenta foi revogado. Isso levou a comunidade de desenvolvimento do Linux (e em particular Linus Torvalds, o criador do Linux) a desenvolver sua própria ferramenta baseada em algumas das lições que aprenderam ao usar o BitKeeper. Alguns dos objetivos do novo sistema eram os seguintes:

- Velocidade
- Design simples
- Forte suporte para desenvolvimento não linear (milhares de branches paralelos)
- Totalmente distribuído
- Capaz de lidar com grandes projetos como o kernel Linux eficientemente (velocidade e tamanho dos dados)

Desde o seu nascimento em 2005, o Git evoluiu e amadureceu para ser fácil de usar, e ainda assim reter essas qualidades iniciais. Ele é incrivelmente rápido, é muito eficiente com grandes projetos e tem um incrível sistema de branching para desenvolvimento não linear (veja Ramificação (Branching) no Git).

O que é Git?

Então, o que é o Git em poucas palavras? Esta é uma seção importante para absorver, porque se você entender o que o Git é e os fundamentos de como ele funciona, então usar o Git de forma eficaz provavelmente será muito mais fácil para você. À medida que você aprende o Git, tente limpar a mente das coisas que você talvez saiba sobre outros VCSs, como CVS, Subversion ou Perforce — fazer isso o ajudará a evitar confusões sutis ao usar a ferramenta. Mesmo que a interface de usuário do Git seja bastante semelhante a esses outros VCSs, o Git armazena e pensa sobre as informações de uma maneira muito diferente, e entender essas diferenças o ajudará a evitar ficar confuso ao usá-lo.

Snapshots, não diferenças

A principal diferença entre o Git e qualquer outro VCS (Subversion e amigos incluídos) é a maneira como o Git pensa sobre seus dados. Conceitualmente, a maioria dos outros sistemas armazena informações como uma lista de alterações baseadas em arquivos. Esses outros sistemas (CVS, Subversion, Perforce e assim por diante) pensam nas informações que armazenam como um conjunto de arquivos e as alterações feitas em cada arquivo ao longo do tempo (isso é comumente descrito como controle de versão *baseado em delta*).

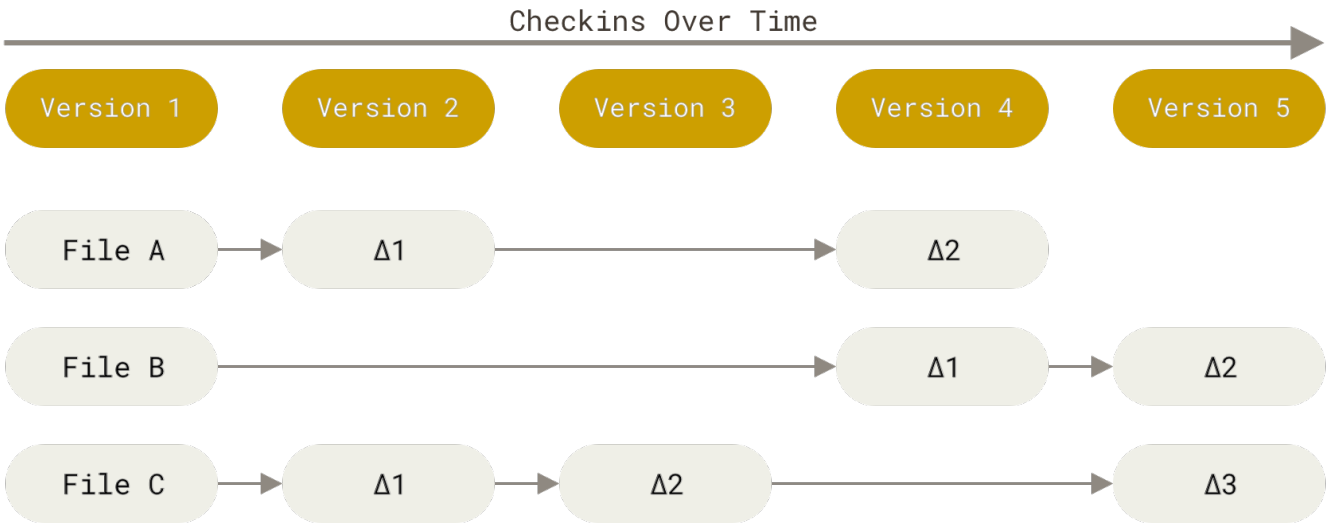


Figure 4. Armazenamento de dados como alterações em uma versão base de cada arquivo

O Git não pensa ou armazena seus dados dessa maneira. Em vez disso, o Git pensa em seus dados mais como uma série de snapshots de um sistema de arquivos em miniatura. Com o Git, toda vez que você faz um commit, ou salva o estado do seu projeto, o Git basicamente tira uma foto de como todos os seus arquivos se parecem naquele momento e armazena uma referência para esse snapshot. Para ser eficiente, se os arquivos não mudaram, o Git não armazena o arquivo novamente, apenas um link para o arquivo idêntico anterior que ele já armazenou. O Git pensa em seus dados mais como um **fluxo de snapshots**.

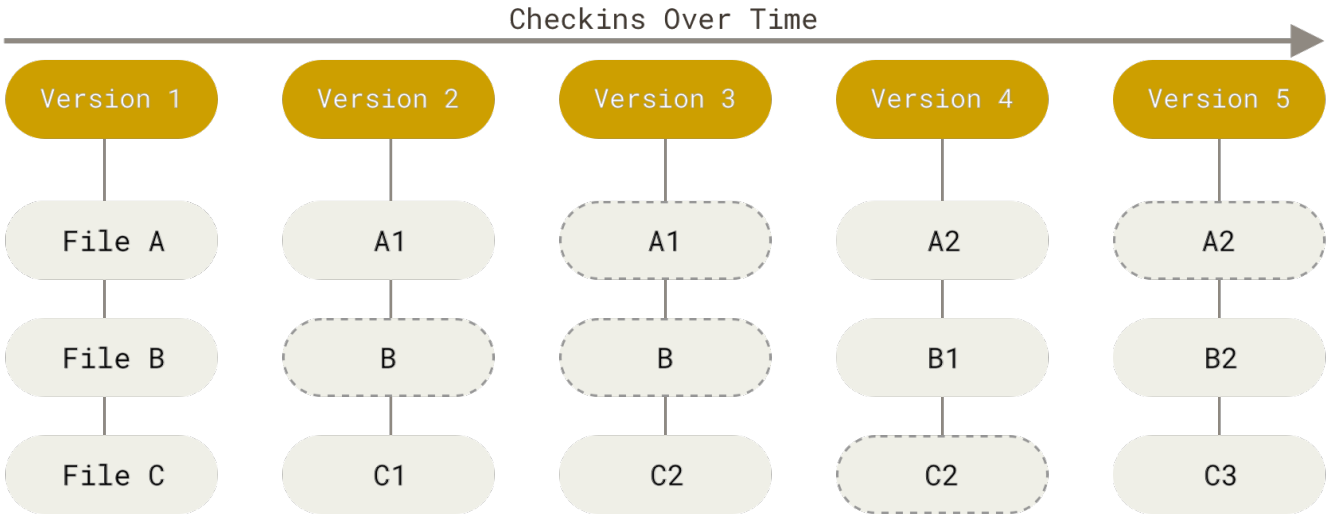


Figure 5. Armazenamento de dados como snapshots do projeto ao longo do tempo

Esta é uma distinção importante entre o Git e quase todos os outros VCSs. Isso faz com que o Git reconsidere quase todos os aspectos do controle de versão que a maioria dos outros sistemas copiou da geração anterior. Isso torna o Git mais como um mini sistema de arquivos com algumas ferramentas incrivelmente poderosas construídas sobre ele, em vez de simplesmente um VCS. Exploraremos alguns dos benefícios que você ganha ao pensar em seus dados dessa maneira quando abordarmos o branching do Git em Ramificação (Branching) no Git.

Quase toda operação é local

A maioria das operações no Git precisa apenas de arquivos e recursos locais para operar — geralmente nenhuma informação de outro computador na sua rede é necessária. Se você está acostumado a um CVCS em que a maioria das operações tem aquele custo adicional de latência

de rede, esse aspecto do Git fará você pensar que os deuses da velocidade abençoaram o Git com poderes de outro mundo. Como você tem todo o histórico do projeto bem ali no seu disco local, a maioria das operações parece quase instantânea.

Por exemplo, para navegar no histórico do projeto, o Git não precisa ir ao servidor para obter o histórico e exibi-lo para você — ele simplesmente o lê direto do seu banco de dados local. Isso significa que você vê o histórico do projeto quase instantaneamente. Se você quiser ver as alterações introduzidas entre a versão atual de um arquivo e o arquivo há um mês, o Git pode consultar o arquivo de um mês atrás e fazer um cálculo de diferença local, em vez de ter que pedir a um servidor remoto para fazer isso ou extrair uma versão mais antiga do arquivo do servidor remoto para fazê-lo localmente.

Isso também significa que há muito pouco que você não pode fazer se estiver offline ou fora da VPN. Se você entrar em um avião ou trem e quiser trabalhar um pouco, pode fazer commits alegremente (na sua cópia *local*, lembra?) até chegar a uma conexão de rede para fazer o upload. Se você for para casa e não conseguir fazer o seu cliente VPN funcionar corretamente, você ainda pode trabalhar. Em muitos outros sistemas, fazer isso é impossível ou doloroso. No Perforce, por exemplo, você não pode fazer muita coisa quando não está conectado ao servidor; no Subversion e CVS, você pode editar arquivos, mas não pode fazer commit das alterações no seu banco de dados (porque o seu banco de dados está offline). Isso pode não parecer grande coisa, mas você pode se surpreender com a grande diferença que isso pode fazer.

O Git tem integridade

Tudo no Git é verificado por soma de verificação (checksummed) antes de ser armazenado e é então referido por essa soma de verificação. Isso significa que é impossível alterar o conteúdo de qualquer arquivo ou diretório sem que o Git saiba disso. Essa funcionalidade é incorporada ao Git nos níveis mais baixos e é essencial à sua filosofia. Você não pode perder informações em trânsito ou sofrer corrupção de arquivos sem que o Git seja capaz de detectar isso.

O mecanismo que o Git usa para essa soma de verificação é chamado de hash SHA-1. Esta é uma string de 40 caracteres composta por caracteres hexadecimais (0–9 e a–f) e calculada com base no conteúdo de um arquivo ou estrutura de diretórios no Git. Um hash SHA-1 se parece com isto:

```
24b9da6552252987aa493b52f8696cd6d3b00373
```

Você verá esses valores de hash em todos os lugares no Git porque ele os usa muito. Na verdade, o Git armazena tudo em seu banco de dados não pelo nome do arquivo, mas pelo valor de hash do seu conteúdo.

O Git geralmente apenas adiciona dados

Quando você executa ações no Git, quase todas elas apenas *adicionam* dados ao banco de dados do Git. É difícil fazer o sistema fazer qualquer coisa que não seja desfazível ou fazê-lo apagar dados de alguma forma. Como com qualquer VCS, você pode perder ou bagunçar alterações que ainda não commitou, mas depois que você faz o commit de um snapshot no Git, é muito difícil perdê-lo, especialmente se você envia (push) regularmente o seu banco de dados para outro repositório.

Isso faz com que usar o Git seja uma alegria, porque sabemos que podemos experimentar sem o perigo de estragar as coisas gravemente. Para uma visão mais aprofundada de como o Git armazena seus dados e como você pode recuperar dados que parecem perdidos, veja Desfazendo Coisas.

Os Três Estados

Preste atenção agora — aqui está a principal coisa a lembrar sobre o Git se você quiser que o resto do seu processo de aprendizado ocorra sem problemas. O Git tem três estados principais nos quais seus arquivos podem residir: *modificado*, *preparado* e *commitado*:

- Modificado significa que você alterou o arquivo, mas ainda não o commitou em seu banco de dados.
- Preparado significa que você marcou um arquivo modificado em sua versão atual para entrar em seu próximo snapshot de commit.
- Commitado significa que os dados estão armazenados com segurança em seu banco de dados local.

Isso nos leva às três seções principais de um projeto Git: a árvore de trabalho, a área de preparação e o diretório Git.

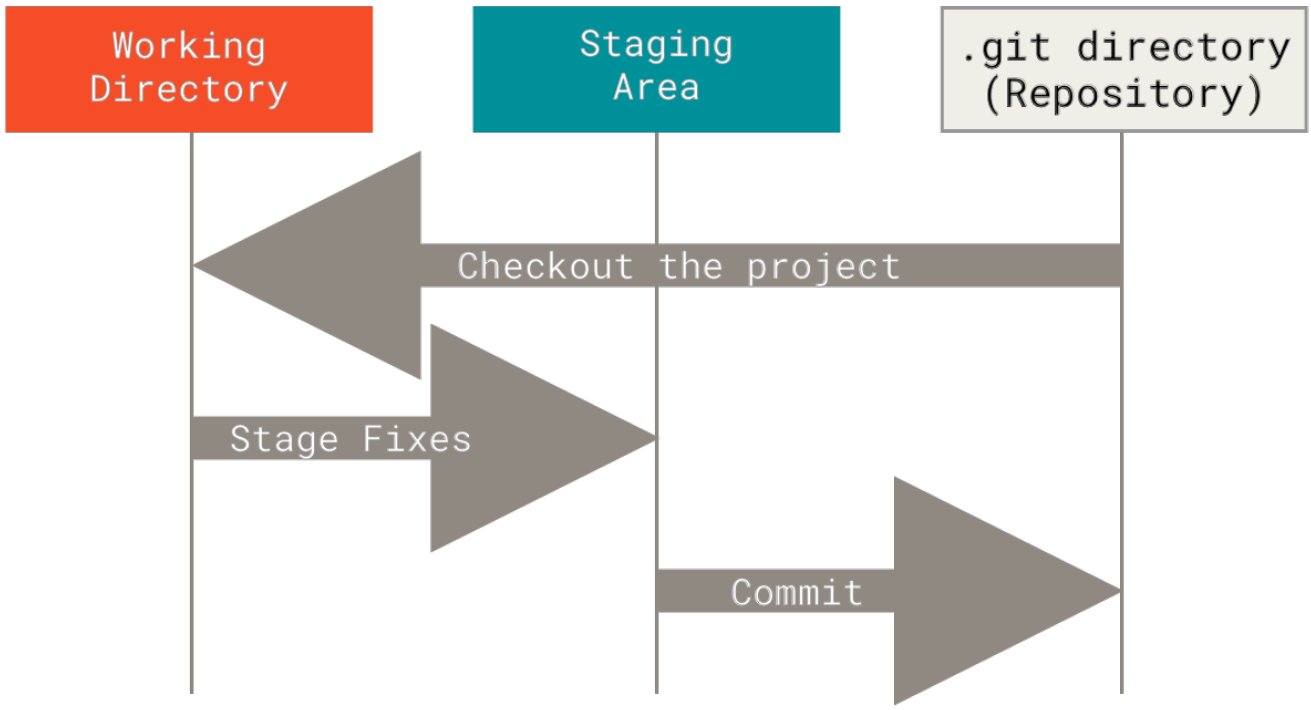


Figure 6. Árvore de trabalho, área de preparação e diretório Git

A árvore de trabalho é um único checkout de uma versão do projeto. Esses arquivos são retirados do banco de dados compactado no diretório Git e colocados no disco para você usar ou modificar.

A área de preparação é um arquivo, geralmente contido no seu diretório Git, que armazena informações sobre o que irá para o seu próximo commit. Seu nome técnico no jargão do Git é o "index", mas a frase "área de preparação" funciona da mesma forma.

O diretório Git é onde o Git armazena os metadados e o banco de dados de objetos para o seu

projeto. Esta é a parte mais importante do Git e é o que é copiado quando você *clona* um repositório de outro computador.

O fluxo de trabalho básico do Git é mais ou menos assim:

1. Você modifica arquivos na sua árvore de trabalho.
2. Você prepara (stage) seletivamente apenas as alterações que deseja que façam parte do seu próximo commit, o que adiciona *apenas* essas alterações à área de preparação.
3. Você faz um commit, o que pega os arquivos como eles estão na área de preparação e armazena esse snapshot permanentemente no seu diretório Git.

Se uma versão particular de um arquivo está no diretório Git, ela é considerada *commitada*. Se ela foi modificada e foi adicionada à área de preparação, ela está *preparada*. E se ela foi alterada desde o checkout, mas não foi preparada, ela está *modificada*. No Fundamentos do Git, você aprenderá mais sobre esses estados e como você pode tirar vantagem deles ou pular a parte preparada inteiramente.

A Linha de Comando

Existem muitas maneiras diferentes de usar o Git. Existem as ferramentas de linha de comando originais, e existem muitas interfaces gráficas de usuário com capacidades variadas. Para este livro, usaremos o Git na linha de comando. Por um lado, a linha de comando é o único lugar onde você pode executar *todos* os comandos do Git — a maioria das GUIs implementa apenas um subconjunto parcial das funcionalidades do Git por simplicidade. Se você sabe como executar a versão de linha de comando, provavelmente também conseguirá descobrir como executar a versão da GUI, enquanto o oposto não é necessariamente verdadeiro. Além disso, enquanto a escolha do cliente gráfico é uma questão de gosto pessoal, *todos* os usuários terão as ferramentas de linha de comando instaladas e disponíveis.

Portanto, esperamos que você saiba como abrir o Terminal no macOS ou o Prompt de Comando ou PowerShell no Windows. Se você não sabe do que estamos falando aqui, pode ser necessário parar e pesquisar isso rapidamente para que você possa acompanhar o restante dos exemplos e descrições neste livro.

Instalando o Git

Antes de começar a usar o Git, você deve disponibilizá-lo no seu computador. Mesmo que já esteja instalado, provavelmente é uma boa ideia atualizar para a versão mais recente. Você pode instalá-lo como um pacote ou por meio de outro instalador, ou baixar o código-fonte e compilá-lo você mesmo.



Este livro foi escrito usando o Git versão 2. Como o Git é excelente em preservar a compatibilidade com versões anteriores, qualquer versão recente deve funcionar bem. Embora a maioria dos comandos que usamos deva funcionar mesmo em versões antigas do Git, alguns deles podem não funcionar ou podem agir de maneira ligeiramente diferente.

Instalando no Linux

Se você deseja instalar as ferramentas básicas do Git no Linux por meio de um instalador binário, geralmente pode fazer isso através da ferramenta de gerenciamento de pacotes que vem com sua distribuição. Se você estiver no Fedora (ou em qualquer distribuição baseada em RPM intimamente relacionada, como RHEL ou CentOS), pode usar o `dnf`:

```
$ sudo dnf install git-all
```

Se você estiver em uma distribuição baseada em Debian, como o Ubuntu, tente o `apt`:

```
$ sudo apt install git-all
```

Para mais opções, existem instruções para instalar em várias distribuições Unix diferentes no site do Git, em https://git-scm.com/download/linux.

Instalando no macOS

Existem várias maneiras de instalar o Git no macOS. A mais fácil é provavelmente instalar as ferramentas de linha de comando do Xcode. No Mavericks (10.9) ou superior, você pode fazer isso simplesmente tentando executar `git` a partir do Terminal na primeira vez.

```
$ git --version
```

Se você ainda não o tiver instalado, será solicitado a instalá-lo.

Se você quiser uma versão mais atualizada, também pode instalá-la por meio de um instalador binário. Um instalador do Git para macOS é mantido e está disponível para download no site do Git, em https://git-scm.com/download/mac.



Figure 7. Instalador do Git para macOS

Instalando no Windows

Também existem algumas maneiras de instalar o Git no Windows. A compilação mais oficial está disponível para download no site do Git. Basta ir em https://git-scm.com/download/win e o download iniciará automaticamente. Note que este é um projeto chamado Git para Windows, que é separado do próprio Git; para mais informações sobre ele, vá em https://gitforwindows.org.

Para obter uma instalação automatizada, você pode usar o pacote Chocolatey do Git. Note que o pacote Chocolatey é mantido pela comunidade.

Instalando a partir do código-fonte

Algumas pessoas podem achar mais útil instalar o Git a partir do código-fonte, porque você obterá a versão mais recente. Os instaladores binários tendem a ficar um pouco para trás, embora, à medida que o Git amadureceu nos últimos anos, isso tenha feito menos diferença.

Se você quiser instalar o Git a partir do código-fonte, precisará ter as seguintes bibliotecas das quais o Git depende: autotools, curl, zlib, openssl, expat e libiconv. Por exemplo, se você estiver em um sistema que tem o `dnf` (como o Fedora) ou `apt-get` (como um sistema baseado em Debian), pode usar um desses comandos para instalar as dependências mínimas para compilar e instalar os binários do Git:

```
$ sudo dnf install dh-autoreconf curl-devel expat-devel gettext-devel \
```

```
  openssl-devel perl-devel zlib-devel
$ sudo apt-get install dh-autoreconf libcurl4-gnutls-dev libexpat1-dev \
  gettext libz-dev libssl-dev
```

Para ser capaz de adicionar a documentação em vários formatos (doc, html, info), estas dependências adicionais são necessárias:

```
$ sudo dnf install asciidoc xmlto docbook2X
$ sudo apt-get install asciidoc xmlto docbook2x
```



Usuários do RHEL e de derivados do RHEL, como CentOS e Scientific Linux, terão que habilitar o repositório EPEL para baixar o pacote `docbook2X`.

Se você estiver usando uma distribuição baseada no Debian (Debian/Ubuntu/derivados do Ubuntu), também precisará do pacote `install-info`:

```
$ sudo apt-get install install-info
```

Se você estiver usando uma distribuição baseada em RPM (Fedora/RHEL/derivados do RHEL), também precisará do pacote `getopt` (que já vem instalado em uma distribuição baseada em Debian):

```
$ sudo dnf install getopt
```

Adicionalmente, se você estiver usando Fedora/RHEL/derivados do RHEL, precisará fazer isso:

```
$ sudo ln -s /usr/bin/db2x_docbook2texi /usr/bin/docbook2x-texi
```

devido a diferenças de nome de binário.

Quando você tiver todas as dependências necessárias, pode ir em frente e pegar o tarball da versão mais recente marcada (tagged release) de vários lugares. Você pode obtê-lo no site kernel.org, em https://www.kernel.org/pub/software/scm/git, ou no espelho do site no GitHub, em https://github.com/git/git/tags. Geralmente é um pouco mais claro qual é a versão mais recente na página do GitHub, mas a página kernel.org também tem assinaturas de lançamento se você quiser verificar seu download.

Então, compile e instale:

```
$ tar -zxf git-2.8.0.tar.gz
$ cd git-2.8.0
$ make configure
$ ./configure --prefix=/usr
$ make all doc info
```

```
$ sudo make install install-doc install-html install-info
```

Depois que isso for feito, você também pode obter o Git por meio do próprio Git para atualizações:

```
$ git clone https://git.kernel.org/pub/scm/git/git.git
```

Configuração inicial do Git

Agora que você tem o Git em seu sistema, você vai querer fazer algumas coisas para personalizar o seu ambiente Git. Você deve ter que fazer essas coisas apenas uma vez em qualquer computador; elas permanecerão entre as atualizações. Você também pode alterá-las a qualquer momento executando os comandos novamente.

O Git vem com uma ferramenta chamada `git config` que permite obter e definir variáveis de configuração que controlam todos os aspectos de como o Git se parece e opera. Essas variáveis podem ser armazenadas em três locais diferentes:

1. Arquivo `[caminho]/etc/gitconfig`: Contém valores aplicados a todos os usuários no sistema e em todos os seus repositórios. Se você passar a opção `--system` para `git config`, ele lê e escreve especificamente a partir deste arquivo. Como este é um arquivo de configuração do sistema, você precisaria de privilégios administrativos ou de superusuário para fazer alterações nele.
2. Arquivo `~/.gitconfig` ou `~/.config/git/config`: Valores específicos pessoalmente para você, o usuário. Você pode fazer com que o Git leia e grave especificamente neste arquivo passando a opção `--global`, e isso afeta *todos* os repositórios com os quais você trabalha no seu sistema.
3. Arquivo `config` no diretório Git (ou seja, `.git/config`) de qualquer repositório que você esteja usando no momento: Específico para aquele único repositório. Você pode forçar o Git a ler e escrever neste arquivo com a opção `--local`, mas isso é de fato o padrão. Sem surpresa, você precisa estar localizado em algum lugar de um repositório Git para que esta opção funcione corretamente.

Cada nível substitui os valores no nível anterior, portanto, os valores em `.git/config` superam os de `[caminho]/etc/gitconfig`.

Em sistemas Windows, o Git procura o arquivo `.gitconfig` no diretório `$HOME` (`C:\Users\$USER` para a maioria das pessoas). Ele também ainda procura por `[caminho]/etc/gitconfig`, embora seja relativo à raiz do MSys, que é onde quer que você decida instalar o Git em seu sistema Windows ao executar o instalador. Se você estiver usando a versão 2.x ou mais recente do Git para Windows, também existe um arquivo de configuração no nível do sistema em `C:\Documents and Settings\All Users\Application Data\Git\config` no Windows XP, e em `C:\ProgramData\Git\config` no Windows Vista e mais recentes. Este arquivo de configuração só pode ser alterado por `git config -f <arquivo>` como administrador.

Você pode visualizar todas as suas configurações e de onde elas vêm usando:

```
$ git config --list --show-origin
```

Sua Identidade

A primeira coisa que você deve fazer ao instalar o Git é definir o seu nome de usuário e endereço de e-mail. Isso é importante porque todo commit do Git usa essas informações e elas estão incorporadas de forma imutável nos commits que você começa a criar:

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

Novamente, você precisa fazer isso apenas uma vez se passar a opção `--global`, porque então o Git sempre usará essas informações para o seu usuário nesse sistema. Se você quiser substituir isso com um nome ou endereço de e-mail diferente para projetos específicos, poderá executar o comando sem a opção `--global` quando estiver nesse projeto.

Muitas das ferramentas de GUI ajudarão você a fazer isso quando as executar pela primeira vez.

Seu Editor

Agora que sua identidade está configurada, você pode configurar o editor de texto padrão que será usado quando o Git precisar que você digite uma mensagem. Se não estiver configurado, o Git usará o editor padrão do seu sistema.

Se você quiser usar um editor de texto diferente, como o Emacs, pode fazer o seguinte:

```
$ git config --global core.editor emacs
```

Em um sistema Windows, se você quiser usar um editor de texto diferente, deverá especificar o caminho completo para seu arquivo executável. Isso pode ser diferente dependendo de como o seu editor está empacotado.

No caso do Notepad++, um editor de programação popular, é provável que você queira usar a versão de 32 bits, já que, no momento da escrita, a versão de 64 bits não suporta todos os plug-ins. Se você estiver em um sistema Windows de 32 bits, ou se tiver um editor de 64 bits em um sistema de 64 bits, digitará algo assim:

```
$ git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe'
-multiInst -notabbar -nosession -noPlugin"
```



O Vim, Emacs e Notepad++ são editores de texto populares frequentemente usados por desenvolvedores em sistemas baseados em Unix como Linux e macOS ou um sistema Windows. Se você estiver usando outro editor, ou uma versão de 32 bits, encontre instruções específicas sobre como configurar o seu editor favorito com o Git em Comandos git config core.editor.



Você pode descobrir que, se não configurar seu editor dessa forma, ficará em um estado muito confuso quando o Git tentar iniciá-lo. Um exemplo em um sistema

> Windows pode incluir uma operação do Git encerrada prematuramente durante uma edição iniciada pelo Git.

O nome do seu branch padrão

Por padrão, o Git criará um branch chamado *master* quando você criar um novo repositório com `git init`. A partir da versão 2.28 do Git, você pode definir um nome diferente para o branch inicial.

Para definir *main* como o nome do branch padrão, faça:

```
$ git config --global init.defaultBranch main
```

Verificando suas configurações

Se você quiser verificar suas definições de configuração, pode usar o comando `git config --list` para listar todas as configurações que o Git pode encontrar naquele ponto:

```
$ git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

Você pode ver as chaves mais de uma vez, porque o Git lê a mesma chave de arquivos diferentes (`[caminho]/etc/gitconfig` e `~/.gitconfig`, por exemplo). Nesse caso, o Git usa o último valor para cada chave única que vê.

Você também pode verificar o que o Git pensa ser o valor de uma chave específica digitando `git config <chave>`:

```
$ git config user.name
John Doe
```



> Como o Git pode ler o valor da mesma variável de configuração de mais de um arquivo, é possível que você tenha um valor inesperado para um desses valores e não saiba o porquê. Em casos assim, você pode consultar o Git quanto à *origem* (*origin*) desse valor, e ele dirá qual arquivo de configuração teve a palavra final na definição desse valor:
>
> ```
> $ git config --show-origin rerere.autoUpdate
> file:/home/johndoe/.gitconfig   false
> ```

Obtendo Ajuda

Se você precisar de ajuda ao usar o Git, existem três maneiras equivalentes de obter a ajuda abrangente da página de manual (manpage) para qualquer um dos comandos do Git:

```
$ git help <verb>
$ git <verb> --help
$ man git-<verb>
```

Por exemplo, você pode obter a ajuda da manpage para o comando `git config` executando isto:

```
$ git help config
```

Esses comandos são ótimos porque você pode acessá-los de qualquer lugar, mesmo offline. Se as manpages e este livro não forem suficientes e você precisar de ajuda em pessoa, pode tentar os canais `#git`, `#github` ou `#gitlab` no servidor IRC Libera Chat, que pode ser encontrado em https://libera.chat/. Esses canais costumam estar cheios de centenas de pessoas que têm muito conhecimento sobre o Git e geralmente estão dispostas a ajudar.

Além disso, se você não precisar da ajuda completa da manpage, mas apenas precisar de uma rápida lembrança sobre as opções disponíveis para um comando do Git, você pode solicitar a saída "help" mais concisa com a opção `-h`, como em:

```
$ git add -h
usage: git add [<options>] [--] <pathspec>...

    -n, --dry-run               dry run
    -v, --verbose               be verbose

    -i, --interactive           interactive picking
    -p, --patch                 select hunks interactively
    -e, --edit                  edit current diff and apply
    -f, --force                 allow adding otherwise ignored files
    -u, --update                update tracked files
    --renormalize               renormalize EOL of tracked files (implies -u)
    -N, --intent-to-add         record only the fact that the path will be added later
    -A, --all                   add changes from all tracked and untracked files
    --ignore-removal            ignore paths removed in the working tree (same as --no
-all)
    --refresh                   don't add, only refresh the index
    --ignore-errors             just skip files which cannot be added because of
errors
    --ignore-missing            check if - even missing - files are ignored in dry run
    --sparse                    allow updating entries outside of the sparse-checkout
cone
    --chmod (+|-)x              override the executable bit of the listed files
    --pathspec-from-file <file> read pathspec from file
```

```
    --pathspec-file-nul         with --pathspec-from-file, pathspec elements are
separated with NUL character
```

Resumo

Você deve ter um entendimento básico do que é o Git e como ele é diferente de quaisquer sistemas de controle de versão centralizados que você possa ter usado anteriormente. Você também deve ter agora uma versão funcional do Git em seu sistema que está configurada com sua identidade pessoal. Agora é a hora de aprender alguns fundamentos do Git.

Fundamentos do Git

Se você puder ler apenas um capítulo para começar a usar o Git, é este. Este capítulo cobre todos os comandos básicos que você precisa para fazer a grande maioria das coisas que você passará seu tempo fazendo com o Git. Ao final do capítulo, você deverá ser capaz de configurar e inicializar um repositório, começar e parar de rastrear arquivos, além de preparar (stage) e fazer commit de alterações. Também mostraremos como configurar o Git para ignorar determinados arquivos e padrões de arquivo, como desfazer erros de maneira rápida e fácil, como navegar pelo histórico do seu projeto e visualizar alterações entre commits, e como enviar (push) e puxar (pull) de repositórios remotos.

Obtendo um Repositório Git

Você normalmente obtém um repositório Git de uma de duas maneiras:

1. Você pode pegar um diretório local que não esteja sob controle de versão, e transformá-lo em um repositório Git, ou
2. Você pode *clonar* um repositório Git existente de outro lugar.

Em ambos os casos, você acaba com um repositório Git em sua máquina local, pronto para o trabalho.

Inicializando um repositório em um diretório existente

Se você tem um diretório de projeto que não está atualmente sob controle de versão e deseja começar a controlá-lo com o Git, você primeiro precisa ir para o diretório desse projeto. Se você nunca fez isso, parece um pouco diferente dependendo de qual sistema você está executando:

para Linux:

```
$ cd /home/user/my_project
```

para macOS:

```
$ cd /Users/user/my_project
```

para Windows:

```
$ cd C:/Users/user/my_project
```

e digite:

```
$ git init
```

Isso cria um novo subdiretório chamado `.git` que contém todos os arquivos de repositório necessários — um esqueleto de repositório Git. Neste ponto, nada em seu projeto é rastreado ainda. Veja Git Internals (Por Dentro do Git) para mais informações sobre exatamente quais arquivos estão contidos no diretório `.git` que você acabou de criar.

Se você quiser começar a controlar a versão de arquivos existentes (em oposição a um diretório vazio), você provavelmente deve começar a rastrear esses arquivos e fazer um commit inicial. Você pode realizar isso com alguns comandos `git add` que especificam os arquivos que você deseja rastrear, seguidos por um `git commit`:

```
$ git add *.c
$ git add LICENSE
$ git commit -m 'Initial project version'
```

Vamos analisar o que esses comandos fazem em apenas um minuto. Neste ponto, você tem um repositório Git com arquivos rastreados e um commit inicial.

Clonando um Repositório Existente

Se você quiser obter uma cópia de um repositório Git existente — por exemplo, um projeto para o qual você gostaria de contribuir — o comando de que você precisa é `git clone`. Se você está familiarizado com outros VCSs como Subversion, notará que o comando é "clone" e não "checkout". Essa é uma distinção importante — em vez de obter apenas uma cópia de trabalho, o Git recebe uma cópia completa de quase todos os dados que o servidor possui. Toda versão de todo arquivo do histórico do projeto é baixada (pulled down) por padrão quando você executa `git clone`. De fato, se o disco do seu servidor for corrompido, muitas vezes você pode usar quase qualquer um dos clones em qualquer cliente para restaurar o servidor ao estado em que estava quando foi clonado (você pode perder alguns hooks do lado do servidor e coisas do tipo, mas todos os dados versionados estariam lá — veja Colocando o Git em um Servidor para mais detalhes).

Você clona um repositório com `git clone <url>`. Por exemplo, se você quiser clonar a biblioteca vinculável (linkable library) do Git chamada `libgit2`, pode fazer isso assim:

```
$ git clone https://github.com/libgit2/libgit2
```

Isso cria um diretório chamado `libgit2`, inicializa um diretório `.git` dentro dele, baixa todos os dados para aquele repositório e faz o checkout de uma cópia de trabalho da versão mais recente. Se você entrar no novo diretório `libgit2` que acabou de ser criado, verá os arquivos do projeto lá, prontos para serem trabalhados ou usados.

Se você quiser clonar o repositório em um diretório com um nome diferente de `libgit2`, pode especificar o novo nome do diretório como um argumento adicional:

```
$ git clone https://github.com/libgit2/libgit2 mylibgit
```

Esse comando faz a mesma coisa que o anterior, mas o diretório de destino é chamado `mylibgit`.

O Git possui vários protocolos de transferência diferentes que você pode usar. O exemplo anterior usa o protocolo `https://`, mas você também pode ver `git://` ou `user@server:path/to/repo.git`, que usa o protocolo de transferência SSH. Colocando o Git em um Servidor apresentará todas as opções disponíveis que o servidor pode configurar para acessar seu repositório Git e os prós e contras de cada uma.

Gravando Alterações no Repositório

Neste ponto, você deve ter um repositório Git *bona fide* na sua máquina local e um checkout ou *cópia de trabalho* (*working copy*) de todos os seus arquivos à sua frente. Normalmente, você vai querer começar a fazer alterações e fazer o commit de snapshots dessas alterações no seu repositório cada vez que o projeto atingir um estado que você deseja gravar.

Lembre-se de que cada arquivo no seu diretório de trabalho pode estar em um de dois estados: *rastreado* (*tracked*) ou *não rastreado* (*untracked*). Arquivos rastreados são arquivos que estavam no último snapshot, bem como quaisquer arquivos recém-preparados; eles podem estar não modificados, modificados ou preparados (staged). Em resumo, arquivos rastreados são arquivos que o Git conhece.

Arquivos não rastreados são tudo o mais — quaisquer arquivos no seu diretório de trabalho que não estavam no seu último snapshot e não estão na sua área de preparação. Quando você clona um repositório pela primeira vez, todos os seus arquivos estarão rastreados e não modificados, porque o Git acabou de fazer o checkout deles e você não editou nada.

À medida que você edita os arquivos, o Git os vê como modificados, porque você os alterou desde o último commit. Conforme você trabalha, você prepara seletivamente esses arquivos modificados e, em seguida, faz o commit de todas essas alterações preparadas, e o ciclo se repete.

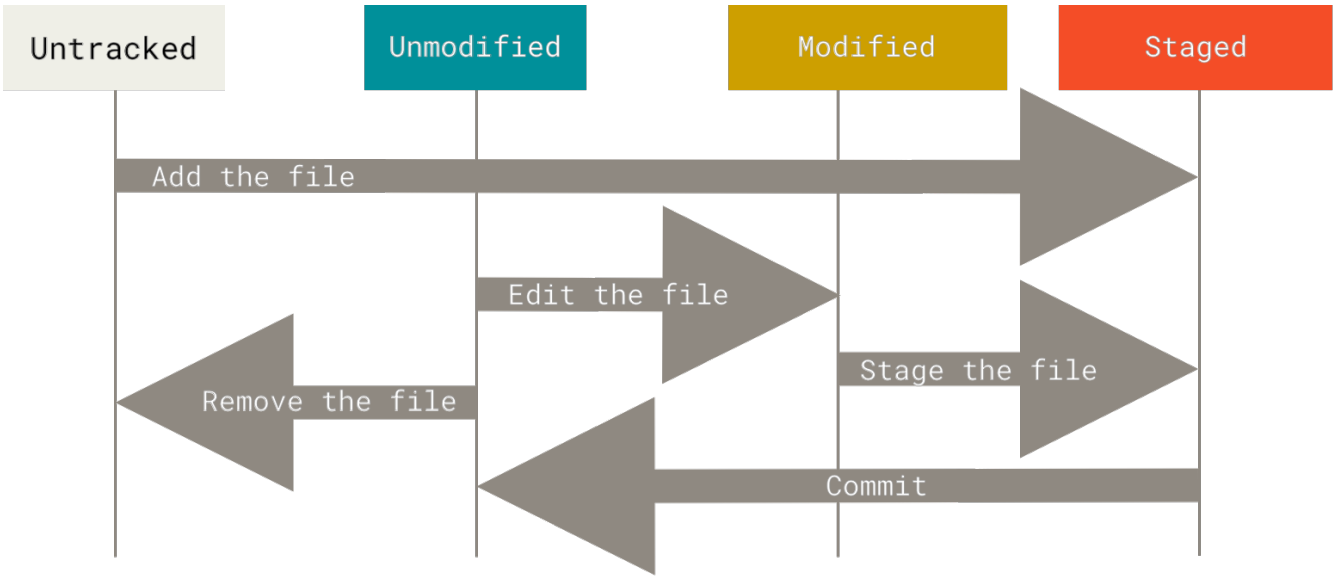


Figure 8. O ciclo de vida do status dos seus arquivos

Verificando o status dos seus arquivos

A principal ferramenta que você usa para determinar quais arquivos estão em qual estado é o comando `git status`. Se você executar este comando logo após um clone, deverá ver algo assim:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working tree clean
```

Isso significa que você tem um diretório de trabalho limpo; em outras palavras, nenhum dos seus arquivos rastreados foi modificado. O Git também não vê nenhum arquivo não rastreado, ou eles estariam listados aqui. Por fim, o comando informa em qual branch você está e informa que não divergiu do mesmo branch no servidor. Por enquanto, esse branch é sempre `master`, que é o padrão; você não vai se preocupar com isso aqui. Ramificação (Branching) no Git abordará branches e referências em detalhes.



O GitHub mudou o nome padrão do branch de `master` para `main` em meados de 2020, e outros hosts Git seguiram o exemplo. Portanto, você pode descobrir que o nome padrão do branch em alguns repositórios recém-criados é `main` e não `master`. Além disso, o nome padrão do branch pode ser alterado (como você viu em O nome do seu branch padrão), então você pode ver um nome diferente para o branch padrão.

No entanto, o próprio Git ainda usa `master` como padrão, então nós o usaremos em todo o livro.

Digamos que você adicione um novo arquivo ao seu projeto, um arquivo `README` simples. Se o arquivo não existia antes e você executar `git status`, verá seu arquivo não rastreado assim:

```
$ echo 'My Project' > README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)

    README

nothing added to commit but untracked files present (use "git add" to track)
```

Você pode ver que seu novo arquivo `README` não é rastreado, porque está sob o cabeçalho “Untracked files” na saída do status. Não rastreado basicamente significa que o Git vê um arquivo que você não tinha no snapshot anterior (commit) e que ainda não foi preparado; o Git não começará a incluí-lo em seus snapshots de commit até que você diga explicitamente a ele para fazer isso. Ele faz isso para que você não comece acidentalmente a incluir arquivos binários gerados ou outros arquivos que você não pretendia incluir. Você quer começar a incluir o `README`, então vamos começar a rastrear o arquivo.

Rastreando Novos Arquivos

Para começar a rastrear um novo arquivo, você usa o comando `git add`. Para começar a rastrear o

arquivo `README`, você pode executar isto:

```
$ git add README
```

Se você executar seu comando status novamente, poderá ver que seu arquivo `README` agora está rastreado e preparado para ser commitado:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)

    new file:   README
```

Você pode dizer que ele está preparado porque está sob o cabeçalho “Changes to be committed”. Se você fizer o commit neste ponto, a versão do arquivo no momento em que você executou o `git add` é a que estará no snapshot histórico subsequente. Você deve se lembrar que, quando executou o `git init` anteriormente, executou o `git add <arquivos>` em seguida — isso foi para começar a rastrear arquivos no seu diretório. O comando `git add` recebe um nome de caminho para um arquivo ou diretório; se for um diretório, o comando adiciona todos os arquivos desse diretório recursivamente.

Preparando Arquivos Modificados

Vamos alterar um arquivo que já estava rastreado. Se você alterar um arquivo previamente rastreado chamado `CONTRIBUTING.md` e depois executar seu comando `git status` novamente, obterá algo parecido com isto:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

O arquivo `CONTRIBUTING.md` aparece em uma seção chamada “Changes not staged for commit” — o que significa que um arquivo que é rastreado foi modificado no diretório de trabalho, mas ainda não foi preparado. Para prepará-lo, você executa o comando `git add`. `git add` é um comando

polivalente — você o usa para começar a rastrear novos arquivos, preparar arquivos e fazer outras coisas como marcar arquivos com conflito de mesclagem (merge-conflicted files) como resolvidos. Pode ser útil pensar nele mais como “adicione precisamente este conteúdo ao próximo commit” em vez de “adicione este arquivo ao projeto”. Vamos executar `git add` agora para preparar o arquivo `CONTRIBUTING.md` e depois executar `git status` novamente:

```
$ git add CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md
```

Ambos os arquivos estão preparados e irão para o seu próximo commit. Neste ponto, suponha que você se lembre de uma pequena alteração que deseja fazer no `CONTRIBUTING.md` antes de fazer o commit dele. Você o abre novamente e faz essa alteração, e está pronto para fazer o commit. No entanto, vamos executar `git status` mais uma vez:

```
$ vim CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

Que diabos? Agora o `CONTRIBUTING.md` está listado como preparado *e* não preparado. Como isso é possível? Acontece que o Git prepara um arquivo exatamente como ele é quando você executa o comando `git add`. Se você fizer o commit agora, a versão de `CONTRIBUTING.md` como era quando você executou o comando `git add` pela última vez é como ela entrará no commit, não a versão do arquivo como aparece no seu diretório de trabalho quando você executa `git commit`. Se você modificar um arquivo depois de executar `git add`, precisará executar `git add` novamente para preparar a versão mais recente do arquivo:

```
$ git add CONTRIBUTING.md
$ git status
```

```
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
    modified:   CONTRIBUTING.md
```

Status Curto

Embora a saída de `git status` seja bem abrangente, ela também é bastante prolixa. O Git também tem uma flag de status curto (short status) para que você possa ver suas alterações de uma maneira mais compacta. Se você executar `git status -s` ou `git status --short`, obterá uma saída muito mais simplificada do comando:

```
$ git status -s
 M README
MM Rakefile
A  lib/git.rb
M  lib/simplegit.rb
?? LICENSE.txt
```

Arquivos novos que não são rastreados têm um `??` ao lado deles, arquivos novos que foram adicionados à área de preparação têm um `A`, arquivos modificados têm um `M` e assim por diante. Existem duas colunas na saída — a coluna da esquerda indica o status da área de preparação e a coluna da direita indica o status da árvore de trabalho (working tree). Portanto, por exemplo nessa saída, o arquivo `README` foi modificado no diretório de trabalho, mas ainda não foi preparado, enquanto o arquivo `lib/simplegit.rb` foi modificado e preparado. O `Rakefile` foi modificado, preparado e modificado novamente, portanto, há alterações nele que estão preparadas e não preparadas.

Ignorando Arquivos

Geralmente, você terá uma classe de arquivos que não deseja que o Git adicione automaticamente ou até mesmo mostre a você como não rastreados. Estes geralmente são arquivos gerados automaticamente, como arquivos de log ou arquivos produzidos pelo seu sistema de compilação (build system). Nesses casos, você pode criar um arquivo listando padrões que correspondam a eles, chamado `.gitignore`. Aqui está um exemplo de arquivo `.gitignore`:

```
$ cat .gitignore
*.[oa]
*~
```

A primeira linha diz ao Git para ignorar quaisquer arquivos que terminem em “.o” ou “.a” — arquivos de objeto e de arquivamento (archive) que podem ser o produto da compilação do seu código. A segunda linha diz ao Git para ignorar todos os arquivos cujos nomes terminam com

um til (`~`), o que é usado por muitos editores de texto como o Emacs para marcar arquivos temporários. Você também pode incluir um diretório de log, tmp ou pid; documentação gerada automaticamente; e assim por diante. Configurar um arquivo `.gitignore` para o seu novo repositório antes de começar é geralmente uma boa ideia, para que você não faça o commit acidentalmente de arquivos que você realmente não deseja em seu repositório Git.

As regras para os padrões que você pode colocar no arquivo `.gitignore` são as seguintes:

- Linhas em branco ou linhas começando com `#` são ignoradas.
- Os padrões de glob (glob patterns) padrão funcionam e serão aplicados recursivamente em toda a árvore de trabalho.
- Você pode iniciar os padrões com uma barra normal (`/`) para evitar a recursividade.
- Você pode terminar os padrões com uma barra normal (`/`) para especificar um diretório.
- Você pode negar um padrão iniciando-o com um ponto de exclamação (`!`).

Os padrões de glob são como expressões regulares simplificadas que os shells usam. Um asterisco (`*`) corresponde a zero ou mais caracteres; `[abc]` corresponde a qualquer caractere dentro dos colchetes (neste caso, a, b ou c); um ponto de interrogação (`?`) corresponde a um único caractere; e os caracteres dentro de colchetes separados por um hífen (`[0-9]`) correspondem a qualquer caractere entre eles (neste caso, de 0 a 9). Você também pode usar dois asteriscos para corresponder a diretórios aninhados; `a/**/z` corresponderia a `a/z`, `a/b/z`, `a/b/c/z` e assim por diante.

Aqui está outro exemplo de arquivo `.gitignore`:

```
# ignore all .a files
*.a

# but do track lib.a, even though you're ignoring .a files above
!lib.a

# only ignore the TODO file in the current directory, not subdir/TODO
/TODO

# ignore all files in any directory named build
build/

# ignore doc/notes.txt, but not doc/server/arch.txt
doc/*.txt

# ignore all .pdf files in the doc/ directory and any of its subdirectories
doc/**/*.pdf
```



O GitHub mantém uma lista bastante abrangente de bons exemplos de arquivos `.gitignore` para dezenas de projetos e linguagens em https://github.com/github/gitignore se você quiser um ponto de partida para o seu projeto.



No caso simples, um repositório pode ter um único arquivo `.gitignore` em seu

> diretório raiz, que se aplica recursivamente a todo o repositório. No entanto, também é possível ter arquivos `.gitignore` adicionais em subdiretórios. As regras nesses arquivos `.gitignore` aninhados se aplicam apenas aos arquivos no diretório onde estão localizados. O repositório de código-fonte do kernel do Linux tem 206 arquivos `.gitignore`.
>
> Está além do escopo deste livro entrar em detalhes de vários arquivos `.gitignore`; veja `man gitignore` para obter os detalhes.

Visualizando suas Alterações Preparadas e Não Preparadas

Se o comando `git status` for muito vago para você — você quer saber exatamente o que você alterou, não apenas quais arquivos foram alterados — você pode usar o comando `git diff`. Abordaremos o `git diff` com mais detalhes posteriormente, mas você provavelmente o usará com mais frequência para responder a estas duas perguntas: O que você alterou, mas ainda não preparou? E o que você preparou que está prestes a fazer o commit? Embora `git status` responda a essas perguntas de maneira muito geral, listando os nomes dos arquivos, `git diff` mostra as linhas exatas adicionadas e removidas — o patch, por assim dizer.

Digamos que você edite e prepare o arquivo `README` novamente e, em seguida, edite o arquivo `CONTRIBUTING.md` sem prepará-lo. Se você executar seu comando `git status`, verá mais uma vez algo assim:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

Para ver o que você alterou, mas ainda não preparou, digite `git diff` sem outros argumentos:

```
$ git diff
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 8ebb991..643e24f 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -65,7 +65,8 @@ branch directly, things can get messy.
 Please include a nice description of your changes when you submit your PR;
 if we have to read the whole diff to figure out why you're contributing
 in the first place, you're less likely to get feedback and have your change
```

```
-merged in.
+merged in. Also, split your changes into comprehensive chunks if your patch is
+longer than a dozen lines.

 If you are starting to work on a particular area, feel free to submit a PR
 that highlights your work in progress (and note in the PR title that it's
```

Esse comando compara o que está no seu diretório de trabalho com o que está na sua área de preparação. O resultado informa as alterações que você fez e que ainda não preparou.

Se você quiser ver o que você preparou e que irá para o seu próximo commit, pode usar `git diff --staged`. Este comando compara suas alterações preparadas com o seu último commit:

```
$ git diff --staged
diff --git a/README b/README
new file mode 100644
index 0000000..03902a1
--- /dev/null
+++ b/README
@@ -0,0 +1 @@
+My Project
```

É importante notar que `git diff` por si só não mostra todas as alterações feitas desde o seu último commit — apenas as alterações que ainda não foram preparadas. Se você tiver preparado todas as suas alterações, o `git diff` não fornecerá nenhuma saída.

Como outro exemplo, se você preparar o arquivo `CONTRIBUTING.md` e depois editá-lo, poderá usar `git diff` para ver as alterações no arquivo que estão preparadas e as alterações que não estão preparadas. Se o nosso ambiente for assim:

```
$ git add CONTRIBUTING.md
$ echo '# test line' >> CONTRIBUTING.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   CONTRIBUTING.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

Agora você pode usar `git diff` para ver o que ainda não foi preparado:

```
$ git diff
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 643e24f..87f08c8 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -119,3 +119,4 @@ at the
 ## Starter Projects

 See our [projects
list](https://github.com/libgit2/libgit2/blob/development/PROJECTS.md).
+# test line
```

e `git diff --cached` para ver o que você preparou até agora (`--staged` e `--cached` são sinônimos):

```
$ git diff --cached
diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md
index 8ebb991..643e24f 100644
--- a/CONTRIBUTING.md
+++ b/CONTRIBUTING.md
@@ -65,7 +65,8 @@ branch directly, things can get messy.
 Please include a nice description of your changes when you submit your PR;
 if we have to read the whole diff to figure out why you're contributing
 in the first place, you're less likely to get feedback and have your change
-merged in.
+merged in. Also, split your changes into comprehensive chunks if your patch is
+longer than a dozen lines.

 If you are starting to work on a particular area, feel free to submit a PR
 that highlights your work in progress (and note in the PR title that it's
```



Git Diff em uma Ferramenta Externa

Continuaremos a usar o comando `git diff` de várias maneiras ao longo do resto do livro. Há outra maneira de ver esses diffs se você preferir um programa de visualização de diff gráfico ou externo. Se você executar `git difftool` em vez de `git diff`, poderá visualizar qualquer um desses diffs em softwares como emerge, vimdiff e muitos outros (incluindo produtos comerciais). Execute `git difftool --tool-help` para ver o que está disponível em seu sistema.

Fazendo o Commit de suas Alterações

Agora que sua área de preparação está configurada do jeito que você deseja, você pode fazer o commit de suas alterações. Lembre-se de que qualquer coisa que ainda não esteja preparada — quaisquer arquivos que você criou ou modificou e nos quais não executou `git add` desde que os editou — não entrará neste commit. Eles permanecerão como arquivos modificados no seu disco. Neste caso, digamos que da última vez que você executou `git status`, viu que tudo estava preparado, então você está pronto para fazer o commit das suas alterações. A maneira mais simples de fazer o commit é digitar `git commit`:

```
$ git commit
```

Isso inicia o seu editor de escolha.



Isso é definido pela variável de ambiente `EDITOR` do seu shell — geralmente vim ou emacs, embora você possa configurá-lo com o que quiser usando o comando `git config --global core.editor`, como você viu em Primeiros Passos.

O editor exibe o seguinte texto (este exemplo é uma tela do Vim):

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch master
# Your branch is up-to-date with 'origin/master'.
#
# Changes to be committed:
#   new file:   README
#   modified:   CONTRIBUTING.md
#
~
~
~
".git/COMMIT_EDITMSG" 9L, 283C
```

Você pode ver que a mensagem de commit padrão contém a saída mais recente do comando `git status` comentada e uma linha vazia no topo. Você pode remover esses comentários e digitar sua mensagem de commit, ou pode deixá-los lá para ajudá-lo a lembrar o que está commitando.



Para um lembrete ainda mais explícito do que você modificou, você pode passar a opção `-v` para o `git commit`. Fazer isso também coloca o diff de sua alteração no editor para que você possa ver exatamente quais alterações está commitando.

Quando você sai do editor, o Git cria o seu commit com essa mensagem de commit (com os comentários e o diff removidos).

Alternativamente, você pode digitar sua mensagem de commit embutida no comando `commit` especificando-a após a flag `-m`, assim:

```
$ git commit -m "Story 182: fix benchmarks for speed"
[master 463dc4f] Story 182: fix benchmarks for speed
 2 files changed, 2 insertions(+)
 create mode 100644 README
```

Agora você criou o seu primeiro commit! Você pode ver que o commit forneceu a você algumas saídas sobre si mesmo: em qual branch você fez o commit (`master`), qual checksum SHA-1 o commit tem (`463dc4f`), quantos arquivos foram alterados e estatísticas sobre as linhas adicionadas e

removidas no commit.

Lembre-se de que o commit registra o snapshot que você configurou na sua área de preparação. Qualquer coisa que você não tenha preparado ainda está lá modificada; você pode fazer outro commit para adicioná-la ao seu histórico. Toda vez que você realiza um commit, está registrando um snapshot do seu projeto para o qual você pode reverter ou comparar posteriormente.

Ignorando a Área de Preparação

Embora possa ser incrivelmente útil para criar commits exatamente da forma como você os deseja, a área de preparação às vezes é um pouco mais complexa do que você precisa em seu fluxo de trabalho. Se você quiser pular a área de preparação, o Git fornece um atalho simples. Adicionar a opção `-a` ao comando `git commit` faz com que o Git prepare automaticamente todos os arquivos que já estão sendo rastreados antes de fazer o commit, permitindo que você ignore a parte do `git add`:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md

no changes added to commit (use "git add" and/or "git commit -a")
$ git commit -a -m 'Add new benchmarks'
[master 83e38c7] Add new benchmarks
 1 file changed, 5 insertions(+), 0 deletions(-)
```

Observe como você não precisa executar `git add` no arquivo `CONTRIBUTING.md` neste caso antes de fazer o commit. Isso ocorre porque a flag `-a` inclui todos os arquivos alterados. Isso é conveniente, mas tenha cuidado; às vezes, essa flag fará com que você inclua alterações indesejadas.

Removendo Arquivos

Para remover um arquivo do Git, você deve removê-lo dos seus arquivos rastreados (mais precisamente, removê-lo da sua área de preparação) e, em seguida, fazer o commit. O comando `git rm` faz isso e também remove o arquivo do seu diretório de trabalho para que você não o veja como um arquivo não rastreado na próxima vez.

Se você simplesmente remover o arquivo do seu diretório de trabalho, ele aparecerá na área "Changes not staged for commit" (ou seja, *não preparado*) da saída do seu `git status`:

```
$ rm PROJECTS.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
```

```
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        deleted:    PROJECTS.md

no changes added to commit (use "git add" and/or "git commit -a")
```

Então, se você executar `git rm`, ele preparará a remoção do arquivo:

```
$ git rm PROJECTS.md
rm 'PROJECTS.md'
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    deleted:    PROJECTS.md
```

Na próxima vez que você fizer commit, o arquivo não existirá mais e não será mais rastreado. Se você modificou o arquivo ou já o adicionou à área de preparação, deve forçar a remoção com a opção `-f`. Esse é um recurso de segurança para evitar a remoção acidental de dados que ainda não foram gravados em um snapshot e que não podem ser recuperados do Git.

Outra coisa útil que você pode querer fazer é manter o arquivo na sua árvore de trabalho, mas removê-lo da sua área de preparação. Em outras palavras, você pode querer manter o arquivo no seu disco rígido, mas não ter mais o Git o rastreando. Isso é particularmente útil se você esqueceu de adicionar algo ao seu arquivo `.gitignore` e acidentalmente o preparou, como um arquivo de log grande ou um monte de arquivos `.a` compilados. Para fazer isso, use a opção `--cached`:

```
$ git rm --cached README
```

Você pode passar arquivos, diretórios e padrões de file-glob para o comando `git rm`. Isso significa que você pode fazer coisas como:

```
$ git rm log/\*.log
```

Observe a barra invertida (`\`) na frente do `*`. Isso é necessário porque o Git faz sua própria expansão de nome de arquivo, além da expansão de nome de arquivo do seu shell. Esse comando remove todos os arquivos que têm a extensão `.log` no diretório `log/`. Ou você pode fazer algo assim:

```
$ git rm \*~
```

Este comando remove todos os arquivos cujos nomes terminam com um `~`.

Movendo Arquivos

Ao contrário de muitos outros VCSs, o Git não rastreia explicitamente o movimento de arquivos. Se você renomear um arquivo no Git, nenhum metadado será armazenado no Git que informe que você renomeou o arquivo. No entanto, o Git é muito inteligente em descobrir isso depois do fato — lidaremos com a detecção de movimento de arquivos um pouco mais tarde.

Portanto, é um pouco confuso que o Git tenha um comando `mv`. Se você quiser renomear um arquivo no Git, você pode executar algo como:

```
$ git mv file_from file_to
```

e funciona perfeitamente. De fato, se você executar algo assim e observar o status, verá que o Git o considera um arquivo renomeado:

```
$ git mv README.md README
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
```

No entanto, isso é equivalente a executar algo assim:

```
$ mv README.md README
$ git rm README.md
$ git add README
```

O Git descobre que é uma renomeação implicitamente, portanto, não importa se você renomeia um arquivo dessa maneira ou com o comando `mv`. A única diferença real é que `git mv` é um comando em vez de três — é uma função de conveniência. Mais importante ainda, você pode usar qualquer ferramenta que desejar para renomear um arquivo e tratar o `add`/`rm` depois, antes de fazer o commit.

Visualizando o Histórico de Commits

Depois de criar vários commits, ou se você tiver clonado um repositório com um histórico de commits existente, você provavelmente vai querer olhar para trás para ver o que aconteceu. A ferramenta mais básica e poderosa para fazer isso é o comando `git log`.

Estes exemplos usam um projeto muito simples chamado “simplegit”. Para obter o projeto, execute:

```
$ git clone https://github.com/schacon/simplegit-progit
```

Quando você executa `git log` neste projeto, você deve obter uma saída semelhante a esta:

```
$ git log
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 16:40:33 2008 -0700

    Remove unnecessary test

commit a11bef06a3f659402fe7563abf99ad00de2209e6
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 10:31:28 2008 -0700

    Initial commit
```

Por padrão, sem argumentos, o `git log` lista os commits feitos naquele repositório em ordem cronológica reversa; ou seja, os commits mais recentes aparecem primeiro. Como você pode ver, este comando lista cada commit com seu checksum SHA-1, o nome e e-mail do autor, a data em que foi escrito e a mensagem de commit.

Um grande número e variedade de opções para o comando `git log` estão disponíveis para mostrar exatamente o que você está procurando. Aqui, mostraremos a você algumas das mais populares.

Uma das opções mais úteis é `-p` ou `--patch`, que mostra a diferença (a saída do *patch*) introduzida em cada commit. Você também pode limitar o número de entradas de log exibidas, como usar `-2` para mostrar apenas as duas últimas entradas.

```
$ git log -p -2
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number

diff --git a/Rakefile b/Rakefile
index a874b73..8f94139 100644
--- a/Rakefile
+++ b/Rakefile
@@ -5,7 +5,7 @@ require 'rake/gempackagetask'
 spec = Gem::Specification.new do |s|
     s.platform  =   Gem::Platform::RUBY
     s.name      =   "simplegit"
-    s.version   =   "0.1.0"
```

```
+    s.version   =   "0.1.1"
     s.author    =   "Scott Chacon"
     s.email     =   "schacon@gee-mail.com"
     s.summary   =   "A simple gem for using Git in Ruby code."

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 16:40:33 2008 -0700

    Remove unnecessary test

diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index a0a60ae..47c6340 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -18,8 +18,3 @@ class SimpleGit
     end

 end
-
-if $0 == __FILE__
-  git = SimpleGit.new
-  puts git.show
-end
```

Esta opção exibe as mesmas informações, mas com um diff logo após cada entrada. Isso é muito útil para revisão de código ou para navegar rapidamente pelo que aconteceu durante uma série de commits que um colaborador adicionou. Você também pode usar uma série de opções de resumo com o `git log`. Por exemplo, se você quiser ver algumas estatísticas abreviadas para cada commit, pode usar a opção `--stat`:

```
$ git log --stat
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number

 Rakefile | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 16:40:33 2008 -0700

    Remove unnecessary test

 lib/simplegit.rb | 5 -----
 1 file changed, 5 deletions(-)
```

```
commit a11bef06a3f659402fe7563abf99ad00de2209e6
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Sat Mar 15 10:31:28 2008 -0700

    Initial commit

 README           |  6 ++++++
 Rakefile         | 23 +++++++++++++++++++++
 lib/simplegit.rb | 25 +++++++++++++++++++++++
 3 files changed, 54 insertions(+)
```

Como você pode ver, a opção `--stat` imprime abaixo de cada entrada de commit uma lista de arquivos modificados, quantos arquivos foram alterados e quantas linhas nesses arquivos foram adicionadas e removidas. Ela também coloca um resumo das informações no final.

Outra opção muito útil é `--pretty`. Esta opção altera a saída do log para formatos diferentes do padrão. Existem alguns valores de opções predefinidos disponíveis para você usar. O valor `oneline` para esta opção imprime cada commit em uma única linha, o que é útil se você estiver observando muitos commits. Além disso, os valores `short`, `full` e `fuller` mostram a saída aproximadamente no mesmo formato, mas com menos ou mais informações, respectivamente:

```
$ git log --pretty=oneline
ca82a6dff817ec66f44342007202690a93763949 Change version number
085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Remove unnecessary test
a11bef06a3f659402fe7563abf99ad00de2209e6 Initial commit
```

O valor de opção mais interessante é `format`, que permite que você especifique seu próprio formato de saída de log. Isso é especialmente útil quando você está gerando saída para análise (parsing) por máquina — como você especifica o formato explicitamente, sabe que ele não será alterado com as atualizações do Git:

```
$ git log --pretty=format:"%h - %an, %ar : %s"
ca82a6d - Scott Chacon, 6 years ago : Change version number
085bb3b - Scott Chacon, 6 years ago : Remove unnecessary test
a11bef0 - Scott Chacon, 6 years ago : Initial commit
```

Especificadores úteis para `git log --pretty=format` lista alguns dos especificadores mais úteis que o `format` aceita.

Table 1. Especificadores úteis para `git log --pretty=format`

Especificador	Descrição da Saída
`%H`	Hash do commit
`%h`	Hash do commit abreviado
`%T`	Hash da árvore (tree)
`%t`	Hash da árvore abreviado

Especificador	Descrição da Saída
`%P`	Hashes pais (parent hashes)
`%p`	Hashes pais abreviados
`%an`	Nome do autor
`%ae`	E-mail do autor
`%ad`	Data do autor (o formato respeita a opção `--date=`)
`%ar`	Data do autor, relativa
`%cn`	Nome do committer
`%ce`	E-mail do committer
`%cd`	Data do committer
`%cr`	Data do committer, relativa
`%s`	Assunto

Você deve estar se perguntando qual é a diferença entre *autor* (*author*) e *committer*. O autor é a pessoa que escreveu o trabalho originalmente, ao passo que o committer é a pessoa que aplicou o trabalho por último. Então, se você enviar um patch para um projeto e um dos membros principais (core members) aplicar o patch, ambos receberão o crédito — você como autor e o membro principal como o committer. Abordaremos essa distinção um pouco mais em Git Distribuído.

Os valores de opção `oneline` e `format` são particularmente úteis com outra opção `log` chamada `--graph`. Esta opção adiciona um pequeno gráfico ASCII bacana mostrando o seu histórico de branches e merges:

```
$ git log --pretty=format:"%h %s" --graph
* 2d3acf9 Ignore errors from SIGCHLD on trap
*  5e3ee11 Merge branch 'master' of https://github.com/dustin/grit.git
|\
| * 420eac9 Add method for getting the current branch
* | 30e367c Timeout code and tests
* | 5a09431 Add timeout protection to grit
* | e1193f8 Support for heads with slashes in them
|/
* d6016bc Require time for xmlschema
*  11d191e Merge branch 'defunkt' into local
```

Este tipo de saída se tornará mais interessante conforme passarmos por branches e merges no próximo capítulo.

Essas são apenas algumas opções simples de formatação de saída para o `git log` — há muitas outras. Opções comuns para `git log` lista as opções que cobrimos até agora, bem como algumas outras opções comuns de formatação que podem ser úteis, junto com a forma como elas alteram a saída do comando `log`.

Table 2. Opções comuns para `git log`

Opção	Descrição
`-p`	Mostra o patch introduzido com cada commit.
`--stat`	Mostra estatísticas de arquivos modificados em cada commit.
`--shortstat`	Exibe apenas a linha changed/insertions/deletions do comando `--stat`.
`--name-only`	Mostra a lista de arquivos modificados após as informações do commit.
`--name-status`	Mostra a lista de arquivos afetados com informações de added/modified/deleted também.
`--abbrev-commit`	Mostra apenas os primeiros caracteres do checksum SHA-1 em vez de todos os 40.
`--relative-date`	Exibe a data em um formato relativo (por exemplo, “2 weeks ago”) em vez de usar o formato de data completo.
`--graph`	Exibe um gráfico ASCII da história de branches e merges ao lado da saída de log.
`--pretty`	Mostra commits em um formato alternativo. Os valores de opções incluem `oneline`, `short`, `full`, `fuller` e `format` (onde você especifica seu próprio formato).
`--oneline`	Uma forma abreviada de `--pretty=oneline --abbrev-commit` usados em conjunto.

Limitando a Saída do Log

Além das opções de formatação de saída, o `git log` aceita várias opções úteis de limitação; ou seja, opções que permitem exibir apenas um subconjunto de commits. Você já viu uma dessas opções — a opção `-2`, que exibe apenas os dois últimos commits. De fato, você pode usar `-<n>`, onde `n` é qualquer número inteiro para mostrar os últimos `n` commits. Na realidade, é improvável que você o use com frequência, porque, por padrão, o Git envia (pipes) toda a saída por meio de um pager para que você veja apenas uma página de saída de log por vez.

No entanto, as opções de limite de tempo, como `--since` e `--until`, são muito úteis. Por exemplo, este comando obtém a lista de commits feitos nas últimas duas semanas:

```
$ git log --since=2.weeks
```

Este comando funciona com vários formatos — você pode especificar uma data específica como `"2008-01-15"`, ou uma data relativa como `"2 years 1 day 3 minutes ago"`.

Você também pode filtrar a lista para os commits que correspondam a alguns critérios de pesquisa. A opção `--author` permite filtrar um autor específico, e a opção `--grep` permite pesquisar palavras-chave nas mensagens de commit.



Você pode especificar mais de uma instância dos critérios de pesquisa `--author` e `--grep`, o que limitará a saída dos commits aos commits que correspondem a *qualquer* um dos padrões `--author` e *qualquer* um dos padrões `--grep`; no entanto, adicionar a opção `--all-match` limita ainda mais a saída apenas aos commits que

> correspondem a *todos* os padrões `--grep`.

Outro filtro realmente útil é a opção `-S` (coloquialmente chamada de opção “pickaxe” do Git), que recebe uma string e mostra apenas os commits que alteraram o número de ocorrências dessa string. Por exemplo, se você quisesse encontrar o último commit que adicionou ou removeu uma referência a uma função específica, você poderia chamar:

```
$ git log -S function_name
```

A última opção realmente útil para passar para `git log` como filtro é um caminho. Se você especificar um diretório ou nome de arquivo, poderá limitar a saída do log aos commits que introduziram uma alteração nesses arquivos. Essa é sempre a última opção e geralmente é precedida por dois hifens (`--`) para separar os caminhos das opções:

```
$ git log -- path/to/file
```

Em Opções para limitar a saída de `git log`, listaremos estas e algumas outras opções comuns para sua referência.

Table 3. Opções para limitar a saída de `git log`

Opção	Descrição
`-<n>`	Mostra apenas os últimos n commits.
`--since`, `--after`	Limita os commits aos feitos após a data especificada.
`--until`, `--before`	Limita os commits aos feitos antes da data especificada.
`--author`	Mostra apenas os commits em que a entrada do autor corresponda à string especificada.
`--committer`	Mostra apenas os commits em que a entrada do committer corresponda à string especificada.
`--grep`	Mostra apenas os commits com uma mensagem de commit contendo a string.
`-S`	Mostra apenas os commits adicionando ou removendo código correspondente à string.

Por exemplo, se você quiser ver quais commits que modificaram os arquivos de teste no histórico de código-fonte do Git foram commitados por Junio Hamano no mês de outubro de 2008 e não são commits de mesclagem (merge commits), você pode executar algo assim:

```
$ git log --pretty="%h - %s" --author='Junio C Hamano' --since="2008-10-01" \
   --before="2008-11-01" --no-merges -- t/
5610e3b - Fix testcase failure when extended attributes are in use
acd3b9e - Enhance hold_lock_file_for_{update,append}() API
f563754 - demonstrate breakage of detached checkout with symbolic link HEAD
d1a43f2 - reset --hard/read-tree --reset -u: remove unmerged new paths
```

```
51a94af - Fix "checkout --track -b newbranch" on detached HEAD
b0ad11e - pull: allow "git pull origin $something:$current_branch" into an unborn
branch
```

Dos quase 40.000 commits no histórico do código-fonte do Git, esse comando mostra os 6 que correspondem a esses critérios.



Prevenindo a exibição de commits de mesclagem (merge commits)

Dependendo do fluxo de trabalho usado no seu repositório, é possível que uma porcentagem considerável dos commits no seu histórico de log sejam apenas commits de mesclagem, que normalmente não são muito informativos. Para evitar que a exibição de commits de mesclagem sobrecarregue (cluttering up) seu histórico de log, basta adicionar a opção `--no-merges` de `log`.

Desfazendo Coisas

Em qualquer estágio, você pode querer desfazer algo. Aqui, revisaremos algumas ferramentas básicas para desfazer alterações que você fez. Tenha cuidado, pois você nem sempre pode desfazer alguns desses desfazimentos. Esta é uma das poucas áreas no Git onde você pode perder algum trabalho se fizer de forma incorreta.

Um dos desfazimentos comuns ocorre quando você faz o commit muito cedo e possivelmente se esquece de adicionar alguns arquivos, ou você bagunça sua mensagem de commit. Se você quiser refazer esse commit, faça as alterações adicionais que esqueceu, prepare-as e faça o commit novamente usando a opção `--amend`:

```
$ git commit --amend
```

Este comando pega sua área de preparação e a utiliza para o commit. Se você não fez nenhuma alteração desde o seu último commit (por exemplo, você executa este comando imediatamente após o commit anterior), então o seu snapshot ficará exatamente o mesmo e tudo o que você alterará será a sua mensagem de commit.

O mesmo editor de mensagem de commit é iniciado, mas já contém a mensagem do seu commit anterior. Você pode editar a mensagem da mesma forma de sempre, mas ela substitui (overwrites) seu commit anterior.

Como exemplo, se você fizer o commit e, em seguida, perceber que se esqueceu de preparar as alterações em um arquivo que queria adicionar a este commit, você pode fazer algo assim:

```
$ git commit -m 'Initial commit'
$ git add forgotten_file
$ git commit --amend
```

Você acaba com um único commit — o segundo commit substitui os resultados do primeiro.



É importante entender que quando você está alterando (amending) seu último commit, você não está tanto corrigindo-o, mas *substituindo-o* inteiramente por um novo e melhorado commit que empurra o antigo commit para fora do caminho e coloca o novo commit em seu lugar. Efetivamente, é como se o commit anterior nunca tivesse acontecido, e ele não aparecerá no histórico do seu repositório.

O valor óbvio de alterar (amending) commits é fazer pequenas melhorias em seu último commit, sem sobrecarregar o histórico do seu repositório com mensagens de commit na forma, "Ops, esqueci de adicionar um arquivo" ou "Droga, corrigindo erro de digitação no último commit".



Altere apenas commits que ainda são locais e não foram enviados (pushed) para algum lugar. Alterar commits enviados anteriormente e forçar o push do branch causará problemas para seus colaboradores. Para saber mais sobre o que acontece quando você faz isso e como se recuperar se estiver do lado do destinatário, leia Os Perigos do Rebase (Perils of Rebasing).

Retirando a Preparação de um Arquivo Preparado (Unstaging)

As próximas duas seções demonstram como trabalhar com a sua área de preparação e com as alterações do diretório de trabalho. A parte boa é que o comando que você usa para determinar o estado dessas duas áreas também o lembra de como desfazer alterações nelas. Por exemplo, digamos que você alterou dois arquivos e quer fazer o commit deles como duas alterações separadas, mas você acidentalmente digitou `git add *` e preparou ambos. Como você pode retirar a preparação de (unstage) um dos dois? O comando `git status` o lembra:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
    modified:   CONTRIBUTING.md
```

Logo abaixo do texto "Changes to be committed", ele diz para usar `git reset HEAD <file>…` para retirar a preparação (unstage). Então, vamos usar esse conselho para retirar a preparação do arquivo `CONTRIBUTING.md`:

```
$ git reset HEAD CONTRIBUTING.md
Unstaged changes after reset:
M   CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)
```

```
    renamed:    README.md -> README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

O comando é um pouco estranho, mas funciona. O arquivo `CONTRIBUTING.md` está modificado, mas mais uma vez não preparado (unstaged).



É verdade que `git reset` pode ser um comando perigoso, especialmente se você fornecer a flag `--hard`. No entanto, no cenário descrito acima, o arquivo no seu diretório de trabalho não é tocado, então é relativamente seguro.

Por enquanto, essa invocação mágica é tudo o que você precisa saber sobre o comando `git reset`. Entraremos em muito mais detalhes sobre o que o `reset` faz e como dominá-lo para fazer coisas realmente interessantes em Reset Desmistificado.

Desfazendo a Modificação de um Arquivo Modificado

E se você perceber que não deseja manter as alterações feitas no arquivo `CONTRIBUTING.md`? Como você pode desfazer as modificações facilmente — revertê-lo para a aparência que tinha da última vez em que fez o commit (ou clonou inicialmente, ou como quer que tenha chegado ao seu diretório de trabalho)? Felizmente, o `git status` também diz como fazer isso. Na saída do último exemplo, a área não preparada se parece com isto:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   CONTRIBUTING.md
```

Ele diz explicitamente como descartar as alterações feitas. Vamos fazer o que ele diz:

```
$ git checkout -- CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    renamed:    README.md -> README
```

Você pode ver que as alterações foram revertidas.



É importante entender que `git checkout -- <file>` é um comando perigoso. Quaisquer alterações locais feitas nesse arquivo são perdidas — o Git

> simplesmente substituiu aquele arquivo pela última versão preparada ou commitada. Nunca use esse comando a menos que você saiba com certeza absoluta que não deseja essas alterações locais não salvas.

Se você gostaria de manter as alterações feitas nesse arquivo, mas ainda precisa tirá-lo do caminho por enquanto, abordaremos as opções de stashing e branching no Ramificação (Branching) no Git; essas são geralmente maneiras melhores de prosseguir.

Lembre-se, qualquer coisa que seja *commitada* no Git quase sempre pode ser recuperada. Até mesmo commits que estavam em branches que foram excluídos ou commits que foram sobrescritos com um commit `--amend` podem ser recuperados (consulte Recuperação de Dados para recuperação de dados). No entanto, qualquer coisa que você perca e que nunca tenha sido commitada provavelmente nunca mais será vista.

Desfazendo coisas com git restore

O Git versão 2.23.0 introduziu um novo comando: `git restore`. É basicamente uma alternativa ao `git reset` que acabamos de abordar. A partir do Git versão 2.23.0, o Git usará `git restore` em vez de `git reset` para muitas operações de desfazer.

Vamos refazer nossos passos e desfazer as coisas com `git restore` em vez de `git reset`.

Retirando a Preparação de um Arquivo Preparado com git restore

As próximas duas seções demonstram como trabalhar com a sua área de preparação e as alterações do diretório de trabalho com `git restore`. A parte boa é que o comando que você usa para determinar o estado dessas duas áreas também o lembra de como desfazer alterações nelas. Por exemplo, digamos que você alterou dois arquivos e quer fazer o commit deles como duas alterações separadas, mas você acidentalmente digitou `git add *` e preparou ambos. Como você pode retirar a preparação de (unstage) um dos dois? O comando `git status` o lembra:

```
$ git add *
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   CONTRIBUTING.md
    renamed:    README.md -> README
```

Logo abaixo do texto “Changes to be committed”, ele diz para usar `git restore --staged <file>…` para retirar a preparação (unstage). Então, vamos usar esse conselho para retirar a preparação do arquivo `CONTRIBUTING.md`:

```
$ git restore --staged CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
```

```
    renamed:    README.md -> README

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
```

O arquivo `CONTRIBUTING.md` está modificado, mas mais uma vez não preparado (unstaged).

Desfazendo a Modificação de um Arquivo Modificado com git restore

E se você perceber que não deseja manter as alterações feitas no arquivo `CONTRIBUTING.md`? Como você pode desfazer as modificações facilmente — revertê-lo para a aparência que tinha da última vez em que fez o commit (ou clonou inicialmente, ou como quer que tenha chegado ao seu diretório de trabalho)? Felizmente, o `git status` também diz como fazer isso. Na saída do último exemplo, a área não preparada se parece com isto:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
```

Ele diz explicitamente como descartar as alterações feitas. Vamos fazer o que ele diz:

```
$ git restore CONTRIBUTING.md
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    README.md -> README
```



É importante entender que `git restore <file>` é um comando perigoso. Quaisquer alterações locais feitas nesse arquivo são perdidas — o Git simplesmente substituiu aquele arquivo pela última versão preparada ou commitada. Nunca use esse comando a menos que você saiba com certeza absoluta que não deseja essas alterações locais não salvas.

Trabalhando com Repositórios Remotos

Para poder colaborar em qualquer projeto Git, você precisa saber como gerenciar seus repositórios remotos. Repositórios remotos são versões do seu projeto que estão hospedadas na Internet ou em algum lugar da rede. Você pode ter vários deles, cada um geralmente sendo somente leitura ou leitura/gravação para você. Colaborar com outras pessoas envolve gerenciar esses repositórios remotos e enviar (push) e extrair (pull) dados de e para eles quando você precisa compartilhar o trabalho. O gerenciamento de repositórios remotos inclui saber como adicionar repositórios remotos, remover remotos que não são mais válidos, gerenciar várias branches remotas e defini-las

como sendo rastreadas (tracked) ou não e muito mais. Nesta seção, abordaremos algumas dessas habilidades de gerenciamento de remotos.



Repositórios remotos podem estar na sua máquina local.

É inteiramente possível que você esteja trabalhando com um repositório "remoto" que, de fato, esteja no mesmo host que você. A palavra "remoto" não implica necessariamente que o repositório esteja em outro lugar na rede ou na Internet, apenas que está em outro lugar. Trabalhar com um repositório remoto assim ainda envolveria todas as operações padrão de push, pull e fetch, como em qualquer outro remoto.

Mostrando seus Remotos

Para ver quais servidores remotos você configurou, você pode executar o comando `git remote`. Ele lista os nomes curtos (shortnames) de cada handle remoto que você especificou. Se você clonou seu repositório, você deve pelo menos ver `origin` — esse é o nome padrão que o Git dá ao servidor do qual você clonou:

```
$ git clone https://github.com/schacon/ticgit
Cloning into 'ticgit'...
remote: Reusing existing pack: 1857, done.
remote: Total 1857 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (1857/1857), 374.35 KiB | 268.00 KiB/s, done.
Resolving deltas: 100% (772/772), done.
Checking connectivity... done.
$ cd ticgit
$ git remote
origin
```

Você também pode especificar `-v`, que mostra as URLs que o Git armazenou para o nome curto (shortname) a ser usado ao ler e gravar nesse remoto:

```
$ git remote -v
origin  https://github.com/schacon/ticgit (fetch)
origin  https://github.com/schacon/ticgit (push)
```

Se você tiver mais de um remoto, o comando lista todos eles. Por exemplo, um repositório com vários remotos para trabalhar com vários colaboradores pode ser parecido com isto.

```
$ cd grit
$ git remote -v
bakkdoor  https://github.com/bakkdoor/grit (fetch)
bakkdoor  https://github.com/bakkdoor/grit (push)
cho45     https://github.com/cho45/grit (fetch)
cho45     https://github.com/cho45/grit (push)
defunkt   https://github.com/defunkt/grit (fetch)
```

```
defunkt   https://github.com/defunkt/grit (push)
koke      git://github.com/koke/grit.git (fetch)
koke      git://github.com/koke/grit.git (push)
origin    git@github.com:mojombo/grit.git (fetch)
origin    git@github.com:mojombo/grit.git (push)
```

Isso significa que podemos extrair (pull) contribuições de qualquer um desses usuários com bastante facilidade. Adicionalmente, podemos ter permissão para enviar (push) para um ou mais deles, embora não possamos dizer isso aqui.

Observe que esses remotos usam uma variedade de protocolos; cobriremos mais sobre isso em Colocando o Git em um Servidor.

Adicionando Repositórios Remotos

Mencionamos e demonstramos como o comando `git clone` adiciona implicitamente o remoto `origin` para você. Veja como adicionar um novo remoto explicitamente. Para adicionar um novo repositório Git remoto como um nome curto (shortname) que você pode referenciar facilmente, execute `git remote add <shortname> <url>`:

```
$ git remote
origin
$ git remote add pb https://github.com/paulboone/ticgit
$ git remote -v
origin  https://github.com/schacon/ticgit (fetch)
origin  https://github.com/schacon/ticgit (push)
pb  https://github.com/paulboone/ticgit (fetch)
pb  https://github.com/paulboone/ticgit (push)
```

Agora você pode usar a string `pb` na linha de comando em vez da URL inteira. Por exemplo, se você quiser buscar (fetch) todas as informações que Paul tem, mas que você ainda não tem em seu repositório, você pode executar `git fetch pb`:

```
$ git fetch pb
remote: Counting objects: 43, done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 43 (delta 10), reused 31 (delta 5)
Unpacking objects: 100% (43/43), done.
From https://github.com/paulboone/ticgit
 * [new branch]      master     -> pb/master
 * [new branch]      ticgit     -> pb/ticgit
```

O branch `master` de Paul agora está acessível localmente como `pb/master` — você pode mesclá-lo (merge) em um dos seus branches, ou você pode fazer checkout em um branch local naquele ponto se quiser inspecioná-lo. Veremos o que são branches e como usá-los com muito mais detalhes em Ramificação (Branching) no Git.

Buscando (fetching) e Extraindo (pulling) de Seus Remotos

Como você acabou de ver, para obter dados de seus projetos remotos, você pode executar:

```
$ git fetch <remote>
```

O comando vai para aquele projeto remoto e puxa (pulls down) todos os dados daquele projeto remoto que você ainda não tem. Depois de fazer isso, você deve ter referências para todos os branches daquele remoto, os quais você pode mesclar (merge) ou inspecionar a qualquer momento.

Se você clonar um repositório, o comando adicionará automaticamente esse repositório remoto com o nome "origin". Portanto, `git fetch origin` busca (fetches) qualquer trabalho novo que tenha sido enviado (pushed) para aquele servidor desde que você o clonou (ou buscou dele pela última vez). É importante observar que o comando `git fetch` apenas baixa os dados para o seu repositório local — ele não os mescla (merge) automaticamente com nenhum de seus trabalhos, nem modifica o que você está trabalhando no momento. Você deve mesclá-lo manualmente em seu trabalho quando estiver pronto.

Se o seu branch atual estiver configurado para rastrear um branch remoto (consulte a próxima seção e Ramificação (Branching) no Git para obter mais informações), você poderá usar o comando `git pull` para buscar (fetch) e, em seguida, mesclar (merge) automaticamente aquele branch remoto em seu branch atual. Esse pode ser um fluxo de trabalho mais fácil ou mais confortável para você; e, por padrão, o comando `git clone` configura automaticamente seu branch `master` local para rastrear o branch `master` remoto (ou qualquer que seja o nome do branch padrão) no servidor de onde você clonou. A execução do `git pull` geralmente busca (fetches) dados do servidor do qual você clonou originalmente e tenta mesclá-los (merge) automaticamente no código em que você está trabalhando no momento.



A partir do Git versão 2.27, o `git pull` emitirá um aviso se a variável `pull.rebase` não estiver definida. O Git continuará avisando você até que você defina a variável.

Se você quiser o comportamento padrão do Git (fast-forward se possível, caso contrário crie um commit de mesclagem (merge commit)): `git config --global pull.rebase "false"`

Se você quiser fazer rebase ao extrair (pulling): `git config --global pull.rebase "true"`

Enviando (pushing) para seus Remotos

Quando você tiver seu projeto em um ponto que deseja compartilhar, você deve enviá-lo (push) para upstream. O comando para isso é simples: `git push <remote> <branch>`. Se você quiser enviar (push) seu branch `master` para o seu servidor `origin` (novamente, a clonagem geralmente configura ambos os nomes para você automaticamente), você pode executar isso para enviar todos os commits que você fez de volta para o servidor:

```
$ git push origin master
```

Este comando funciona apenas se você clonou de um servidor no qual você tem acesso de gravação e se ninguém fez o push nesse meio tempo. Se você e outra pessoa clonarem ao mesmo tempo, e ela enviar (push) para upstream e depois você enviar (push) para upstream, o seu push será corretamente rejeitado. Você terá que buscar (fetch) o trabalho deles primeiro e incorporá-lo ao seu antes de ter permissão para fazer o push. Consulte Ramificação (Branching) no Git para obter informações mais detalhadas sobre como fazer push para servidores remotos.

Inspecionando um Remoto

Se você quiser ver mais informações sobre um remoto específico, você pode usar o comando `git remote show <remote>`. Se você executar este comando com um nome curto (shortname) específico, como `origin`, você obterá algo assim:

```
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/schacon/ticgit
  Push  URL: https://github.com/schacon/ticgit
  HEAD branch: master
  Remote branches:
    master                               tracked
    dev-branch                           tracked
  Local branch configured for 'git pull':
    master merges with remote master
  Local ref configured for 'git push':
    master pushes to master (up to date)
```

Ele lista a URL para o repositório remoto, bem como as informações do branch de rastreamento (tracking branch). O comando informa utilmente que se você estiver no branch `master` e executar `git pull`, ele mesclará (merge) automaticamente o branch `master` do remoto no local depois de ser buscado (fetched). Ele também lista todas as referências remotas que puxou (pulled down).

Esse é um exemplo simples que você provavelmente encontrará. Quando você estiver usando o Git de forma mais intensa, no entanto, poderá ver muito mais informações do `git remote show`:

```
$ git remote show origin
* remote origin
  URL: https://github.com/my-org/complex-project
  Fetch URL: https://github.com/my-org/complex-project
  Push  URL: https://github.com/my-org/complex-project
  HEAD branch: master
  Remote branches:
    master                           tracked
    dev-branch                       tracked
    markdown-strip                   tracked
    issue-43                         new (next fetch will store in remotes/origin)
```

```
    issue-45                     new (next fetch will store in remotes/origin)
    refs/remotes/origin/issue-11 stale (use 'git remote prune' to remove)
  Local branches configured for 'git pull':
    dev-branch merges with remote dev-branch
    master     merges with remote master
  Local refs configured for 'git push':
    dev-branch                   pushes to dev-branch                   (up to
date)
    markdown-strip               pushes to markdown-strip               (up to
date)
    master                       pushes to master                       (up to
date)
```

Este comando mostra para qual branch o envio é feito automaticamente quando você executa `git push` enquanto está em certos branches. Ele também mostra quais branches remotos no servidor você ainda não tem, quais branches remotos você tem que foram removidos do servidor e vários branches locais que podem ser mesclados automaticamente com seus branches de rastreamento remotos (remote-tracking branch) quando você executa `git pull`.

Renomeando e Removendo Remotos

Você pode executar `git remote rename` para alterar o nome curto (shortname) de um remoto. Por exemplo, se você quiser renomear `pb` para `paul`, você pode fazê-lo com `git remote rename`:

```
$ git remote rename pb paul
$ git remote
origin
paul
```

Vale a pena mencionar que isso também altera todos os nomes dos seus branches de rastreamento remoto (remote-tracking branch). O que costumava ser referenciado em `pb/master` agora está em `paul/master`.

Se você quiser remover um remoto por algum motivo — você mudou de servidor ou não está mais usando um espelho (mirror) específico, ou talvez um colaborador não esteja mais contribuindo — você pode usar `git remote remove` ou `git remote rm`:

```
$ git remote remove paul
$ git remote
origin
```

Depois de excluir a referência a um remoto dessa maneira, todos os branches de rastreamento remoto (remote-tracking branches) e definições de configuração associados a esse remoto também serão excluídos.

Criando Tags

Como a maioria dos VCSs, o Git tem a capacidade de marcar (tag) pontos específicos no histórico de um repositório como sendo importantes. Normalmente, as pessoas usam essa funcionalidade para marcar pontos de lançamento (`v1.0`, `v2.0` e assim por diante). Nesta seção, você aprenderá como listar as tags existentes, como criar e excluir tags, e quais são os diferentes tipos de tags.

Listando Suas Tags

Listar as tags existentes no Git é simples. Basta digitar `git tag` (com o opcional `-l` ou `--list`):

```
$ git tag
v1.0
v2.0
```

Este comando lista as tags em ordem alfabética; a ordem na qual elas são exibidas não tem importância real.

Você também pode pesquisar tags que correspondam a um padrão específico. O repositório de código-fonte do Git, por exemplo, contém mais de 500 tags. Se você estiver interessado apenas em observar a série 1.8.5, você pode executar isto:

```
$ git tag -l "v1.8.5*"
v1.8.5
v1.8.5-rc0
v1.8.5-rc1
v1.8.5-rc2
v1.8.5-rc3
v1.8.5.1
v1.8.5.2
v1.8.5.3
v1.8.5.4
v1.8.5.5
```



A listagem de curingas (wildcards) de tags requer a opção `-l` ou `--list`

Se você quiser apenas a lista inteira de tags, executar o comando `git tag` assume implicitamente que você deseja uma lista e fornece uma; o uso de `-l` ou `--list` neste caso é opcional.

Se, no entanto, você estiver fornecendo um padrão curinga (wildcard) para corresponder a nomes de tags, o uso de `-l` ou `--list` é obrigatório.

Criando Tags

O Git suporta dois tipos de tags: *leves (lightweight)* e *anotadas (annotated)*.

Uma tag leve (lightweight tag) é muito parecida com um branch que não muda — é apenas um

ponteiro para um commit específico.

As tags anotadas (annotated tags), no entanto, são armazenadas como objetos completos no banco de dados do Git. Elas têm checksum; contêm o nome do criador da tag (tagger), e-mail e data; possuem uma mensagem de tag; e podem ser assinadas e verificadas com o GNU Privacy Guard (GPG). Geralmente é recomendado que você crie tags anotadas para que possa ter todas essas informações; mas se você quiser uma tag temporária ou, por algum motivo, não quiser manter as outras informações, as tags leves também estão disponíveis.

Tags Anotadas (Annotated Tags)

Criar uma tag anotada no Git é simples. A maneira mais fácil é especificar `-a` quando você executar o comando `tag`:

```
$ git tag -a v1.4 -m "my version 1.4"
$ git tag
v0.1
v1.3
v1.4
```

O `-m` especifica uma mensagem de tag, que é armazenada junto com ela. Se você não especificar uma mensagem para uma tag anotada, o Git iniciará o seu editor para que você possa digitá-la.

Você pode ver os dados da tag junto com o commit que foi tagueado usando o comando `git show`:

```
$ git show v1.4
tag v1.4
Tagger: Ben Straub <ben@straub.cc>
Date:   Sat May 3 20:19:12 2014 -0700

my version 1.4

commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number
```

Isso mostra as informações de quem criou a tag (tagger), a data em que o commit foi tagueado e a mensagem de anotação antes de mostrar as informações do commit.

Tags Leves (Lightweight Tags)

Outra forma de taguear commits é com uma tag leve (lightweight tag). Isso é basicamente o checksum do commit armazenado em um arquivo — nenhuma outra informação é mantida. Para criar uma tag leve, não forneça nenhuma das opções `-a`, `-s` ou `-m`, forneça apenas o nome da tag:

```
$ git tag v1.4-lw
$ git tag
v0.1
v1.3
v1.4
v1.4-lw
v1.5
```

Desta vez, se você executar `git show` na tag, você não verá as informações extras da tag. O comando apenas mostra o commit:

```
$ git show v1.4-lw
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    Change version number
```

Tagueando Depois

Você também pode taguear commits depois de ter passado por eles. Suponha que o seu histórico de commits seja parecido com este:

```
$ git log --pretty=oneline
15027957951b64cf874c3557a0f3547bd83b3ff6 Merge branch 'experiment'
a6b4c97498bd301d84096da251c98a07c7723e65 Create write support
0d52aaab4479697da7686c15f77a3d64d9165190 One more thing
6d52a271eda8725415634dd79daabbc4d9b6008e Merge branch 'experiment'
0b7434d86859cc7b8c3d5e1dddfed66ff742fcbc Add commit function
4682c3261057305bdd616e23b64b0857d832627b Add todo file
166ae0c4d3f420721acbb115cc33848dfcc2121a Create write support
9fceb02d0ae598e95dc970b74767f19372d61af8 Update rakefile
964f16d36dfccde844893cac5b347e7b3d44abbc Commit the todo
8a5cbc430f1a9c3d00faaeffd07798508422908a Update readme
```

Agora, suponha que você esqueceu de taguear o projeto na v1.2, que estava no commit “Update rakefile”. Você pode adicioná-la depois do fato. Para taguear aquele commit, você especifica o checksum do commit (ou parte dele) no final do comando:

```
$ git tag -a v1.2 9fceb02
```

Você pode ver que tagueou o commit:

```
$ git tag
v0.1
```

```
v1.2
v1.3
v1.4
v1.4-lw
v1.5

$ git show v1.2
tag v1.2
Tagger: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Feb 9 15:32:16 2009 -0800

version 1.2
commit 9fceb02d0ae598e95dc970b74767f19372d61af8
Author: Magnus Chacon <mchacon@gee-mail.com>
Date:   Sun Apr 27 20:43:35 2008 -0700

    Update rakefile
...
```

Compartilhando Tags

Por padrão, o comando `git push` não transfere tags para servidores remotos. Você terá que enviar (push) as tags explicitamente para um servidor compartilhado depois de tê-las criado. Esse processo é igual a compartilhar branches remotos — você pode executar `git push origin <tagname>`.

```
$ git push origin v1.5
Counting objects: 14, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.05 KiB | 0 bytes/s, done.
Total 14 (delta 3), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]         v1.5 -> v1.5
```

Se você tiver muitas tags que deseja enviar (push) de uma só vez, também poderá usar a opção `--tags` do comando `git push`. Isso transferirá todas as suas tags para o servidor remoto que ainda não estão lá.

```
$ git push origin --tags
Counting objects: 1, done.
Writing objects: 100% (1/1), 160 bytes | 0 bytes/s, done.
Total 1 (delta 0), reused 0 (delta 0)
To git@github.com:schacon/simplegit.git
 * [new tag]         v1.4 -> v1.4
 * [new tag]         v1.4-lw -> v1.4-lw
```

Agora, quando outra pessoa clonar ou extrair (pull) do seu repositório, ela também obterá todas as

suas tags.



`git push` *envia ambos os tipos de tags*

`git push <remote> --tags` enviará tags leves e anotadas. No momento, não há opção para enviar apenas tags leves, mas se você usar `git push <remote> --follow-tags` apenas as tags anotadas serão enviadas para o remoto.

Excluindo Tags

Para excluir uma tag no seu repositório local, você pode usar `git tag -d <tagname>`. Por exemplo, poderíamos remover a nossa tag leve acima da seguinte maneira:

```
$ git tag -d v1.4-lw
Deleted tag 'v1.4-lw' (was e7d5add)
```

Observe que isso não remove a tag de nenhum servidor remoto. Existem duas variações comuns para excluir uma tag de um servidor remoto.

A primeira variação é `git push <remote> :refs/tags/<tagname>`:

```
$ git push origin :refs/tags/v1.4-lw
To /git@github.com:schacon/simplegit.git
 - [deleted]         v1.4-lw
```

A maneira de interpretar o comando acima é lê-lo como o valor nulo antes dos dois pontos (colon) sendo enviado para o nome da tag remota, efetivamente excluindo-a.

A segunda (e mais intuitiva) maneira de excluir uma tag remota é com:

```
$ git push origin --delete <tagname>
```

Fazendo Checkout de Tags

Se você quiser ver as versões dos arquivos para as quais uma tag está apontando, você pode fazer um `git checkout` daquela tag, embora isso coloque seu repositório no estado “detached HEAD”, o que tem alguns efeitos colaterais ruins:

```
$ git checkout v2.0.0
Note: switching to 'v2.0.0'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by performing another checkout.

If you want to create a new branch to retain commits you create, you may
```

```
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 99ada87... Merge pull request #89 from schacon/appendix-final

$ git checkout v2.0-beta-0.1
Previous HEAD position was 99ada87... Merge pull request #89 from schacon/appendix-
final
HEAD is now at df3f601... Add atlas.json and cover image
```

No estado “detached HEAD”, se você fizer alterações e então criar um commit, a tag permanecerá a mesma, mas seu novo commit não pertencerá a nenhum branch e ficará inacessível, exceto pelo hash exato do commit. Portanto, se você precisar fazer alterações — digamos que esteja corrigindo um bug em uma versão mais antiga, por exemplo — geralmente você desejará criar um branch:

```
$ git checkout -b version2 v2.0.0
Switched to a new branch 'version2'
```

Se você fizer isso e criar um commit, seu branch `version2` será ligeiramente diferente da sua tag `v2.0.0`, pois ele avançará com as suas novas alterações, portanto tenha cuidado.

Aliases (Apelidos) no Git

Antes de passarmos para o próximo capítulo, queremos introduzir um recurso que pode tornar a sua experiência com o Git mais simples, fácil e familiar: os aliases (apelidos). Para fins de clareza, não os usaremos em nenhum outro lugar neste livro, mas se você continuar a usar o Git com alguma regularidade, aliases são algo que você deve conhecer.

O Git não infere automaticamente o seu comando se você digitá-lo parcialmente. Se você não quiser digitar o texto inteiro de cada um dos comandos do Git, poderá configurar facilmente um alias para cada comando usando `git config`. Aqui estão alguns exemplos que você pode querer configurar:

```
$ git config --global alias.co checkout
$ git config --global alias.br branch
$ git config --global alias.ci commit
$ git config --global alias.st status
```

Isso significa que, por exemplo, em vez de digitar `git commit`, você só precisa digitar `git ci`. À medida que continuar usando o Git, você provavelmente usará outros comandos com frequência também; não hesite em criar novos aliases.

Esta técnica também pode ser muito útil para criar comandos que você acha que deveriam existir. Por exemplo, para corrigir o problema de usabilidade que você encontrou ao retirar a preparação (unstage) de um arquivo, você pode adicionar seu próprio alias de unstage ao Git:

```
$ git config --global alias.unstage 'reset HEAD --'
```

Isso torna os dois comandos a seguir equivalentes:

```
$ git unstage fileA
$ git reset HEAD -- fileA
```

Isso parece um pouco mais claro. Também é comum adicionar um comando `last`, assim:

```
$ git config --global alias.last 'log -1 HEAD'
```

Desta forma, você pode ver o último commit facilmente:

```
$ git last
commit 66938dae3329c7aebe598c2246a8e6af90d04646
Author: Josh Goebel <dreamer3@example.com>
Date:   Tue Aug 26 19:48:51 2008 +0800

    Test for current head

    Signed-off-by: Scott Chacon <schacon@example.com>
```

Como você pode ver, o Git simplesmente substitui o novo comando pelo que você criou como alias. No entanto, talvez você queira executar um comando externo, em vez de um subcomando do Git. Nesse caso, você inicia o comando com um caractere `!`. Isso é útil se você escrever suas próprias ferramentas que funcionam com um repositório Git. Podemos demonstrar isso criando um alias para `git visual` para executar o `gitk`:

```
$ git config --global alias.visual '!gitk'
```

Resumo

Neste ponto, você pode fazer todas as operações locais básicas do Git — criar ou clonar um repositório, fazer alterações, preparar (stage) e fazer commit dessas alterações, e visualizar o histórico de todas as alterações pelas quais o repositório passou. Em seguida, abordaremos o recurso matador do Git: seu modelo de branching.

Ramificação (Branching) no Git

Quase todos os VCSs têm alguma forma de suporte à ramificação (branching). Branching (ramificação) significa que você diverge da linha principal de desenvolvimento e continua a fazer o trabalho sem mexer nessa linha principal. Em muitas ferramentas de VCS, este é um processo um tanto custoso, muitas vezes exigindo que você crie uma nova cópia do diretório de código-fonte, o que pode levar muito tempo em projetos grandes.

Algumas pessoas se referem ao modelo de ramificação do Git como seu "recurso matador (killer feature)", e ele certamente destaca o Git na comunidade de VCS. Por que é tão especial? A maneira como o Git ramifica é incrivelmente leve, tornando as operações de ramificação quase instantâneas, e alternar entre as ramificações (branches) geralmente é igualmente rápido. Ao contrário de muitos outros VCSs, o Git incentiva fluxos de trabalho que ramificam (branch) e mesclam (merge) frequentemente, até várias vezes ao dia. Compreender e dominar esse recurso fornece a você uma ferramenta poderosa e única, e pode mudar completamente a maneira como você desenvolve.

Branches em Poucas Palavras

Para realmente entender a maneira como o Git faz ramificação (branching), precisamos dar um passo atrás e examinar como o Git armazena seus dados.

Como você deve se lembrar do O que é Git?, o Git não armazena dados como uma série de conjuntos de alterações (changesets) ou diferenças, mas sim como uma série de *snapshots* (instantâneos).

Quando você faz um commit, o Git armazena um objeto de commit que contém um ponteiro para o snapshot do conteúdo que você preparou (staged). Este objeto também contém o nome e o endereço de e-mail do autor, a mensagem que você digitou e ponteiros para o commit ou commits que vieram diretamente antes deste commit (seu pai ou pais): zero pais para o commit inicial, um pai para um commit normal e vários pais para um commit que resulta de uma mesclagem (merge) de dois ou mais branches.

Para visualizar isso, vamos assumir que você tem um diretório contendo três arquivos, e você prepara (stage) todos eles e faz o commit. Preparar (staging) os arquivos calcula um checksum para cada um (o hash SHA-1 que mencionamos em O que é Git?), armazena essa versão do arquivo no repositório Git (o Git se refere a eles como *blobs*) e adiciona esse checksum à área de preparação (staging area):

```
$ git add README test.rb LICENSE
$ git commit -m 'Initial commit'
```

Quando você cria o commit executando `git commit`, o Git calcula o checksum de cada subdiretório (neste caso, apenas o diretório raiz do projeto) e os armazena como um objeto de árvore (tree) no repositório Git. O Git então cria um objeto de commit que tem os metadados e um ponteiro para a árvore do projeto raiz para que ele possa recriar aquele snapshot quando necessário.

Seu repositório Git agora contém cinco objetos: três *blobs* (cada um representando o conteúdo de um dos três arquivos), uma *árvore (tree)* que lista o conteúdo do diretório e especifica quais nomes de arquivo são armazenados como quais blobs, e um *commit* com o ponteiro para essa árvore raiz e todos os metadados do commit.

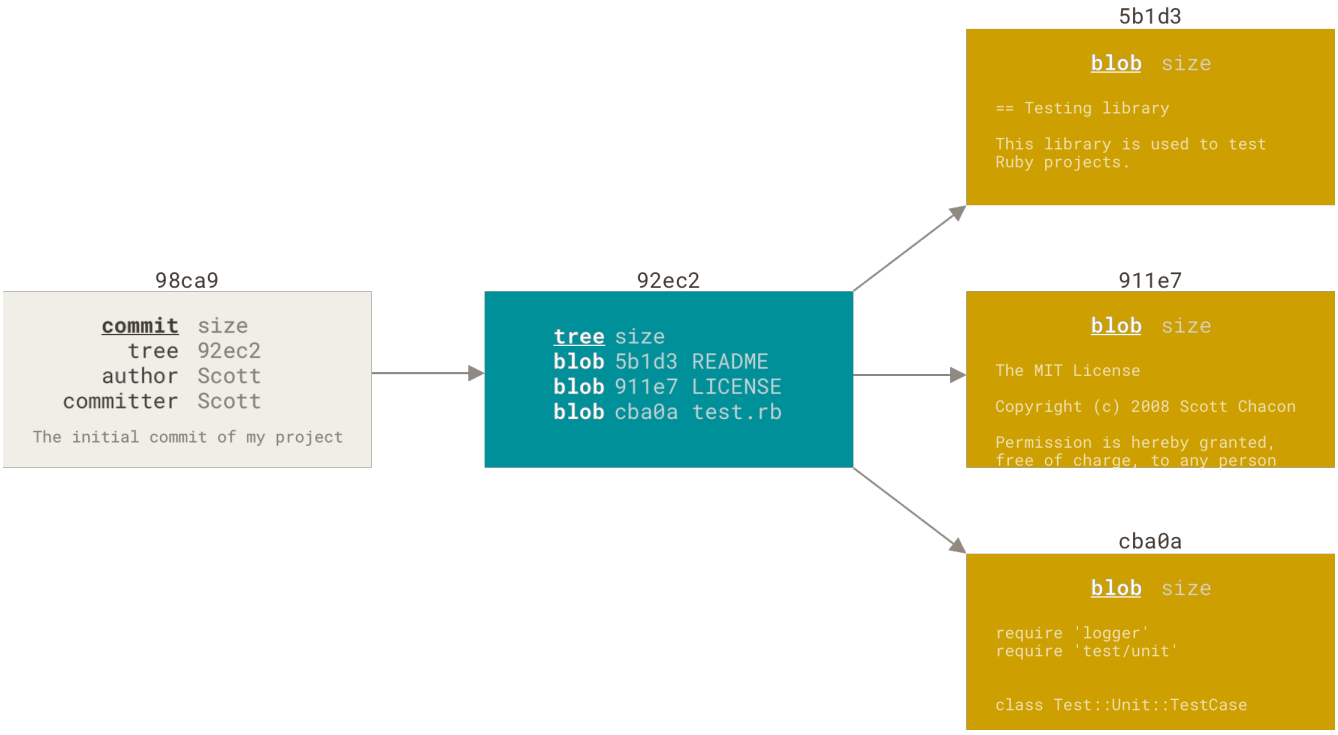


Figure 9. Um commit e sua árvore

Se você fizer algumas alterações e fizer commit novamente, o próximo commit armazenará um ponteiro para o commit que veio imediatamente antes dele.

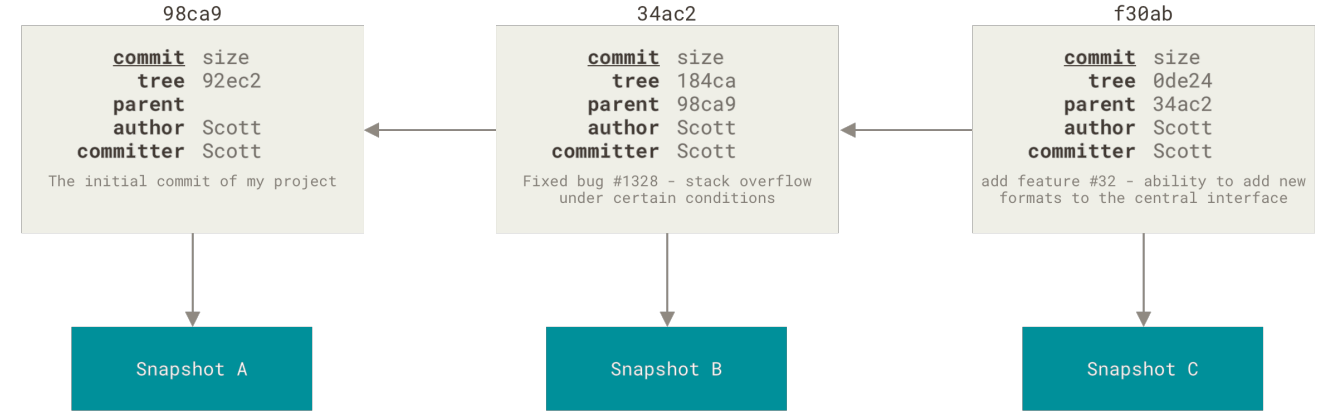


Figure 10. Commits e seus pais

Um branch no Git é simplesmente um ponteiro móvel e leve para um desses commits. O nome do branch padrão no Git é `master`. À medida que você começa a fazer commits, você recebe um branch `master` que aponta para o último commit que você fez. Toda vez que você faz um commit, o ponteiro do branch `master` avança automaticamente.



O branch “master” no Git não é um branch especial. É exatamente como qualquer outro branch. A única razão pela qual quase todo repositório tem um é que o comando `git init` o cria por padrão e a maioria das pessoas não se dá ao trabalho

de alterá-lo.

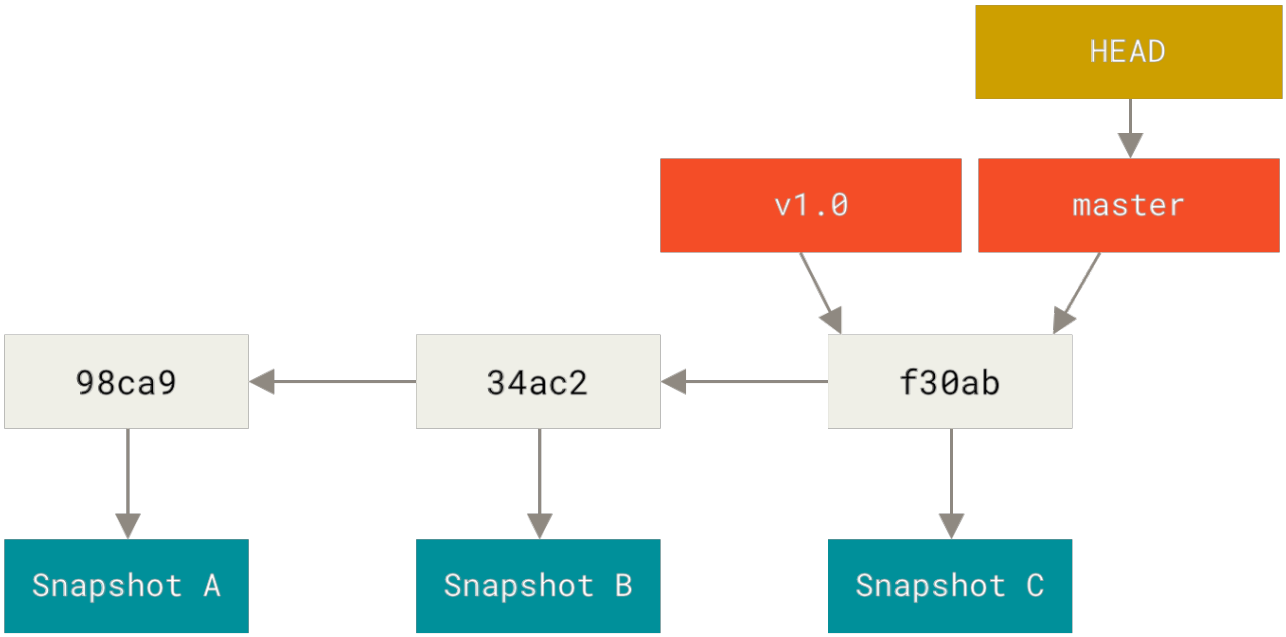


Figure 11. Um branch e seu histórico de commits

Criando um Novo Branch

O que acontece quando você cria um novo branch? Bem, fazer isso cria um novo ponteiro para você mover por aí. Digamos que você queira criar um novo branch chamado `testing`. Você faz isso com o comando `git branch`:

```
$ git branch testing
```

Isso cria um novo ponteiro para o mesmo commit em que você está no momento.

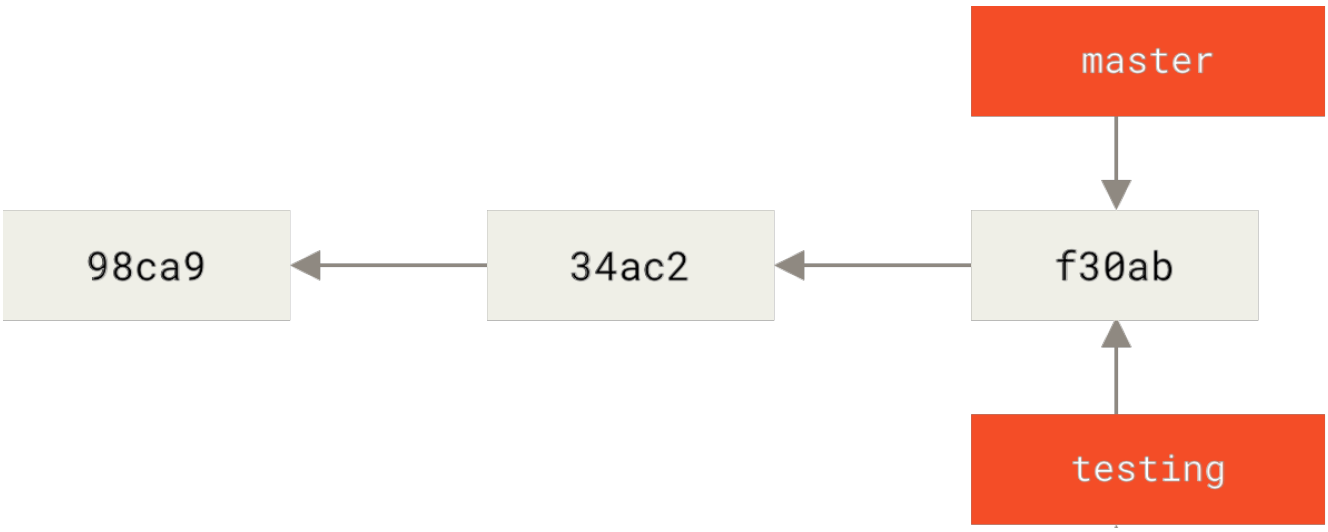


Figure 12. Dois branches apontando para a mesma série de commits

Como o Git sabe em qual branch você está no momento? Ele mantém um ponteiro especial chamado `HEAD`. Observe que isso é muito diferente do conceito de `HEAD` em outros VCSs aos quais

você pode estar acostumado, como Subversion ou CVS. No Git, este é um ponteiro para o branch local em que você está no momento. Neste caso, você ainda está no `master`. O comando `git branch` apenas *criou* um novo branch — ele não mudou para aquele branch.

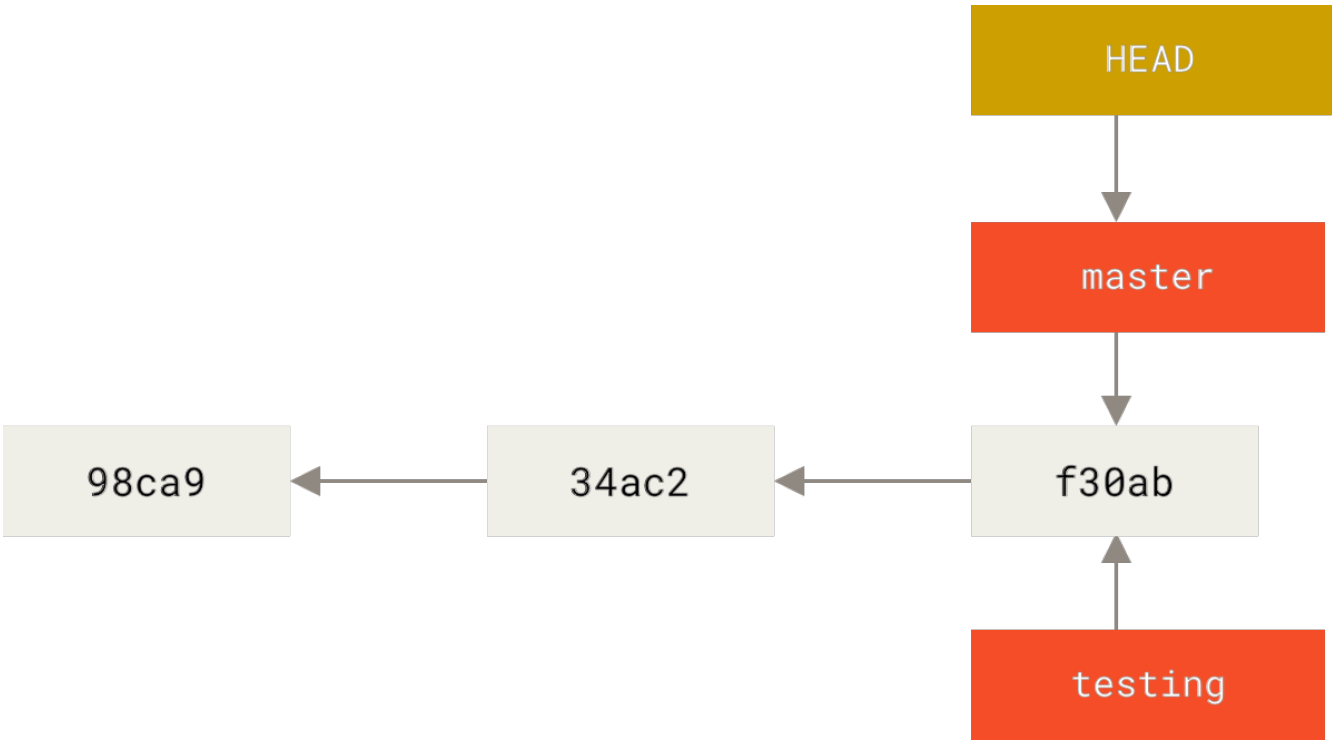


Figure 13. HEAD apontando para um branch

Você pode ver isso facilmente executando um comando simples `git log` que mostra para onde os ponteiros de branch estão apontando. Esta opção é chamada `--decorate`.

```
$ git log --oneline --decorate
f30ab (HEAD -> master, testing) Add feature #32 - ability to add new formats to the
central interface
34ac2 Fix bug #1328 - stack overflow under certain conditions
98ca9 Initial commit
```

Você pode ver os branches `master` e `testing` que estão bem ali ao lado do commit `f30ab`.

Mudando de Branches

Para mudar para um branch existente, você executa o comando `git checkout`. Vamos mudar para o novo branch `testing`:

```
$ git checkout testing
```

Isso move o `HEAD` para apontar para o branch `testing`.

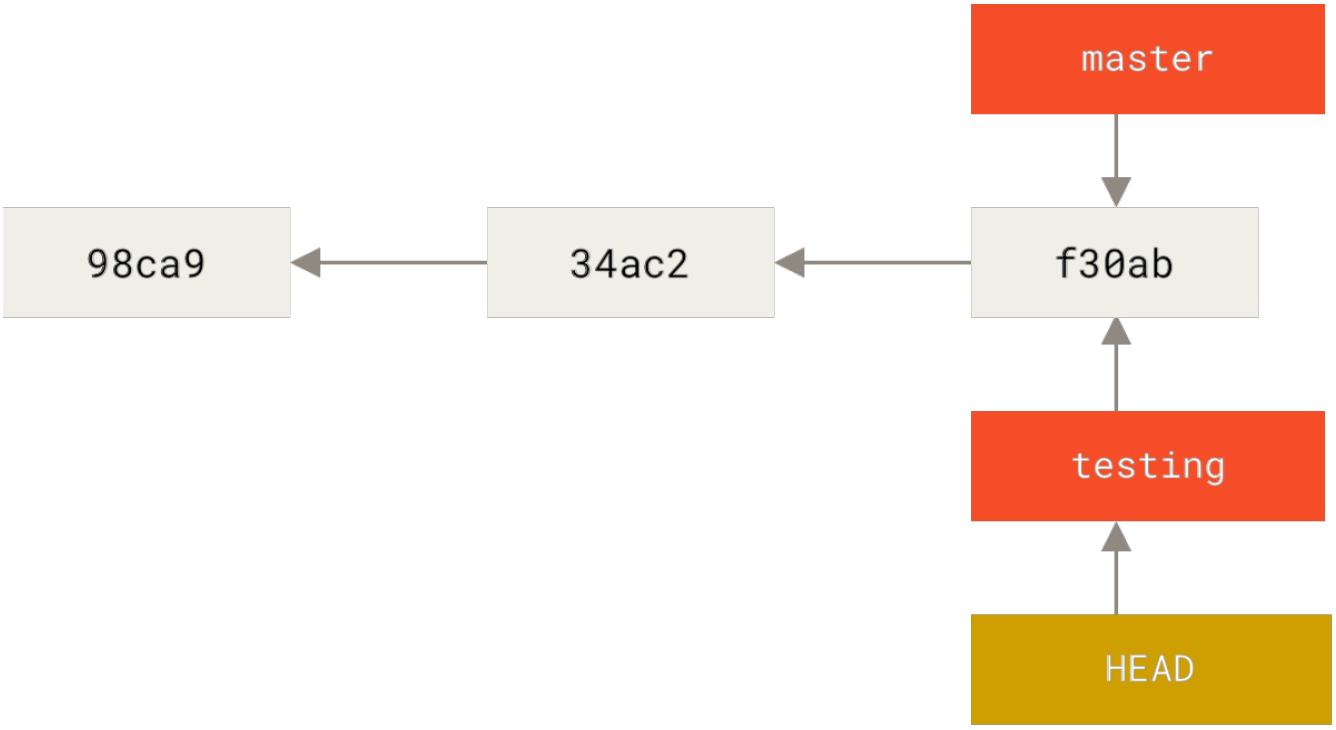


Figure 14. HEAD aponta para o branch atual

Qual o significado disso? Bem, vamos fazer outro commit:

```
$ vim test.rb
$ git commit -a -m 'Make a change'
```

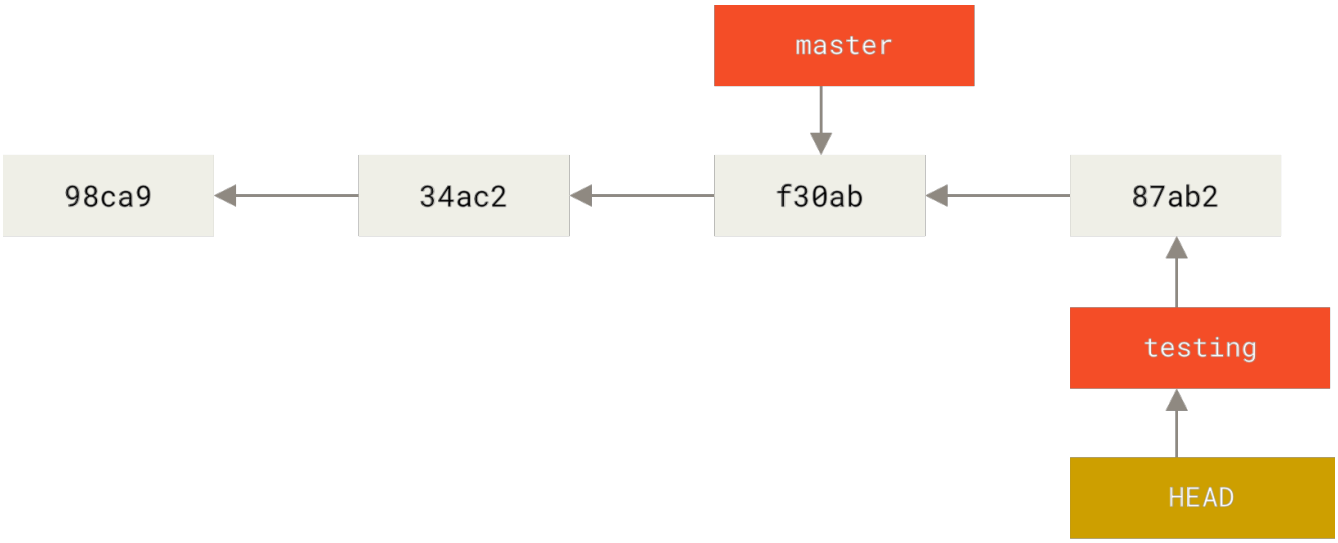


Figure 15. O branch HEAD avança quando um commit é feito

Isso é interessante, porque agora o seu branch `testing` avançou, mas o seu branch `master` ainda aponta para o commit em que você estava quando executou `git checkout` para mudar de branch. Vamos voltar para o branch `master`:

```
$ git checkout master
```



`git log` *não mostra todos os branches o tempo todo*

Se você executasse `git log` agora mesmo, poderia se perguntar para onde foi o branch "testing" que você acabou de criar, pois ele não apareceria na saída.

O branch não desapareceu; o Git apenas não sabe que você está interessado naquele branch e está tentando mostrar o que ele acha que você está interessado. Em outras palavras, por padrão, o `git log` mostrará apenas o histórico de commits abaixo do branch que você fez o checkout.

Para mostrar o histórico de commits do branch desejado, você deve especificá-lo explicitamente: `git log testing`. Para mostrar todos os branches, adicione `--all` ao seu comando `git log`.

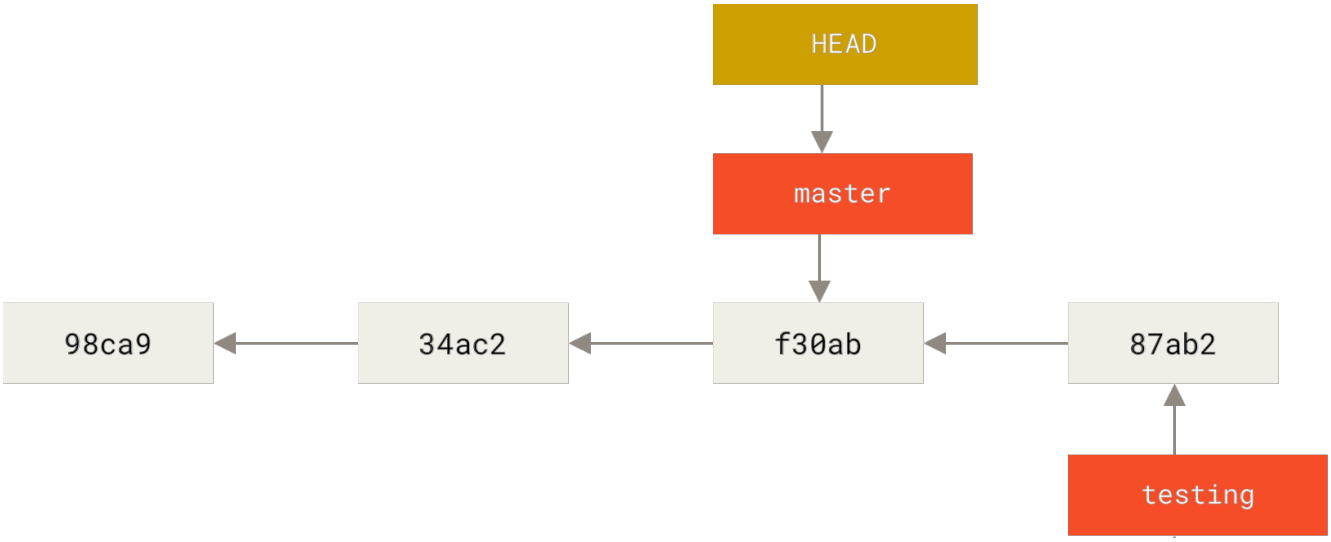


Figure 16. HEAD se move quando você faz checkout

Aquele comando fez duas coisas. Ele moveu o ponteiro HEAD de volta para apontar para o branch `master`, e reverteu os arquivos no seu diretório de trabalho de volta para o snapshot para o qual o `master` aponta. Isso também significa que as alterações feitas a partir deste ponto divergirão de uma versão anterior do projeto. Ele essencialmente retrocede (rewinds) o trabalho que você fez no seu branch `testing` para que você possa seguir em uma direção diferente.



Mudar de branch altera os arquivos no seu diretório de trabalho

É importante observar que quando você muda de branch no Git, os arquivos no seu diretório de trabalho serão alterados. Se você mudar para um branch mais antigo, seu diretório de trabalho será revertido para ficar como na última vez em que você fez commit naquele branch. Se o Git não conseguir fazer isso de forma limpa (cleanly), ele não permitirá a mudança.

Vamos fazer algumas alterações e commitar novamente:

```
$ vim test.rb
$ git commit -a -m 'Make other changes'
```

Agora o histórico do seu projeto divergiu (veja Histórico divergente). Você criou e mudou para um branch, fez algum trabalho nele, e então mudou de volta para o seu branch principal e fez outro

trabalho. Ambas as alterações estão isoladas em branches separados: você pode alternar entre os branches e mesclá-los quando estiver pronto. E você fez tudo isso com simples comandos `branch`, `checkout` e `commit`.

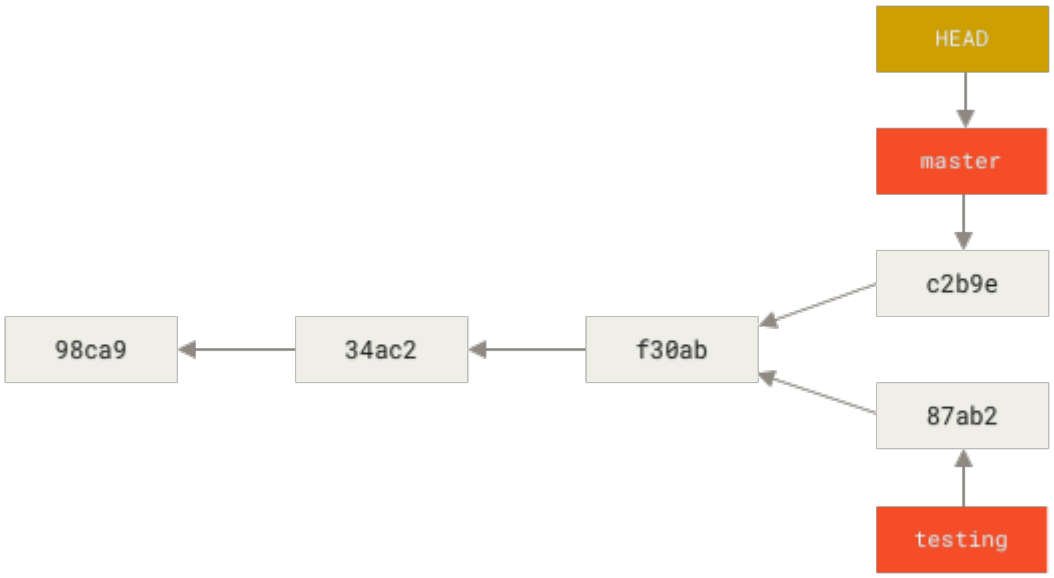


Figure 17. Histórico divergente

Você também pode ver isso facilmente com o comando `git log`. Se você executar `git log --oneline --decorate --graph --all` ele imprimirá o histórico de seus commits, mostrando onde estão os ponteiros do seu branch e como o seu histórico divergiu.

```
$ git log --oneline --decorate --graph --all
* c2b9e (HEAD, master) Make other changes
| * 87ab2 (testing) Make a change
|/
* f30ab Add feature #32 - ability to add new formats to the central interface
* 34ac2 Fix bug #1328 - stack overflow under certain conditions
* 98ca9 Initial commit of my project
```

Como um branch no Git é na verdade um arquivo simples que contém o checksum SHA-1 de 40 caracteres do commit para o qual ele aponta, os branches são baratos de criar e destruir. Criar um novo branch é tão rápido e simples quanto gravar 41 bytes em um arquivo (40 caracteres e uma nova linha).

Isso está em nítido contraste com a maneira como a maioria das ferramentas de VCS mais antigas ramificam (branch), o que envolve copiar todos os arquivos do projeto para um segundo diretório. Isso pode levar alguns segundos ou até minutos, dependendo do tamanho do projeto, enquanto que no Git o processo é sempre instantâneo. Além disso, como estamos registrando os pais quando fazemos o commit, encontrar uma base de mesclagem (merge base) adequada para a mesclagem é feito automaticamente para nós e geralmente é muito fácil de fazer. Esses recursos ajudam a

incentivar os desenvolvedores a criar e usar branches com frequência.

Vamos ver por que você deve fazer isso.



Criando um novo branch e mudando para ele ao mesmo tempo

É comum criar um novo branch e querer mudar para ele ao mesmo tempo — isso pode ser feito em uma única operação com `git checkout -b <newbranchname>`.



A partir do Git versão 2.23 em diante, você pode usar `git switch` em vez de `git checkout` para:

- Mudar para um branch existente: `git switch testing-branch`.
- Criar um novo branch e mudar para ele: `git switch -c new-branch`. A flag `-c` significa create (criar), você também pode usar a flag completa: `--create`.
- Retornar ao branch do qual você fez checkout anteriormente: `git switch -`.

Ramificação (Branching) e Mesclagem (Merging) Básicas

Vamos passar por um exemplo simples de ramificação (branching) e mesclagem (merging) com um fluxo de trabalho que você pode usar no mundo real. Você seguirá as seguintes etapas:

1. Fazer algum trabalho em um site.
2. Criar um branch para uma nova história de usuário (user story) na qual você está trabalhando.
3. Fazer algum trabalho naquele branch.

Nesta fase, você receberá uma chamada informando que outro problema é crítico e você precisa de um hotfix. Você fará o seguinte:

1. Mudar para o seu branch de produção.
2. Criar um branch para adicionar o hotfix.
3. Depois que for testado, mesclar (merge) o branch de hotfix e enviar (push) para produção.
4. Mudar de volta para a sua história de usuário original e continuar trabalhando.

Ramificação Básica

Primeiro, digamos que você esteja trabalhando no seu projeto e já tenha alguns commits no branch `master`.

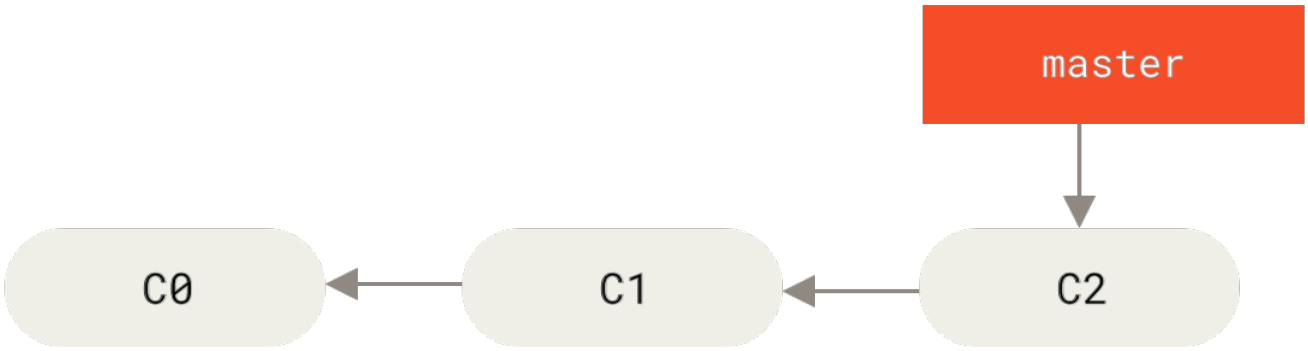


Figure 18. Um histórico de commits simples

Você decidiu que vai trabalhar na issue #53 em qualquer sistema de rastreamento de problemas (issue-tracking) que a sua empresa utilize. Para criar um novo branch e mudar para ele ao mesmo tempo, você pode executar o comando `git checkout` com a chave (switch) `-b`:

```
$ git checkout -b iss53
Switched to a new branch "iss53"
```

Isso é um atalho para:

```
$ git branch iss53
$ git checkout iss53
```

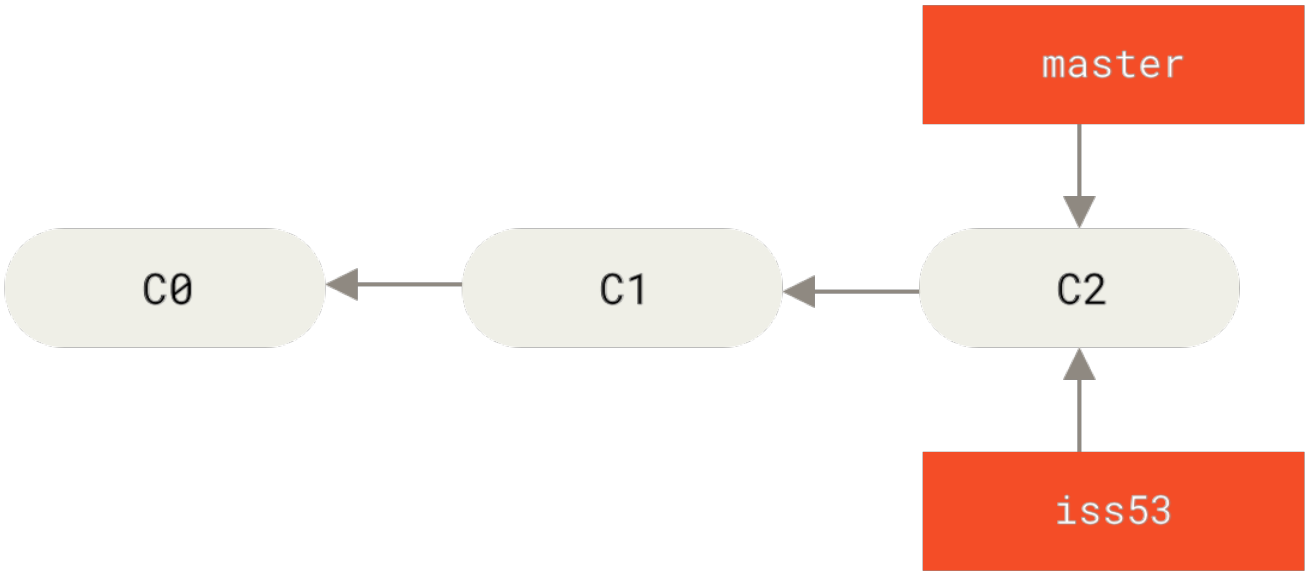


Figure 19. Criando um novo ponteiro de branch

Você trabalha no seu site e faz alguns commits. Fazer isso move o branch `iss53` para frente, porque você fez checkout dele (ou seja, seu `HEAD` está apontando para ele):

```
$ vim index.html
$ git commit -a -m 'Create new footer [issue 53]'
```

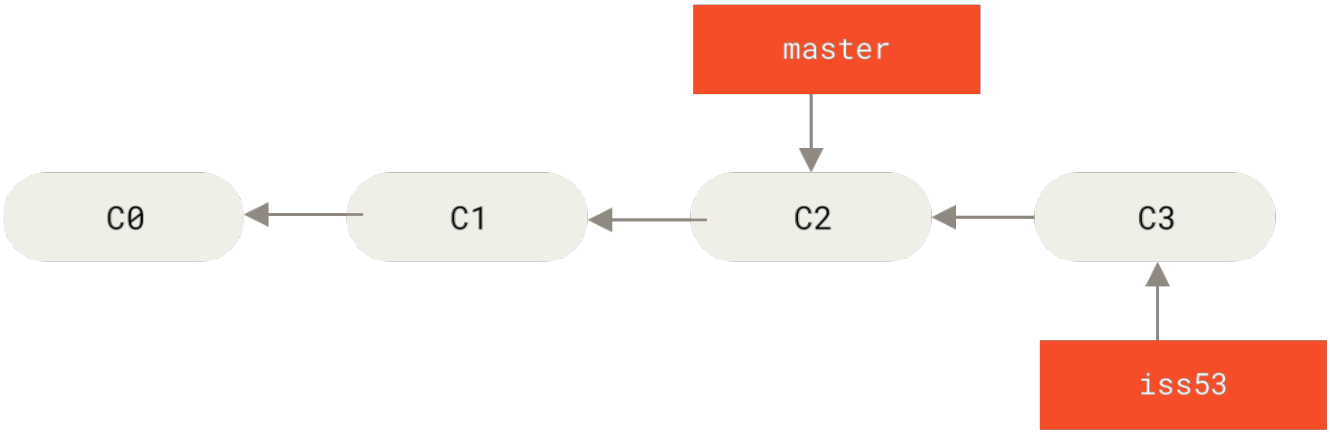


Figure 20. O branch `iss53` avançou com o seu trabalho

Agora você recebe a chamada de que há um problema com o site e você precisa corrigi-lo imediatamente. Com o Git, você não precisa implantar (deploy) a sua correção junto com as alterações da `iss53` que você fez, e não precisa fazer muito esforço para reverter essas alterações antes de poder trabalhar na aplicação da sua correção ao que está em produção. Tudo o que você precisa fazer é mudar de volta para o seu branch `master`.

No entanto, antes de fazer isso, observe que se o seu diretório de trabalho ou a área de preparação tiverem alterações não commitadas que conflitam com o branch para o qual você está fazendo o checkout, o Git não deixará você mudar de branch. É melhor ter um estado de trabalho limpo quando você muda de branch. Existem maneiras de contornar isso (como, por exemplo, o stashing e a alteração de commits) que abordaremos mais tarde, em Fazendo Stash e Limpando. Por enquanto, vamos assumir que você fez commit de todas as suas alterações, então você pode mudar de volta para o seu branch `master`:

```
$ git checkout master
Switched to branch 'master'
```

Neste ponto, o diretório de trabalho do seu projeto está exatamente da mesma forma que estava antes de você começar a trabalhar na issue #53, e você pode se concentrar no seu hotfix. Este é um ponto importante a ser lembrado: quando você muda de branch, o Git redefine (resets) seu diretório de trabalho para que ele fique como na última vez em que você fez commit naquele branch. Ele adiciona, remove e modifica arquivos automaticamente para garantir que a sua cópia de trabalho seja exatamente como o branch estava no seu último commit nele.

Em seguida, você tem um hotfix a fazer. Vamos criar um branch `hotfix` no qual trabalharemos até que seja concluído:

```
$ git checkout -b hotfix
Switched to a new branch 'hotfix'
$ vim index.html
$ git commit -a -m 'Fix broken email address'
[hotfix 1fb7853] Fix broken email address
 1 file changed, 2 insertions(+)
```

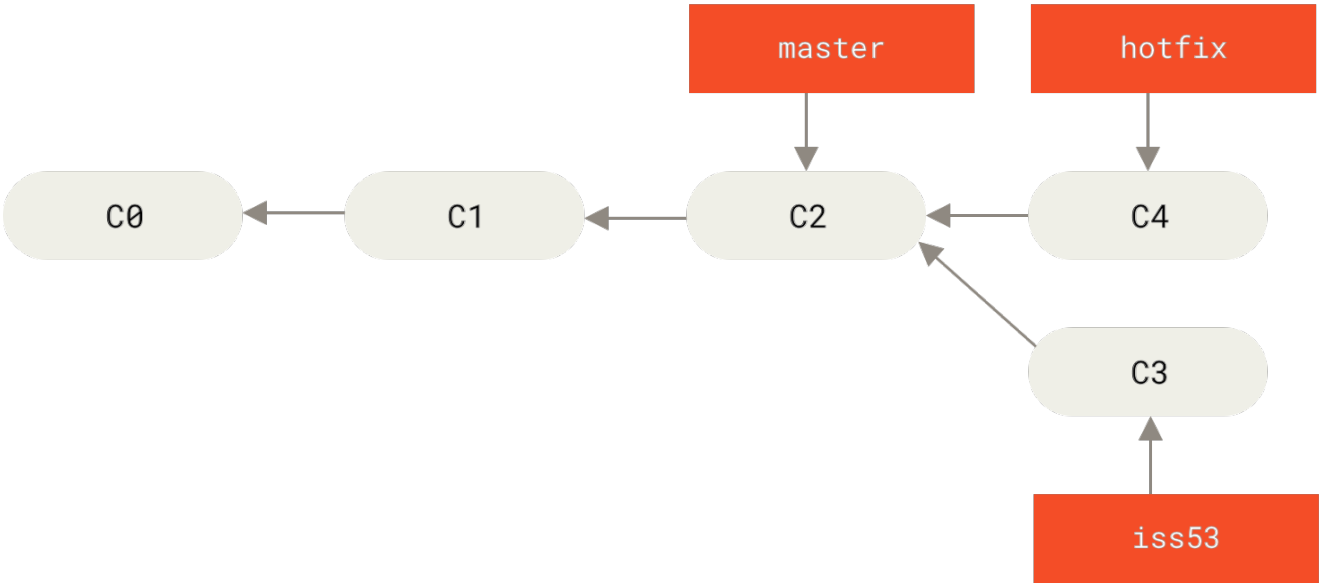


Figure 21. Branch hotfix baseado no `master`

Você pode executar os seus testes, certificar-se de que o hotfix é o que você deseja, e, finalmente, mesclar o branch `hotfix` de volta para o seu branch `master` para implantar na produção. Você faz isso com o comando `git merge`:

```
$ git checkout master
$ git merge hotfix
Updating f42c576..3a0874c
Fast-forward
 index.html | 2 ++
 1 file changed, 2 insertions(+)
```

Você notará a frase “fast-forward” (avanço rápido) nessa mesclagem. Como o commit `C4` apontado pelo branch `hotfix` que você mesclou estava diretamente à frente do commit `C2` em que você está, o Git simplesmente move o ponteiro para a frente. Em outras palavras, quando você tenta mesclar um commit com um commit que pode ser alcançado seguindo o histórico do primeiro commit, o Git simplifica as coisas movendo o ponteiro para a frente, pois não há trabalho divergente a ser mesclado — isso é chamado de “fast-forward.”

A sua alteração agora está no snapshot do commit apontado pelo branch `master`, e você pode implantar a correção.

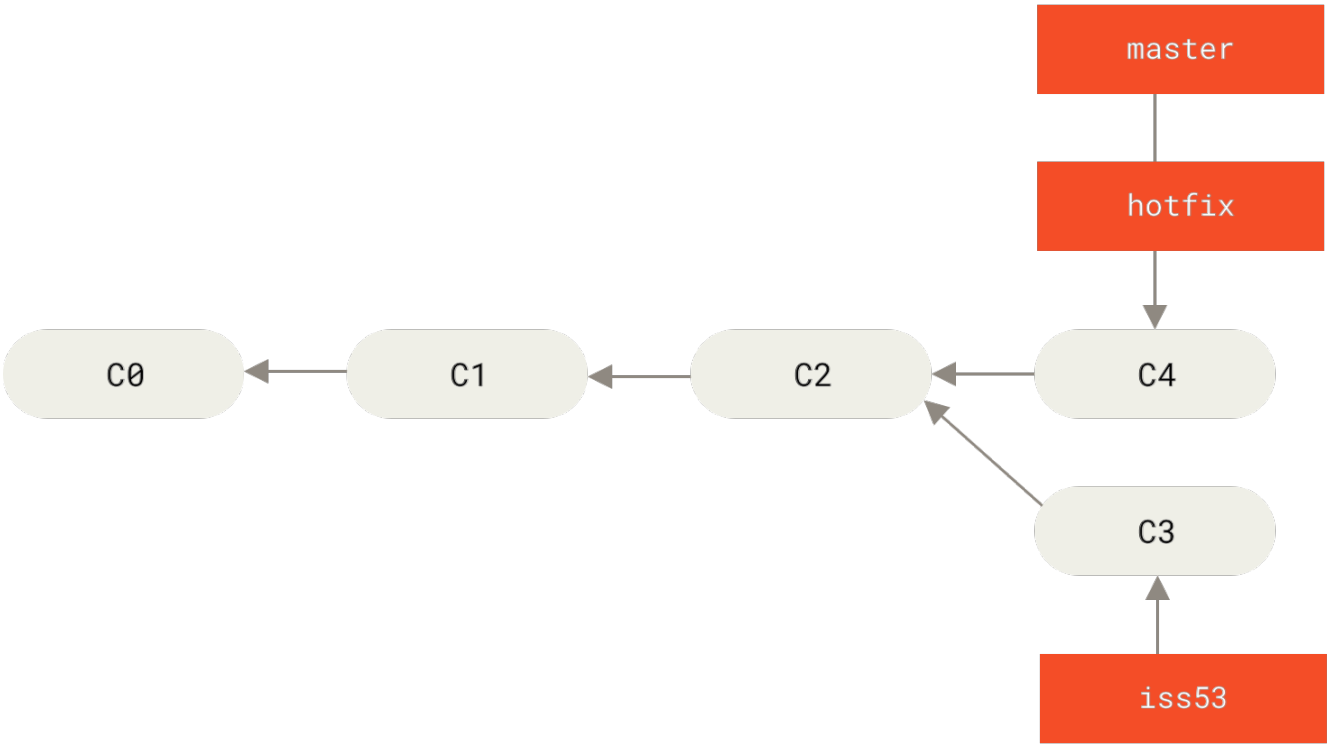


Figure 22. `master` é avançado (fast-forwarded) para `hotfix`

Depois que sua correção superimportante for implantada, você estará pronto para voltar ao trabalho que estava fazendo antes de ser interrompido. No entanto, primeiro você excluirá o branch `hotfix`, pois não precisa mais dele — o branch `master` aponta para o mesmo lugar. Você pode excluí-lo com a opção `-d` do comando `git branch`:

```
$ git branch -d hotfix
Deleted branch hotfix (3a0874c).
```

Agora você pode voltar ao seu branch de trabalho em andamento na issue #53 e continuar trabalhando nela.

```
$ git checkout iss53
Switched to branch "iss53"
$ vim index.html
$ git commit -a -m 'Finish the new footer [issue 53]'
[iss53 ad82d7a] Finish the new footer [issue 53]
1 file changed, 1 insertion(+)
```

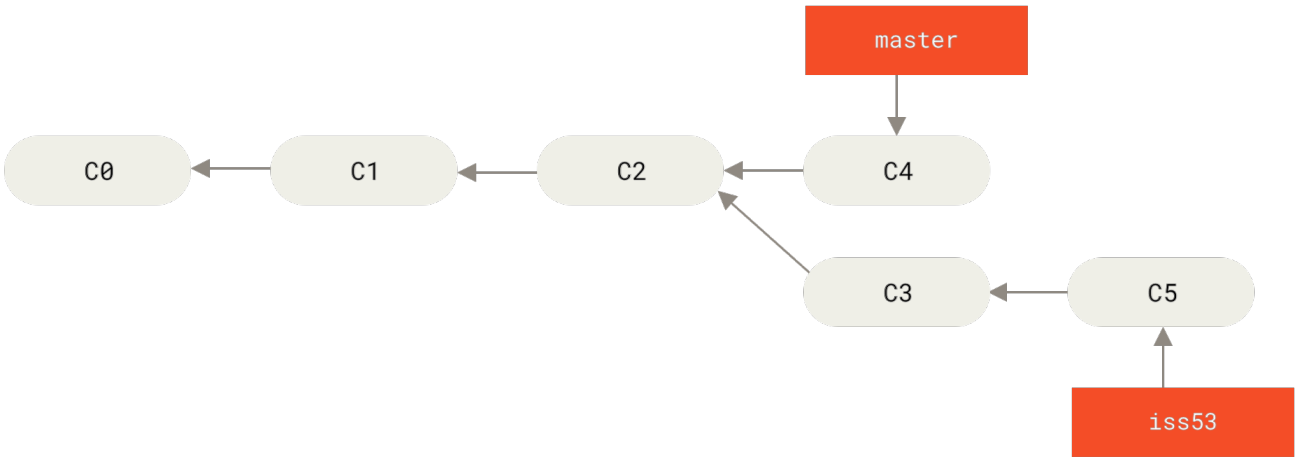


Figure 23. O trabalho continua no `iss53`

Vale ressaltar aqui que o trabalho que você fez no seu branch `hotfix` não está contido nos arquivos do seu branch `iss53`. Se você precisar incorporá-lo (pull it in), você pode mesclar (merge) o seu branch `master` para dentro do seu branch `iss53` executando `git merge master`, ou pode esperar para integrar essas alterações até decidir incorporar o branch `iss53` de volta ao `master` mais tarde.

Mesclagem (Merging) Básica

Suponha que você decidiu que seu trabalho na issue #53 está completo e pronto para ser mesclado (merged) no seu branch `master`. Para fazer isso, você mesclará seu branch `iss53` no `master`, de forma semelhante à mesclagem do seu branch `hotfix` anteriormente. Tudo o que você precisa fazer é fazer checkout do branch para o qual deseja mesclar (merge into) e depois executar o comando `git merge`:

```
$ git checkout master
Switched to branch 'master'
$ git merge iss53
Merge made by the 'recursive' strategy.
index.html |    1 +
1 file changed, 1 insertion(+)
```

Isso parece um pouco diferente da mesclagem do `hotfix` que você fez anteriormente. Neste caso, o seu histórico de desenvolvimento divergiu de algum ponto anterior. Como o commit do branch em que você está não é um ancestral direto do branch que você está mesclando, o Git tem que fazer algum trabalho. Nesse caso, o Git faz uma simples mesclagem de três vias (three-way merge), usando os dois snapshots apontados pelas pontas dos branches (branch tips) e o ancestral comum aos dois.

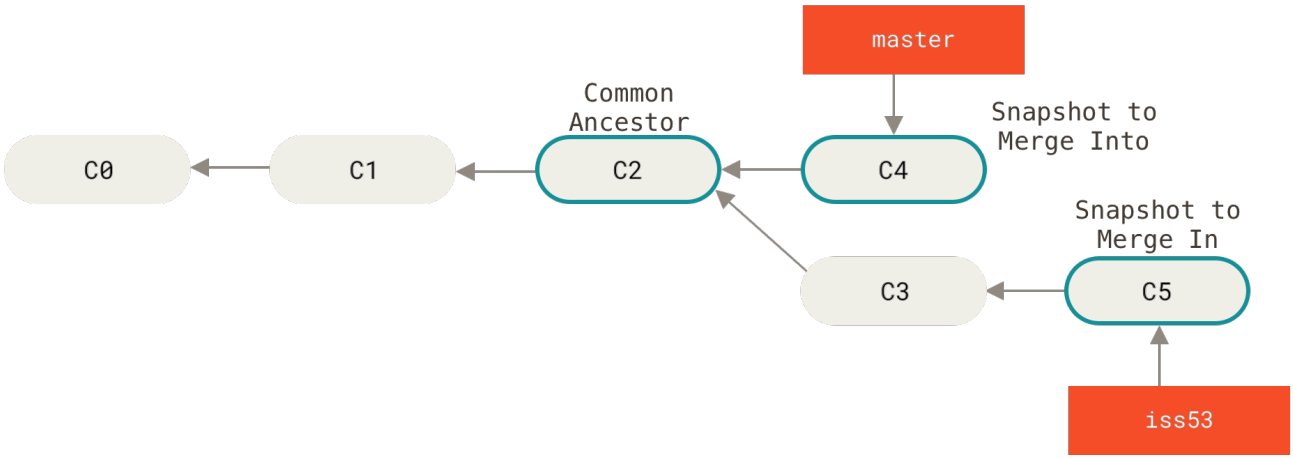


Figure 24. Três snapshots usados em uma mesclagem típica

Em vez de apenas mover o ponteiro do branch para a frente, o Git cria um novo snapshot que resulta dessa mesclagem de três vias (three-way merge) e cria automaticamente um novo commit que aponta para ele. Isso é conhecido como um merge commit (commit de mesclagem), e é especial porque tem mais de um pai.

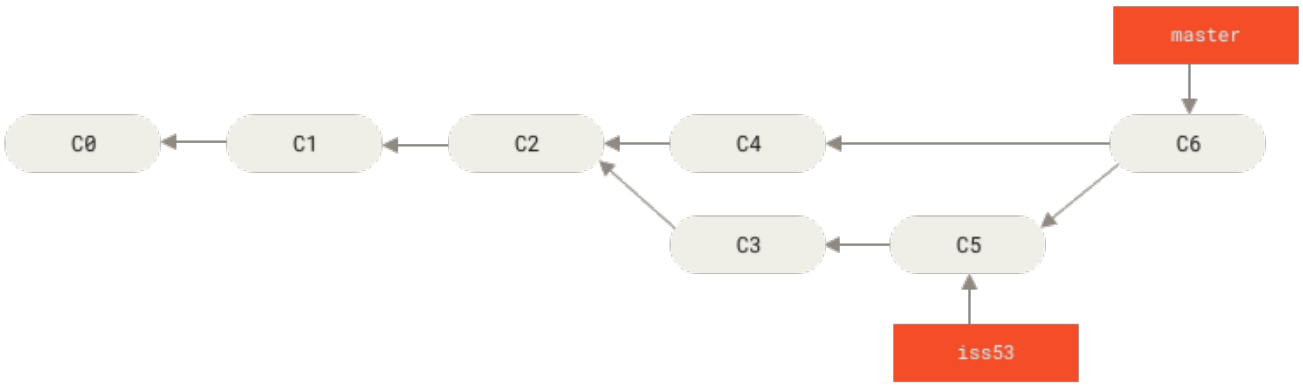


Figure 25. Um merge commit

Agora que o seu trabalho foi mesclado (merged), você não tem mais necessidade do branch `iss53`. Você pode fechar a issue no seu sistema de rastreamento de problemas e excluir o branch:

```
$ git branch -d iss53
```

Conflitos Básicos de Mesclagem (Merge Conflicts)

Ocasionalmente, esse processo não ocorre sem problemas. Se você alterou a mesma parte do mesmo arquivo de forma diferente nos dois branches que está mesclando, o Git não será capaz de mesclá-los de forma limpa (cleanly). Se a sua correção para a issue #53 modificou a mesma parte de um arquivo que o branch `hotfix`, você terá um conflito de mesclagem (merge conflict) parecido com

este:

```
$ git merge iss53
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

O Git não criou automaticamente um novo merge commit. Ele pausou o processo enquanto você resolve o conflito. Se você quiser ver quais arquivos não foram mesclados (unmerged) em qualquer ponto após um conflito de mesclagem, você pode executar o `git status`:

```
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add <file>..." to mark resolution)

    both modified:      index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Qualquer coisa que tenha conflitos de mesclagem e não tenha sido resolvida é listada como não mesclada (unmerged). O Git adiciona marcadores de resolução de conflitos padrão aos arquivos que têm conflitos, para que você possa abri-los manualmente e resolver esses conflitos. O seu arquivo contém uma seção parecida com esta:

```
<<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=======
<div id="footer">
 please contact us at support@github.com
</div>
>>>>>>> iss53:index.html
```

Isso significa que a versão no `HEAD` (seu branch `master`, porque era esse que você havia feito checkout quando executou o comando merge) é a parte superior daquele bloco (tudo acima de `=======`), enquanto a versão no seu branch `iss53` se parece com tudo o que está na parte inferior. Para resolver o conflito, você deve escolher um lado ou o outro ou mesclar o conteúdo você mesmo. Por exemplo, você pode resolver esse conflito substituindo o bloco inteiro por isso:

```
<div id="footer">
please contact us at email.support@github.com
</div>
```

Essa resolução tem um pouco de cada seção, e as linhas `<<<<<<<`, `=======` e `>>>>>>>` foram completamente removidas. Depois de resolver cada uma dessas seções em cada arquivo em conflito, execute `git add` em cada arquivo para marcá-lo como resolvido. Preparar o arquivo (staging) marca-o como resolvido no Git.

Se você quiser usar uma ferramenta gráfica para resolver esses problemas, você pode executar o `git mergetool`, que aciona uma ferramenta de mesclagem visual apropriada e o guia através dos conflitos:

```
$ git mergetool

This message is displayed because 'merge.tool' is not configured.
See 'git mergetool --tool-help' or 'git help config' for more details.
'git mergetool' will now attempt to use one of the following tools:
opendiff kdiff3 tkdiff xxdiff meld tortoisemerge gvimdiff diffuse diffmerge ecmerge
p4merge araxis bc3 codecompare vimdiff emerge
Merging:
index.html

Normal merge conflict for 'index.html':
  {local}: modified file
  {remote}: modified file
Hit return to start merge resolution tool (opendiff):
```

Se você quiser usar uma ferramenta de mesclagem diferente da padrão (o Git escolheu `opendiff` neste caso porque o comando foi executado no macOS), você pode ver todas as ferramentas suportadas listadas no topo após “one of the following tools.” Basta digitar o nome da ferramenta que você prefere usar.



Se você precisar de ferramentas mais avançadas para resolver conflitos de mesclagem complicados, cobriremos mais sobre mesclagem em Merging Avançado.

Depois de sair da ferramenta de mesclagem, o Git pergunta se a mesclagem foi bem-sucedida. Se você disser ao script que sim, ele prepara (stages) o arquivo para marcá-lo como resolvido para você. Você pode executar `git status` novamente para verificar se todos os conflitos foram resolvidos:

```
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:

    modified:   index.html
```

Se você estiver satisfeito com isso e verificar que tudo o que tinha conflitos foi preparado (staged), você pode digitar `git commit` para finalizar o merge commit. A mensagem de commit, por padrão, se parece com isto:

```
Merge branch 'iss53'

Conflicts:
    index.html
#
# It looks like you may be committing a merge.
# If this is not correct, please remove the file
#   .git/MERGE_HEAD
# and try again.


# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch master
# All conflicts fixed but you are still merging.
#
# Changes to be committed:
#   modified:   index.html
#
```

Se você acha que seria útil para outras pessoas analisarem essa mesclagem no futuro, você pode modificar esta mensagem de commit com detalhes sobre como você resolveu a mesclagem e explicar por que você fez as alterações que fez, se elas não forem óbvias.

Gerenciamento de Branches

Agora que você criou, mesclou (merged) e excluiu alguns branches, vamos dar uma olhada em algumas ferramentas de gerenciamento de branches que serão úteis quando você começar a usar branches o tempo todo.

O comando `git branch` faz mais do que apenas criar e excluir branches. Se você executá-lo sem argumentos, obterá uma listagem simples de seus branches atuais:

```
$ git branch
  iss53
* master
  testing
```

Observe o caractere `*` que antecede o branch `master`: ele indica o branch em que você fez checkout no momento (ou seja, o branch para o qual o `HEAD` aponta). Isso significa que, se você fizer um commit neste ponto, o branch `master` será movido para frente com o seu novo trabalho. Para ver o último commit em cada branch, você pode executar `git branch -v`:

```
$ git branch -v
  iss53   93b412c Fix javascript issue
* master  7a98805 Merge branch 'iss53'
  testing 782fd34 Add scott to the author list in the readme
```

As opções úteis `--merged` e `--no-merged` podem filtrar esta lista para branches que você tem ou ainda não mesclou no branch em que você está atualmente. Para ver quais branches já estão mesclados (merged) no branch em que você está, você pode executar `git branch --merged`:

```
$ git branch --merged
  iss53
* master
```

Como você já mesclou o `iss53` anteriormente, você o vê na sua lista. Branches nesta lista sem o `*` na frente geralmente podem ser excluídos sem problemas com o `git branch -d`; você já incorporou o trabalho deles a outro branch, então não perderá nada.

Para ver todos os branches que contêm trabalho que você ainda não mesclou, você pode executar `git branch --no-merged`:

```
$ git branch --no-merged
  testing
```

Isso mostra o seu outro branch. Como ele contém trabalho que ainda não foi mesclado, tentar excluí-lo com `git branch -d` falhará:

```
$ git branch -d testing
error: The branch 'testing' is not fully merged.
If you are sure you want to delete it, run 'git branch -D testing'.
```

Se você realmente deseja excluir o branch e perder esse trabalho, você pode forçar a exclusão com `-D`, como a mensagem de ajuda (helpful message) aponta.



As opções descritas acima, `--merged` e `--no-merged`, se não receberem um nome de commit ou de branch como argumento, mostrarão a você o que está, respectivamente, mesclado (merged) ou não mesclado (not merged) no seu branch *atual*.

Você sempre pode fornecer um argumento adicional para perguntar sobre o estado de mesclagem (merge state) em relação a algum outro branch sem fazer checkout desse outro branch primeiro, como em: o que não está mesclado no branch `master`?

```
$ git checkout testing
$ git branch --no-merged master
```

```
  topicA
  featureB
```

Alterando o nome de um branch



Não renomeie branches que ainda estão em uso por outros colaboradores. Não renomeie um branch como master/main/mainline sem ter lido a seção Alterando o nome do branch master.

Suponha que você tenha um branch chamado `bad-branch-name` e queira alterá-lo para `corrected-branch-name`, mantendo todo o histórico. Você também deseja alterar o nome do branch no remoto (GitHub, GitLab, outro servidor). Como você faz isso?

Renomeie o branch localmente com o comando `git branch --move`:

```
$ git branch --move bad-branch-name corrected-branch-name
```

Isso substitui o seu `bad-branch-name` por `corrected-branch-name`, mas essa alteração é apenas local por enquanto. Para permitir que outras pessoas vejam o branch corrigido no remoto, faça o push dele:

```
$ git push --set-upstream origin corrected-branch-name
```

Agora daremos uma breve olhada onde estamos agora:

```
$ git branch --all
* corrected-branch-name
  main
  remotes/origin/bad-branch-name
  remotes/origin/corrected-branch-name
  remotes/origin/main
```

Observe que você está no branch `corrected-branch-name` e ele está disponível no remoto. No entanto, o branch com o nome ruim também ainda está presente lá, mas você pode excluí-lo executando o seguinte comando:

```
$ git push origin --delete bad-branch-name
```

Agora, o nome do branch ruim foi totalmente substituído pelo nome do branch corrigido.

Alterando o nome do branch master



Alterar o nome de um branch como master/main/mainline/default quebrará as integrações, os serviços, os utilitários auxiliares e os scripts de

> compilação/lançamento (build/release) que o seu repositório usa. Antes de fazer isso, certifique-se de consultar os seus colaboradores. Além disso, certifique-se de fazer uma pesquisa minuciosa no seu repositório e atualizar quaisquer referências ao nome antigo do branch no seu código e scripts.

Renomeie o seu branch `master` local para `main` com o seguinte comando:

```
$ git branch --move master main
```

Não há mais branch `master` local, pois ele foi renomeado para branch `main`.

Para permitir que outras pessoas vejam o novo branch `main`, você precisa fazer o push dele para o remoto. Isso torna o branch renomeado disponível no remoto.

```
$ git push --set-upstream origin main
```

Agora acabamos com o seguinte estado:

```
$ git branch --all
* main
  remotes/origin/HEAD -> origin/master
  remotes/origin/main
  remotes/origin/master
```

O seu branch `master` local se foi, pois foi substituído pelo branch `main`. O branch `main` está presente no remoto. No entanto, o branch `master` antigo ainda está presente no remoto. Outros colaboradores continuarão a usar o branch `master` como base de seu trabalho, até que você faça algumas alterações adicionais.

Agora você tem mais algumas tarefas pela frente para concluir a transição:

- Quaisquer projetos que dependam deste precisarão atualizar o seu código e/ou configuração.
- Atualizar quaisquer arquivos de configuração do test-runner.
- Ajustar scripts de compilação (build) e lançamento (release).
- Redirecionar as configurações no host do seu repositório para coisas como o branch padrão do repositório, regras de mesclagem e outras coisas que correspondam a nomes de branch.
- Atualizar as referências ao branch antigo na documentação.
- Fechar ou mesclar quaisquer pull requests direcionados ao branch antigo.

Depois de concluir todas essas tarefas e ter certeza de que o branch `main` funciona exatamente como o branch `master`, você pode excluir o branch `master`:

```
$ git push origin --delete master
```

Fluxos de Trabalho de Branches (Branching Workflows)

Agora que você tem o básico de ramificação e mesclagem (branching e merging), o que você pode ou deve fazer com eles? Nesta seção, abordaremos alguns fluxos de trabalho comuns que essa ramificação leve torna possível, para que você possa decidir se gostaria de incorporá-los ao seu próprio ciclo de desenvolvimento.

Branches de Longa Duração (Long-Running Branches)

Como o Git usa uma simples mesclagem de três vias (three-way merge), mesclar de um branch para outro várias vezes durante um longo período é geralmente fácil de fazer. Isso significa que você pode ter vários branches que estão sempre abertos e que você usa para diferentes estágios do seu ciclo de desenvolvimento; você pode mesclar (merge) regularmente de alguns deles para outros.

Muitos desenvolvedores Git têm um fluxo de trabalho que adota essa abordagem, como ter apenas código que seja totalmente estável no seu branch `master` — possivelmente apenas código que foi ou será lançado (released). Eles têm outro branch paralelo chamado `develop` ou `next` com o qual trabalham ou usam para testar a estabilidade — não é necessariamente sempre estável, mas sempre que chega a um estado estável, pode ser mesclado (merged) no `master`. Ele é usado para extrair (pull in) branches de tópicos (branches de curta duração, como seu branch `iss53` anterior) quando eles estiverem prontos, para garantir que passem em todos os testes e não introduzam bugs.

Na realidade, estamos falando sobre ponteiros movendo-se na linha de commits que você está fazendo. Os branches estáveis estão mais abaixo na linha do seu histórico de commits, e os branches de ponta (bleeding-edge) estão mais acima no histórico.

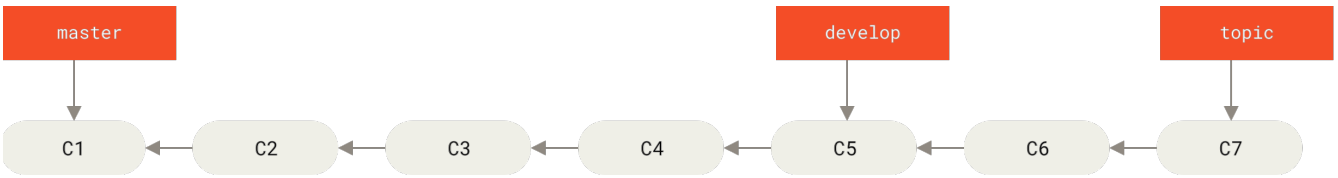


Figure 26. Uma visão linear da ramificação de estabilidade progressiva

Geralmente, é mais fácil pensar neles como silos de trabalho, onde conjuntos de commits se formam para um silo mais estável quando são totalmente testados.

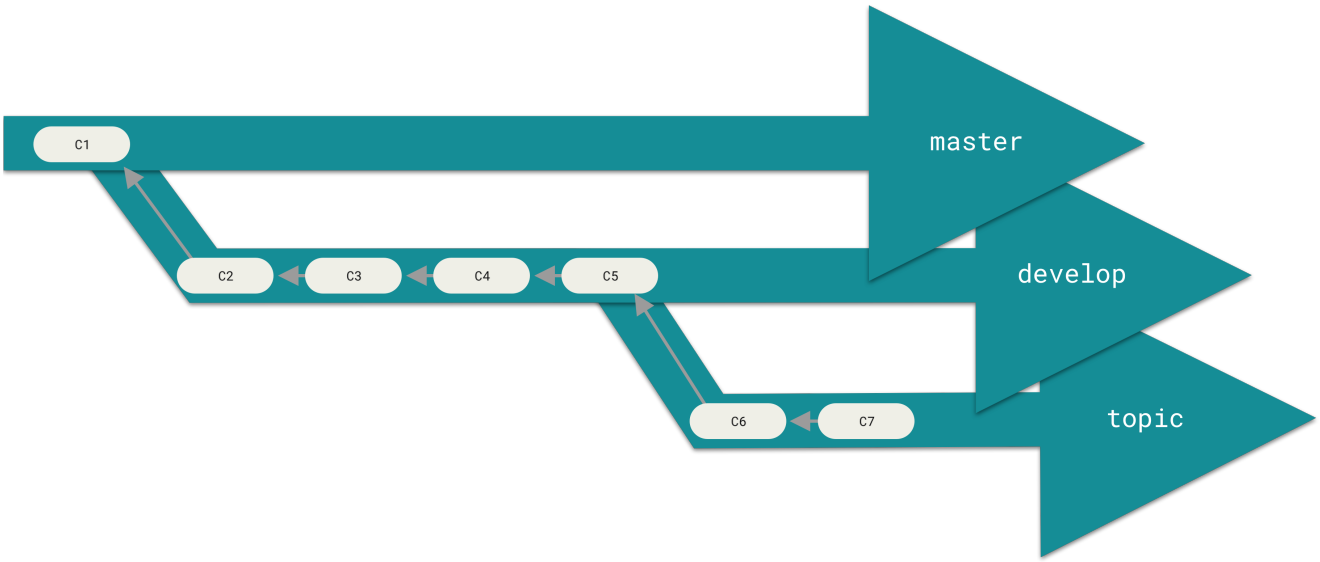


Figure 27. Uma visão em "silo" da ramificação de estabilidade progressiva

Você pode continuar fazendo isso por vários níveis de estabilidade. Alguns projetos maiores também têm um branch `proposed` ou `pu` (atualizações propostas) que integrou branches que podem não estar prontos para ir para o branch `next` ou `master`. A ideia é que os seus branches estejam em vários níveis de estabilidade; quando eles atingem um nível mais estável, eles são mesclados (merged) no branch acima deles. Novamente, ter vários branches de longa duração não é necessário, mas costuma ser útil, especialmente quando você está lidando com projetos muito grandes ou complexos.

Branches de Tópicos (Topic Branches)

Os branches de tópicos, no entanto, são úteis em projetos de qualquer tamanho. Um branch de tópico é um branch de curta duração que você cria e usa para um único recurso específico (feature) ou trabalho relacionado. Isso é algo que você provavelmente nunca fez com um VCS (Sistema de Controle de Versão) antes, porque geralmente é muito caro criar e mesclar branches. Mas no Git é comum criar, trabalhar, mesclar e excluir branches várias vezes ao dia.

Você viu isso na última seção com os branches `iss53` e `hotfix` que você criou. Você fez alguns commits neles e os excluiu diretamente após mesclá-los (merging) no seu branch principal. Esta técnica permite que você mude de contexto (context-switch) de forma rápida e completa — como o seu trabalho é separado em silos onde todas as alterações daquele branch têm a ver com aquele tópico, é mais fácil ver o que aconteceu durante a revisão de código e afins. Você pode manter as alterações lá por minutos, dias ou meses, e mesclá-las (merge) quando estiverem prontas, independentemente da ordem em que foram criadas ou trabalhadas.

Considere um exemplo de fazer algum trabalho (no `master`), ramificando (branching) para uma issue (`iss91`), trabalhando nela um pouco, ramificando o segundo branch para tentar outra maneira de lidar com a mesma coisa (`iss91v2`), voltando ao seu branch `master` e trabalhando lá por um tempo e, em seguida, ramificando para fazer um trabalho que você não tem certeza se é uma boa ideia (branch `dumbidea`). O seu histórico de commits ficará parecido com isto:

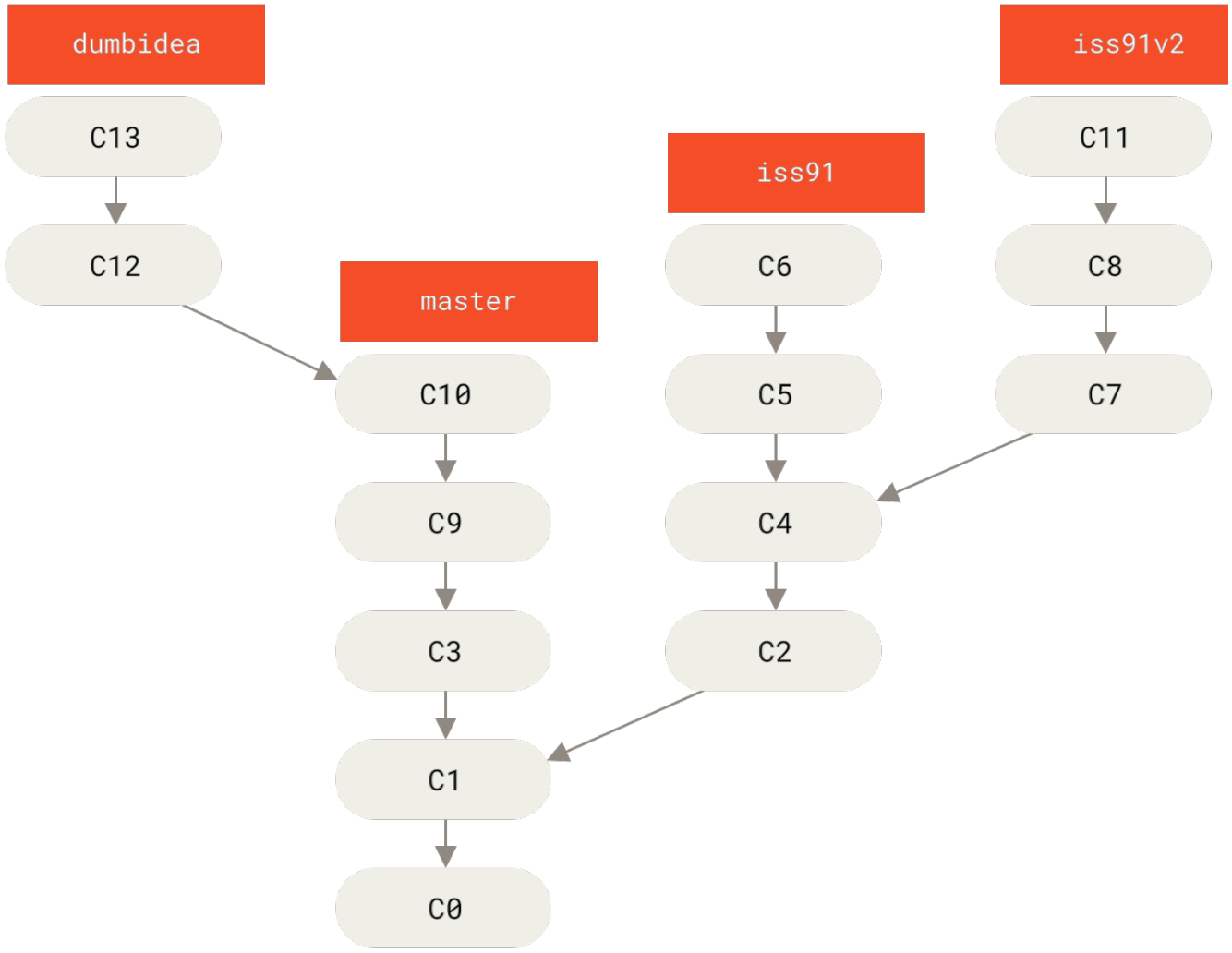


Figure 28. Múltiplos branches de tópicos

Agora, digamos que você decida que gosta mais da segunda solução para o seu problema (`iss91v2`); e você mostrou o branch `dumbidea` aos seus colegas de trabalho e ele acabou sendo genial. Você pode descartar (throw away) o branch original `iss91` (perdendo os commits `C5` e `C6`) e mesclar os outros dois. O seu histórico então fica assim:

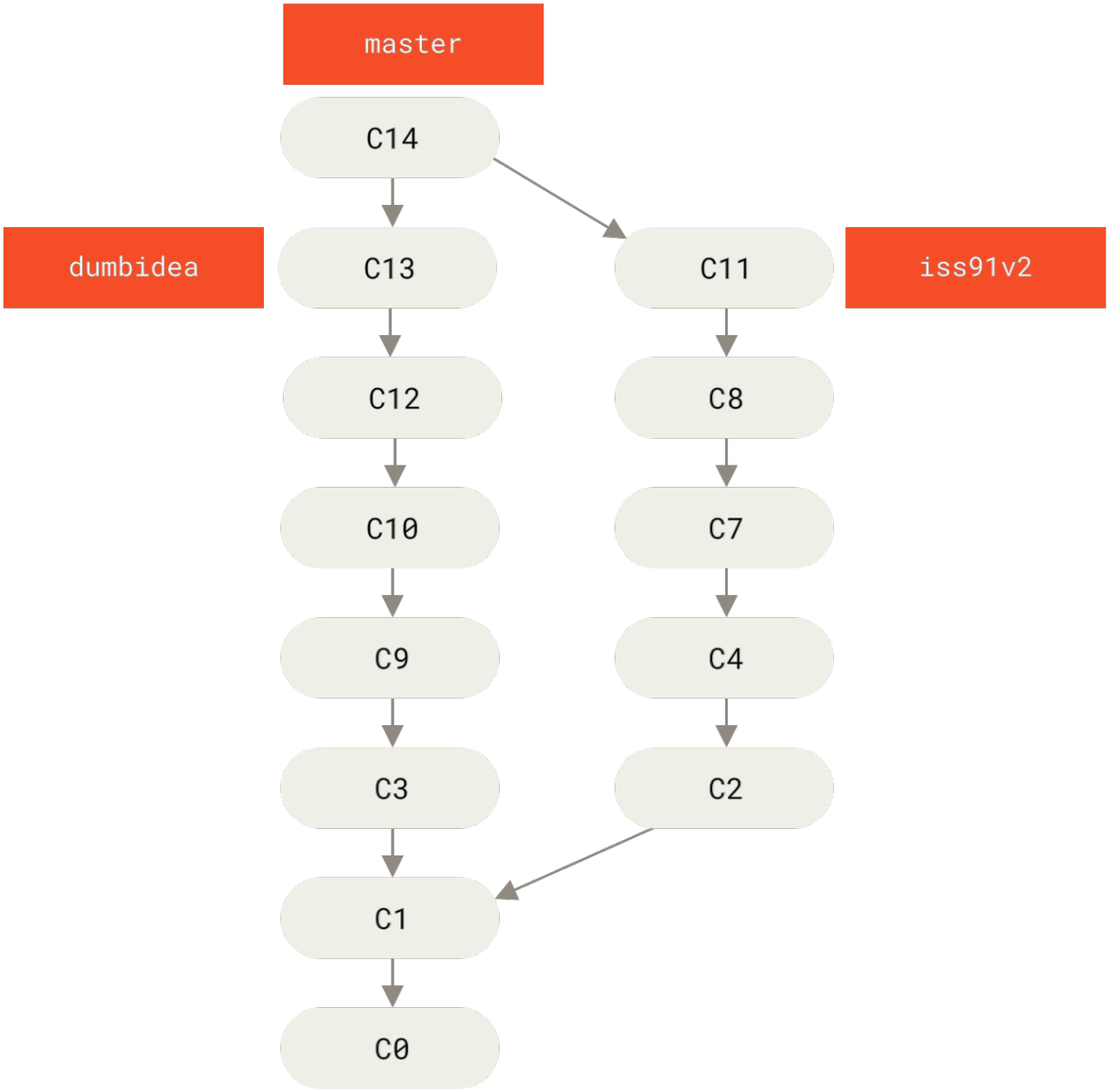


Figure 29. Histórico após mesclar `dumbidea` *e* `iss91v2`

Entraremos em mais detalhes sobre os vários fluxos de trabalho possíveis para o seu projeto Git em Git Distribuído, portanto, antes de decidir qual esquema de ramificação (branching scheme) o seu próximo projeto usará, certifique-se de ler esse capítulo.

É importante lembrar ao fazer tudo isso que esses branches são completamente locais. Quando você está ramificando e mesclando (branching e merging), tudo é feito apenas no seu repositório Git — não há comunicação com o servidor.

Branches Remotos (Remote Branches)

Referências remotas são referências (ponteiros) nos seus repositórios remotos, incluindo branches, tags e assim por diante. Você pode obter uma lista completa de referências remotas explicitamente com `git ls-remote <remote>`, ou `git remote show <remote>` para branches remotos, bem como mais informações. No entanto, uma maneira mais comum é aproveitar os branches de rastreamento

remoto (remote-tracking branches).

Branches de rastreamento remoto são referências ao estado dos branches remotos. Eles são referências locais que você não pode mover; o Git os move para você sempre que você faz qualquer comunicação de rede, para garantir que eles representem com precisão o estado do repositório remoto. Pense neles como favoritos, para lembrá-lo de onde estavam os branches nos seus repositórios remotos da última vez que você se conectou a eles.

Nomes de branches de rastreamento remoto assumem a forma `<remote>/<branch>`. Por exemplo, se você quisesse ver como o branch `master` no seu remoto `origin` estava da última vez que você se comunicou com ele, você verificaria o branch `origin/master`. Se você estivesse trabalhando em uma issue com um parceiro e ele fizesse o push de um branch `iss53`, você poderia ter o seu próprio branch local `iss53`, mas o branch no servidor seria representado pelo branch de rastreamento remoto `origin/iss53`.

Isso pode ser um pouco confuso, então vamos ver um exemplo. Digamos que você tenha um servidor Git na sua rede em `git.ourcompany.com`. Se você clonar a partir dele, o comando `clone` do Git o nomeia automaticamente como `origin` para você, baixa todos os seus dados, cria um ponteiro para onde o branch `master` dele está e o nomeia como `origin/master` localmente. O Git também fornece a você o seu próprio branch `master` local começando no mesmo lugar que o branch `master` do origin, para que você tenha algo a partir do qual trabalhar.



“origin” não é especial

Assim como o nome do branch “master” não tem nenhum significado especial no Git, o “origin” também não. Enquanto “master” é o nome padrão para um branch inicial quando você executa o `git init`, o que é a única razão pela qual ele é amplamente utilizado, “origin” é o nome padrão para um remoto quando você executa o `git clone`. Se, em vez disso, você executar `git clone -o booyah`, você terá `booyah/master` como o seu branch remoto padrão.

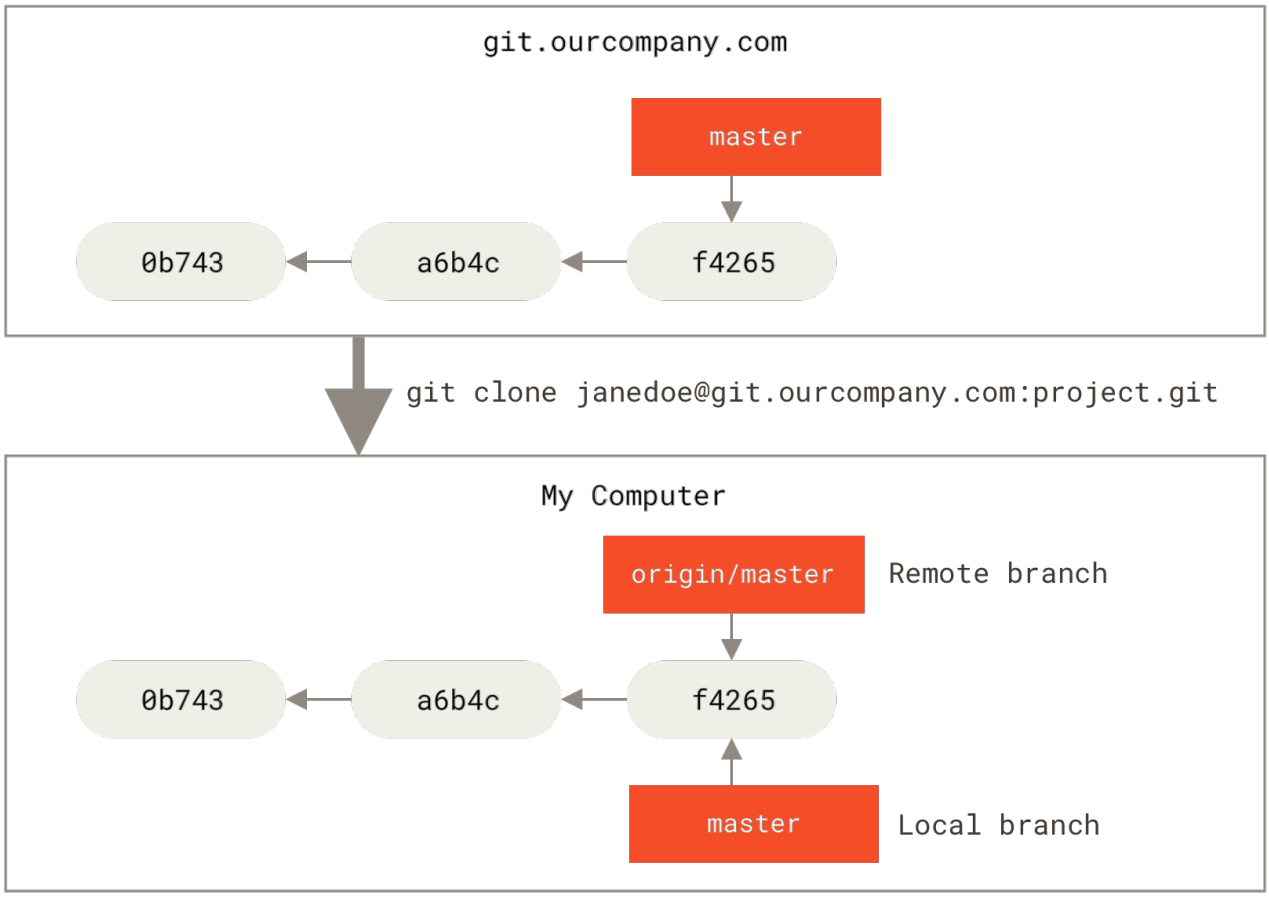


Figure 30. Repositórios locais e do servidor após clonar

Se você fizer algum trabalho no seu branch local `master` e, nesse meio-tempo, outra pessoa fizer push para `git.ourcompany.com` e atualizar o branch `master` de lá, os seus históricos avançarão de forma diferente. Além disso, enquanto você não se comunicar com o seu servidor `origin`, o seu ponteiro `origin/master` não se moverá.

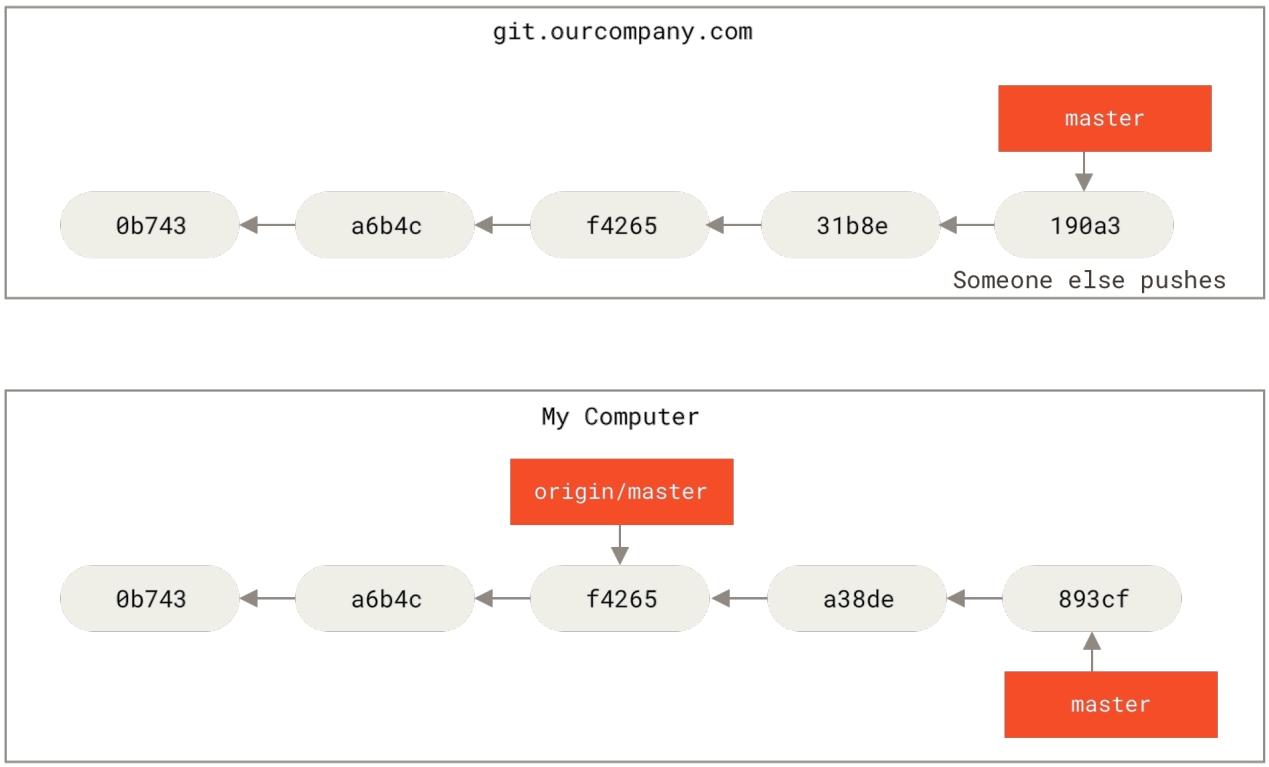


Figure 31. O trabalho local e o remoto podem divergir

Para sincronizar o seu trabalho com um determinado remoto, você executa o comando `git fetch <remote>` (no nosso caso, `git fetch origin`). Esse comando verifica qual servidor é o "origin" (neste caso, é `git.ourcompany.com`), busca os dados que você ainda não tem e atualiza o seu banco de dados local, movendo o seu ponteiro `origin/master` para a sua posição nova e mais atualizada.

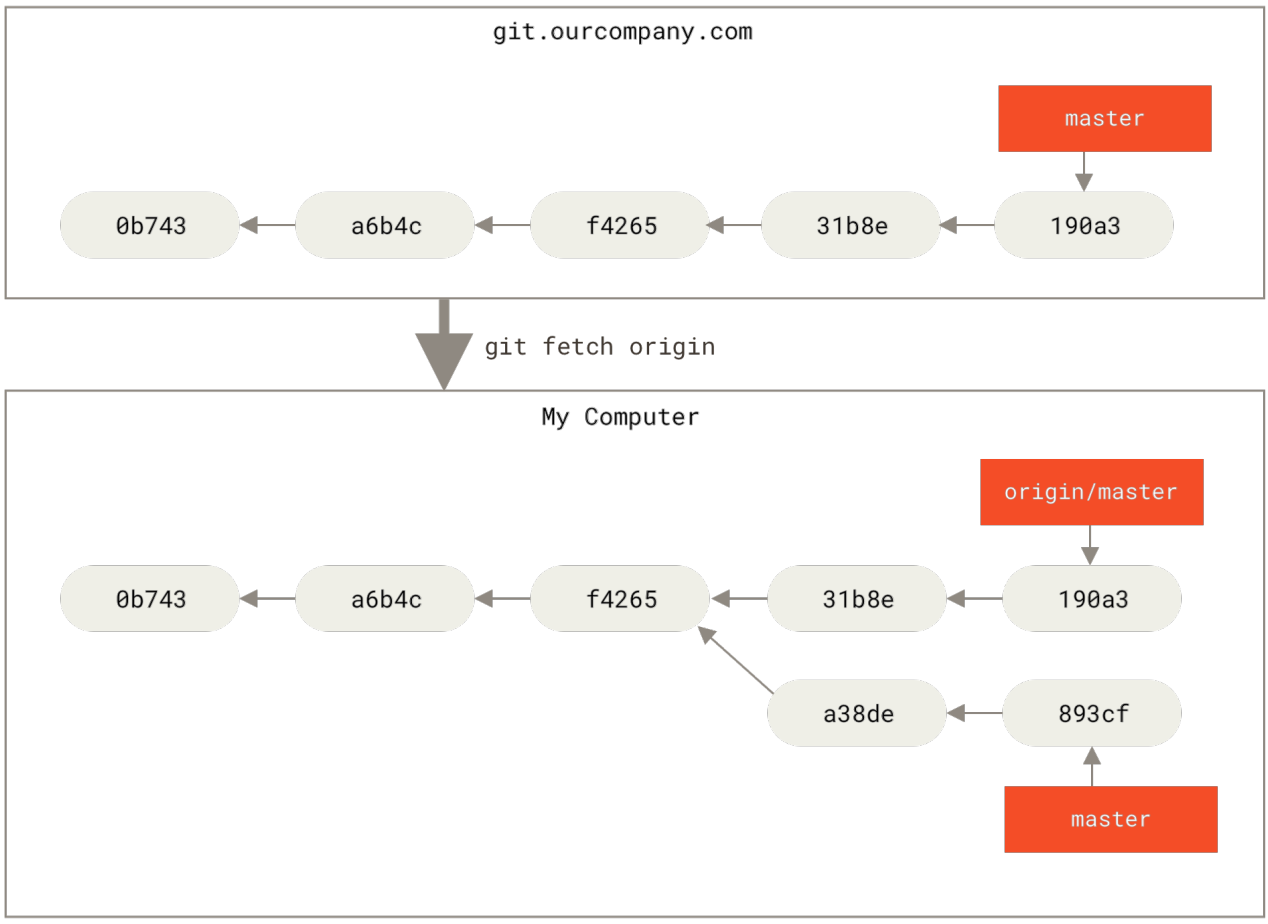


Figure 32. `git fetch` atualiza os seus branches de rastreamento remoto

Para demonstrar como ter vários servidores remotos e como se parecem os branches remotos para esses projetos remotos, vamos supor que você tenha outro servidor Git interno que é usado apenas para desenvolvimento por uma das suas equipes de sprint. Este servidor está em `git.team1.ourcompany.com`. Você pode adicioná-lo como uma nova referência remota ao projeto em que você está trabalhando atualmente executando o comando `git remote add`, conforme abordamos no Fundamentos do Git. Dê a este remoto o nome `teamone`, que será o seu nome abreviado (shortname) para toda a URL.

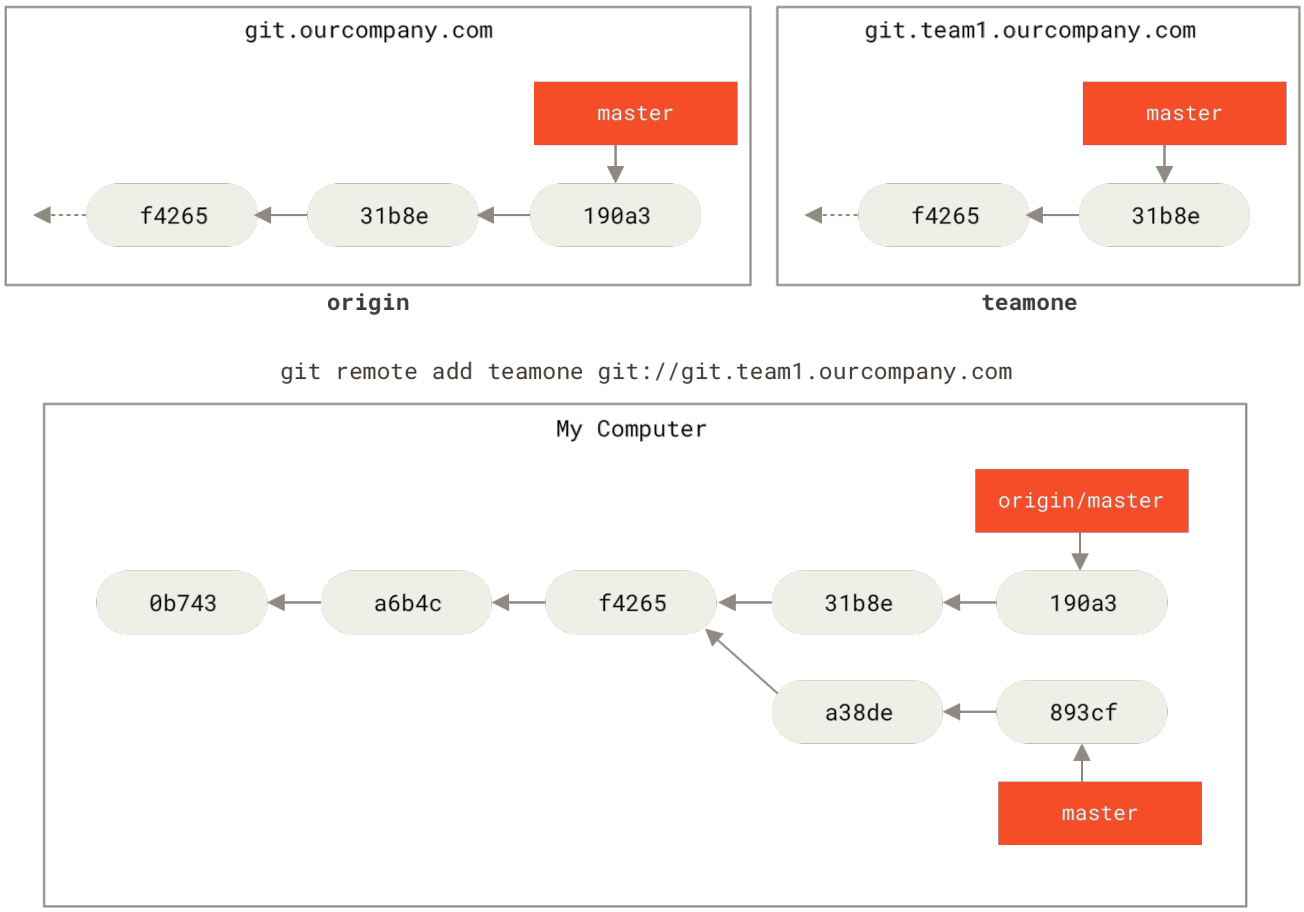


Figure 33. Adicionando outro servidor como remoto

Agora, você pode executar o `git fetch teamone` para buscar tudo o que o servidor remoto `teamone` tem e que você ainda não tem. Como esse servidor tem um subconjunto dos dados que o seu servidor `origin` tem no momento, o Git não busca dados, mas define um branch de rastreamento remoto chamado `teamone/master` para apontar para o commit que `teamone` tem como sendo o seu branch `master`.

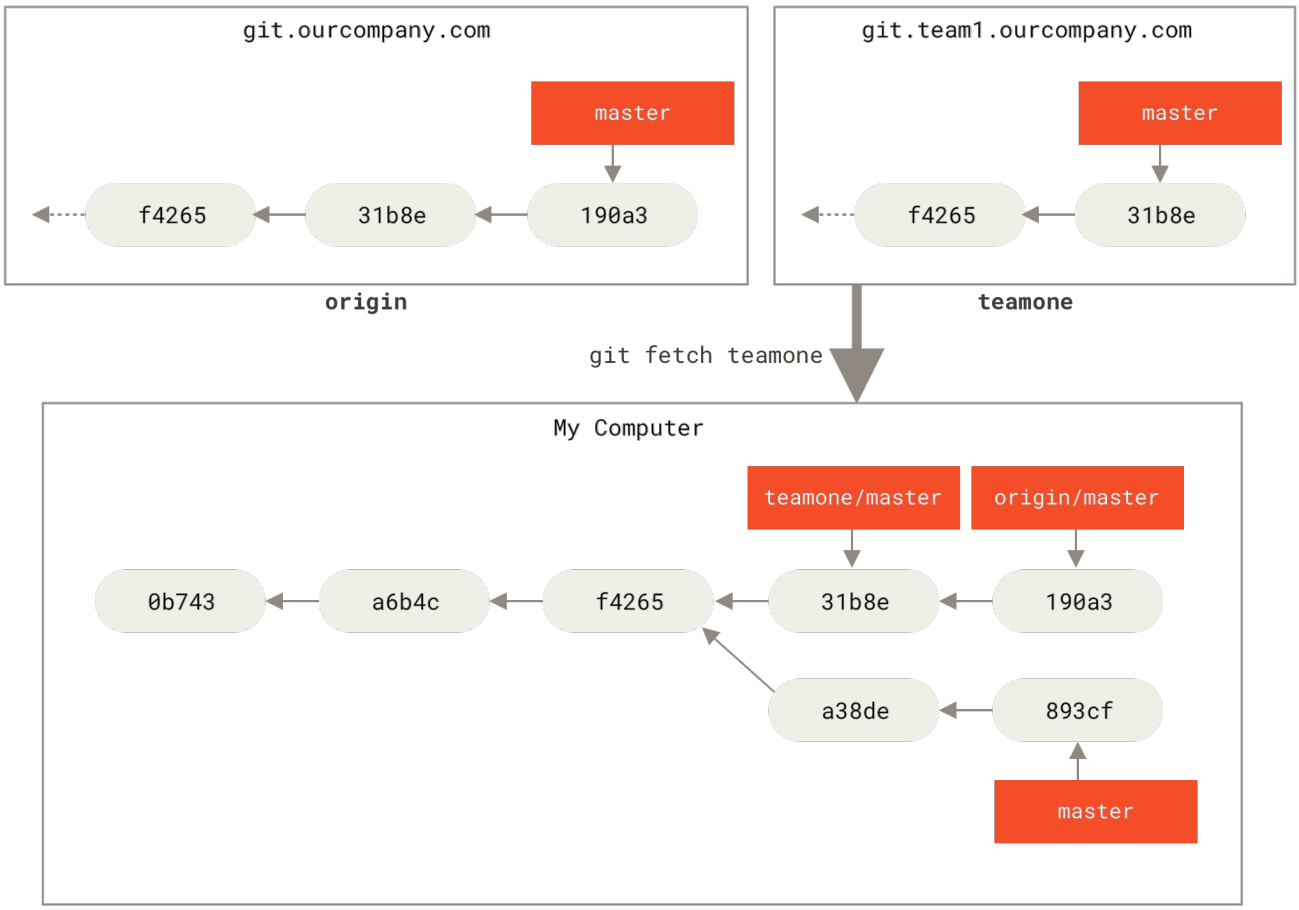


Figure 34. Branch de rastreamento remoto para `teamone/master`

Enviando (Pushing)

Quando você quiser compartilhar um branch com o mundo, você precisa fazer o push dele para um remoto no qual você tenha acesso de gravação. Os seus branches locais não são sincronizados automaticamente com os remotos nos quais você grava — você tem que fazer o push explicitamente dos branches que deseja compartilhar. Dessa forma, você pode usar branches privados para trabalhos que não deseja compartilhar e fazer o push apenas dos branches de tópicos nos quais deseja colaborar.

Se você tiver um branch chamado `serverfix` no qual deseja trabalhar com outras pessoas, você pode fazer o push dele da mesma forma que fez o push do seu primeiro branch. Execute `git push <remote> <branch>`:

```
$ git push origin serverfix
Counting objects: 24, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (15/15), done.
Writing objects: 100% (24/24), 1.91 KiB | 0 bytes/s, done.
Total 24 (delta 2), reused 0 (delta 0)
To https://github.com/schacon/simplegit
 * [new branch]      serverfix -> serverfix
```

Isso é um pouco de atalho. O Git expande automaticamente o nome do branch `serverfix` para

`refs/heads/serverfix:refs/heads/serverfix`, o que significa: “Pegue meu branch local `serverfix` e faça o push para atualizar o branch `serverfix` do remoto.” Abordaremos a parte `refs/heads/` em detalhes em Git Internals (Por Dentro do Git), mas geralmente você pode omiti-la. Você também pode fazer `git push origin serverfix:serverfix`, que faz a mesma coisa — ele diz: “Pegue meu serverfix e faça dele o serverfix do remoto.” Você pode usar esse formato para fazer o push de um branch local em um branch remoto com nome diferente. Se você não quisesse que ele fosse chamado de `serverfix` no remoto, você poderia executar `git push origin serverfix:awesomebranch` para fazer o push do seu branch local `serverfix` para o branch `awesomebranch` no projeto remoto.



Não digite sua senha toda vez

Se você estiver usando uma URL HTTPS para fazer o push, o servidor Git solicitará o seu nome de usuário e senha para autenticação. Por padrão, ele solicitará essas informações no terminal para que o servidor possa saber se você tem permissão para fazer o push.

Se você não quiser digitá-la todas as vezes que fizer um push, você pode configurar um “cache de credenciais” (credential cache). A maneira mais simples é mantê-la na memória por alguns minutos, o que você pode configurar facilmente executando `git config --global credential.helper cache`.

Para obter mais informações sobre as várias opções de cache de credenciais disponíveis, consulte Armazenamento de Credenciais.

Na próxima vez que um de seus colaboradores fizer um fetch do servidor, ele obterá uma referência de onde a versão do `serverfix` do servidor está no branch remoto `origin/serverfix`:

```
$ git fetch origin
remote: Counting objects: 7, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://github.com/schacon/simplegit
 * [new branch]      serverfix    -> origin/serverfix
```

É importante observar que, ao fazer um fetch que traz novos branches de rastreamento remoto, você não obtém automaticamente cópias locais editáveis deles. Em outras palavras, neste caso, você não tem um novo branch `serverfix` — você tem apenas um ponteiro `origin/serverfix` que não pode modificar.

Para mesclar (merge) esse trabalho no seu branch de trabalho atual, você pode executar `git merge origin/serverfix`. Se você quiser o seu próprio branch `serverfix` no qual possa trabalhar, você pode baseá-lo no seu branch de rastreamento remoto:

```
$ git checkout -b serverfix origin/serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

Isso fornece a você um branch local no qual você pode trabalhar e que começa onde `origin/serverfix` está.

Branches de Rastreamento (Tracking Branches)

Fazer o checkout de um branch local a partir de um branch de rastreamento remoto cria automaticamente o que é chamado de "branch de rastreamento" (tracking branch) (e o branch que ele rastreia é chamado de "branch upstream"). Branches de rastreamento são branches locais que têm uma relação direta com um branch remoto. Se você estiver em um branch de rastreamento e digitar `git pull`, o Git saberá automaticamente de qual servidor buscar (fetch) e qual branch mesclar (merge in).

Quando você clona um repositório, ele geralmente cria automaticamente um branch `master` que rastreia `origin/master`. No entanto, você pode configurar outros branches de rastreamento, se desejar — aqueles que rastreiam branches em outros remotos, ou não rastreiam o branch `master`. O caso simples é o exemplo que você acabou de ver, executando `git checkout -b <branch> <remote>/<branch>`. Esta é uma operação comum o suficiente para que o Git forneça o atalho `--track`:

```
$ git checkout --track origin/serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

Na verdade, isso é tão comum que há até um atalho para esse atalho. Se o nome do branch que você está tentando fazer checkout (a) não existir e (b) corresponder exatamente a um nome em apenas um remoto, o Git criará um branch de rastreamento para você:

```
$ git checkout serverfix
Branch serverfix set up to track remote branch serverfix from origin.
Switched to a new branch 'serverfix'
```

Para configurar um branch local com um nome diferente do branch remoto, você pode usar facilmente a primeira versão com um nome de branch local diferente:

```
$ git checkout -b sf origin/serverfix
Branch sf set up to track remote branch serverfix from origin.
Switched to a new branch 'sf'
```

Agora, o seu branch local `sf` fará o pull automaticamente de `origin/serverfix`.

Se você já tem um branch local e deseja defini-lo para um branch remoto do qual você acabou de fazer o pull (pulled down), ou deseja alterar o branch upstream que você está rastreando, você pode usar a opção `-u` ou `--set-upstream-to` para `git branch` para defini-lo explicitamente a qualquer momento.

```
$ git branch -u origin/serverfix
```

```
Branch serverfix set up to track remote branch serverfix from origin.
```



Atalho Upstream

Quando você tem um branch de rastreamento configurado, você pode referenciar o seu branch upstream com o atalho `@{upstream}` ou `@{u}`. Portanto, se você estiver no branch `master` e ele estiver rastreando `origin/master`, você pode dizer algo como `git merge @{u}` em vez de `git merge origin/master` se desejar.

Se você quiser ver quais branches de rastreamento você configurou, você pode usar a opção `-vv` do `git branch`. Isso listará os seus branches locais com mais informações, incluindo o que cada branch está rastreando e se o seu branch local está à frente (ahead), atrás (behind) ou ambos.

```
$ git branch -vv
  iss53     7e424c3 [origin/iss53: ahead 2] Add forgotten brackets
  master    1ae2a45 [origin/master] Deploy index fix
* serverfix f8674d9 [teamone/server-fix-good: ahead 3, behind 1] This should do it
  testing   5ea463a Try something new
```

Então, aqui podemos ver que o nosso branch `iss53` está rastreando `origin/iss53` e está “à frente” (ahead) por dois, o que significa que temos dois commits localmente que não foram enviados (pushed) para o servidor. Também podemos ver que o nosso branch `master` está rastreando `origin/master` e está atualizado. A seguir, podemos ver que o nosso branch `serverfix` está rastreando o branch `server-fix-good` no nosso servidor `teamone` e está à frente por três e atrás (behind) por um, o que significa que há um commit no servidor que ainda não mesclamos (merged in) e três commits localmente que não fizemos o push. Finalmente, podemos ver que o nosso branch `testing` não está rastreando nenhum branch remoto.

É importante observar que esses números são apenas desde a última vez que você fez um fetch de cada servidor. Esse comando não chega aos servidores, ele está dizendo o que tem em cache desses servidores localmente. Se você deseja números totalmente atualizados sobre o que está à frente e atrás, você precisará fazer um fetch de todos os seus remotos logo antes de executar isso. Você pode fazer isso assim:

```
$ git fetch --all; git branch -vv
```

Obtendo (Pulling)

Embora o comando `git fetch` busque todas as alterações no servidor que você ainda não tem, ele não modificará o seu diretório de trabalho de forma alguma. Ele simplesmente obterá os dados para você e permitirá que você os mescle (merge) você mesmo. No entanto, há um comando chamado `git pull` que é essencialmente um `git fetch` imediatamente seguido por um `git merge` na maioria dos casos. Se você tiver um branch de rastreamento configurado conforme demonstrado na seção anterior, definindo-o explicitamente ou fazendo com que seja criado para você pelos comandos `clone` ou `checkout`, o `git pull` procurará qual servidor e branch o seu branch atual está rastreando, fará o fetch desse servidor e tentará mesclar (merge in) esse branch remoto.

Excluindo Branches Remotos

Suponha que você tenha terminado com um branch remoto — digamos que você e seus colaboradores tenham terminado um recurso (feature) e o tenham mesclado (merged) no branch `master` do seu remoto (ou em qualquer branch em que a sua linha de código estável esteja). Você pode excluir um branch remoto usando a opção `--delete` para `git push`. Se você deseja excluir o seu branch `serverfix` do servidor, execute o seguinte:

```
$ git push origin --delete serverfix
To https://github.com/schacon/simplegit
 - [deleted]         serverfix
```

Basicamente, tudo o que isso faz é remover o ponteiro do servidor. O servidor Git geralmente manterá os dados lá por um tempo até que uma coleta de lixo (garbage collection) seja executada, então, se ele for excluído acidentalmente, geralmente é fácil de recuperar.

Rebase (Rebasing)

No Git, existem duas maneiras principais de integrar as alterações de um branch em outro: o `merge` (mesclagem) e o `rebase` (rebaseamento). Nesta seção, você aprenderá o que é o rebase, como fazê-lo, por que é uma ferramenta tão incrível e em quais casos você não vai querer usá-lo.

O Rebase Básico

Se você voltar a um exemplo anterior em Mesclagem (Merging) Básica, você pode ver que você divergiu o seu trabalho e fez commits em dois branches diferentes.

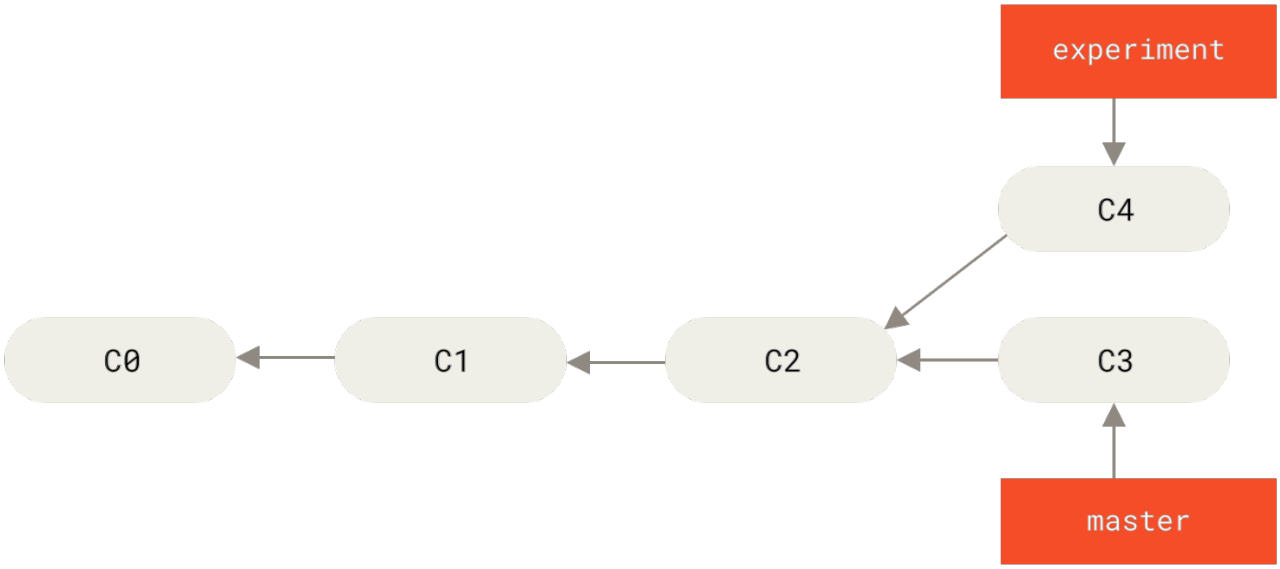


Figure 35. Histórico divergente simples

A maneira mais fácil de integrar os branches, como já abordamos, é com o comando `merge`. Ele executa uma mesclagem de três vias (three-way merge) entre os dois snapshots (instantâneos) de branch mais recentes (`C3` e `C4`) e o ancestral comum mais recente dos dois (`C2`), criando um novo snapshot (e commit).

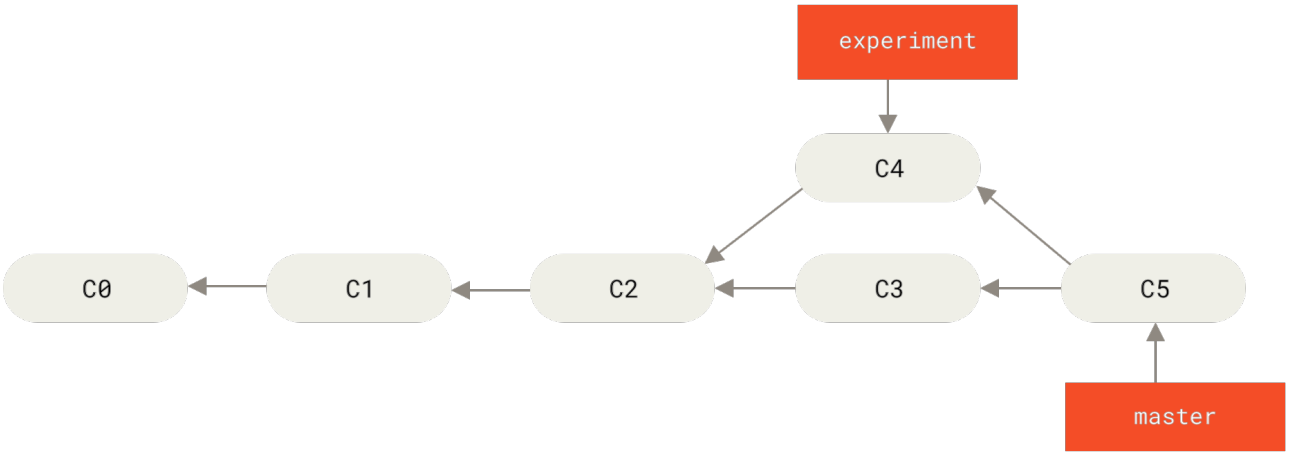


Figure 36. Mesclando (Merging) para integrar histórico de trabalho divergente

No entanto, há outra maneira: você pode pegar o patch da alteração que foi introduzida no `C4` e reaplicá-lo no topo do `C3`. No Git, isso é chamado de *rebasing* (rebaseamento). Com o comando `rebase`, você pode pegar todas as alterações que foram comitadas em um branch e repeti-las (replay) em um branch diferente.

Para este exemplo, você faria o checkout do branch `experiment` e então faria o rebase dele no branch `master` da seguinte maneira:

```
$ git checkout experiment
$ git rebase master
First, rewinding head to replay your work on top of it...
Applying: added staged command
```

Essa operação funciona indo ao ancestral comum dos dois branches (aquele em que você está e aquele para o qual você está fazendo o rebase), obtendo o diff introduzido por cada commit do branch em que você está, salvando esses diffs em arquivos temporários, redefinindo (resetting) o branch atual para o mesmo commit do branch para o qual você está fazendo o rebase e, finalmente, aplicando cada alteração uma por uma.

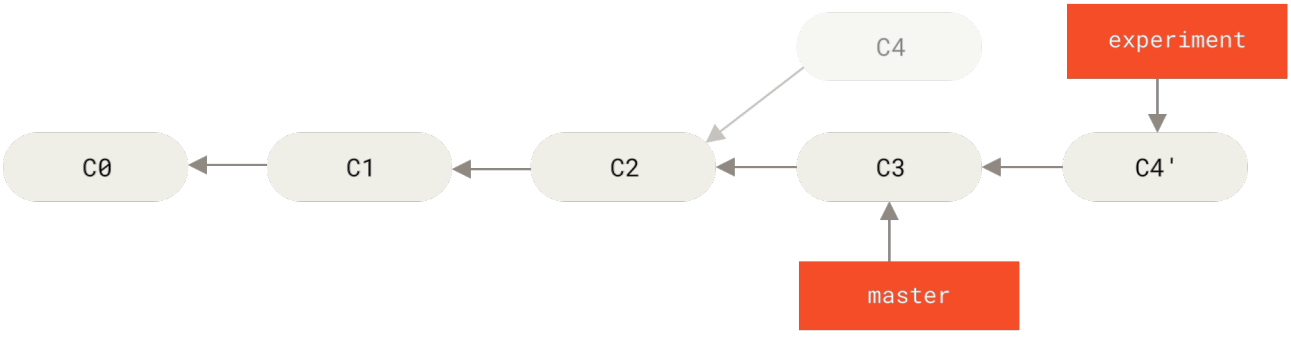


Figure 37. Fazendo o rebase da alteração introduzida em `C4` no `C3`

Neste ponto, você pode voltar ao branch `master` e fazer uma mesclagem de avanço rápido (fast-forward merge).

```
$ git checkout master
```

```
$ git merge experiment
```

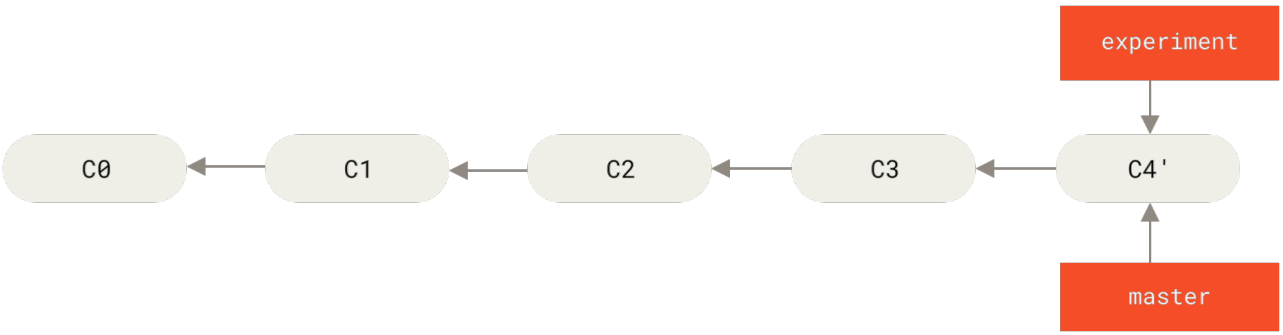


Figure 38. Avanço rápido (Fast-forwarding) do branch `master`

Agora, o snapshot apontado por `C4'` é exatamente o mesmo que o apontado por `C5` no exemplo de mesclagem (merge). Não há diferença no produto final da integração, mas o rebase proporciona um histórico mais limpo. Se você examinar o log de um branch com rebase, ele se parece com um histórico linear: parece que todo o trabalho aconteceu em série, mesmo quando originalmente aconteceu em paralelo.

Geralmente, você fará isso para garantir que os seus commits sejam aplicados de forma limpa em um branch remoto — talvez em um projeto para o qual você esteja tentando contribuir, mas que não mantém. Neste caso, você faria o seu trabalho em um branch e então faria o rebase do seu trabalho em `origin/master` quando estivesse pronto para enviar os seus patches para o projeto principal. Dessa forma, o mantenedor não precisa fazer nenhum trabalho de integração — apenas um avanço rápido (fast-forward) ou uma aplicação limpa.

Observe que o snapshot apontado pelo commit final que você obtém, seja ele o último dos commits rebaseados para um rebase ou o commit de mesclagem (merge) final após um merge, é o mesmo snapshot — apenas o histórico é diferente. O rebase repete (replays) as alterações de uma linha de trabalho para outra na ordem em que foram introduzidas, ao passo que a mesclagem (merging) pega os pontos finais e os mescla.

Rebases Mais Interessantes

Você também pode fazer com que o seu rebase seja repetido (replay) em algo diferente do branch de destino do rebase. Pegue um histórico como o Um histórico com um branch de tópico ramificado de outro branch de tópico, por exemplo. Você ramificou (branched) um branch de tópico (`server`) para adicionar algumas funcionalidades do lado do servidor ao seu projeto e fez um commit. Em seguida, você ramificou a partir dele para fazer as alterações no lado do cliente (`client`) e fez alguns commits. Por fim, você voltou ao seu branch `server` e fez mais alguns commits.

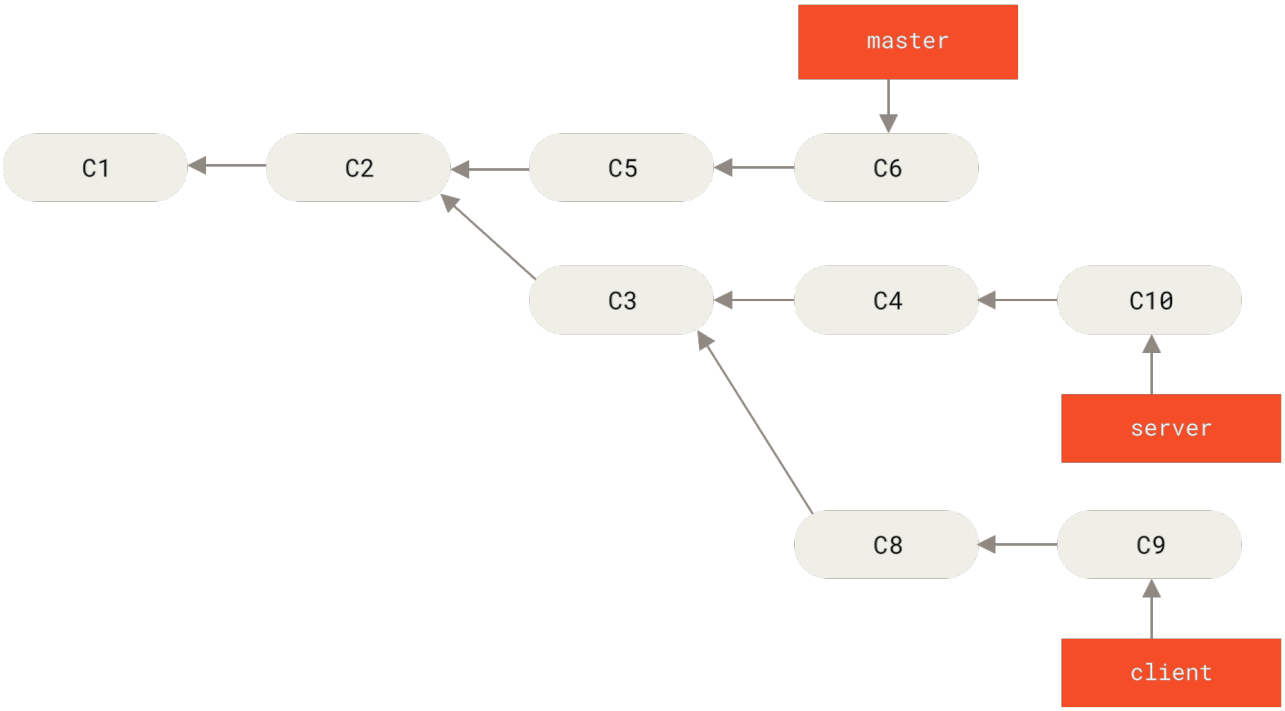


Figure 39. Um histórico com um branch de tópico ramificado de outro branch de tópico

Suponha que você decida que deseja mesclar as suas alterações no lado do cliente na sua linha principal (mainline) para um lançamento (release), mas deseja adiar as alterações no lado do servidor até que elas sejam mais testadas. Você pode pegar as alterações no `client` que não estão no `server` (`C8` e `C9`) e repeti-las (replay) no seu branch `master` usando a opção `--onto` do `git rebase`:

```
$ git rebase --onto master server client
```

Isso basicamente diz: "Pegue o branch `client`, descubra os patches desde que ele divergiu do branch `server` e repita (replay) esses patches no branch `client` como se ele fosse baseado diretamente no branch `master`." É um pouco complexo, mas o resultado é muito legal.

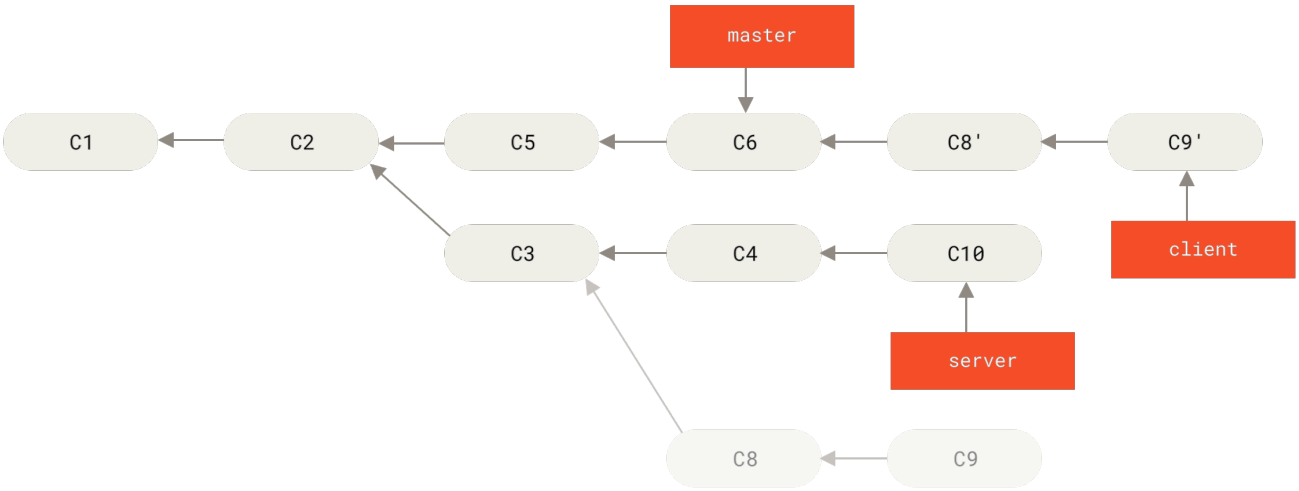


Figure 40. Fazendo o rebase de um branch de tópico a partir de outro branch de tópico

Agora você pode fazer o avanço rápido (fast-forward) do seu branch `master` (consulte Avanço rápido (Fast-forwarding) do seu branch `master` para incluir as alterações do branch `client`):

```
$ git checkout master
$ git merge client
```

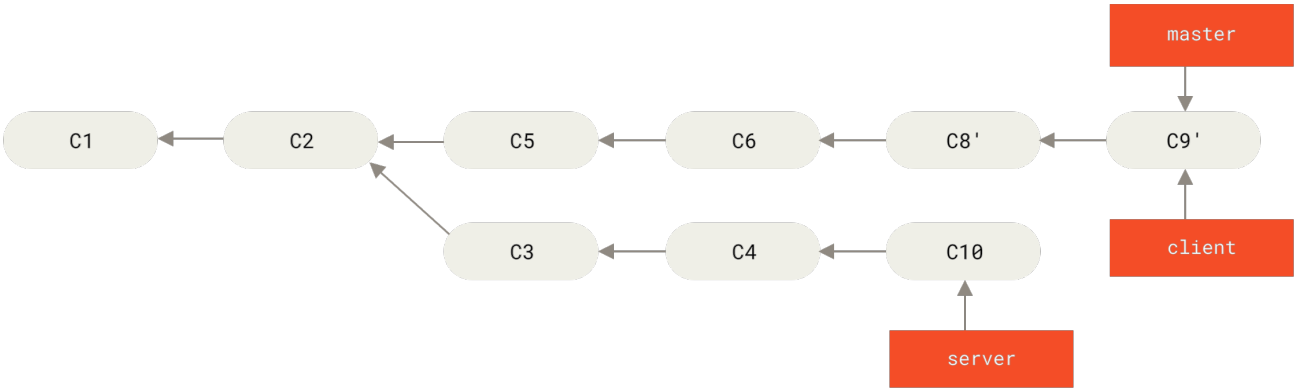


Figure 41. Avanço rápido (Fast-forwarding) do seu branch `master` para incluir as alterações do branch `client`

Digamos que você decida fazer o pull do seu branch `server` também. Você pode fazer o rebase do branch `server` no branch `master` sem precisar fazer o checkout dele primeiro executando `git rebase <basebranch> <topicbranch>` — que faz o checkout do branch de tópico (neste caso, `server`) para você e o repete (replays) no branch base (`master`):

```
$ git rebase master server
```

Isso repete (replays) o seu trabalho do `server` sobre o seu trabalho do `master`, como mostrado em Fazendo o rebase do seu branch `server` sobre o seu branch `master`.

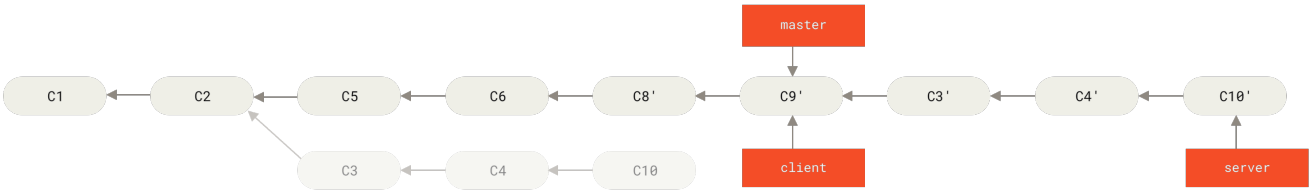


Figure 42. Fazendo o rebase do seu branch `server` sobre o seu branch `master`

Em seguida, você pode fazer o avanço rápido (fast-forward) do branch base (`master`):

```
$ git checkout master
$ git merge server
```

Você pode remover os branches `client` e `server` porque todo o trabalho está integrado e você não precisa mais deles, deixando o seu histórico de todo este processo parecido com o Histórico de commits final:

```
$ git branch -d client
$ git branch -d server
```

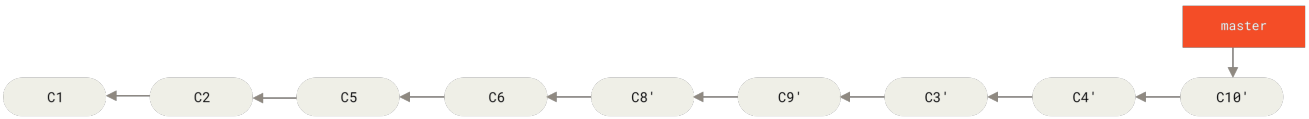


Figure 43. Histórico de commits final

Os Perigos do Rebase (Perils of Rebasing)

Ahh, mas a alegria do rebase não existe sem suas desvantagens, que podem ser resumidas em uma única linha:

Não faça rebase de commits que existem fora do seu repositório e nos quais as pessoas possam ter baseado o trabalho delas.

Se você seguir essa diretriz, ficará tudo bem. Se não o fizer, as pessoas o odiarão, e você será desprezado por amigos e familiares.

Quando você faz rebase de coisas, você está abandonando os commits existentes e criando novos que são parecidos, mas diferentes. Se você fizer o push de commits para algum lugar e outras pessoas fizerem o pull deles (pull down) e basearem o trabalho neles, e então você reescrever esses commits com o `git rebase` e fizer o push deles novamente, os seus colaboradores terão que mesclar (re-merge) o trabalho deles novamente e as coisas ficarão bagunçadas quando você tentar fazer o pull do trabalho deles de volta para o seu.

Vejamos um exemplo de como fazer o rebase de um trabalho que você tornou público pode causar problemas. Suponha que você clone de um servidor central e faça algum trabalho a partir dele. O seu histórico de commits fica assim:

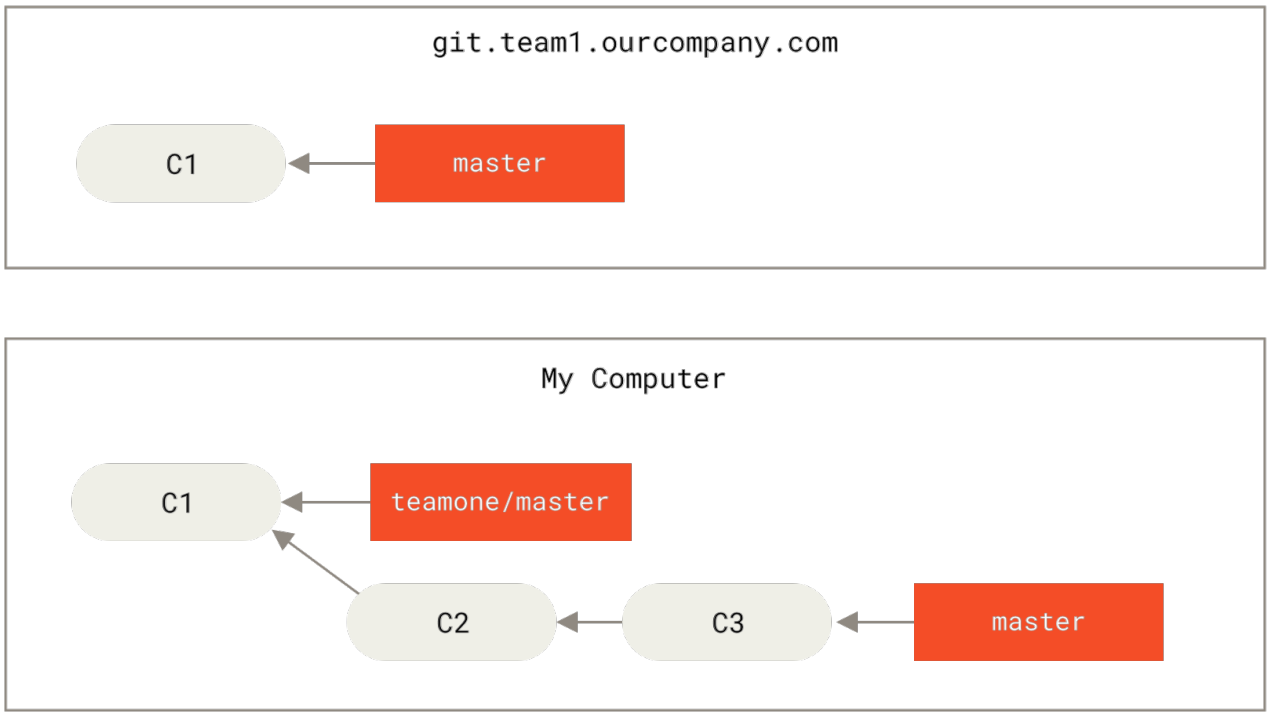


Figure 44. Clonar um repositório e basear algum trabalho nele

Agora, outra pessoa faz mais trabalho que inclui uma mesclagem (merge) e faz o push desse

trabalho para o servidor central. Você faz um fetch e mescla (merge) o novo branch remoto no seu trabalho, fazendo com que o seu histórico fique parecido com isto:

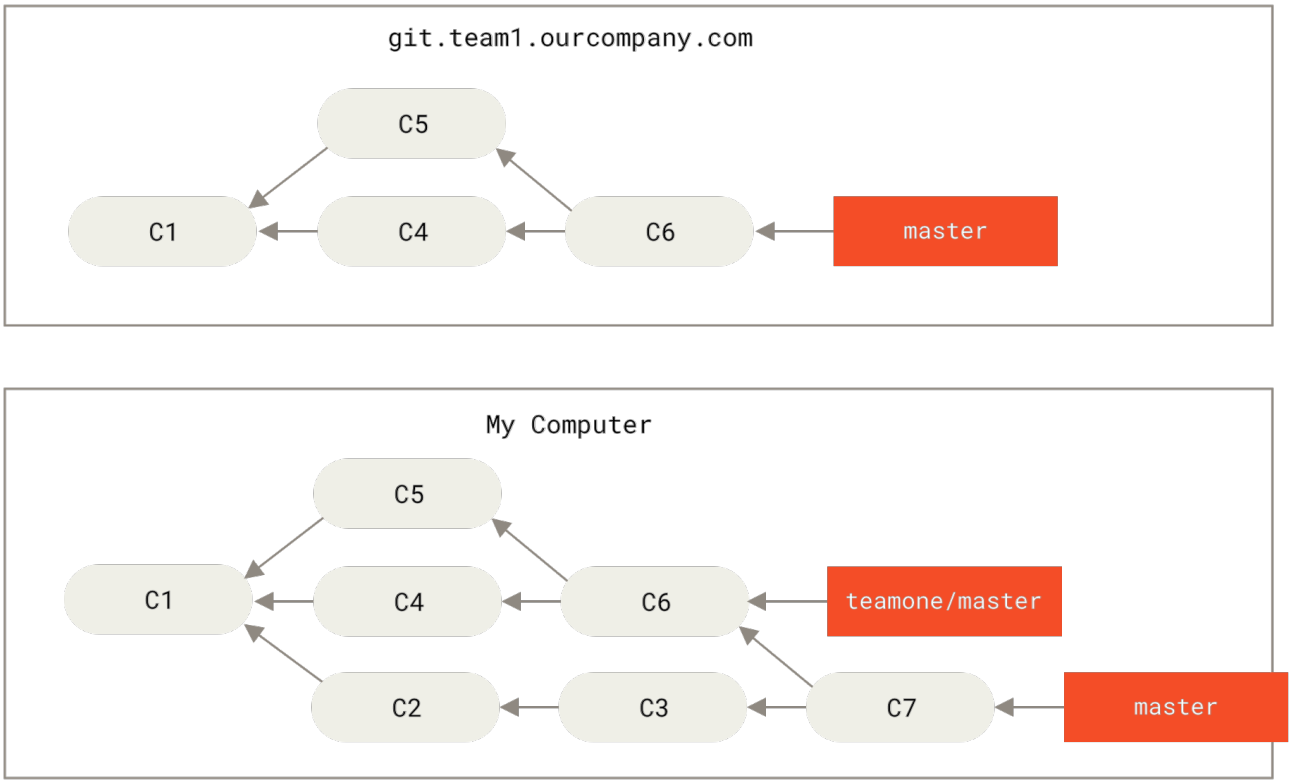


Figure 45. Buscar (Fetch) mais commits e mesclá-los (merge) no seu trabalho

A seguir, a pessoa que fez o push do trabalho mesclado decide voltar e, em vez disso, fazer o rebase do trabalho dela; ela faz um `git push --force` para substituir o histórico no servidor. Você então faz um fetch desse servidor, baixando os novos commits.

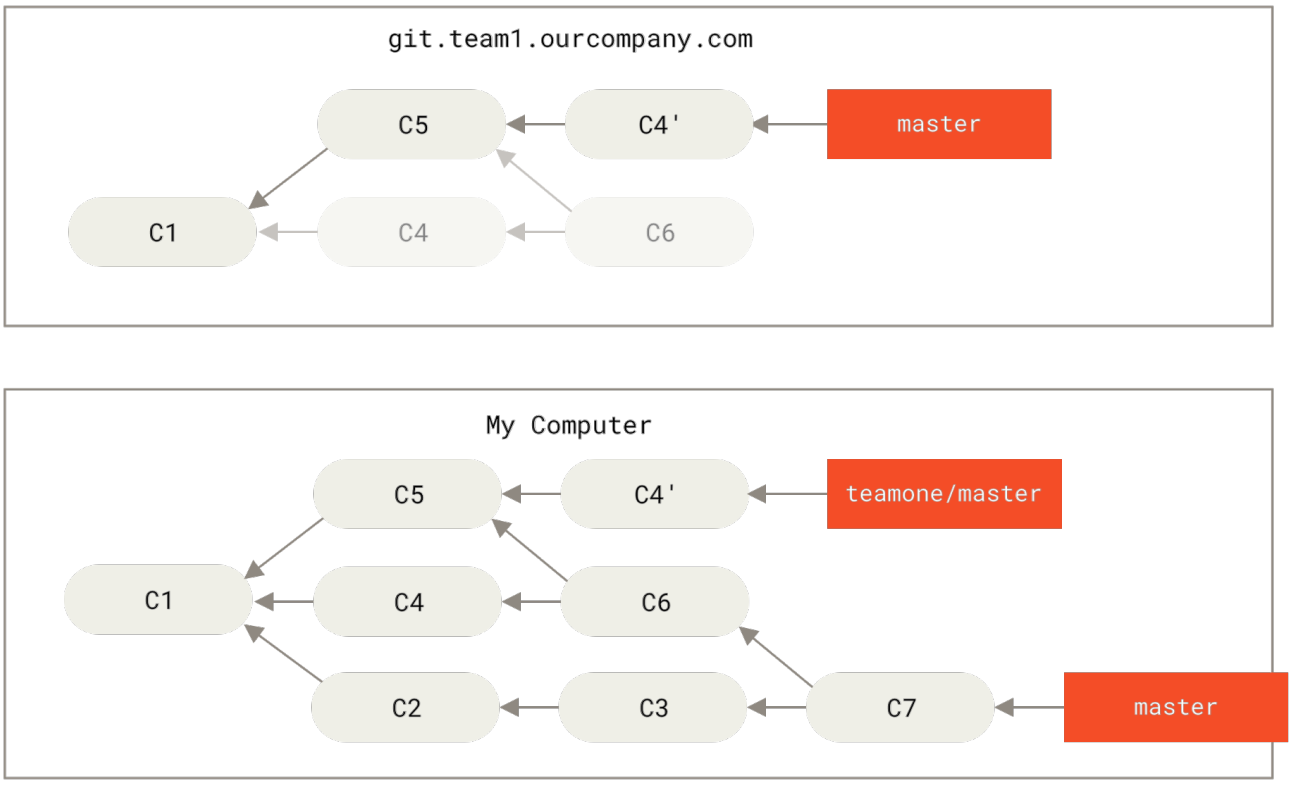


Figure 46. Alguém envia (pushes) commits em rebase, abandonando os commits nos quais você baseou o seu trabalho

Agora vocês dois estão em apuros. Se você fizer um `git pull`, você criará um commit de mesclagem (merge) que inclui ambas as linhas de histórico e o seu repositório ficará assim:

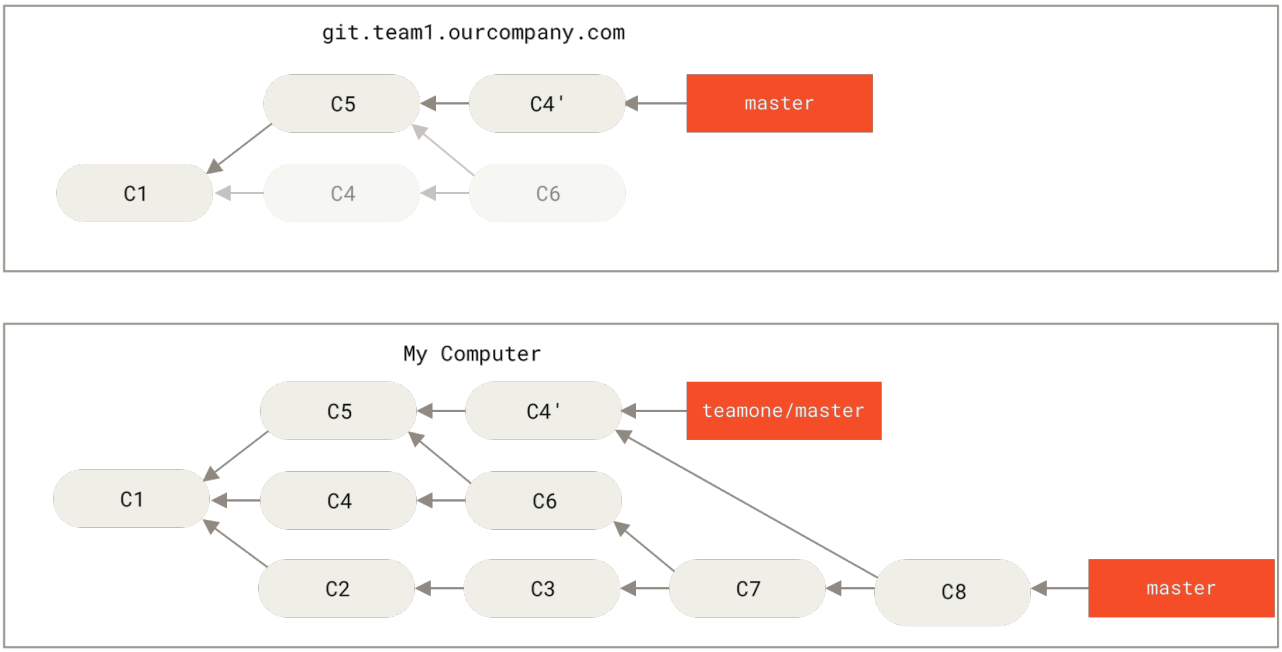


Figure 47. Você mescla (merge in) o mesmo trabalho novamente em um novo commit de mesclagem (merge commit)

Se você executar um `git log` quando o seu histórico estiver assim, você verá dois commits que têm o mesmo autor, data e mensagem, o que será confuso. Além disso, se você fizer o push desse

histórico de volta para o servidor, você reintroduzirá todos esses commits com rebase no servidor central, o que pode confundir ainda mais as pessoas. É bastante seguro supor que o outro desenvolvedor não deseja que `C4` e `C6` estejam no histórico; foi por isso que ele fez o rebase em primeiro lugar.

Rebase Quando Você Fizer Rebase

Se você **se** encontrar em uma situação como esta, o Git tem um pouco mais de mágica que pode ajudá-lo. Se alguém na sua equipe forçar o push de alterações que substituem um trabalho no qual você baseou o seu trabalho, o seu desafio é descobrir o que é seu e o que foi reescrito.

Acontece que, além da soma de verificação (checksum) SHA-1 do commit, o Git também calcula um checksum baseado apenas no patch introduzido com o commit. Isso é chamado de "patch-id".

Se você fizer o pull de um trabalho que foi reescrito e fizer o rebase no topo dos novos commits do seu parceiro, o Git geralmente consegue descobrir com sucesso o que é exclusivamente seu e aplicá-lo novamente no topo do novo branch.

Por exemplo, no cenário anterior, se, em vez de fazer uma mesclagem (merge) quando estamos em Alguém envia (pushes) commits em rebase, abandonando os commits nos quais você baseou o seu trabalho, executarmos `git rebase teamone/master`, o Git irá:

- Determinar qual trabalho é exclusivo do nosso branch (`C2`, `C3`, `C4`, `C6`, `C7`)
- Determinar quais não são commits de mesclagem (merge commits) (`C2`, `C3`, `C4`)
- Determinar quais não foram reescritos no branch de destino (apenas `C2` e `C3`, já que `C4` é o mesmo patch que `C4'`)
- Aplicar esses commits no topo de `teamone/master`

Então, em vez do resultado que vemos em Você mescla (merge in) o mesmo trabalho novamente em um novo commit de mesclagem (merge commit), terminaríamos com algo mais parecido com Rebase no topo do trabalho com rebase e push forçado.

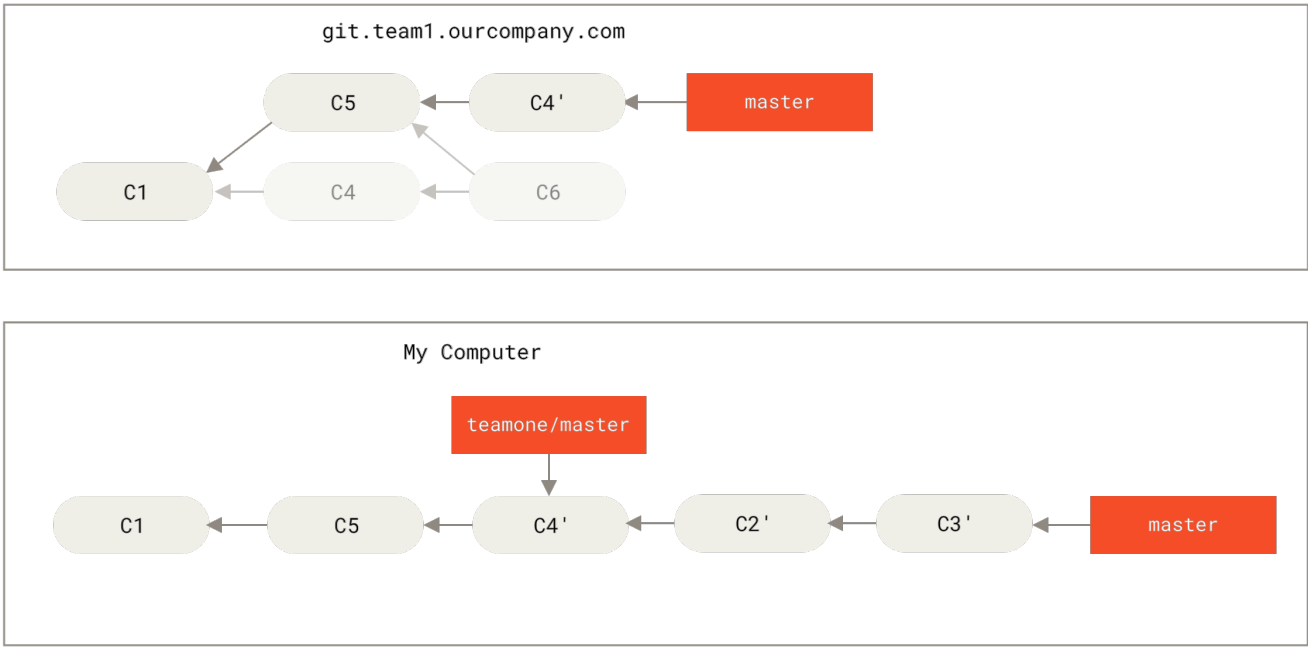


Figure 48. Rebase no topo do trabalho com rebase e push forçado

Isso só funciona se o `C4` e o `C4'` que o seu parceiro fez forem quase exatamente o mesmo patch. Caso contrário, o rebase não conseguirá perceber que se trata de uma duplicata e adicionará outro patch semelhante a `C4` (que provavelmente falhará ao ser aplicado de forma limpa, já que as alterações já estariam lá pelo menos em parte).

Você também pode simplificar isso executando um `git pull --rebase` em vez de um `git pull` normal. Ou você pode fazer isso manualmente com um `git fetch` seguido por um `git rebase teamone/master`, neste caso.

Se você está usando `git pull` e deseja tornar `--rebase` o padrão, você pode definir o valor de configuração `pull.rebase` com algo como `git config --global pull.rebase true`.

Se você fizer o rebase apenas de commits que nunca saíram do seu próprio computador, tudo ficará bem. Se você fizer o rebase de commits que foram enviados (pushed), mas que ninguém mais baseou os seus commits neles, você também ficará bem. Se você fizer rebase de commits que já foram enviados publicamente e as pessoas possam ter baseado trabalho nesses commits, então você poderá ter alguns problemas frustrantes e o desprezo dos seus colegas de equipe.

Se você ou um parceiro achar necessário em algum momento, certifique-se de que todos saibam que devem executar `git pull --rebase` para tentar tornar a dor um pouco mais simples depois que isso acontecer.

Rebase vs. Mesclar (Merge)

Agora que você viu o rebase e o merge em ação, você pode estar se perguntando qual deles é o melhor. Antes de respondermos a isso, vamos dar um passo atrás e falar sobre o que significa o histórico.

Um ponto de vista sobre isso é que o histórico de commits do seu repositório é um **registro do que realmente aconteceu.** É um documento histórico, valioso por si só, e não deve ser adulterado.

Desse ângulo, alterar o histórico de commits é quase uma blasfêmia; você está *mentindo* sobre o que realmente aconteceu. E daí se houve uma série bagunçada de commits de mesclagem (merge)? Foi assim que aconteceu, e o repositório deve preservar isso para a posteridade.

O ponto de vista oposto é que o histórico de commits é a **história de como o seu projeto foi feito.** Você não publicaria o primeiro rascunho de um livro, então por que mostrar o seu trabalho bagunçado? Ao trabalhar em um projeto, você pode precisar de um registro de todos os seus erros e becos sem saída, mas quando chegar a hora de mostrar o seu trabalho ao mundo, você pode querer contar uma história mais coerente de como ir de A a B. As pessoas nesse campo (camp) usam ferramentas como `rebase` e `filter-branch` para reescrever os seus commits antes de serem mesclados (merged) no branch da linha principal (mainline). Eles usam ferramentas como `rebase` e `filter-branch` para contar a história da maneira que for melhor para os futuros leitores.

Agora, sobre a questão de saber se mesclar (merging) ou fazer rebase é melhor: esperamos que você perceba que não é tão simples. O Git é uma ferramenta poderosa e permite que você faça muitas coisas no seu histórico e com ele, mas cada equipe e cada projeto são diferentes. Agora que você sabe como as duas coisas funcionam, cabe a você decidir qual é a melhor para a sua situação em particular.

Você pode obter o melhor dos dois mundos: faça o rebase das alterações locais antes de enviar (pushing) para limpar o seu trabalho, mas nunca faça o rebase de nada que você já tenha enviado (pushed) para algum lugar.

Resumo

Cobrimos o básico sobre ramificação (branching) e mesclagem (merging) no Git. Você deve se sentir confortável criando e mudando para novos branches, alternando entre branches e mesclando branches locais. Você também deve ser capaz de compartilhar seus branches enviando-os (push) para um servidor compartilhado, trabalhando com outras pessoas em branches compartilhados e fazendo rebase de seus branches antes de eles serem compartilhados. A seguir, abordaremos o que você precisará para executar seu próprio servidor de hospedagem de repositórios Git.

Git no Servidor

Neste ponto, você deve ser capaz de realizar a maioria das tarefas do dia a dia para as quais usará o Git. No entanto, para fazer qualquer colaboração no Git, você precisará ter um repositório Git remoto. Embora você possa tecnicamente fazer push de alterações e pull de alterações dos repositórios de indivíduos, fazer isso é desencorajado porque você pode facilmente confundir no que eles estão trabalhando se não for cuidadoso. Além disso, você quer que seus colaboradores possam acessar o repositório mesmo se o seu computador estiver offline — ter um repositório comum mais confiável costuma ser útil. Portanto, o método preferido para colaborar com alguém é configurar um repositório intermediário ao qual ambos tenham acesso e fazer push e pull dele.

Executar um servidor Git é bastante simples. Primeiro, você escolhe quais protocolos deseja que o seu servidor suporte. A primeira seção deste capítulo abordará os protocolos disponíveis e os prós e contras de cada um. As próximas seções explicarão algumas configurações típicas usando esses protocolos e como colocar o seu servidor para rodar com eles. Por último, abordaremos algumas opções hospedadas (hosted), se você não se importa em hospedar o seu código no servidor de outra pessoa e não quer passar pelo aborrecimento de configurar e manter o seu próprio servidor.

Se você não tem interesse em executar o seu próprio servidor, pode pular para a última seção do capítulo para ver algumas opções para configurar uma conta hospedada (hosted account) e depois passar para o próximo capítulo, onde discutimos os vários detalhes do trabalho em um ambiente de controle de fonte distribuído.

Um repositório remoto geralmente é um *bare repository* (repositório nu) — um repositório Git que não tem diretório de trabalho. Como o repositório é usado apenas como um ponto de colaboração, não há motivo para ter um snapshot com checkout no disco; são apenas os dados do Git. Nos termos mais simples, um repositório nu é o conteúdo do diretório `.git` do seu projeto e nada mais.

Os Protocolos

O Git pode usar quatro protocolos distintos para transferir dados: Local, HTTP, Secure Shell (SSH) e Git. Aqui, discutiremos o que eles são e em quais circunstâncias básicas você gostaria (ou não gostaria) de usá-los.

Protocolo Local

O mais básico é o *protocolo Local*, no qual o repositório remoto está em outro diretório no mesmo host. Isso é frequentemente usado se todos na sua equipe tiverem acesso a um sistema de arquivos compartilhado, como uma montagem NFS, ou no caso menos provável de que todos façam login no mesmo computador. Este último não seria o ideal, pois todas as suas instâncias de repositório de código residiriam no mesmo computador, tornando uma perda catastrófica muito mais provável.

Se você tiver um sistema de arquivos compartilhado montado, poderá clonar, fazer o push e o pull de um repositório local baseado em arquivo. Para clonar um repositório como esse ou para adicioná-lo como remoto a um projeto existente, use o caminho para o repositório como a URL. Por exemplo, para clonar um repositório local, você pode executar algo assim:

```
$ git clone /srv/git/project.git
```

Ou você pode fazer isso:

```
$ git clone file:///srv/git/project.git
```

O Git opera de forma ligeiramente diferente se você especificar explicitamente `file://` no início da URL. Se você apenas especificar o caminho, o Git tentará usar hardlinks (links físicos) ou copiar diretamente os arquivos de que precisa. Se você especificar `file://`, o Git aciona os processos que ele normalmente usa para transferir dados por uma rede, o que geralmente é muito menos eficiente. O principal motivo para especificar o prefixo `file://` é se você quiser uma cópia limpa do repositório, sem as referências estranhas (extraneous) ou os objetos deixados de fora — geralmente após uma importação de outro VCS ou algo semelhante (consulte Git Internals (Por Dentro do Git) para tarefas de manutenção). Usaremos o caminho normal aqui porque fazer isso é quase sempre mais rápido.

Para adicionar um repositório local a um projeto Git existente, você pode executar algo assim:

```
$ git remote add local_proj /srv/git/project.git
```

Então, você pode fazer o push e o pull desse remoto através do seu novo nome de remoto `local_proj` como se estivesse fazendo isso por uma rede.

Os Prós

Os prós dos repositórios baseados em arquivos são que eles são simples e usam as permissões de arquivos e o acesso à rede existentes. Se você já tem um sistema de arquivos compartilhado ao qual toda a sua equipe tem acesso, configurar um repositório é muito fácil. Você coloca a cópia do repositório nu em algum lugar ao qual todos tenham acesso compartilhado e define as permissões de leitura/gravação como faria para qualquer outro diretório compartilhado. Discutiremos como exportar uma cópia do repositório nu para este propósito em Colocando o Git em um Servidor.

Essa também é uma ótima opção para obter rapidamente o trabalho do repositório de trabalho de outra pessoa. Se você e um colega de trabalho estiverem trabalhando no mesmo projeto e eles quiserem que você verifique algo, executar um comando como `git pull /home/john/project` costuma ser mais fácil do que eles fazerem o push para um servidor remoto e você, em seguida, fazer o fetch a partir dele.

Os Contras

Os contras deste método são que o acesso compartilhado geralmente é mais difícil de configurar e alcançar de vários locais do que o acesso básico à rede. Se você deseja fazer o push do seu laptop quando está em casa, precisa montar o disco remoto, o que pode ser difícil e lento em comparação com o acesso baseado em rede.

É importante mencionar que esta não é necessariamente a opção mais rápida se você estiver

usando um tipo de montagem compartilhada. Um repositório local só é rápido se você tiver acesso rápido aos dados. Um repositório em NFS geralmente é mais lento do que o repositório via SSH no mesmo servidor, permitindo que o Git seja executado nos discos locais de cada sistema.

Finalmente, esse protocolo não protege o repositório contra danos acidentais. Cada usuário tem acesso shell total ao diretório "remoto", e não há nada que o impeça de alterar ou remover arquivos internos do Git e corromper o repositório.

Os Protocolos HTTP

O Git pode se comunicar via HTTP usando dois modos diferentes. Antes do Git 1.6.6, havia apenas uma maneira de fazer isso, que era muito simples e geralmente apenas para leitura (read-only). Na versão 1.6.6, um protocolo novo e mais inteligente foi introduzido, que envolvia o Git sendo capaz de negociar inteligentemente a transferência de dados de uma maneira semelhante a como ele faz via SSH. Nos últimos anos, esse novo protocolo HTTP se tornou muito popular, pois é mais simples para o usuário e mais inteligente sobre como se comunica. A versão mais recente costuma ser chamada de protocolo HTTP *Inteligente* (Smart HTTP) e a maneira mais antiga como HTTP *Burro* (Dumb HTTP). Abordaremos primeiro o novo protocolo Smart HTTP.

Smart HTTP

O Smart HTTP opera de forma muito semelhante aos protocolos SSH ou Git, mas roda sobre portas HTTPS padrão e pode usar vários mecanismos de autenticação HTTP, o que significa que geralmente é mais fácil para o usuário do que algo como o SSH, já que você pode usar coisas como autenticação de nome de usuário/senha em vez de ter que configurar chaves SSH.

Provavelmente se tornou a maneira mais popular de usar o Git agora, já que ele pode ser configurado tanto para servir anonimamente como o protocolo `git://`, quanto para fazer push com autenticação e criptografia como o protocolo SSH. Em vez de ter que configurar URLs diferentes para essas coisas, agora você pode usar uma única URL para ambas. Se você tentar fazer o push e o repositório exigir autenticação (o que normalmente deveria), o servidor pode solicitar um nome de usuário e uma senha. O mesmo vale para o acesso de leitura.

Na verdade, para serviços como o GitHub, a URL que você usa para visualizar o repositório online (por exemplo, https://github.com/schacon/simplegit) é a mesma URL que você pode usar para clonar e, se tiver acesso, para fazer push.

Dumb HTTP

Se o servidor não responder com um serviço inteligente (smart service) HTTP do Git, o cliente Git tentará recorrer (fall back) ao protocolo *Dumb* HTTP, mais simples. O protocolo Dumb espera que o repositório Git nu (bare) seja servido como arquivos normais do servidor da web. A beleza do Dumb HTTP é a simplicidade de configurá-lo. Basicamente, tudo o que você precisa fazer é colocar um repositório Git nu (bare) sob a raiz do documento HTTP e configurar um gancho (hook) específico de `post-update`, e pronto (consulte Hooks do Git). A partir desse ponto, qualquer um que possa acessar o servidor da web no qual você colocou o repositório também poderá cloná-lo. Para permitir acesso de leitura ao seu repositório via HTTP, faça algo assim:

```
$ cd /var/www/htdocs/
```

```
$ git clone --bare /path/to/git_project gitproject.git
$ cd gitproject.git
$ mv hooks/post-update.sample hooks/post-update
$ chmod a+x hooks/post-update
```

Isso é tudo. O hook `post-update` que vem com o Git por padrão executa o comando apropriado (`git update-server-info`) para fazer com que a busca (fetching) e a clonagem HTTP funcionem corretamente. Este comando é executado quando você faz o push para este repositório (via SSH, talvez); então, outras pessoas podem clonar via algo como:

```
$ git clone https://example.com/gitproject.git
```

Neste caso particular, estamos usando o caminho `/var/www/htdocs`, que é comum para configurações do Apache, mas você pode usar qualquer servidor da web estático — basta colocar o repositório nu no seu caminho. Os dados do Git são servidos como arquivos estáticos básicos (consulte o capítulo Git Internals (Por Dentro do Git) para obter detalhes sobre como exatamente ele é servido).

Geralmente, você optaria por executar um servidor Smart HTTP de leitura/gravação ou simplesmente ter os arquivos acessíveis como leitura (read-only) da maneira Dumb. É raro executar uma mistura dos dois serviços.

Os Prós

Nós nos concentraremos nos prós da versão Smart do protocolo HTTP.

A simplicidade de ter uma única URL para todos os tipos de acesso e ter o servidor solicitando autenticação apenas quando ela é necessária torna as coisas muito fáceis para o usuário final. Ser capaz de se autenticar com um nome de usuário e uma senha também é uma grande vantagem sobre o SSH, já que os usuários não precisam gerar chaves SSH localmente e fazer o upload da chave pública deles para o servidor antes de poder interagir com ele. Para usuários menos sofisticados, ou usuários em sistemas em que o SSH é menos comum, essa é uma grande vantagem na usabilidade. Também é um protocolo muito rápido e eficiente, semelhante ao SSH.

Você também pode servir os seus repositórios no modo de leitura (read-only) via HTTPS, o que significa que você pode criptografar a transferência de conteúdo; ou você pode ir tão longe a ponto de fazer os clientes usarem certificados SSL assinados específicos.

Outra coisa boa é que HTTP e HTTPS são protocolos tão comumente usados que os firewalls corporativos costumam ser configurados para permitir o tráfego em suas portas.

Os Contras

O Git via HTTPS pode ser um pouco mais complicado de configurar em comparação com o SSH em alguns servidores. Fora isso, há muito pouca vantagem que outros protocolos tenham sobre o Smart HTTP para servir conteúdo Git.

Se você estiver usando HTTP para fazer o push autenticado, fornecer as suas credenciais às vezes é mais complicado do que usar chaves via SSH. Há, no entanto, várias ferramentas de

armazenamento em cache (caching tools) de credenciais que você pode usar, incluindo o acesso ao Keychain no macOS e o Gerenciador de Credenciais (Credential Manager) no Windows, para tornar isso bem indolor. Leia Armazenamento de Credenciais para ver como configurar o cache de senha HTTP seguro no seu sistema.

O Protocolo SSH

Um protocolo de transporte comum para o Git ao fazer a auto-hospedagem (self-hosting) é sobre SSH. Isso ocorre porque o acesso SSH a servidores já está configurado na maioria dos lugares — e se não estiver, é fácil de fazer. O SSH também é um protocolo de rede autenticado e, por ser onipresente, geralmente é fácil de configurar e usar.

Para clonar um repositório Git via SSH, você pode especificar uma URL `ssh://` como esta:

```
$ git clone ssh://[user@]server/project.git
```

Ou você pode usar a sintaxe mais curta, semelhante a scp, para o protocolo SSH:

```
$ git clone [user@]server:project.git
```

Em ambos os casos acima, se você não especificar o nome de usuário opcional, o Git assume o usuário com o qual você está logado no momento.

Os Prós

Os prós do uso de SSH são muitos. Primeiro, o SSH é relativamente fácil de configurar — os daemons SSH são comuns, muitos administradores de rede têm experiência com eles e muitas distribuições de sistema operacional são configuradas com eles ou têm ferramentas para gerenciá-los. Em seguida, o acesso via SSH é seguro — toda a transferência de dados é criptografada e autenticada. Por último, como os protocolos HTTPS, Git e Local, o SSH é eficiente, tornando os dados os mais compactos possíveis antes de transferi-los.

Os Contras

O aspecto negativo do SSH é que ele não suporta acesso anônimo ao seu repositório Git. Se você estiver usando SSH, as pessoas *devem* ter acesso SSH à sua máquina, mesmo em uma capacidade somente leitura (read-only), o que não torna o SSH propício para projetos de código aberto (open source) nos quais as pessoas podem simplesmente querer clonar o seu repositório para examiná-lo. Se você estiver usando-o apenas na sua rede corporativa, o SSH pode ser o único protocolo com o qual você precise lidar. Se você quiser permitir acesso anônimo somente leitura (read-only) aos seus projetos e também quiser usar SSH, terá que configurar o SSH para você fazer push, mas usar outra coisa para que os outros façam fetch.

O Protocolo Git

Por fim, temos o protocolo Git. Trata-se de um daemon especial que vem empacotado com o Git; ele escuta em uma porta dedicada (9418) que fornece um serviço semelhante ao protocolo SSH, mas

sem nenhuma autenticação ou criptografia. Para que um repositório seja servido por meio do protocolo Git, você deve criar um arquivo `git-daemon-export-ok` — o daemon não servirá um repositório sem que esse arquivo esteja lá — mas, além disso, não há segurança. O repositório Git está disponível para qualquer pessoa clonar ou não está. Isso significa que geralmente não há push por meio deste protocolo. Você pode habilitar o acesso de push, mas, dada a falta de autenticação, qualquer pessoa na internet que encontre a URL do seu projeto pode fazer push para esse projeto. Basta dizer que isso é raro.

Os Prós

O protocolo Git geralmente é o protocolo de transferência de rede mais rápido disponível. Se você estiver servindo muito tráfego para um projeto público ou servindo um projeto muito grande que não requer autenticação de usuário para acesso de leitura (read access), é provável que você queira configurar um daemon do Git para servir o seu projeto. Ele usa o mesmo mecanismo de transferência de dados que o protocolo SSH, mas sem a sobrecarga (overhead) de criptografia e autenticação.

Os Contras

Devido à falta de TLS ou outra criptografia, a clonagem por meio de `git://` pode levar a uma vulnerabilidade de execução arbitrária de código e, portanto, deve ser evitada, a menos que você saiba o que está fazendo.

- Se você executar `git clone git://example.com/project.git`, um invasor que controle, por exemplo, o seu roteador, poderá modificar o repositório que você acabou de clonar, inserindo código malicioso nele. Se você então compilar/executar o código que acabou de clonar, você executará o código malicioso. Executar `git clone http://example.com/project.git` deve ser evitado pelo mesmo motivo.
- Executar `git clone https://example.com/project.git` não sofre do mesmo problema (a menos que o invasor consiga fornecer um certificado TLS para example.com). Executar `git clone git@example.com:project.git` só sofre deste problema se você aceitar uma impressão digital (fingerprint) de chave SSH errada.

Também não possui autenticação, ou seja, qualquer pessoa pode clonar o repositório (embora isso seja frequentemente exatamente o que você deseja). Também é, provavelmente, o protocolo mais difícil de configurar. Ele deve executar o seu próprio daemon, o que requer configuração do `xinetd` ou `systemd` ou algo semelhante, o que nem sempre é um passeio no parque (walk in the park). Também requer acesso de firewall à porta 9418, que não é uma porta padrão que os firewalls corporativos sempre permitem. Atrás de grandes firewalls corporativos, essa porta obscura geralmente é bloqueada.

Colocando o Git em um Servidor

Agora, abordaremos a configuração de um serviço Git executando esses protocolos no seu próprio servidor.



Aqui estaremos demonstrando os comandos e passos necessários para fazer instalações básicas e simplificadas em um servidor baseado em Linux, embora

> também seja possível executar esses serviços em servidores macOS ou Windows. Na verdade, configurar um servidor de produção na sua infraestrutura certamente acarretará diferenças nas medidas de segurança ou nas ferramentas do sistema operacional, mas esperamos que isso lhe dê uma ideia geral do que está envolvido.

Para configurar inicialmente qualquer servidor Git, você deve exportar um repositório existente para um novo repositório nu (bare repository) — um repositório que não contém um diretório de trabalho. Geralmente é simples de fazer. Para clonar o seu repositório a fim de criar um novo repositório nu, você executa o comando clone com a opção `--bare`. Por convenção, os nomes de diretório de repositórios nus terminam com o sufixo `.git`, assim:

```
$ git clone --bare my_project my_project.git
Cloning into bare repository 'my_project.git'...
done.
```

Agora você deve ter uma cópia dos dados do diretório Git no seu diretório `my_project.git`.

Isso é aproximadamente equivalente a algo como:

```
$ cp -Rf my_project/.git my_project.git
```

Existem algumas pequenas diferenças no arquivo de configuração, mas, para o seu propósito, isso é quase a mesma coisa. Ele pega o repositório Git por si só, sem um diretório de trabalho, e cria um diretório especificamente apenas para ele.

Colocando o Repositório Nu (Bare) em um Servidor

Agora que você tem uma cópia nua do seu repositório, tudo o que você precisa fazer é colocá-la em um servidor e configurar os seus protocolos. Digamos que você configurou um servidor chamado `git.example.com` ao qual você tem acesso SSH e deseja armazenar todos os seus repositórios Git no diretório `/srv/git`. Supondo que `/srv/git` exista nesse servidor, você pode configurar o seu novo repositório copiando o seu repositório nu:

```
$ scp -r my_project.git user@git.example.com:/srv/git
```

Neste ponto, outros usuários que tenham acesso de leitura baseado em SSH ao diretório `/srv/git` naquele servidor podem clonar o seu repositório executando:

```
$ git clone user@git.example.com:/srv/git/my_project.git
```

Se um usuário fizer SSH em um servidor e tiver acesso de gravação ao diretório `/srv/git/my_project.git`, ele também terá acesso de push automaticamente.

O Git adicionará automaticamente permissões de gravação de grupo a um repositório adequadamente se você executar o comando `git init` com a opção `--shared`. Observe que, ao

executar este comando, você não destruirá nenhum commit, referência, etc. no processo.

```
$ ssh user@git.example.com
$ cd /srv/git/my_project.git
$ git init --bare --shared
```

Você vê como é fácil pegar um repositório Git, criar uma versão nua e colocá-lo em um servidor ao qual você e os seus colaboradores tenham acesso SSH. Agora você está pronto para colaborar no mesmo projeto.

É importante notar que isso é literalmente tudo o que você precisa fazer para executar um servidor Git útil ao qual várias pessoas tenham acesso — basta adicionar contas com acesso SSH em um servidor e colocar um repositório nu em algum lugar onde todos esses usuários tenham acesso de leitura e gravação. Você está pronto para começar — nada mais é necessário.

Nas próximas seções, você verá como expandir para configurações mais sofisticadas. Essa discussão incluirá não ter que criar contas de usuário para cada usuário, adicionar acesso público de leitura a repositórios, configurar IUs da web e muito mais. No entanto, tenha em mente que para colaborar com algumas pessoas em um projeto privado, tudo o que você *precisa* é de um servidor SSH e um repositório nu.

Pequenas Configurações

Se você é uma pequena equipe ou está apenas experimentando o Git na sua organização e tem apenas alguns desenvolvedores, as coisas podem ser simples para você. Um dos aspectos mais complicados da configuração de um servidor Git é o gerenciamento de usuários. Se você quiser que alguns repositórios sejam somente leitura para determinados usuários e leitura/gravação para outros, o acesso e as permissões podem ser um pouco mais difíceis de organizar.

Acesso SSH

Se você tem um servidor ao qual todos os seus desenvolvedores já têm acesso SSH, geralmente é mais fácil configurar o seu primeiro repositório lá, porque você não tem quase nenhum trabalho (como abordamos na última seção). Se você quiser permissões de tipo de controle de acesso mais complexas nos seus repositórios, pode lidar com elas com as permissões normais do sistema de arquivos do sistema operacional do seu servidor.

Se você quiser colocar os seus repositórios em um servidor que não tenha contas para todos na sua equipe para quem você deseja conceder acesso de gravação, você deve configurar o acesso SSH para eles. Nós assumimos que se você tiver um servidor para fazer isso, você já tem um servidor SSH instalado, e é assim que você está acessando o servidor.

Existem algumas maneiras de dar acesso a todos na sua equipe. A primeira é configurar contas para todos, o que é simples, mas pode ser trabalhoso. Você pode não querer executar `adduser` (ou a possível alternativa `useradd`) e ter que definir senhas temporárias para cada novo usuário.

Um segundo método é criar uma única conta de usuário 'git' na máquina, pedir a todos os usuários que terão acesso de gravação que lhe enviem uma chave pública SSH e adicionar essa chave ao arquivo `~/.ssh/authorized_keys` dessa nova conta 'git'. Nesse ponto, todos poderão acessar essa

máquina através da conta 'git'. Isso não afeta os dados do commit de forma alguma — o usuário SSH como o qual você se conecta não afeta os commits que você gravou.

Outra maneira de fazer isso é fazer com que o seu servidor SSH se autentique a partir de um servidor LDAP ou alguma outra fonte de autenticação centralizada que você já possa ter configurado. Desde que cada usuário possa obter acesso shell na máquina, qualquer mecanismo de autenticação SSH que você imaginar deve funcionar.

Gerando a sua Chave Pública SSH

Muitos servidores Git se autenticam usando chaves públicas SSH. Para fornecer uma chave pública, cada usuário no seu sistema deve gerar uma se ainda não tiver. Este processo é semelhante em todos os sistemas operacionais. Primeiro, você deve verificar se já não tem uma chave. Por padrão, as chaves SSH de um usuário são armazenadas no diretório `~/.ssh` desse usuário. Você pode verificar facilmente se já tem uma chave acessando esse diretório e listando o conteúdo:

```
$ cd ~/.ssh
$ ls
authorized_keys2  id_dsa       known_hosts
config            id_dsa.pub
```

Você está procurando um par de arquivos chamados algo como `id_dsa` ou `id_rsa` e um arquivo correspondente com a extensão `.pub`. O arquivo `.pub` é a sua chave pública, e o outro arquivo é a chave privada correspondente. Se você não tiver esses arquivos (ou nem mesmo tiver um diretório `.ssh`), pode criá-los executando um programa chamado `ssh-keygen`, que é fornecido com o pacote SSH nos sistemas Linux/macOS e vem com o Git para Windows:

```
$ ssh-keygen -o
Generating public/private rsa key pair.
Enter file in which to save the key (/home/schacon/.ssh/id_rsa):
Created directory '/home/schacon/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/schacon/.ssh/id_rsa.
Your public key has been saved in /home/schacon/.ssh/id_rsa.pub.
The key fingerprint is:
d0:82:24:8e:d7:f1:bb:9b:33:53:96:93:49:da:9b:e3 schacon@mylaptop.local
```

Primeiro ele confirma onde você quer salvar a chave (`.ssh/id_rsa`), e então pede duas vezes uma senha (passphrase), que você pode deixar em branco se não quiser digitar uma senha ao usar a chave. No entanto, se você usar uma senha, certifique-se de adicionar a opção `-o`; ela salva a chave privada num formato que é mais resistente à quebra de senhas por força bruta do que o formato padrão. Você também pode usar a ferramenta `ssh-agent` para evitar ter de digitar a senha toda vez.

Agora, cada usuário que faz isso precisa enviar a sua chave pública para você ou para quem estiver administrando o servidor Git (presumindo que você esteja usando uma configuração de servidor SSH que requer chaves públicas). Tudo o que eles precisam fazer é copiar o conteúdo do arquivo

`.pub` e enviá-lo por e-mail. As chaves públicas se parecem com algo assim:

```
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAklOUpkDHrfHY17SbrmTIpNLTGK9Tjom/BWDSU
GPl+nafzlHDTYW7hdI4yZ5ew18JH4JW9jbhUFrviQzM7xlELEVf4h9lFX5QVkbPppSwg0cda3
Pbv7kOdJ/MTyBlWXFCR+HAo3FXRitBqxiX1nKhXpHAZsMciLq8V6RjsNAQwdsdMFvSlVK/7XA
t3FaoJoAsncM1Q9x5+3V0Ww68/eIFmb1zuUFljQJKprrX88XypNDvjYNby6vw/Pb0rwert/En
mZ+AW4OZPnTPI89ZPmVMLuayrD2cE86Z/il8b+gw3r3+1nKatmIkjn2so1d01QraTlMqVSsbx
NrRFi9wrf+M7Q== schacon@mylaptop.local
```

Para um tutorial mais detalhado sobre como criar uma chave SSH em vários sistemas operacionais, veja o guia do GitHub sobre chaves SSH em https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent.

Configurando o Servidor

Vamos percorrer a configuração do acesso SSH no lado do servidor. Neste exemplo, você usará o método `authorized_keys` para autenticar os seus usuários. Também presumimos que você esteja executando uma distribuição Linux padrão como o Ubuntu.



Grande parte do que é descrito aqui pode ser automatizado usando o comando `ssh-copy-id`, em vez de copiar e instalar manualmente chaves públicas.

Primeiro, você cria uma conta de usuário `git` e um diretório `.ssh` para esse usuário.

```
$ sudo adduser git
$ su git
$ cd
$ mkdir .ssh && chmod 700 .ssh
$ touch .ssh/authorized_keys && chmod 600 .ssh/authorized_keys
```

Em seguida, você precisa adicionar algumas chaves públicas SSH de desenvolvedor ao arquivo `authorized_keys` para o usuário `git`. Vamos supor que você tenha algumas chaves públicas confiáveis e as tenha salvo em arquivos temporários. Novamente, as chaves públicas se parecem com algo assim:

```
$ cat /tmp/id_rsa.john.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCB007n/ww+ouN4gSLKssMxXnBOvf9LGt4L
ojG6rs6hPB09j9R/T17/x4lhJA0F3FR1rP6kYBRsWj2aThGw6HXLm9/5zytK6Ztg3RPKK+4k
Yjh6541NYsnEAZuXz0jTTyAUfrtU3Z5E003C4oxOj6H0rfIF1kKI9MAQLMdpGW1GYEIgS9Ez
Sdfd8AcCIicTDWbqLAcU4UpkaX8KyGlLwsNuuGztobF8m72ALC/nLF6JLtPofwFBlgc+myiv
O7TCUSBdLQlgMVOFq1I2uPWQOkOWQAHukEOmfjy2jctxSDBQ220ymjaNsHT4kgtZg2AYYgPq
dAv8JggJICUvax2T9va5 gsg-keypair
```

Basta anexá-las ao arquivo `authorized_keys` do usuário `git` em seu diretório `.ssh`:

```
$ cat /tmp/id_rsa.john.pub >> ~/.ssh/authorized_keys
$ cat /tmp/id_rsa.josie.pub >> ~/.ssh/authorized_keys
$ cat /tmp/id_rsa.jessica.pub >> ~/.ssh/authorized_keys
```

Agora, você pode configurar um repositório vazio para eles executando `git init` com a opção `--bare`, que inicializa o repositório sem um diretório de trabalho:

```
$ cd /srv/git
$ mkdir project.git
$ cd project.git
$ git init --bare
Initialized empty Git repository in /srv/git/project.git/
```

Então, John, Josie ou Jessica podem fazer o push da primeira versão do projeto deles para esse repositório, adicionando-o como remoto e fazendo o push de um branch. Observe que alguém deve acessar o shell da máquina e criar um repositório nu toda vez que você quiser adicionar um projeto. Vamos usar `gitserver` como o hostname do servidor no qual você configurou o seu usuário `git` e repositório. Se você o estiver executando internamente e configurar o DNS do `gitserver` para apontar para esse servidor, poderá usar os comandos da mesma forma (assumindo que `myproject` é um projeto existente com arquivos nele):

```
# on John's computer
$ cd myproject
$ git init
$ git add .
$ git commit -m 'Initial commit'
$ git remote add origin git@gitserver:/srv/git/project.git
$ git push origin master
```

Neste ponto, os outros podem clonar e fazer o push das alterações de volta com a mesma facilidade:

```
$ git clone git@gitserver:/srv/git/project.git
$ cd project
$ vim README
$ git commit -am 'Fix for README file'
$ git push origin master
```

Com este método, você pode colocar um servidor Git de leitura/gravação em funcionamento rapidamente para um punhado de desenvolvedores.

Você deve notar que atualmente todos esses usuários também podem logar no servidor e obter um shell como o usuário `git`. Se você quiser restringir isso, terá que alterar o shell para outra coisa no arquivo `/etc/passwd`.

Você pode restringir facilmente a conta de usuário `git` a apenas atividades relacionadas ao Git com

uma ferramenta de shell limitado chamada `git-shell` que vem com o Git. Se você definir isso como o login shell da conta de usuário `git`, então essa conta não poderá ter acesso de shell normal ao seu servidor. Para usar isso, especifique `git-shell` em vez de `bash` ou `csh` para o login shell dessa conta. Para fazer isso, você deve primeiro adicionar o nome do caminho completo (pathname) do comando `git-shell` em `/etc/shells`, se ele ainda não estiver lá:

```
$ cat /etc/shells   # see if git-shell is already in there. If not...
$ which git-shell   # make sure git-shell is installed on your system.
$ sudo -e /etc/shells  # and add the path to git-shell from last command
```

Agora você pode editar o shell para um usuário usando `chsh <username> -s <shell>`:

```
$ sudo chsh git -s $(which git-shell)
```

Agora, o usuário `git` ainda pode usar a conexão SSH para fazer o push e o pull de repositórios Git, mas não pode usar o shell na máquina. Se você tentar, verá uma rejeição de login como esta:

```
$ ssh git@gitserver
fatal: Interactive git shell is not enabled.
hint: ~/git-shell-commands should exist and have read and execute access.
Connection to gitserver closed.
```

Neste ponto, os usuários ainda conseguem usar o encaminhamento de porta SSH para acessar qualquer host que o servidor git consiga alcançar. Se você quiser evitar isso, pode editar o arquivo `authorized_keys` e adicionar as seguintes opções antes de cada chave que gostaria de restringir:

```
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty
```

O resultado deve ficar assim:

```
$ cat ~/.ssh/authorized_keys
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQCB007n/ww+ouN4gSLKssMxXnBOvf9LGt4LojG6rs6h
PB09j9R/T17/x4lhJA0F3FR1rP6kYBRsWj2aThGw6HXLm9/5zytK6Ztg3RPKK+4kYjh6541N
YsnEAZuXz0jTTyAUfrtU3Z5E003C4oxOj6H0rfIF1kKI9MAQLMdpGW1GYEIgS9EzSdfd8AcC
IicTDWbqLAcU4UpkaX8KyGlLwsNuuGztobF8m72ALC/nLF6JLtPofwFBlgc+myivO7TCUSBd
LQlgMVOFq1I2uPWQOkOWQAHukEOmfjy2jctxSDBQ220ymjaNsHT4kgtZg2AYYgPqdAv8JggJ
ICUvax2T9va5 gsg-keypair

no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABAQDEwENNMomTboYI+LJieaAY16qiXiH3wuvENhBG...
```

Agora, os comandos de rede do Git ainda funcionarão perfeitamente, mas os usuários não conseguirão obter um shell. Como a saída (output) indica, você também pode configurar um

diretório no diretório inicial (home) do usuário `git` que personaliza um pouco o comando `git-shell`. Por exemplo, você pode restringir os comandos Git que o servidor aceitará ou pode personalizar a mensagem que os usuários veem se tentarem fazer SSH dessa forma. Execute `git help shell` para obter mais informações sobre a personalização do shell.

Git Daemon

Em seguida, configuraremos um daemon para servir repositórios usando o protocolo "Git". Esta é uma escolha comum para acesso rápido e não autenticado aos seus dados Git. Lembre-se de que, como este não é um serviço autenticado, qualquer coisa que você sirva através deste protocolo será pública dentro da sua rede.

Se você estiver executando isso em um servidor fora do seu firewall, deve ser usado apenas para projetos que são publicamente visíveis para o mundo. Se o servidor em que você o executa estiver dentro do seu firewall, você pode usá-lo para projetos aos quais um grande número de pessoas ou computadores (servidores de integração contínua ou de build) tem acesso de leitura, quando você não quer ter que adicionar uma chave SSH para cada um.

De qualquer forma, o protocolo Git é relativamente fácil de configurar. Basicamente, você precisa executar este comando de maneira daemonizada (em segundo plano):

```
$ git daemon --reuseaddr --base-path=/srv/git/ /srv/git/
```

A opção `--reuseaddr` permite que o servidor reinicie sem esperar que as conexões antigas atinjam o tempo limite, enquanto a opção `--base-path` permite que as pessoas clonem projetos sem especificar o caminho inteiro, e o caminho no final informa ao Git daemon onde procurar os repositórios a serem exportados. Se você estiver executando um firewall, também precisará abrir um buraco (punch a hole) nele na porta 9418 da máquina em que está configurando isso.

Você pode daemonizar esse processo de várias maneiras, dependendo do sistema operacional que estiver executando.

Como o `systemd` é o sistema init mais comum entre as distribuições Linux modernas, você pode usá-lo para esse fim. Basta colocar um arquivo em `/etc/systemd/system/git-daemon.service` com este conteúdo:

```
[Unit]
Description=Start Git Daemon

[Service]
ExecStart=/usr/bin/git daemon --reuseaddr --base-path=/srv/git/ /srv/git/

Restart=always
RestartSec=500ms

StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=git-daemon
```

```
User=git
Group=git

[Install]
WantedBy=multi-user.target
```

Você deve ter notado que o Git daemon é iniciado aqui com `git` como grupo e usuário. Modifique-o para atender às suas necessidades e certifique-se de que o usuário fornecido exista no sistema. Além disso, verifique se o binário do Git está realmente localizado em `/usr/bin/git` e altere o caminho se necessário.

Finalmente, você executará `systemctl enable git-daemon` para iniciar o serviço automaticamente na inicialização, e pode iniciar e parar o serviço com, respectivamente, `systemctl start git-daemon` e `systemctl stop git-daemon`.

Em outros sistemas, você pode querer usar `xinetd`, um script no seu sistema `sysvinit`, ou alguma outra coisa — desde que você consiga que esse comando seja daemonizado e monitorado de alguma forma.

Em seguida, você precisa informar ao Git a quais repositórios ele deve permitir o acesso não autenticado baseado no servidor Git. Você pode fazer isso em cada repositório criando um arquivo chamado `git-daemon-export-ok`.

```
$ cd /path/to/project.git
$ touch git-daemon-export-ok
```

A presença desse arquivo informa ao Git que não há problema em servir este projeto sem autenticação.

Smart HTTP

Agora temos acesso autenticado via SSH e acesso não autenticado via `git://`, mas também há um protocolo que pode fazer as duas coisas ao mesmo tempo. Configurar o Smart HTTP é basicamente habilitar um script CGI que é fornecido com o Git chamado `git-http-backend` no servidor. Este CGI lerá o caminho e os cabeçalhos enviados por um `git fetch` ou `git push` para uma URL HTTP e determinará se o cliente pode se comunicar por HTTP (o que é verdadeiro para qualquer cliente desde a versão 1.6.6). Se o CGI vir que o cliente é inteligente (smart), ele se comunicará inteligentemente com ele; caso contrário, recorrerá (fall back) ao comportamento burro (dumb behavior) (portanto, é retrocompatível para leituras com clientes mais antigos).

Vamos analisar uma configuração muito básica. Vamos configurar isso com o Apache como servidor CGI. Se você não tem o Apache configurado, você pode fazê-lo em uma máquina Linux com algo assim:

```
$ sudo apt-get install apache2 apache2-utils
```

```
$ a2enmod cgi alias env
```

Isso também habilita os módulos `mod_cgi`, `mod_alias` e `mod_env`, que são todos necessários para que isso funcione corretamente.

Você também precisará definir o grupo de usuários Unix dos diretórios `/srv/git` para `www-data` para que o seu servidor web possa ter acesso de leitura e gravação aos repositórios, porque a instância do Apache executando o script CGI estará (por padrão) sendo executada como aquele usuário:

```
$ chgrp -R www-data /srv/git
```

Em seguida, precisamos adicionar algumas coisas à configuração do Apache para executar o `git-http-backend` como o manipulador (handler) de qualquer coisa que chegue ao caminho `/git` do seu servidor web.

```
SetEnv GIT_PROJECT_ROOT /srv/git
SetEnv GIT_HTTP_EXPORT_ALL
ScriptAlias /git/ /usr/lib/git-core/git-http-backend/
```

Se você omitir a variável de ambiente `GIT_HTTP_EXPORT_ALL`, o Git servirá aos clientes não autenticados apenas os repositórios com o arquivo `git-daemon-export-ok` neles, assim como o Git daemon fez.

Finalmente, você vai querer dizer ao Apache para permitir requisições ao `git-http-backend` e fazer com que as gravações sejam autenticadas de alguma forma, possivelmente com um bloco Auth como este:

```
<Files "git-http-backend">
    AuthType Basic
    AuthName "Git Access"
    AuthUserFile /srv/git/.htpasswd
    Require expr !(%{QUERY_STRING} -strmatch '*service=git-receive-pack*' ||
%{REQUEST_URI} =~ m#/git-receive-pack$#)
    Require valid-user
</Files>
```

Isso exigirá que você crie um arquivo `.htpasswd` contendo as senhas de todos os usuários válidos. Aqui está um exemplo de adição de um usuário “schacon” ao arquivo:

```
$ htpasswd -c /srv/git/.htpasswd schacon
```

Há inúmeras maneiras de fazer o Apache autenticar usuários, você terá que escolher e implementar uma delas. Este é apenas o exemplo mais simples que pudemos imaginar. Você quase certamente também vai querer configurar isso com SSL para que todos esses dados sejam criptografados.

Não queremos nos aprofundar muito na toca do coelho (rabbit hole) dos detalhes da configuração do Apache, pois você pode muito bem estar usando um servidor diferente ou ter necessidades de autenticação diferentes. A ideia é que o Git vem com um CGI chamado `git-http-backend` que, quando invocado, fará toda a negociação para enviar e receber dados via HTTP. Ele não implementa nenhuma autenticação em si, mas isso pode ser facilmente controlado na camada do servidor web que o invoca. Você pode fazer isso com quase qualquer servidor web com capacidade para CGI, então escolha o que você conhece melhor.



Para mais informações sobre a configuração de autenticação no Apache, confira a documentação do Apache aqui: https://httpd.apache.org/docs/current/howto/auth.html.

GitWeb

Agora que você tem acesso básico de leitura/gravação e somente leitura ao seu projeto, pode querer configurar um visualizador simples baseado na web. O Git vem com um script CGI chamado GitWeb que às vezes é usado para isso.

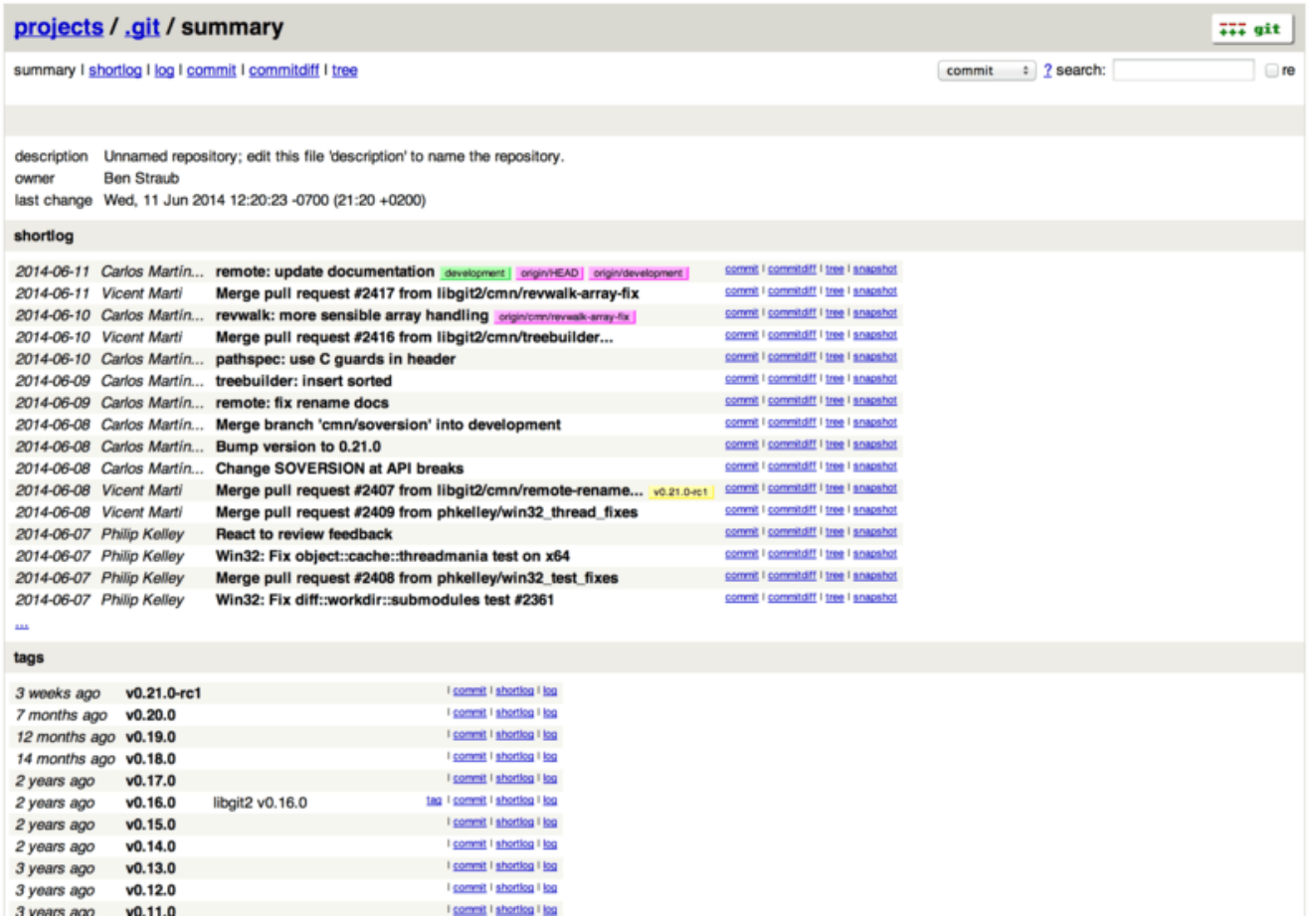


Figure 49. A interface de usuário baseada na web GitWeb

Se você quiser conferir como o GitWeb ficaria no seu projeto, o Git vem com um comando para acionar uma instância temporária se você tiver um servidor web leve (lightweight) no seu sistema, como `lighttpd` ou `webrick`. Em máquinas Linux, o `lighttpd` costuma estar instalado, então você pode conseguir executá-lo digitando `git instaweb` no diretório do seu projeto. Se você estiver executando o macOS, o Leopard vem pré-instalado com Ruby, então o `webrick` pode ser a sua melhor aposta.

Para iniciar o `instaweb` com um manipulador não-lighttpd, você pode executá-lo com a opção `--httpd`.

```
$ git instaweb --httpd=webrick
[2009-02-21 10:02:21] INFO  WEBrick 1.3.1
[2009-02-21 10:02:21] INFO  ruby 1.8.6 (2008-03-03) [universal-darwin9.0]
```

Isso inicia um servidor HTTPD na porta 1234 e então inicia automaticamente um navegador da web que abre nessa página. É muito fácil da sua parte. Quando terminar e quiser desligar o servidor, você pode executar o mesmo comando com a opção `--stop`:

```
$ git instaweb --httpd=webrick --stop
```

Se você quiser executar a interface da web em um servidor o tempo todo para a sua equipe ou para um projeto de código aberto que você está hospedando, precisará configurar o script CGI para ser servido pelo seu servidor web normal. Algumas distribuições Linux têm um pacote `gitweb` que você pode conseguir instalar via `apt` ou `dnf`, então você pode querer tentar isso primeiro. Veremos como instalar o GitWeb manualmente de forma muito rápida. Primeiro, você precisa obter o código-fonte do Git, que vem com o GitWeb, e gerar o script CGI personalizado:

```
$ git clone https://git.kernel.org/pub/scm/git/git.git
$ cd git/
$ make GITWEB_PROJECTROOT="/srv/git" prefix=/usr gitweb
    SUBDIR gitweb
    SUBDIR ../
make[2]: `GIT-VERSION-FILE' is up to date.
    GEN gitweb.cgi
    GEN static/gitweb.js
$ sudo cp -Rf gitweb /var/www/
```

Observe que você deve dizer ao comando onde encontrar os seus repositórios Git com a variável `GITWEB_PROJECTROOT`. Agora, você precisa fazer com que o Apache use CGI para esse script, para o qual você pode adicionar um VirtualHost:

```
<VirtualHost *:80>
    ServerName gitserver
    DocumentRoot /var/www/gitweb
    <Directory /var/www/gitweb>
        Options +ExecCGI +FollowSymLinks +SymLinksIfOwnerMatch
        AllowOverride All
        order allow,deny
        Allow from all
        AddHandler cgi-script cgi
        DirectoryIndex gitweb.cgi
    </Directory>
```

```
</VirtualHost>
```

Novamente, o GitWeb pode ser servido com qualquer servidor web capaz de rodar CGI ou Perl; se você preferir usar outra coisa, não deve ser difícil de configurar. Neste ponto, você deve conseguir visitar `http://gitserver/` para visualizar os seus repositórios online.

GitLab

O GitWeb é bem simplista, no entanto. Se você está procurando um servidor Git moderno e completo, há várias soluções de código aberto que você pode instalar. Como o GitLab é um dos mais populares, vamos abordar a sua instalação e uso como exemplo. Isso é mais difícil que a opção GitWeb e exigirá mais manutenção, mas é uma opção completa.

Instalação

O GitLab é uma aplicação web com suporte a banco de dados (database-backed), então a sua instalação é mais complexa do que a de alguns outros servidores Git. Felizmente, esse processo é bem documentado e suportado. O GitLab recomenda fortemente a instalação do GitLab no seu servidor através do pacote oficial Omnibus GitLab.

As outras opções de instalação são:

- Gráfico (chart) GitLab Helm, para uso com o Kubernetes.
- Pacotes GitLab em Docker para uso com o Docker.
- A partir dos arquivos fonte (source files).
- Provedores de nuvem como AWS, Google Cloud Platform, Azure, OpenShift e Digital Ocean.

Para mais informações, leia o leia-me (readme) do GitLab Community Edition (CE).

Administração

A interface de administração do GitLab é acessada pela web. Basta apontar o seu navegador para o hostname ou endereço IP onde o GitLab está instalado e fazer o login como o usuário `root`. A senha dependerá do seu tipo de instalação, mas por padrão, o Omnibus GitLab gera automaticamente uma senha e a armazena em /etc/gitlab/initial_root_password por pelo menos 24 horas. Siga a documentação para mais detalhes. Depois de logar, clique no ícone “Admin area” (área de Administração) no menu superior direito.

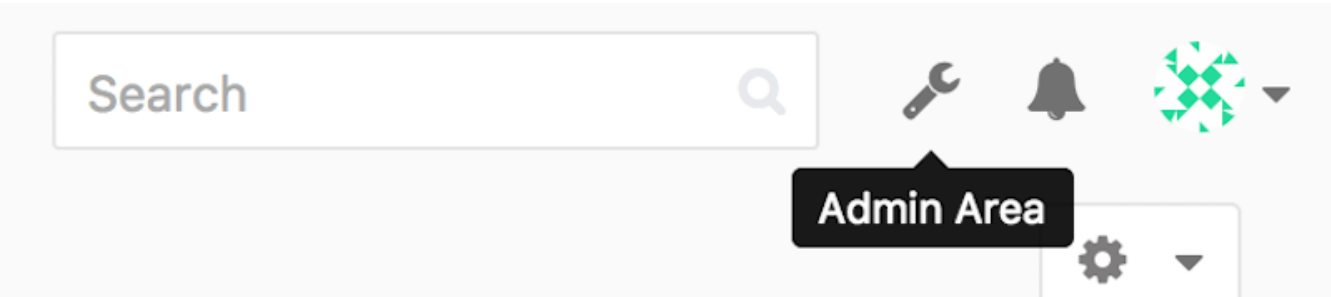


Figure 50. O item “Admin area” no menu do GitLab

Usuários

Todo mundo que usa o seu servidor GitLab deve ter uma conta de usuário. As contas de usuário são bem simples, elas contêm principalmente informações pessoais anexadas aos dados de login. Cada conta de usuário tem um **namespace**, que é um agrupamento lógico de projetos que pertencem a esse usuário. Se a usuária jane tivesse um projeto chamado project, a URL desse projeto seria `http://server/jane/project`.

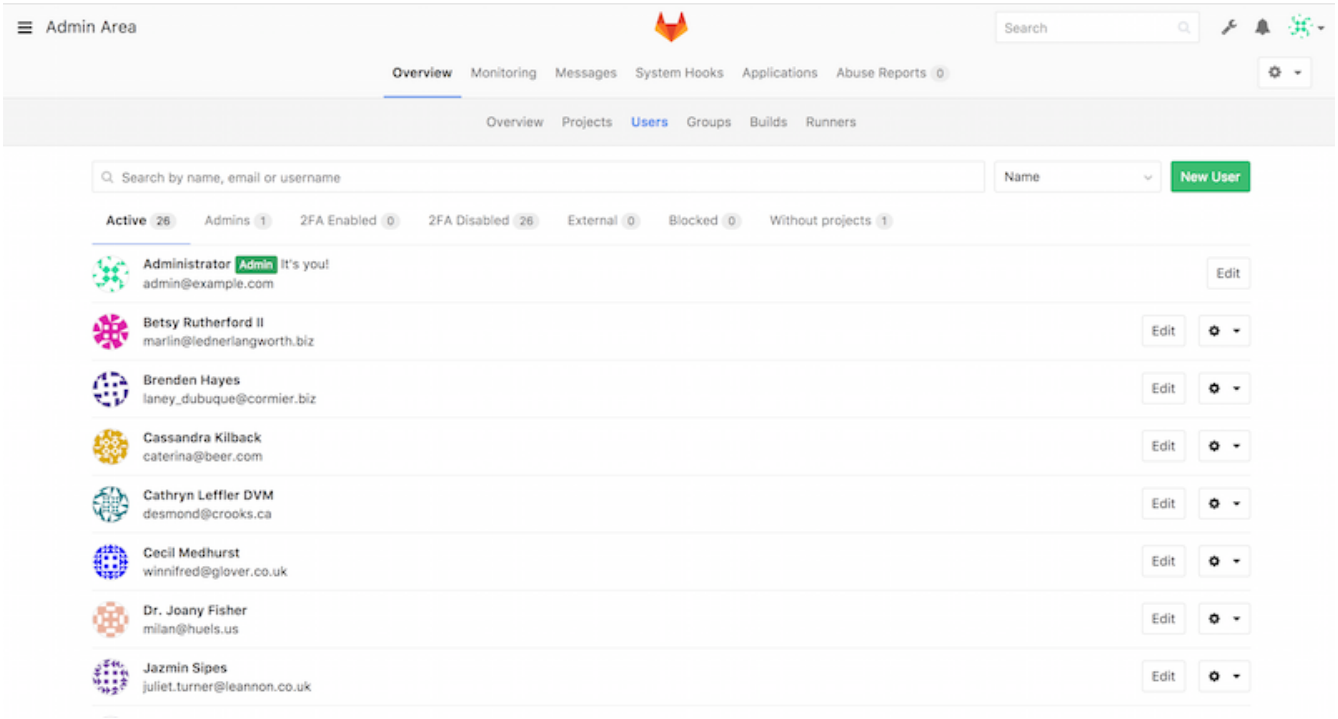


Figure 51. A tela de administração de usuários do GitLab

Você pode remover uma conta de usuário de duas maneiras: “Bloquear” (Blocking) um usuário o impede de fazer login na instância do GitLab, mas todos os dados sob o namespace daquele usuário serão preservados, e os commits assinados com o endereço de e-mail daquele usuário ainda estarão vinculados ao seu perfil.

“Destruir” (Destroying) um usuário, por outro lado, o remove completamente do banco de dados e do sistema de arquivos. Todos os projetos e dados no namespace dele são removidos, e quaisquer grupos que ele possua também serão removidos. Esta é obviamente uma ação muito mais permanente e destrutiva, e você raramente precisará dela.

Grupos

Um grupo do GitLab é uma coleção de projetos, juntamente com dados sobre como os usuários podem acessar esses projetos. Cada grupo tem um namespace de projeto (da mesma forma que os usuários), então se o grupo training tem um projeto materials, a sua URL seria `http://server/training/materials`.

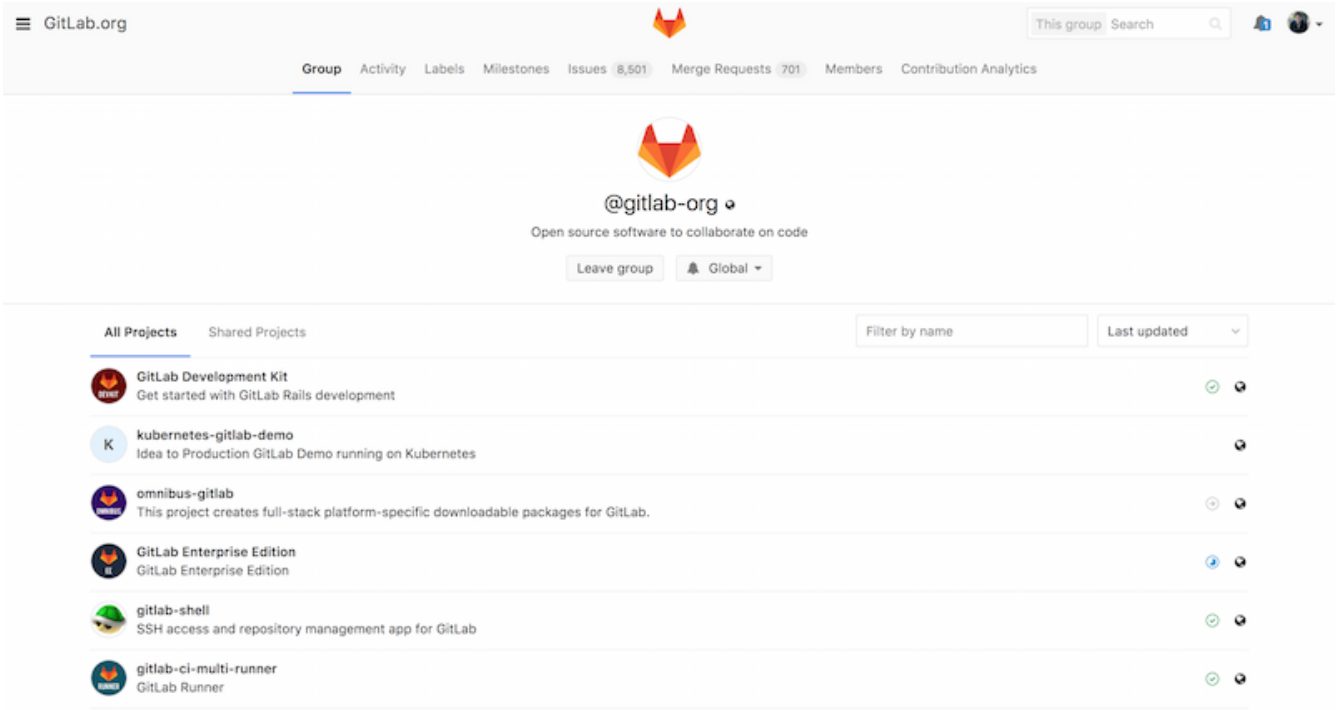


Figure 52. A tela de administração de grupos do GitLab

Cada grupo está associado a um número de usuários, cada um dos quais tem um nível de permissões para os projetos do grupo e para o próprio grupo. Estes variam de "Convidado" (Guest - apenas issues e chat) a "Proprietário" (Owner - controle total do grupo, seus membros e seus projetos). Os tipos de permissões são numerosos demais para serem listados aqui, mas o GitLab tem um link útil na tela de administração.

Projetos

Um projeto do GitLab corresponde aproximadamente a um único repositório Git. Cada projeto pertence a um único namespace, seja de um usuário ou de um grupo. Se o projeto pertencer a um usuário, o proprietário do projeto tem controle direto sobre quem tem acesso ao projeto; se o projeto pertencer a um grupo, as permissões de nível de usuário do grupo entrarão em vigor.

Cada projeto tem um nível de visibilidade, que controla quem tem acesso de leitura às páginas e ao repositório desse projeto. Se um projeto for *Privado*, o proprietário do projeto deve conceder acesso explicitamente a usuários específicos. Um projeto *Interno* é visível para qualquer usuário logado, e um projeto *Público* é visível para qualquer pessoa. Observe que isso controla tanto o acesso `git fetch` quanto o acesso à UI web para esse projeto.

Ganchos (Hooks)

O GitLab inclui suporte para ganchos (hooks), tanto no nível de projeto quanto de sistema. Para qualquer um desses, o servidor GitLab executará um HTTP POST com algum JSON descritivo sempre que eventos relevantes ocorrerem. Essa é uma ótima maneira de conectar os seus repositórios Git e a instância do GitLab ao resto da sua automação de desenvolvimento, como servidores de CI, salas de bate-papo (chat rooms) ou ferramentas de implantação (deployment).

Uso Básico

A primeira coisa que você vai querer fazer com o GitLab é criar um novo projeto. Você pode fazer

isso clicando no ícone “+” na barra de ferramentas. Você será solicitado a informar o nome do projeto, a qual namespace ele deve pertencer e qual deve ser o seu nível de visibilidade. A maior parte do que você especifica aqui não é permanente e pode ser alterada posteriormente através da interface de configurações. Clique em “Criar Projeto” (Create Project), e você está pronto.

Depois que o projeto existir, você provavelmente desejará conectá-lo a um repositório Git local. Cada projeto é acessível via HTTPS ou SSH, qualquer um dos quais pode ser usado para configurar um remoto Git. As URLs são visíveis no topo da página inicial do projeto. Para um repositório local existente, este comando criará um remoto chamado `gitlab` para o local hospedado:

```
$ git remote add gitlab https://server/namespace/project.git
```

Se você não tiver uma cópia local do repositório, pode simplesmente fazer isto:

```
$ git clone https://server/namespace/project.git
```

A interface web fornece acesso a várias visualizações úteis do próprio repositório. A página inicial de cada projeto mostra a atividade recente, e os links na parte superior o levarão a visualizações dos arquivos e do log de commits do projeto.

Trabalhando Juntos

A maneira mais simples de trabalhar junto em um projeto do GitLab é dando a cada usuário acesso direto de push ao repositório Git. Você pode adicionar um usuário a um projeto acessando a seção “Membros” (Members) das configurações desse projeto e associando o novo usuário a um nível de acesso (os diferentes níveis de acesso são discutidos um pouco em Grupos). Ao dar a um usuário um nível de acesso “Desenvolvedor” (Developer) ou superior, esse usuário pode fazer o push de commits e branches diretamente para o repositório.

Outra forma de colaboração, mais desacoplada, é usar merge requests (solicitações de mesclagem). Esse recurso permite que qualquer usuário que possa ver um projeto contribua com ele de forma controlada. Usuários com acesso direto podem simplesmente criar um branch, fazer o push de commits para ele e abrir um merge request de seu branch de volta para a `master` ou qualquer outro branch. Os usuários que não têm permissão de push para um repositório podem fazer o “fork” dele para criar sua própria cópia, fazer o push de commits para a cópia *deles* e abrir um merge request de seu fork de volta para o projeto principal. Esse modelo permite que o proprietário tenha controle total sobre o que entra no repositório e quando, ao mesmo tempo em que permite contribuições de usuários não confiáveis.

Os merge requests e os issues (problemas/tarefas) são as principais unidades de discussão de longa duração no GitLab. Cada merge request permite uma discussão linha por linha da alteração proposta (o que suporta um tipo leve de revisão de código), bem como um tópico de discussão geral (discussion thread). Ambos podem ser atribuídos a usuários ou organizados em marcos (milestones).

Esta seção é focada principalmente nos recursos do GitLab relacionados ao Git, mas como um projeto maduro, ele fornece muitos outros recursos para ajudar a sua equipe a trabalhar em

conjunto, como wikis de projeto e ferramentas de manutenção do sistema. Um benefício do GitLab é que, uma vez que o servidor esteja configurado e funcionando, você raramente precisará ajustar um arquivo de configuração ou acessar o servidor via SSH; a maior parte da administração e do uso geral pode ser feita por meio da interface no navegador.

Opções de Hospedagem de Terceiros

Se você não quiser passar por todo o trabalho de configurar o seu próprio servidor Git, você tem várias opções para hospedar os seus projetos Git em um site de hospedagem externo dedicado. Fazer isso oferece uma série de vantagens: um site de hospedagem geralmente é rápido de configurar e fácil para iniciar projetos, e nenhuma manutenção ou monitoramento de servidor está envolvido. Mesmo se você configurar e executar o seu próprio servidor internamente, ainda pode querer usar um site de hospedagem público para o seu código de fonte aberto (open source) — geralmente é mais fácil para a comunidade de código aberto encontrar e ajudá-lo.

Atualmente, você tem um grande número de opções de hospedagem para escolher, cada uma com diferentes vantagens e desvantagens. Para ver uma lista atualizada, confira a página GitHosting na wiki principal do Git em https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/GitHosting.html.

Abordaremos o uso do GitHub em detalhes no GitHub, pois é o maior host Git existente e você pode precisar interagir com projetos hospedados nele de qualquer forma, mas há dezenas de outros para escolher se você não quiser configurar o seu próprio servidor Git.

Resumo

Você tem várias opções para colocar um repositório Git remoto em funcionamento para poder colaborar com outras pessoas ou compartilhar o seu trabalho.

Executar o seu próprio servidor lhe dá muito controle e permite que você execute o servidor dentro do seu próprio firewall, mas um servidor assim geralmente requer uma boa quantidade do seu tempo para configurar e manter. Se você colocar os seus dados em um servidor hospedado, é fácil configurar e manter; no entanto, você precisa ser capaz de manter o seu código nos servidores de outra pessoa, e algumas organizações não permitem isso.

Deve ser bastante simples determinar qual solução ou combinação de soluções é apropriada para você e a sua organização.

Git Distribuído

Agora que você tem um repositório Git remoto configurado como um ponto focal para todos os desenvolvedores compartilharem os seus códigos, e está familiarizado com os comandos básicos do Git em um fluxo de trabalho local, verá como utilizar alguns dos fluxos de trabalho distribuídos que o Git oferece.

Neste capítulo, você verá como trabalhar com o Git em um ambiente distribuído como contribuidor (contributor) e integrador (integrator). Ou seja, você aprenderá como contribuir com código para um projeto com sucesso e torná-lo o mais fácil possível para você e para o mantenedor do projeto, e também como manter um projeto com sucesso com vários desenvolvedores contribuindo.

Fluxos de Trabalho Distribuídos

Em contraste com os Sistemas de Controle de Versão Centralizados (CVCSs), a natureza distribuída do Git permite que você seja muito mais flexível na forma como os desenvolvedores colaboram em projetos. Em sistemas centralizados, cada desenvolvedor é um nó trabalhando de forma mais ou menos igual com um hub central. No Git, no entanto, todo desenvolvedor é potencialmente tanto um nó quanto um hub; ou seja, todo desenvolvedor pode contribuir com código para outros repositórios e manter um repositório público no qual outros podem basear o seu trabalho e para o qual podem contribuir. Isso apresenta uma vasta gama de possibilidades de fluxo de trabalho para o seu projeto e/ou equipe, então abordaremos alguns paradigmas comuns que aproveitam essa flexibilidade. Analisaremos os pontos fortes e as possíveis fraquezas de cada design; você pode escolher apenas um para usar, ou pode misturar e combinar (mix and match) recursos de cada um.

Fluxo de Trabalho Centralizado

Em sistemas centralizados, geralmente há um único modelo de colaboração — o fluxo de trabalho centralizado. Um hub central, ou *repositório*, pode aceitar código, e todos sincronizam o seu trabalho com ele. Um número de desenvolvedores são nós — consumidores daquele hub — e sincronizam com esse local centralizado.

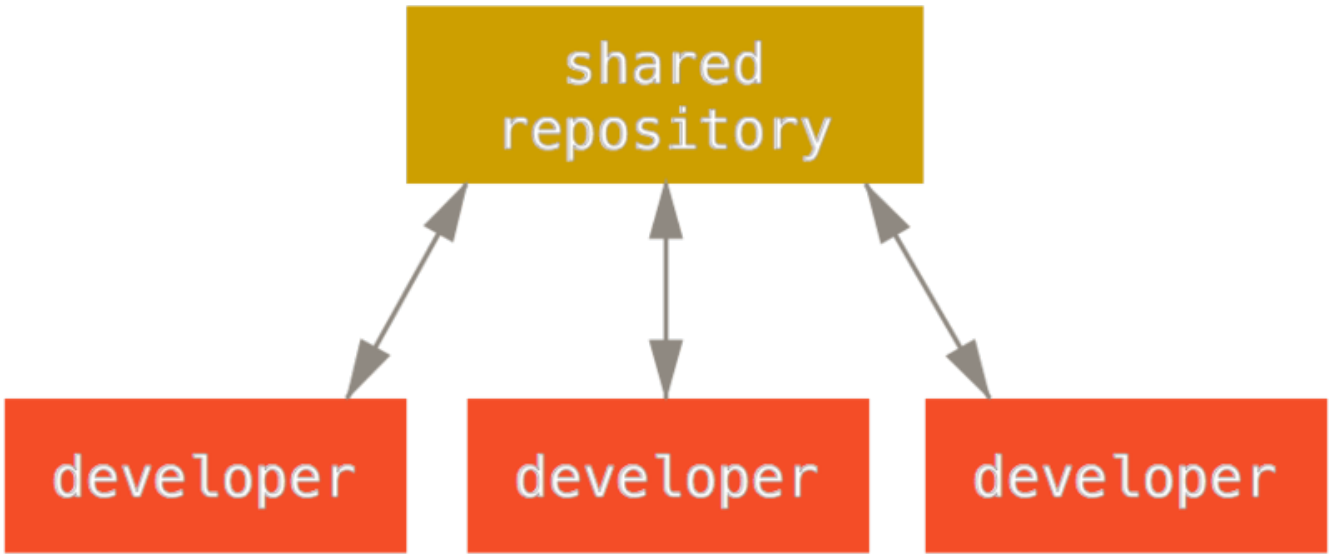


Figure 53. Fluxo de trabalho centralizado

Isso significa que, se dois desenvolvedores clonarem do hub e ambos fizerem alterações, o primeiro desenvolvedor a fazer o push das suas alterações de volta pode fazê-lo sem problemas. O segundo desenvolvedor deve fazer o merge do trabalho do primeiro antes de fazer o push das alterações, para não substituir as alterações do primeiro desenvolvedor. Esse conceito é tão verdadeiro no Git quanto no Subversion (ou em qualquer CVCS), e esse modelo funciona perfeitamente bem no Git.

Se você já estiver confortável com um fluxo de trabalho centralizado na sua empresa ou equipe, poderá continuar usando esse fluxo de trabalho com o Git facilmente. Basta configurar um único repositório e dar acesso de push a todos na sua equipe; o Git não permitirá que os usuários sobrescrevam os trabalhos uns dos outros.

Digamos que John e Jessica comecem a trabalhar ao mesmo tempo. John termina a sua alteração e faz o push para o servidor. Então, Jessica tenta fazer o push de suas alterações, mas o servidor as rejeita. Ela é informada de que está tentando fazer um push de alterações non-fast-forward e que não conseguirá fazê-lo até fazer o fetch e o merge. Este fluxo de trabalho é atraente para muitas pessoas porque é um paradigma com o qual muitos estão familiarizados e confortáveis.

Isso também não se limita a equipes pequenas. Com o modelo de ramificação (branching model) do Git, é possível que centenas de desenvolvedores trabalhem com sucesso em um único projeto através de dezenas de branches simultaneamente.

Fluxo de Trabalho do Gerente de Integração (Integration-Manager)

Como o Git permite que você tenha vários repositórios remotos, é possível ter um fluxo de trabalho em que cada desenvolvedor tem acesso de gravação ao seu próprio repositório público e acesso de leitura aos de todos os outros. Esse cenário geralmente inclui um repositório canônico que representa o projeto “oficial”. Para contribuir com esse projeto, você cria o seu próprio clone público do projeto e faz o push das suas alterações para ele. Então, você pode enviar uma solicitação ao mantenedor do projeto principal para fazer o pull das suas alterações. O mantenedor pode então adicionar o seu repositório como remoto, testar as suas alterações localmente, mesclá-las no branch dele e fazer o push de volta para o repositório dele. O processo funciona da seguinte forma (veja Fluxo de trabalho do Gerente de Integração (Integration-manager)):

1. O mantenedor do projeto faz o push para o seu repositório público.
2. Um contribuidor clona esse repositório e faz alterações.
3. O contribuidor faz o push para a sua própria cópia pública.
4. O contribuidor envia um e-mail ao mantenedor pedindo que ele faça o pull das alterações.
5. O mantenedor adiciona o repositório do contribuidor como um remoto e faz o merge localmente.
6. O mantenedor faz o push das alterações mescladas (merged changes) para o repositório principal.

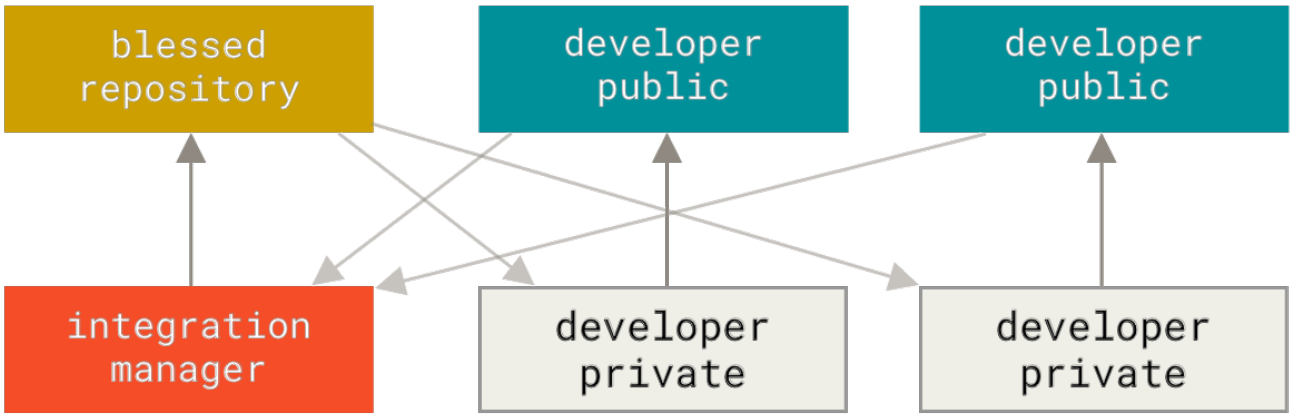


Figure 54. Fluxo de trabalho do Gerente de Integração (Integration-manager)

Este é um fluxo de trabalho muito comum com ferramentas baseadas em hub como o GitHub ou GitLab, onde é fácil fazer um fork de um projeto e fazer o push das suas alterações para o seu fork para que todos vejam. Uma das principais vantagens dessa abordagem é que você pode continuar a trabalhar, e o mantenedor do repositório principal pode fazer o pull das suas alterações a qualquer momento. Os contribuidores não precisam esperar que o projeto incorpore as suas alterações — cada parte pode trabalhar no seu próprio ritmo.

Fluxo de Trabalho Ditador e Tenentes (Dictator and Lieutenants)

Esta é uma variante de um fluxo de trabalho de múltiplos repositórios. Geralmente é usado por projetos enormes com centenas de colaboradores; um exemplo famoso é o kernel do Linux. Vários gerentes de integração são responsáveis por certas partes do repositório; eles são chamados de *tenentes* (lieutenants). Todos os tenentes têm um gerente de integração conhecido como o ditador benevolente (benevolent dictator). O ditador benevolente faz o push do seu diretório para um repositório de referência do qual todos os colaboradores precisam fazer o pull. O processo funciona assim (veja Fluxo de trabalho do ditador benevolente (Benevolent dictator)):

1. Desenvolvedores regulares trabalham em seu branch de tópico e fazem o rebase do seu trabalho no topo da `master`. O branch `master` é aquele do repositório de referência para o qual o ditador faz o push.
2. Os tenentes mesclam (merge) os branches de tópico dos desenvolvedores em seu branch `master`.
3. O ditador mescla os branches `master` dos tenentes no branch `master` do ditador.
4. Finalmente, o ditador faz o push desse branch `master` para o repositório de referência para que os outros desenvolvedores possam fazer o rebase nele.

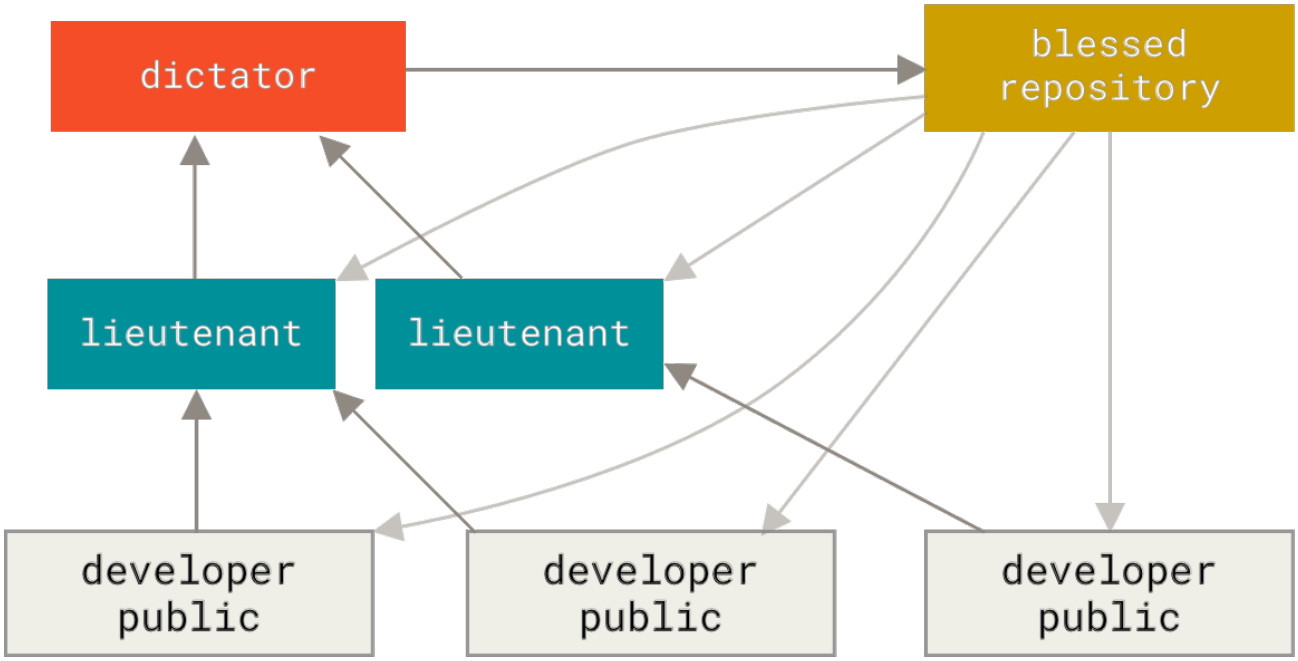


Figure 55. Fluxo de trabalho do ditador benevolente (Benevolent dictator)

Esse tipo de fluxo de trabalho não é comum, mas pode ser útil em projetos muito grandes ou em ambientes altamente hierárquicos. Ele permite que o líder do projeto (o ditador) delegue grande parte do trabalho e colete grandes subconjuntos de código em vários pontos antes de integrá-los.

Padrões para Gerenciar Branches de Código-Fonte



Martin Fowler criou um guia "Patterns for Managing Source Code Branches" (Padrões para Gerenciar Branches de Código-Fonte). Este guia abrange todos os fluxos de trabalho comuns do Git e explica como/quando usá-los. Há também uma seção comparando frequências de integração altas e baixas.

https://martinfowler.com/articles/branching-patterns.html

Resumo dos Fluxos de Trabalho

Estes são alguns fluxos de trabalho comumente usados que são possíveis com um sistema distribuído como o Git, mas você pode ver que muitas variações são possíveis para se adequar ao seu fluxo de trabalho particular do mundo real. Agora que você pode (esperamos) determinar qual combinação de fluxo de trabalho pode funcionar para você, abordaremos alguns exemplos mais específicos de como cumprir os papéis principais que compõem os diferentes fluxos. Na próxima seção, você aprenderá sobre alguns padrões comuns para contribuir com um projeto.

Contribuindo com um Projeto

A principal dificuldade em descrever como contribuir para um projeto são as inúmeras variações sobre como fazer isso. Como o Git é muito flexível, as pessoas podem e trabalham juntas de muitas maneiras, e é problemático descrever como você deve contribuir — cada projeto é um pouco diferente. Algumas das variáveis envolvidas são a contagem de contribuidores ativos, o fluxo de trabalho escolhido, o seu acesso de commit e possivelmente o método de contribuição externa.

A primeira variável é a contagem de contribuidores ativos — quantos usuários estão contribuindo ativamente com código para este projeto e com que frequência? Em muitos casos, você terá dois ou três desenvolvedores com alguns commits por dia, ou possivelmente menos para projetos um tanto inativos (dormant). Para empresas ou projetos maiores, o número de desenvolvedores pode ser na casa dos milhares, com centenas ou milhares de commits chegando a cada dia. Isso é importante porque, com cada vez mais desenvolvedores, você se depara com mais problemas para garantir que o seu código se aplique de forma limpa ou possa ser facilmente mesclado. As alterações que você envia podem se tornar obsoletas ou gravemente corrompidas por trabalhos que são mesclados enquanto você estava trabalhando ou enquanto as suas alterações estavam aguardando para serem aprovadas ou aplicadas. Como você pode manter o seu código consistentemente atualizado e os seus commits válidos?

A próxima variável é o fluxo de trabalho em uso para o projeto. Ele é centralizado, com cada desenvolvedor tendo igual acesso de gravação à linha de código principal? O projeto tem um mantenedor ou gerente de integração que verifica todos os patches? Todos os patches são revisados por pares e aprovados? Você está envolvido nesse processo? Existe um sistema de tenentes (lieutenant) em vigor, e você tem que submeter o seu trabalho a eles primeiro?

A próxima variável é o seu acesso de commit. O fluxo de trabalho necessário para contribuir para um projeto é muito diferente se você tem acesso de gravação ao projeto do que se você não tem. Se você não tem acesso de gravação, como o projeto prefere aceitar o trabalho contribuído? Ele sequer tem uma política? Quanto trabalho você está contribuindo de uma vez? Com que frequência você contribui?

Todas essas questões podem afetar como você contribui de forma eficaz para um projeto e quais fluxos de trabalho são preferidos ou estão disponíveis para você. Abordaremos aspectos de cada uma delas em uma série de casos de uso, passando do simples para o mais complexo; você deve ser capaz de construir os fluxos de trabalho específicos de que precisa na prática a partir desses exemplos.

Diretrizes de Commit

Antes de começarmos a examinar os casos de uso específicos, aqui vai uma nota rápida sobre mensagens de commit. Ter uma boa diretriz para criar commits e cumpri-la torna o trabalho com o Git e a colaboração com outras pessoas muito mais fáceis. O projeto Git fornece um documento que expõe várias boas dicas para criar commits a partir dos quais enviar patches — você pode lê-lo no código-fonte do Git, no arquivo `Documentation/SubmittingPatches`.

Primeiro, as suas submissões não devem conter erros de espaço em branco (whitespace). O Git fornece uma maneira fácil de verificar isso — antes de fazer o commit, execute `git diff --check`, o qual identifica possíveis erros de espaço em branco e os lista para você.

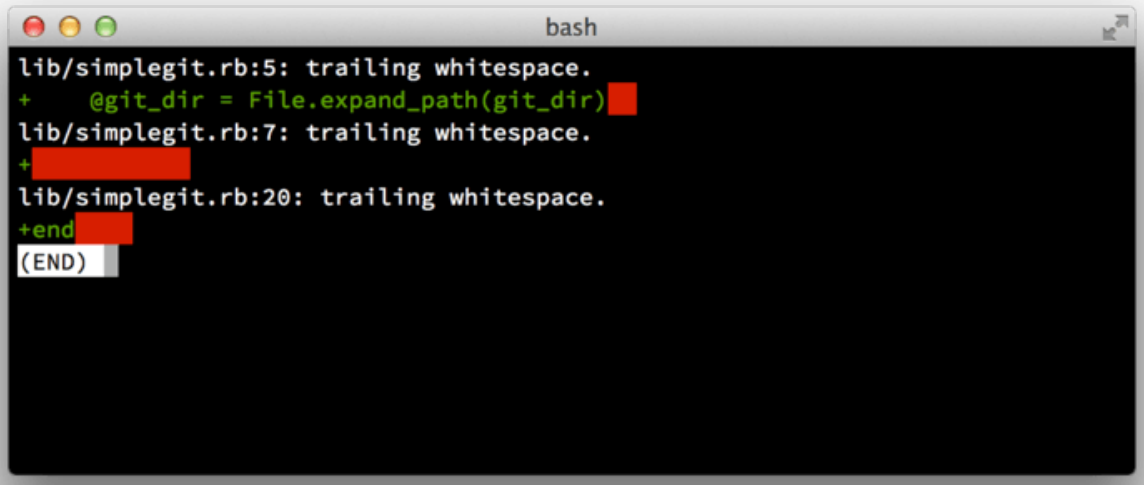


Figure 56. Saída do `git diff --check`

Se você executar esse comando antes de fazer o commit, poderá saber se está prestes a commitar problemas de espaço em branco que podem irritar outros desenvolvedores.

Em seguida, tente fazer de cada commit um conjunto de alterações (changeset) logicamente separado. Se puder, tente tornar as suas alterações digeríveis — não codifique um fim de semana inteiro sobre cinco issues diferentes e depois as envie todas como um único commit enorme na segunda-feira. Mesmo que você não faça commits durante o fim de semana, use a área de preparo (staging area) na segunda-feira para dividir o seu trabalho em pelo menos um commit por issue, com uma mensagem útil por commit. Se algumas das alterações modificarem o mesmo arquivo, tente usar `git add --patch` para preparar arquivos parcialmente (abordado em detalhes em Área de Stage Interativa). O snapshot do projeto na ponta do branch é idêntico quer você faça um ou cinco commits, desde que todas as alterações sejam adicionadas em algum momento, então tente facilitar as coisas para os seus colegas desenvolvedores quando eles tiverem que revisar as suas alterações.

Esta abordagem também torna mais fácil extrair ou reverter um dos conjuntos de alterações se você precisar mais tarde. A seção Reescrevendo o Histórico descreve uma série de truques úteis do Git para reescrever o histórico e preparar arquivos interativamente — use essas ferramentas para ajudar a criar um histórico limpo e compreensível antes de enviar o trabalho para outra pessoa.

A última coisa a ter em mente é a mensagem de commit. Adquirir o hábito de criar mensagens de commit de qualidade torna o uso e a colaboração com o Git muito mais fáceis. Como regra geral, as suas mensagens devem começar com uma única linha que não tenha mais de 50 caracteres e que descreva o conjunto de alterações de forma concisa, seguida por uma linha em branco, seguida por uma explicação mais detalhada. O projeto Git requer que a explicação mais detalhada inclua a sua motivação para a alteração e contraste a sua implementação com o comportamento anterior — esta é uma boa diretriz a seguir. Escreva sua mensagem de commit no imperativo: "Fix bug", e não "Fixed bug" nem "Fixes bug.". Aqui está um modelo (template) que você pode seguir, o qual adaptamos levemente de um originalmente escrito por Tim Pope:

```
Capitalized, short (50 chars or less) summary
```

```
More detailed explanatory text, if necessary.  Wrap it to about 72
characters or so.  In some contexts, the first line is treated as the
subject of an email and the rest of the text as the body.  The blank
line separating the summary from the body is critical (unless you omit
the body entirely); tools like rebase will confuse you if you run the
two together.

Write your commit message in the imperative: "Fix bug" and not "Fixed bug"
or "Fixes bug."  This convention matches up with commit messages generated
by commands like git merge and git revert.

Further paragraphs come after blank lines.

- Bullet points are okay, too

- Typically a hyphen or asterisk is used for the bullet, followed by a
  single space, with blank lines in between, but conventions vary here

- Use a hanging indent
```

Se todas as suas mensagens de commit seguirem este modelo, as coisas serão muito mais fáceis para você e para os desenvolvedores com quem você colabora. O projeto Git tem mensagens de commit bem formatadas — tente rodar `git log --no-merges` nele para ver como se parece o histórico de commits de um projeto bem formatado.



Faça o que dizemos, não o que fazemos.

Para fins de concisão, muitos dos exemplos neste livro não têm mensagens de commit bem formatadas como esta; em vez disso, nós simplesmente usamos a opção `-m` com o `git commit`.

Em resumo, faça o que dizemos, não o que fazemos.

Equipe Pequena Privada

A configuração mais simples que você provavelmente encontrará é um projeto privado com um ou dois outros desenvolvedores. “Privado” (Private), neste contexto, significa código fechado (closed-source) — não acessível ao mundo exterior. Você e todos os outros desenvolvedores têm acesso de push ao repositório.

Neste ambiente, você pode seguir um fluxo de trabalho semelhante ao que faria ao usar o Subversion ou outro sistema centralizado. Você ainda obtém as vantagens de coisas como commits offline e ramificações e mesclagens muito mais simples, mas o fluxo de trabalho pode ser muito semelhante; a principal diferença é que os merges acontecem do lado do cliente em vez de no servidor na hora do commit. Vamos ver como isso pode ser quando dois desenvolvedores começam a trabalhar juntos com um repositório compartilhado. O primeiro desenvolvedor, John, clona o repositório, faz uma alteração e faz o commit localmente. As mensagens de protocolo foram substituídas por ⋯ nesses exemplos para encurtá-las um pouco.

```
# John's Machine
$ git clone john@githost:simplegit.git
Cloning into 'simplegit'...
...
$ cd simplegit/
$ vim lib/simplegit.rb
$ git commit -am 'Remove invalid default value'
[master 738ee87] Remove invalid default value
 1 files changed, 1 insertions(+), 1 deletions(-)
```

A segunda desenvolvedora, Jessica, faz a mesma coisa — clona o repositório e faz o commit de uma alteração:

```
# Jessica's Machine
$ git clone jessica@githost:simplegit.git
Cloning into 'simplegit'...
...
$ cd simplegit/
$ vim TODO
$ git commit -am 'Add reset task'
[master fbff5bc] Add reset task
 1 files changed, 1 insertions(+), 0 deletions(-)
```

Agora, Jessica faz o push do seu trabalho para o servidor, o que funciona muito bem:

```
# Jessica's Machine
$ git push origin master
...
To jessica@githost:simplegit.git
   1edee6b..fbff5bc  master -> master
```

A última linha da saída acima mostra uma mensagem de retorno útil da operação push. O formato básico é `<oldref>..<newref> fromref → toref`, onde `oldref` significa a referência antiga, `newref` significa a nova referência, `fromref` é o nome da referência local sofrendo o push, e `toref` é o nome da referência remota sendo atualizada. Você verá saídas semelhantes a esta nas discussões a seguir, portanto, ter uma ideia básica do significado ajudará a entender os vários estados dos repositórios. Mais detalhes estão disponíveis na documentação do git-push.

Continuando com este exemplo, pouco tempo depois, John faz algumas alterações, faz o commit delas no seu repositório local e tenta fazer o push para o mesmo servidor:

```
# John's Machine
$ git push origin master
To john@githost:simplegit.git
 ! [rejected]        master -> master (non-fast forward)
error: failed to push some refs to 'john@githost:simplegit.git'
```

Neste caso, o push de John falha por causa do push anterior de Jessica com as alterações *dela*. Isso é especialmente importante de entender se você estiver acostumado com o Subversion, porque você notará que os dois desenvolvedores não editaram o mesmo arquivo. Embora o Subversion faça tal mesclagem (merge) automaticamente no servidor se arquivos diferentes forem editados, com o Git, você deve *primeiro* mesclar os commits localmente. Em outras palavras, John deve primeiro fazer o fetch das alterações upstream de Jessica e mesclá-las no seu repositório local antes de ter permissão para fazer o push.

Como um primeiro passo, John faz o fetch do trabalho de Jessica (isso apenas *faz o fetch* do trabalho upstream de Jessica, ainda não o mescla no trabalho de John):

```
$ git fetch origin
...
From john@githost:simplegit
 + 049d078...fbff5bc master     -> origin/master
```

Neste ponto, o repositório local de John se parece com isso:

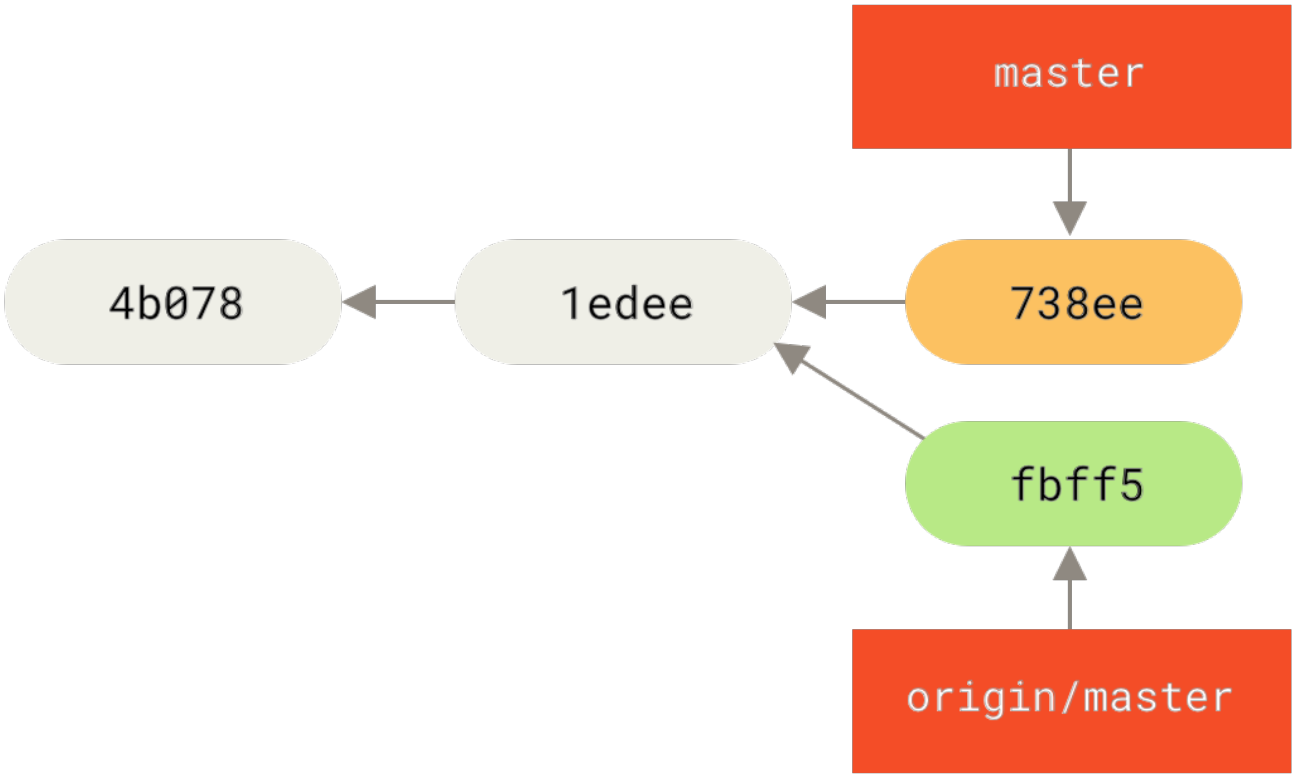


Figure 57. Histórico divergente de John

Agora John pode mesclar o trabalho de Jessica que ele buscou (fetched) no seu próprio trabalho local:

```
$ git merge origin/master
Merge made by the 'recursive' strategy.
 TODO |    1 +
 1 files changed, 1 insertions(+), 0 deletions(-)
```

Desde que essa mesclagem local ocorra sem problemas, o histórico atualizado de John agora se parecerá com isso:

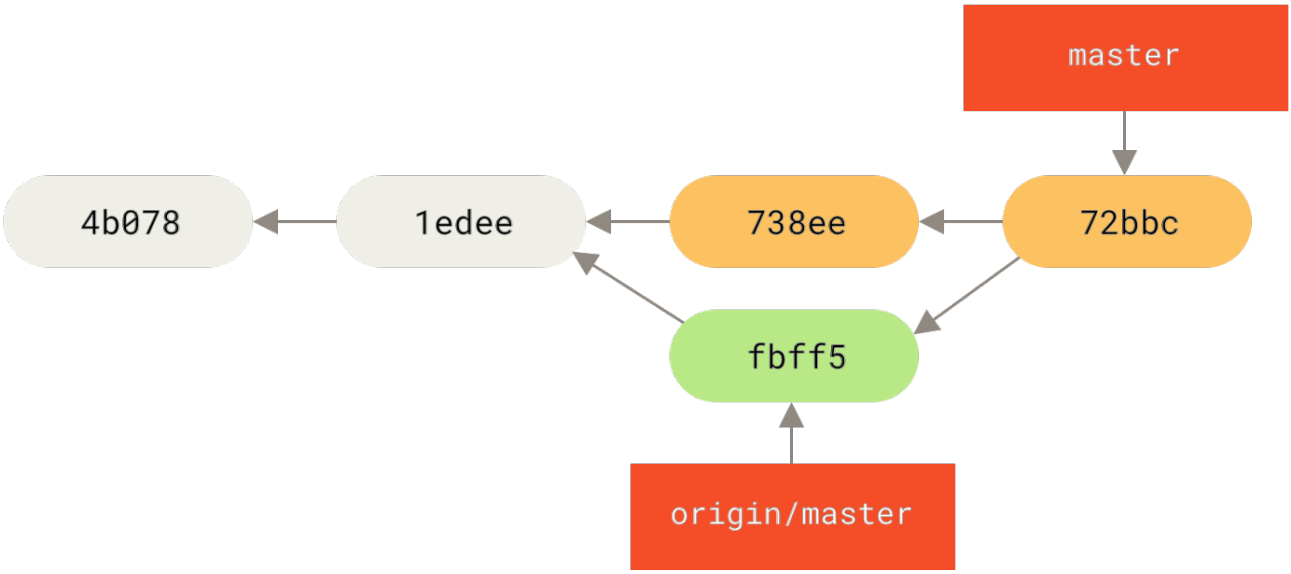


Figure 58. Repositório de John após mesclar `origin/master`

Neste ponto, John pode querer testar este novo código para garantir que nenhum dos trabalhos de Jessica afete o dele e, desde que tudo pareça bem, ele pode finalmente fazer o push do novo trabalho mesclado para o servidor:

```
$ git push origin master
...
To john@githost:simplegit.git
   fbff5bc..72bbc59  master -> master
```

No final, o histórico de commits de John ficará assim:

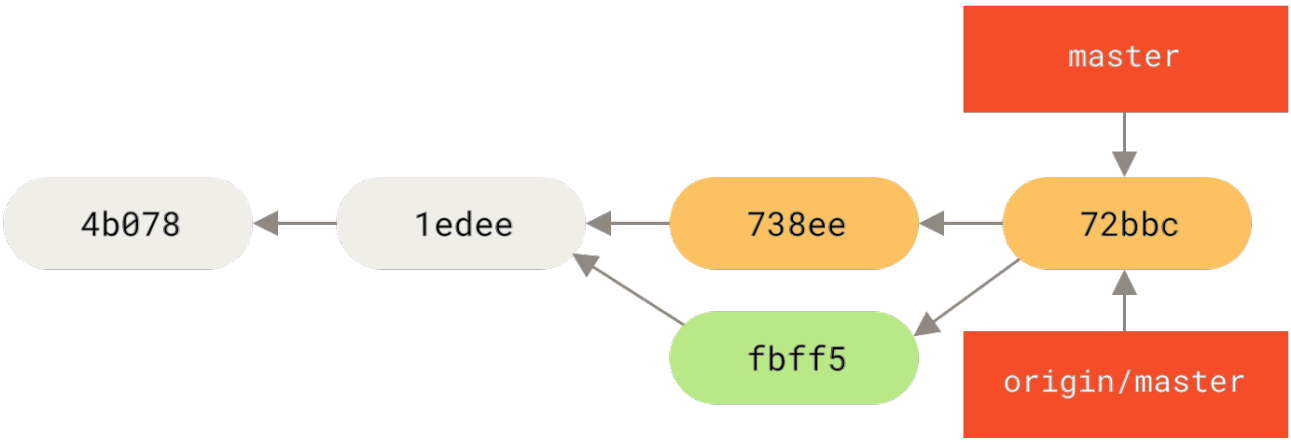


Figure 59. Histórico de John após o push para o servidor `origin`

Enquanto isso, Jessica criou um novo branch de tópico chamado `issue54`, e fez três commits nesse branch. Ela ainda não fez o fetch das alterações de John, então o histórico de commits dela é este:

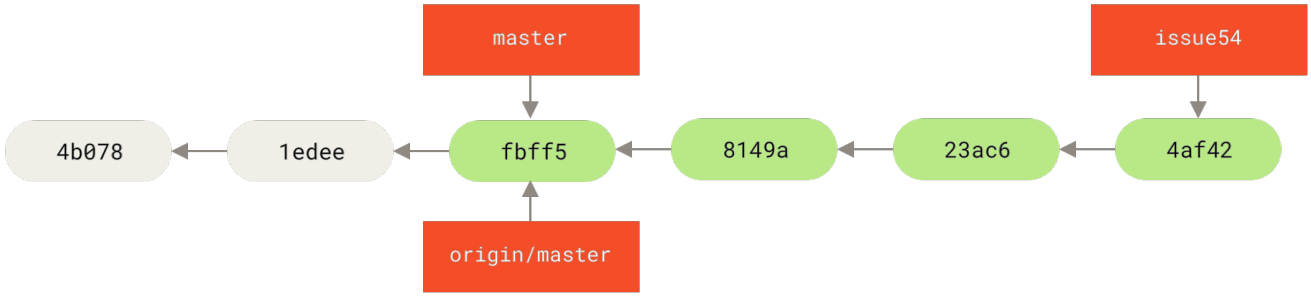


Figure 60. Branch de tópico de Jessica

De repente, Jessica descobre que John fez o push de um novo trabalho para o servidor e ela quer dar uma olhada, então ela pode fazer o fetch de todo o conteúdo novo do servidor que ela ainda não tem, com:

```
# Jessica's Machine
$ git fetch origin
...
From jessica@githost:simplegit
   fbff5bc..72bbc59  master     -> origin/master
```

Isso puxa (pulls down) o trabalho que John fez o push enquanto isso. O histórico de Jessica agora está assim:

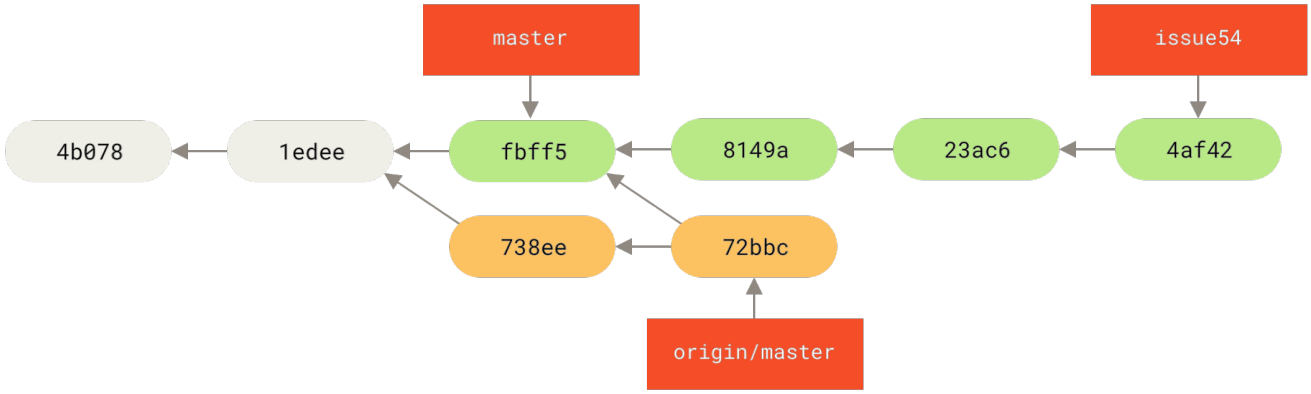


Figure 61. Histórico de Jessica após buscar (fetching) as alterações de John

Jessica acha que o seu branch de tópico está pronto, mas ela quer saber qual parte do trabalho que ela buscou de John ela tem que mesclar no trabalho dela para que ela possa fazer o push. Ela executa `git log` para descobrir:

```
$ git log --no-merges issue54..origin/master
commit 738ee872852dfaa9d6634e0dea7a324040193016
Author: John Smith <jsmith@example.com>
Date:   Fri May 29 16:01:27 2009 -0700

   Remove invalid default value
```

A sintaxe `issue54..origin/master` é um filtro de log que pede ao Git para exibir apenas os commits

que estão no último branch (neste caso `origin/master`) e que não estão no primeiro branch (neste caso `issue54`). Abordaremos essa sintaxe detalhadamente em Intervalos (Ranges) de Commits.

A partir da saída acima, podemos ver que há um único commit que John fez que Jessica não mesclou em seu trabalho local. Se ela mesclar `origin/master`, esse é o único commit que modificará o seu trabalho local.

Agora, Jessica pode mesclar o seu trabalho de tópico no seu branch `master`, mesclar o trabalho de John (`origin/master`) no seu branch `master` e, em seguida, fazer o push de volta para o servidor novamente.

Primeiro (tendo feito o commit de todo o trabalho no seu branch de tópico `issue54`), Jessica muda de volta para o seu branch `master` em preparação para integrar todo esse trabalho:

```
$ git checkout master
Switched to branch 'master'
Your branch is behind 'origin/master' by 2 commits, and can be fast-forwarded.
```

Jessica pode mesclar tanto `origin/master` quanto `issue54` primeiro — ambos são upstream, então a ordem não importa. O snapshot final deve ser idêntico, não importa qual ordem ela escolha; apenas o histórico será diferente. Ela escolhe mesclar o branch `issue54` primeiro:

```
$ git merge issue54
Updating fbff5bc..4af4298
Fast forward
 README           |    1 +
 lib/simplegit.rb |    6 +++++-
 2 files changed, 6 insertions(+), 1 deletions(-)
```

Nenhum problema ocorre; como você pode ver, foi uma simples mesclagem fast-forward. Jessica agora conclui o processo de mesclagem local mesclando o trabalho de John buscado (fetched) anteriormente que está no branch `origin/master`:

```
$ git merge origin/master
Auto-merging lib/simplegit.rb
Merge made by the 'recursive' strategy.
 lib/simplegit.rb |    2 +-
 1 files changed, 1 insertions(+), 1 deletions(-)
```

Tudo se mescla de forma limpa, e o histórico de Jessica agora fica assim:

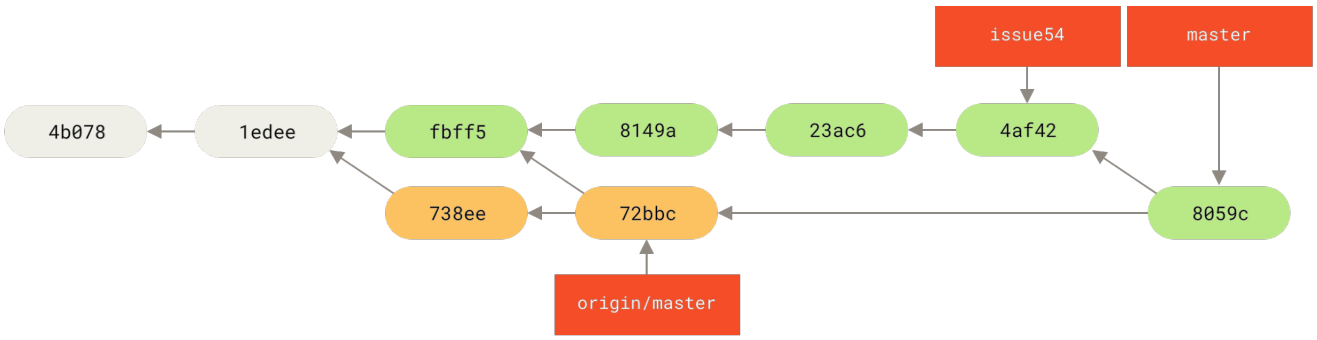


Figure 62. Histórico de Jessica após mesclar as alterações de John

Agora `origin/master` pode ser alcançado a partir do branch `master` de Jessica, então ela deve ser capaz de fazer o push com sucesso (supondo que John não tenha feito o push de ainda mais alterações nesse meio tempo):

```
$ git push origin master
...
To jessica@githost:simplegit.git
   72bbc59..8059c15  master -> master
```

Cada desenvolvedor fez alguns commits e mesclou o trabalho do outro com sucesso.

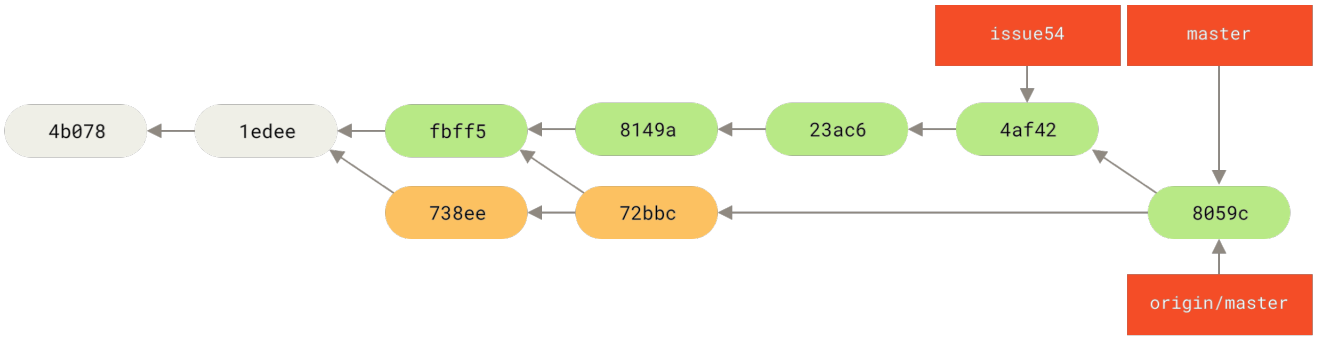


Figure 63. Histórico de Jessica após o push de todas as alterações de volta para o servidor

Esse é um dos fluxos de trabalho mais simples. Você trabalha por um tempo (geralmente em um branch de tópico), e mescla esse trabalho no seu branch `master` quando ele estiver pronto para ser integrado. Quando você quiser compartilhar esse trabalho, você faz o fetch e o merge do seu `master` a partir do `origin/master` se ele tiver mudado, e finalmente faz o push para o branch `master` no servidor. A sequência geral é algo assim:

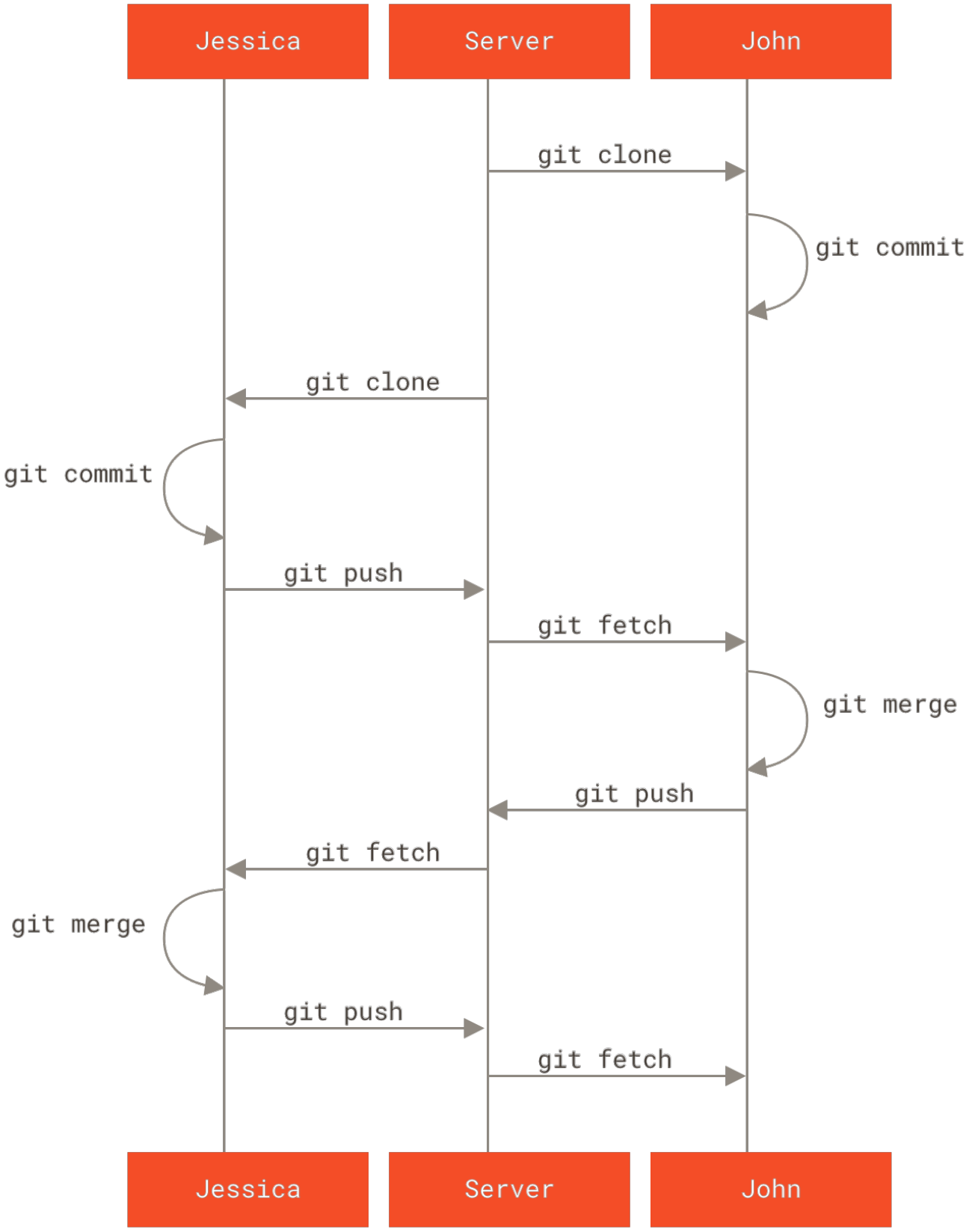


Figure 64. Sequência geral de eventos para um fluxo de trabalho Git simples com múltiplos desenvolvedores

Equipe Gerenciada Privada

Neste próximo cenário, você analisará os papéis dos contribuidores em um grupo privado maior. Você aprenderá como trabalhar em um ambiente onde pequenos grupos colaboram em recursos, após o qual essas contribuições baseadas em equipe são integradas por outra parte.

Digamos que John e Jessica estejam trabalhando juntos em um recurso (vamos chamá-lo de "featureA"), enquanto Jessica e uma terceira desenvolvedora, Josie, estão trabalhando em um segundo (digamos, "featureB"). Neste caso, a empresa está usando um tipo de fluxo de trabalho de gerente de integração onde o trabalho dos grupos individuais é integrado apenas por certos engenheiros, e o branch `master` do repositório principal pode ser atualizado apenas por esses engenheiros. Neste cenário, todo o trabalho é feito em branches baseados em equipe e puxados (pulled) juntos pelos integradores mais tarde.

Vamos seguir o fluxo de trabalho de Jessica enquanto ela trabalha nos seus dois recursos, colaborando em paralelo com dois desenvolvedores diferentes neste ambiente. Supondo que ela já tenha o seu repositório clonado, ela decide trabalhar em `featureA` primeiro. Ela cria um novo branch para o recurso e trabalha um pouco nele lá:

```
# Jessica's Machine
$ git checkout -b featureA
Switched to a new branch 'featureA'
$ vim lib/simplegit.rb
$ git commit -am 'Add limit to log function'
[featureA 3300904] Add limit to log function
 1 files changed, 1 insertions(+), 1 deletions(-)
```

Neste ponto, ela precisa compartilhar o seu trabalho com John, então ela faz o push dos commits do seu branch `featureA` para o servidor. Jessica não tem acesso de push ao branch `master` — apenas os integradores têm — então ela tem que fazer o push para outro branch para colaborar com John:

```
$ git push -u origin featureA
...
To jessica@githost:simplegit.git
 * [new branch]      featureA -> featureA
```

Jessica envia um e-mail para John para avisá-lo que ela fez o push de algum trabalho em um branch chamado `featureA` e que ele pode dar uma olhada agora. Enquanto ela espera pelo feedback de John, Jessica decide começar a trabalhar no `featureB` com Josie. Para começar, ela inicia um novo branch de recurso, baseando-o no branch `master` do servidor:

```
# Jessica's Machine
$ git fetch origin
$ git checkout -b featureB origin/master
Switched to a new branch 'featureB'
```

Agora, Jessica faz alguns commits no branch `featureB`:

```
$ vim lib/simplegit.rb
$ git commit -am 'Make ls-tree function recursive'
[featureB e5b0fdc] Make ls-tree function recursive
 1 files changed, 1 insertions(+), 1 deletions(-)
```

```
$ vim lib/simplegit.rb
$ git commit -am 'Add ls-files'
[featureB 8512791] Add ls-files
 1 files changed, 5 insertions(+), 0 deletions(-)
```

O repositório de Jessica agora está assim:

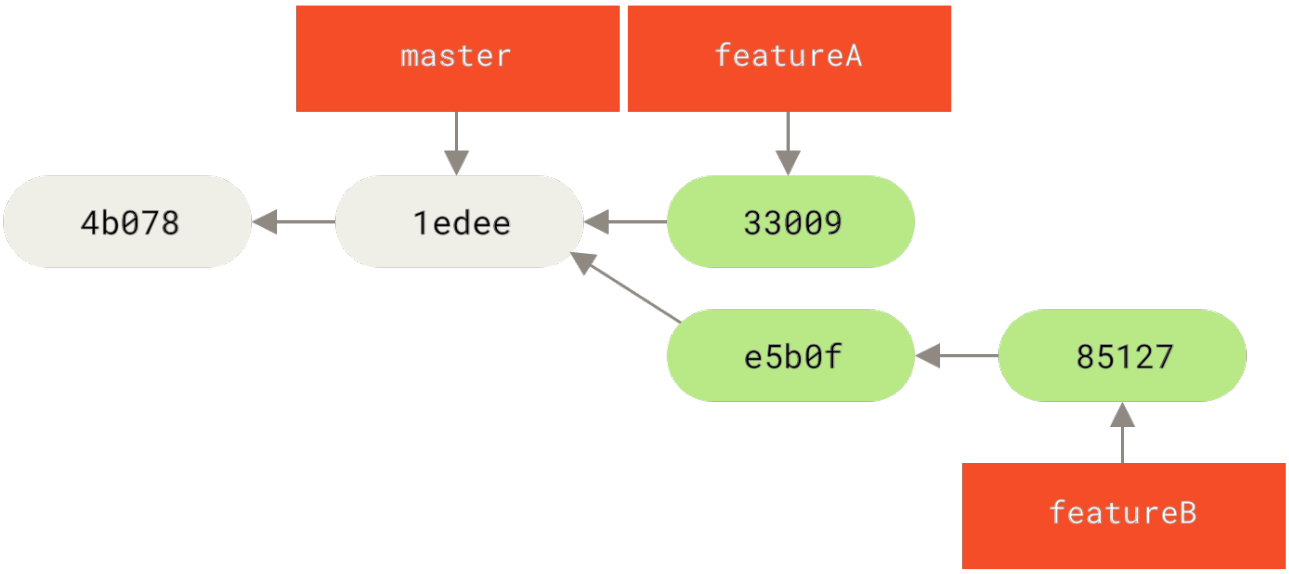


Figure 65. Histórico inicial de commits de Jessica

Ela está pronta para fazer o push do seu trabalho, mas recebe um e-mail de Josie dizendo que um branch com algum trabalho inicial de "featureB" já sofreu push no servidor como o branch `featureBee`. Jessica precisa mesclar essas alterações com as suas próprias antes que ela possa fazer o push do seu trabalho para o servidor. Primeiro, Jessica faz o fetch das alterações de Josie com `git fetch`:

```
$ git fetch origin
...
From jessica@githost:simplegit
 * [new branch]      featureBee -> origin/featureBee
```

Supondo que Jessica ainda esteja no seu branch `featureB` verificado (checked-out), ela agora pode mesclar o trabalho de Josie nesse branch com `git merge`:

```
$ git merge origin/featureBee
Auto-merging lib/simplegit.rb
Merge made by the 'recursive' strategy.
 lib/simplegit.rb |    4 ++++
 1 files changed, 4 insertions(+), 0 deletions(-)
```

Neste ponto, Jessica quer fazer o push de todo esse trabalho "featureB" mesclado de volta para o servidor, mas ela não quer simplesmente fazer o push do seu próprio branch `featureB`. Em vez disso, como Josie já iniciou um branch upstream `featureBee`, Jessica quer fazer o push para *esse*

branch, o que ela faz com:

```
$ git push -u origin featureB:featureBee
...
To jessica@githost:simplegit.git
   fba9af8..cd685d1  featureB -> featureBee
```

Isso é chamado de *refspec* (especificação de referência). Consulte Refspec para uma discussão mais detalhada sobre refspecs do Git e coisas diferentes que você pode fazer com eles. Observe também a flag `-u`; esta é uma abreviação para `--set-upstream`, que configura os branches para facilitar pushes e pulls (puxar) posteriormente.

De repente, Jessica recebe um e-mail de John, que lhe diz que ele fez o push de algumas alterações para o branch `featureA` no qual eles estão colaborando, e ele pede que Jessica dê uma olhada nelas. Novamente, Jessica executa um simples `git fetch` para buscar *todo* o conteúdo novo do servidor, incluindo (claro) o trabalho mais recente de John:

```
$ git fetch origin
...
From jessica@githost:simplegit
   3300904..aad881d  featureA   -> origin/featureA
```

Jessica pode exibir o log do novo trabalho de John comparando o conteúdo do branch `featureA` recém-buscado (fetched) com a sua cópia local do mesmo branch:

```
$ git log featureA..origin/featureA
commit aad881d154acdaeb2b6b18ea0e827ed8a6d671e6
Author: John Smith <jsmith@example.com>
Date:   Fri May 29 19:57:33 2009 -0700

    Increase log output to 30 from 25
```

Se Jessica gostar do que vê, ela pode mesclar o novo trabalho de John no seu branch `featureA` local com:

```
$ git checkout featureA
Switched to branch 'featureA'
$ git merge origin/featureA
Updating 3300904..aad881d
Fast forward
 lib/simplegit.rb |   10 +++++++++-
1 files changed, 9 insertions(+), 1 deletions(-)
```

Por fim, Jessica pode querer fazer algumas pequenas alterações em todo esse conteúdo mesclado, então ela é livre para fazer essas alterações, fazer o commit delas no seu branch `featureA` local e fazer o push do resultado final de volta para o servidor:

```
$ git commit -am 'Add small tweak to merged content'
[featureA 774b3ed] Add small tweak to merged content
 1 files changed, 1 insertions(+), 1 deletions(-)
$ git push
...
To jessica@githost:simplegit.git
   3300904..774b3ed  featureA -> featureA
```

O histórico de commits de Jessica agora está mais ou menos assim:

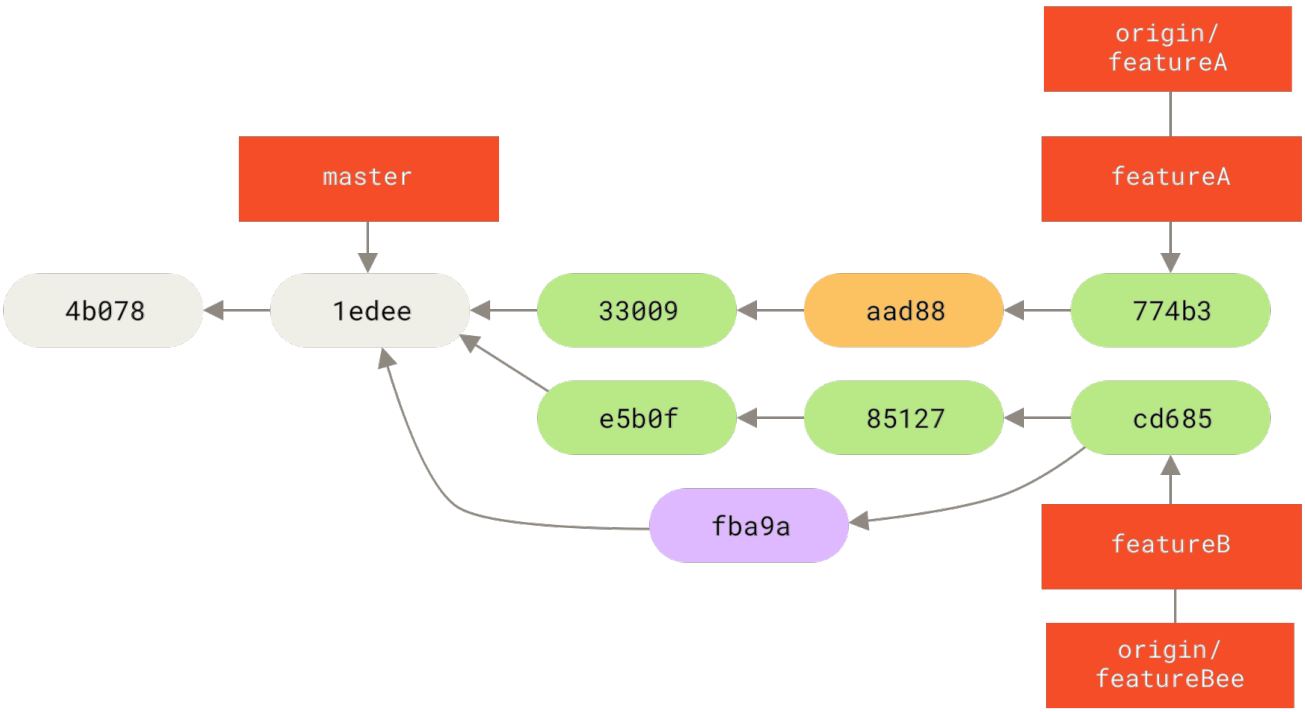


Figure 66. Histórico de Jessica após fazer um commit num branch de recurso

Em algum momento, Jessica, Josie e John informam aos integradores que os branches `featureA` e `featureBee` no servidor estão prontos para integração na linha principal (mainline). Depois que os integradores mesclam esses branches na linha principal, um fetch trará o novo commit de mesclagem (merge commit), deixando o histórico assim:

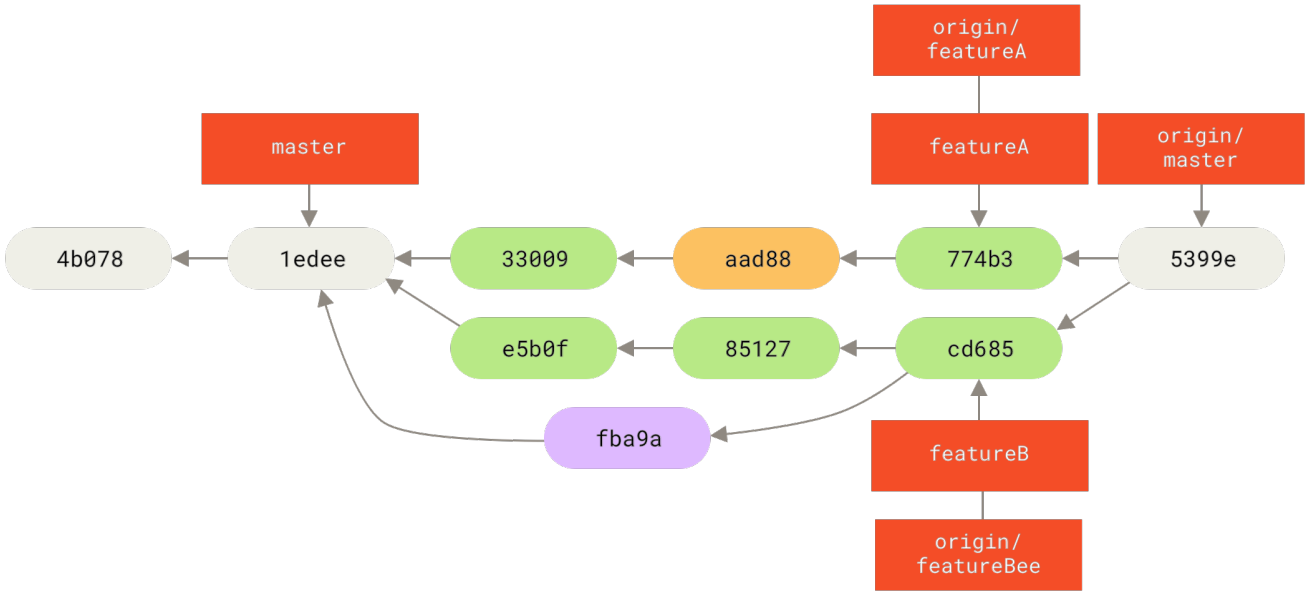


Figure 67. Histórico de Jessica após mesclar os seus dois branches de tópico

Muitos grupos mudam para o Git devido a essa capacidade de ter várias equipes trabalhando em paralelo, mesclando as diferentes linhas de trabalho no final do processo. A capacidade de subgrupos menores de uma equipe de colaborarem via branches remotos, sem necessariamente ter que envolver ou impedir a equipe inteira, é um benefício enorme do Git. A sequência para o fluxo de trabalho que você viu aqui é algo assim:

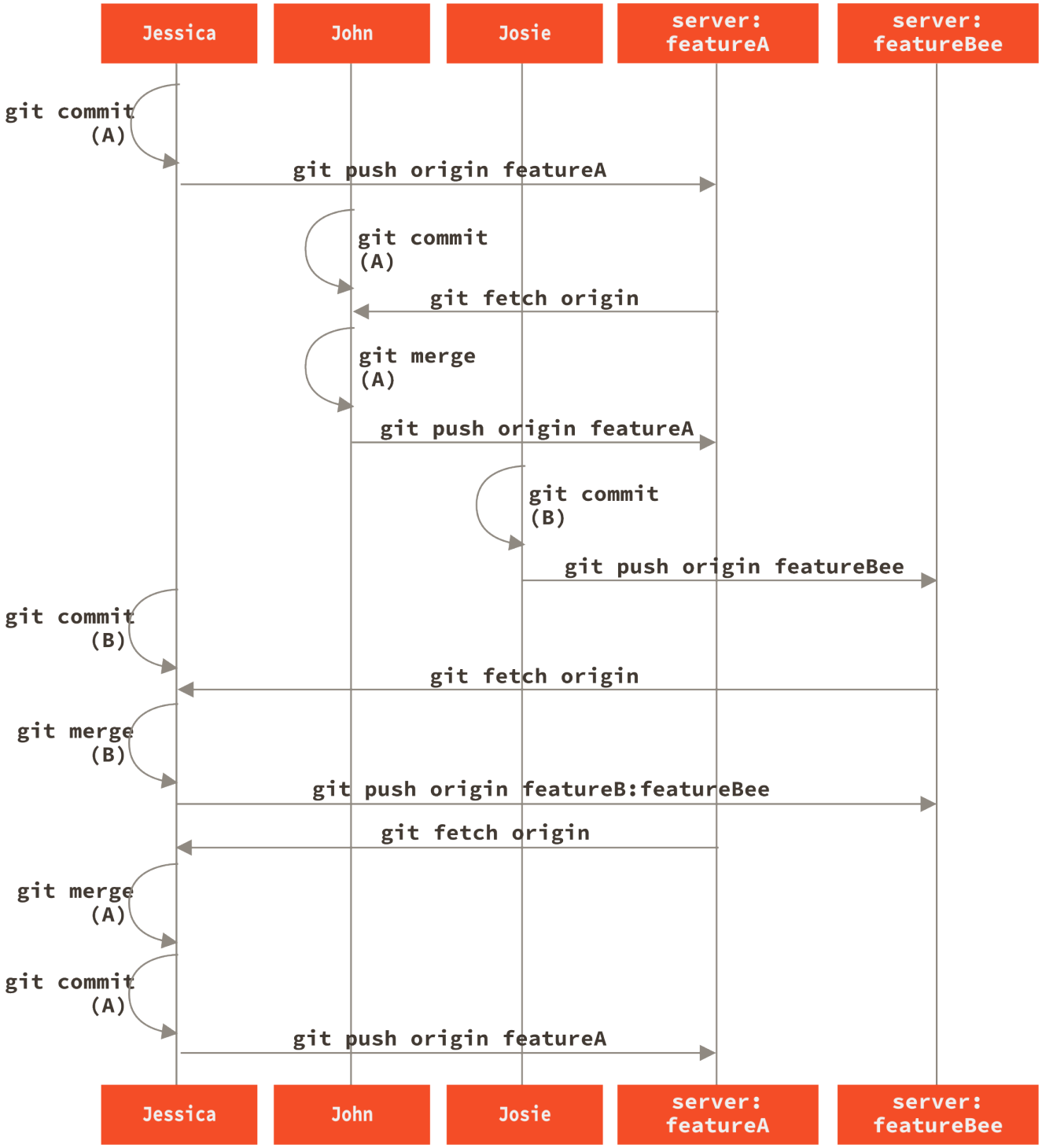


Figure 68. Sequência básica deste fluxo de trabalho de equipe gerenciada

Projeto Público Bifurcado (Forked)

Contribuir para projetos públicos é um pouco diferente. Como você não tem as permissões para atualizar diretamente os branches no projeto, tem que levar o trabalho aos mantenedores de alguma outra forma. Este primeiro exemplo descreve a contribuição por meio de bifurcação (forking) em hosts Git que suportam forking facilmente. Muitos sites de hospedagem suportam isso (incluindo GitHub, BitBucket, repo.or.cz e outros), e muitos mantenedores de projetos esperam esse estilo de contribuição. A próxima seção trata de projetos que preferem aceitar patches contribuídos por e-mail.

Primeiro, você provavelmente vai querer clonar o repositório principal, criar um branch de tópico

para o patch ou série de patches com os quais planeja contribuir, e fazer o seu trabalho nele. A sequência se parece basicamente com isto:

```
$ git clone <url>
$ cd project
$ git checkout -b featureA
  ... work ...
$ git commit
  ... work ...
$ git commit
```



Você pode querer usar `rebase -i` para esmagar (squash) o seu trabalho em um único commit ou reorganizar o trabalho nos commits para tornar o patch mais fácil para o mantenedor revisar — veja Reescrevendo o Histórico para obter mais informações sobre rebasing interativo.

Quando o trabalho do seu branch terminar e você estiver pronto para contribuir de volta aos mantenedores, vá para a página original do projeto e clique no botão “Fork”, criando o seu próprio fork gravável do projeto. Você então precisará adicionar a URL deste repositório como um novo remoto do seu repositório local; neste exemplo, vamos chamá-lo de `myfork`:

```
$ git remote add myfork <url>
```

Você então precisará fazer o push do seu novo trabalho para este repositório. É mais fácil fazer o push do branch de tópico em que você está trabalhando para o seu repositório originado de um fork, em vez de mesclar esse trabalho no seu branch `master` e fazer o push disso. O motivo é que, se o seu trabalho não for aceito ou for escolhido a dedo (cherry-picked), você não precisa voltar atrás no seu branch `master` (a operação `cherry-pick` do Git é abordada com mais detalhes em Fluxos de Trabalho de Rebase e Cherry-Pick). Se os mantenedores usarem `merge`, `rebase` ou `cherry-pick` no seu trabalho, você eventualmente o receberá de volta puxando (pulling) do repositório deles de qualquer maneira.

Em qualquer caso, você pode fazer o push do seu trabalho com:

```
$ git push -u myfork featureA
```

Depois que o seu trabalho for enviado para o fork do repositório, você precisará notificar os mantenedores do projeto original que tem um trabalho que gostaria que eles mesclassem. Isso costuma ser chamado de *pull request* (solicitação de pull), e você normalmente gera essa solicitação por meio do site — o GitHub tem o seu próprio mecanismo “Pull Request” que abordaremos em GitHub — ou você pode executar o comando `git request-pull` e enviar a saída resultante por e-mail para o mantenedor do projeto manualmente.

O comando `git request-pull` recebe o branch base no qual você deseja que o seu branch de tópico seja puxado (pulled) e a URL do repositório Git de onde você quer que eles o puxem, e produz um resumo de todas as alterações que você está solicitando. Por exemplo, se Jessica quiser enviar um

pull request a John e tiver feito dois commits no branch de tópico que ela acabou de dar push, ela pode executar isto:

```
$ git request-pull origin/master myfork
The following changes since commit 1edee6b1d61823a2de3b09c160d7080b8d1b3a40:
Jessica Smith (1):
        Create new function

are available in the git repository at:

  https://githost/simplegit.git featureA

Jessica Smith (2):
      Add limit to log function
      Increase log output to 30 from 25

 lib/simplegit.rb |   10 +++++++++-
 1 files changed, 9 insertions(+), 1 deletions(-)
```

Esta saída pode ser enviada para o mantenedor — ela informa a ele de onde o trabalho foi ramificado (branched), resume os commits e identifica de onde o novo trabalho deve ser puxado.

Em um projeto do qual você não é o mantenedor, geralmente é mais fácil ter um branch como `master` que rastreie o `origin/master` e fazer o seu trabalho em branches de tópicos que podem ser facilmente descartados caso sejam rejeitados. Ter os temas de trabalho isolados em branches de tópicos também facilita o rebase do seu trabalho se a ponta (tip) do repositório principal se moveu nesse meio tempo e seus commits não se aplicam mais de forma limpa. Por exemplo, se você quiser submeter um segundo tópico de trabalho para o projeto, não continue trabalhando no branch de tópico que você acabou de dar o push — comece do zero a partir do branch `master` do repositório principal:

```
$ git checkout -b featureB origin/master
  ... work ...
$ git commit
$ git push myfork featureB
$ git request-pull origin/master myfork
  ... email generated request pull to maintainer ...
$ git fetch origin
```

Agora, cada um de seus tópicos está contido em um silo — semelhante a uma fila de patches — que você pode reescrever, fazer o rebase e modificar sem que os tópicos interfiram ou dependam uns dos outros, assim:

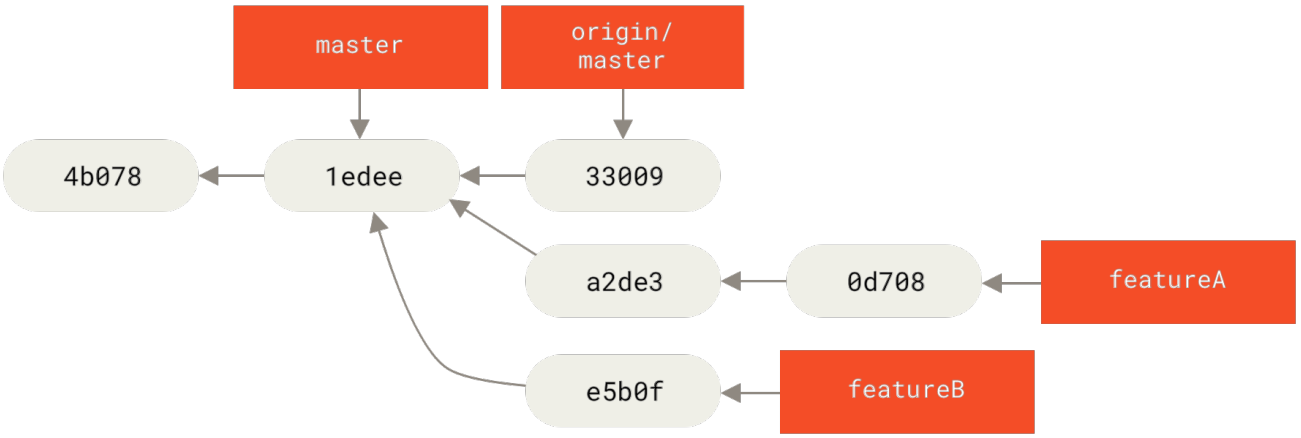


Figure 69. Histórico inicial de commits com o trabalho `featureB`

Digamos que o mantenedor do projeto puxou vários outros patches e tentou o seu primeiro branch, mas ele não se mescla mais de forma limpa. Neste caso, você pode tentar fazer o rebase desse branch no topo do `origin/master`, resolver os conflitos para o mantenedor, e então reenviar as suas alterações:

```
$ git checkout featureA
$ git rebase origin/master
$ git push -f myfork featureA
```

Isso reescreve o seu histórico para agora se parecer com Histórico de commits após o trabalho `featureA`.

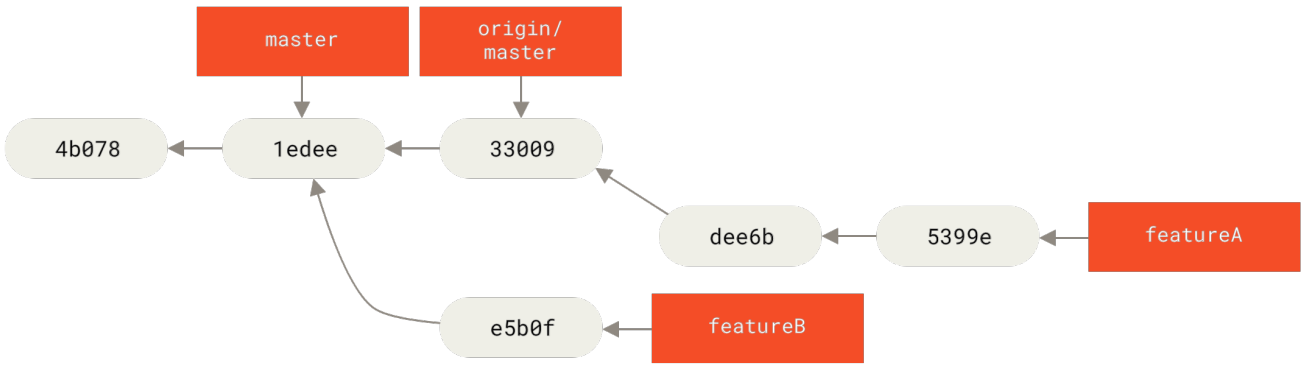


Figure 70. Histórico de commits após o trabalho `featureA`

Como você fez o rebase do branch, você tem que especificar o `-f` para o seu comando de push a fim de conseguir substituir o branch `featureA` no servidor por um commit que não é descendente dele. Uma alternativa seria fazer o push deste novo trabalho para um branch diferente no servidor (talvez chamado `featureAv2`).

Vamos analisar mais um cenário possível: o mantenedor olhou o trabalho no seu segundo branch e gostou do conceito, mas gostaria que você mudasse um detalhe de implementação. Você também aproveitará essa oportunidade para mover o trabalho e baseá-lo no branch `master` atual do projeto. Você inicia um novo branch baseado no branch `origin/master` atual, usa o squash nas alterações de `featureB` ali, resolve os conflitos, faz a alteração de implementação e, em seguida, faz o push disso

como um novo branch:

```
$ git checkout -b featureBv2 origin/master
$ git merge --squash featureB
  ... change implementation ...
$ git commit
$ git push myfork featureBv2
```

A opção `--squash` pega todo o trabalho do branch mesclado e usa squash num único changeset, produzindo o estado do repositório como se uma mesclagem (merge) real tivesse acontecido, sem na verdade criar um merge commit. Isso significa que o seu commit futuro terá apenas um pai e permite que você introduza todas as alterações de outro branch e então faça mais alterações antes de registrar o novo commit. Além disso, a opção `--no-commit` pode ser útil para atrasar o merge commit no caso do processo de merge padrão.

Neste ponto, você pode notificar o mantenedor que você fez as alterações solicitadas, e que eles podem encontrar essas alterações no seu branch `featureBv2`.

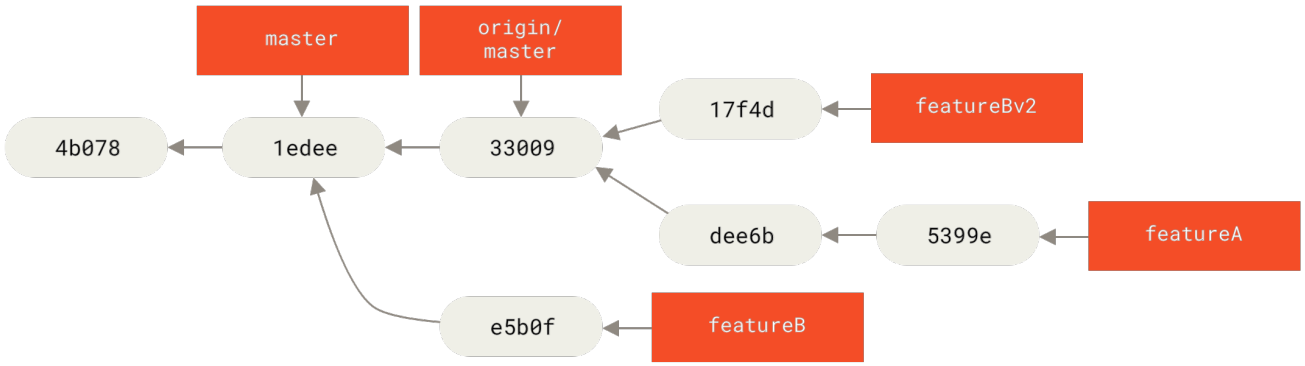


Figure 71. Histórico de commits após o trabalho `featureBv2`

Projeto Público Grande

Muitos projetos maiores estabeleceram procedimentos para aceitar patches — você precisará verificar as regras específicas de cada projeto, porque elas serão diferentes. Como há vários projetos mais antigos e maiores que aceitam patches via uma lista de discussão de desenvolvedores, examinaremos um exemplo disso agora.

O fluxo de trabalho é semelhante ao caso de uso anterior — você cria branches de tópicos para cada série de patches em que trabalha. A diferença está em como você as submete ao projeto. Em vez de fazer o fork do projeto e fazer o push para a sua própria versão gravável, você gera versões de e-mail de cada série de commits e as envia por e-mail para a lista de discussão de desenvolvedores:

```
$ git checkout -b topicA
  ... work ...
$ git commit
  ... work ...
```

```
$ git commit
```

Agora você tem dois commits que deseja enviar para a lista de discussão. Você usa `git format-patch` para gerar os arquivos no formato mbox que você pode enviar por e-mail para a lista — ele transforma cada commit em uma mensagem de e-mail com a primeira linha da mensagem de commit como o assunto e o resto da mensagem mais o patch que o commit introduz como o corpo. O lado bom disso é que aplicar um patch a partir de um e-mail gerado com `format-patch` preserva adequadamente todas as informações do commit.

```
$ git format-patch -M origin/master
0001-add-limit-to-log-function.patch
0002-increase-log-output-to-30-from-25.patch
```

O comando `format-patch` imprime os nomes dos arquivos de patch que ele cria. A opção `-M` diz ao Git para procurar por renomeações. Os arquivos acabam ficando assim:

```
$ cat 0001-add-limit-to-log-function.patch
From 330090432754092d704da8e76ca5c05c198e71a8 Mon Sep 17 00:00:00 2001
From: Jessica Smith <jessica@example.com>
Date: Sun, 6 Apr 2008 10:17:23 -0700
Subject: [PATCH 1/2] Add limit to log function

Limit log functionality to the first 20

---
 lib/simplegit.rb |    2 +-
 1 files changed, 1 insertions(+), 1 deletions(-)

diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index 76f47bc..f9815f1 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -14,7 +14,7 @@ class SimpleGit
   end

   def log(treeish = 'master')
-    command("git log #{treeish}")
+    command("git log -n 20 #{treeish}")
   end

   def ls_tree(treeish = 'master')
--
2.1.0
```

Você também pode editar esses arquivos de patch para adicionar mais informações para a lista de e-mail que você não quer que apareçam na mensagem de commit. Se você adicionar texto entre a linha `---` e o início do patch (a linha `diff --git`), os desenvolvedores poderão lê-lo, mas esse

conteúdo é ignorado pelo processo de aplicação de patch.

Para enviar isso por e-mail a uma lista de discussão, você pode colar o arquivo no seu programa de e-mail ou enviá-lo através de um programa de linha de comando. Colar o texto muitas vezes causa problemas de formatação, especialmente com clientes "mais inteligentes" que não preservam as quebras de linha e outros espaços em branco adequadamente. Felizmente, o Git fornece uma ferramenta para ajudá-lo a enviar patches devidamente formatados via IMAP, o que pode ser mais fácil para você. Nós demonstraremos como enviar um patch via Gmail, que por acaso é o agente de e-mail que conhecemos melhor; você pode ler instruções detalhadas para diversos programas de e-mail no final do já mencionado arquivo `Documentation/SubmittingPatches` no código-fonte do Git.

Primeiro, você precisa configurar a seção imap no seu arquivo `~/.gitconfig`. Você pode definir cada valor separadamente com uma série de comandos `git config`, ou você pode adicioná-los manualmente, mas no final seu arquivo de configuração deve se parecer com isso:

```
[imap]
  folder = "[Gmail]/Drafts"
  host = imaps://imap.gmail.com
  user = user@gmail.com
  pass = YX]8g76G_2^sFbd
  port = 993
  sslverify = false
```

Se o seu servidor IMAP não usar SSL, as duas últimas linhas provavelmente não são necessárias, e o valor do host será `imap://` em vez de `imaps://`. Quando isso estiver configurado, você pode usar `git imap-send` para colocar a série de patches na pasta Drafts (Rascunhos) do servidor IMAP especificado:

```
$ cat *.patch |git imap-send
Resolving imap.gmail.com... ok
Connecting to [74.125.142.109]:993... ok
Logging in...
sending 2 messages
100% (2/2) done
```

Neste ponto, você deve ser capaz de ir até a sua pasta de Rascunhos, alterar o campo Para (To) para a lista de discussão para a qual você está enviando o patch, possivelmente colocar em cópia (CC) o mantenedor ou a pessoa responsável por aquela seção, e enviá-lo.

Você também pode enviar os patches por meio de um servidor SMTP. Como antes, você pode definir cada valor separadamente com uma série de comandos `git config`, ou pode adicioná-los manualmente na seção sendemail do seu arquivo `~/.gitconfig`:

```
[sendemail]
  smtpencryption = tls
  smtpserver = smtp.gmail.com
  smtpuser = user@gmail.com
```

```
  smtpserverport = 587
```

Depois que isso for feito, você pode usar `git send-email` para enviar os seus patches:

```
$ git send-email *.patch
0001-add-limit-to-log-function.patch
0002-increase-log-output-to-30-from-25.patch
Who should the emails appear to be from? [Jessica Smith <jessica@example.com>]
Emails will be sent from: Jessica Smith <jessica@example.com>
Who should the emails be sent to? jessica@example.com
Message-ID to be used as In-Reply-To for the first email? y
```

Então, o Git cospe um monte de informações de log parecidas com isto para cada patch que você está enviando:

```
(mbox) Adding cc: Jessica Smith <jessica@example.com> from
  \line 'From: Jessica Smith <jessica@example.com>'
OK. Log says:
Sendmail: /usr/sbin/sendmail -i jessica@example.com
From: Jessica Smith <jessica@example.com>
To: jessica@example.com
Subject: [PATCH 1/2] Add limit to log function
Date: Sat, 30 May 2009 13:29:15 -0700
Message-Id: <1243715356-61726-1-git-send-email-jessica@example.com>
X-Mailer: git-send-email 1.6.2.rc1.20.g8c5b.dirty
In-Reply-To: <y>
References: <y>

Result: OK
```



Para obter ajuda sobre a configuração do seu sistema e e-mail, mais dicas e truques e um sandbox para enviar um patch de teste por e-mail, acesse git-send-email.io.

Resumo

Nesta seção, abordamos vários fluxos de trabalho e falamos sobre as diferenças entre trabalhar como parte de uma pequena equipe em projetos de código fechado em vez de contribuir para um grande projeto público. Você sabe verificar erros de espaço em branco antes de fazer um commit e sabe escrever uma ótima mensagem de commit. Você aprendeu como formatar patches e enviá-los por e-mail a uma lista de discussão de desenvolvedores. Lidar com mesclagens também foi abordado no contexto dos diferentes fluxos de trabalho. Agora você está bem preparado para colaborar em qualquer projeto.

A seguir, você verá como trabalhar do outro lado da moeda: mantendo um projeto Git. Você aprenderá como ser um ditador benevolente ou gerente de integração.

Mantendo um Projeto

Além de saber como contribuir efetivamente para um projeto, você provavelmente precisará saber como manter um. Isso pode consistir em aceitar e aplicar patches gerados via `format-patch` e enviados por e-mail para você, ou integrar alterações em branches remotos de repositórios que você adicionou como remotos ao seu projeto. Quer você mantenha um repositório canônico ou queira ajudar verificando ou aprovando patches, você precisa saber como aceitar o trabalho de uma forma que seja mais clara para outros contribuidores e sustentável para você a longo prazo.

Trabalhando em Branches de Tópicos

Quando você estiver pensando em integrar um novo trabalho, geralmente é uma boa ideia testá-lo em um *branch de tópico* (topic branch) — um branch temporário feito especificamente para testar esse novo trabalho. Dessa forma, é fácil ajustar um patch individualmente e deixá-lo se não estiver funcionando até que você tenha tempo de voltar a ele. Se você criar um nome de branch simples com base no tema do trabalho que vai testar, como `ruby_client` ou algo similarmente descritivo, poderá se lembrar dele facilmente caso precise abandoná-lo por um tempo e voltar mais tarde. O mantenedor do projeto Git também tende a colocar esses branches em namespaces — como `sc/ruby_client`, onde `sc` é a abreviação da pessoa que contribuiu com o trabalho. Como você deve se lembrar, você pode criar o branch com base no seu branch `master` assim:

```
$ git branch sc/ruby_client master
```

Ou, se você também quiser mudar para ele imediatamente, pode usar a opção `checkout -b`:

```
$ git checkout -b sc/ruby_client master
```

Agora você está pronto para adicionar o trabalho contribuído que você recebeu a este branch de tópico e determinar se deseja mesclá-lo (merge) em seus branches de longo prazo.

Aplicando Patches a partir de E-mail

Se você receber um patch por e-mail que precisa integrar ao seu projeto, você precisa aplicar o patch no seu branch de tópico para avaliá-lo. Existem duas maneiras de aplicar um patch enviado por e-mail: com `git apply` ou com `git am`.

Aplicando um Patch com `apply`

Se você recebeu o patch de alguém que o gerou com `git diff` ou alguma variação do comando `diff` do Unix (o que não é recomendado; veja a próxima seção), você pode aplicá-lo com o comando `git apply`. Supondo que você tenha salvo o patch em `/tmp/patch-ruby-client.patch`, você pode aplicar o patch assim:

```
$ git apply /tmp/patch-ruby-client.patch
```

Isso modifica os arquivos no seu diretório de trabalho. É quase idêntico a executar um comando `patch -p1` para aplicar o patch, embora seja mais paranoico e aceite menos correspondências difusas (fuzzy matches) do que o patch. Ele também lida com adições, exclusões e renomeações de arquivos se estiverem descritos no formato `git diff`, o que o `patch` não faz. Finalmente, o `git apply` é um modelo de "aplique tudo ou aborte tudo" (apply all or abort all) onde tudo é aplicado ou nada é, enquanto o `patch` pode aplicar os arquivos de patch parcialmente, deixando o seu diretório de trabalho num estado estranho. O `git apply` é, no geral, muito mais conservador do que o `patch`. Ele não criará um commit para você — após executá-lo, você deve preparar (stage) e fazer o commit das alterações introduzidas manualmente.

Você também pode usar `git apply` para ver se um patch se aplica de forma limpa antes de tentar aplicá-lo de fato — você pode rodar `git apply --check` com o patch:

```
$ git apply --check 0001-see-if-this-helps-the-gem.patch
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
```

Se não houver saída, então o patch deve se aplicar de forma limpa. Esse comando também sai com um status diferente de zero se a verificação falhar, então você pode usá-lo em scripts se quiser.

Aplicando um Patch com `am`

Se o contribuidor for um usuário do Git e foi bom o suficiente para usar o comando `format-patch` para gerar o patch dele, então o seu trabalho é mais fácil porque o patch contém as informações do autor e uma mensagem de commit para você. Se você puder, encoraje os seus contribuidores a usar o `format-patch` em vez do `diff` para gerar patches para você. Você só deveria ter que usar o `git apply` para patches legados e coisas desse tipo.

Para aplicar um patch gerado pelo `format-patch`, você usa o `git am` (o comando é chamado de `am` porque é usado para "aplicar uma série de patches de uma caixa de correio"). Tecnicamente, o `git am` foi construído para ler um arquivo mbox, que é um formato simples de texto puro (plain-text) para armazenar uma ou mais mensagens de e-mail em um único arquivo de texto. Fica parecido com isso:

```
From 330090432754092d704da8e76ca5c05c198e71a8 Mon Sep 17 00:00:00 2001
From: Jessica Smith <jessica@example.com>
Date: Sun, 6 Apr 2008 10:17:23 -0700
Subject: [PATCH 1/2] Add limit to log function

Limit log functionality to the first 20
```

Este é o começo da saída do comando `git format-patch` que você viu na seção anterior; ele também representa um formato de e-mail mbox válido. Se alguém lhe enviou o patch de forma adequada por e-mail usando `git send-email`, e você fez o download dele no formato mbox, você pode apontar o `git am` para esse arquivo mbox, e ele começará a aplicar todos os patches que encontrar. Se você executa um cliente de e-mail que pode salvar vários e-mails no formato mbox, pode salvar uma série inteira de patches num arquivo e então usar o `git am` para aplicá-los um de cada vez.

No entanto, se alguém fez upload de um arquivo de patch gerado via `git format-patch` em um sistema de tickets ou algo semelhante, você pode salvar o arquivo localmente e depois passar esse arquivo salvo no seu disco para o `git am` aplicá-lo:

```
$ git am 0001-limit-log-function.patch
Applying: Add limit to log function
```

Você pode ver que ele foi aplicado de forma limpa e automaticamente criou o novo commit para você. As informações do autor são retiradas dos cabeçalhos `From` e `Date` do e-mail, e a mensagem do commit é retirada do `Subject` e do corpo (antes do patch) do e-mail. Por exemplo, se esse patch foi aplicado a partir do exemplo de mbox acima, o commit gerado se pareceria com isso:

```
$ git log --pretty=fuller -1
commit 6c5e70b984a60b3cecd395edd5b48a7575bf58e0
Author:     Jessica Smith <jessica@example.com>
AuthorDate: Sun Apr 6 10:17:23 2008 -0700
Commit:     Scott Chacon <schacon@gmail.com>
CommitDate: Thu Apr 9 09:19:06 2009 -0700

   Add limit to log function

   Limit log functionality to the first 20
```

A informação `Commit` indica a pessoa que aplicou o patch e a hora em que foi aplicado. A informação `Author` (Autor) é o indivíduo que originalmente criou o patch e quando ele foi originalmente criado.

Mas é possível que o patch não seja aplicado de forma limpa. Talvez o seu branch principal tenha divergido muito do branch a partir do qual o patch foi construído, ou o patch depende de outro patch que você ainda não aplicou. Nesse caso, o processo `git am` falhará e perguntará o que você deseja fazer:

```
$ git am 0001-see-if-this-helps-the-gem.patch
Applying: See if this helps the gem
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
Patch failed at 0001.
When you have resolved this problem run "git am --resolved".
If you would prefer to skip this patch, instead run "git am --skip".
To restore the original branch and stop patching run "git am --abort".
```

Este comando coloca marcadores de conflito em quaisquer arquivos com os quais tenha problemas, muito parecido com uma operação de merge ou rebase em conflito. Você resolve esse problema da mesma maneira — edite o arquivo para resolver o conflito, prepare (stage) o novo arquivo, e então execute `git am --resolved` para continuar com o próximo patch:

```
$ (fix the file)
```

```
$ git add ticgit.gemspec
$ git am --resolved
Applying: See if this helps the gem
```

Se você quiser que o Git tente resolver o conflito de forma um pouco mais inteligente, você pode passar uma opção `-3` para ele, o que faz com que o Git tente um merge de três vias (three-way merge). Essa opção não está ativada por padrão porque ela não funciona se o commit no qual o patch diz que foi baseado não estiver no seu repositório. Se você tiver esse commit — se o patch for baseado em um commit público — então a opção `-3` é geralmente muito mais inteligente para aplicar um patch conflitante:

```
$ git am -3 0001-see-if-this-helps-the-gem.patch
Applying: See if this helps the gem
error: patch failed: ticgit.gemspec:1
error: ticgit.gemspec: patch does not apply
Using index info to reconstruct a base tree...
Falling back to patching base and 3-way merge...
No changes -- Patch already applied.
```

Neste caso, sem a opção `-3`, o patch teria sido considerado como um conflito. Como a opção `-3` foi usada, o patch foi aplicado de forma limpa.

Se você estiver aplicando vários patches a partir de um mbox, você também pode executar o comando `am` no modo interativo, que para em cada patch que encontra e pergunta se você deseja aplicá-lo:

```
$ git am -3 -i mbox
Commit Body is:
--------------------------
See if this helps the gem
--------------------------
Apply? [y]es/[n]o/[e]dit/[v]iew patch/[a]ccept all
```

Isso é bom se você tiver vários patches salvos, porque você pode visualizar o patch primeiro se não se lembrar do que se trata, ou não aplicar o patch se já o tiver feito.

Quando todos os patches para o seu tópico forem aplicados e tiverem sido confirmados (committed) no seu branch, você pode escolher se e como integrá-los em um branch de longa duração.

Fazendo Checkout de Branches Remotos

Se a sua contribuição veio de um usuário do Git que configurou seu próprio repositório, fez o push de várias alterações para ele e depois lhe enviou a URL do repositório e o nome do branch remoto onde as alterações estão, você pode adicioná-lo como remoto e fazer os merges localmente.

Por exemplo, se Jessica lhe enviar um e-mail dizendo que ela tem um ótimo novo recurso no branch `ruby-client` de seu repositório, você pode testá-lo adicionando o remoto e fazendo checkout

desse branch localmente:

```
$ git remote add jessica https://github.com/jessica/myproject.git
$ git fetch jessica
$ git checkout -b rubyclient jessica/ruby-client
```

Se ela lhe enviar um e-mail novamente mais tarde com outro branch contendo outro ótimo recurso, você poderia fazer um `fetch` e `checkout` diretamente porque já tem a configuração do remoto.

Isso é mais útil se você estiver trabalhando com uma pessoa de forma consistente. Se alguém tiver apenas um único patch para contribuir de vez em quando, então aceitá-lo por e-mail pode consumir menos tempo do que exigir que todos executem seus próprios servidores e ter que adicionar e remover remotos continuamente para obter alguns patches. Também é improvável que você queira ter centenas de remotos, cada um para alguém que contribui com apenas um ou dois patches. No entanto, scripts e serviços hospedados podem facilitar isso — depende muito de como você desenvolve e como os seus contribuidores desenvolvem.

A outra vantagem dessa abordagem é que você também obtém o histórico dos commits. Embora você possa ter problemas de merge legítimos, você sabe em que ponto da sua história o trabalho deles se baseia; um merge de três vias adequado é o padrão em vez de ter que fornecer um `-3` e torcer para que o patch tenha sido gerado a partir de um commit público ao qual você tem acesso.

Se você não estiver trabalhando com uma pessoa consistentemente, mas ainda assim quiser fazer um pull dela dessa maneira, pode fornecer a URL do repositório remoto ao comando `git pull`. Isso faz um pull único e não salva a URL como uma referência de remoto:

```
$ git pull https://github.com/onetimeguy/project
From https://github.com/onetimeguy/project
 * branch            HEAD       -> FETCH_HEAD
Merge made by the 'recursive' strategy.
```

Determinando o Que é Introduzido

Agora você tem um branch de tópico que contém o trabalho contribuído. Neste ponto, você pode determinar o que gostaria de fazer com ele. Esta seção revisita alguns comandos para que você possa ver como pode usá-los para revisar exatamente o que você introduzirá se fizer o merge disto no seu branch principal.

Muitas vezes é útil obter uma revisão de todos os commits que estão neste branch mas que não estão no seu branch `master`. Você pode excluir os commits no branch `master` adicionando a opção `--not` antes do nome do branch. Isso faz a mesma coisa que o formato `master..contrib` que usamos anteriormente. Por exemplo, se o seu contribuidor lhe enviar dois patches e você criar um branch chamado `contrib` e aplicou esses patches lá, você pode executar isto:

```
$ git log contrib --not master
commit 5b6235bd297351589efc4d73316f0a68d484f118
Author: Scott Chacon <schacon@gmail.com>
```

```
Date:   Fri Oct 24 09:53:59 2008 -0700

    See if this helps the gem

commit 7482e0d16d04bea79d0dba8988cc78df655f16a0
Author: Scott Chacon <schacon@gmail.com>
Date:   Mon Oct 22 19:38:36 2008 -0700

    Update gemspec to hopefully work better
```

Para ver as alterações que cada commit introduz, lembre-se de que você pode passar a opção `-p` para o `git log` e ele anexará o diff introduzido a cada commit.

Para ver um diff completo do que aconteceria se você fosse mesclar esse branch de tópico com outro branch, você pode ter que usar um truque estranho para obter os resultados corretos. Você pode pensar em executar isso:

```
$ git diff master
```

Este comando fornece um diff, mas pode ser enganoso. Se o seu branch `master` avançou desde que você criou o branch de tópico a partir dele, então você obterá resultados aparentemente estranhos. Isso acontece porque o Git compara diretamente os snapshots do último commit do branch de tópico em que você está e o snapshot do último commit no branch `master`. Por exemplo, se você adicionou uma linha num arquivo no branch `master`, uma comparação direta dos snapshots fará parecer que o branch de tópico vai remover essa linha.

Se o `master` for um ancestral direto do seu branch de tópico, isso não é um problema; mas se as duas histórias tiverem divergido, o diff fará parecer que você está adicionando todas as coisas novas no seu branch de tópico e removendo tudo o que é exclusivo do branch `master`.

O que você realmente quer ver são as alterações adicionadas ao branch de tópico — o trabalho que você introduzirá se mesclar esse branch com o `master`. Você faz isso fazendo com que o Git compare o último commit no seu branch de tópico com o primeiro ancestral comum que ele tem com o branch `master`.

Tecnicamente, você pode fazer isso descobrindo explicitamente o ancestral comum e então rodando o seu diff sobre ele:

```
$ git merge-base contrib master
36c7dba2c95e6bbb78dfa822519ecfec6e1ca649
$ git diff 36c7db
```

ou, de forma mais concisa:

```
$ git diff $(git merge-base contrib master)
```

No entanto, nenhum desses é particularmente conveniente, então o Git fornece outra abreviação (shorthand) para fazer a mesma coisa: a sintaxe de três pontos (triple-dot syntax). No contexto do comando `git diff`, você pode colocar três pontos após outro branch para fazer um `diff` entre o último commit do branch em que você está e o seu ancestral comum com o outro branch:

```
$ git diff master...contrib
```

Esse comando mostra apenas o trabalho que o seu atual branch de tópico introduziu desde o seu ancestral comum com o `master`. Essa é uma sintaxe muito útil para se lembrar.

Integrando o Trabalho Contribuído

Quando todo o trabalho no seu branch de tópico estiver pronto para ser integrado a um branch de linha principal (mainline branch), a questão é como fazê-lo. Além disso, qual fluxo de trabalho geral você deseja usar para manter o seu projeto? Você tem várias opções, então abordaremos algumas delas.

Fluxos de Trabalho de Merge

Um fluxo de trabalho básico é simplesmente mesclar todo esse trabalho diretamente no seu branch `master`. Neste cenário, você tem um branch `master` que contém código basicamente estável. Quando você tem trabalho num branch de tópico que você acha que concluiu, ou trabalho com o qual outra pessoa contribuiu e você verificou, você o mescla no seu branch master, exclui o branch de tópico recém-mesclado e repete.

Por exemplo, se temos um repositório com trabalho em dois branches chamados `ruby_client` e `php_client` que se parece com Histórico com vários branches de tópicos, e mesclamos `ruby_client` seguido de `php_client`, o seu histórico acabará ficando parecido com Após um merge de branch de tópico.

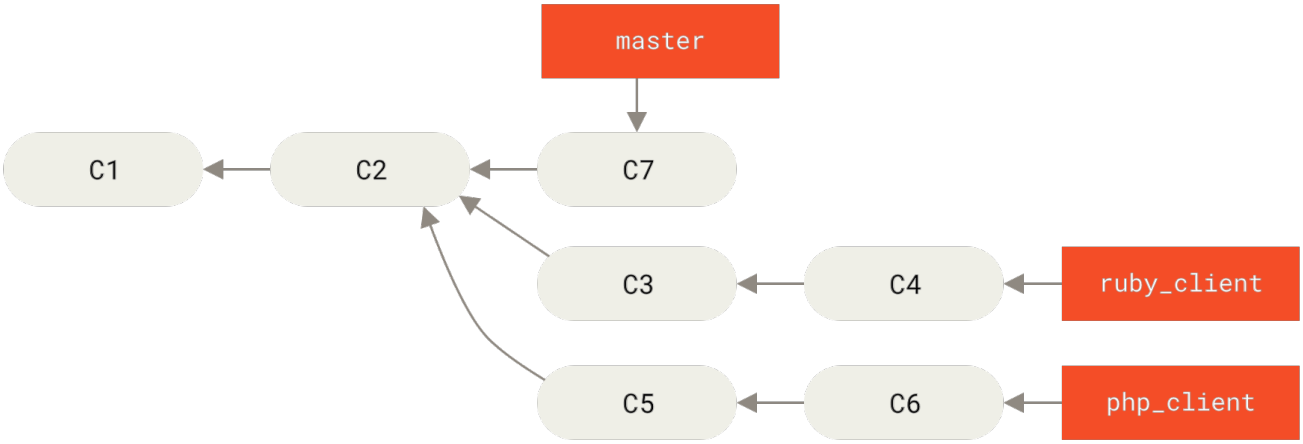


Figure 72. Histórico com vários branches de tópicos

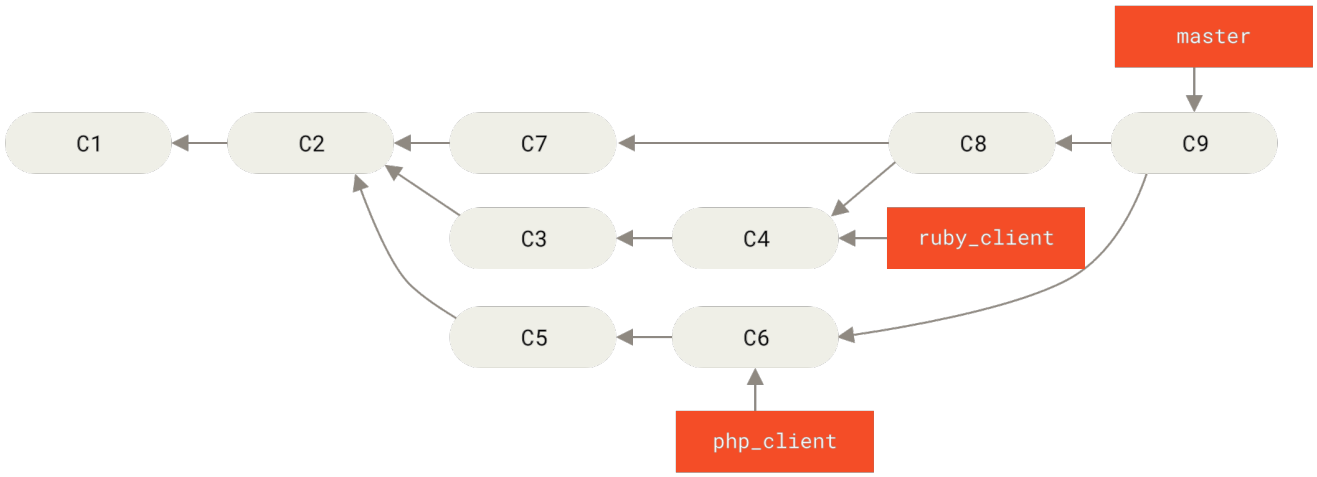


Figure 73. Após um merge de branch de tópico

Esse é provavelmente o fluxo de trabalho mais simples, mas pode possivelmente ser problemático se você estiver lidando com projetos maiores ou mais estáveis, onde você quer ser muito cuidadoso com o que introduz.

Se você tem um projeto mais importante, talvez queira usar um ciclo de merge de duas fases. Neste cenário, você tem dois branches de longa duração, `master` e `develop`, em que você determina que o `master` é atualizado apenas quando uma versão (release) muito estável é criada, e todo o novo código é integrado ao branch `develop`. Você faz regularmente o push de ambos os branches para o repositório público. Toda vez que você tem um novo branch de tópico para mesclar (Antes de um merge de branch de tópico), você o mescla em `develop` (Após um merge de branch de tópico); em seguida, quando você cria uma tag de um lançamento, você faz um fast-forward do `master` para onde quer que o agora estável branch `develop` esteja (Após um lançamento (release) do projeto).

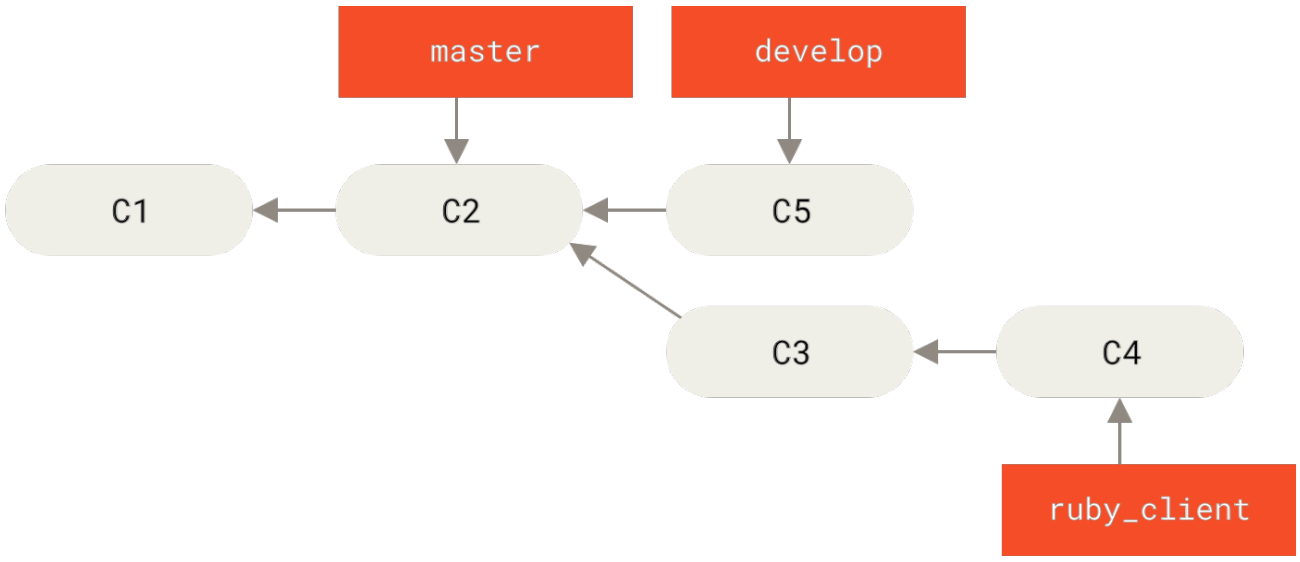


Figure 74. Antes de um merge de branch de tópico

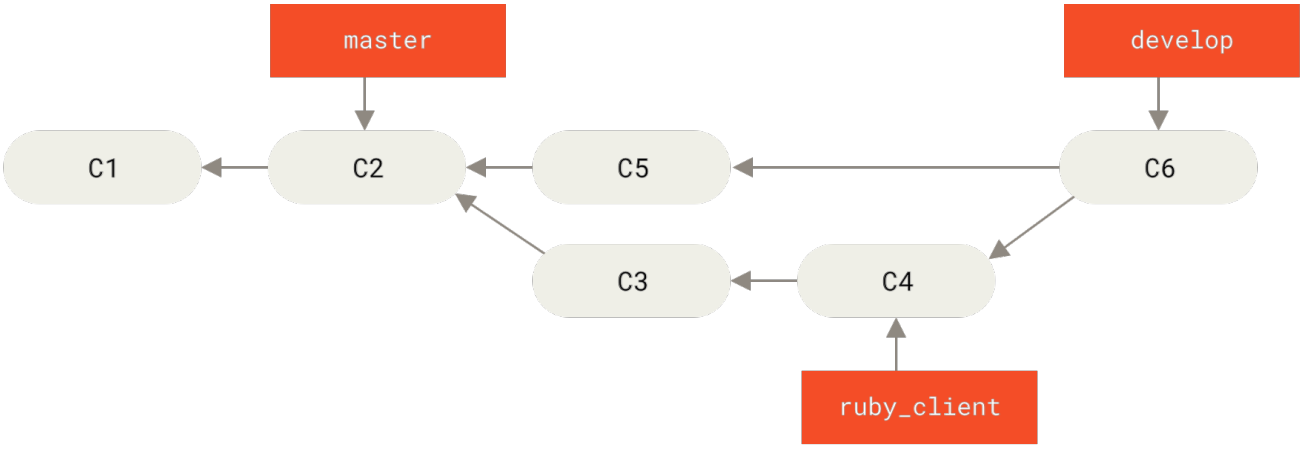


Figure 75. Após um merge de branch de tópico

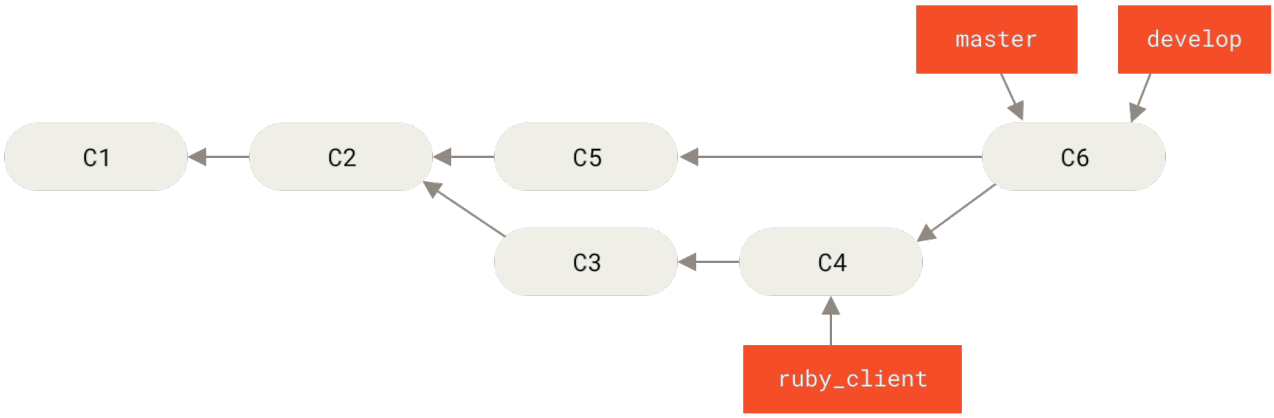


Figure 76. Após um lançamento (release) do projeto

Desta forma, quando as pessoas clonarem o repositório do seu projeto, elas podem fazer o checkout do `master` para construir a versão estável mais recente e manter-se atualizadas com facilidade, ou podem fazer o checkout do `develop`, que é o conteúdo mais inovador (cutting-edge). Você também pode estender esse conceito tendo um branch `integrate` onde todo o trabalho é mesclado. Então, quando a base de código nesse branch estiver estável e passar nos testes, você o mescla em um branch `develop`; e quando ele provar estar estável por um tempo, você faz um fast-forward no seu branch `master`.

Fluxos de Trabalho de Mesclagem em Grande Escala

O projeto Git tem quatro branches de longa duração: `master`, `next`, e `seen` (antigamente 'pu' — proposed updates, ou atualizações propostas) para trabalhos novos, e `maint` para backports de manutenção. Quando um novo trabalho é introduzido por contribuidores, ele é coletado em branches de tópicos no repositório do mantenedor de uma maneira semelhante à que descrevemos (veja Gerenciando uma série complexa de branches de tópicos contribuídos paralelamente). Neste ponto, os tópicos são avaliados para determinar se eles são seguros e estão prontos para consumo ou se precisam de mais trabalho. Se forem seguros, eles são mesclados em `next`, e esse branch recebe um push para que todos possam testar os tópicos integrados juntos.

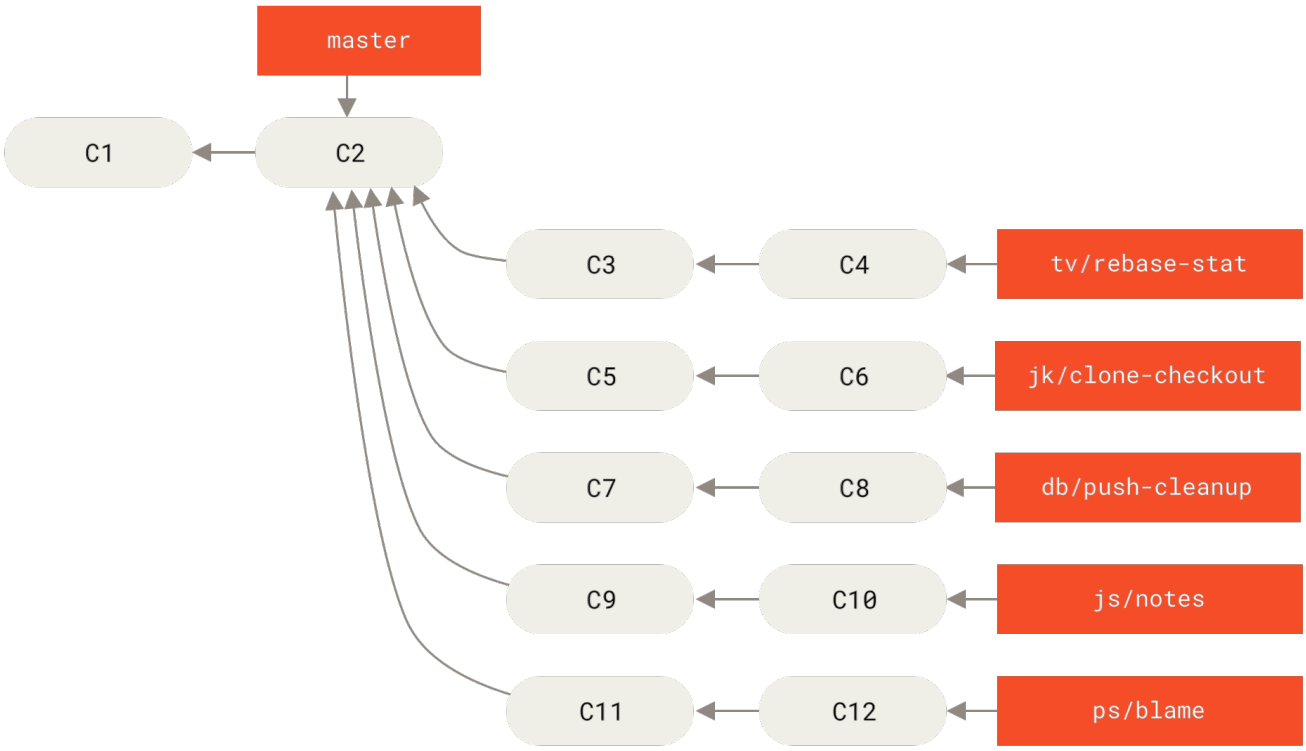


Figure 77. Gerenciando uma série complexa de branches de tópicos contribuídos paralelamente

Se os tópicos ainda precisarem de trabalho, eles são mesclados em `seen`. Quando for determinado que eles estão totalmente estáveis, os tópicos são mesclados novamente em `master`. Os branches `next` e `seen` são então reconstruídos a partir de `master`. Isso significa que o `master` quase sempre se move para frente, o `next` sofre rebase ocasionalmente e o `seen` sofre rebase ainda com mais frequência:

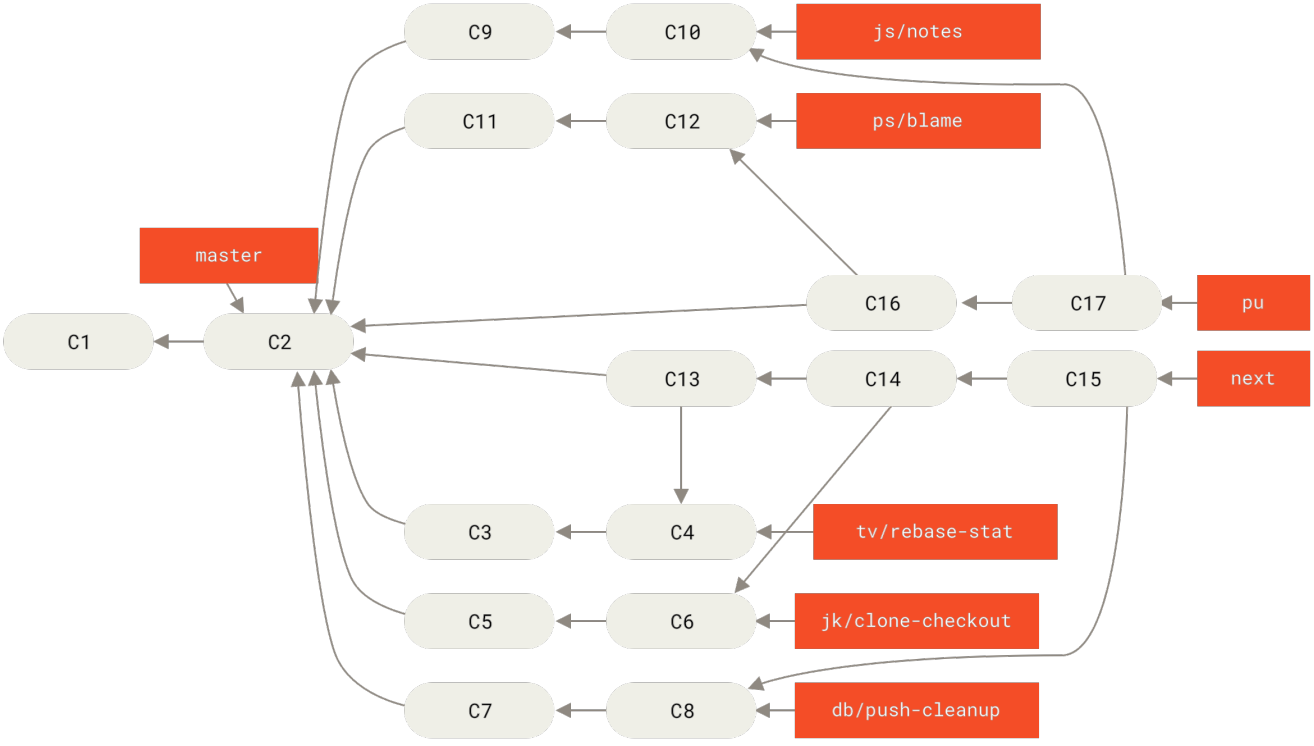


Figure 78. Mesclando branches de tópicos contribuídos em branches de integração de longo prazo

Quando um branch de tópico finalmente é mesclado no `master`, ele é removido do repositório. O

projeto Git também tem um branch `maint` originado a partir do fork do último lançamento, a fim de fornecer patches de backport caso um lançamento de manutenção seja necessário. Assim, quando você clona o repositório Git, você tem quatro branches nos quais pode fazer checkout para avaliar o projeto em diferentes estágios de desenvolvimento, dependendo do quão inovador você deseja ser ou de como deseja contribuir; e o mantenedor tem um fluxo de trabalho estruturado para ajudá-lo a examinar (vet) novas contribuições. O fluxo de trabalho do projeto Git é especializado. Para entender claramente isso, você poderia conferir o Guia do Mantenedor do Git.

Fluxos de Trabalho de Rebase e Cherry-Pick

Outros mantenedores preferem fazer rebase ou cherry-pick do trabalho contribuído no topo do seu branch `master`, em vez de fazer o merge dele, para manter um histórico majoritariamente linear. Quando você tem trabalho em um branch de tópico e determina que deseja integrá-lo, você se move para esse branch e executa o comando rebase para reconstruir as alterações no topo do seu branch `master` (ou `develop`, e assim por diante) atual. Se isso funcionar bem, você pode fazer um fast-forward no seu branch `master`, e acabará com um histórico de projeto linear.

A outra maneira de mover o trabalho introduzido de um branch para outro é fazer o cherry-pick dele. Um cherry-pick no Git é como um rebase para um único commit. Ele pega o patch que foi introduzido em um commit e tenta reaplicá-lo no branch em que você está no momento. Isso é útil se você tem vários commits num branch de tópico e quer integrar apenas um deles, ou se tem apenas um commit num branch de tópico e prefere fazer o cherry-pick em vez de executar o rebase. Por exemplo, suponha que você tenha um projeto parecido com este:

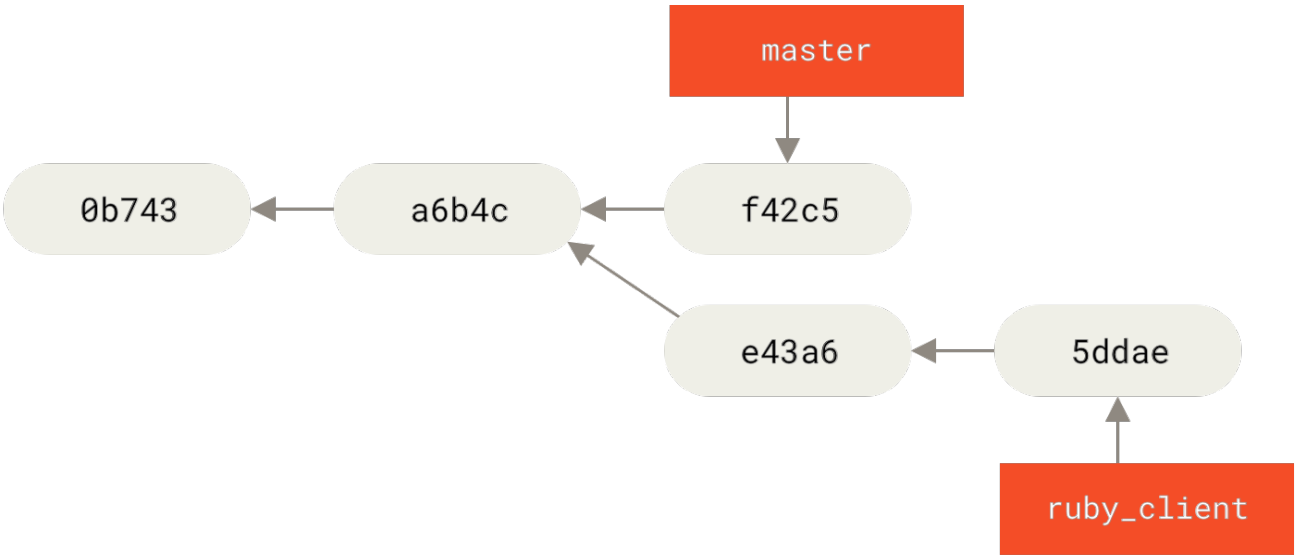


Figure 79. Exemplo de histórico antes de um cherry-pick

Se você quiser puxar (pull) o commit `e43a6` para o seu branch `master`, você pode executar:

```
$ git cherry-pick e43a6
Finished one cherry-pick.
[master]: created a0a41a9: "More friendly message when locking the index fails."
 3 files changed, 17 insertions(+), 3 deletions(-)
```

Isso puxa a mesma alteração introduzida em `e43a6`, mas você obtém um novo valor SHA-1 de commit, porque a data de aplicação é diferente. Agora o seu histórico se parece com isso:

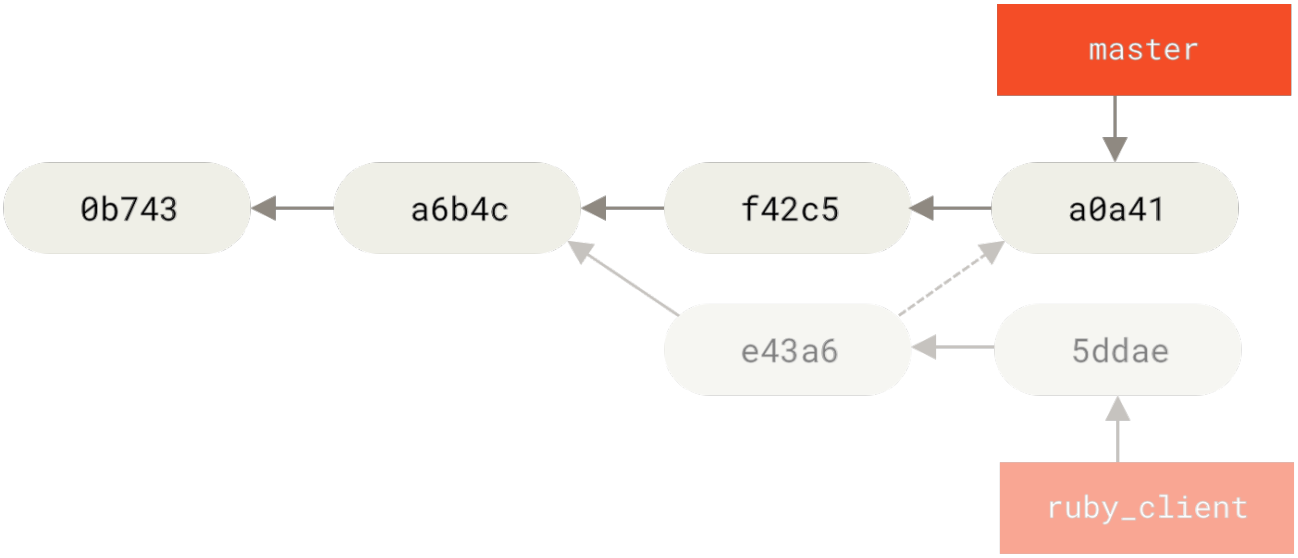


Figure 80. Histórico após fazer o cherry-pick de um commit num branch de tópico

Agora você pode remover o seu branch de tópico e descartar os commits que você não quis puxar.

Rerere

Se você está fazendo muitos merges e rebases, ou está mantendo um branch de tópico de longa vida, o Git tem um recurso chamado "rerere" que pode ajudar.

Rerere significa "reuse recorded resolution" (reutilizar resolução registrada) — é uma forma de atalhar a resolução manual de conflitos. Quando o rerere está ativado, o Git mantém um conjunto de pré e pós-imagens de merges bem-sucedidos e, se perceber que há um conflito exatamente igual a um que você já corrigiu, ele usará a correção da última vez, sem incomodá-lo com isso.

Esse recurso vem em duas partes: uma configuração e um comando. A configuração é `rerere.enabled`, e é útil o suficiente para ser colocada em sua configuração global:

```
$ git config --global rerere.enabled true
```

Agora, sempre que você fizer um merge que resolva conflitos, a resolução será registrada no cache, caso você precise dela no futuro.

Se precisar, você pode interagir com o cache do rerere usando o comando `git rerere`. Quando é invocado sozinho, o Git verifica seu banco de dados de resoluções e tenta encontrar uma correspondência com quaisquer conflitos de merge atuais e resolvê-los (embora isso seja feito automaticamente se `rerere.enabled` estiver definido como `true`). Existem também subcomandos para ver o que será registrado, para apagar resoluções específicas do cache e para limpar todo o cache. Cobriremos o rerere em mais detalhes em Rerere.

Adicionando Tags aos Seus Lançamentos

Quando você tiver decidido cortar um lançamento (release), provavelmente desejará atribuir uma tag para poder recriar esse lançamento em qualquer momento no futuro. Você pode criar uma nova tag conforme discutido em Fundamentos do Git. Se você decidir assinar a tag como mantenedor, a marcação com tag pode se parecer com isso:

```
$ git tag -s v1.5 -m 'my signed 1.5 tag'
You need a passphrase to unlock the secret key for
user: "Scott Chacon <schacon@gmail.com>"
1024-bit DSA key, ID F721C45A, created 2009-02-09
```

Se você realmente assinar as suas tags, pode ter o problema de distribuir a chave pública PGP usada para assinar as suas tags. O mantenedor do projeto Git resolveu esse problema incluindo a sua chave pública como um blob no repositório e então adicionando uma tag que aponta diretamente para esse conteúdo. Para fazer isso, você pode descobrir qual chave deseja executando `gpg --list-keys`:

```
$ gpg --list-keys
/Users/schacon/.gnupg/pubring.gpg
---------------------------------
pub   1024D/F721C45A 2009-02-09 [expires: 2010-02-09]
uid                  Scott Chacon <schacon@gmail.com>
sub   2048g/45D02282 2009-02-09 [expires: 2010-02-09]
```

Em seguida, você pode importar diretamente a chave para o banco de dados do Git, exportando-a e passando o resultado via pipe pelo `git hash-object`, que grava um novo blob com esse conteúdo no Git e retorna o SHA-1 do blob para você:

```
$ gpg -a --export F721C45A | git hash-object -w --stdin
659ef797d181633c87ec71ac3f9ba29fe5775b92
```

Agora que você tem o conteúdo de sua chave no Git, pode criar uma tag que aponte diretamente para ele, especificando o novo valor SHA-1 que o comando `hash-object` lhe deu:

```
$ git tag -a maintainer-pgp-pub 659ef797d181633c87ec71ac3f9ba29fe5775b92
```

Se você rodar `git push --tags`, a tag `maintainer-pgp-pub` será compartilhada com todos. Se alguém quiser verificar uma tag, eles podem importar diretamente a sua chave PGP, retirando o blob diretamente do banco de dados e importando-o para o GPG:

```
$ git show maintainer-pgp-pub | gpg --import
```

Eles podem usar essa chave para verificar todas as suas tags assinadas. Além disso, se você incluir

instruções na mensagem da tag, rodar `git show <tag>` permitirá que você dê ao usuário final instruções mais específicas sobre a verificação de tags.

Gerando um Número de Build

Como o Git não possui números monotonicamente crescentes como 'v123' ou o equivalente para acompanhar cada commit, se você deseja ter um nome legível por humanos para acompanhar um commit, você pode rodar o `git describe` nesse commit. Em resposta, o Git gera uma string que consiste no nome da tag mais recente anterior a esse commit, seguida pelo número de commits desde essa tag e, finalmente, por um valor SHA-1 parcial do commit que está sendo descrito (prefixado com a letra "g" que significa Git):

```
$ git describe master
v1.6.2-rc1-20-g8c5b85c
```

Dessa forma, você pode exportar um snapshot ou build e nomeá-lo com algo compreensível para as pessoas. Na verdade, se você compilar o Git a partir do código-fonte clonado do repositório Git, o `git --version` lhe dará algo que se parece com isto. Se você estiver descrevendo um commit que marcou diretamente com uma tag, ele lhe dará simplesmente o nome da tag.

Por padrão, o comando `git describe` requer tags anotadas (tags criadas com a flag `-a` ou `-s`); se você quiser aproveitar também as tags leves (lightweight tags, não anotadas), adicione a opção `--tags` ao comando. Você também pode usar essa string como o alvo de um comando `git checkout` ou `git show`, embora ele dependa do valor SHA-1 abreviado no final, portanto pode não ser válido para sempre. Por exemplo, o kernel do Linux pulou recentemente de 8 para 10 caracteres para garantir a exclusividade do objeto SHA-1, portanto, nomes de saída mais antigos do `git describe` foram invalidados.

Preparando um Lançamento (Release)

Agora você quer lançar (release) um build. Uma das coisas que você vai querer fazer é criar um arquivo compactado (archive) do snapshot mais recente do seu código para aquelas pobres almas que não usam o Git. O comando para fazer isso é `git archive`:

```
$ git archive master --prefix='project/' | gzip > `git describe master`.tar.gz
$ ls *.tar.gz
v1.6.2-rc1-20-g8c5b85c.tar.gz
```

Se alguém abrir esse tarball, obterá o snapshot mais recente do seu projeto dentro de um diretório `project`. Você também pode criar um arquivo zip da mesma maneira, mas passando a opção `--format=zip` para o `git archive`:

```
$ git archive master --prefix='project/' --format=zip > `git describe master`.zip
```

Agora você tem um belo tarball e um arquivo zip do lançamento do seu projeto que você pode fazer upload para o seu site ou enviar por e-mail para as pessoas.

O Shortlog

É hora de enviar um e-mail para a sua lista de discussão de pessoas que querem saber o que está acontecendo no seu projeto. Uma maneira legal de obter rapidamente uma espécie de changelog do que foi adicionado ao seu projeto desde o seu último lançamento ou e-mail é usar o comando `git shortlog`. Ele resume todos os commits no intervalo que você fornece; por exemplo, o seguinte lhe dá um resumo de todos os commits desde o seu último lançamento, se o seu último lançamento se chamasse `v1.0.1`:

```
$ git shortlog --no-merges master --not v1.0.1
Chris Wanstrath (6):
      Add support for annotated tags to Grit::Tag
      Add packed-refs annotated tag support.
      Add Grit::Commit#to_patch
      Update version and History.txt
      Remove stray `puts`
      Make ls_tree ignore nils

Tom Preston-Werner (4):
      fix dates in history
      dynamic version method
      Version bump to 1.0.2
      Regenerated gemspec for version 1.0.2
```

Você obtém um resumo limpo de todos os commits desde a `v1.0.1`, agrupados por autor, que você pode enviar por e-mail para a sua lista.

Resumo

Você deve se sentir bastante confortável contribuindo para um projeto no Git, bem como mantendo o seu próprio projeto ou integrando contribuições de outros usuários. Parabéns por ser um desenvolvedor Git eficaz! No próximo capítulo, você aprenderá sobre como usar o maior e mais popular serviço de hospedagem Git, o GitHub.

GitHub

O GitHub é o maior provedor de hospedagem de repositórios Git, e é o ponto central de colaboração para milhões de desenvolvedores e projetos. Uma grande porcentagem de todos os repositórios Git está hospedada no GitHub, e muitos projetos de código aberto o utilizam para hospedagem Git, rastreamento de problemas (issue tracking), revisão de código e outras coisas. Portanto, embora não seja uma parte direta do projeto de código aberto Git, há uma boa chance de você querer ou precisar interagir com o GitHub em algum momento enquanto usa o Git profissionalmente.

Este capítulo é sobre como usar o GitHub de forma eficaz. Nós abordaremos como se inscrever e gerenciar uma conta, criar e usar repositórios Git, fluxos de trabalho comuns para contribuir com projetos e aceitar contribuições para os seus, a interface programática do GitHub e várias dicas para facilitar a sua vida em geral.

Se você não estiver interessado em usar o GitHub para hospedar seus próprios projetos ou colaborar com outros projetos que estão hospedados no GitHub, você pode pular para Ferramentas do Git com segurança.



Alterações de Interface

É importante notar que, como muitos sites ativos, os elementos de interface de usuário (UI) nestas capturas de tela estão sujeitos a alterações ao longo do tempo. Esperamos que a ideia geral do que estamos tentando realizar aqui ainda esteja lá, mas se você quiser versões mais atualizadas dessas telas, as versões online deste livro podem ter capturas de tela mais novas.

Configuração e Ajustes de Conta

A primeira coisa que você precisa fazer é configurar uma conta de usuário gratuita. Basta visitar https://github.com, escolher um nome de usuário que ainda não tenha sido usado, fornecer um endereço de e-mail e uma senha, e clicar no botão verde grande “Sign up for GitHub”.

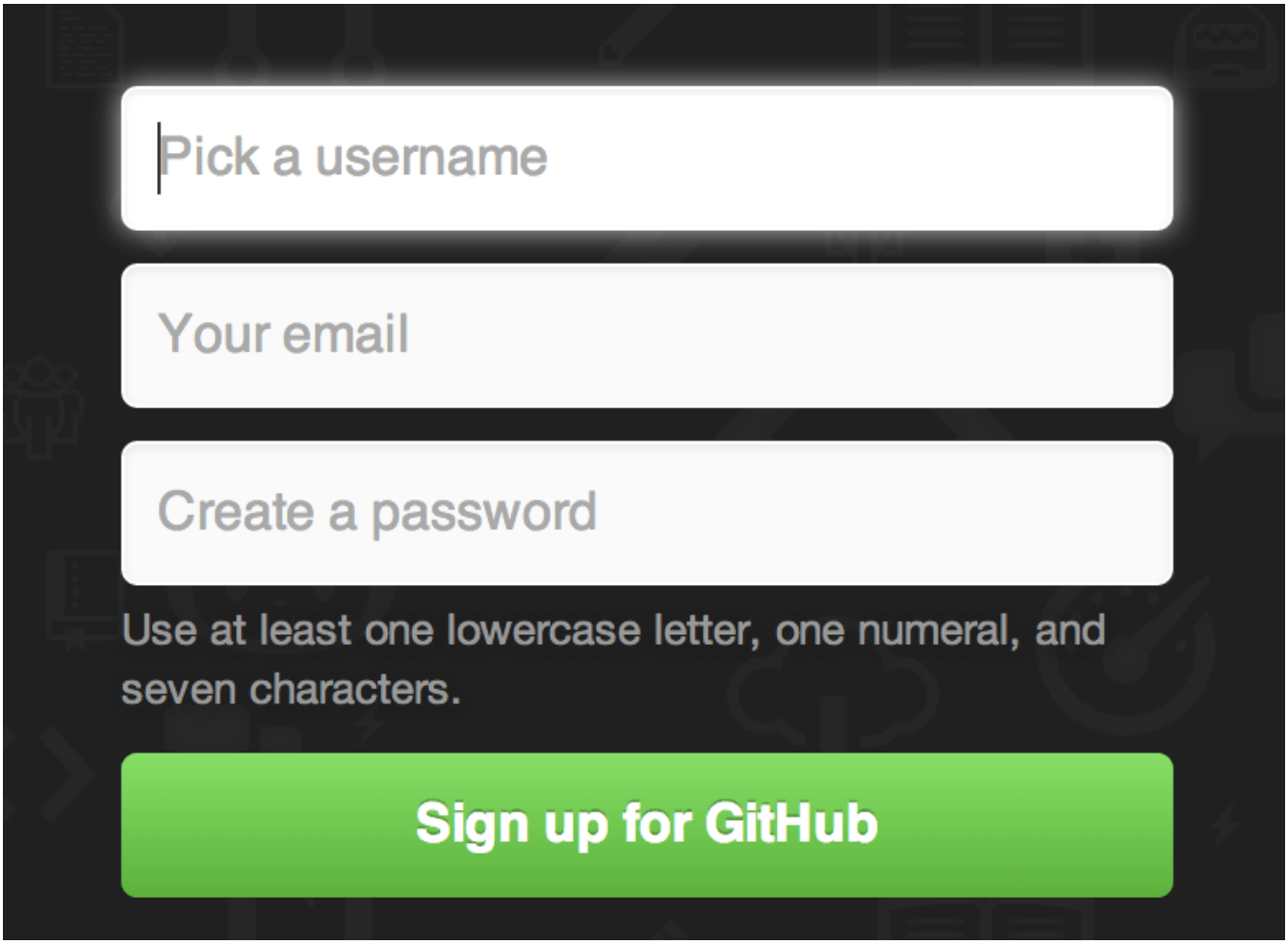


Figure 81. O formulário de inscrição do GitHub

A próxima coisa que você verá é a página de preços para planos atualizados, mas é seguro ignorá-la por enquanto. O GitHub lhe enviará um e-mail para verificar o endereço que você forneceu. Vá em frente e faça isso; é muito importante (como veremos mais adiante).



O GitHub fornece quase toda a sua funcionalidade em contas gratuitas, exceto alguns recursos avançados.

Os planos pagos do GitHub incluem ferramentas e recursos avançados, bem como limites maiores para serviços gratuitos, mas não os abordaremos neste livro. Para obter mais informações sobre os planos disponíveis e sua comparação, visite https://github.com/pricing.

Clicar no logotipo Octocat no canto superior esquerdo da tela levará você para a página do seu painel. Agora você está pronto para usar o GitHub.

Acesso SSH

A partir de agora, você está totalmente apto a se conectar com repositórios Git usando o protocolo `https://`, autenticando-se com o nome de usuário e senha que você acabou de configurar. No entanto, para simplesmente clonar projetos públicos, você nem precisa se inscrever - a conta que acabamos de criar entra em jogo quando fazemos fork de projetos e fazemos push para nossos forks um pouco mais tarde.

Se você quiser usar remotos SSH, precisará configurar uma chave pública. Se você ainda não tiver uma, veja Gerando a sua Chave Pública SSH. Abra as configurações da sua conta usando o link no canto superior direito da janela:

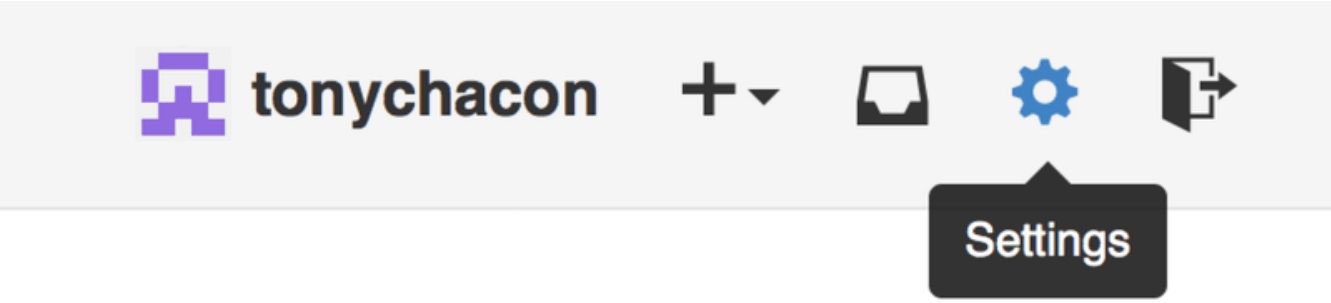


Figure 82. O link “Account settings”

Em seguida, selecione a seção “SSH keys” ao longo do lado esquerdo.

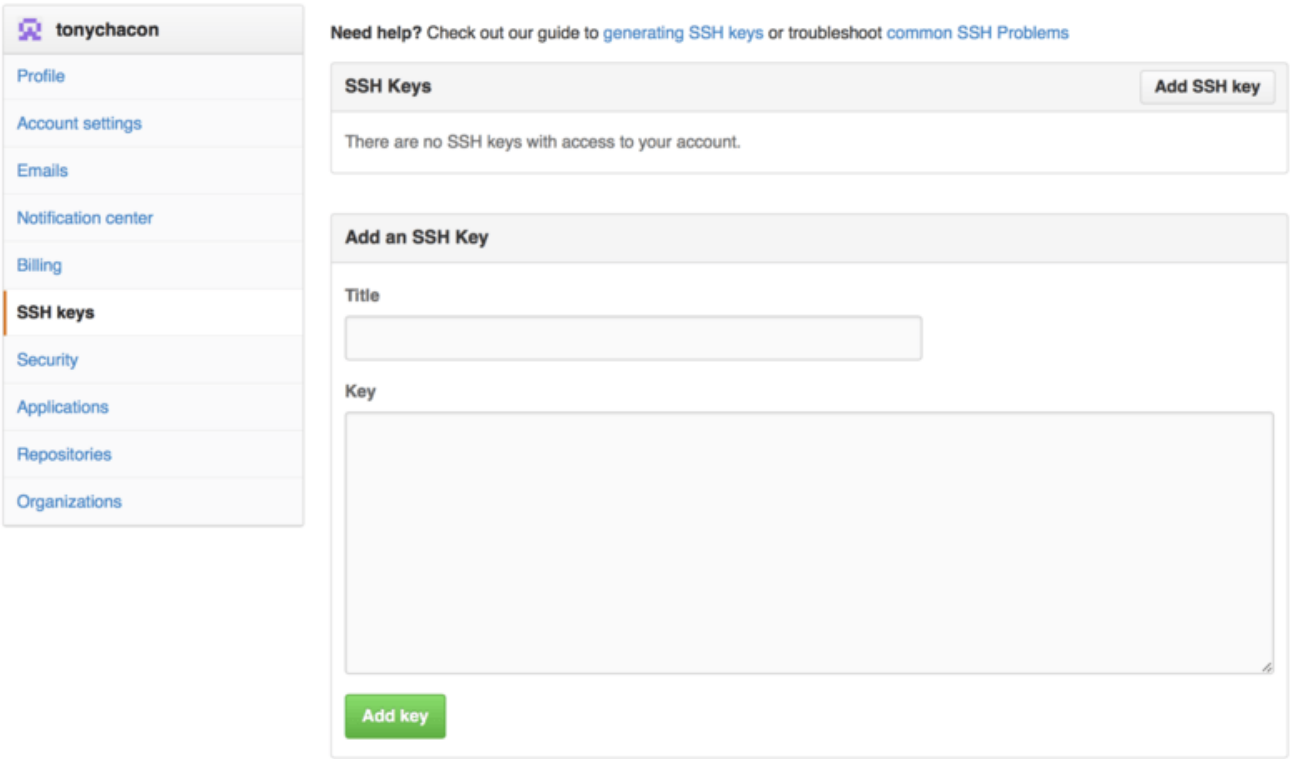


Figure 83. O link “SSH keys”

A partir daí, clique no botão “Add an SSH key”, dê um nome à sua chave, cole o conteúdo do seu arquivo de chave pública `~/.ssh/id_rsa.pub` (ou qualquer que seja o nome dele) na área de texto e clique em “Add key”.



Certifique-se de nomear sua chave SSH com algo que você possa lembrar. Você pode nomear cada uma de suas chaves (ex: "Meu Laptop" ou "Conta de Trabalho") para que, se precisar revogar uma chave mais tarde, possa saber facilmente qual está procurando.

O Seu Avatar

Em seguida, se desejar, você pode substituir o avatar gerado para você por uma imagem de sua escolha. Primeiro vá para a aba “Profile” (perfil, acima da aba SSH Keys) e clique em “Upload new picture” (carregar nova imagem).

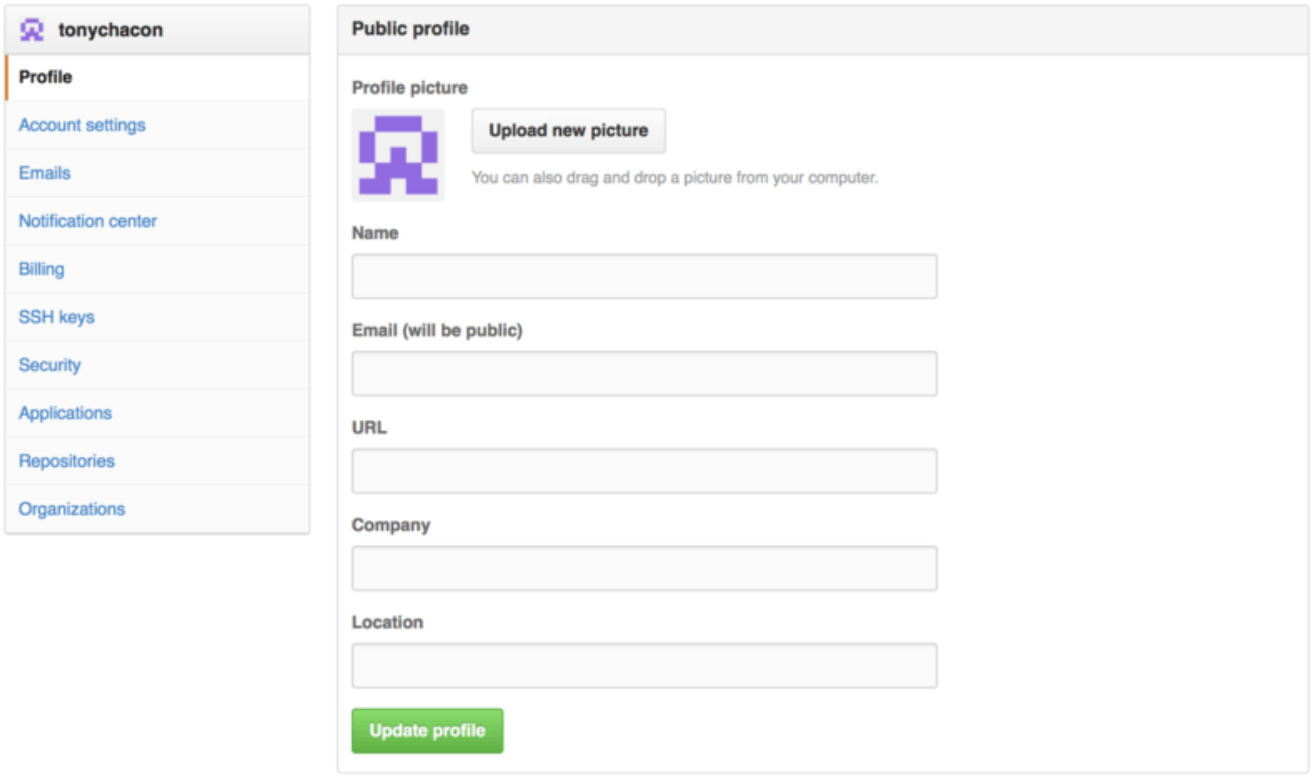


Figure 84. O link “Profile”

Nós escolheremos uma cópia do logotipo do Git que está no nosso disco rígido e então teremos a chance de recortá-lo.

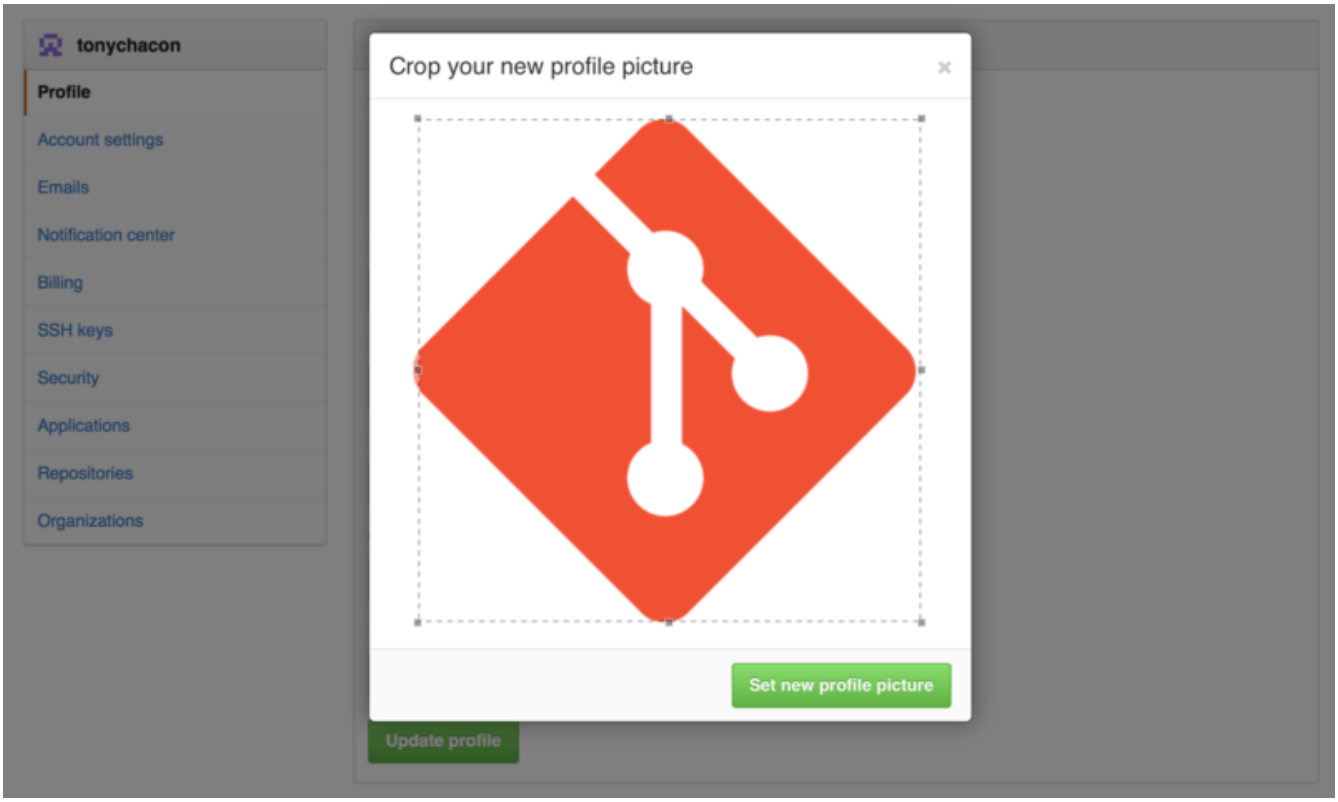


Figure 85. Recorte o seu avatar carregado

Agora, em qualquer lugar que você interagir no site, as pessoas verão o seu avatar ao lado do seu nome de usuário.

Se por acaso você tiver carregado um avatar no popular serviço Gravatar (frequentemente usado para contas WordPress), esse avatar será usado por padrão e você não precisa fazer esta etapa.

Os Seus Endereços de E-mail

A maneira como o GitHub mapeia os seus commits do Git para o seu usuário é pelo endereço de e-mail. Se você usa vários endereços de e-mail nos seus commits e quer que o GitHub os vincule adequadamente, precisa adicionar todos os endereços de e-mail que você usou na seção Emails da seção de administração.

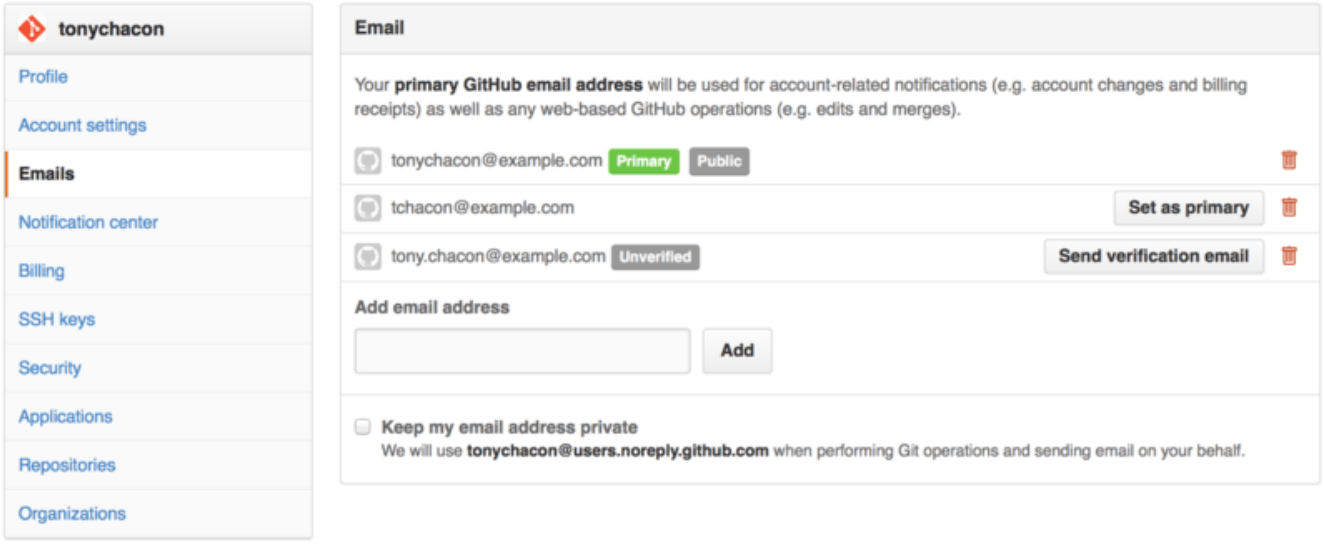


Figure 86. Adicione todos os seus endereços de e-mail

Em Adicione todos os seus endereços de e-mail podemos ver alguns dos diferentes estados que são possíveis. O endereço do topo é verificado e definido como o endereço principal, o que significa que é lá que você receberá quaisquer notificações e recibos. O segundo endereço é verificado e, portanto, pode ser definido como principal se você desejar trocá-los. O último endereço não é verificado, o que significa que você não pode torná-lo o seu endereço principal. Se o GitHub vir qualquer um desses nas mensagens de commit em qualquer repositório no site, ele será vinculado ao seu usuário agora.

Autenticação de Dois Fatores

Finalmente, para segurança extra, você definitivamente deve configurar a Autenticação de Dois Fatores ou "2FA" (Two-factor Authentication). A Autenticação de Dois Fatores é um mecanismo de autenticação que está se tornando cada vez mais popular recentemente para mitigar o risco de a sua conta ser comprometida se a sua senha for roubada de alguma forma. Ao ativá-la, o GitHub pedirá dois métodos diferentes de autenticação, de modo que, se um deles for comprometido, um invasor não conseguirá acessar a sua conta.

Você pode encontrar a configuração de Autenticação de Dois Fatores na aba Security (Segurança) das configurações da sua Conta.

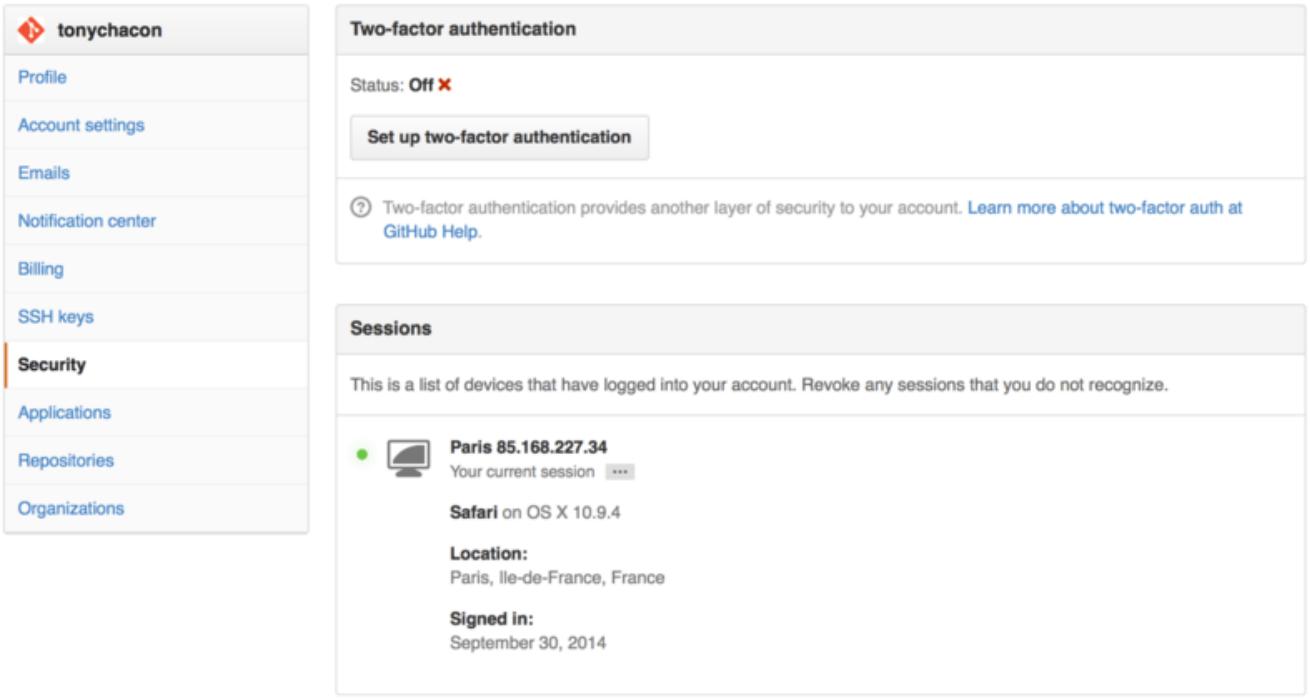


Figure 87. 2FA na aba Security

Se você clicar no botão "Set up two-factor authentication", ele o levará a uma página de configuração onde você pode escolher usar um aplicativo de telefone para gerar o seu código secundário (uma "senha de uso único baseada em tempo" ou time based one-time password), ou pode fazer com que o GitHub lhe envie um código via SMS cada vez que você precisar fazer login.

Depois de escolher o método de sua preferência e seguir as instruções para configurar o 2FA, a sua conta ficará um pouco mais segura e você terá que fornecer um código além da sua senha sempre que fizer login no GitHub.

Contribuindo para um Projeto

Agora que a nossa conta está configurada, vamos examinar alguns detalhes que podem ser úteis para ajudá-lo a contribuir com um projeto existente.

Fazendo Fork de Projetos

Se você quiser contribuir com um projeto existente ao qual não tem acesso de push, você pode fazer um "fork" do projeto. Quando você faz um "fork" de um projeto, o GitHub faz uma cópia do projeto que é inteiramente sua; ela vive no seu namespace e você pode fazer push para ela.



Historicamente, o termo "fork" tem sido um tanto negativo em seu contexto, significando que alguém levou um projeto de código aberto para uma direção diferente, às vezes criando um projeto concorrente e dividindo os contribuidores. No GitHub, um "fork" é simplesmente o mesmo projeto no seu próprio namespace, permitindo que você faça alterações em um projeto publicamente como uma forma de contribuir de maneira mais aberta.

Dessa forma, os projetos não precisam se preocupar em adicionar usuários como colaboradores para lhes dar acesso de push. As pessoas podem fazer o fork de um projeto, fazer push nele e contribuir com as suas alterações de volta ao repositório original criando o que é chamado de Pull Request, que abordaremos a seguir. Isso abre um tópico de discussão com revisão de código, e o proprietário e o contribuidor podem então se comunicar sobre a alteração até que o proprietário esteja satisfeito com ela, momento em que o proprietário pode mesclá-la.

Para fazer o fork de um projeto, visite a página do projeto e clique no botão "Fork" no canto superior direito da página.

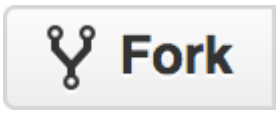


Figure 88. O botão "Fork"

Depois de alguns segundos, você será levado para a página do seu novo projeto, com a sua própria cópia editável do código.

O Fluxo do GitHub (GitHub Flow)

O GitHub é projetado em torno de um fluxo de trabalho de colaboração particular, centrado em Pull Requests. Esse fluxo funciona quer você esteja colaborando com uma equipe unida em um único repositório compartilhado, ou com uma empresa distribuída globalmente ou uma rede de estranhos contribuindo para um projeto por meio de dezenas de forks. Ele é centrado no fluxo de trabalho de Branches de Tópicos (Topic Branches) abordado em Ramificação (Branching) no Git.

Eis como isso geralmente funciona:

1. Faça um fork do projeto.
2. Crie um branch de tópico a partir do `master`.

3. Faça alguns commits para melhorar o projeto.
4. Faça um push deste branch para o seu projeto no GitHub.
5. Abra um Pull Request no GitHub.
6. Discuta e, opcionalmente, continue a fazer commits.
7. O proprietário do projeto mescla (merge) ou fecha (close) o Pull Request.
8. Sincronize o `master` atualizado de volta para o seu fork.

Este é basicamente o fluxo de trabalho de Gerente de Integração (Integration Manager) abordado em Fluxo de Trabalho do Gerente de Integração (Integration-Manager), mas em vez de usar e-mail para comunicar e revisar as alterações, as equipes usam as ferramentas baseadas na web do GitHub.

Vamos analisar um exemplo de proposição de uma alteração a um projeto de código aberto hospedado no GitHub usando este fluxo.



Você pode usar a ferramenta oficial **GitHub CLI** em vez da interface web do GitHub para a maioria das coisas. A ferramenta pode ser usada em sistemas Windows, macOS e Linux. Vá para a página inicial do GitHub CLI para instruções de instalação e o manual.

Criando um Pull Request

Tony está procurando um código para rodar no seu microcontrolador programável Arduino e encontrou um ótimo arquivo de programa no GitHub em https://github.com/schacon/blink.

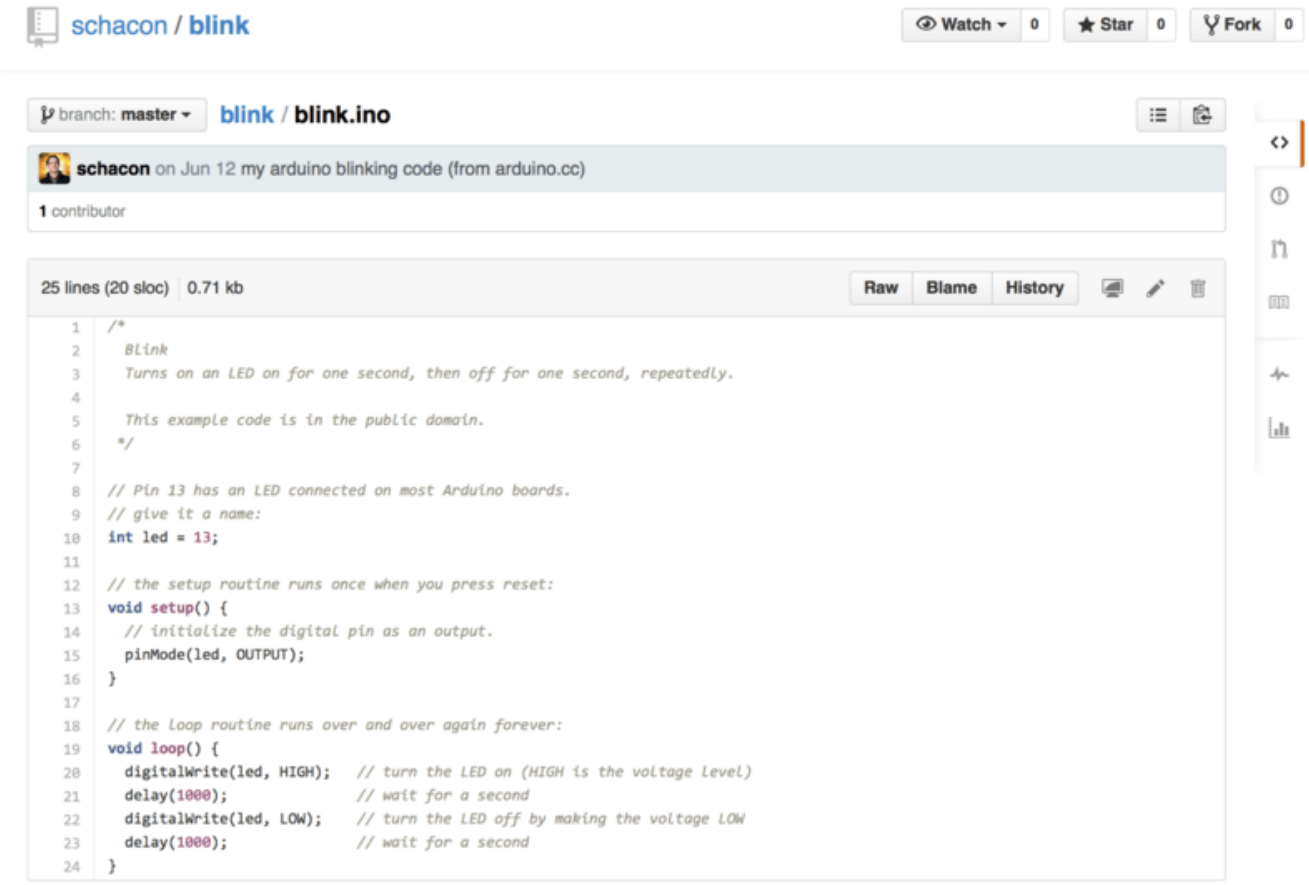


Figure 89. O projeto com o qual queremos contribuir

O único problema é que a taxa de piscar está muito rápida. Achamos que é muito mais agradável esperar 3 segundos em vez de 1 entre cada mudança de estado. Então, vamos melhorar o programa e enviá-lo de volta ao projeto como uma alteração proposta.

Primeiro, clicamos no botão 'Fork' (Fazer Fork) como mencionado anteriormente para obtermos a nossa própria cópia do projeto. O nosso nome de usuário aqui é “tonychacon”, então a nossa cópia deste projeto está em https://github.com/tonychacon/blink e é lá que podemos editá-lo. Vamos cloná-lo localmente, criar um branch de tópico, fazer a alteração no código e finalmente fazer o push dessa alteração de volta para o GitHub.

```
$ git clone https://github.com/tonychacon/blink ①
Cloning into 'blink'...

$ cd blink
$ git checkout -b slow-blink ②
Switched to a new branch 'slow-blink'

$ sed -i '' 's/1000/3000/' blink.ino (macOS) ③
# If you're on a Linux system, do this instead:
# $ sed -i 's/1000/3000/' blink.ino ③

$ git diff --word-diff ④
diff --git a/blink.ino b/blink.ino
index 15b9911..a6cc5a5 100644
```

```
--- a/blink.ino
+++ b/blink.ino
@@ -18,7 +18,7 @@ void setup() {
// the loop routine runs over and over again forever:
void loop() {
  digitalWrite(led, HIGH);   // turn the LED on (HIGH is the voltage level)
  [-delay(1000);-]{+delay(3000);+}               // wait for a second
  digitalWrite(led, LOW);    // turn the LED off by making the voltage LOW
  [-delay(1000);-]{+delay(3000);+}               // wait for a second
}

$ git commit -a -m 'Change delay to 3 seconds' ⑤
[slow-blink 5ca509d] Change delay to 3 seconds
 1 file changed, 2 insertions(+), 2 deletions(-)

$ git push origin slow-blink ⑥
Username for 'https://github.com': tonychacon
Password for 'https://tonychacon@github.com':
Counting objects: 5, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 340 bytes | 0 bytes/s, done.
Total 3 (delta 1), reused 0 (delta 0)
To https://github.com/tonychacon/blink
 * [new branch]      slow-blink -> slow-blink
```

① Clone o nosso fork do projeto localmente.

② Crie um branch de tópico descritivo.

③ Faça a nossa alteração no código.

④ Verifique se a alteração está boa.

⑤ Faça o commit da nossa alteração para o branch de tópico.

⑥ Faça o push do nosso novo branch de tópico de volta para o nosso fork no GitHub.

Agora, se voltarmos para o nosso fork no GitHub, podemos ver que o GitHub notou que fizemos o push de um novo branch de tópico e nos apresenta um grande botão verde para verificar as nossas alterações e abrir um Pull Request para o projeto original.

Alternativamente, você pode ir para a página “Branches” em `https://github.com/<user>/<project>/branches` para localizar o seu branch e abrir um novo Pull Request a partir de lá.

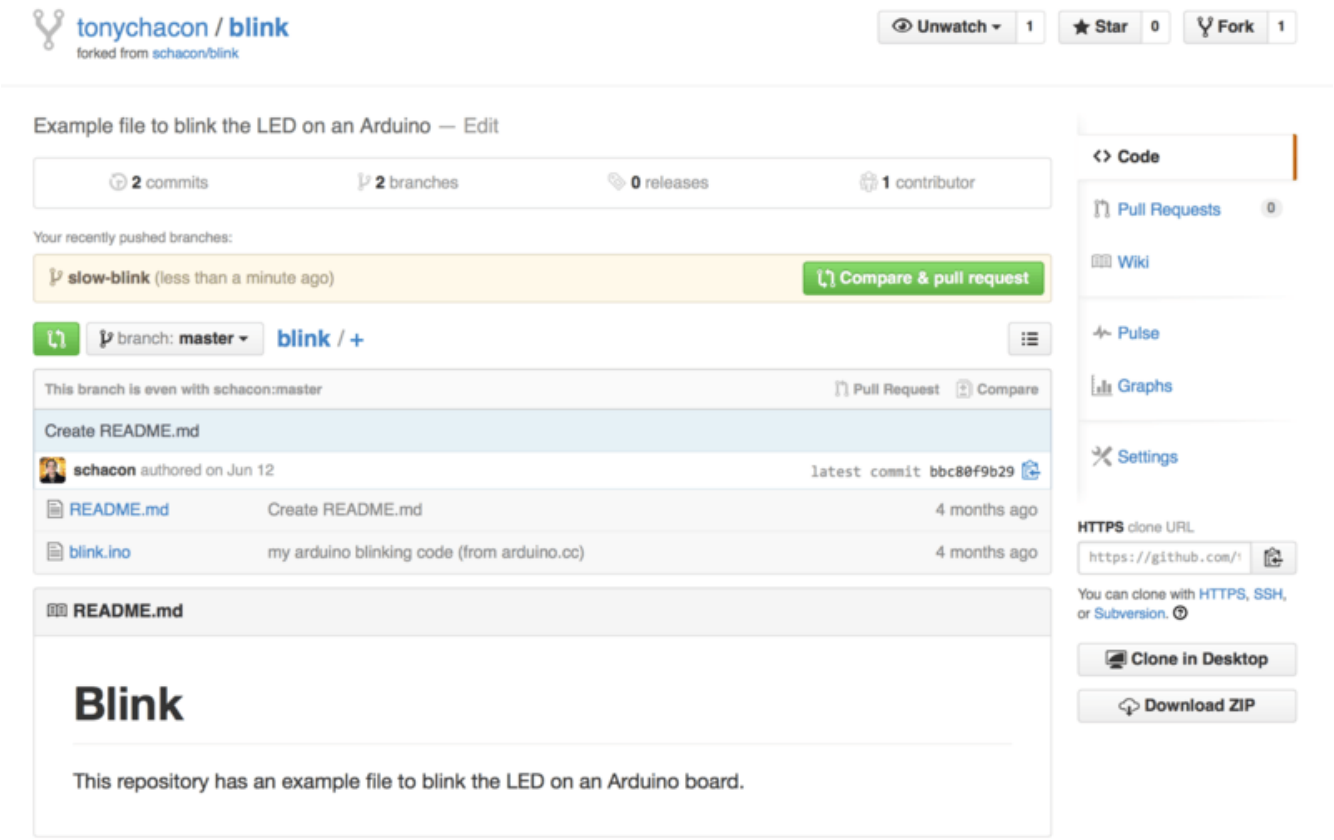


Figure 90. Botão de Pull Request

Se clicarmos naquele botão verde, veremos uma tela que nos pede para dar um título e uma descrição ao nosso Pull Request. Quase sempre vale a pena colocar algum esforço nisso, já que uma boa descrição ajuda o proprietário do projeto original a determinar o que você estava tentando fazer, se as alterações propostas estão corretas e se a aceitação das alterações melhoraria o projeto original.

Também vemos uma lista dos commits em nosso branch de tópico que estão “à frente” (ahead) do branch `master` (neste caso, apenas um) e um diff unificado de todas as alterações que serão feitas caso este branch seja mesclado pelo proprietário do projeto.

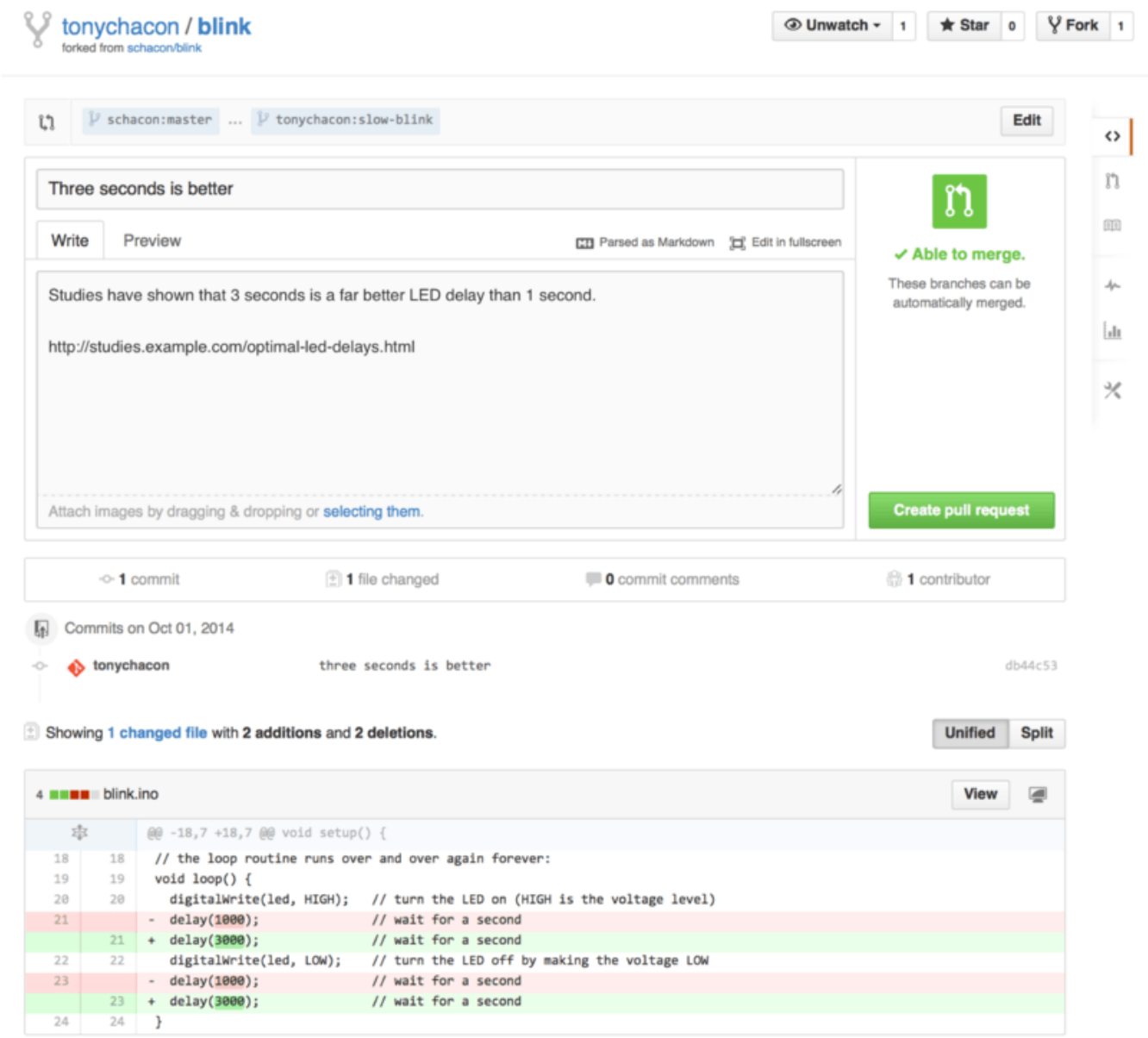


Figure 91. Página de criação de Pull Request

Quando você clica no botão 'Create pull request' nesta tela, o proprietário do projeto do qual você fez fork receberá uma notificação de que alguém está sugerindo uma alteração e fornecerá um link para uma página que contém todas essas informações.



Embora Pull Requests sejam comumente usados para projetos públicos como este quando o contribuidor tem uma alteração completa pronta para ser feita, eles também são frequentemente usados em projetos internos *no início* do ciclo de desenvolvimento. Como você pode continuar fazendo push para o branch de tópico mesmo **após** a abertura do Pull Request, ele costuma ser aberto no início e usado como uma forma de iterar no trabalho em equipe dentro de um contexto, em vez de ser aberto apenas no final do processo.

Iterando em um Pull Request

Neste ponto, o proprietário do projeto pode examinar a alteração sugerida e mesclá-la, rejeitá-la ou comentá-la. Digamos que ele goste da ideia, mas preferisse um tempo um pouco maior para a luz ficar apagada do que acesa.

Onde essa conversa poderia ocorrer por e-mail nos fluxos de trabalho apresentados em Git Distribuído, no GitHub ela acontece online. O proprietário do projeto pode revisar o diff unificado e deixar um comentário clicando em qualquer uma das linhas.

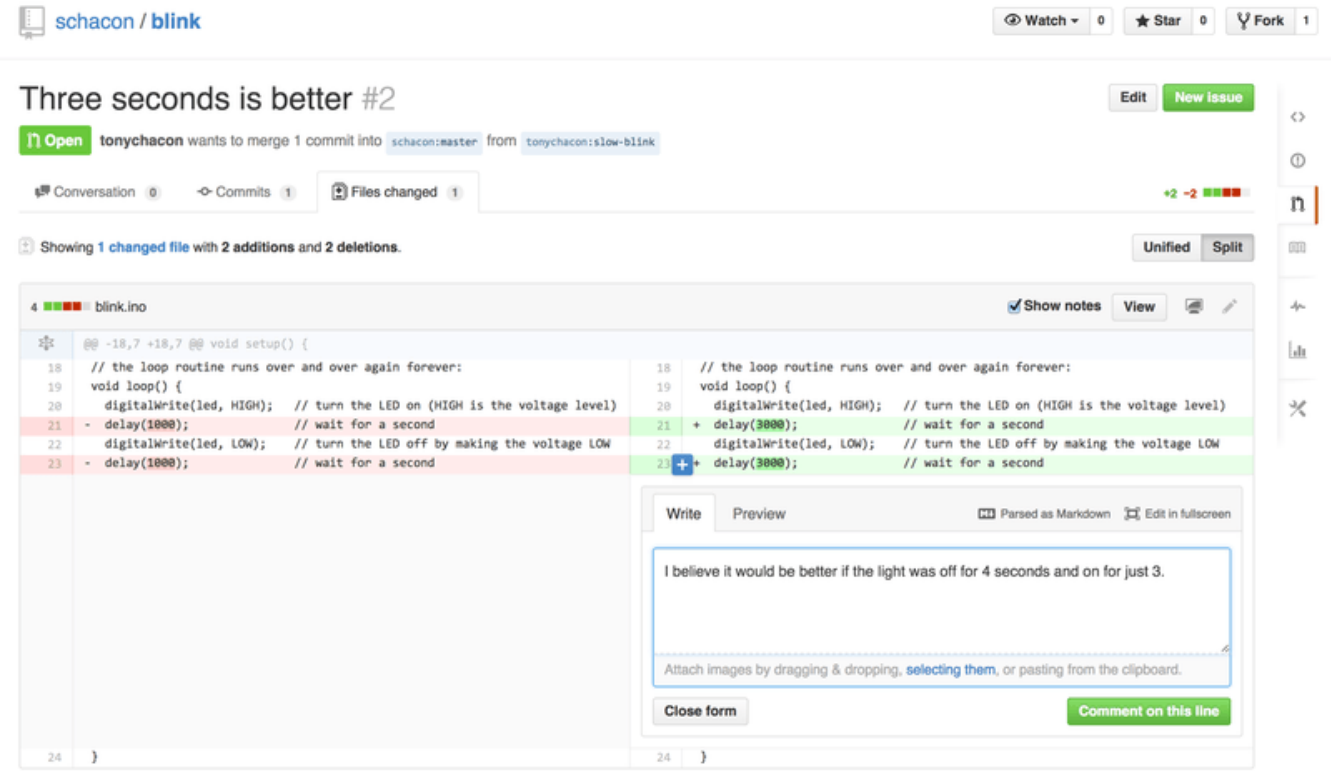


Figure 92. Comentário em uma linha de código específica em um Pull Request

Assim que o mantenedor faz esse comentário, a pessoa que abriu o Pull Request (e, na verdade, qualquer outra pessoa que esteja observando o repositório) receberá uma notificação. Abordaremos a personalização disso mais tarde, mas se tivesse as notificações por e-mail ativadas, Tony receberia um e-mail como este:



Figure 93. Comentários enviados como notificações por e-mail

Qualquer pessoa também pode deixar comentários gerais sobre o Pull Request. Em Página de discussão de Pull Request podemos ver um exemplo do proprietário do projeto tanto comentando em uma linha de código quanto deixando um comentário geral na seção de discussão. Você pode ver que os comentários de código também são trazidos para a conversa.

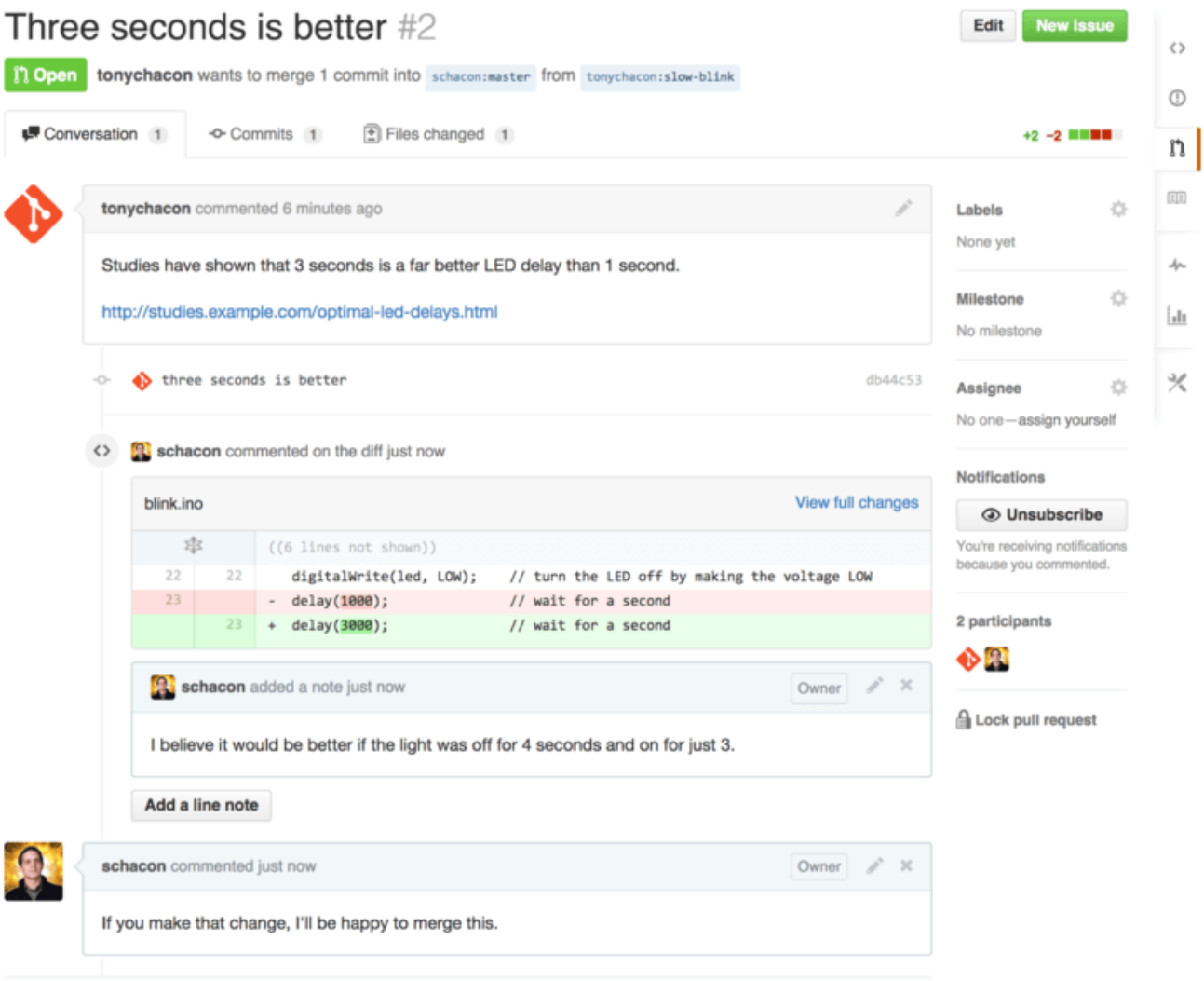


Figure 94. Página de discussão de Pull Request

Agora o contribuidor pode ver o que precisa fazer para que a sua alteração seja aceita. Felizmente, isso é muito simples. Enquanto por e-mail você pode ter que reescrever (re-roll) a sua série e reenviá-la para a lista de discussão, com o GitHub você simplesmente faz um commit no branch de tópico novamente e faz o push, o que atualizará automaticamente o Pull Request. Em Final do Pull Request, você também pode ver que o comentário do código antigo foi ocultado no Pull Request atualizado, uma vez que foi feito em uma linha que já foi alterada.

Adicionar commits a um Pull Request existente não aciona uma notificação, então, assim que Tony envia as suas correções, ele decide deixar um comentário para informar ao proprietário do projeto que fez a alteração solicitada.

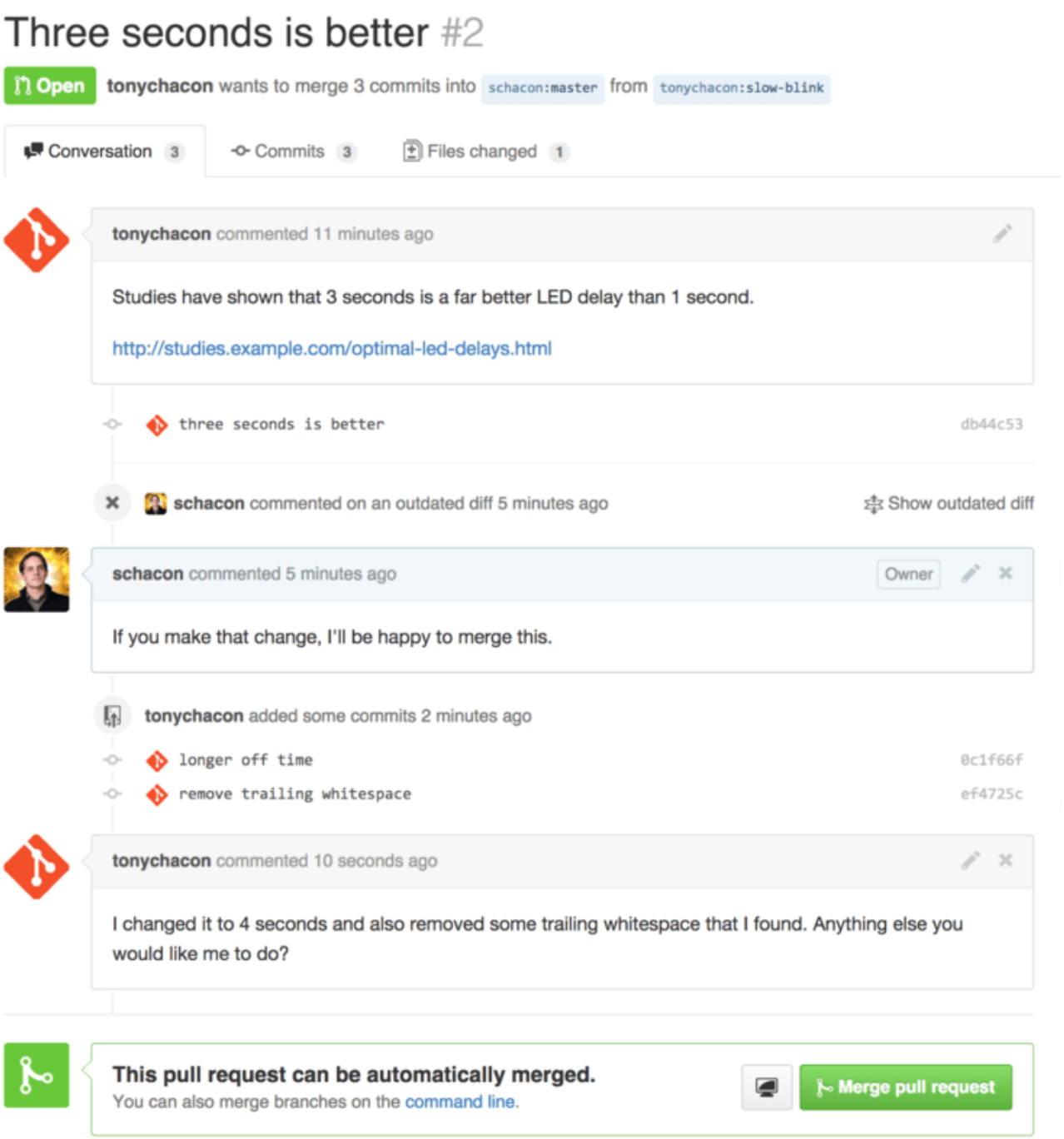


Figure 95. Final do Pull Request

Uma coisa interessante de se notar é que se você clicar na aba “Files Changed” (Arquivos Alterados) neste Pull Request, você obterá o diff “unificado” — isto é, a diferença total agregada que seria introduzida no seu branch principal se este branch de tópico fosse mesclado. Em termos de `git diff`, ele basicamente mostra automaticamente `git diff master…<branch>` para o branch no qual este Pull Request se baseia. Veja Determinando o Que é Introduzido para saber mais sobre esse tipo de diff.

A outra coisa que você notará é que o GitHub verifica se o Pull Request se mescla de forma limpa e fornece um botão para fazer o merge para você no servidor. Este botão só aparece se você tiver acesso de gravação (write access) ao repositório e um merge trivial for possível. Se você clicar nele, o GitHub realizará um merge “non-fast-forward”, o que significa que mesmo que o merge **pudesse** ser um fast-forward, ele ainda criará um commit de merge.

Se preferir, você pode simplesmente fazer o pull do branch e mesclá-lo localmente. Se você mesclar este branch no branch `master` e fizer o push para o GitHub, o Pull Request será fechado automaticamente.

Este é o fluxo de trabalho básico que a maioria dos projetos do GitHub usa. Branches de tópico são criados, Pull Requests são abertos neles, uma discussão se segue, possivelmente mais trabalho é feito no branch e, eventualmente, a solicitação é fechada ou mesclada.



Não Apenas Forks

É importante notar que você também pode abrir um Pull Request entre dois branches no mesmo repositório. Se você estiver trabalhando em um recurso com alguém e ambos tiverem acesso de gravação ao projeto, você pode fazer o push de um branch de tópico para o repositório e abrir um Pull Request dele para o branch `master` desse mesmo projeto para iniciar o processo de revisão de código e discussão. Nenhum fork é necessário.

Pull Requests Avançados

Agora que cobrimos os fundamentos de contribuição para um projeto no GitHub, vamos abordar algumas dicas e truques interessantes sobre Pull Requests para que você possa ser mais eficaz ao usá-los.

Pull Requests como Patches

É importante entender que muitos projetos não pensam realmente nos Pull Requests como filas de patches perfeitos que devem ser aplicados de forma limpa em ordem, como a maioria dos projetos baseados em lista de discussão pensa das contribuições de séries de patches. A maioria dos projetos no GitHub pensa nos branches de Pull Request como conversas iterativas em torno de uma alteração proposta, culminando em um diff unificado que é aplicado por meio de merge.

Esta é uma distinção importante, porque geralmente a alteração é sugerida antes que o código seja considerado perfeito, o que é muito mais raro com contribuições de séries de patches baseadas em listas de discussão. Isso permite uma conversa mais cedo com os mantenedores para que chegar à solução adequada seja mais um esforço da comunidade. Quando um código é proposto com um Pull Request e os mantenedores ou a comunidade sugerem uma alteração, a série de patches geralmente não é reescrita, mas sim a diferença é enviada como um novo commit para o branch, avançando a conversa com o contexto do trabalho anterior intacto.

Por exemplo, se você voltar e olhar novamente para Final do Pull Request, notará que o contribuidor não fez um rebase do seu commit nem enviou outro Pull Request. Em vez disso, eles adicionaram novos commits e fizeram push para o branch existente. Dessa forma, se você voltar e analisar este Pull Request no futuro, poderá facilmente encontrar todo o contexto do porquê as decisões foram tomadas. Clicar no botão “Merge” no site cria intencionalmente um commit de merge que faz referência ao Pull Request, para que seja fácil voltar e pesquisar a conversa original, se necessário.

Acompanhando o Upstream

Se o seu Pull Request ficar desatualizado ou não puder ser mesclado de forma limpa, você vai querer consertá-lo para que o mantenedor possa mesclá-lo facilmente. O GitHub testará isso para você e informará na parte inferior de cada Pull Request se o merge é trivial ou não.

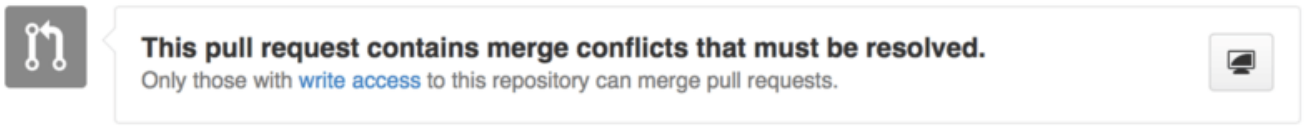


Figure 96. Pull Request não se mescla de forma limpa

Se você vir algo como Pull Request não se mescla de forma limpa, você vai querer consertar o seu branch para que ele fique verde e o mantenedor não tenha que fazer trabalho extra.

Você tem duas opções principais para fazer isso. Você pode fazer um rebase do seu branch em cima de qualquer que seja o branch de destino (normalmente o branch `master` do repositório do qual você fez fork), ou pode mesclar (merge) o branch de destino no seu branch.

A maioria dos desenvolvedores no GitHub escolherá fazer o último, pelas mesmas razões que acabamos de ver na seção anterior. O que importa é o histórico e o merge final, então fazer rebase não está lhe dando muito além de um histórico ligeiramente mais limpo e, em troca, é **muito** mais difícil e propenso a erros.

Se você quiser mesclar o branch de destino para tornar o seu Pull Request passível de merge, você adicionaria o repositório original como um novo remoto, faria um fetch dele, mesclaria o branch principal daquele repositório no seu branch de tópico, consertaria quaisquer problemas e finalmente faria o push de volta para o mesmo branch em que você abriu o Pull Request.

Por exemplo, digamos que no exemplo do “tonychacon” que estávamos usando antes, o autor original tenha feito uma alteração que criaria um conflito no Pull Request. Vamos percorrer essas etapas.

```
$ git remote add upstream https://github.com/schacon/blink ①

$ git fetch upstream ②
remote: Counting objects: 3, done.
remote: Compressing objects: 100% (3/3), done.
Unpacking objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0)
From https://github.com/schacon/blink
 * [new branch]      master     -> upstream/master

$ git merge upstream/master ③
Auto-merging blink.ino
CONFLICT (content): Merge conflict in blink.ino
Automatic merge failed; fix conflicts and then commit the result.

$ vim blink.ino ④
$ git add blink.ino
```

```
$ git commit
[slow-blink 3c8d735] Merge remote-tracking branch 'upstream/master' \
    into slower-blink

$ git push origin slow-blink ⑤
Counting objects: 6, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 682 bytes | 0 bytes/s, done.
Total 6 (delta 2), reused 0 (delta 0)
To https://github.com/tonychacon/blink
   ef4725c..3c8d735  slower-blink -> slow-blink
```

① Adicione o repositório original como um remoto chamado `upstream`.

② Faça um fetch do trabalho mais recente daquele remoto.

③ Mescle o branch principal daquele repositório no seu branch de tópico.

④ Conserte o conflito que ocorreu.

⑤ Faça o push de volta para o mesmo branch de tópico.

Assim que você fizer isso, o Pull Request será automaticamente atualizado e verificado novamente para ver se ele se mescla de forma limpa.

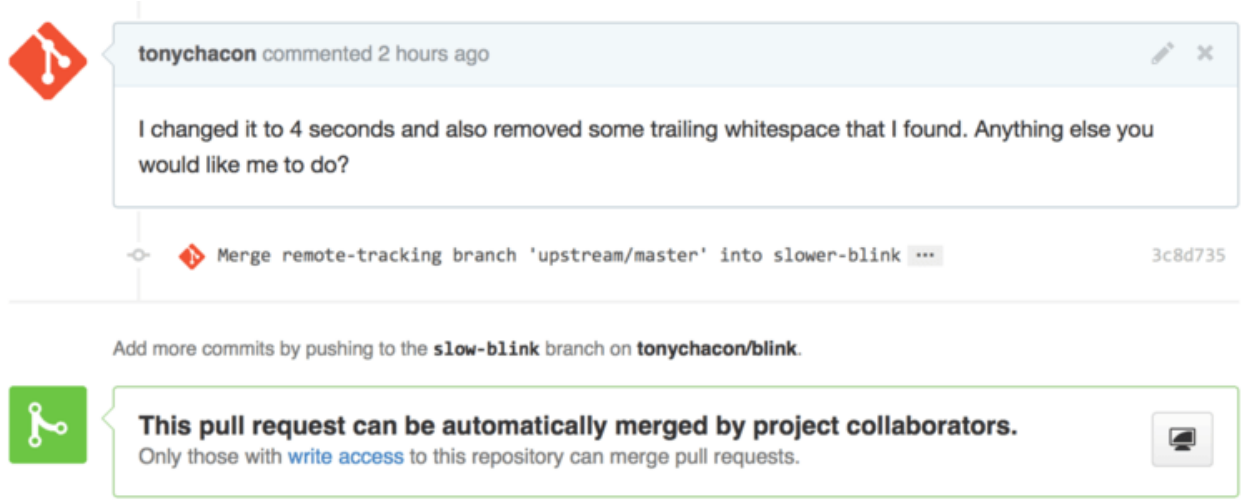


Figure 97. O Pull Request agora se mescla de forma limpa

Uma das grandes coisas sobre o Git é que você pode fazer isso continuamente. Se você tiver um projeto muito longo, poderá facilmente fazer merge do branch de destino repetidas vezes e só terá que lidar com os conflitos que surgiram desde a última vez que você fez merge, tornando o processo muito gerenciável.

Se você absolutamente deseja fazer rebase do branch para limpá-lo, certamente pode fazê-lo, mas é altamente recomendável não fazer um push forçado (force push) sobre o branch em que o Pull Request já está aberto. Se outras pessoas tiverem feito o pull dele e feito mais trabalho nele, você encontrará todos os problemas descritos em Os Perigos do Rebase (Perils of Rebasing). Em vez disso, faça o push do branch que sofreu rebase para um novo branch no GitHub e abra um Pull Request totalmente novo fazendo referência ao antigo, depois feche o original.

Referências

A sua próxima pergunta pode ser “Como faço referência ao Pull Request antigo?”. Acontece que existem muitas, muitas maneiras de referenciar outras coisas em quase qualquer lugar onde você pode escrever no GitHub.

Vamos começar com como fazer referência cruzada a outro Pull Request ou a uma Issue. Todos os Pull Requests e Issues recebem números e eles são únicos dentro do projeto. Por exemplo, você não pode ter o Pull Request #3 *e* a Issue #3. Se você quiser referenciar qualquer Pull Request ou Issue a partir de qualquer outro, você pode simplesmente colocar `#<num>` em qualquer comentário ou descrição. Você também pode ser mais específico se a Issue ou Pull Request estiver em outro lugar; escreva `username#<num>` se estiver se referindo a uma Issue ou Pull Request em um fork do repositório em que você está, ou `username/repo#<num>` para referenciar algo em outro repositório.

Vamos ver um exemplo. Digamos que fizemos um rebase do branch no exemplo anterior, criamos um novo pull request para ele e agora queremos referenciar o pull request antigo a partir do novo. Nós também queremos referenciar uma issue no fork do repositório e uma issue em um projeto completamente diferente. Podemos preencher a descrição exatamente como em Referências cruzadas em um Pull Request.

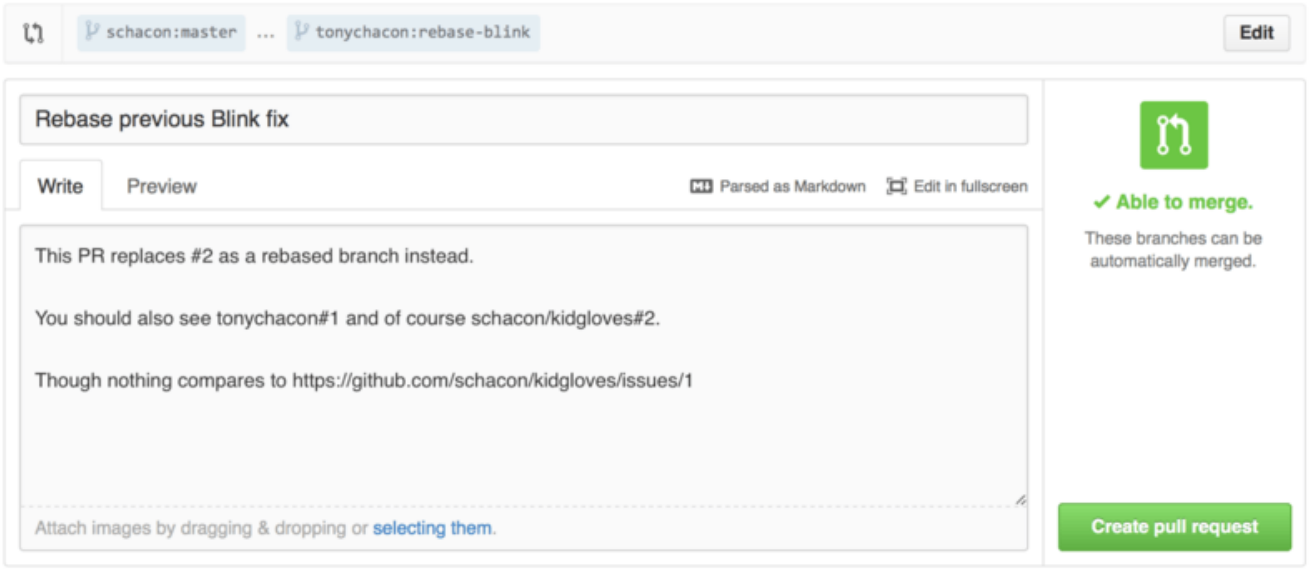


Figure 98. Referências cruzadas em um Pull Request

Quando submetermos este pull request, veremos tudo isso renderizado como em Referências cruzadas renderizadas em um Pull Request.

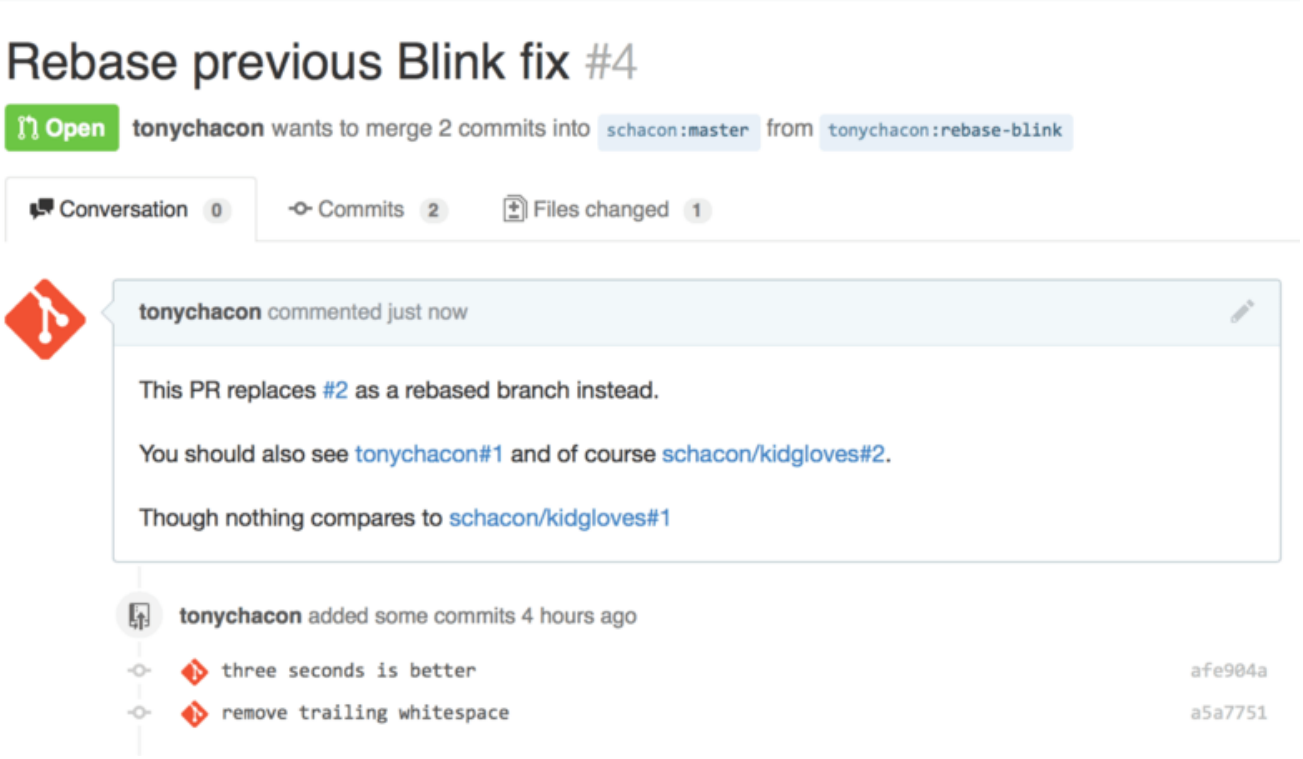


Figure 99. Referências cruzadas renderizadas em um Pull Request

Observe que a URL completa do GitHub que colocamos lá foi encurtada apenas para as informações necessárias.

Agora, se Tony voltar e fechar o Pull Request original, podemos ver que, ao mencioná-lo no novo, o GitHub criou automaticamente um evento de trackback (rastreamento) na linha do tempo do Pull Request. Isso significa que qualquer um que visite este Pull Request e veja que ele está fechado pode facilmente encontrar um link para aquele que o substituiu. O link se parecerá com Link de volta para o novo Pull Request na linha do tempo do Pull Request fechado.

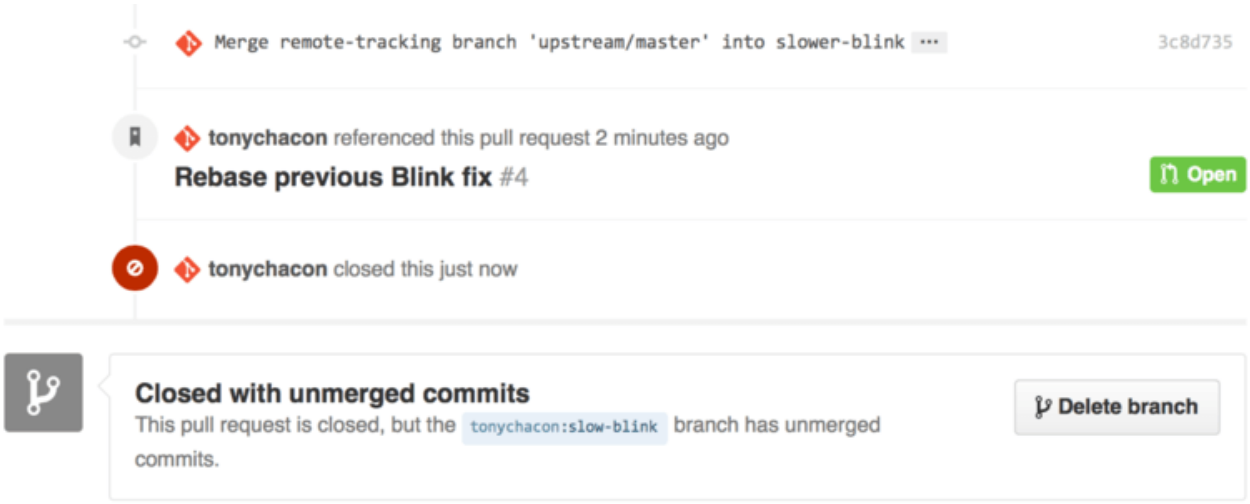


Figure 100. Link de volta para o novo Pull Request na linha do tempo do Pull Request fechado

Além dos números de issue, você também pode referenciar um commit específico pelo seu SHA-1. Você tem que especificar um SHA-1 completo de 40 caracteres, mas se o GitHub vir isso em um comentário, ele criará um link direto para o commit. Novamente, você pode referenciar commits em forks ou outros repositórios da mesma forma que fez com issues.

Markdown do GitHub (GitHub Flavored Markdown)

Vincular a outras Issues é apenas o começo de coisas interessantes que você pode fazer com quase qualquer caixa de texto no GitHub. Em descrições de Issue e Pull Request, comentários, comentários de código e muito mais, você pode usar o que é chamado de "GitHub Flavored Markdown". Markdown é como escrever em texto simples, mas que é renderizado com riqueza de detalhes.

Veja Um exemplo de GitHub Flavored Markdown como escrito e como renderizado para um exemplo de como os comentários ou textos podem ser escritos e depois renderizados usando Markdown.

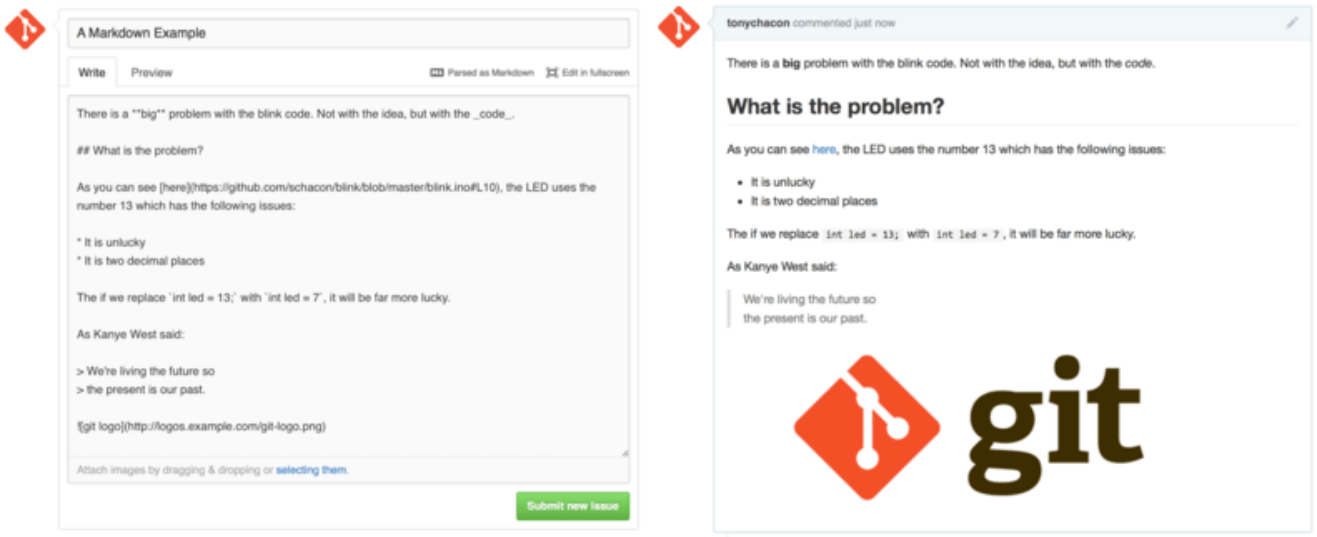


Figure 101. Um exemplo de GitHub Flavored Markdown como escrito e como renderizado

A variação de Markdown do GitHub adiciona mais coisas que você pode fazer além da sintaxe Markdown básica. Tudo isso pode ser realmente útil ao criar comentários ou descrições úteis de Pull Request ou Issue.

Listas de Tarefas (Task Lists)

A primeira funcionalidade Markdown realmente útil específica do GitHub, especialmente para uso em Pull Requests, é a Lista de Tarefas (Task List). Uma lista de tarefas é uma lista de caixas de seleção de coisas que você deseja que sejam feitas. Colocá-las em uma Issue ou Pull Request normalmente indica coisas que você deseja que sejam feitas antes de considerar o item completo.

Você pode criar uma lista de tarefas como esta:

```
- [X] Write the code
- [ ] Write all the tests
- [ ] Document the code
```

Se incluirmos isso na descrição do nosso Pull Request ou Issue, o veremos renderizado como em Listas de tarefas renderizadas em um comentário Markdown.

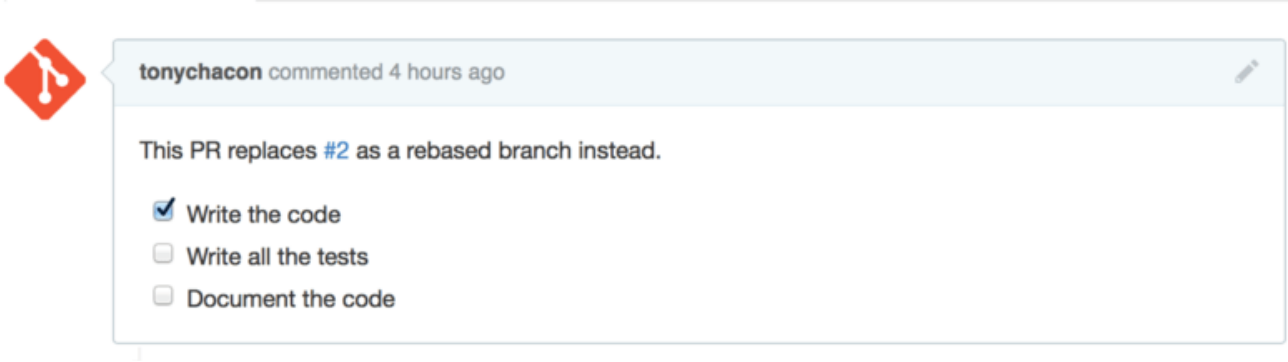


Figure 102. Listas de tarefas renderizadas em um comentário Markdown

Isso é frequentemente usado em Pull Requests para indicar tudo o que você gostaria de ver feito no branch antes que o Pull Request esteja pronto para o merge. A parte realmente legal é que você pode simplesmente clicar nas caixas de seleção para atualizar o comentário — você não precisa editar o Markdown diretamente para marcar as tarefas como concluídas.

Além do mais, o GitHub procurará por listas de tarefas em suas Issues e Pull Requests e as mostrará como metadados nas páginas que os listam. Por exemplo, se você tem um Pull Request com tarefas e olha na página de visão geral de todos os Pull Requests, pode ver o quanto ele está concluído. Isso ajuda as pessoas a dividirem Pull Requests em subtarefas e ajuda outras pessoas a acompanhar o progresso do branch. Você pode ver um exemplo disso em Resumo da lista de tarefas na lista de Pull Request.



Figure 103. Resumo da lista de tarefas na lista de Pull Request

Elas são incrivelmente úteis quando você abre um Pull Request logo no início e o usa para rastrear o seu progresso através da implementação do recurso.

Trechos de Código (Code Snippets)

Você também pode adicionar trechos de código (code snippets) aos comentários. Isso é especialmente útil se você quiser apresentar algo que você *poderia* tentar fazer antes de realmente implementá-lo como um commit no seu branch. Isso também é usado com frequência para adicionar código de exemplo do que não está funcionando ou o que este Pull Request poderia implementar.

Para adicionar um trecho de código você tem que “cercá-lo” (fence) com crases.

````
```java
for(int i=0 ; i < 5 ; i++)
{
   System.out.println("i is : " + i);
}
```
````

```
```

Se você adicionar um nome de linguagem como fizemos ali com 'java', o GitHub também tentará destacar a sintaxe do trecho. No caso do exemplo acima, ele acabaria renderizando como em Exemplo de código cercado renderizado.



Figure 104. Exemplo de código cercado renderizado

Citando (Quoting)

Se você estiver respondendo a uma pequena parte de um comentário longo, você pode citar seletivamente do outro comentário precedendo as linhas com o caractere `>`. De fato, isso é tão comum e tão útil que existe um atalho de teclado para isso. Se você destacar texto em um comentário ao qual deseja responder diretamente e pressionar a tecla `r`, ele citará esse texto na caixa de comentário para você.

As citações se parecem mais ou menos assim:

```
> Whether 'tis Nobler in the mind to suffer
> The Slings and Arrows of outrageous Fortune,

How big are these slings and in particular, these arrows?
```

Uma vez renderizado, o comentário se parecerá com Exemplo de citação renderizado.

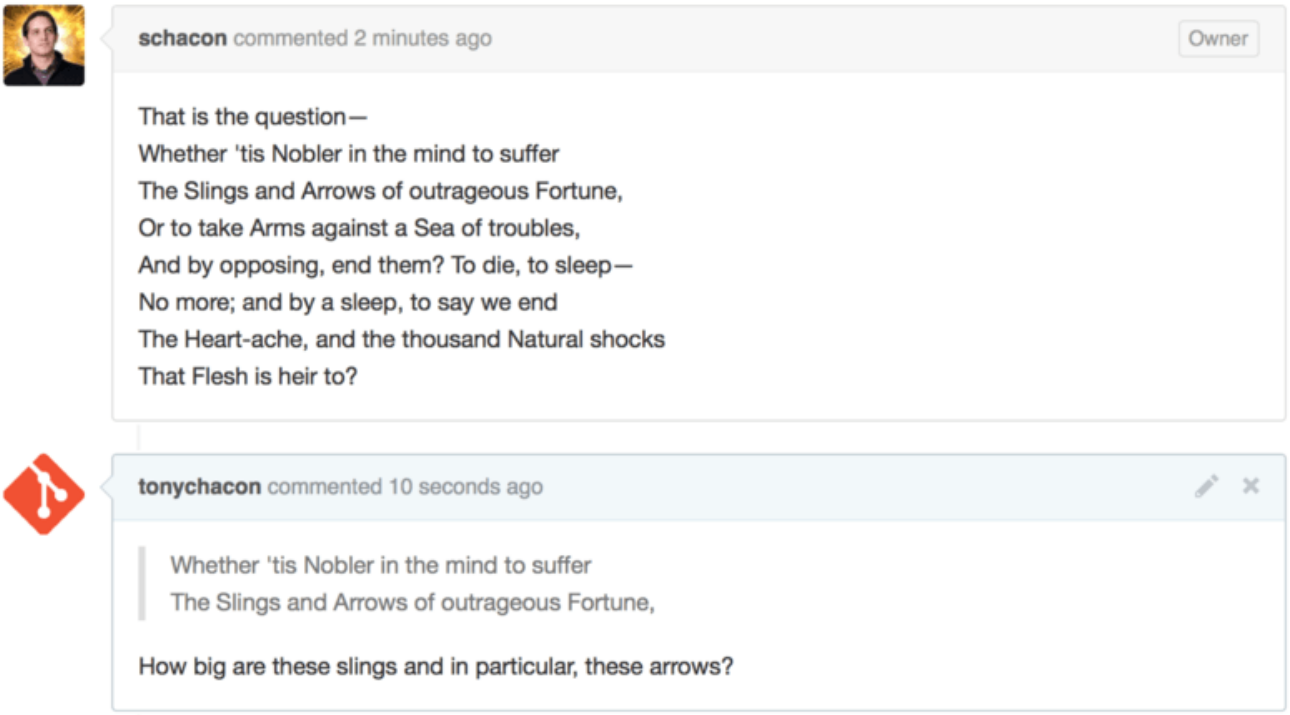


Figure 105. Exemplo de citação renderizado

Emoji

Finalmente, você também pode usar emoji em seus comentários. Isso é, na verdade, usado extensivamente em comentários que você vê em muitas Issues e Pull Requests no GitHub. Há até mesmo um assistente de emoji no GitHub. Se você estiver digitando um comentário e começar com um caractere `:`, um recurso de autocompletar ajudará a encontrar o que você está procurando.

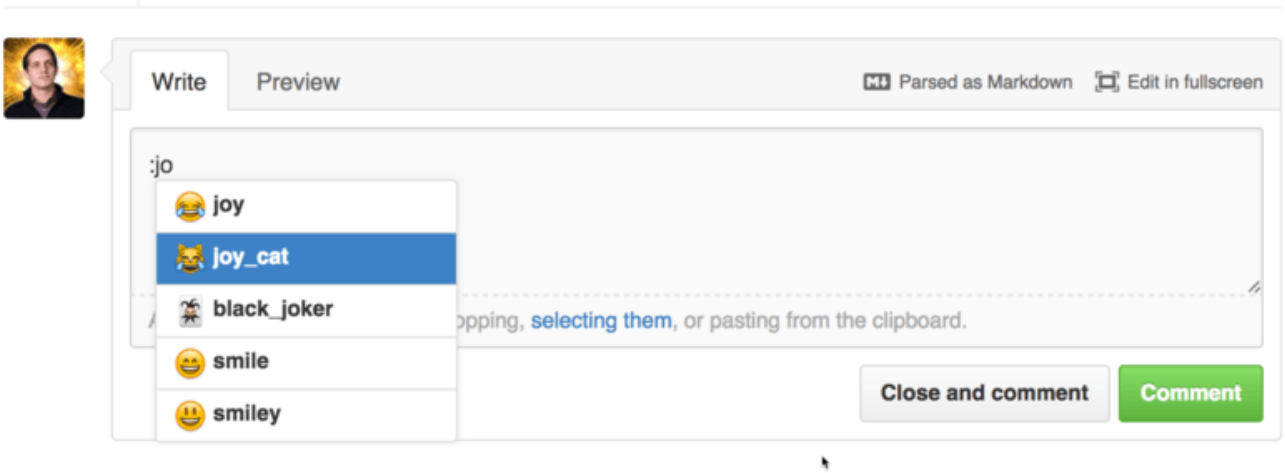


Figure 106. Recurso de autocompletar emoji em ação

Os emojis assumem a forma de `:<name>:` em qualquer lugar no comentário. Por exemplo, você poderia escrever algo assim:

```
I :eyes: that :bug: and I :cold_sweat:.

:trophy: for :microscope: it.

:+1: and :sparkles: on this :ship:, it's :fire::poop:!
```

```
:clap::tada::panda_face:
```

Quando renderizado, ficaria parecido com Comentários repletos de emoji.

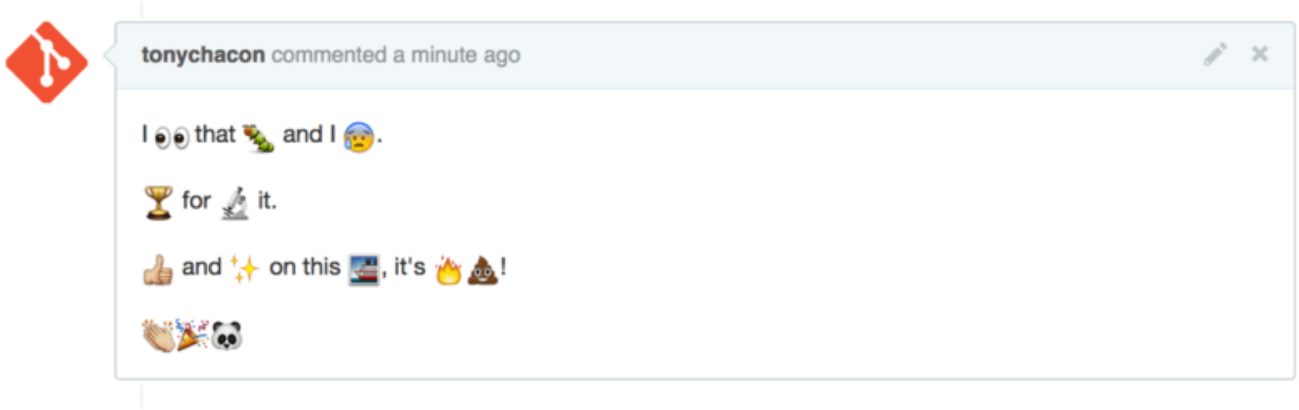


Figure 107. Comentários repletos de emoji

Não que isso seja incrivelmente útil, mas adiciona um elemento de diversão e emoção a um meio no qual, de outra forma, é difícil transmitir emoção.



Na verdade, há um bom número de serviços da web que fazem uso de caracteres emoji hoje em dia. Um ótimo guia de referência (cheat sheet) para encontrar o emoji que expressa o que você quer dizer pode ser encontrado em:

https://www.webfx.com/tools/emoji-cheat-sheet/

Imagens

Isso não é tecnicamente GitHub Flavored Markdown, mas é incrivelmente útil. Além de adicionar links de imagens Markdown aos comentários, que podem ser difíceis de encontrar e incorporar URLs, o GitHub permite que você arraste e solte imagens em áreas de texto para incorporá-las.

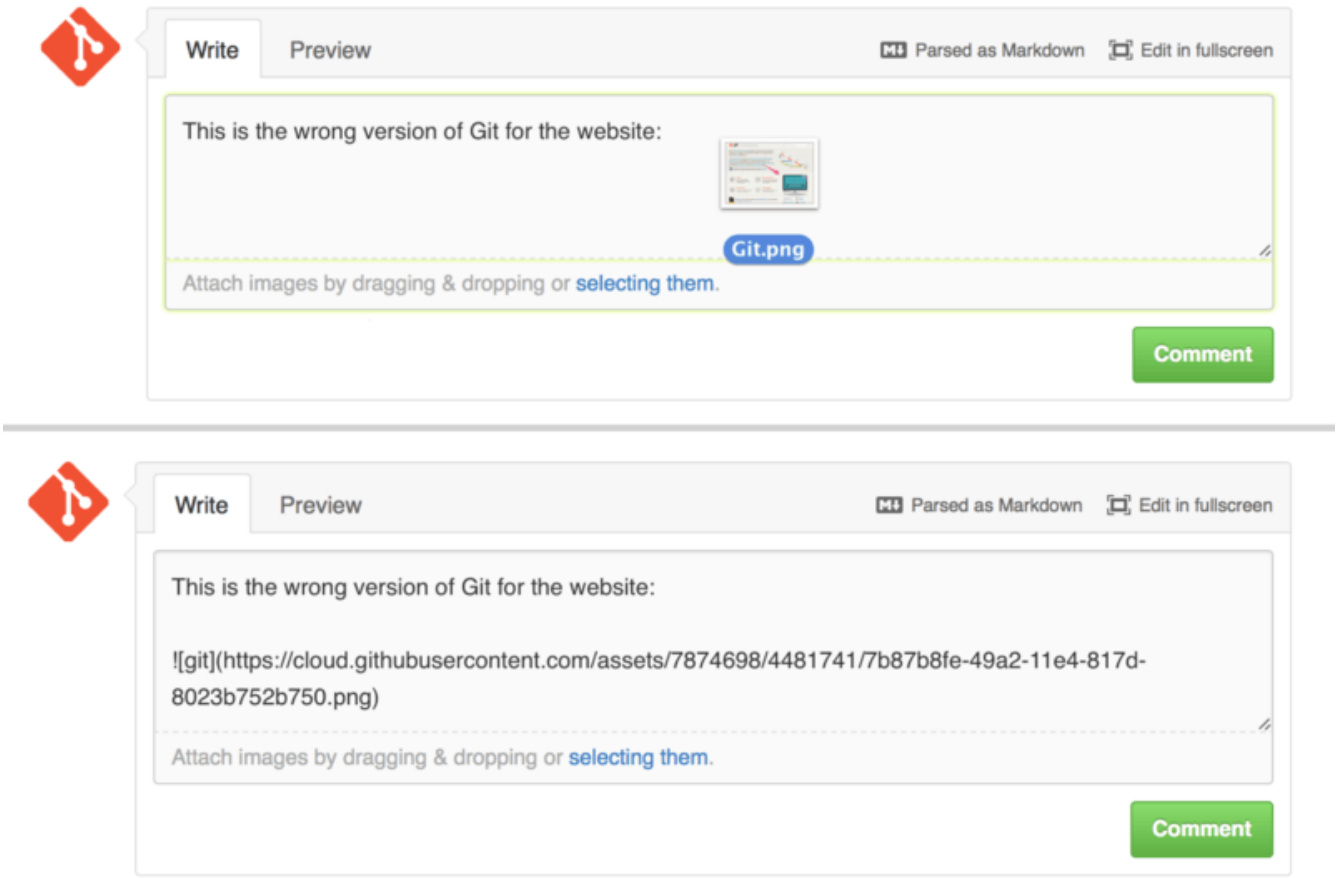


Figure 108. Arraste e solte imagens para carregá-las e incorporá-las automaticamente

Se você olhar para Arraste e solte imagens para carregá-las e incorporá-las automaticamente, poderá ver uma pequena dica “Parsed as Markdown” (Analisado como Markdown) acima da área de texto. Clicar nela lhe dará um guia de referência completo de tudo o que você pode fazer com Markdown no GitHub.

Mantenha o seu repositório público do GitHub atualizado

Depois que você faz o fork de um repositório no GitHub, o seu repositório (o seu "fork") existe independentemente do original. Em particular, quando o repositório original tiver novos commits, o GitHub o informará com uma mensagem como:

```
This branch is 5 commits behind progit:master.
```

Mas o seu repositório no GitHub nunca será atualizado automaticamente pelo GitHub; isso é algo que você mesmo deve fazer. Felizmente, isso é muito fácil de fazer.

Uma possibilidade para fazer isso não requer configuração. Por exemplo, se você fez fork de `https://github.com/progit/progit2.git`, você pode manter o seu branch `master` atualizado desta forma:

```
$ git checkout master ①
$ git pull https://github.com/progit/progit2.git ②
$ git push origin master ③
```

① Se você estava em outro branch, retorne ao `master`.

② Faça um fetch das alterações de `https://github.com/progit/progit2.git` e as mescle no `master`.

③ Faça o push do seu branch `master` para a `origin`.

Isso funciona, mas é um pouco entediante ter que soletrar a URL do fetch todas as vezes. Você pode automatizar esse trabalho com um pouco de configuração:

```
$ git remote add progit https://github.com/progit/progit2.git ①
$ git fetch progit ②
$ git branch --set-upstream-to=progit/master master ③
$ git config --local remote.pushDefault origin ④
```

① Adicione o repositório de origem e dê um nome a ele. Aqui, escolhi chamá-lo de `progit`.

② Obtenha uma referência aos branches do progit, em particular ao `master`.

③ Configure o seu branch `master` para fazer fetch do remoto `progit`.

④ Defina o repositório padrão de push para `origin`.

Uma vez feito isso, o fluxo de trabalho se torna muito mais simples:

```
$ git checkout master ①
$ git pull ②
$ git push ③
```

① Se você estava em outro branch, retorne ao `master`.

② Faça um fetch das alterações do `progit` e mescle as alterações no `master`.

③ Faça o push do seu branch `master` para a `origin`.

Essa abordagem pode ser útil, mas não deixa de ter desvantagens. O Git ficará feliz em fazer esse trabalho para você silenciosamente, mas não o avisará se você fizer um commit no `master`, fizer um pull de `progit` e depois fizer um push para `origin` — todas essas operações são válidas com esta configuração. Então, você terá que tomar cuidado para nunca comitar diretamente no `master`, já que esse branch pertence efetivamente ao repositório upstream.

Mantendo um Projeto

Agora que nos sentimos confortáveis contribuindo com um projeto, vamos olhar para o outro lado: criar, manter e administrar o seu próprio projeto.

Criando um Novo Repositório

Vamos criar um novo repositório para compartilhar o código do nosso projeto. Comece clicando no botão “New repository” (Novo repositório) no lado direito do painel, ou a partir do botão `+` na barra de ferramentas superior ao lado do seu nome de usuário, como visto em O menu suspenso “New repository”.

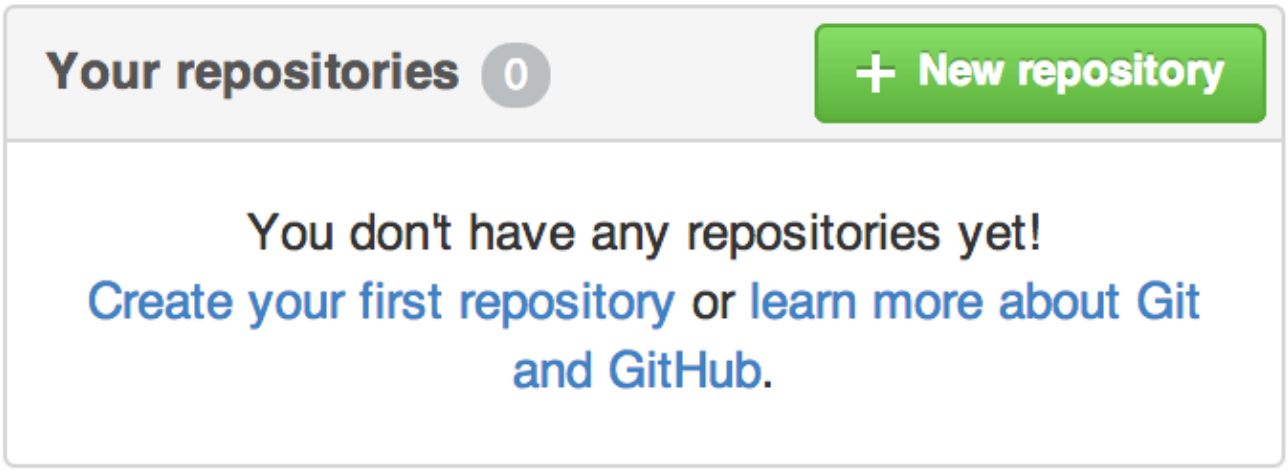


Figure 109. A área “Your repositories”

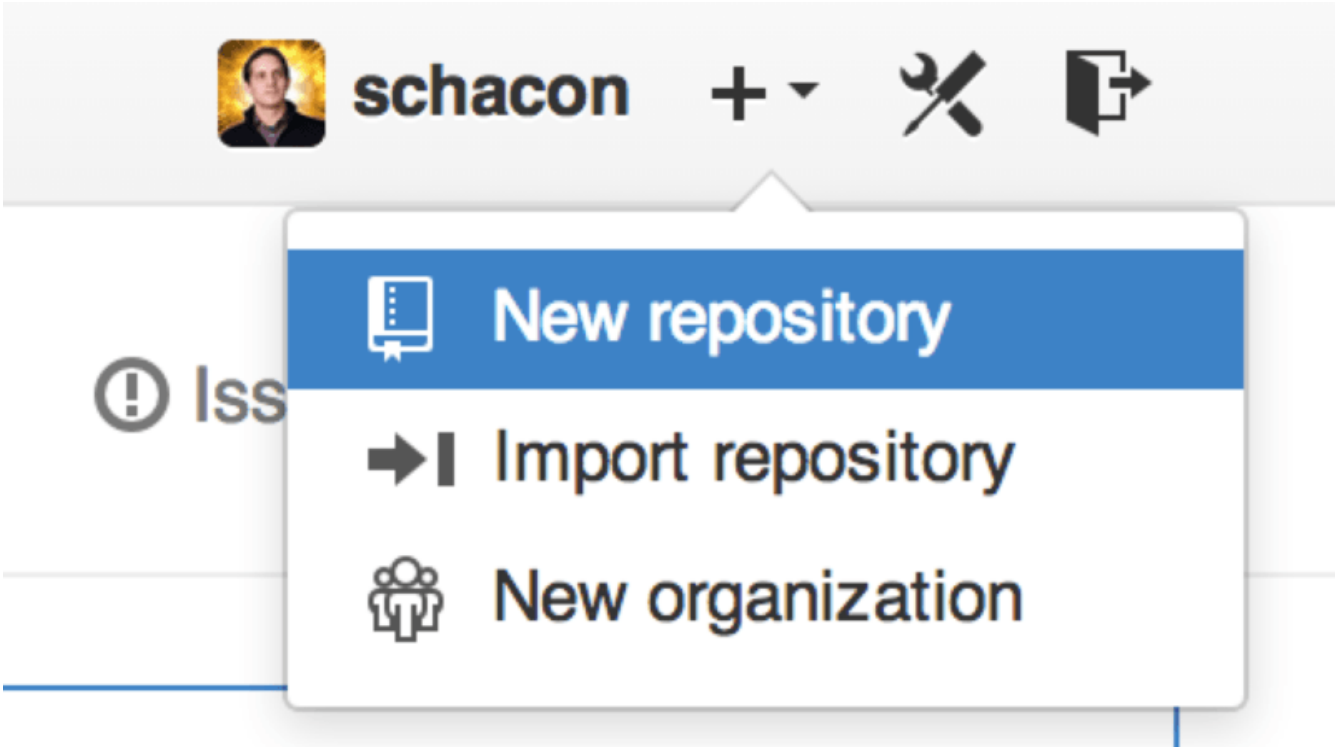


Figure 110. O menu suspenso “New repository”

Isso leva você ao formulário “new repository” (novo repositório):

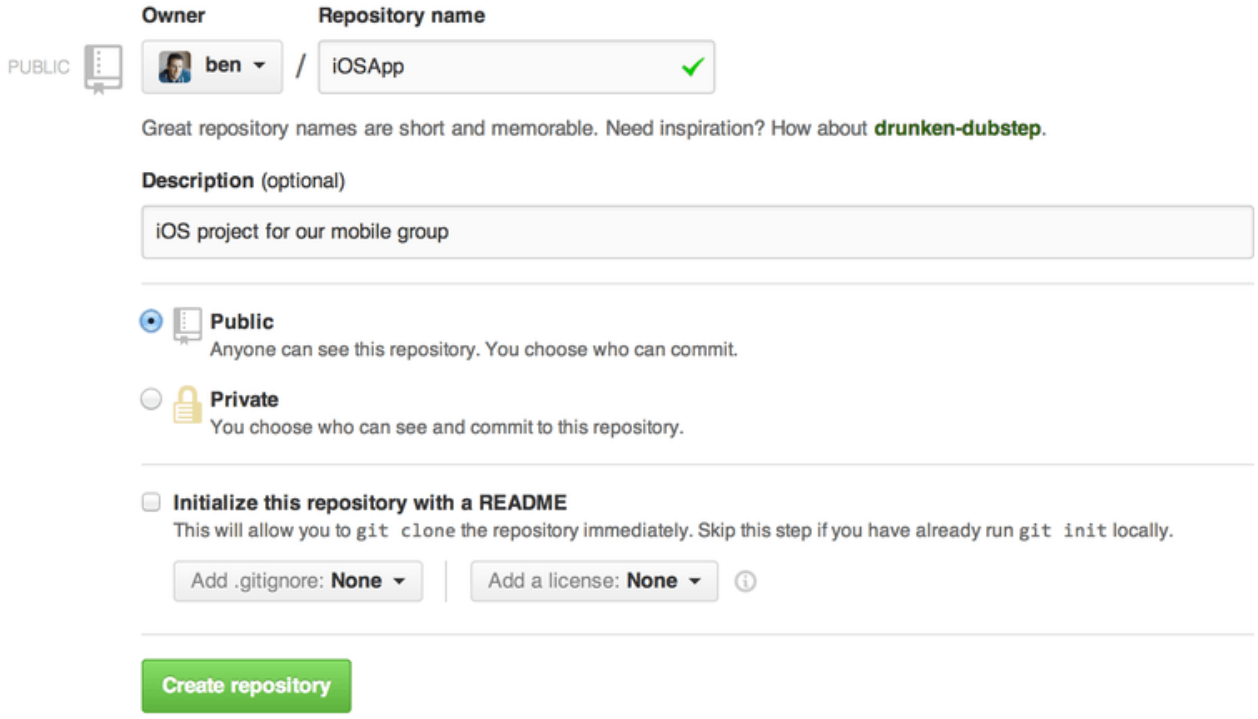


Figure 111. O formulário "new repository"

Tudo o que você realmente precisa fazer aqui é fornecer o nome de um projeto; o restante dos campos é totalmente opcional. Por enquanto, basta clicar no botão "Create Repository" (Criar Repositório), e bum — você tem um novo repositório no GitHub, chamado `<user>/<project_name>`.

Como você ainda não tem nenhum código lá, o GitHub lhe mostrará instruções de como criar um repositório Git totalmente novo ou conectar um projeto Git existente. Não vamos nos estender sobre isso aqui; se você precisar de uma atualização, verifique Fundamentos do Git.

Agora que o seu projeto está hospedado no GitHub, você pode fornecer a URL a qualquer pessoa com quem queira compartilhar o seu projeto. Todo projeto no GitHub é acessível via HTTPS em `https://github.com/<user>/<project_name>`, e via SSH em `git@github.com:<user>/<project_name>`. O Git pode fazer fetch de e push para ambas as URLs, mas elas têm controle de acesso com base nas credenciais do usuário que se conecta a elas.



Muitas vezes é preferível compartilhar a URL baseada em HTTPS para um projeto público, já que o usuário não precisa ter uma conta no GitHub para acessá-lo para clonagem. Os usuários terão que ter uma conta e uma chave SSH carregada para acessar o seu projeto se você lhes der a URL SSH. A versão em HTTPS também é exatamente a mesma URL que eles colariam em um navegador para ver o projeto lá.

Adicionando Colaboradores

Se você estiver trabalhando com outras pessoas a quem deseja dar acesso de commit, você precisará adicioná-las como "collaborators" (colaboradores). Se Ben, Jeff e Louise se inscreverem para contas no GitHub, e você quiser dar a eles acesso de push ao seu repositório, você pode adicioná-los ao seu projeto. Ao fazer isso, você dará a eles acesso de "push", o que significa que eles terão acesso tanto de leitura quanto de gravação ao projeto e ao repositório Git.

Clique no link “Settings” (Configurações) na parte inferior da barra lateral direita.

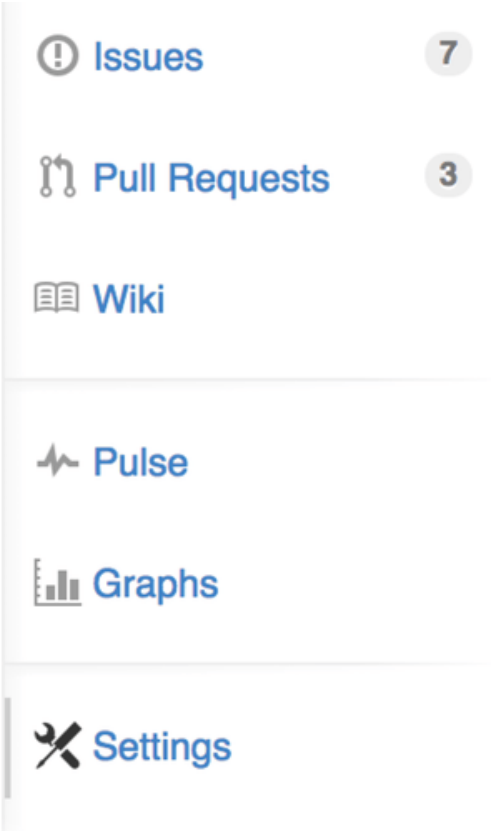


Figure 112. O link de configurações do repositório

Em seguida, selecione “Collaborators” (Colaboradores) no menu do lado esquerdo. Em seguida, basta digitar um nome de usuário na caixa e clicar em “Add collaborator” (Adicionar colaborador). Você pode repetir isso quantas vezes quiser para conceder acesso a todos que desejar. Se precisar revogar o acesso, basta clicar no “X” no lado direito da linha deles.

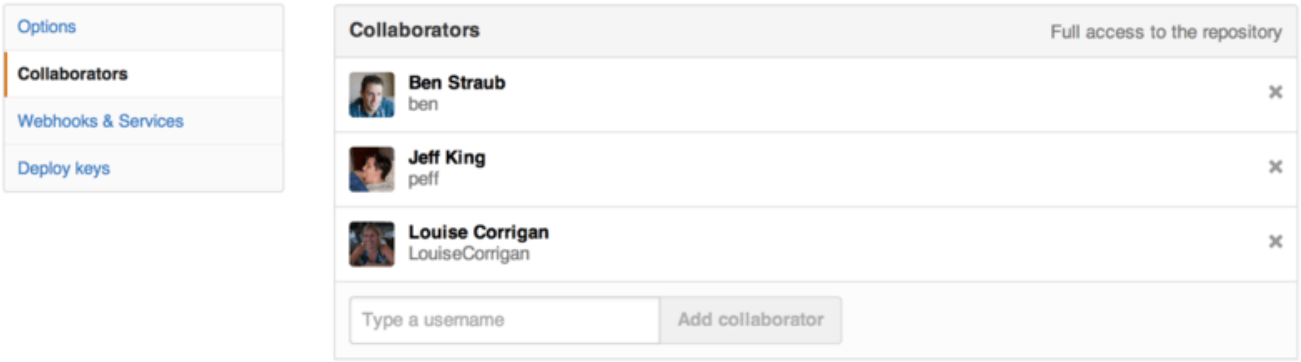


Figure 113. A caixa de colaboradores do repositório

Gerenciando Pull Requests

Agora que você tem um projeto com algum código nele e talvez até mesmo alguns colaboradores que também têm acesso de push, vamos ver o que fazer quando você mesmo receber um Pull Request.

Os Pull Requests podem vir de um branch em um fork do seu repositório ou podem vir de outro branch no mesmo repositório. A única diferença é que os que estão em um fork costumam ser de

pessoas onde você não pode fazer push para o branch delas e elas não podem fazer push para o seu, enquanto em Pull Requests internos, geralmente, ambas as partes podem acessar o branch.

Para estes exemplos, vamos assumir que você seja “tonychacon” e tenha criado um novo projeto de código Arduino chamado “fade”.

Notificações por E-mail

Alguém chega, faz uma alteração no seu código e lhe envia um Pull Request. Você deve receber um e-mail notificando-o sobre o novo Pull Request e ele deve se parecer com Notificação por e-mail de um novo Pull Request.


[fade] Wait longer to see the dimming effect better (#1)

Scott Chacon <notifications@github.com> 10:05 AM (0 minutes ago)
to tonychacon/fade Unsubscribe

One needs to wait another 10 ms to properly see the fade.

You can merge this Pull Request by running

```
git pull https://github.com/schacon/fade patch-1
```

Or view, comment on, or merge it at:

https://github.com/tonychacon/fade/pull/1

Commit Summary

- wait longer to see the dimming effect better

File Changes

- **M** fade.ino (2)

Patch Links:

- https://github.com/tonychacon/fade/pull/1.patch
- https://github.com/tonychacon/fade/pull/1.diff

—
Reply to this email directly or view it on GitHub.

Figure 114. Notificação por e-mail de um novo Pull Request

Existem algumas coisas a serem notadas sobre este e-mail. Ele lhe dará um pequeno diffstat — uma lista de arquivos que foram alterados no Pull Request e em que quantidade. Ele lhe dá um link para o Pull Request no GitHub. Ele também lhe dá algumas URLs que você pode usar a partir da linha de comando.

Se você notar a linha que diz `git pull <url> patch-1`, esta é uma forma simples de mesclar (merge) um branch remoto sem ter que adicionar um remoto. Nós analisamos isso rapidamente em Fazendo Checkout de Branches Remotos. Se desejar, você pode criar e mudar para um branch de tópico e, em seguida, executar este comando para mesclar as alterações do Pull Request.

As outras URLs interessantes são as URLs `.diff` e `.patch`, que, como você pode adivinhar, fornecem versões diff unificadas e patch do Pull Request. Você poderia tecnicamente mesclar o trabalho do Pull Request com algo como isto:

```
$ curl https://github.com/tonychacon/fade/pull/1.patch | git am
```

Colaborando no Pull Request

Como abordamos em O Fluxo do GitHub (GitHub Flow), agora você pode ter uma conversa com a pessoa que abriu o Pull Request. Você pode comentar em linhas de código específicas, comentar em commits inteiros ou comentar no Pull Request inteiro em si, usando o GitHub Flavored Markdown em toda parte.

Toda vez que alguém comenta no Pull Request, você continuará a receber notificações por e-mail, para saber que há atividade acontecendo. Cada um deles terá um link para o Pull Request onde a atividade está acontecendo e você também pode responder diretamente ao e-mail para comentar na thread do Pull Request.

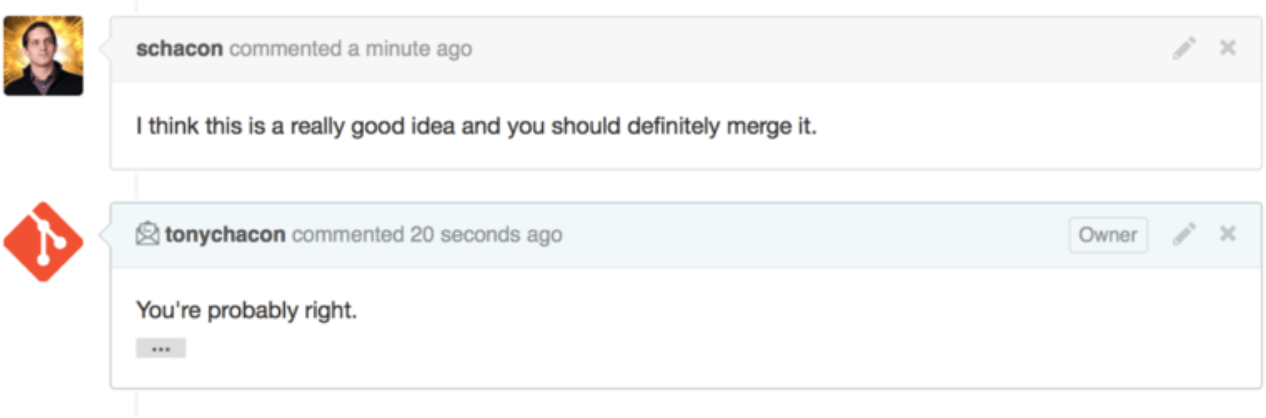


Figure 115. Respostas a e-mails são incluídas na thread

Uma vez que o código esteja de uma forma que você goste e queira mesclá-lo, você pode fazer o pull do código e mesclá-lo localmente, seja com a sintaxe `git pull <url> <branch>` que vimos anteriormente, ou adicionando o fork como um remoto e fazendo fetch e merge.

Se o merge for trivial, você também pode simplesmente clicar no botão “Merge” no site do GitHub. Isso fará um merge “non-fast-forward”, criando um commit de merge mesmo que um merge fast-forward fosse possível. Isso significa que, aconteça o que acontecer, toda vez que você clica no botão de merge, um commit de merge é criado. Como você pode ver em Botão Merge e instruções para fazer o merge de um Pull Request manualmente, o GitHub lhe dá todas essas informações se você clicar no link de dica.

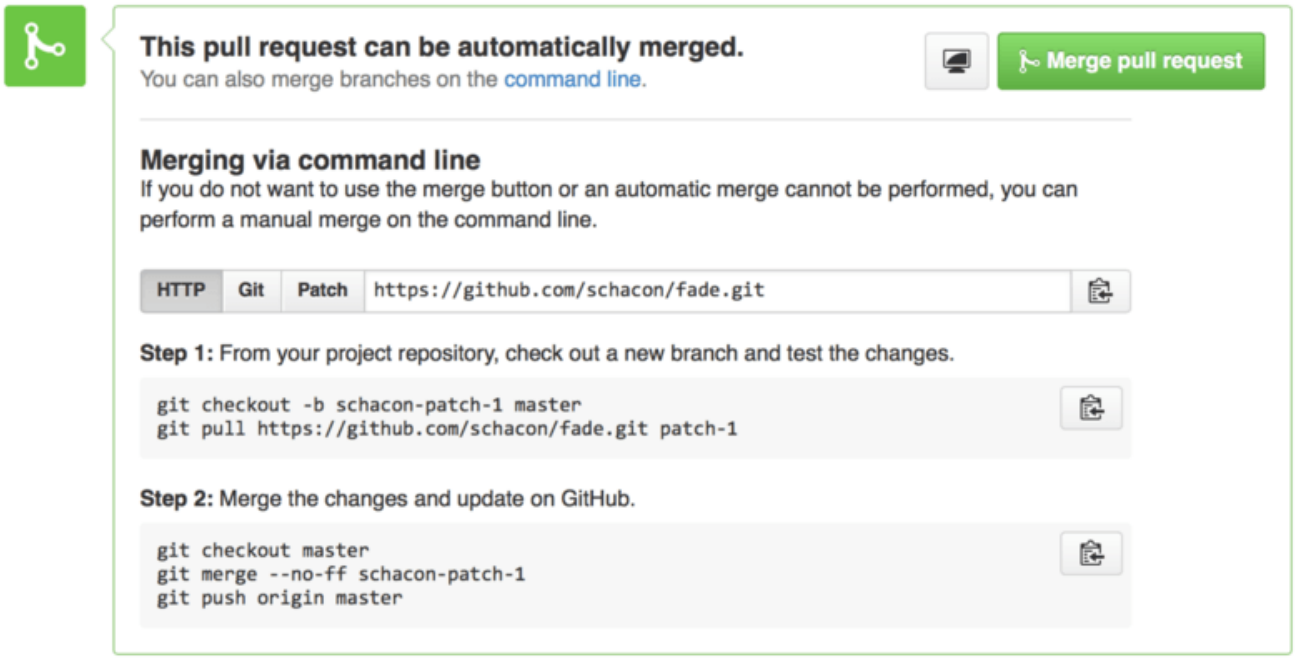


Figure 116. Botão Merge e instruções para fazer o merge de um Pull Request manualmente

Se você decidir que não quer mesclá-lo, você também pode simplesmente fechar o Pull Request e a pessoa que o abriu será notificada.

Refs de Pull Request

Se você estiver lidando com um **monte** de Pull Requests e não quiser adicionar um monte de remotos ou fazer pulls pontuais toda vez, há um truque bacana que o GitHub permite que você faça. Este é um truque um pouco avançado e vamos analisar os detalhes disso um pouco mais em Refspec, mas ele pode ser bastante útil.

Na verdade, o GitHub anuncia (advertises) os branches de Pull Request de um repositório como uma espécie de pseudo-branches no servidor. Por padrão, você não os obtém quando clona, mas eles estão lá de forma obscura e você pode acessá-los com bastante facilidade.

Para demonstrar isso, usaremos um comando de baixo nível (frequentemente referido como um comando de “plumbing” ou encanamento, sobre o qual leremos mais em Encanamento e Porcelana (Plumbing and Porcelain)) chamado `ls-remote`. Esse comando geralmente não é usado em operações do dia a dia no Git, mas é útil para nos mostrar quais referências estão presentes no servidor.

Se executarmos esse comando no repositório “blink” que estávamos usando antes, obteremos uma lista de todos os branches, tags e outras referências no repositório.

```
$ git ls-remote https://github.com/schacon/blink
10d539600d86723087810ec636870a504f4fee4d    HEAD
10d539600d86723087810ec636870a504f4fee4d    refs/heads/master
6a83107c62950be9453aac297bb0193fd743cd6e    refs/pull/1/head
afe83c2d1a70674c9505cc1d8b7d380d5e076ed3    refs/pull/1/merge
3c8d735ee16296c242be7a9742ebfbc2665adec1    refs/pull/2/head
15c9f4f80973a2758462ab2066b6ad9fe8dcf03d    refs/pull/2/merge
```

```
a5a7751a33b7e86c5e9bb07b26001bb17d775d1a    refs/pull/4/head
31a45fc257e8433c8d8804e3e848cf61c9d3166c    refs/pull/4/merge
```

É claro, se você estiver em seu repositório e executar `git ls-remote origin` ou qualquer remoto que deseje verificar, ele mostrará algo semelhante a isso.

Se o repositório estiver no GitHub e você tiver Pull Requests abertos, você obterá essas referências prefixadas com `refs/pull/`. Estas são basicamente branches, mas como elas não estão sob `refs/heads/`, você normalmente não as obtém ao clonar ou fazer fetch do servidor — o processo de fetch normalmente as ignora.

Existem duas referências por Pull Request - a que termina em `/head` aponta exatamente para o mesmo commit que o último commit no branch do Pull Request. Então, se alguém abrir um Pull Request no nosso repositório e seu branch for nomeado `bug-fix` apontando para o commit `a5a775`, em **nosso** repositório não teremos um branch `bug-fix` (já que está no fork deles), mas *teremos* `pull/<pr#>/head` apontando para `a5a775`. Isso significa que podemos muito facilmente baixar todo branch de Pull Request de uma só vez sem ter que adicionar um monte de remotos.

Agora, você poderia fazer algo como o fetch da referência diretamente.

```
$ git fetch origin refs/pull/958/head
From https://github.com/libgit2/libgit2
 * branch            refs/pull/958/head -> FETCH_HEAD
```

Isto diz ao Git: “Conecte-se ao remoto `origin` e baixe o ref chamado `refs/pull/958/head`.” O Git obedece de bom grado e baixa tudo o que você precisa para construir esse ref, e coloca um ponteiro para o commit que você deseja sob `.git/FETCH_HEAD`. Você pode seguir isso com `git merge FETCH_HEAD` no branch onde deseja testar a alteração, mas a mensagem do commit de merge parecerá um pouco estranha. Além disso, se você estiver revisando um **monte** de pull requests, isso se torna entediante.

Há também uma forma de fazer fetch de *todos* os pull requests e mantê-los atualizados sempre que você se conectar ao remoto. Abra o `.git/config` em seu editor favorito e procure pelo remoto `origin`. Deve se parecer um pouco com isto:

```
[remote "origin"]
    url = https://github.com/libgit2/libgit2
    fetch = +refs/heads/*:refs/remotes/origin/*
```

A linha que começa com `fetch =` é um “refspec”. É uma forma de mapear nomes no remoto com nomes no seu diretório `.git` local. Esse em particular diz ao Git: "as coisas no remoto que estão sob `refs/heads` devem ir para o meu repositório local sob `refs/remotes/origin`." Você pode modificar esta seção para adicionar outro refspec:

```
[remote "origin"]
    url = https://github.com/libgit2/libgit2.git
    fetch = +refs/heads/*:refs/remotes/origin/*
```

```
    fetch = +refs/pull/*/head:refs/remotes/origin/pr/*
```

Essa última linha diz ao Git: "Todos os refs que se parecem com `refs/pull/123/head` devem ser armazenados localmente como `refs/remotes/origin/pr/123`." Agora, se você salvar esse arquivo e fizer um `git fetch`:

```
$ git fetch
# …
 * [new ref]         refs/pull/1/head -> origin/pr/1
 * [new ref]         refs/pull/2/head -> origin/pr/2
 * [new ref]         refs/pull/4/head -> origin/pr/4
# …
```

Agora, todos os pull requests remotos são representados localmente com refs que atuam de forma muito parecida com as branches de rastreamento (tracking branches); eles são somente leitura e são atualizados quando você faz um fetch. Isso torna super fácil testar o código de um pull request localmente:

```
$ git checkout pr/2
Checking out files: 100% (3769/3769), done.
Branch pr/2 set up to track remote branch pr/2 from origin.
Switched to a new branch 'pr/2'
```

Os mais observadores entre vocês devem ter notado o `head` no final da porção remota do refspec. Há também um ref `refs/pull/#/merge` no lado do GitHub, que representa o commit que resultaria se você clicasse no botão "merge" no site. Isso pode permitir que você teste o merge antes mesmo de apertar o botão.

Pull Requests de Pull Requests

Você não apenas pode abrir Pull Requests que visam o branch principal ou `master`, como na verdade pode abrir um Pull Request visando qualquer branch na rede. De fato, você pode até mesmo visar outro Pull Request.

Se você vir um Pull Request que está indo na direção certa e tiver uma ideia para uma alteração que dependa dele ou sobre a qual não tenha certeza se é uma boa ideia, ou simplesmente não tiver acesso de push ao branch de destino, você pode abrir um Pull Request diretamente para ele.

Quando você vai abrir um Pull Request, há uma caixa na parte superior da página que especifica de qual branch você está solicitando fazer o pull e para qual branch você está solicitando fazer o pull. Se você clicar no botão "Edit" (Editar) à direita dessa caixa, poderá alterar não apenas os branches, mas também de qual fork.

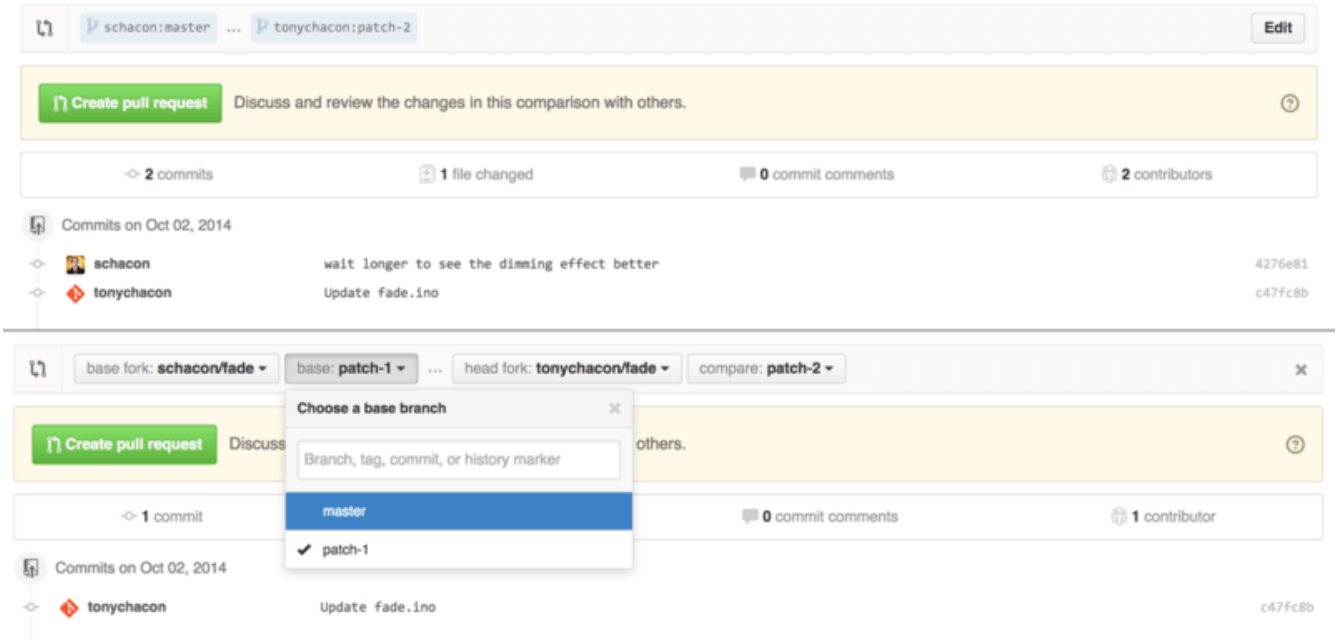


Figure 117. Altere manualmente o fork e o branch de destino do Pull Request

Aqui, você pode facilmente especificar o merge do seu novo branch em outro Pull Request ou outro fork do projeto.

Menções e Notificações

O GitHub também possui um sistema de notificações embutido muito bom que pode ser útil quando você tem dúvidas ou precisa de feedback de indivíduos ou equipes específicas.

Em qualquer comentário, você pode começar a digitar um caractere `@` e ele começará a preencher automaticamente (autocomplete) com os nomes e nomes de usuário de pessoas que são colaboradoras ou contribuidoras do projeto.

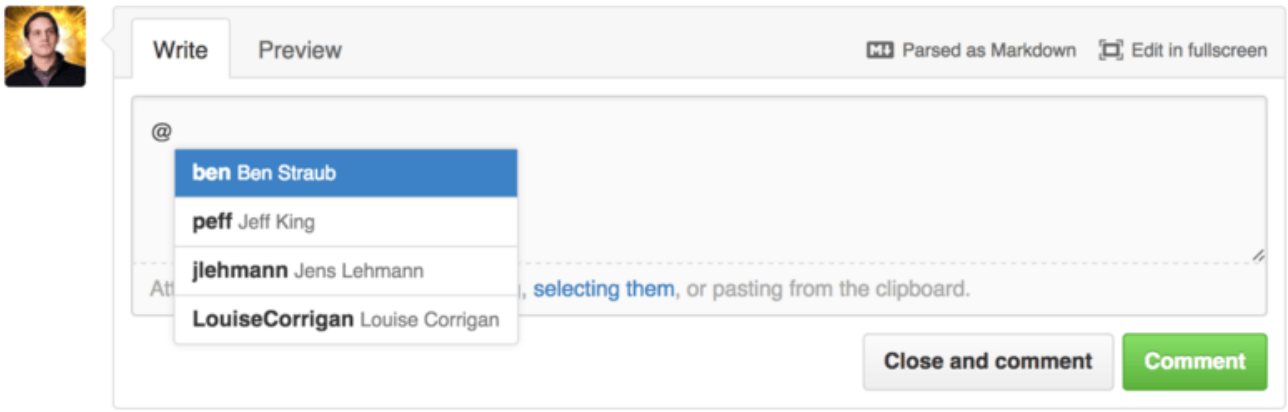


Figure 118. Comece a digitar @ para mencionar alguém

Você também pode mencionar um usuário que não está nesse menu suspenso, mas muitas vezes o recurso de autocompletar pode torná-lo mais rápido.

Assim que você publicar um comentário com uma menção a um usuário, esse usuário será notificado. Isso significa que essa pode ser uma forma realmente eficaz de puxar as pessoas para conversas em vez de fazê-las pesquisar (poll). Com muita frequência, em Pull Requests no GitHub, as pessoas puxam outras pessoas de suas equipes ou de suas empresas para revisar uma Issue ou

Pull Request.

Se alguém for mencionado em um Pull Request ou Issue, ele será “inscrito” (subscribed) nele e continuará recebendo notificações sempre que alguma atividade ocorrer. Você também será inscrito em algo se o abriu, se estiver observando (watching) o repositório ou se comentar sobre algo. Se você não deseja mais receber notificações, há um botão “Unsubscribe” (Cancelar Inscrição) na página que você pode clicar para parar de receber atualizações sobre ele.

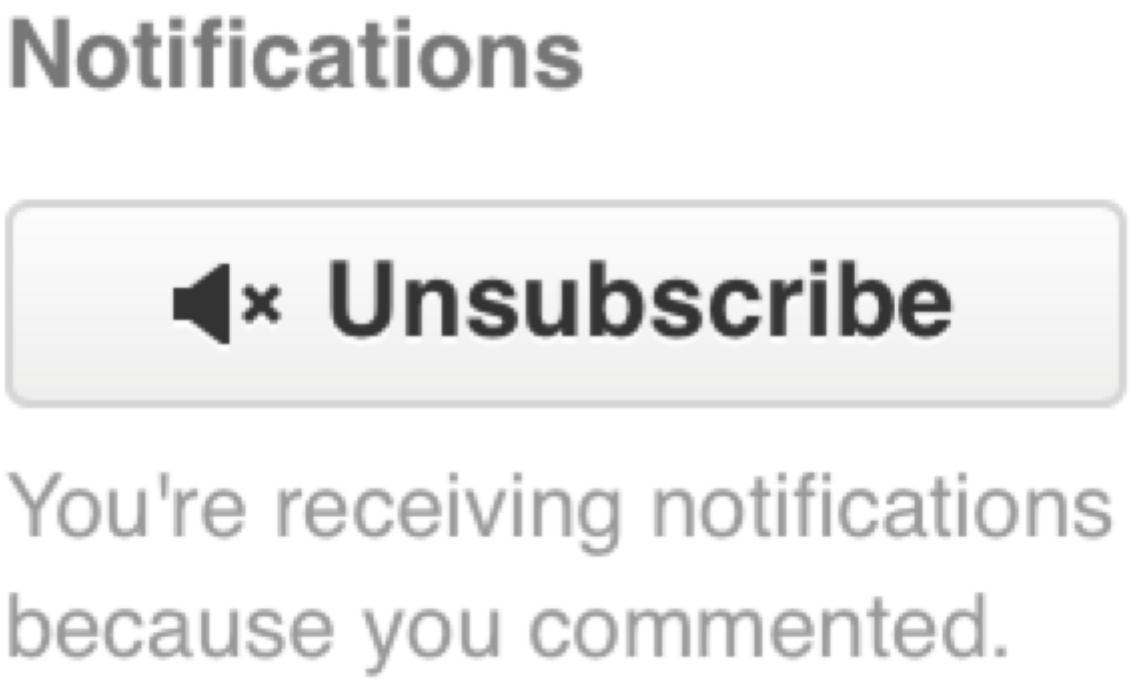


Figure 119. Cancele a inscrição de uma Issue ou Pull Request

A Página de Notificações

Quando mencionamos “notificações” aqui em relação ao GitHub, queremos dizer uma maneira específica que o GitHub tenta entrar em contato com você quando ocorrem eventos e há algumas maneiras diferentes de configurá-las. Se você for para a guia “Notification center” (Centro de notificações) na página de configurações, poderá ver algumas das opções que você tem.

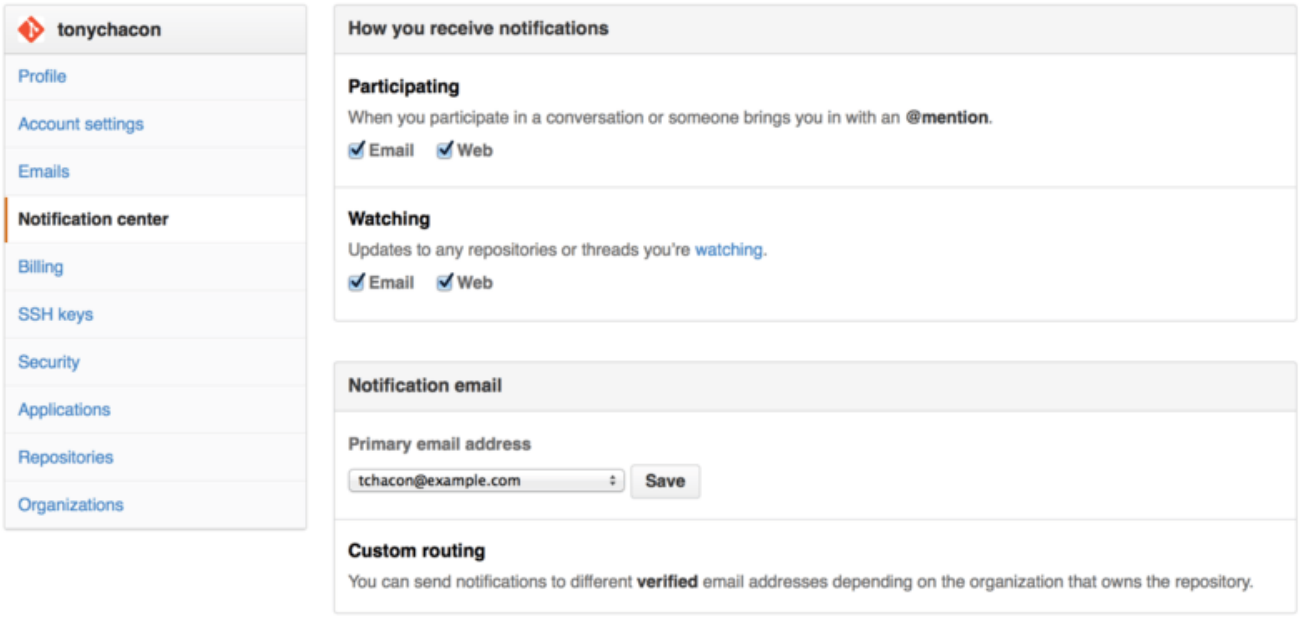


Figure 120. Opções do centro de notificações

As duas opções são receber notificações por “Email” e pela “Web” e você pode escolher uma, nenhuma ou ambas para quando participar ativamente de coisas e para atividades em repositórios que você está observando.

Notificações na Web

Notificações na Web só existem no GitHub e você só pode verificá-las no GitHub. Se você tiver essa opção selecionada em suas preferências e uma notificação for acionada para você, você verá um pequeno ponto azul sobre o ícone de notificações na parte superior da tela, como visto em Centro de notificações.

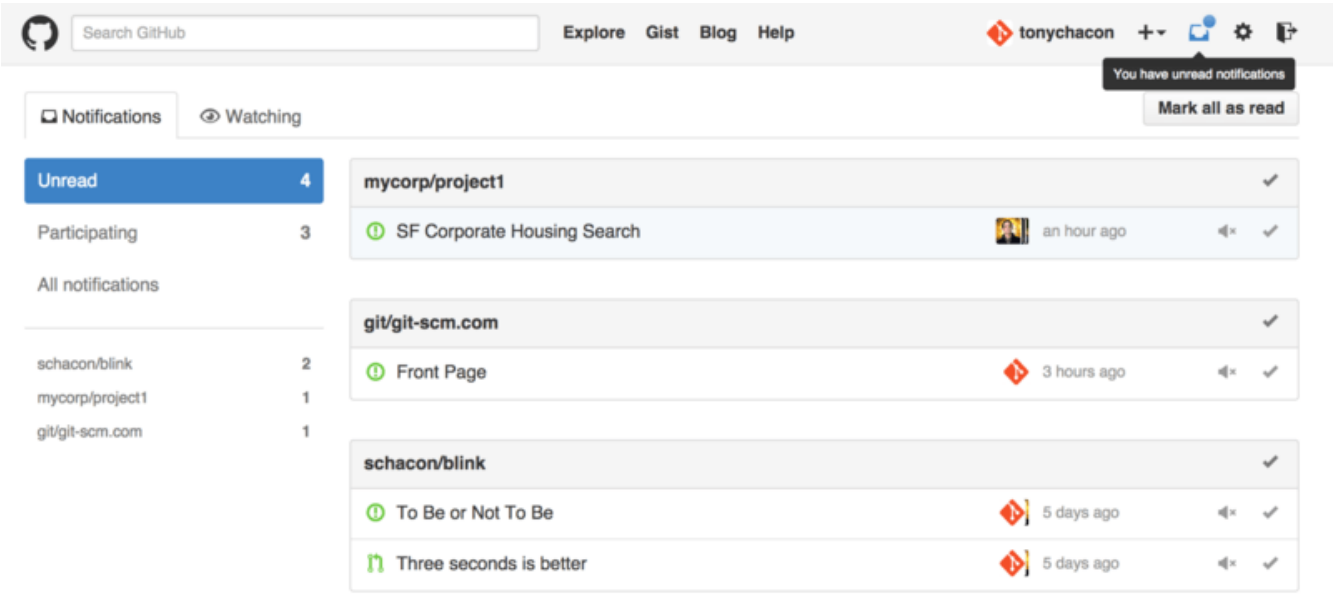


Figure 121. Centro de notificações

Se você clicar nele, verá uma lista de todos os itens sobre os quais foi notificado, agrupados por projeto. Você pode filtrar as notificações de um projeto específico clicando em seu nome na barra

lateral esquerda. Você também pode confirmar a leitura (acknowledge) da notificação clicando no ícone de marca de seleção ao lado de qualquer notificação, ou confirmar a leitura de *todas* as notificações em um projeto clicando na marca de seleção na parte superior do grupo. Há também um botão de silenciar (mute) ao lado de cada marca de seleção que você pode clicar para não receber mais notificações sobre aquele item.

Todas essas ferramentas são muito úteis para lidar com um grande número de notificações. Muitos usuários avançados (power users) do GitHub simplesmente desligarão totalmente as notificações por e-mail e gerenciarão todas as suas notificações através desta tela.

Notificações por E-mail

Notificações por e-mail são a outra forma de lidar com notificações pelo GitHub. Se você tiver isso ativado, receberá e-mails para cada notificação. Vimos exemplos disso em Comentários enviados como notificações por e-mail e Notificação por e-mail de um novo Pull Request. Os e-mails também serão agrupados em threads corretamente, o que é bom se você estiver usando um cliente de e-mail com suporte a threads.

Há também uma boa quantidade de metadados incorporados aos cabeçalhos dos e-mails que o GitHub lhe envia, o que pode ser muito útil para configurar filtros e regras personalizadas.

Por exemplo, se olharmos os cabeçalhos de e-mail reais enviados para Tony no e-mail mostrado em Notificação por e-mail de um novo Pull Request, veremos as seguintes informações entre as enviadas:

```
To: tonychacon/fade <fade@noreply.github.com>
Message-ID: <tonychacon/fade/pull/1@github.com>
Subject: [fade] Wait longer to see the dimming effect better (#1)
X-GitHub-Recipient: tonychacon
List-ID: tonychacon/fade <fade.tonychacon.github.com>
List-Archive: https://github.com/tonychacon/fade
List-Post: <mailto:reply+i-4XXX@reply.github.com>
List-Unsubscribe: <mailto:unsub+i-XXX@reply.github.com>,...
X-GitHub-Recipient-Address: tchacon@example.com
```

Há algumas coisas interessantes aqui. Se você quiser destacar ou redirecionar e-mails para este projeto ou mesmo Pull Request em particular, as informações em `Message-ID` lhe dão todos os dados no formato `<usuario>/<projeto>/<tipo>/<id>`. Se isso fosse uma issue, por exemplo, o campo `<tipo>` teria sido “issues” em vez de “pull”.

Os campos `List-Post` e `List-Unsubscribe` significam que, se você tiver um cliente de e-mail que os compreenda, você pode postar facilmente na lista ou cancelar a inscrição (“Unsubscribe”) na thread. Isso seria essencialmente o mesmo que clicar no botão de silenciar (“mute”) na versão web da notificação ou em “Unsubscribe” na própria página da Issue ou Pull Request.

Também vale a pena notar que se você tiver ambas as notificações por e-mail e pela web ativadas e ler a versão por e-mail da notificação, a versão web também será marcada como lida se você tiver imagens permitidas em seu cliente de e-mail.

Arquivos Especiais

Existem alguns arquivos especiais que o GitHub notará se estiverem presentes em seu repositório.

README

O primeiro é o arquivo `README`, que pode ser de quase qualquer formato que o GitHub reconheça como prosa. Por exemplo, pode ser `README`, `README.md`, `README.asciidoc`, etc. Se o GitHub vir um arquivo `README` no seu código-fonte, ele o renderizará na página de destino (landing page) do projeto.

Muitas equipes usam este arquivo para armazenar todas as informações relevantes do projeto para alguém que possa ser novo no repositório ou projeto. Geralmente, isso inclui coisas como:

- Para que serve o projeto
- Como configurá-lo e instalá-lo
- Um exemplo de como usá-lo ou colocá-lo para rodar
- A licença sob a qual o projeto é oferecido
- Como contribuir com ele

Uma vez que o GitHub renderizará este arquivo, você pode incorporar imagens ou links nele para maior facilidade de compreensão.

CONTRIBUTING

O outro arquivo especial que o GitHub reconhece é o arquivo `CONTRIBUTING`. Se você tiver um arquivo chamado `CONTRIBUTING` com qualquer extensão de arquivo, o GitHub mostrará Abrindo um Pull Request quando existe um arquivo CONTRIBUTING quando alguém começar a abrir um Pull Request.

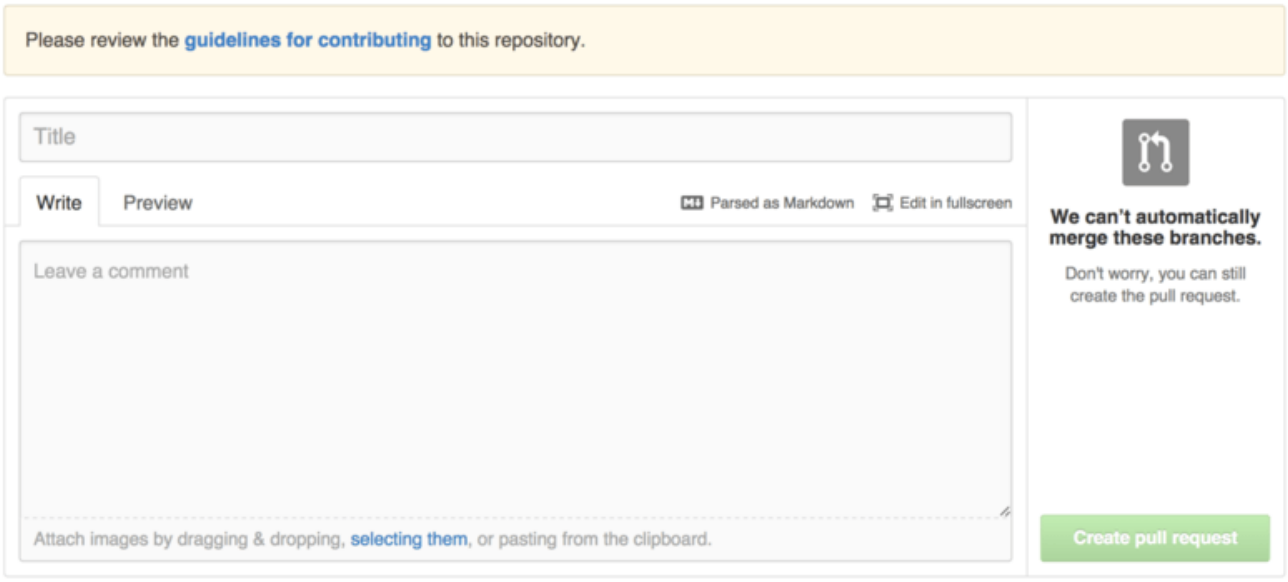


Figure 122. Abrindo um Pull Request quando existe um arquivo CONTRIBUTING

A ideia aqui é que você pode especificar coisas específicas que você quer ou não quer em um Pull

Request enviado para o seu projeto. Dessa forma, as pessoas podem de fato ler as diretrizes antes de abrirem o Pull Request.

Administração de Projeto

Geralmente não há muitas coisas administrativas que você possa fazer com um único projeto, mas há alguns itens que podem ser de interesse.

Mudando o Branch Padrão

Se você estiver usando um branch diferente de “master” como seu branch padrão, no qual você quer que as pessoas abram Pull Requests ou vejam por padrão, você pode alterar isso na página de configurações do seu repositório, na guia “Options” (Opções).

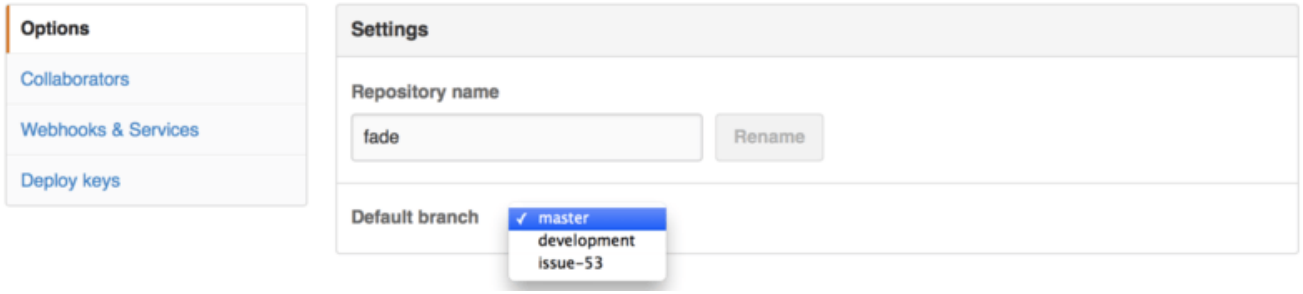


Figure 123. Mude o branch padrão de um projeto

Basta alterar o branch padrão no menu suspenso e ele será o padrão para todas as principais operações a partir de então, incluindo qual branch é feito o checkout por padrão quando alguém clona o repositório.

Transferindo um Projeto

Se você quiser transferir um projeto para outro usuário ou para uma organização no GitHub, há uma opção “Transfer ownership” (Transferir propriedade) na parte inferior da mesma guia “Options” (Opções) da página de configurações do seu repositório que permite fazer isso.

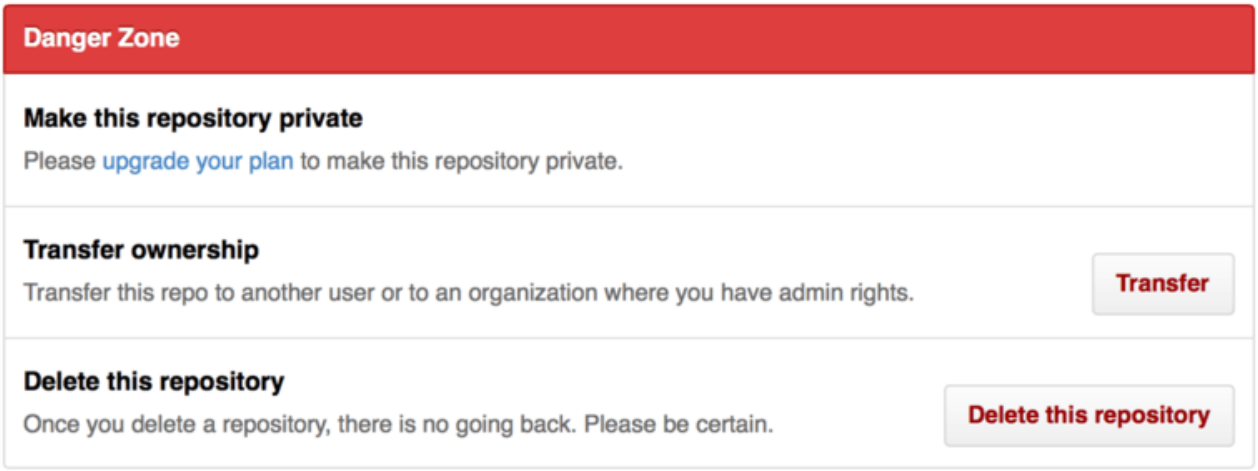


Figure 124. Transfira um projeto para outro usuário ou Organização do GitHub

Isso é útil se você estiver abandonando um projeto e alguém quiser assumi-lo, ou se o seu projeto

estiver crescendo e você quiser movê-lo para uma organização.

Isso não apenas move o repositório, juntamente com todos os seus observadores (watchers) e estrelas (stars), para outro lugar, mas também configura um redirecionamento da sua URL para o novo lugar. Ele também redirecionará clones e fetches do Git, não apenas solicitações da web.

Gerenciando uma Organização

Além das contas de usuário único, o GitHub tem o que são chamadas de Organizações (Organizations). Como as contas pessoais, as contas de Organizações têm um namespace onde todos os seus projetos existem, mas muitas outras coisas são diferentes. Essas contas representam um grupo de pessoas com propriedade compartilhada de projetos e existem muitas ferramentas para gerenciar subgrupos dessas pessoas. Normalmente, essas contas são usadas por grupos Open Source (como “perl” ou “rails”) ou empresas (como “google” ou “twitter”).

O Básico de uma Organização

Uma organização é muito fácil de criar; basta clicar no ícone “+” no canto superior direito de qualquer página do GitHub e selecionar “New organization” (Nova organização) no menu.

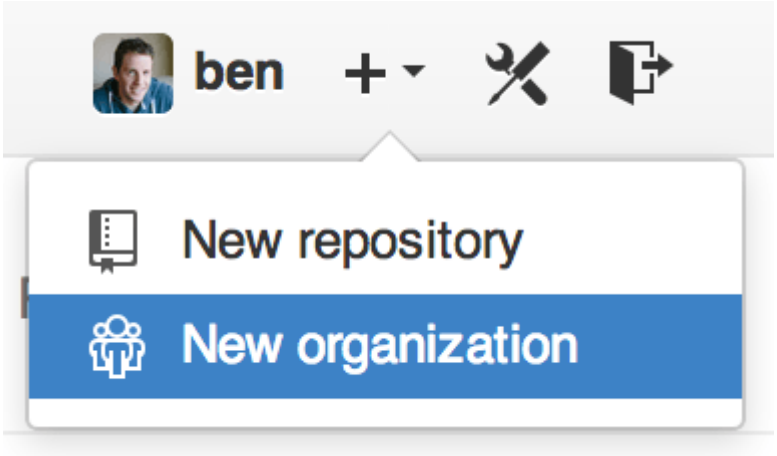


Figure 125. O item de menu “New organization”

Primeiro você precisará nomear a sua organização e fornecer um endereço de e-mail para um ponto de contato principal para o grupo. Em seguida, você pode convidar outros usuários para serem coproprietários da conta, se desejar.

Siga essas etapas e você logo será o proprietário de uma organização novinha em folha. Como as contas pessoais, as organizações são gratuitas se tudo o que você planeja armazenar lá for de código aberto.

Como proprietário de uma organização, ao fazer fork de um repositório, você terá a opção de fazer o fork dele para o namespace da sua organização. Ao criar novos repositórios, você pode criá-los em sua conta pessoal ou em qualquer uma das organizações das quais é proprietário. Você também observa (“watch”) automaticamente qualquer novo repositório criado sob essas organizações.

Assim como em O Seu Avatar, você pode fazer o upload de um avatar para a sua organização para personalizá-la um pouco. Também como as contas pessoais, você tem uma página de destino (landing page) para a organização que lista todos os seus repositórios e pode ser visualizada por

outras pessoas.

Agora vamos cobrir algumas das coisas que são um pouco diferentes com uma conta organizacional.

Equipes (Teams)

Organizações são associadas a pessoas individuais por meio de equipes, que são simplesmente um agrupamento de contas de usuários individuais e repositórios dentro da organização, e o tipo de acesso que essas pessoas têm nesses repositórios.

Por exemplo, digamos que a sua empresa tenha três repositórios: `frontend`, `backend` e `deployscripts`. Você gostaria que os seus desenvolvedores de HTML/CSS/JavaScript tivessem acesso ao `frontend` e talvez ao `backend`, e que o seu pessoal de Operações tivesse acesso ao `backend` e `deployscripts`. As equipes tornam isso fácil, sem ter que gerenciar os colaboradores de cada repositório individual.

A página da Organização mostra um painel simples de todos os repositórios, usuários e equipes que estão sob essa organização.

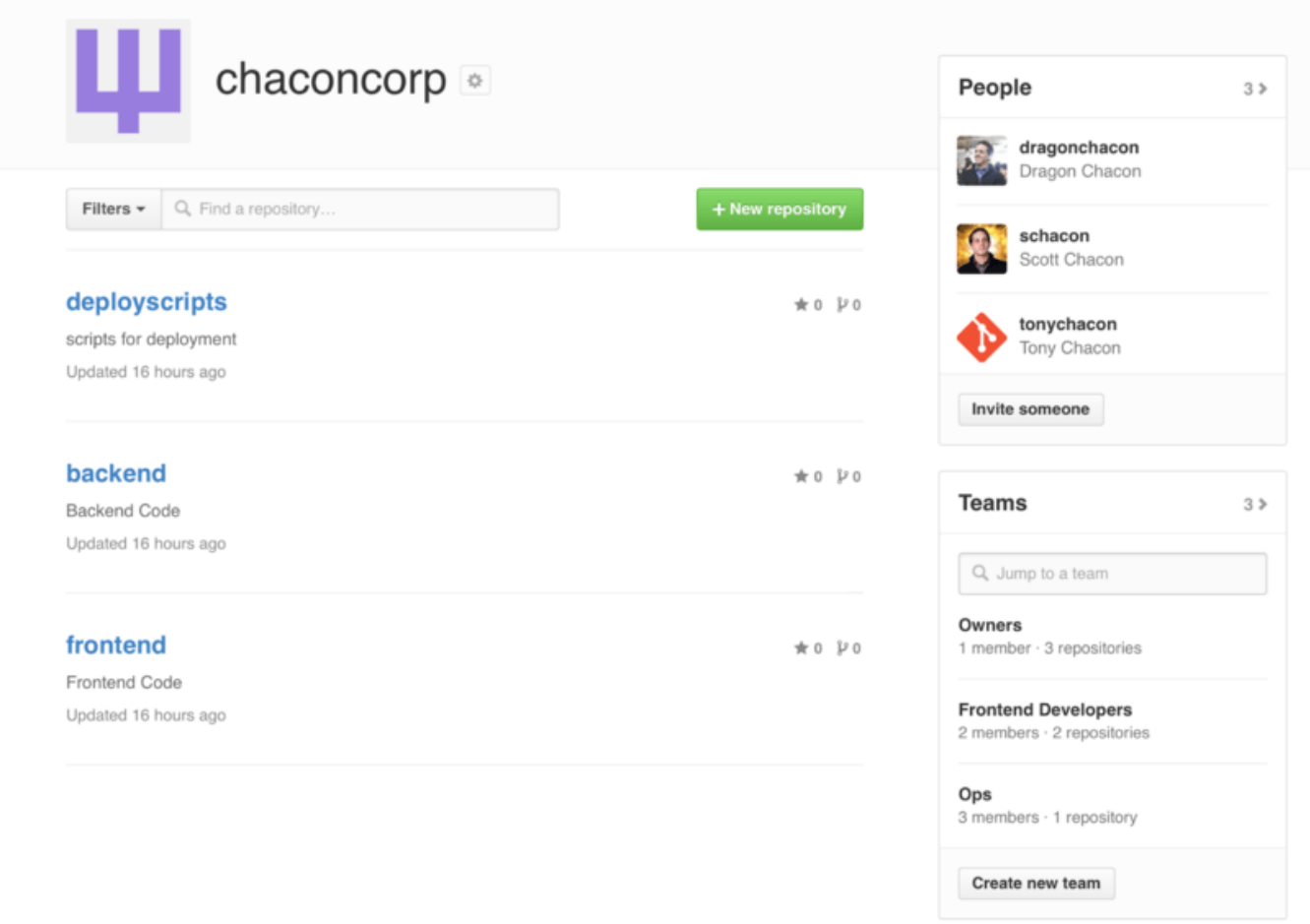


Figure 126. A página da Organização

Para gerenciar as suas Equipes (Teams), você pode clicar na barra lateral Equipes (Teams) no lado direito da página em A página da Organização. Isso levará você a uma página que você pode usar para adicionar membros à equipe, adicionar repositórios à equipe ou gerenciar as configurações e níveis de controle de acesso para a equipe. Cada equipe pode ter acesso somente de leitura, leitura/gravação ou administrativo aos repositórios. Você pode alterar esse nível clicando no botão

“Settings” (Configurações) em A página da Equipe.

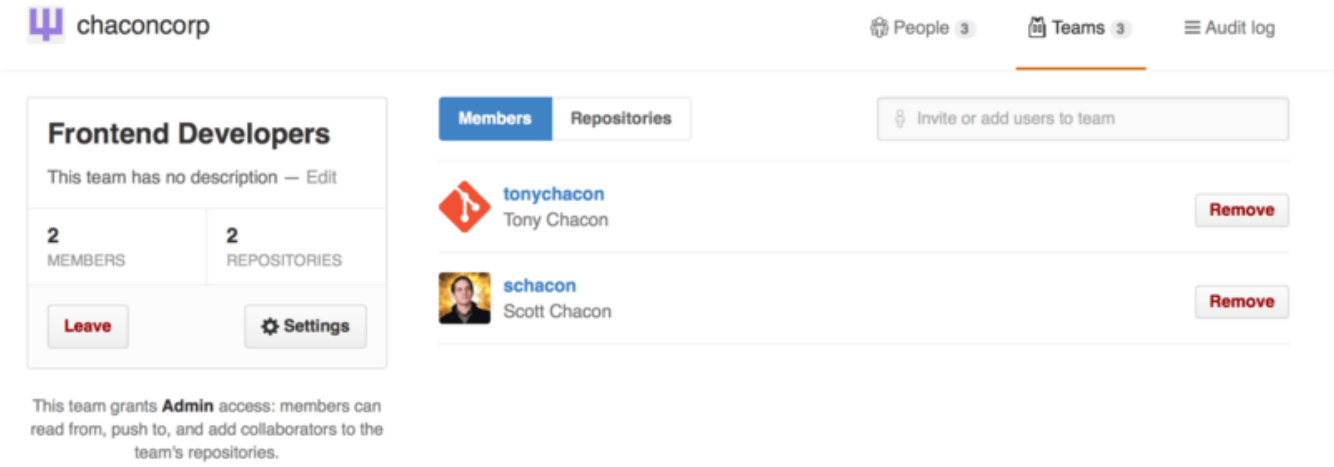


Figure 127. A página da Equipe

Quando você convidar alguém para uma equipe, ele receberá um e-mail avisando que foi convidado.

Além disso, as menções à equipe, `@mentions` (como `@acmecorp/frontend`), funcionam da mesma forma que com usuários individuais, exceto que **todos** os membros da equipe são então inscritos na thread. Isso é útil se você quiser a atenção de alguém de uma equipe, mas não sabe exatamente para quem perguntar.

Um usuário pode pertencer a qualquer número de equipes, então não se limite apenas a equipes de controle de acesso. Equipes de interesse especial como `ux`, `css` ou `refactoring` (refatoração) são úteis para certos tipos de perguntas, e outras como `legal` (jurídico) e `colorblind` (daltônico) para um tipo totalmente diferente.

Log de Auditoria (Audit Log)

As organizações também dão aos proprietários acesso a todas as informações sobre o que aconteceu sob a organização. Você pode ir até a aba 'Audit Log' (Log de Auditoria) e ver quais eventos aconteceram em um nível de organização, quem os fez e em que lugar do mundo eles foram feitos.

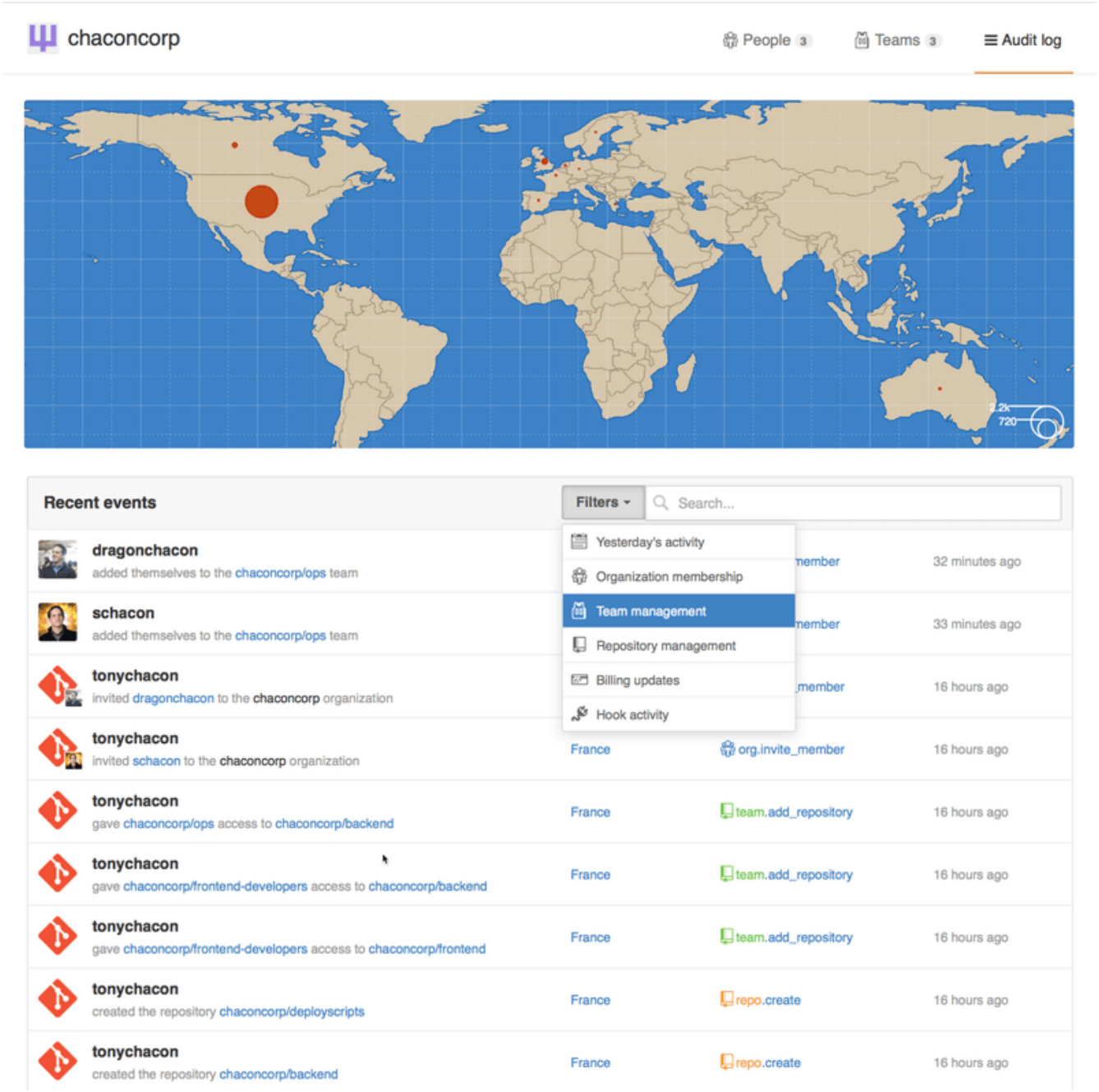


Figure 128. O Log de Auditoria

Você também pode filtrar por tipos específicos de eventos, locais específicos ou pessoas específicas.

Criando Scripts para o GitHub

Então, agora cobrimos todos os principais recursos e fluxos de trabalho do GitHub, mas qualquer grande grupo ou projeto terá personalizações que podem querer fazer ou serviços externos que podem querer integrar.

Para nossa sorte, o GitHub é realmente bastante hackeável de muitas maneiras. Nesta seção cobriremos como usar o sistema de hooks do GitHub e a sua API para fazer o GitHub funcionar como nós queremos.

Serviços e Hooks

A seção Hooks and Services da administração de repositório do GitHub é a maneira mais fácil de fazer o GitHub interagir com sistemas externos.

Serviços

Primeiro daremos uma olhada em Serviços. Ambas as integrações de Hooks e Serviços podem ser encontradas na seção Settings do seu repositório, onde vimos anteriormente como adicionar Colaboradores e alterar o branch padrão do seu projeto. Na aba “Webhooks and Services” (Webhooks e Serviços), você verá algo parecido com Seção de configuração de Serviços e Hooks.

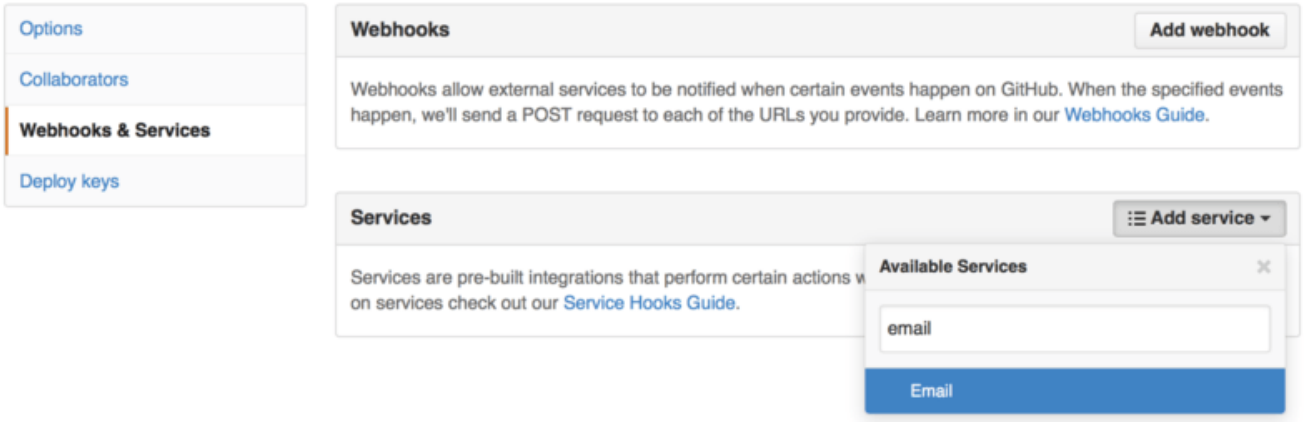


Figure 129. Seção de configuração de Serviços e Hooks

Existem dezenas de serviços para escolher, a maioria integrações com outros sistemas comerciais e de código aberto. A maioria deles é para serviços de Integração Contínua, rastreadores de bugs e issues, sistemas de sala de bate-papo e sistemas de documentação. Passaremos pela configuração de um bem simples, o hook de E-mail. Se você escolher “email” no menu suspenso “Add Service” (Adicionar Serviço), você obterá uma tela de configuração como Configuração de serviço de e-mail.

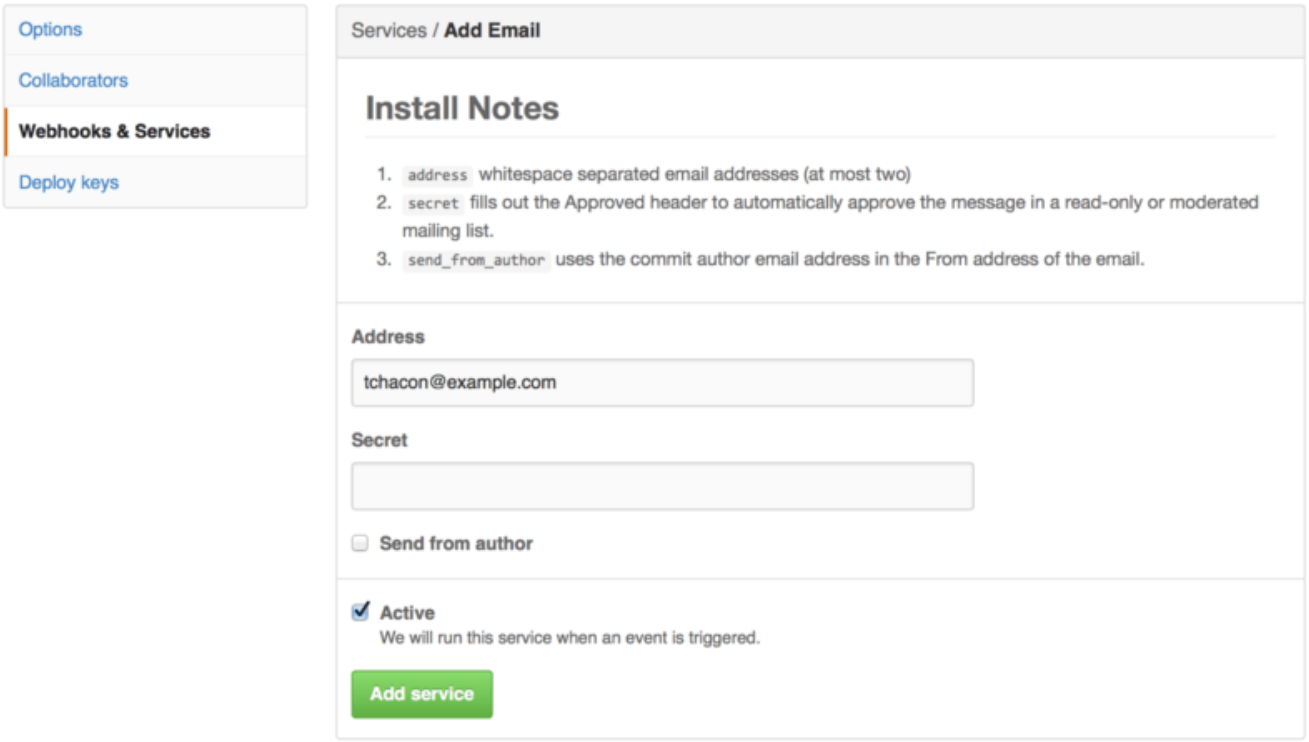


Figure 130. Configuração de serviço de e-mail

Neste caso, se clicarmos no botão “Add service” (Adicionar serviço), o endereço de e-mail que especificamos receberá um e-mail toda vez que alguém fizer um push para o repositório. Os serviços podem escutar muitos tipos diferentes de eventos, mas a maioria apenas escuta eventos de push e, em seguida, faz algo com esses dados.

Se houver um sistema que você esteja usando e que gostaria de integrar com o GitHub, você deve verificar aqui para ver se há uma integração de serviço existente disponível. Por exemplo, se você estiver usando o Jenkins para executar testes em sua base de código, poderá habilitar a integração do serviço embutido do Jenkins para iniciar uma execução de teste toda vez que alguém fizer push no seu repositório.

Hooks

Se você precisar de algo mais específico ou quiser se integrar a um serviço ou site que não está incluído nesta lista, você pode usar o sistema de hooks mais genérico. Os hooks de repositório do GitHub são bem simples. Você especifica uma URL e o GitHub fará o post de um payload HTTP para essa URL em qualquer evento que você desejar.

Geralmente, a maneira como isso funciona é que você pode configurar um pequeno serviço web para escutar um payload do hook do GitHub e então fazer algo com os dados quando eles forem recebidos.

Para ativar um hook, você clica no botão “Add webhook” em Seção de configuração de Serviços e Hooks. Isso o levará a uma página semelhante a Configuração de Web hook.

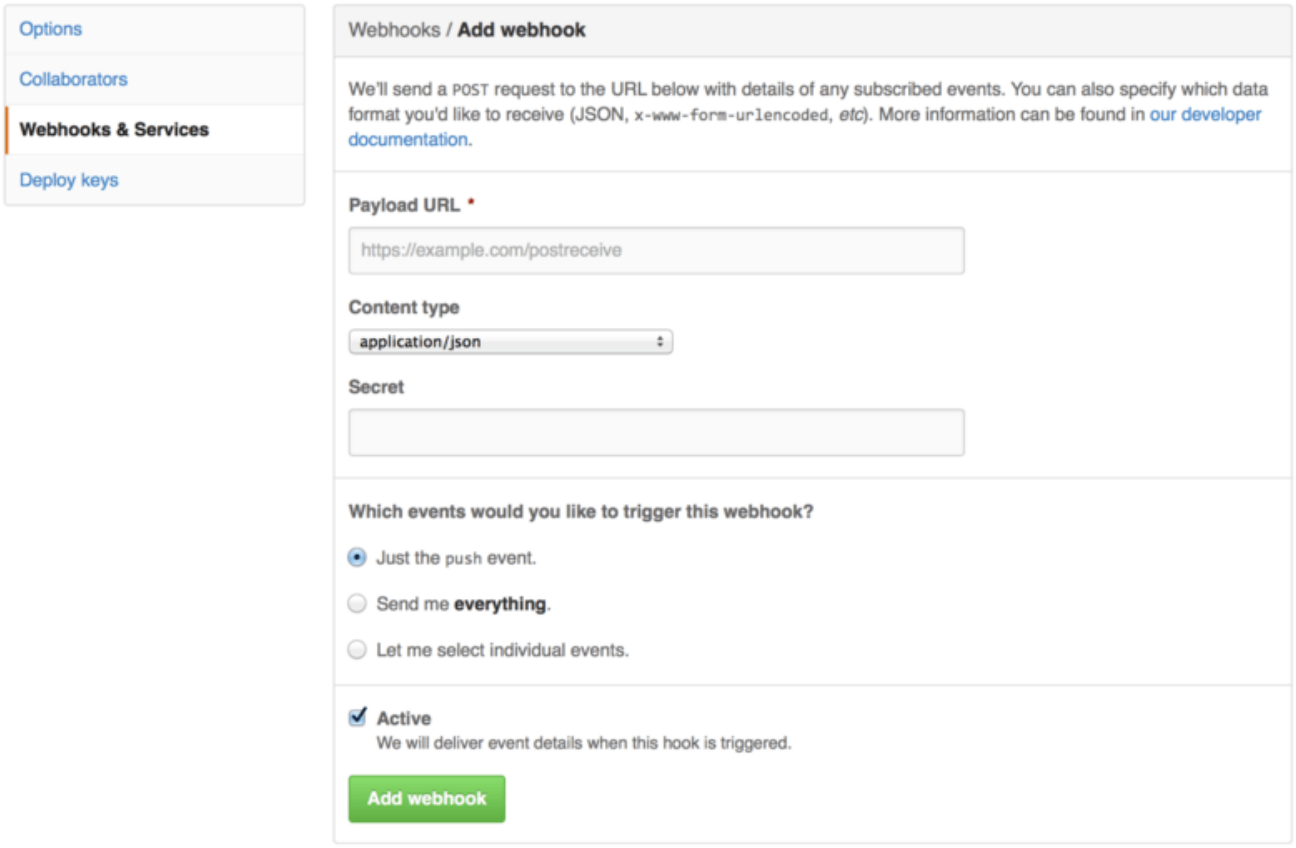


Figure 131. Configuração de Web hook

A configuração para um web hook é bastante simples. Na maioria dos casos, você simplesmente insere uma URL e uma chave secreta e aperta “Add webhook”. Existem algumas opções para quais eventos você deseja que o GitHub envie um payload — o padrão é receber um payload apenas para o evento `push`, quando alguém envia (push) novo código para qualquer branch do seu repositório.

Vejamos um pequeno exemplo de um serviço web que você pode configurar para lidar com um web hook. Nós usaremos o framework web Ruby Sinatra, já que ele é bastante conciso e você deve ser capaz de ver facilmente o que estamos fazendo.

Digamos que queremos receber um e-mail se uma pessoa específica fizer push em um branch específico do nosso projeto, modificando um arquivo específico. Nós poderíamos muito facilmente fazer isso com um código como este:

```
require 'sinatra'
require 'json'
require 'mail'

post '/payload' do
  push = JSON.parse(request.body.read) # parse the JSON

  # gather the data we're looking for
  pusher = push["pusher"]["name"]
  branch = push["ref"]

  # get a list of all the files touched
  files = push["commits"].map do |commit|
```

```
    commit['added'] + commit['modified'] + commit['removed']
  end
  files = files.flatten.uniq

  # check for our criteria
  if pusher == 'schacon' &&
     branch == 'ref/heads/special-branch' &&
     files.include?('special-file.txt')

    Mail.deliver do
      from     'tchacon@example.com'
      to       'tchacon@example.com'
      subject  'Scott Changed the File'
      body     "ALARM"
    end
  end
end
```

Aqui estamos pegando o payload JSON que o GitHub nos entrega e procurando quem fez o push, em qual branch fizeram o push e quais arquivos foram tocados em todos os commits que sofreram push. Em seguida, verificamos isso contra os nossos critérios e enviamos um e-mail caso corresponda.

Para desenvolver e testar algo assim, você tem um console de desenvolvedor legal na mesma tela onde você configura o hook. Você pode ver as últimas entregas que o GitHub tentou fazer para aquele webhook. Para cada hook, você pode analisar quando ele foi entregue, se foi bem-sucedido e o corpo e os cabeçalhos tanto para a requisição quanto para a resposta. Isso torna incrivelmente fácil testar e depurar seus hooks.

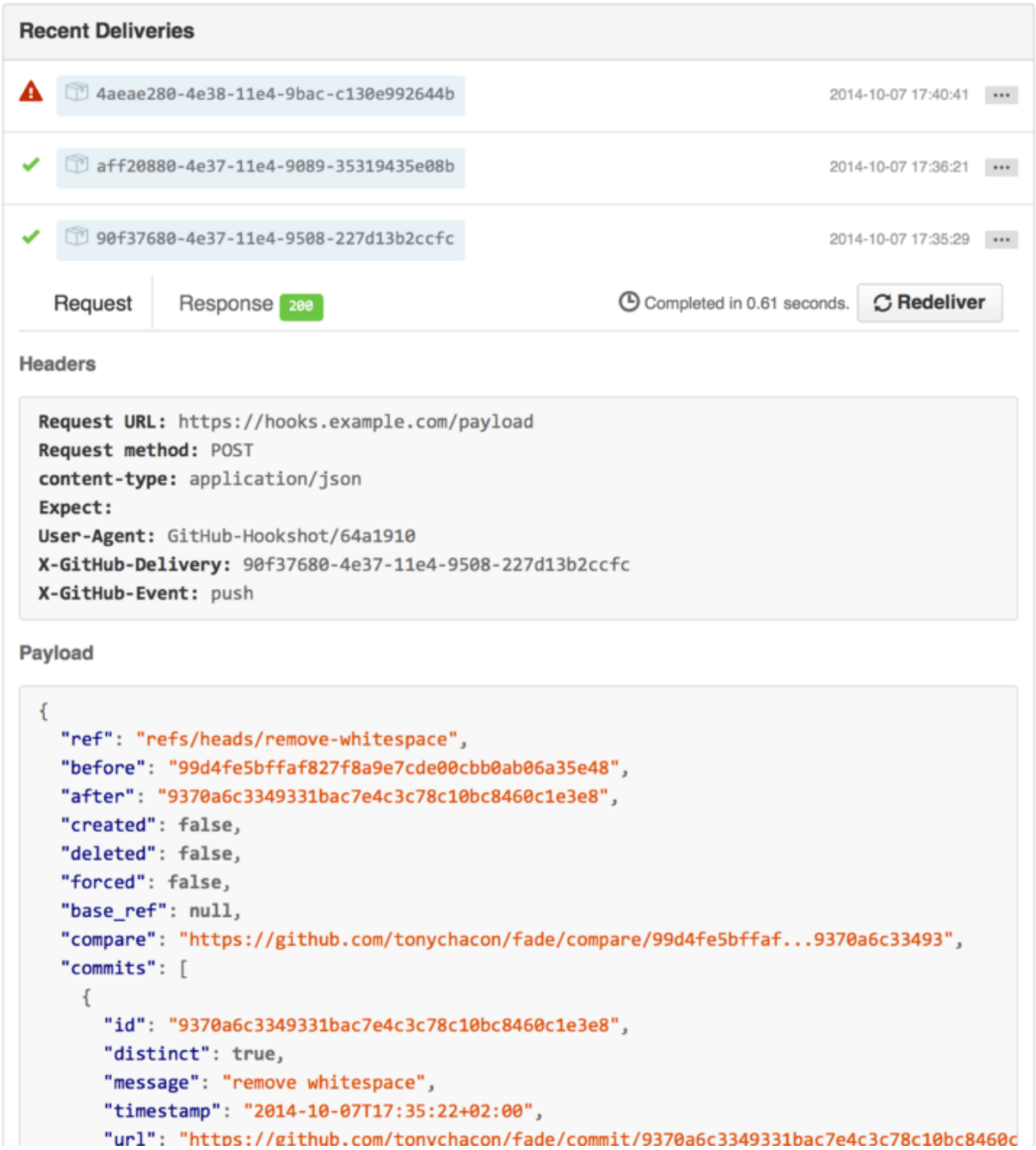


Figure 132. Informações de depuração do Web hook

O outro grande recurso disso é que você pode reenviar qualquer um dos payloads para testar o seu serviço facilmente.

Para mais informações sobre como escrever webhooks e todos os diferentes tipos de eventos que você pode escutar, vá para a documentação do GitHub Developer em https://docs.github.com/en/webhooks-and-events/webhooks/about-webhooks.

A API do GitHub

Serviços e hooks oferecem uma forma de receber notificações push sobre eventos que acontecem nos seus repositórios, mas e se você precisar de mais informações sobre esses eventos? E se você precisar automatizar algo como adicionar colaboradores ou rotular issues?

É aqui que a API do GitHub é útil. O GitHub tem muitos endpoints de API para fazer de forma automatizada quase tudo que você pode fazer no site. Nesta seção, aprenderemos como autenticar e se conectar à API, como comentar em uma issue e como alterar o status de um Pull Request através da API.

Uso Básico

A coisa mais básica que você pode fazer é uma simples requisição GET num endpoint que não requer autenticação. Isto pode ser um usuário ou uma informação somente leitura num projeto de código aberto. Por exemplo, se quisermos saber mais sobre um usuário chamado “schacon”, podemos rodar algo como isto:

```
$ curl https://api.github.com/users/schacon
{
  "login": "schacon",
  "id": 70,
  "avatar_url": "https://avatars.githubusercontent.com/u/70",
# …
  "name": "Scott Chacon",
  "company": "GitHub",
  "following": 19,
  "created_at": "2008-01-27T17:19:28Z",
  "updated_at": "2014-06-10T02:37:23Z"
}
```

Há toneladas de endpoints como esse para obter informações sobre organizações, projetos, issues, commits — basicamente qualquer coisa que você possa ver publicamente no GitHub. Você pode até mesmo usar a API para renderizar Markdown arbitrário ou encontrar um template `.gitignore`.

```
$ curl https://api.github.com/gitignore/templates/Java
{
  "name": "Java",
  "source": "*.class

# Mobile Tools for Java (J2ME)
.mtj.tmp/

# Package Files #
*.jar
*.war
*.ear

# virtual machine crash logs, see
https://www.java.com/en/download/help/error_hotspot.xml
hs_err_pid*
"
}
```

Comentando em uma Issue

No entanto, se você quiser fazer uma ação no site como comentar em uma Issue ou Pull Request ou se você quiser visualizar ou interagir com conteúdo privado, você precisará se autenticar.

Existem várias maneiras de se autenticar. Você pode usar autenticação básica com apenas seu nome de usuário e senha, mas geralmente é uma ideia melhor usar um token de acesso pessoal. Você pode gerar isso a partir da aba “Applications” (Aplicações) da sua página de configurações.

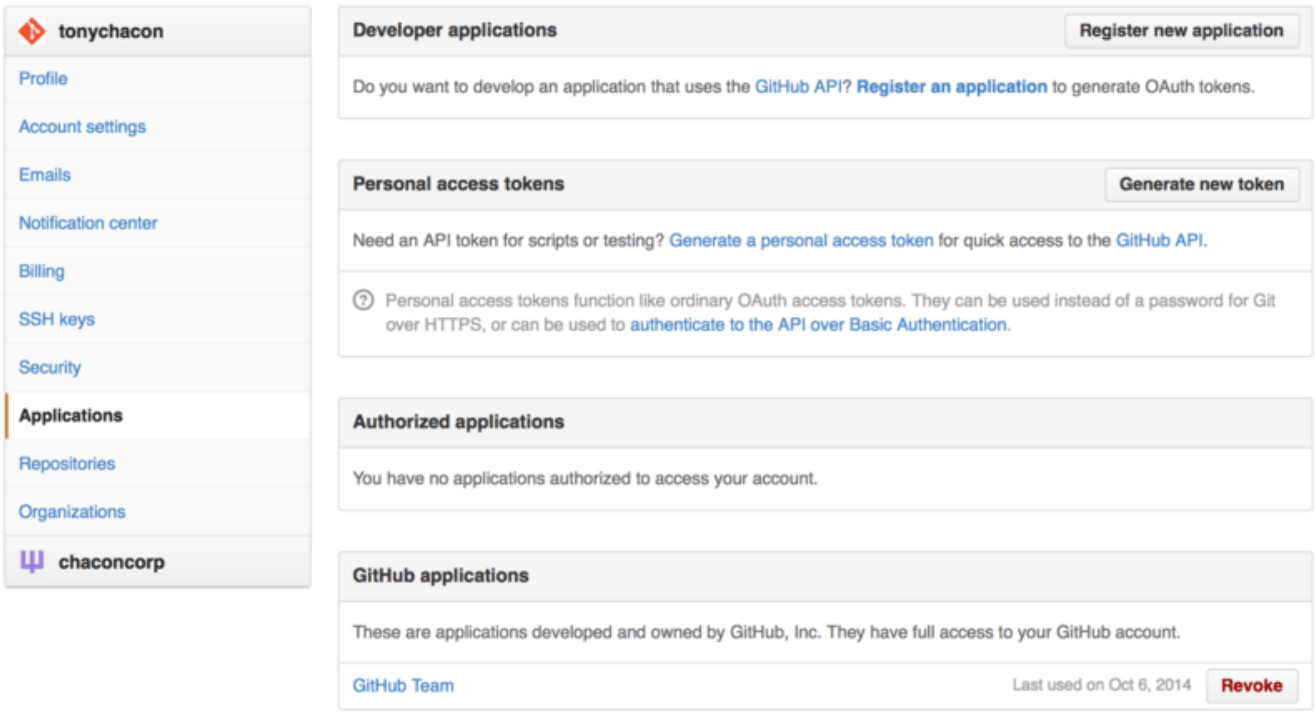


Figure 133. Gere seu token de acesso a partir da aba “Applications” da sua página de configurações

Ele perguntará quais escopos você deseja para esse token e uma descrição. Certifique-se de usar uma boa descrição para se sentir confortável ao remover o token quando o seu script ou aplicação não for mais usado.

O GitHub mostrará o token apenas uma vez, então certifique-se de copiá-lo. Você pode agora usar isso para se autenticar no seu script em vez de usar um nome de usuário e senha. Isso é legal porque você pode limitar o escopo do que deseja fazer e o token é revogável.

Isso também tem a vantagem adicional de aumentar o seu limite de requisições (rate limit). Sem autenticação, você será limitado a 60 requisições por hora. Se você se autenticar, pode fazer até 5.000 requisições por hora.

Então, vamos usá-lo para fazer um comentário em uma de nossas issues. Digamos que queremos deixar um comentário em uma issue específica, a Issue #6. Para fazer isso, temos que fazer uma requisição HTTP POST para `repos/<usuario>/<repo>/issues/<num>/comments` com o token que acabamos de gerar como um cabeçalho Authorization.

```
$ curl -H "Content-Type: application/json" \
       -H "Authorization: token TOKEN" \
       --data '{"body":"A new comment, :+1:"}' \
```

```
      https://api.github.com/repos/schacon/blink/issues/6/comments
{
  "id": 58322100,
  "html_url": "https://github.com/schacon/blink/issues/6#issuecomment-58322100",
  ...
  "user": {
    "login": "tonychacon",
    "id": 7874698,
    "avatar_url": "https://avatars.githubusercontent.com/u/7874698?v=2",
    "type": "User",
  },
  "created_at": "2014-10-08T07:48:19Z",
  "updated_at": "2014-10-08T07:48:19Z",
  "body": "A new comment, :+1:"
}
```

Agora, se você for a essa issue, poderá ver o comentário que acabamos de postar com sucesso, como em Um comentário postado da API do GitHub.

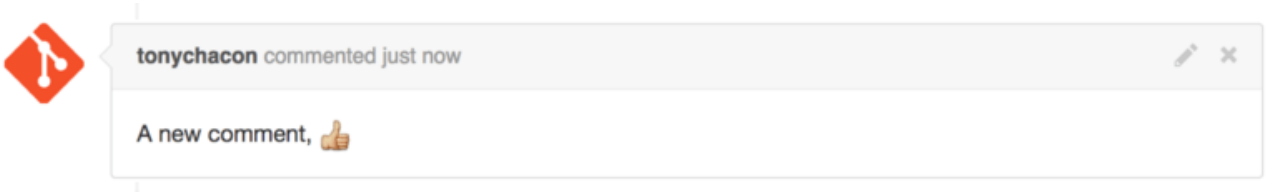


Figure 134. Um comentário postado da API do GitHub

Você pode usar a API para fazer basicamente qualquer coisa que você pode fazer no site — criar e configurar marcos (milestones), atribuir pessoas a Issues e Pull Requests, criar e alterar rótulos (labels), acessar dados de commit, criar novos commits e branches, abrir, fechar ou mesclar Pull Requests, criar e editar equipes, comentar em linhas de código em um Pull Request, pesquisar no site e muito mais.

Alterando o Status de um Pull Request

Há um último exemplo que veremos, já que é realmente útil se você estiver trabalhando com Pull Requests. Cada commit pode ter um ou mais status associados a ele e há uma API para adicionar e consultar esse status.

A maioria dos serviços de Integração Contínua e testes faz uso desta API para reagir a pushes, testando o código que sofreu push, e então relatando de volta se aquele commit passou em todos os testes. Você também poderia usar isso para verificar se a mensagem de commit está formatada corretamente, se o remetente seguiu todas as suas diretrizes de contribuição, se o commit foi validamente assinado — qualquer número de coisas.

Digamos que você configure um webhook no seu repositório que atinge um pequeno serviço web que verifica uma string `Signed-off-by` na mensagem de commit.

```
require 'httparty'
require 'sinatra'
```

```
require 'json'

post '/payload' do
  push = JSON.parse(request.body.read) # parse the JSON
  repo_name = push['repository']['full_name']

  # look through each commit message
  push["commits"].each do |commit|

    # look for a Signed-off-by string
    if /Signed-off-by/.match commit['message']
      state = 'success'
      description = 'Successfully signed off!'
    else
      state = 'failure'
      description = 'No signoff found.'
    end

    # post status to GitHub
    sha = commit["id"]
    status_url = "https://api.github.com/repos/#{repo_name}/statuses/#{sha}"

    status = {
      "state"       => state,
      "description" => description,
      "target_url"  => "http://example.com/how-to-signoff",
      "context"     => "validate/signoff"
    }
    HTTParty.post(status_url,
      :body => status.to_json,
      :headers => {
        'Content-Type'  => 'application/json',
        'User-Agent'    => 'tonychacon/signoff',
        'Authorization' => "token #{ENV['TOKEN']}" }
    )
  end
end
```

Espero que isso seja bastante simples de acompanhar. Neste manipulador de web hook (handler), olhamos cada commit que acabou de sofrer push, procuramos pela string 'Signed-off-by' na mensagem de commit e finalmente fazemos um POST via HTTP para o endpoint da API `/repos/<usuario>/<repo>/statuses/<commit_sha>` com o status.

Neste caso, você pode enviar um estado ('success', 'failure', 'error'), uma descrição do que aconteceu, uma URL de destino (target URL) onde o usuário pode ir para obter mais informações e um “context” (contexto) no caso de haver vários status para um único commit. Por exemplo, um serviço de teste pode fornecer um status e um serviço de validação como este também pode fornecer um status — o campo “context” é como eles são diferenciados.

Se alguém abrir um novo Pull Request no GitHub e este hook estiver configurado, você poderá ver

algo como Status do commit via API.

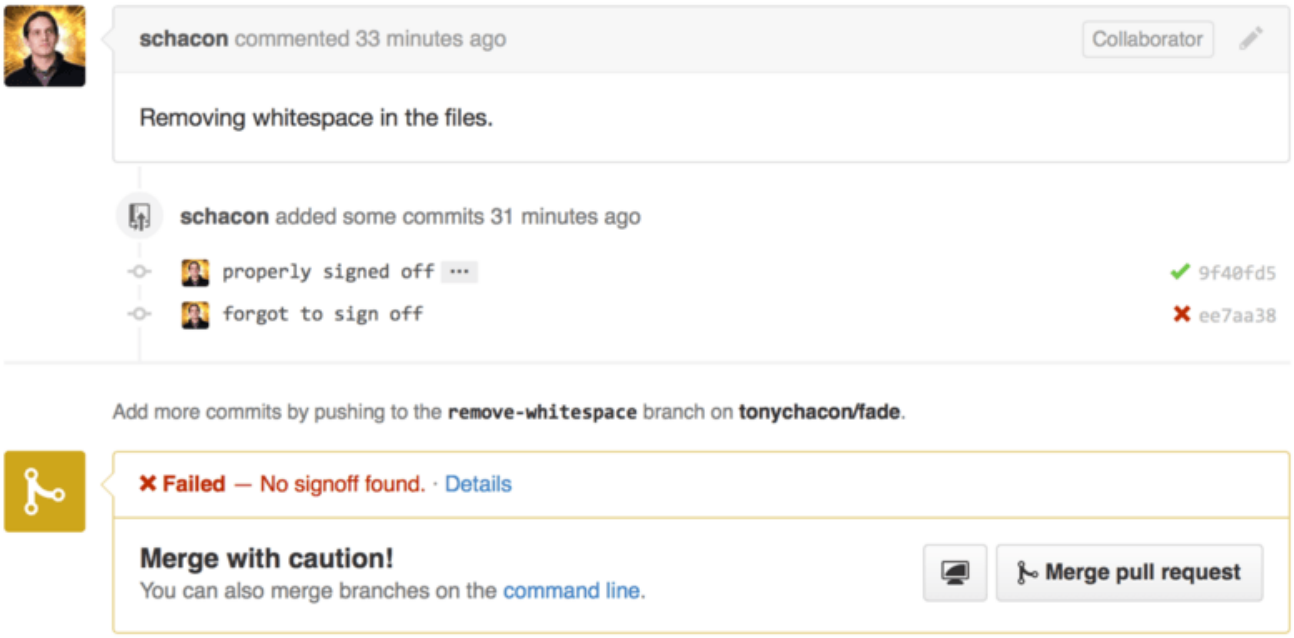


Figure 135. Status do commit via API

Você pode ver agora uma pequena marca de seleção (check mark) verde ao lado do commit que tem uma string "Signed-off-by" na mensagem e uma cruz vermelha naquele em que o autor esqueceu de assinar (sign off). Você também pode ver que o Pull Request pega o status do último commit no branch e o avisa se for uma falha. Isso é realmente útil se você estiver usando esta API para resultados de testes, para que não mescle acidentalmente algo em que o último commit esteja falhando nos testes.

Octokit

Embora tenhamos feito quase tudo através de `curl` e simples requisições HTTP nestes exemplos, existem várias bibliotecas de código aberto que disponibilizam esta API de uma maneira mais idiomática. No momento em que este livro foi escrito, as linguagens suportadas incluíam Go, Objective-C, Ruby e .NET. Confira https://github.com/octokit para obter mais informações sobre eles, pois eles lidam com grande parte do HTTP para você.

Esperamos que estas ferramentas possam ajudá-lo a personalizar e modificar o GitHub para funcionar melhor com os seus fluxos de trabalho específicos. Para a documentação completa de toda a API, bem como guias para tarefas comuns, confira https://docs.github.com/.

Resumo

Agora você é um usuário do GitHub. Você sabe como criar uma conta, gerenciar uma organização, criar e fazer push para repositórios, contribuir para projetos de outras pessoas e aceitar contribuições de terceiros. No próximo capítulo, você aprenderá sobre ferramentas e dicas mais poderosas para lidar com situações complexas, o que fará de você verdadeiramente um mestre no Git.

Ferramentas do Git

A esta altura, você aprendeu a maioria dos comandos e fluxos de trabalho do dia a dia de que precisa para gerenciar ou manter um repositório Git para o seu controle de código-fonte. Você concluiu as tarefas básicas de rastreamento e commit de arquivos, e utilizou o poder da área de stage, do branch de tópicos leve (lightweight topic branching) e merge.

Agora você explorará várias coisas muito poderosas que o Git pode fazer, as quais você pode não usar necessariamente no dia a dia, mas pode precisar em algum momento.

Seleção de Revisão

O Git permite que você se refira a um único commit, conjunto de commits ou intervalo (range) de commits de várias maneiras. Elas não são necessariamente óbvias, mas são úteis de se conhecer.

Revisões Únicas

Você pode obviamente se referir a qualquer commit único pelo seu hash SHA-1 completo de 40 caracteres, mas também existem formas mais amigáveis aos humanos de se referir a commits. Esta seção descreve as várias maneiras pelas quais você pode se referir a qualquer commit.

SHA-1 Curto

O Git é inteligente o suficiente para descobrir a qual commit você está se referindo se você fornecer os primeiros caracteres do hash SHA-1, desde que esse hash parcial tenha pelo menos quatro caracteres e seja inequívoco; ou seja, nenhum outro objeto no banco de dados de objetos pode ter um hash que comece com o mesmo prefixo.

Por exemplo, para examinar um commit específico onde você sabe que adicionou certa funcionalidade, você pode primeiro executar o comando `git log` para localizar o commit:

```
$ git log
commit 734713bc047d87bf7eac9674765ae793478c50d3
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri Jan 2 18:32:33 2009 -0800

    Fix refs handling, add gc auto, update tests

commit d921970aadf03b3cf0e71becdaab3147ba71cdef
Merge: 1c002dd... 35cfb2b...
Author: Scott Chacon <schacon@gmail.com>
Date:   Thu Dec 11 15:08:43 2008 -0800

    Merge commit 'phedders/rdocs'

commit 1c002dd4b536e7479fe34593e72e6c6c1819e53b
Author: Scott Chacon <schacon@gmail.com>
Date:   Thu Dec 11 14:58:32 2008 -0800
```

```
    Add some blame and merge stuff
```

Neste caso, digamos que você esteja interessado no commit cujo hash começa com `1c002dd…`. Você pode inspecionar esse commit com qualquer uma das seguintes variações de `git show` (supondo que as versões mais curtas sejam inequívocas):

```
$ git show 1c002dd4b536e7479fe34593e72e6c6c1819e53b
$ git show 1c002dd4b536e7479f
$ git show 1c002d
```

O Git pode descobrir uma abreviação curta e única para os seus valores SHA-1. Se você passar `--abbrev-commit` para o comando `git log`, a saída usará valores mais curtos, mas os manterá únicos; o padrão é usar sete caracteres, mas os torna mais longos se necessário para manter o SHA-1 inequívoco:

```
$ git log --abbrev-commit --pretty=oneline
ca82a6d Change the version number
085bb3b Remove unnecessary test code
a11bef0 Initial commit
```

Geralmente, oito a dez caracteres são mais do que suficientes para serem únicos em um projeto. Por exemplo, a partir de fevereiro de 2019, o kernel do Linux (que é um projeto bastante considerável) tem mais de 875.000 commits e quase sete milhões de objetos no seu banco de dados de objetos, sem que dois objetos tenham SHA-1 idênticos nos primeiros 12 caracteres.



UMA BREVE NOTA SOBRE O SHA-1

Muitas pessoas começam a se preocupar em algum momento de que elas, por acaso, terão dois objetos distintos em seu repositório com o mesmo valor de hash SHA-1. E aí?

Se você por acaso comitar um objeto que tenha o mesmo valor de hash SHA-1 que um objeto anterior *diferente* no seu repositório, o Git verá o objeto anterior já no seu banco de dados do Git, assumirá que ele já foi gravado e simplesmente o reutilizará. Se você tentar fazer o checkout desse objeto novamente em algum momento, você sempre obterá os dados do primeiro objeto.

No entanto, você deve estar ciente de quão ridiculamente improvável é esse cenário. O digest SHA-1 tem 20 bytes ou 160 bits. O número de objetos com hash aleatório necessários para garantir uma probabilidade de 50% de uma única colisão é cerca de 2^{80} (a fórmula para determinar a probabilidade de colisão é `p = (n(n-1)/2) * (1/2^160)`). 2^{80} é 1,2 x 10^{24} ou 1 milhão de bilhões de bilhões. Isso é 1.200 vezes o número de grãos de areia na Terra.

Aqui está um exemplo para se ter uma ideia do que seria necessário para obter uma colisão SHA-1. Se todos os 6,5 bilhões de humanos na Terra estivessem

> programando, e a cada segundo, cada um estivesse produzindo código que fosse o equivalente a toda a história do kernel do Linux (6,5 milhões de objetos Git) e enviando-o para um enorme repositório Git, demoraria cerca de 2 anos até que o repositório contivesse objetos suficientes para ter 50% de probabilidade de uma única colisão de objeto SHA-1. Assim, uma colisão orgânica do SHA-1 é menos provável do que todos os membros da sua equipe de programação serem atacados e mortos por lobos em incidentes não relacionados na mesma noite.
>
> Se você dedicar milhares de dólares em poder computacional a isso, é possível sintetizar dois arquivos com o mesmo hash, como comprovado em https://shattered.io/ em fevereiro de 2017. O Git está caminhando para o uso do SHA256 como algoritmo de hashing padrão, o qual é muito mais resiliente a ataques de colisão, e tem código no lugar para ajudar a mitigar este ataque (embora não possa eliminá-lo completamente).

Referências de Branch

Uma maneira direta de se referir a um commit em particular é se ele for o commit na ponta (tip) de um branch; nesse caso, você pode simplesmente usar o nome do branch em qualquer comando do Git que espere uma referência a um commit. Por exemplo, se você quiser examinar o último objeto de commit em um branch, os seguintes comandos são equivalentes, assumindo que o branch `topic1` aponte para o commit `ca82a6d···`:

```
$ git show ca82a6dff817ec66f44342007202690a93763949
$ git show topic1
```

Se você quiser ver para qual SHA-1 específico um branch aponta, ou se você quiser ver o que qualquer um desses exemplos se resume em termos de SHA-1s, você pode usar uma ferramenta de encanamento (plumbing) do Git chamada `rev-parse`. Você pode ver Git Internals (Por Dentro do Git) para mais informações sobre ferramentas de plumbing; basicamente, `rev-parse` existe para operações de nível mais baixo e não foi projetado para ser usado em operações do dia a dia. No entanto, às vezes pode ser útil quando você precisar ver o que realmente está acontecendo. Aqui você pode rodar o `rev-parse` no seu branch.

```
$ git rev-parse topic1
ca82a6dff817ec66f44342007202690a93763949
```

Nomes Curtos do RefLog

Uma das coisas que o Git faz em segundo plano enquanto você está trabalhando é manter um "reflog" — um log de onde o seu HEAD e referências de branch estiveram nos últimos meses.

Você pode ver o seu reflog usando `git reflog`:

```
$ git reflog
734713b HEAD@{0}: commit: Fix refs handling, add gc auto, update tests
```

```
d921970 HEAD@{1}: merge phedders/rdocs: Merge made by the 'recursive' strategy.
1c002dd HEAD@{2}: commit: Add some blame and merge stuff
1c36188 HEAD@{3}: rebase -i (squash): updating HEAD
95df984 HEAD@{4}: commit: # This is a combination of two commits.
1c36188 HEAD@{5}: rebase -i (squash): updating HEAD
7e05da5 HEAD@{6}: rebase -i (pick): updating HEAD
```

Toda vez que a ponta do seu branch é atualizada por qualquer motivo, o Git armazena essa informação para você neste histórico temporário. Você também pode usar seus dados do reflog para se referir a commits mais antigos. Por exemplo, se você quiser ver o quinto valor anterior do HEAD do seu repositório, você pode usar a referência `@{5}` que você vê na saída do reflog:

```
$ git show HEAD@{5}
```

Você também pode usar essa sintaxe para ver onde um branch estava há um tempo específico. Por exemplo, para ver onde o seu branch `master` estava ontem, você pode digitar:

```
$ git show master@{yesterday}
```

Isso mostraria a você onde a ponta do seu branch `master` estava ontem. Esta técnica só funciona para dados que ainda estão no seu reflog, então você não pode usá-la para procurar commits mais antigos que alguns meses.

Para ver as informações do reflog formatadas como a saída do `git log`, você pode executar `git log -g`:

```
$ git log -g master
commit 734713bc047d87bf7eac9674765ae793478c50d3
Reflog: master@{0} (Scott Chacon <schacon@gmail.com>)
Reflog message: commit: Fix refs handling, add gc auto, update tests
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri Jan 2 18:32:33 2009 -0800

    Fix refs handling, add gc auto, update tests

commit d921970aadf03b3cf0e71becdaab3147ba71cdef
Reflog: master@{1} (Scott Chacon <schacon@gmail.com>)
Reflog message: merge phedders/rdocs: Merge made by recursive.
Author: Scott Chacon <schacon@gmail.com>
Date:   Thu Dec 11 15:08:43 2008 -0800

    Merge commit 'phedders/rdocs'
```

É importante notar que as informações do reflog são estritamente locais — é um log apenas do que *você* fez no *seu* repositório. As referências não serão as mesmas na cópia do repositório de outra pessoa; além disso, logo após você clonar um repositório inicialmente, você terá um reflog vazio, já

que nenhuma atividade ocorreu ainda no seu repositório. Rodar `git show HEAD@{2.months.ago}` mostrará o commit correspondente apenas se você clonou o projeto há pelo menos dois meses — se você clonou mais recentemente do que isso, verá apenas seu primeiro commit local.



Pense no reflog como a versão do Git do histórico do shell

Se você tem experiência em UNIX ou Linux, pode pensar no reflog como a versão do Git do histórico do shell, o que enfatiza que o que está lá é claramente relevante apenas para você e sua "sessão", e não tem nada a ver com qualquer outra pessoa que possa estar trabalhando na mesma máquina.



Escapando chaves no PowerShell

Ao usar o PowerShell, as chaves como `{` e `}` são caracteres especiais e devem ser escapados. Você pode escapá-los com uma crase `` ` `` ou colocar a referência do commit entre aspas:

```
$ git show HEAD@{0}     # will NOT work
$ git show HEAD@`{0`}   # OK
$ git show "HEAD@{0}"   # OK
```

Referências de Ascendência (Ancestry)

A outra maneira principal de especificar um commit é por meio de sua ascendência. Se você colocar um `^` (acento circunflexo) no final de uma referência, o Git resolverá isso para significar o pai daquele commit. Suponha que você observe o histórico do seu projeto:

```
$ git log --pretty=format:'%h %s' --graph
* 734713b Fix refs handling, add gc auto, update tests
*   d921970 Merge commit 'phedders/rdocs'
|\
| * 35cfb2b Some rdoc changes
* | 1c002dd Add some blame and merge stuff
|/
* 1c36188 Ignore *.gem
* 9b29157 Add open3_detach to gemspec file list
```

Então, você pode ver o commit anterior especificando `HEAD^`, que significa "o pai do HEAD":

```
$ git show HEAD^
commit d921970aadf03b3cf0e71becdaab3147ba71cdef
Merge: 1c002dd... 35cfb2b...
Author: Scott Chacon <schacon@gmail.com>
Date:   Thu Dec 11 15:08:43 2008 -0800

    Merge commit 'phedders/rdocs'
```



Escapando o acento circunflexo no Windows

No Windows em `cmd.exe`, `^` é um caractere especial e precisa ser tratado de forma diferente. Você pode duplicá-lo ou colocar a referência do commit entre aspas:

```
$ git show HEAD^     # will NOT work on Windows
$ git show HEAD^^    # OK
$ git show "HEAD^"   # OK
```

Você também pode especificar um número após o `^` para identificar *qual* pai você deseja; por exemplo, `d921970^2` significa "o segundo pai de d921970". Esta sintaxe é útil apenas para commits de merge, que têm mais de um pai — o *primeiro* pai de um commit de merge é do branch em que você estava quando fez o merge (frequentemente `master`), enquanto o *segundo* pai de um commit de merge é do branch que foi mesclado (digamos, `topic`):

```
$ git show d921970^
commit 1c002dd4b536e7479fe34593e72e6c6c1819e53b
Author: Scott Chacon <schacon@gmail.com>
Date:   Thu Dec 11 14:58:32 2008 -0800

    Add some blame and merge stuff

$ git show d921970^2
commit 35cfb2b795a55793d7cc56a6cc2060b4bb732548
Author: Paul Hedderly <paul+git@mjr.org>
Date:   Wed Dec 10 22:22:03 2008 +0000

    Some rdoc changes
```

A outra principal especificação de ascendência é o `~` (til). Isto também se refere ao primeiro pai, então `HEAD~` e `HEAD^` são equivalentes. A diferença torna-se aparente quando você especifica um número. `HEAD~2` significa "o primeiro pai do primeiro pai" ou "o avô" — ele percorre os primeiros pais o número de vezes que você especificar. Por exemplo, no histórico listado anteriormente, `HEAD~3` seria:

```
$ git show HEAD~3
commit 1c3618887afb5fbcbea25b7c013f4e2114448b8d
Author: Tom Preston-Werner <tom@mojombo.com>
Date:   Fri Nov 7 13:47:59 2008 -0500

    Ignore *.gem
```

Isso também pode ser escrito como `HEAD~~~`, que novamente é o primeiro pai do primeiro pai do primeiro pai:

```
$ git show HEAD~~~
```

```
commit 1c3618887afb5fbcbea25b7c013f4e2114448b8d
Author: Tom Preston-Werner <tom@mojombo.com>
Date:   Fri Nov 7 13:47:59 2008 -0500

    Ignore *.gem
```

Você também pode combinar essas sintaxes — você pode obter o segundo pai da referência anterior (supondo que tenha sido um commit de merge) usando `HEAD~3^2`, e assim por diante.

Intervalos (Ranges) de Commits

Agora que você pode especificar commits individuais, vamos ver como especificar intervalos de commits. Isso é particularmente útil para gerenciar seus branches — se você tem muitos branches, pode usar as especificações de intervalo para responder a perguntas como: "Qual trabalho está neste branch que eu ainda não mesclei (merged) no meu branch principal?"

Ponto Duplo (Double Dot)

A especificação de intervalo mais comum é a sintaxe de ponto duplo. Isto basicamente pede ao Git para resolver um intervalo de commits que são alcançáveis a partir de um commit, mas não são alcançáveis a partir de outro. Por exemplo, digamos que você tenha um histórico de commits parecido com Exemplo de histórico para seleção de intervalo.

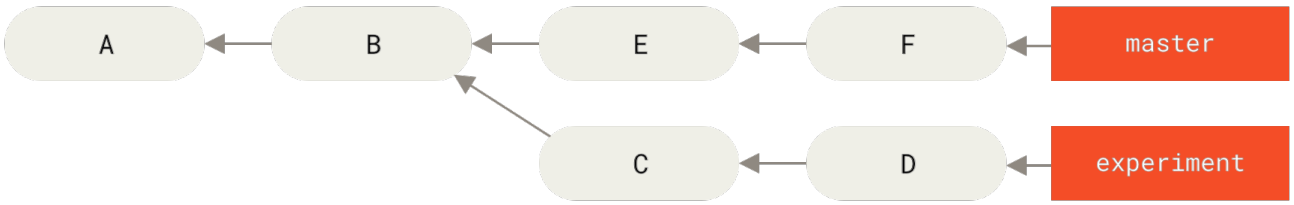


Figure 136. Exemplo de histórico para seleção de intervalo

Digamos que você queira ver o que está no seu branch `experiment` que ainda não foi mesclado (merged) no seu branch `master`. Você pode pedir ao Git para mostrar um log de apenas esses commits com `master..experiment` — isso significa "todos os commits alcançáveis a partir de `experiment` que não são alcançáveis a partir de `master`." Por uma questão de brevidade e clareza nesses exemplos, as letras dos objetos de commit do diagrama são usadas no lugar da saída de log real, na ordem em que seriam exibidas:

```
$ git log master..experiment
D
C
```

Se, por outro lado, você quiser ver o oposto — todos os commits no `master` que não estão no `experiment` — você pode inverter os nomes dos branches. `experiment..master` mostra tudo o que está no `master` e não é alcançável a partir do `experiment`:

```
$ git log experiment..master
F
```

```
E
```

Isso é útil se você quiser manter o branch `experiment` atualizado e visualizar o que está prestes a mesclar. Outro uso frequente dessa sintaxe é ver o que você está prestes a fazer push (enviar) para um repositório remoto:

```
$ git log origin/master..HEAD
```

Este comando mostra quaisquer commits no seu branch atual que não estão no branch `master` no seu repositório remoto `origin`. Se você executar um `git push` e o seu branch atual estiver rastreando `origin/master`, os commits listados por `git log origin/master..HEAD` são os commits que serão transferidos para o servidor. Você também pode omitir um lado da sintaxe para fazer com que o Git assuma `HEAD`. Por exemplo, você pode obter os mesmos resultados do exemplo anterior digitando `git log origin/master..` — o Git substitui `HEAD` se um dos lados estiver faltando.

Múltiplos Pontos

A sintaxe de ponto duplo é útil como um atalho, mas talvez você queira especificar mais do que dois branches para indicar a sua revisão, como ver quais commits estão em qualquer um de vários branches que não estão no branch em que você se encontra atualmente. O Git permite que você faça isso usando o caractere `^` ou `--not` antes de qualquer referência da qual você não deseja ver os commits alcançáveis. Assim, os três comandos a seguir são equivalentes:

```
$ git log refA..refB
$ git log ^refA refB
$ git log refB --not refA
```

Isso é legal porque com essa sintaxe você pode especificar mais de duas referências na sua consulta, o que não pode fazer com a sintaxe de ponto duplo. Por exemplo, se você quiser ver todos os commits que são alcançáveis a partir do `refA` ou `refB` mas não do `refC`, você pode usar qualquer um destes:

```
$ git log refA refB ^refC
$ git log refA refB --not refC
```

Isso cria um sistema de consulta de revisão muito poderoso que deve ajudá-lo a descobrir o que há nos seus branches.

Ponto Triplo (Triple Dot)

A última grande sintaxe de seleção de intervalo é a sintaxe de ponto triplo, que especifica todos os commits que são alcançáveis por *qualquer uma* das duas referências, mas não por ambas. Veja novamente o histórico de commits de exemplo em Exemplo de histórico para seleção de intervalo. Se você quiser ver o que está no `master` ou `experiment`, mas sem quaisquer referências comuns, você pode executar:

```
$ git log master...experiment
F
E
D
C
```

Mais uma vez, isso fornece a saída normal de `log`, mas mostra apenas as informações de commit para aqueles quatro commits, aparecendo na ordem tradicional de data de commit.

Um switch comum para usar com o comando `log` neste caso é o `--left-right`, que mostra em qual lado do intervalo cada commit está. Isso ajuda a tornar a saída mais útil:

```
$ git log --left-right master...experiment
< F
< E
> D
> C
```

Com essas ferramentas, você pode informar ao Git de forma muito mais fácil qual ou quais commits você deseja inspecionar.

Área de Stage Interativa

Nesta seção, você verá alguns comandos interativos do Git que podem ajudá-lo a criar seus commits para incluir apenas certas combinações e partes de arquivos. Essas ferramentas são úteis se você modificar vários arquivos extensivamente, então decide que quer que essas alterações sejam divididas em vários commits focados em vez de um único commit grande e confuso. Dessa forma, você pode certificar-se de que seus commits são conjuntos de alterações separados logicamente e podem ser revisados facilmente pelos desenvolvedores que trabalham com você.

Se você executar `git add` com a opção `-i` ou `--interactive`, o Git entra em um modo shell interativo, exibindo algo como isto:

```
$ git add -i
           staged     unstaged path
  1:    unchanged        +0/-1 TODO
  2:    unchanged        +1/-1 index.html
  3:    unchanged        +5/-1 lib/simplegit.rb

*** Commands ***
  1: [s]tatus     2: [u]pdate      3: [r]evert     4: [a]dd untracked
  5: [p]atch      6: [d]iff        7: [q]uit       8: [h]elp
What now>
```

Você pode ver que este comando mostra uma visão da sua área de stage muito diferente da que você provavelmente está acostumado — basicamente, a mesma informação que você obtém com

`git status`, mas um pouco mais sucinta e informativa. Ela lista as alterações que estão na área de stage à esquerda e as não preparadas (unstaged) à direita.

Depois disso vem uma seção "Commands" (Comandos), que permite fazer uma série de coisas como preparar e retirar arquivos da área de stage, preparar partes de arquivos, adicionar arquivos não rastreados e exibir diffs do que foi preparado (staged).

Adicionando e Removendo Arquivos do Stage

Se você digitar `u` ou `2` (para update) no prompt `What now>`, você será questionado sobre quais arquivos deseja preparar (stage):

```
What now> u
           staged     unstaged path
  1:    unchanged        +0/-1 TODO
  2:    unchanged        +1/-1 index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
Update>>
```

Para preparar (stage) os arquivos `TODO` e `index.html`, você pode digitar os números:

```
Update>> 1,2
           staged     unstaged path
* 1:    unchanged        +0/-1 TODO
* 2:    unchanged        +1/-1 index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
Update>>
```

O `*` próximo a cada arquivo significa que o arquivo foi selecionado para ser preparado. Se você pressionar Enter após não digitar nada no prompt `Update>>`, o Git pegará qualquer coisa selecionada e a preparará para você:

```
Update>>
updated 2 paths

*** Commands ***
  1: [s]tatus     2: [u]pdate      3: [r]evert     4: [a]dd untracked
  5: [p]atch      6: [d]iff        7: [q]uit       8: [h]elp
What now> s
           staged     unstaged path
  1:        +0/-1      nothing TODO
  2:        +1/-1      nothing index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
```

Agora você pode ver que os arquivos `TODO` e `index.html` estão preparados (staged) e o arquivo `simplegit.rb` ainda não está (unstaged). Se você deseja remover (unstage) o arquivo `TODO` neste momento, use a opção `r` ou `3` (para revert):

```
*** Commands ***
  1: [s]tatus     2: [u]pdate      3: [r]evert     4: [a]dd untracked
  5: [p]atch      6: [d]iff        7: [q]uit       8: [h]elp
What now> r
           staged     unstaged path
  1:        +0/-1      nothing TODO
  2:        +1/-1      nothing index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
Revert>> 1
           staged     unstaged path
* 1:        +0/-1      nothing TODO
  2:        +1/-1      nothing index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
Revert>> [enter]
reverted one path
```

Olhando para o seu status do Git novamente, você pode ver que você retirou o arquivo `TODO` do stage:

```
*** Commands ***
  1: [s]tatus     2: [u]pdate      3: [r]evert     4: [a]dd untracked
  5: [p]atch      6: [d]iff        7: [q]uit       8: [h]elp
What now> s
           staged     unstaged path
  1:    unchanged        +0/-1 TODO
  2:        +1/-1      nothing index.html
  3:    unchanged        +5/-1 lib/simplegit.rb
```

Para ver o diff do que você preparou, você pode usar o comando `d` ou `6` (para diff). Ele mostra uma lista dos seus arquivos preparados, e você pode selecionar aqueles para os quais gostaria de ver o diff. Isso é muito parecido com especificar `git diff --cached` na linha de comando:

```
*** Commands ***
  1: [s]tatus     2: [u]pdate      3: [r]evert     4: [a]dd untracked
  5: [p]atch      6: [d]iff        7: [q]uit       8: [h]elp
What now> d
           staged     unstaged path
  1:        +1/-1      nothing index.html
Review diff>> 1
diff --git a/index.html b/index.html
index 4d07108..4335f49 100644
--- a/index.html
+++ b/index.html
@@ -16,7 +16,7 @@ Date Finder

 <p id="out">...</p>

-<div id="footer">contact : support@github.com</div>
```

```
+<div id="footer">contact : email.support@github.com</div>

 <script type="text/javascript">
```

Com esses comandos básicos, você pode usar o modo interativo de adição para lidar com a sua área de stage com um pouco mais de facilidade.

Patches na Área de Stage

Também é possível para o Git preparar (stage) certas *partes* de arquivos e não o resto. Por exemplo, se você fizer duas alterações no seu arquivo `simplegit.rb` e quiser preparar uma delas e não a outra, fazer isso é muito fácil no Git. A partir do mesmo prompt interativo explicado na seção anterior, digite `p` ou `5` (para patch). O Git perguntará quais arquivos você gostaria de preparar parcialmente; então, para cada seção dos arquivos selecionados, ele exibirá pedaços (hunks) do diff do arquivo e perguntará se você gostaria de prepará-los, um por um:

```
diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index dd5ecc4..57399e0 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -22,7 +22,7 @@ class SimpleGit
   end

   def log(treeish = 'master')
-    command("git log -n 25 #{treeish}")
+    command("git log -n 30 #{treeish}")
   end

   def blame(path)
Stage this hunk [y,n,a,d,/,j,J,g,e,?]?
```

Você tem muitas opções neste ponto. Digitar `?` mostra uma lista do que você pode fazer:

```
Stage this hunk [y,n,a,d,/,j,J,g,e,?]? ?
y - stage this hunk
n - do not stage this hunk
a - stage this and all the remaining hunks in the file
d - do not stage this hunk nor any of the remaining hunks in the file
g - select a hunk to go to
/ - search for a hunk matching the given regex
j - leave this hunk undecided, see next undecided hunk
J - leave this hunk undecided, see next hunk
k - leave this hunk undecided, see previous undecided hunk
K - leave this hunk undecided, see previous hunk
s - split the current hunk into smaller hunks
e - manually edit the current hunk
? - print help
```

Geralmente, você digitará `y` ou `n` se quiser preparar (stage) cada pedaço (hunk), mas preparar todos eles em certos arquivos ou pular a decisão de um hunk para mais tarde também pode ser útil. Se você preparar (stage) uma parte do arquivo e deixar outra parte não preparada (unstaged), a saída do seu status ficará assim:

```
What now> 1
           staged     unstaged path
  1:    unchanged        +0/-1 TODO
  2:        +1/-1      nothing index.html
  3:        +1/-1        +4/-0 lib/simplegit.rb
```

O status do arquivo `simplegit.rb` é interessante. Ele mostra que algumas linhas estão na área de stage e algumas não estão. Você preparou parcialmente este arquivo. Neste ponto, você pode sair do script de adição interativa e executar `git commit` para comitar os arquivos parcialmente preparados.

Você também não precisa estar no modo de adição interativa para fazer o stage parcial do arquivo — você pode iniciar o mesmo script usando `git add -p` ou `git add --patch` na linha de comando.

Além disso, você pode usar o modo patch para resetar parcialmente arquivos com o comando `git reset --patch`, para fazer checkout de partes de arquivos com o comando `git checkout --patch` e para fazer stash de partes de arquivos com o comando `git stash save --patch`. Entraremos em mais detalhes sobre cada um deles quando chegarmos a usos mais avançados desses comandos.

Fazendo Stash e Limpando

Frequentemente, quando você está trabalhando em parte do seu projeto, as coisas estão em um estado confuso e você quer mudar de branch por um tempo para trabalhar em outra coisa. O problema é que você não quer fazer um commit de um trabalho feito pela metade só para poder voltar a este ponto mais tarde. A resposta para esse problema é o comando `git stash`.

O stashing pega o estado sujo (dirty) do seu diretório de trabalho — ou seja, seus arquivos modificados rastreados e alterações preparadas (staged) — e o salva em uma pilha de alterações não concluídas que você pode reaplicar a qualquer momento (mesmo em um branch diferente).



Migrando para `git stash push`

A partir do final de outubro de 2017, houve uma extensa discussão na lista de discussão do Git, na qual o comando `git stash save` está sendo descontinuado (deprecated) em favor da alternativa existente `git stash push`. O principal motivo para isso é que o `git stash push` introduz a opção de fazer stash de *pathspecs* selecionados, algo que o `git stash save` não suporta.

O `git stash save` não vai desaparecer tão cedo, então não se preocupe com ele sumindo de repente. Mas você pode querer começar a migrar para a alternativa `push` pelas novas funcionalidades.

Fazendo Stash do Seu Trabalho

Para demonstrar o stashing, você irá para o seu projeto e começará a trabalhar em alguns arquivos e, possivelmente, preparará (stage) uma das alterações. Se você rodar `git status`, poderá ver o seu estado sujo:

```
$ git status
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   lib/simplegit.rb
```

Agora você deseja mudar de branch, mas ainda não quer comitar o que tem trabalhado, então fará o stash das alterações. Para enviar um novo stash para sua pilha, execute `git stash` ou `git stash push`:

```
$ git stash
Saved working directory and index state \
  "WIP on master: 049d078 Create index file"
HEAD is now at 049d078 Create index file
(To restore them type "git stash apply")
```

Agora você pode ver que o seu diretório de trabalho está limpo:

```
$ git status
# On branch master
nothing to commit, working directory clean
```

Neste ponto, você pode mudar de branch e trabalhar em outro lugar; as suas alterações estão armazenadas na sua pilha. Para ver quais stashes você armazenou, você pode usar o comando `git stash list`:

```
$ git stash list
stash@{0}: WIP on master: 049d078 Create index file
stash@{1}: WIP on master: c264051 Revert "Add file_size"
stash@{2}: WIP on master: 21d80a5 Add number to log
```

Neste caso, dois stashes foram salvos anteriormente, então você tem acesso a três trabalhos em stash diferentes. Você pode reaplicar aquele que você acabou de fazer o stash usando o comando mostrado na saída de ajuda do comando stash original: `git stash apply`. Se você deseja aplicar um

dos stashes mais antigos, você pode especificá-lo nomeando-o, assim: `git stash apply stash@{2}`. Se você não especificar um stash, o Git assume que é o stash mais recente e tenta aplicá-lo:

```
$ git stash apply
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   index.html
    modified:   lib/simplegit.rb

no changes added to commit (use "git add" and/or "git commit -a")
```

Você pode ver que o Git modifica novamente os arquivos que você reverteu quando salvou o stash. Neste caso, você tinha um diretório de trabalho limpo quando tentou aplicar o stash, e você tentou aplicá-lo no mesmo branch em que o salvou. Ter um diretório de trabalho limpo e aplicá-lo no mesmo branch não são necessários para aplicar um stash com sucesso. Você pode salvar um stash em um branch, mudar para outro branch mais tarde e tentar reaplicar as alterações. Você também pode ter arquivos modificados e não comitados no seu diretório de trabalho quando aplicar um stash — o Git informará conflitos de merge se algo não for mais aplicável de forma limpa.

As alterações nos seus arquivos foram reaplicadas, mas o arquivo que você havia preparado (staged) antes não foi preparado novamente. Para fazer isso, você deve executar o comando `git stash apply` com a opção `--index` para dizer ao comando que tente reaplicar as alterações preparadas. Se você tivesse rodado isso em vez do outro comando, teria retornado à sua posição original:

```
$ git stash apply --index
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   lib/simplegit.rb
```

A opção `apply` tenta apenas aplicar o trabalho do stash — você continua a tê-lo na sua pilha. Para removê-lo, você pode executar o `git stash drop` com o nome do stash a remover:

```
$ git stash list
stash@{0}: WIP on master: 049d078 Create index file
stash@{1}: WIP on master: c264051 Revert "Add file_size"
```

```
stash@{2}: WIP on master: 21d80a5 Add number to log
$ git stash drop stash@{0}
Dropped stash@{0} (364e91f3f268f0900bc3ee613f9f733e82aaed43)
```

Você também pode executar `git stash pop` para aplicar o stash e então imediatamente removê-lo da sua pilha.

Fazendo Stash Criativo

Existem algumas variantes de stash que também podem ser úteis. A primeira opção que é bastante popular é a opção `--keep-index` do comando `git stash`. Isso diz ao Git para não apenas incluir todo o conteúdo preparado (staged) no stash sendo criado, mas simultaneamente deixá-lo no index.

```
$ git status -s
M  index.html
 M lib/simplegit.rb

$ git stash --keep-index
Saved working directory and index state WIP on master: 1b65b17 added the index file
HEAD is now at 1b65b17 added the index file

$ git status -s
M  index.html
```

Outra coisa comum que você pode querer fazer com o stash é fazer o stash dos arquivos não rastreados, bem como dos rastreados. Por padrão, `git stash` fará o stash apenas de arquivos *rastreados* modificados e preparados. Se você especificar `--include-untracked` ou `-u`, o Git incluirá os arquivos não rastreados no stash sendo criado. No entanto, incluir arquivos não rastreados no stash ainda não incluirá arquivos explicitamente *ignorados*; para incluir também arquivos ignorados, use `--all` (ou apenas `-a`).

```
$ git status -s
M  index.html
 M lib/simplegit.rb
?? new-file.txt

$ git stash -u
Saved working directory and index state WIP on master: 1b65b17 added the index file
HEAD is now at 1b65b17 added the index file

$ git status -s
$
```

Finalmente, se você especificar a flag `--patch`, o Git não fará o stash de tudo o que está modificado, mas, em vez disso, perguntará interativamente quais das alterações você gostaria de fazer o stash e quais gostaria de manter no seu diretório de trabalho.

```
$ git stash --patch
diff --git a/lib/simplegit.rb b/lib/simplegit.rb
index 66d332e..8bb5674 100644
--- a/lib/simplegit.rb
+++ b/lib/simplegit.rb
@@ -16,6 +16,10 @@ class SimpleGit
         return `#{git_cmd} 2>&1`.chomp
       end
     end
+
+    def show(treeish = 'master')
+      command("git show #{treeish}")
+    end

 end
 test
Stash this hunk [y,n,q,a,d,/,e,?]? y

Saved working directory and index state WIP on master: 1b65b17 added the index file
```

Criando um Branch a partir de um Stash

Se você fizer o stash de algum trabalho, deixá-lo lá por um tempo e continuar no branch do qual fez o stash, poderá ter problemas ao reaplicar o trabalho. Se a aplicação tentar modificar um arquivo que você modificou desde então, você terá um conflito de merge e terá que tentar resolvê-lo. Se você quiser uma maneira mais fácil de testar novamente as alterações no stash, pode rodar `git stash branch <nome do novo branch>`, que cria um novo branch para você com o nome do branch selecionado, faz checkout do commit em que você estava quando fez o stash do trabalho, reaplica seu trabalho lá e, em seguida, descarta o stash se for aplicado com sucesso:

```
$ git stash branch testchanges
M   index.html
M   lib/simplegit.rb
Switched to a new branch 'testchanges'
On branch testchanges
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    modified:   index.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   lib/simplegit.rb

Dropped refs/stash@{0} (29d385a81d163dfd45a452a2ce816487a6b8b014)
```

Este é um atalho útil para recuperar o trabalho em stash de forma fácil e trabalhar nele em um novo branch.

Limpando Seu Diretório de Trabalho

Por fim, você pode não querer fazer o stash de algum trabalho ou arquivos no seu diretório de trabalho, mas simplesmente se livrar deles; é para isso que serve o comando `git clean`.

Algumas razões comuns para limpar seu diretório de trabalho podem ser remover lixo (cruft) que foi gerado por merges ou ferramentas externas ou para remover artefatos de build a fim de executar um build limpo.

Você deve ter muito cuidado com este comando, já que ele foi projetado para remover arquivos do seu diretório de trabalho que não são rastreados. Se você mudar de ideia, frequentemente não há como recuperar o conteúdo desses arquivos. Uma opção mais segura é rodar `git stash --all` para remover tudo, mas salvar em um stash.

Assumindo que você realmente deseja remover arquivos inúteis (cruft) ou limpar seu diretório de trabalho, você pode fazer isso com `git clean`. Para remover todos os arquivos não rastreados no seu diretório de trabalho, você pode rodar `git clean -f -d`, que remove quaisquer arquivos e também quaisquer subdiretórios que se tornem vazios como resultado. O `-f` significa 'force' (forçar) ou “realmente faça isso”, e é obrigatório se a variável de configuração do Git `clean.requireForce` não for explicitamente definida como falsa.

Se você quiser ver o que ele faria, pode executar o comando com a opção `--dry-run` (ou `-n`), que significa “faça uma simulação (dry run) e me diga o que você *teria* removido”.

```
$ git clean -d -n
Would remove test.o
Would remove tmp/
```

Por padrão, o comando `git clean` removerá apenas os arquivos não rastreados que não são ignorados. Qualquer arquivo que corresponda a um padrão no seu `.gitignore` ou outros arquivos de ignore não será removido. Se você quiser remover esses arquivos também, como por exemplo remover todos os arquivos `.o` gerados a partir de um build para que possa fazer um build totalmente limpo, pode adicionar um `-x` ao comando `clean`.

```
$ git status -s
 M lib/simplegit.rb
?? build.TMP
?? tmp/

$ git clean -n -d
Would remove build.TMP
Would remove tmp/

$ git clean -n -d -x
Would remove build.TMP
```

```
Would remove test.o
Would remove tmp/
```

Se você não sabe o que o comando `git clean` fará, sempre o execute com um `-n` primeiro para verificar duas vezes antes de mudar o `-n` para um `-f` e fazê-lo de verdade. A outra maneira de ser cuidadoso com o processo é executá-lo com a flag `-i` ou "interactive".

Isso rodará o comando `clean` num modo interativo.

```
$ git clean -x -i
Would remove the following items:
  build.TMP  test.o
*** Commands ***
    1: clean                2: filter by pattern    3: select by numbers    4: ask
each             5: quit
    6: help
What now>
```

Dessa forma, você pode percorrer cada arquivo individualmente ou especificar padrões para exclusão interativamente.



Há uma situação peculiar em que você pode precisar ser mais enérgico ao pedir ao Git para limpar o seu diretório de trabalho. Se por acaso você estiver em um diretório de trabalho sob o qual você copiou ou clonou outros repositórios Git (talvez como submódulos), até mesmo o `git clean -fd` se recusará a excluir esses diretórios. Nesses casos, você precisa adicionar uma segunda opção `-f` para enfatizar.

Assinando Seu Trabalho

O Git é criptograficamente seguro, mas não é infalível. Se você está pegando o trabalho de outras pessoas na internet e quer verificar se os commits são realmente de uma fonte confiável, o Git tem algumas maneiras de assinar e verificar o trabalho usando GPG.

Introdução ao GPG

Em primeiro lugar, se você quiser assinar alguma coisa, você precisa configurar o GPG e ter a sua chave pessoal instalada.

```
$ gpg --list-keys
/Users/schacon/.gnupg/pubring.gpg
---------------------------------
pub   2048R/0A46826A 2014-06-04
uid                  Scott Chacon (Git signing key) <schacon@gmail.com>
sub   2048R/874529A9 2014-06-04
```

Se você não tem uma chave instalada, você pode gerar uma com `gpg --gen-key`.

```
$ gpg --gen-key
```

Depois que você tiver uma chave privada para assinar, você pode configurar o Git para usá-la para assinar coisas, definindo a configuração `user.signingkey`.

```
$ git config --global user.signingkey 0A46826A!
```

Agora o Git usará a sua chave por padrão para assinar tags e commits, se você quiser.

Assinando Tags

Se você tem uma chave privada GPG configurada, você pode agora usá-la para assinar novas tags. Tudo o que você tem a fazer é usar `-s` em vez de `-a`:

```
$ git tag -s v1.5 -m 'my signed 1.5 tag'

You need a passphrase to unlock the secret key for
user: "Ben Straub <ben@straub.cc>"
2048-bit RSA key, ID 800430EB, created 2014-05-04
```

Se você executar `git show` nessa tag, poderá ver sua assinatura GPG anexada a ela:

```
$ git show v1.5
tag v1.5
Tagger: Ben Straub <ben@straub.cc>
Date:   Sat May 3 20:29:41 2014 -0700

my signed 1.5 tag
-----BEGIN PGP SIGNATURE-----
Version: GnuPG v1

iQEcBAABAgAGBQJTZbQlAAoJEF0+sviABDDrZbQH/09PfE51KPVPlanr6q1v4/Ut
LQxfojUWiLQdg2ESJItkcuweYg+kc3HCyFejeDIBw9dpXt00rY26p05qrpnG+85b
hM1/PswpPLuBSr+oCIDj5GMC2r2iEKsfv2fJbNW8iWAXVLoWZRF8B0MfqX/YTMbm
ecorc4iXzQu7tupRihslbNkfvfciMnSDeSvzCpWAHl7h8Wj6hhqePmLm9lAYqnKp
8S5B/1SSQuEAjRZgI4IexpZoeKGVDptPHxLLS38fozsyi0QyDyzEgJxcJQVMXxVi
RUysgqjcpT8+iQM1PblGfHR4XAhuOqN5Fx06PSaFZhqvWFezJ28/CLyX5q+oIVk=
=EFTF
-----END PGP SIGNATURE-----

commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700
```

```
    Change version number
```

Verificando Tags

Para verificar uma tag assinada, você usa `git tag -v <nome-da-tag>`. Este comando usa o GPG para verificar a assinatura. Você precisa da chave pública de quem assinou em seu chaveiro (keyring) para que isso funcione corretamente:

```
$ git tag -v v1.4.2.1
object 883653babd8ee7ea23e6a5c392bb739348b1eb61
type commit
tag v1.4.2.1
tagger Junio C Hamano <junkio@cox.net> 1158138501 -0700

GIT 1.4.2.1

Minor fixes since 1.4.2, including git-mv and git-http with alternates.
gpg: Signature made Wed Sep 13 02:08:25 2006 PDT using DSA key ID F3119B9A
gpg: Good signature from "Junio C Hamano <junkio@cox.net>"
gpg:                 aka "[jpeg image of size 1513]"
Primary key fingerprint: 3565 2A26 2040 E066 C9A7  4A7D C0C6 D9A4 F311 9B9A
```

Se você não tiver a chave pública de quem assinou, obterá algo assim em vez disso:

```
gpg: Signature made Wed Sep 13 02:08:25 2006 PDT using DSA key ID F3119B9A
gpg: Can't check signature: public key not found
error: could not verify the tag 'v1.4.2.1'
```

Assinando Commits

Nas versões mais recentes do Git (v1.7.9 e superior), agora você também pode assinar commits individuais. Se você estiver interessado em assinar commits diretamente em vez de apenas as tags, tudo o que você precisa fazer é adicionar um `-S` ao seu comando `git commit`.

```
$ git commit -a -S -m 'Signed commit'

You need a passphrase to unlock the secret key for
user: "Scott Chacon (Git signing key) <schacon@gmail.com>"
2048-bit RSA key, ID 0A46826A, created 2014-06-04

[master 5c3386c] Signed commit
 4 files changed, 4 insertions(+), 24 deletions(-)
 rewrite Rakefile (100%)
 create mode 100644 lib/git.rb
```

Para ver e verificar essas assinaturas, há também a opção `--show-signature` para o `git log`.

```
$ git log --show-signature -1
commit 5c3386cf54bba0a33a32da706aa52bc0155503c2
gpg: Signature made Wed Jun  4 19:49:17 2014 PDT using RSA key ID 0A46826A
gpg: Good signature from "Scott Chacon (Git signing key) <schacon@gmail.com>"
Author: Scott Chacon <schacon@gmail.com>
Date:   Wed Jun 4 19:49:17 2014 -0700

    Signed commit
```

Além disso, você pode configurar o `git log` para verificar quaisquer assinaturas que ele encontre e listá-las em sua saída com o formato `%G?`.

```
$ git log --pretty="format:%h %G? %aN  %s"

5c3386c G Scott Chacon  Signed commit
ca82a6d N Scott Chacon  Change the version number
085bb3b N Scott Chacon  Remove unnecessary test code
a11bef0 N Scott Chacon  Initial commit
```

Aqui podemos ver que apenas o commit mais recente é assinado e válido, e os commits anteriores não são.

No Git 1.8.3 e versões posteriores, você pode dizer ao `git merge` e `git pull` para inspecionarem e rejeitarem a mesclagem de um commit que não carrega uma assinatura GPG confiável com o comando `--verify-signatures`.

Se você usar essa opção ao mesclar (fazer merge de) um branch e ele contiver commits que não estão assinados e válidos, a mesclagem não funcionará.

```
$ git merge --verify-signatures non-verify
fatal: Commit ab06180 does not have a GPG signature.
```

Se a mesclagem contiver apenas commits assinados válidos, o comando de merge mostrará a você todas as assinaturas que verificou e então prosseguirá com a mesclagem.

```
$ git merge --verify-signatures signed-branch
Commit 13ad65e has a good GPG signature by Scott Chacon (Git signing key)
<schacon@gmail.com>
Updating 5c3386c..13ad65e
Fast-forward
 README | 2 ++
 1 file changed, 2 insertions(+)
```

Você também pode usar a opção `-S` com o comando `git merge` para assinar o próprio commit de merge resultante. O exemplo a seguir verifica se todo commit no branch a ser mesclado está assinado e, além disso, assina o commit de merge resultante.

```
$ git merge --verify-signatures -S  signed-branch
Commit 13ad65e has a good GPG signature by Scott Chacon (Git signing key)
<schacon@gmail.com>

You need a passphrase to unlock the secret key for
user: "Scott Chacon (Git signing key) <schacon@gmail.com>"
2048-bit RSA key, ID 0A46826A, created 2014-06-04

Merge made by the 'recursive' strategy.
 README | 2 ++
 1 file changed, 2 insertions(+)
```

Todos Devem Assinar

Assinar tags e commits é ótimo, mas se você decidir usar isso em seu fluxo de trabalho normal, terá que se certificar de que todos em sua equipe entendam como fazê-lo. Isso pode ser alcançado pedindo a todos que trabalham com o repositório para rodar `git config --local commit.gpgsign true` para que todos os seus commits no repositório sejam automaticamente assinados por padrão. Se você não o fizer, acabará passando muito tempo ajudando as pessoas a descobrir como reescrever seus commits com versões assinadas. Certifique-se de que você entende o GPG e os benefícios de assinar as coisas antes de adotar isso como parte de seu fluxo de trabalho padrão.

Pesquisando (Searching)

Com uma base de código de quase qualquer tamanho, você frequentemente precisará encontrar onde uma função é chamada ou definida, ou exibir o histórico de um método. O Git fornece algumas ferramentas úteis para procurar através do código e dos commits armazenados no seu banco de dados de forma rápida e fácil. Analisaremos algumas delas.

Git Grep

O Git vem com um comando chamado `grep` que permite que você pesquise facilmente através de qualquer árvore commitada, o diretório de trabalho, ou mesmo o index em busca de uma string ou expressão regular. Para os exemplos a seguir, pesquisaremos no próprio código fonte do Git.

Por padrão, o `git grep` examinará os arquivos no seu diretório de trabalho. Como uma primeira variação, você pode usar as opções `-n` ou `--line-number` para imprimir os números das linhas onde o Git encontrou correspondências (matches):

```
$ git grep -n gmtime_r
compat/gmtime.c:3:#undef gmtime_r
compat/gmtime.c:8:      return git_gmtime_r(timep, &result);
compat/gmtime.c:11:struct tm *git_gmtime_r(const time_t *timep, struct tm *result)
compat/gmtime.c:16:     ret = gmtime_r(timep, result);
compat/mingw.c:826:struct tm *gmtime_r(const time_t *timep, struct tm *result)
compat/mingw.h:206:struct tm *gmtime_r(const time_t *timep, struct tm *result);
date.c:482:             if (gmtime_r(&now, &now_tm))
```

```
date.c:545:             if (gmtime_r(&time, tm)) {
date.c:758:             /* gmtime_r() in match_digit() may have clobbered it */
git-compat-util.h:1138:struct tm *git_gmtime_r(const time_t *, struct tm *);
git-compat-util.h:1140:#define gmtime_r git_gmtime_r
```

Além da pesquisa básica mostrada acima, `git grep` suporta uma infinidade de outras opções interessantes.

Por exemplo, em vez de imprimir todas as correspondências, você pode pedir ao `git grep` para resumir a saída, mostrando apenas quais arquivos continham a string de pesquisa e quantas correspondências havia em cada arquivo com a opção `-c` ou `--count`:

```
$ git grep --count gmtime_r
compat/gmtime.c:4
compat/mingw.c:1
compat/mingw.h:1
date.c:3
git-compat-util.h:2
```

Se você estiver interessado no *contexto* de uma string de pesquisa, poderá exibir o método ou função envolvente para cada string correspondente com as opções `-p` ou `--show-function`:

```
$ git grep -p gmtime_r *.c
date.c=static int match_multi_number(timestamp_t num, char c, const char *date,
date.c:         if (gmtime_r(&now, &now_tm))
date.c=static int match_digit(const char *date, struct tm *tm, int *offset, int
*tm_gmt)
date.c:         if (gmtime_r(&time, tm)) {
date.c=int parse_date_basic(const char *date, timestamp_t *timestamp, int *offset)
date.c:         /* gmtime_r() in match_digit() may have clobbered it */
```

Como você pode ver, a rotina `gmtime_r` é chamada tanto pela função `match_multi_number` quanto por `match_digit` no arquivo `date.c` (a terceira correspondência exibida representa apenas a string aparecendo em um comentário).

Você também pode pesquisar por combinações complexas de strings com a flag `--and`, que garante que múltiplas correspondências devem ocorrer na mesma linha de texto. Por exemplo, vamos procurar quaisquer linhas que definam uma constante cujo nome contenha *qualquer uma* das substrings “LINK” ou “BUF_MAX”, especificamente em uma versão mais antiga da base de código do Git representada pela tag `v1.8.0` (incluiremos as opções `--break` e `--heading` que ajudam a dividir a saída em um formato mais legível):

```
$ git grep --break --heading \
    -n -e '#define' --and \( -e LINK -e BUF_MAX \) v1.8.0
v1.8.0:builtin/index-pack.c
62:#define FLAG_LINK (1u<<20)
```

```
v1.8.0:cache.h
73:#define S_IFGITLINK  0160000
74:#define S_ISGITLINK(m)       (((m) & S_IFMT) == S_IFGITLINK)

v1.8.0:environment.c
54:#define OBJECT_CREATION_MODE OBJECT_CREATION_USES_HARDLINKS

v1.8.0:strbuf.c
326:#define STRBUF_MAXLINK (2*PATH_MAX)

v1.8.0:symlinks.c
53:#define FL_SYMLINK  (1 << 2)

v1.8.0:zlib.c
30:/* #define ZLIB_BUF_MAX ((uInt)-1) */
31:#define ZLIB_BUF_MAX ((uInt) 1024 * 1024 * 1024) /* 1GB */
```

O comando `git grep` tem algumas vantagens em relação a comandos de pesquisa normais como `grep` e `ack`. A primeira é que ele é realmente rápido; a segunda é que você pode pesquisar em qualquer árvore no Git, e não apenas no diretório de trabalho. Como vimos no exemplo acima, procuramos por termos em uma versão mais antiga do código fonte do Git, e não na versão em que o checkout foi feito atualmente.

Pesquisa no Git Log

Talvez você não esteja procurando *onde* um termo existe, mas *quando* ele existiu ou foi introduzido. O comando `git log` possui uma série de ferramentas poderosas para encontrar commits específicos pelo conteúdo de suas mensagens ou mesmo pelo conteúdo do diff que eles introduzem.

Se, por exemplo, quisermos descobrir quando a constante `ZLIB_BUF_MAX` foi originalmente introduzida, podemos usar a opção `-S` (coloquialmente chamada de opção Git “pickaxe”) para dizer ao Git que nos mostre apenas os commits que alteraram o número de ocorrências dessa string.

```
$ git log -S ZLIB_BUF_MAX --oneline
e01503b zlib: allow feeding more than 4GB in one go
ef49a7a zlib: zlib can only process 4GB at a time
```

Se olharmos para o diff desses commits, podemos ver que em `ef49a7a` a constante foi introduzida e em `e01503b` ela foi modificada.

Se precisar ser mais específico, você pode fornecer uma expressão regular para pesquisar com a opção `-G`.

Pesquisa no Histórico da Linha (Line Log Search)

Outra pesquisa de log bastante avançada e absurdamente útil é a pesquisa de histórico de linha. Basta executar o `git log` com a opção `-L`, e ele mostrará o histórico de uma função ou linha de código na sua base de código.

Por exemplo, se quiséssemos ver todas as alterações feitas na função `git_deflate_bound` no arquivo `zlib.c`, poderíamos rodar `git log -L :git_deflate_bound:zlib.c`. Isso tentará descobrir quais são os limites dessa função e, em seguida, examinará o histórico e nos mostrará todas as alterações feitas na função como uma série de patches até o momento em que a função foi criada pela primeira vez.

```
$ git log -L :git_deflate_bound:zlib.c
commit ef49a7a0126d64359c974b4b3b71d7ad42ee3bca
Author: Junio C Hamano <gitster@pobox.com>
Date:   Fri Jun 10 11:52:15 2011 -0700

    zlib: zlib can only process 4GB at a time

diff --git a/zlib.c b/zlib.c
--- a/zlib.c
+++ b/zlib.c
@@ -85,5 +130,5 @@
-unsigned long git_deflate_bound(z_streamp strm, unsigned long size)
+unsigned long git_deflate_bound(git_zstream *strm, unsigned long size)
 {
-       return deflateBound(strm, size);
+       return deflateBound(&strm->z, size);
 }


commit 225a6f1068f71723a910e8565db4e252b3ca21fa
Author: Junio C Hamano <gitster@pobox.com>
Date:   Fri Jun 10 11:18:17 2011 -0700

    zlib: wrap deflateBound() too

diff --git a/zlib.c b/zlib.c
--- a/zlib.c
+++ b/zlib.c
@@ -81,0 +85,5 @@
+unsigned long git_deflate_bound(z_streamp strm, unsigned long size)
+{
+       return deflateBound(strm, size);
+}
+
```

Se o Git não conseguir descobrir como combinar uma função ou método em sua linguagem de programação, você também pode fornecer a ele uma expressão regular (ou *regex*). Por exemplo, isso teria feito o mesmo que o exemplo acima: `git log -L '/unsigned long git_deflate_bound/',/^}/:zlib.c`. Você também poderia fornecer um intervalo de linhas ou um único número de linha e obteria o mesmo tipo de saída.

Reescrevendo o Histórico

Muitas vezes, ao trabalhar com o Git, você pode querer revisar seu histórico de commits local. Uma das grandes coisas sobre o Git é que ele permite que você tome decisões no último momento possível. Você pode decidir quais arquivos vão em quais commits logo antes de comitar usando a área de stage, pode decidir que não queria estar trabalhando em algo ainda com o `git stash` e pode reescrever commits que já aconteceram para que pareçam ter acontecido de uma maneira diferente. Isso pode envolver alterar a ordem dos commits, alterar mensagens ou modificar arquivos em um commit, juntar (squash) ou dividir commits, ou remover commits inteiramente — tudo antes de você compartilhar o seu trabalho com outras pessoas.

Nesta seção, você verá como realizar essas tarefas para que possa fazer com que seu histórico de commits fique do jeito que você quer antes de compartilhá-lo com outras pessoas.



Não faça push do seu trabalho até que esteja feliz com ele

Uma das regras fundamentais do Git é que, como grande parte do trabalho é local no seu clone, você tem uma grande liberdade para reescrever o seu histórico *localmente*. No entanto, depois de fazer push do seu trabalho, a história é completamente diferente, e você deve considerar o trabalho empurrado (pushed) como final, a menos que tenha um bom motivo para alterá-lo. Resumindo, você deve evitar fazer push do seu trabalho até que esteja feliz com ele e pronto para compartilhá-lo com o resto do mundo.

Alterando o Último Commit

Alterar o seu commit mais recente é provavelmente a reescrita de histórico mais comum que você fará. Muitas vezes, você desejará fazer duas coisas básicas no seu último commit: simplesmente alterar a mensagem do commit ou alterar o conteúdo real do commit adicionando, removendo e modificando arquivos.

Se você simplesmente quiser modificar a mensagem do seu último commit, isso é fácil:

```
$ git commit --amend
```

O comando acima carrega a mensagem do commit anterior em uma sessão de editor, onde você pode fazer alterações na mensagem, salvar essas alterações e sair. Quando você salva e fecha o editor, o editor grava um novo commit contendo a mensagem de commit atualizada e o torna o seu novo último commit.

Se, por outro lado, você quiser alterar o *conteúdo* real do seu último commit, o processo funciona basicamente da mesma maneira — primeiro faça as alterações que você acha que esqueceu, prepare (stage) essas alterações, e o subsequente `git commit --amend` *substitui* o último commit pelo seu novo e aprimorado commit.

Você precisa ter cuidado com essa técnica porque o amend (alteração) muda o SHA-1 do commit. É como um rebase muito pequeno — não faça amend no seu último commit se já tiver feito push dele.

Um commit alterado pode (ou não) precisar de uma mensagem de commit alterada

Quando você altera um commit, tem a oportunidade de mudar tanto a mensagem do commit quanto o conteúdo dele. Se você alterar o conteúdo do commit de forma substancial, quase certamente deverá atualizar a mensagem do commit para refletir esse conteúdo alterado.



Por outro lado, se as suas alterações forem suficientemente triviais (corrigir um erro de digitação bobo ou adicionar um arquivo que você esqueceu de preparar) de modo que a mensagem de commit anterior esteja boa, você pode simplesmente fazer as alterações, prepará-las (stage) e evitar totalmente a sessão desnecessária do editor com:

```
$ git commit --amend --no-edit
```

Alterando Várias Mensagens de Commit

Para modificar um commit que está mais atrás no seu histórico, você deve usar ferramentas mais complexas. O Git não tem uma ferramenta de modificar o histórico, mas você pode usar a ferramenta de rebase para fazer o rebase de uma série de commits no HEAD em que eles se basearam originalmente, em vez de movê-los para outro. Com a ferramenta de rebase interativo, você pode parar após cada commit que deseja modificar e alterar a mensagem, adicionar arquivos ou fazer o que quiser. Você pode rodar o rebase interativamente adicionando a opção `-i` ao `git rebase`. Você deve indicar o quão longe no passado você quer reescrever os commits, dizendo ao comando em qual commit fazer o rebase.

Por exemplo, se quiser alterar as três últimas mensagens de commit, ou qualquer uma das mensagens de commit nesse grupo, você fornece como argumento para o `git rebase -i` o pai do último commit que deseja editar, que é `HEAD~2^` ou `HEAD~3`. Pode ser mais fácil lembrar do `~3` porque você está tentando editar os últimos três commits, mas lembre-se de que, na verdade, você está designando o quarto commit atrás, o pai do último commit que você deseja editar:

```
$ git rebase -i HEAD~3
```

Lembre-se novamente que este é um comando de rebasing — cada commit no intervalo `HEAD~3..HEAD` com uma mensagem alterada *e todos os seus descendentes* serão reescritos. Não inclua nenhum commit que você já tenha feito push para um servidor central — fazer isso confundirá outros desenvolvedores ao fornecer uma versão alternativa da mesma alteração.

Executar este comando dá a você uma lista de commits no seu editor de texto que se parece com isto:

```
pick f7f3f6d Change my name a bit
pick 310154e Update README formatting and add blame
pick a5f4a0d Add cat-file

# Rebase 710f0f8..a5f4a0d onto 710f0f8
```

```
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup <commit> = like "squash", but discard this commit's log message
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified). Use -c <commit> to reword the commit message.
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
# Note that empty commits are commented out
```

É importante notar que esses commits são listados na ordem oposta daquela que você normalmente vê usando o comando `log`. Se você rodar um `log`, verá algo como isto:

```
$ git log --pretty=format:"%h %s" HEAD~3..HEAD
a5f4a0d Add cat-file
310154e Update README formatting and add blame
f7f3f6d Change my name a bit
```

Observe a ordem inversa. O rebase interativo fornece um script que ele vai rodar. Ele começará no commit que você especificou na linha de comando (`HEAD~3`) e reproduzirá (replay) as alterações introduzidas em cada um desses commits, de cima para baixo. Ele lista o mais antigo no topo, em vez do mais novo, porque esse é o primeiro que ele reproduzirá.

Você precisa editar o script para que ele pare no commit que deseja editar. Para fazer isso, altere a palavra “pick” para a palavra “edit” para cada um dos commits que você deseja que o script pare depois. Por exemplo, para modificar apenas a terceira mensagem de commit, você altera o arquivo para que fique assim:

```
edit f7f3f6d Change my name a bit
pick 310154e Update README formatting and add blame
pick a5f4a0d Add cat-file
```

Quando você salvar e sair do editor, o Git voltará para o último commit nessa lista e o deixará na

linha de comando com a seguinte mensagem:

```
$ git rebase -i HEAD~3
Stopped at f7f3f6d... Change my name a bit
You can amend the commit now, with

       git commit --amend

Once you're satisfied with your changes, run

       git rebase --continue
```

Estas instruções dizem exatamente o que fazer. Digite:

```
$ git commit --amend
```

Altere a mensagem do commit e saia do editor. Então, rode:

```
$ git rebase --continue
```

Este comando aplicará os outros dois commits automaticamente e então você terminou. Se você alterar `pick` para `edit` em mais linhas, pode repetir essas etapas para cada commit que você alterou para `edit`. Cada vez, o Git irá parar, deixará você alterar o commit e continuará quando você terminar.

Reordenando Commits

Você também pode usar rebases interativos para reordenar ou remover commits completamente. Se você quiser remover o commit "Add cat-file" e alterar a ordem em que os outros dois commits são introduzidos, você pode alterar o script de rebase disto:

```
pick f7f3f6d Change my name a bit
pick 310154e Update README formatting and add blame
pick a5f4a0d Add cat-file
```

para isto:

```
pick 310154e Update README formatting and add blame
pick f7f3f6d Change my name a bit
```

Quando você salva e sai do editor, o Git retrocede a sua branch para o pai desses commits, aplica `310154e` e depois `f7f3f6d`, e então para. Você efetivamente alterou a ordem desses commits e removeu o commit "Add cat-file" completamente.

Juntando Commits (Squashing)

Também é possível pegar uma série de commits e juntá-los em um único commit com a ferramenta de rebase interativo. O script coloca instruções úteis na mensagem de rebase:

```
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup <commit> = like "squash", but discard this commit's log message
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified). Use -c <commit> to reword the commit message.
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
# Note that empty commits are commented out
```

Se, em vez de “pick” ou “edit”, você especificar “squash”, o Git aplica tanto essa alteração quanto a alteração imediatamente anterior a ela e faz com que você mescle as mensagens de commit. Então, se você quiser fazer um único commit a partir desses três commits, faça o script parecer com isto:

```
pick f7f3f6d Change my name a bit
squash 310154e Update README formatting and add blame
squash a5f4a0d Add cat-file
```

Quando você salva e sai do editor, o Git aplica todas as três alterações e então coloca você de volta no editor para mesclar as três mensagens de commit:

```
# This is a combination of 3 commits.
# The first commit's message is:
Change my name a bit

# This is the 2nd commit message:

Update README formatting and add blame
```

```
# This is the 3rd commit message:

Add cat-file
```

Quando você salva isso, tem um único commit que introduz as alterações de todos os três commits anteriores.

Dividindo um Commit

Dividir um commit desfaz um commit e então prepara (stages) e comita parcialmente quantas vezes forem os commits com os quais você deseja terminar. Por exemplo, suponha que você queira dividir o commit do meio dos seus três commits. Em vez de "Update README formatting and add blame", você quer dividi-lo em dois commits: "Update README formatting" para o primeiro, e "Add blame" para o segundo. Você pode fazer isso no script do `rebase -i` alterando a instrução no commit que deseja dividir para "edit":

```
pick f7f3f6d Change my name a bit
edit 310154e Update README formatting and add blame
pick a5f4a0d Add cat-file
```

Então, quando o script deixar você na linha de comando, você reseta esse commit, pega as alterações que foram resetadas e cria vários commits a partir delas. Quando você salva e sai do editor, o Git volta (rewinds) para o pai do primeiro commit na sua lista, aplica o primeiro commit (`f7f3f6d`), aplica o segundo (`310154e`) e deixa você no console. Lá, você pode fazer um reset misto (mixed) desse commit com `git reset HEAD^`, que efetivamente desfaz esse commit e deixa os arquivos modificados sem stage (unstaged). Agora você pode preparar e comitar arquivos até ter vários commits, e rodar `git rebase --continue` quando terminar:

```
$ git reset HEAD^
$ git add README
$ git commit -m 'Update README formatting'
$ git add lib/simplegit.rb
$ git commit -m 'Add blame'
$ git rebase --continue
```

O Git aplica o último commit (`a5f4a0d`) no script, e o seu histórico fica assim:

```
$ git log -4 --pretty=format:"%h %s"
1c002dd Add cat-file
9b29157 Add blame
35cfb2b Update README formatting
f7f3f6d Change my name a bit
```

Isso altera os SHA-1s dos três commits mais recentes na sua lista, então certifique-se de que nenhum commit alterado apareça nessa lista que você já tenha feito push para um repositório

compartilhado. Observe que o último commit (`f7f3f6d`) na lista permanece inalterado. Apesar deste commit ser mostrado no script, por ter sido marcado como "pick" e ter sido aplicado antes de quaisquer alterações de rebase, o Git deixa o commit inalterado.

Excluindo um commit

Se você quiser se livrar de um commit, pode excluí-lo usando o script `rebase -i`. Na lista de commits, coloque a palavra "drop" (descartar) antes do commit que deseja excluir (ou simplesmente exclua essa linha do script de rebase):

```
pick 461cb2a This commit is OK
drop 5aecc10 This commit is broken
```

Devido à forma como o Git constrói objetos de commit, excluir ou alterar um commit fará com que todos os commits subsequentes sejam reescritos. Quanto mais para trás você for no histórico do seu repositório, mais commits precisarão ser recriados. Isso pode causar muitos conflitos de merge se você tiver muitos commits posteriores na sequência que dependam daquele que acabou de excluir.

Se você chegar no meio de um rebase como este e decidir que não é uma boa ideia, pode parar a qualquer momento. Digite `git rebase --abort`, e seu repositório retornará ao estado em que estava antes de você começar o rebase.

Se você terminar um rebase e decidir que não é o que queria, pode usar `git reflog` para recuperar uma versão anterior do seu branch. Veja Recuperação de Dados para obter mais informações sobre o comando `reflog`.



Drew DeVault criou um guia prático (hands-on) com exercícios para aprender a usar o `git rebase`. Você pode encontrá-lo em: https://git-rebase.io/

A Opção Nuclear: filter-branch

Existe outra opção para reescrever o histórico que você pode usar se precisar reescrever um grande número de commits de alguma forma que possa ser roteirizada (scriptable) — por exemplo, alterar seu endereço de e-mail globalmente ou remover um arquivo de todos os commits. O comando é `filter-branch`, e ele pode reescrever enormes partes do seu histórico, então você provavelmente não deveria usá-lo a menos que seu projeto ainda não seja público e outras pessoas não tenham baseado seu trabalho nos commits que você está prestes a reescrever. No entanto, pode ser muito útil. Você aprenderá alguns de seus usos comuns para ter uma ideia de algumas das coisas que ele é capaz de fazer.



O `git filter-branch` tem muitas armadilhas e não é mais a maneira recomendada de reescrever o histórico. Em vez disso, considere usar o `git-filter-repo`, que é um script em Python que faz um trabalho melhor para a maioria das aplicações nas quais você normalmente usaria o `filter-branch`. Sua documentação e código-fonte podem ser encontrados em https://github.com/newren/git-filter-repo.

Removendo um Arquivo de Todos os Commits

Isso ocorre com bastante frequência. Alguém comita acidentalmente um arquivo binário enorme com um `git add .` sem pensar, e você deseja removê-lo de todos os lugares. Talvez você tenha comitado acidentalmente um arquivo que continha uma senha e queira tornar o seu projeto open source. O `filter-branch` é a ferramenta que você provavelmente quer usar para limpar todo o seu histórico. Para remover um arquivo chamado `passwords.txt` de todo o seu histórico, você pode usar a opção `--tree-filter` do comando `filter-branch`:

```
$ git filter-branch --tree-filter 'rm -f passwords.txt' HEAD
Rewrite 6b9b3cf04e7c5686a9cb838c3f36a8cb6a0fc2bd (21/21)
Ref 'refs/heads/master' was rewritten
```

A opção `--tree-filter` executa o comando especificado após cada checkout do projeto e depois comita os resultados novamente. Neste caso, você remove um arquivo chamado `passwords.txt` de cada snapshot, independentemente de ele existir ou não. Se você deseja remover todos os arquivos de backup do editor comitados acidentalmente, pode executar algo como `git filter-branch --tree-filter 'rm -f *~' HEAD`.

Você será capaz de assistir o Git reescrevendo árvores e commits e depois movendo o ponteiro do branch no final. Geralmente, é uma boa ideia fazer isso num branch de testes e depois fazer um hard-reset no seu branch `master` depois de determinar que o resultado é o que você realmente quer. Para rodar o `filter-branch` em todas as suas branches, você pode passar a opção `--all` ao comando.

Tornando um Subdiretório na Nova Raiz (Root)

Suponha que você fez uma importação de outro sistema de controle de versão e tem subdiretórios que não fazem sentido (`trunk`, `tags`, e assim por diante). Se você quiser tornar o subdiretório `trunk` a nova raiz do projeto para todos os commits, o `filter-branch` pode ajudar com isso, também:

```
$ git filter-branch --subdirectory-filter trunk HEAD
Rewrite 856f0bf61e41a27326cdae8f09fe708d679f596f (12/12)
Ref 'refs/heads/master' was rewritten
```

Agora, sua nova raiz do projeto é o que estava no subdiretório `trunk` a cada vez. O Git também removerá automaticamente commits que não afetaram o subdiretório.

Alterando Endereços de E-mail Globalmente

Outro caso comum é você ter esquecido de rodar `git config` para configurar seu nome e endereço de e-mail antes de começar a trabalhar, ou talvez você queira abrir o código de um projeto no trabalho (open-source) e alterar todos os seus endereços de e-mail de trabalho para o seu endereço pessoal. De qualquer forma, você também pode alterar os endereços de e-mail em vários commits em lote (batch) com o `filter-branch`. Você precisa tomar cuidado para alterar apenas os endereços de e-mail que são seus, então você usa `--commit-filter`:

```
$ git filter-branch --commit-filter '
```

```
        if [ "$GIT_AUTHOR_EMAIL" = "schacon@localhost" ];
        then
                GIT_AUTHOR_NAME="Scott Chacon";
                GIT_AUTHOR_EMAIL="schacon@example.com";
                git commit-tree "$@";
        else
                git commit-tree "$@";
        fi' HEAD
```

Isso passará e reescreverá cada commit para ter o seu novo endereço. Como os commits contêm os valores SHA-1 de seus pais, esse comando altera cada SHA-1 de commit no seu histórico, e não apenas aqueles que têm o endereço de e-mail correspondente.

Reset Desmistificado

Antes de passar para ferramentas mais especializadas, vamos falar sobre os comandos `reset` e `checkout` do Git. Estes comandos são duas das partes mais confusas do Git quando você os encontra pela primeira vez. Eles fazem tantas coisas que parece impossível realmente entendê-los e empregá-los adequadamente. Para isso, recomendamos uma metáfora simples.

As Três Árvores

Uma maneira mais fácil de pensar sobre `reset` e `checkout` é através do modelo mental de que o Git é um gerenciador de conteúdo de três árvores diferentes. Por “árvore” aqui, queremos dizer realmente “coleção de arquivos”, não especificamente a estrutura de dados. Há alguns casos em que o index não age exatamente como uma árvore, mas para nossos propósitos é mais fácil pensar sobre ele dessa forma por enquanto.

O Git, como sistema, gerencia e manipula três árvores em sua operação normal:

Árvore (Tree)	Papel
HEAD	Snapshot do último commit, próximo pai
Index	Snapshot proposto do próximo commit
Working Directory (Diretório de Trabalho)	Sandbox (Área de testes)

O HEAD

O HEAD é o ponteiro para a referência da branch atual, que por sua vez é um ponteiro para o último commit feito naquela branch. Isso significa que o HEAD será o pai do próximo commit que for criado. Geralmente é mais simples pensar no HEAD como o snapshot do **seu último commit naquela branch**.

Na verdade, é bem fácil ver como é esse snapshot. Aqui está um exemplo de como obter a listagem real do diretório e os checksums SHA-1 para cada arquivo no snapshot do HEAD:

```
$ git cat-file -p HEAD
```

```
tree cfda3bf379e4f8dba8717dee55aab78aef7f4daf
author Scott Chacon  1301511835 -0700
committer Scott Chacon  1301511835 -0700

initial commit

$ git ls-tree -r HEAD
100644 blob a906cb2a4a904a152...   README
100644 blob 8f94139338f9404f2...   Rakefile
040000 tree 99f1a6d12cb4b6f19...   lib
```

Os comandos Git `cat-file` e `ls-tree` são comandos "plumbing" (de encanamento/baixo nível) que são usados para coisas de nível mais baixo e não são realmente usados no trabalho diário, mas eles nos ajudam a ver o que está acontecendo aqui.

O Index

O *index* é o seu **próximo commit proposto**. Também nos referimos a esse conceito como "Staging Area" (Área de Stage) do Git, pois é para lá que o Git olha quando você roda `git commit`.

O Git preenche esse index com uma lista de todo o conteúdo dos arquivos que sofreram checkout pela última vez no seu diretório de trabalho e como eles se pareciam quando originalmente sofreram checkout. Você então substitui alguns desses arquivos por novas versões deles, e o `git commit` converte isso na árvore para um novo commit.

```
$ git ls-files -s
100644 a906cb2a4a904a152e80877d4088654daad0c859 0   README
100644 8f94139338f9404f26296befa88755fc2598c289 0   Rakefile
100644 47c6340d6459e05787f644c2447d2595f5d3a54b 0   lib/simplegit.rb
```

Novamente, aqui estamos usando `git ls-files`, que é mais um comando de bastidores que mostra como o seu index se parece atualmente.

O index não é tecnicamente uma estrutura de árvore — ele é implementado como um manifesto nivelado (flattened) — mas para nossos propósitos, é próximo o suficiente.

O Diretório de Trabalho (Working Directory)

Finalmente, você tem o seu *diretório de trabalho* (também comumente chamado de "working tree" ou árvore de trabalho). As outras duas árvores armazenam o seu conteúdo de forma eficiente, porém inconveniente, dentro da pasta `.git`. O diretório de trabalho descompacta-os em arquivos reais, o que torna muito mais fácil para você editá-los. Pense no diretório de trabalho como uma **sandbox** (área de testes), onde você pode experimentar as alterações antes de comitá-las na sua staging area (index) e então no histórico.

```
$ tree
.
├─── README
```

```
├──── Rakefile
└──── lib
    └──── simplegit.rb

1 directory, 3 files
```

O Fluxo de Trabalho (Workflow)

O fluxo de trabalho típico do Git é gravar snapshots do seu projeto em estados sucessivamente melhores, manipulando essas três árvores.

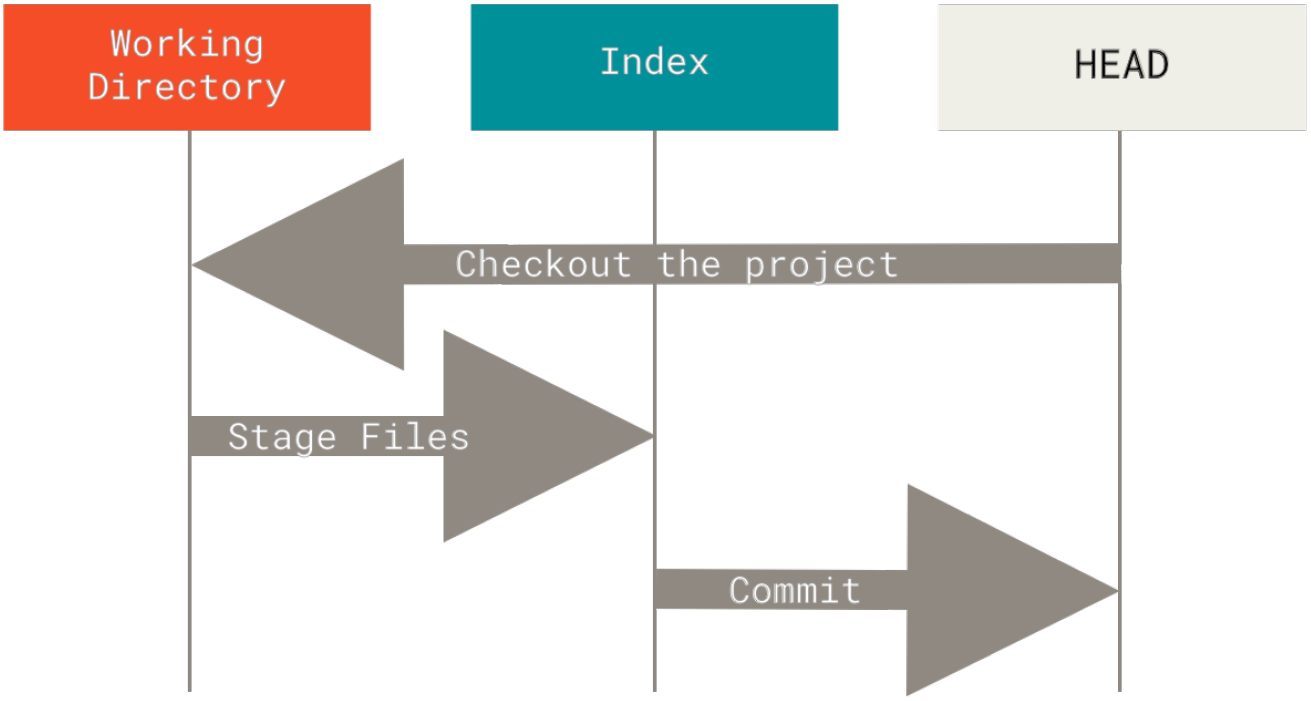


Figure 137. Fluxo de trabalho típico do Git

Vamos visualizar esse processo: digamos que você entre num diretório novo com um único arquivo nele. Chamaremos esta de **v1** do arquivo, e a indicaremos em azul. Agora nós rodamos `git init`, o que criará um repositório Git com uma referência HEAD que aponta para a branch `master` não nascida (unborn).

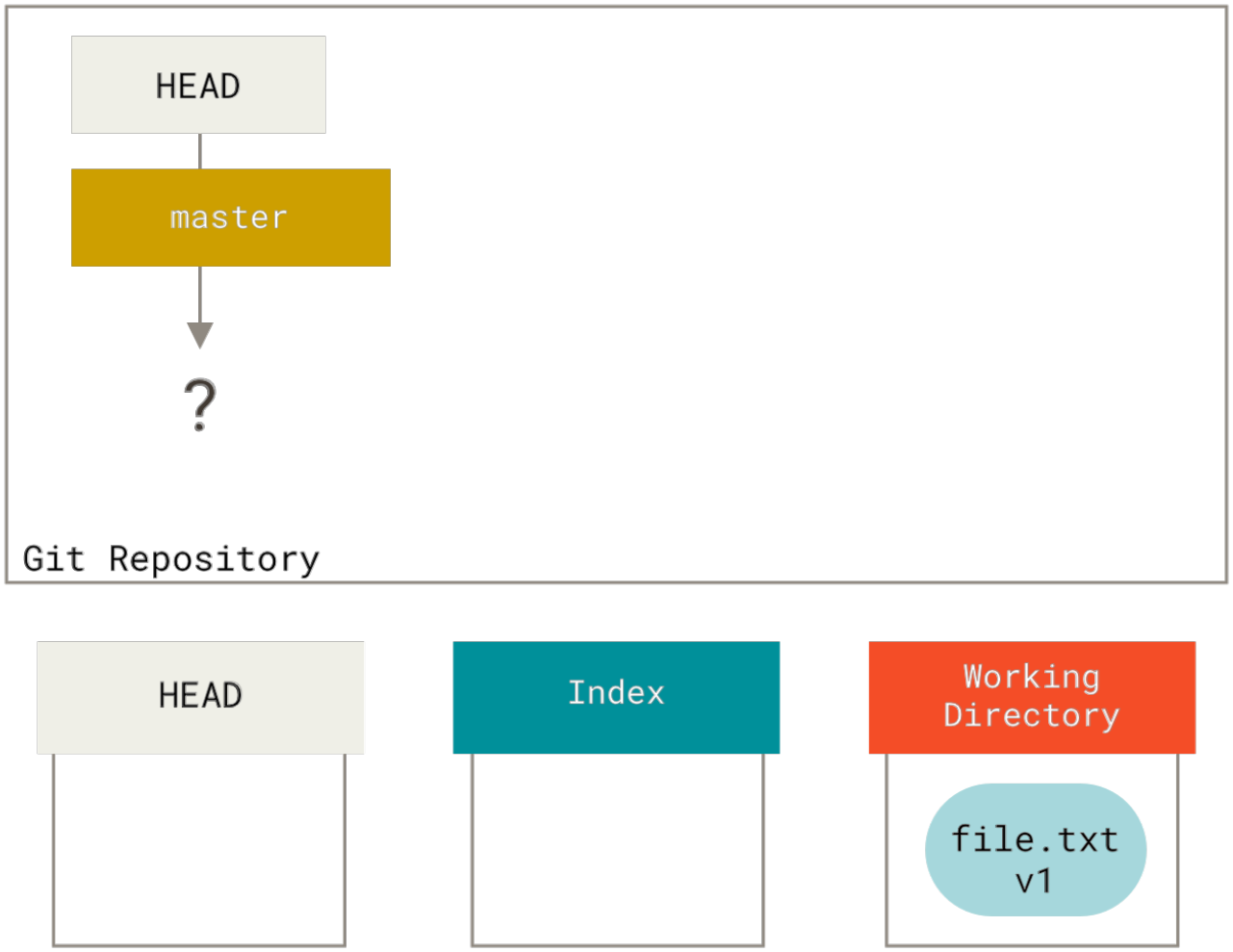


Figure 138. Repositório Git recém-inicializado com arquivo unstaged no diretório de trabalho

Neste ponto, apenas a árvore do diretório de trabalho tem algum conteúdo.

Agora queremos comitar este arquivo, então usamos `git add` para pegar o conteúdo do diretório de trabalho e copiá-lo para o index.

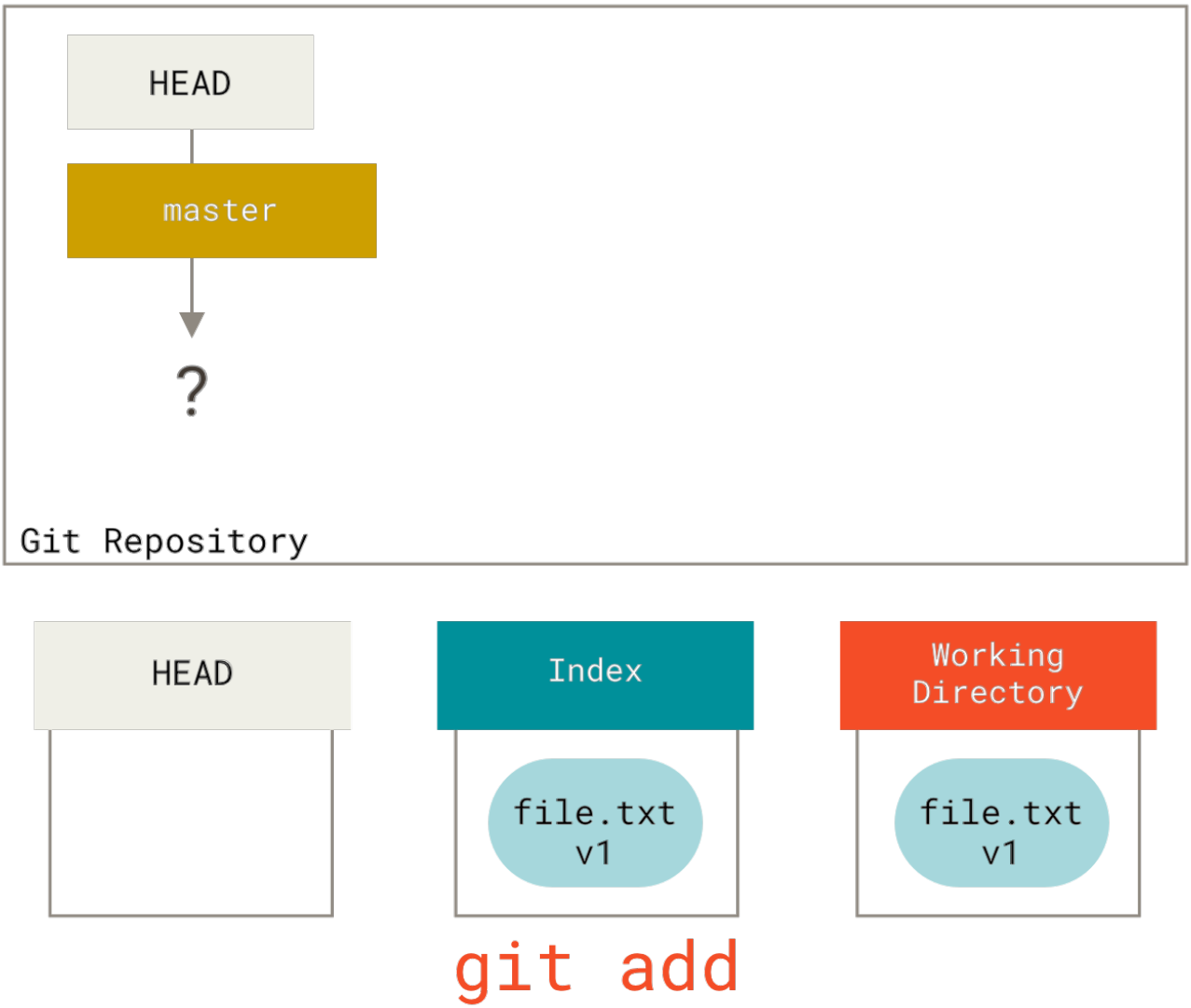


Figure 139. Arquivo é copiado para o index no `git add`

Então nós rodamos `git commit`, o que pega o conteúdo do index e o salva como um snapshot permanente, cria um objeto de commit que aponta para esse snapshot, e atualiza a `master` para apontar para esse commit.

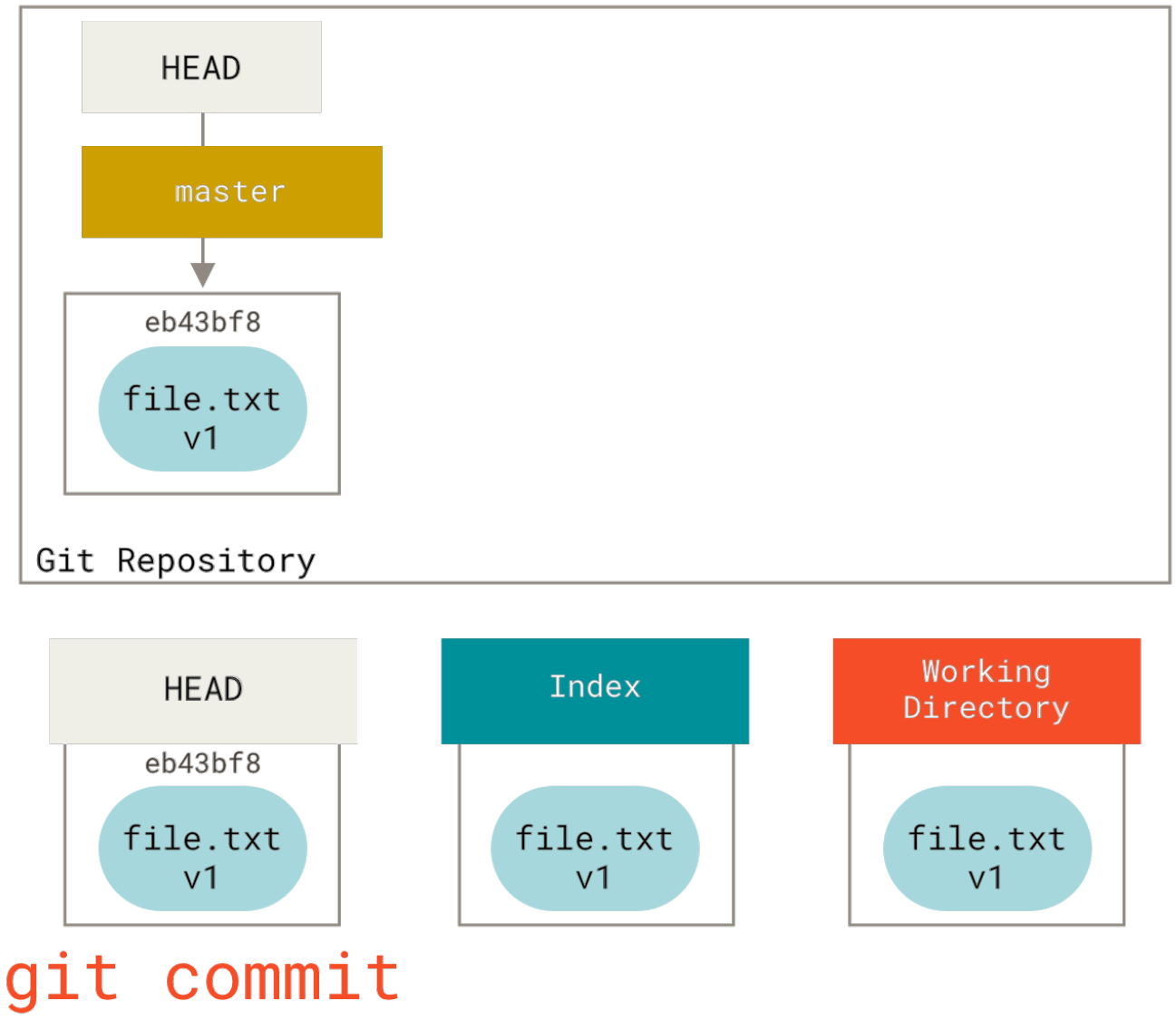


Figure 140. O passo do `git commit`

Se rodarmos `git status`, não veremos alterações, porque todas as três árvores são as mesmas.

Agora queremos fazer uma alteração nesse arquivo e comitá-lo. Passaremos pelo mesmo processo; primeiro, nós alteramos o arquivo no nosso diretório de trabalho. Vamos chamar isso de **v2** do arquivo, e indicá-la em vermelho.

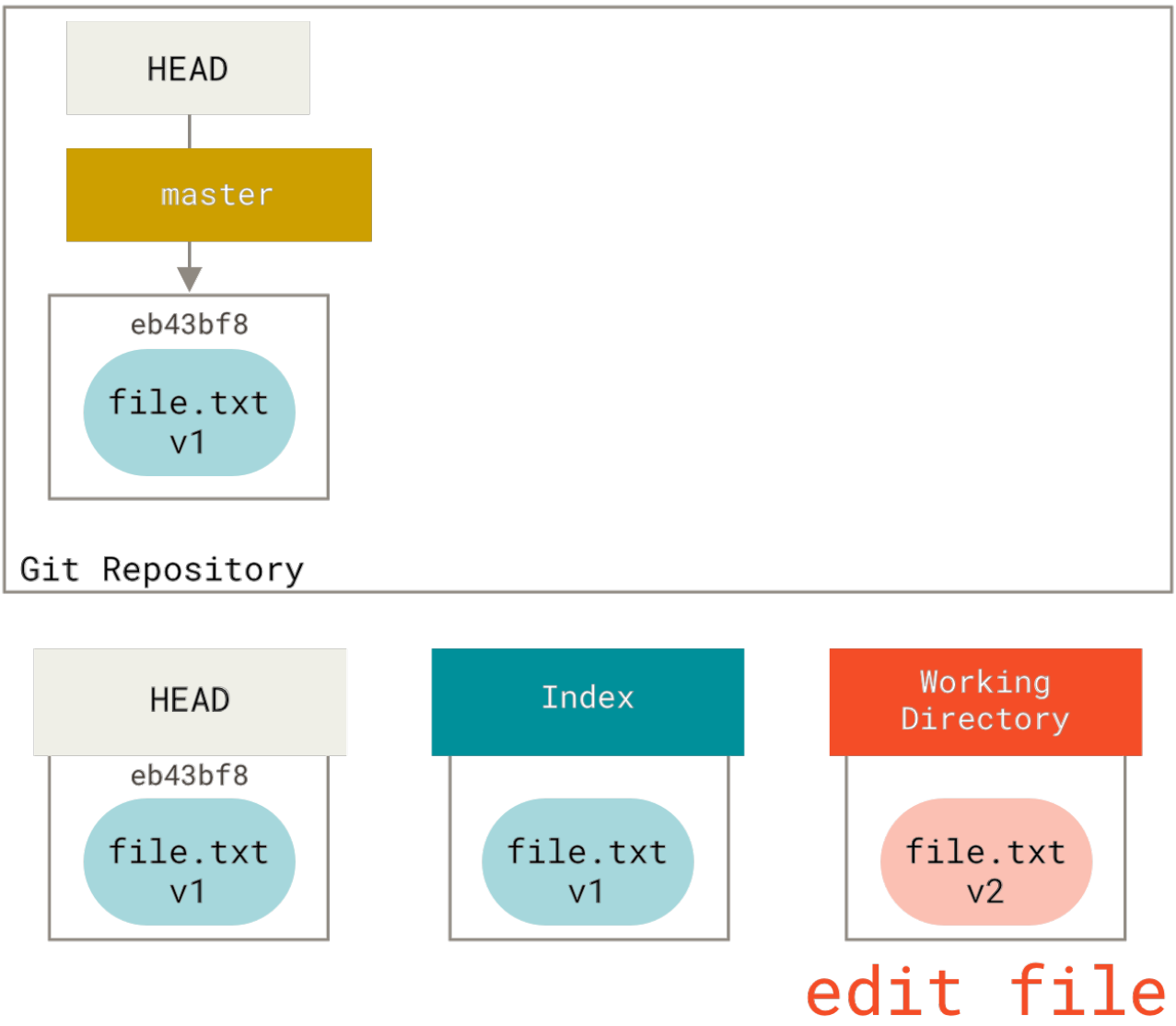


Figure 141. Repositório Git com arquivo alterado no diretório de trabalho

Se rodarmos `git status` agora, veremos o arquivo em vermelho como “Changes not staged for commit” (Alterações não preparadas para commit), porque essa entrada difere entre o index e o diretório de trabalho. Em seguida, nós rodamos `git add` nele para prepará-lo (stage) no nosso index.

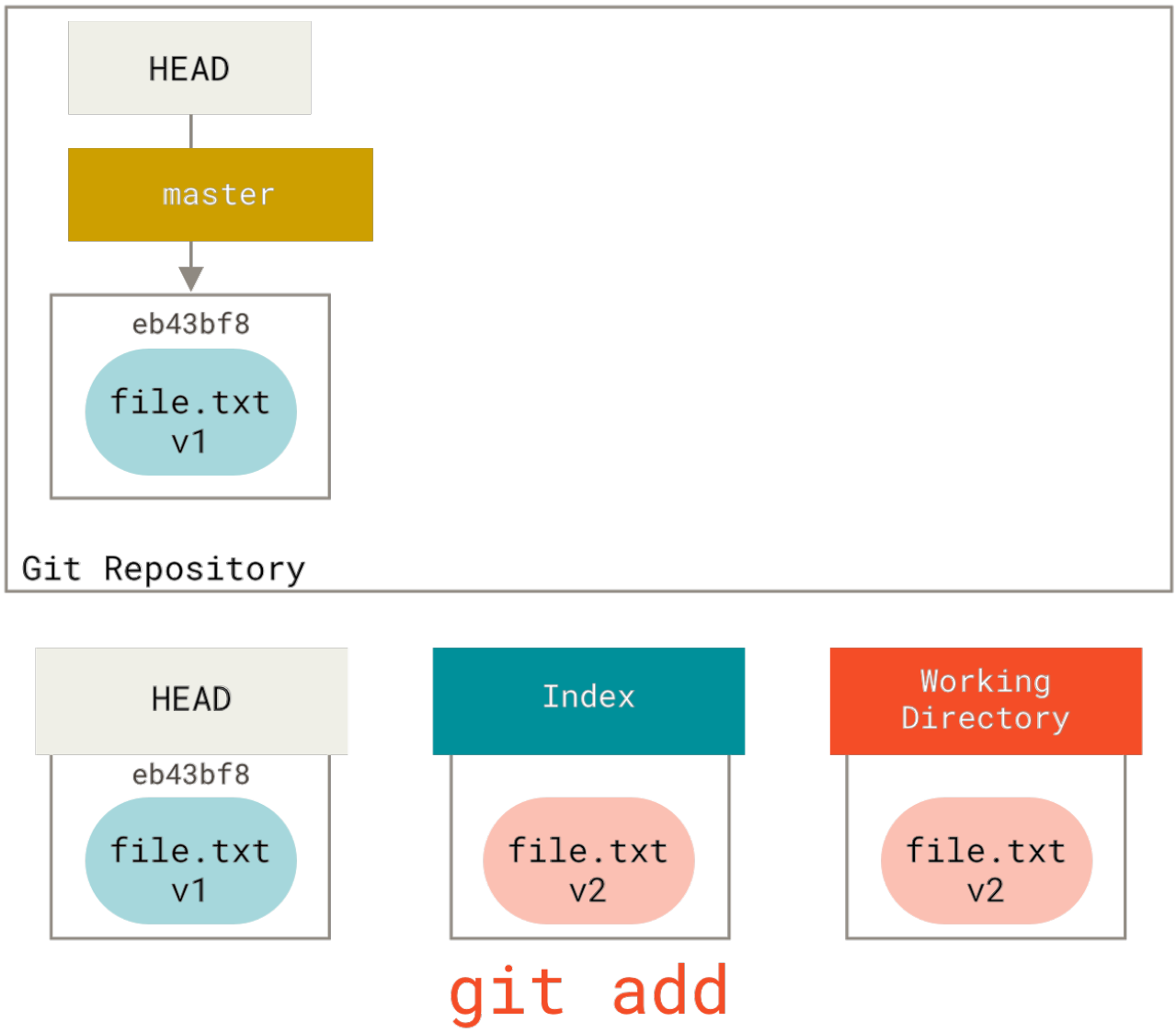


Figure 142. Preparando (Staging) a alteração para o index

Neste ponto, se rodarmos `git status`, veremos o arquivo em verde sob “Changes to be committed” (Alterações a serem comitadas) porque o index e o HEAD diferem — isto é, o nosso próximo commit proposto é agora diferente do nosso último commit. Finalmente, nós rodamos `git commit` para finalizar o commit.

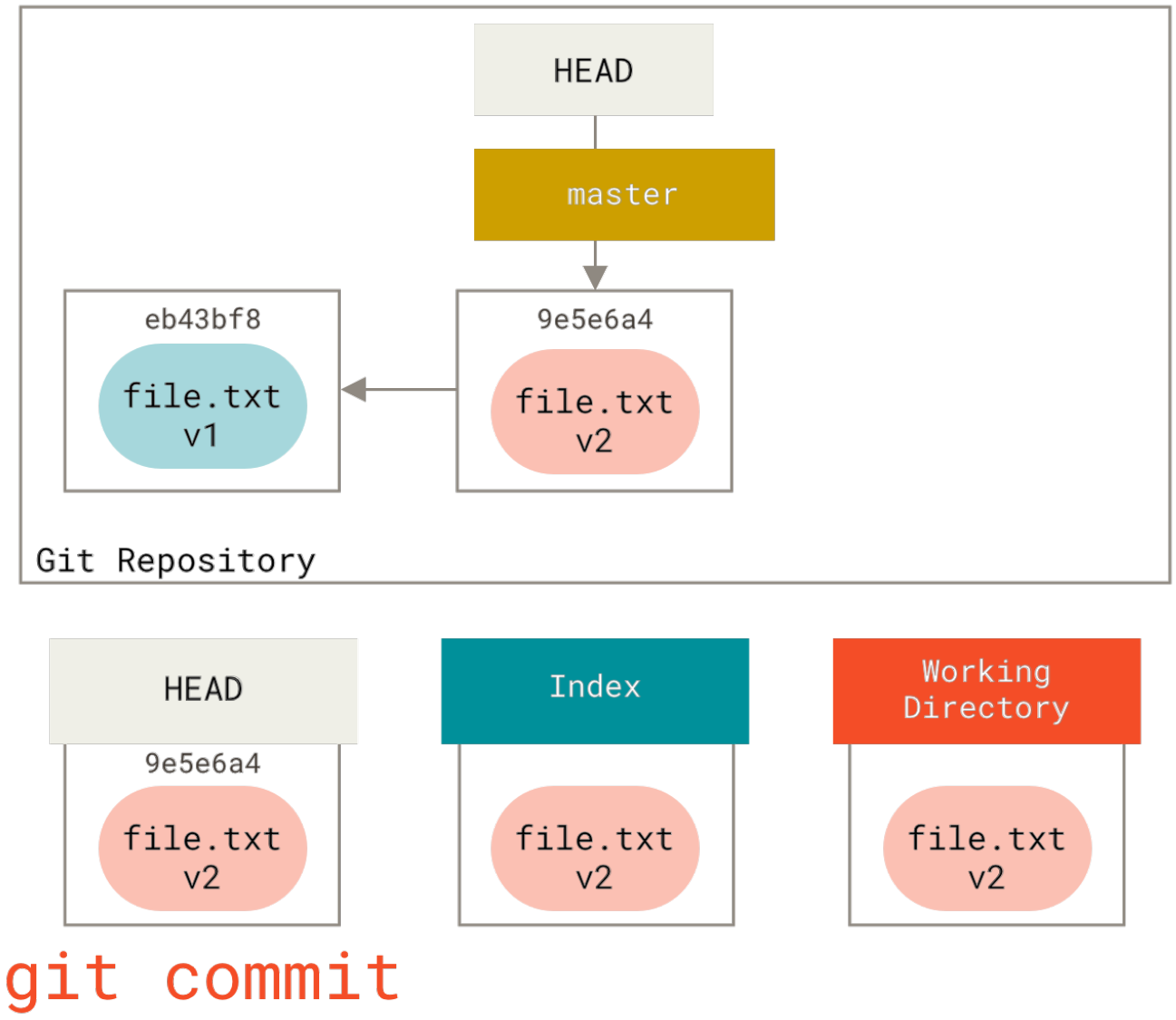


Figure 143. O passo do `git commit` com arquivo alterado

Agora, `git status` não nos dará nenhuma saída, porque todas as três árvores estão iguais novamente.

Trocar de branch ou clonar passa por um processo semelhante. Quando você faz checkout num branch, isso altera o **HEAD** para apontar para a nova ref de branch, popula o seu **index** com o snapshot desse commit, e então copia o conteúdo do **index** para o seu **diretório de trabalho**.

O Papel do Reset

O comando `reset` faz mais sentido quando visto neste contexto.

Para os propósitos destes exemplos, digamos que nós modificamos `file.txt` novamente e o comitamos uma terceira vez. Então agora nosso histórico está assim:

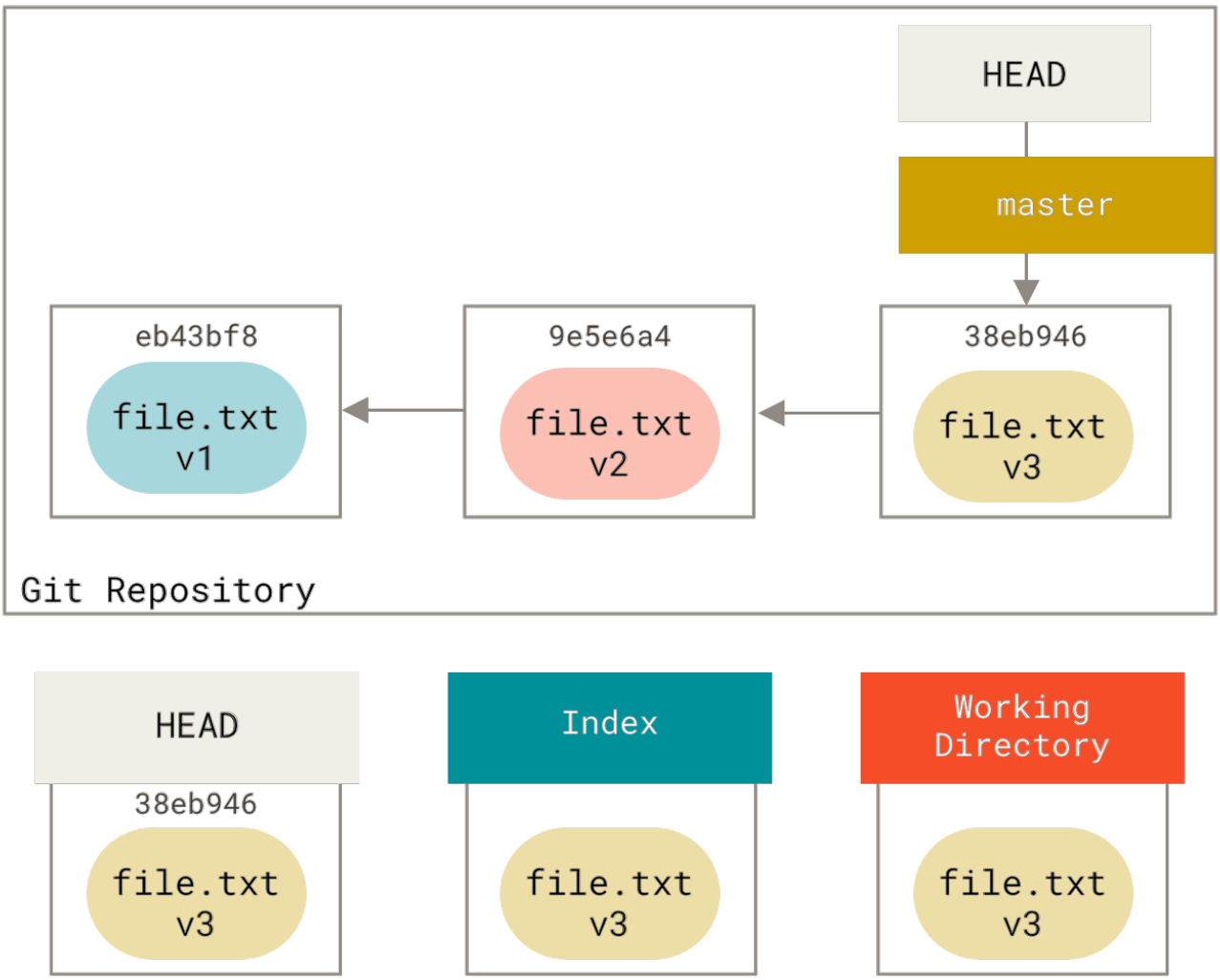


Figure 144. Repositório Git com três commits

Vamos agora percorrer exatamente o que o `reset` faz quando você o chama. Ele manipula diretamente estas três árvores de uma forma simples e previsível. Ele faz até três operações básicas.

Passo 1: Mover o HEAD

A primeira coisa que o `reset` fará é mover para o que o HEAD aponta. Isso não é o mesmo que alterar o próprio HEAD (que é o que o `checkout` faz); `reset` move a branch para a qual o HEAD está apontando. Isso significa que se o HEAD estiver configurado para a branch `master` (i.e. você está atualmente na branch `master`), rodar `git reset 9e5e6a4` começará por fazer a `master` apontar para `9e5e6a4`.

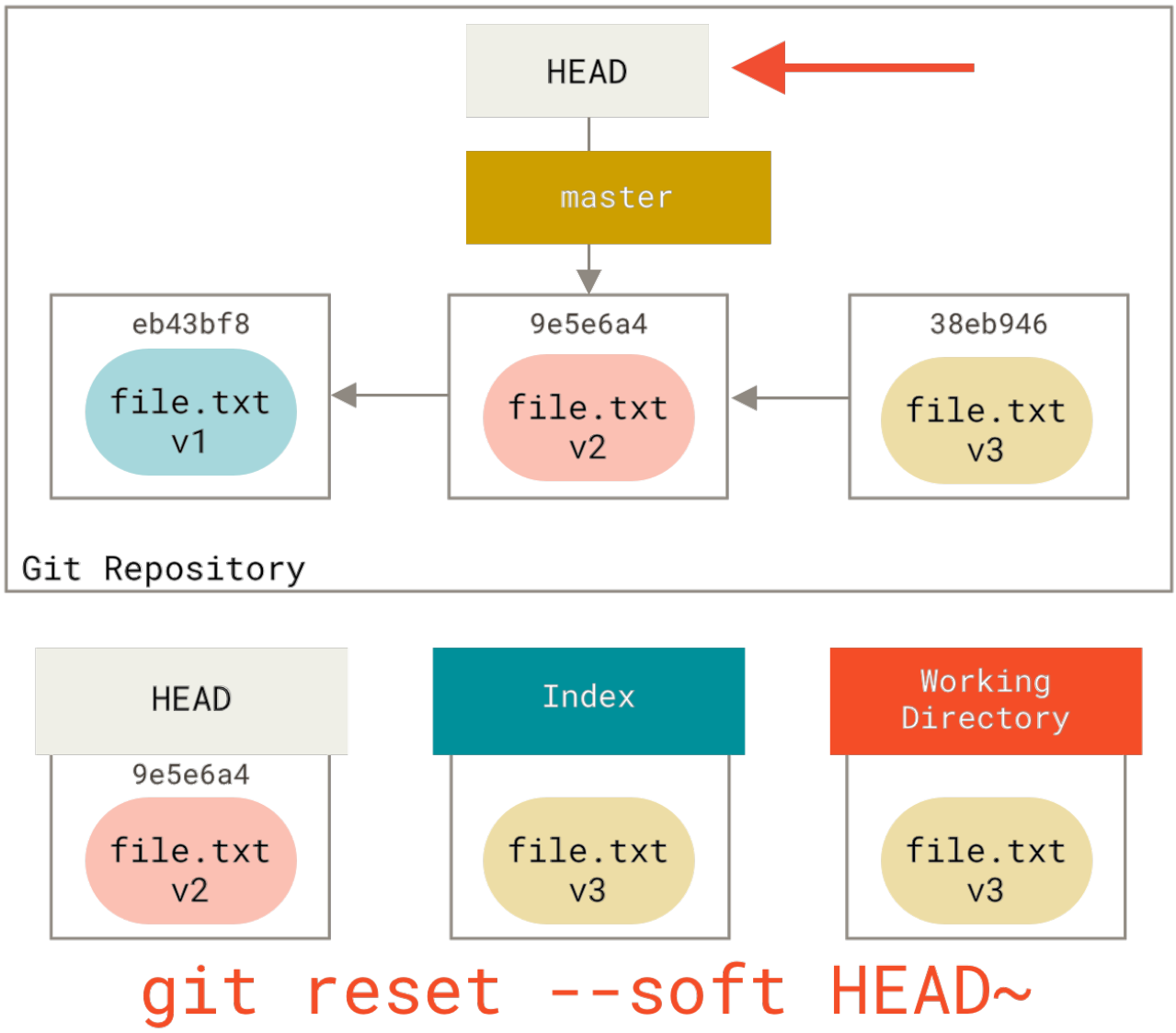


Figure 145. Reset soft

Não importa qual forma de `reset` com um commit você invoque, essa é a primeira coisa que ele sempre tentará fazer. Com o `reset --soft`, ele simplesmente parará por aí.

Agora, reserve um segundo para olhar aquele diagrama e perceber o que aconteceu: ele essencialmente desfez o último comando `git commit`. Quando você roda o `git commit`, o Git cria um novo commit e move a branch para a qual o HEAD aponta para ele. Quando você faz `reset` voltando para `HEAD~` (o pai do HEAD), você está movendo a branch de volta para onde ela estava, sem alterar o index ou o diretório de trabalho. Você poderia agora atualizar o index e rodar `git commit` novamente para realizar o que o `git commit --amend` teria feito (veja Alterando o Último Commit).

Passo 2: Atualizando o Index (`--mixed`)

Note que se você rodar `git status` agora, verá em verde a diferença entre o index e o que é o novo HEAD.

A próxima coisa que o `reset` fará é atualizar o index com o conteúdo do snapshot para o qual o HEAD aponta agora.

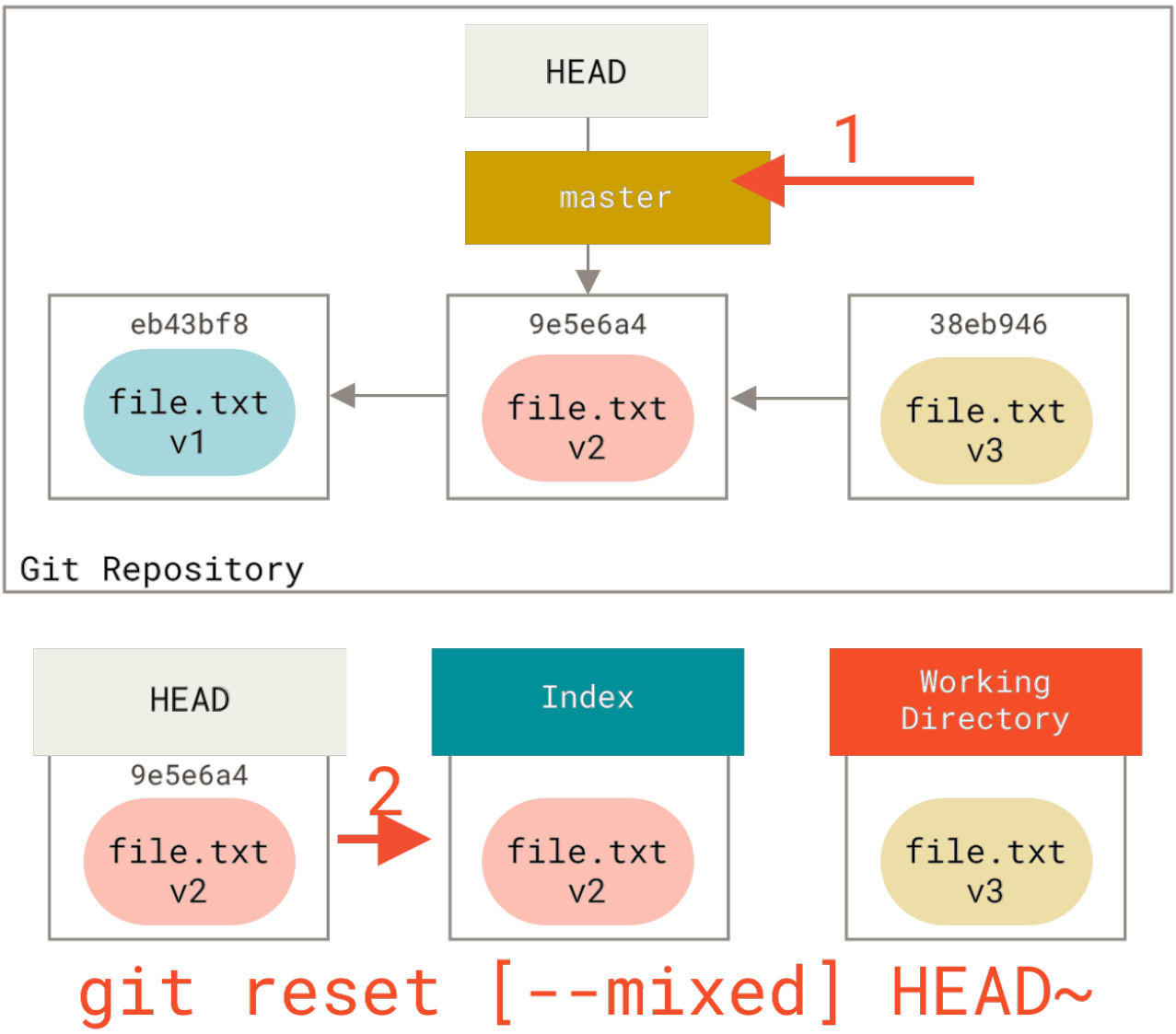


Figure 146. Reset mixed

Se você especificar a opção `--mixed`, o `reset` parará neste ponto. Este também é o padrão, então se você não especificar nenhuma opção (apenas `git reset HEAD~` neste caso), é aqui que o comando vai parar.

Agora, reserve outro segundo para olhar aquele diagrama e perceber o que aconteceu: ele ainda desfez o seu último `commit`, mas também tirou do stage (*unstaged*) tudo. Você reverteu (rolled back) para antes de rodar todos os seus comandos `git add` e `git commit`.

Passo 3: Atualizando o Diretório de Trabalho (`--hard`)

A terceira coisa que o `reset` fará é fazer o diretório de trabalho se parecer com o index. Se você usar a opção `--hard`, ele continuará para esta etapa.

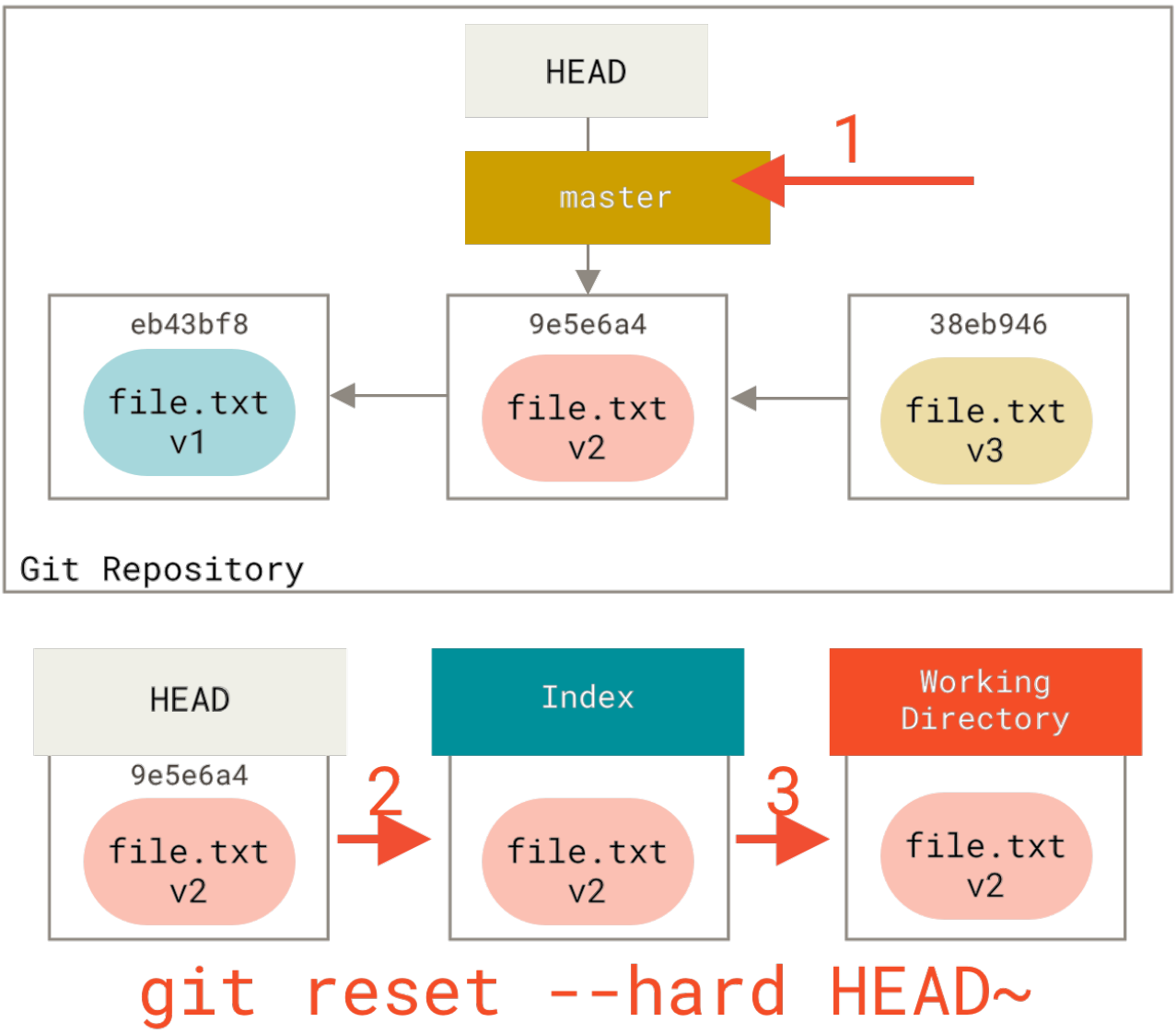


Figure 147. Reset hard

Então, vamos pensar sobre o que acabou de acontecer. Você desfez o seu último commit, os comandos `git add` e `git commit`, **e** todo o trabalho que você fez no seu diretório de trabalho.

É importante notar que esta flag (`--hard`) é a única maneira de tornar o comando `reset` perigoso, e um dos pouquíssimos casos em que o Git realmente destruirá dados. Qualquer outra invocação de `reset` pode ser facilmente desfeita, mas a opção `--hard` não, pois ela substitui forçosamente (overwrites) arquivos no diretório de trabalho. Neste caso particular, nós ainda temos a versão **v3** do nosso arquivo em um commit no nosso banco de dados (DB) do Git, e poderíamos recuperá-la olhando para o nosso `reflog`, mas se não a tivéssemos comitado, o Git ainda assim teria sobrescrito o arquivo e ele seria irrecuperável.

Recapitulação

O comando `reset` substitui (overwrites) estas três árvores numa ordem específica, parando quando você manda:

1. Move a branch para a qual o HEAD aponta *(pare aqui se usar `--soft`)*.
2. Faz o index ficar igual ao HEAD *(pare aqui a menos que use `--hard`)*.

3. Faz o diretório de trabalho ficar igual ao index.

Reset com um Caminho (Path)

Isso cobre o comportamento do `reset` em sua forma básica, mas você também pode fornecer a ele um caminho (path) para agir sobre. Se você especificar um caminho, o `reset` pulará o passo 1 e limitará o restante de suas ações a um arquivo ou conjunto de arquivos específico. Isso até que faz sentido — o HEAD é apenas um ponteiro, e você não pode apontar para parte de um commit e parte de outro. Mas o index e o diretório de trabalho *podem* ser parcialmente atualizados, então o reset prossegue com os passos 2 e 3.

Então, assuma que rodamos `git reset file.txt`. Esta forma (já que você não especificou um SHA-1 de commit ou branch, e não especificou `--soft` ou `--hard`) é um atalho para `git reset --mixed HEAD file.txt`, o qual irá:

1. Mover a branch para a qual o HEAD aponta *(pulado)*.
2. Fazer o index ficar igual ao HEAD *(pare aqui)*.

Então, essencialmente, ele apenas copia o `file.txt` do HEAD para o index.

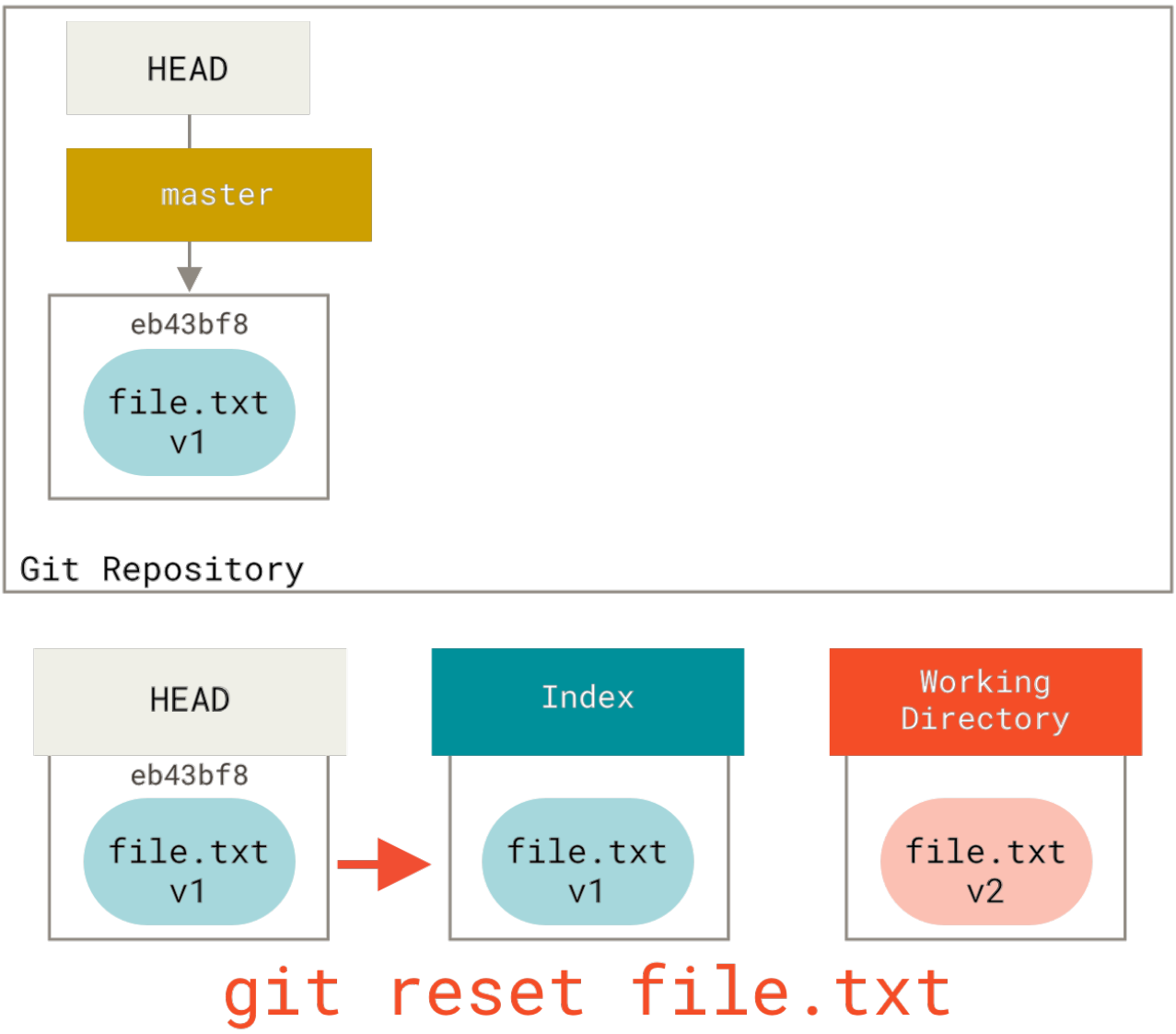


Figure 148. Reset mixed com um caminho (path)

Isso tem o efeito prático de tirar o arquivo do stage (*unstaging*). Se olharmos para o diagrama desse comando e pensarmos sobre o que o `git add` faz, eles são exatamente o oposto.

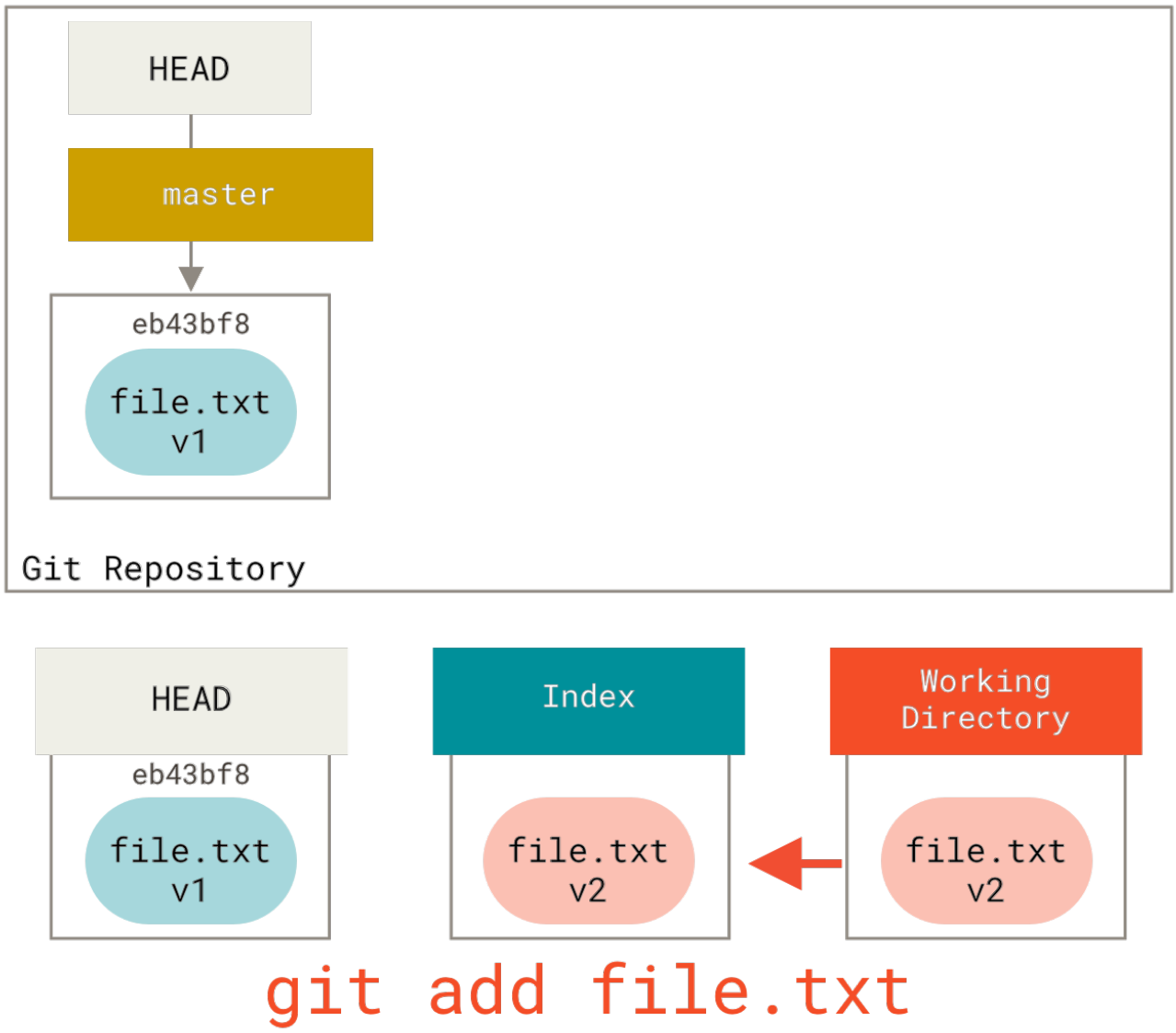


Figure 149. Preparando arquivo (Staging) para o index

É por isso que a saída do comando `git status` sugere que você rode isto para tirar um arquivo do stage (veja Retirando a Preparação de um Arquivo Preparado (Unstaging) para mais informações sobre isso).

Poderíamos facilmente não deixar o Git assumir que queríamos dizer “pegue os dados do HEAD” especificando um commit específico para puxar (pull) aquela versão do arquivo. Nós apenas rodaríamos algo como `git reset eb43bf file.txt`.

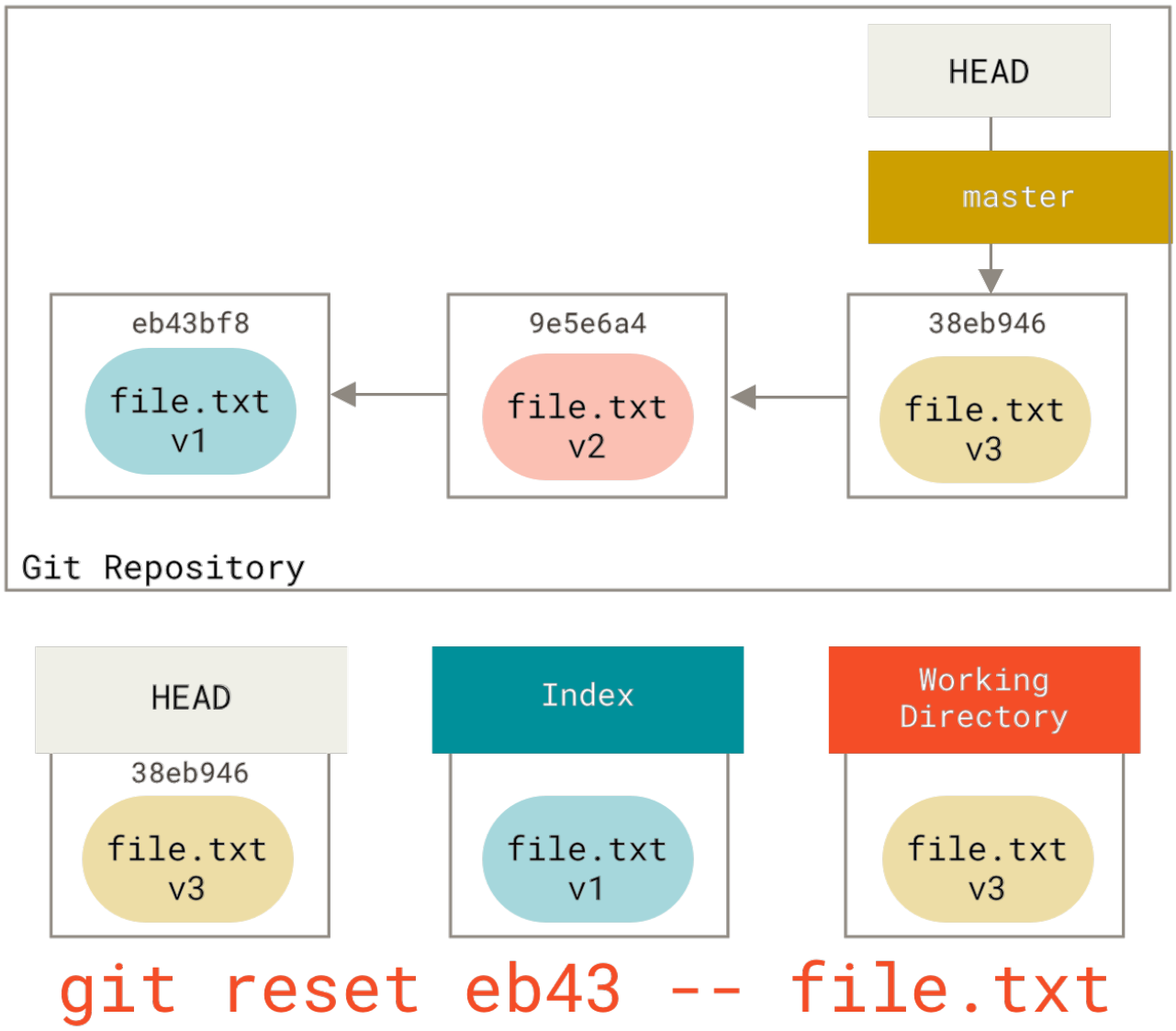


Figure 150. Reset soft com um caminho para um commit específico

Isso efetivamente faz a mesma coisa como se tivéssemos revertido o conteúdo do arquivo para **v1** no diretório de trabalho, rodado `git add` nele, e então revertido de volta para **v3** novamente (sem realmente passar por todos esses passos). Se rodarmos `git commit` agora, ele registrará uma alteração que reverte esse arquivo de volta para **v1**, mesmo que nunca o tenhamos realmente tido no nosso diretório de trabalho novamente.

Também é interessante notar que, assim como o `git add`, o comando `reset` aceitará a opção `--patch` para remover partes do conteúdo do stage trecho por trecho (hunk-by-hunk). Assim, você pode remover do stage ou reverter conteúdos seletivamente.

Juntando (Squashing)

Vamos olhar como fazer algo interessante com este poder recém-descoberto — juntar (squashing) commits.

Digamos que você tenha uma série de commits com mensagens como “oops.”, “WIP” (work in progress) e “forgot this file” (esqueci desse arquivo). Você pode usar o `reset` para juntá-los rápida e facilmente em um único commit que faz você parecer muito esperto. A seção Juntando Commits

(Squashing) mostra outra maneira de fazer isso, mas neste exemplo é mais simples usar o `reset`.

Digamos que você tenha um projeto onde o primeiro commit tem um arquivo, o segundo commit adicionou um novo arquivo e alterou o primeiro, e o terceiro commit alterou o primeiro arquivo novamente. O segundo commit foi um trabalho em andamento e você quer juntá-lo.

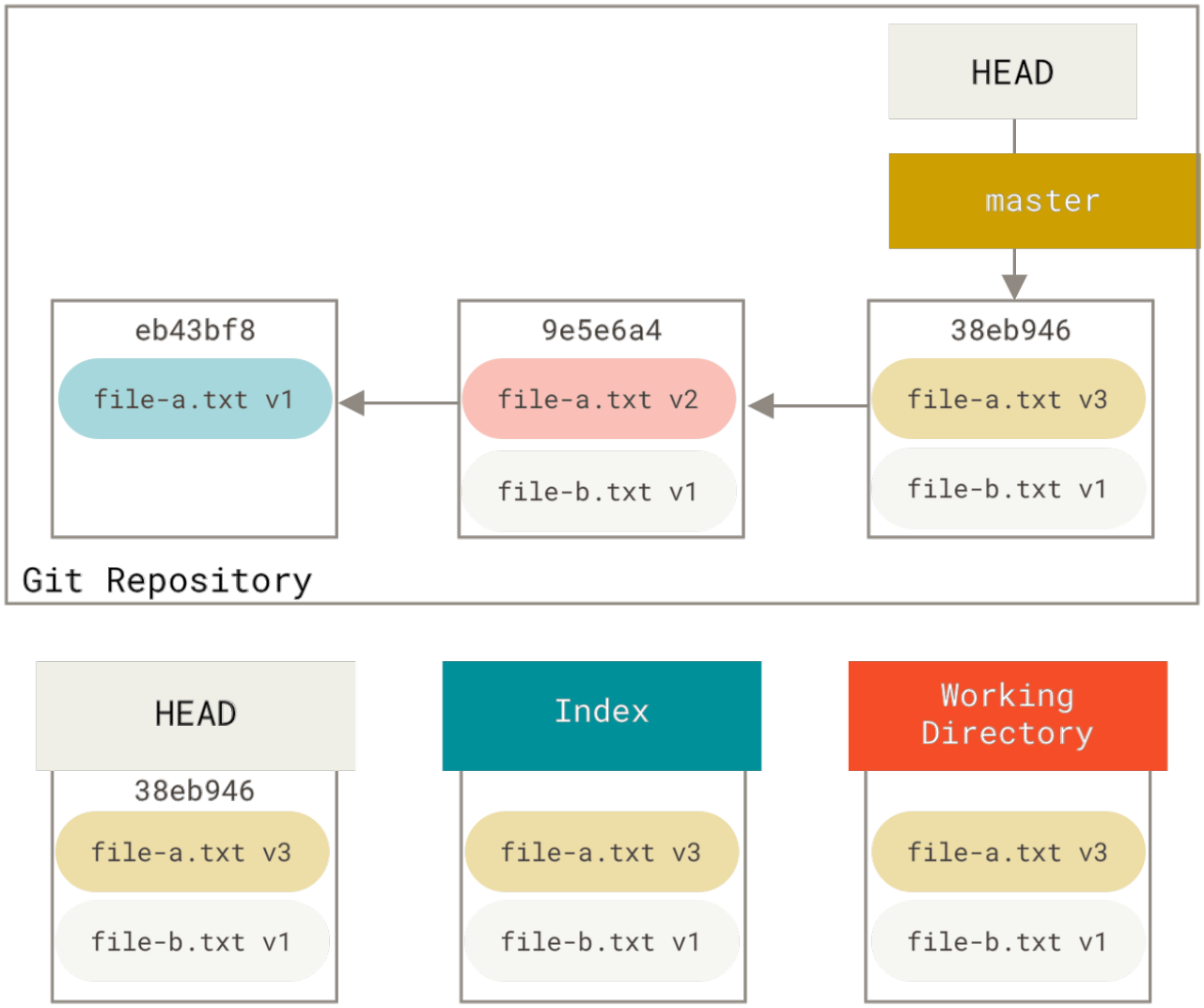


Figure 151. Repositório Git

Você pode rodar `git reset --soft HEAD~2` para mover a branch HEAD de volta para um commit mais antigo (o commit mais recente que você quer manter):

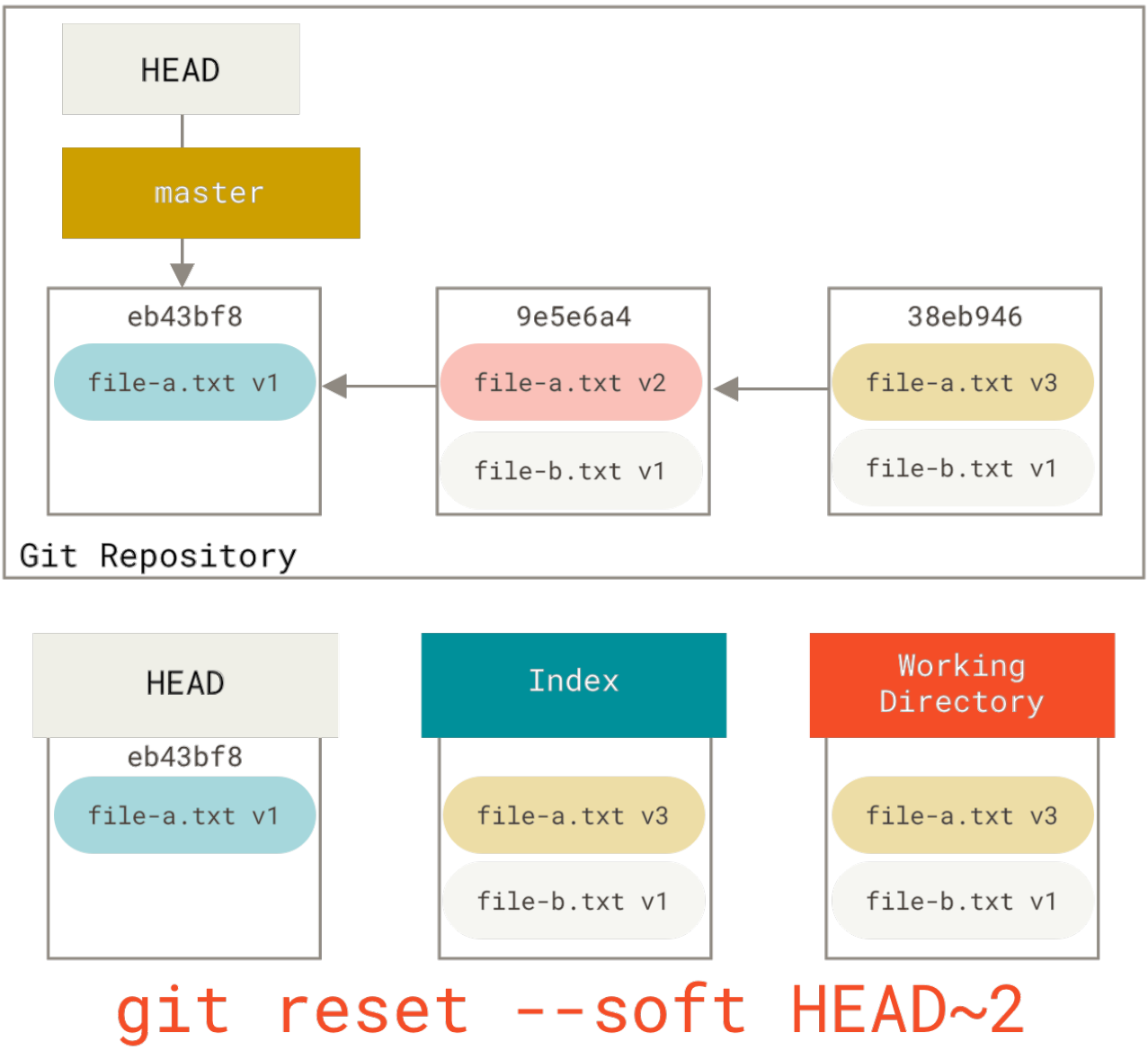


Figure 152. Movendo HEAD com reset soft

E então simplesmente rodar `git commit` novamente:

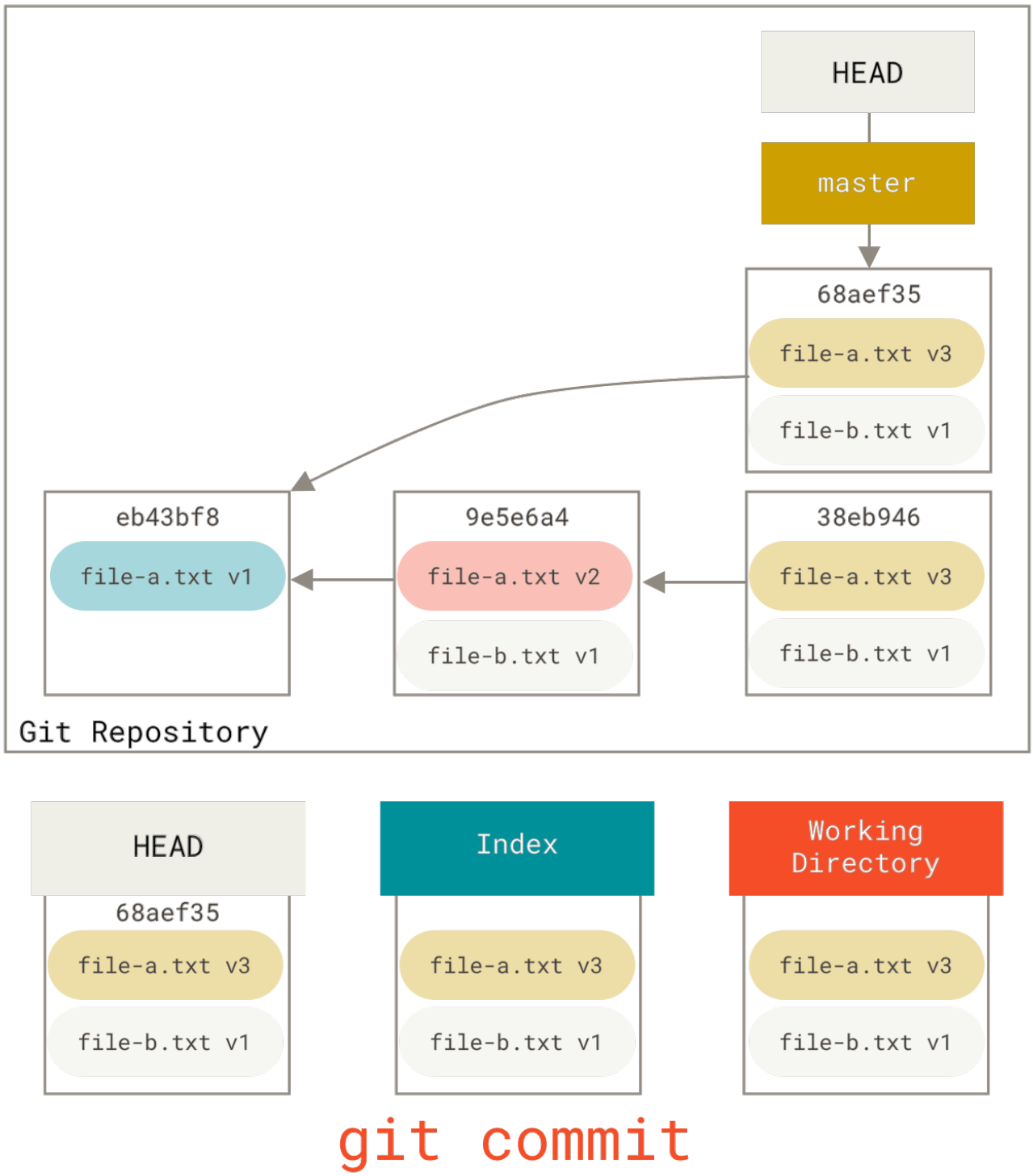


Figure 153. Repositório Git com commit juntado (squashed)

Agora você pode ver que o seu histórico alcançável, o histórico que você faria push, agora parece que você teve um commit com `file-a.txt` **v1**, então um segundo que modificou `file-a.txt` para **v3** e adicionou `file-b.txt`. O commit com a versão **v2** do arquivo não está mais no histórico.

Fazer Checkout (Check It Out)

Finalmente, você pode se perguntar qual é a diferença entre `checkout` e `reset`. Assim como o `reset`, o `checkout` manipula as três árvores, e é um pouco diferente dependendo se você passa um caminho de arquivo para o comando ou não.

Sem Caminhos (Paths)

Rodar `git checkout [branch]` é muito parecido com rodar `git reset --hard [branch]` no sentido de que ele atualiza todas as três árvores para você para ficarem parecidas com `[branch]`, mas há duas diferenças importantes.

Primeiro, diferentemente do `reset --hard`, o `checkout` é seguro para o diretório de trabalho; ele verificará para se certificar de que não está apagando arquivos que tenham alterações neles. Na verdade, ele é um pouco mais inteligente que isso — ele tenta fazer um merge trivial no diretório de trabalho, de modo que todos os arquivos que você *não* alterou serão atualizados. O `reset --hard`, por outro lado, substituirá tudo de forma generalizada sem verificar.

A segunda diferença importante é como o `checkout` atualiza o HEAD. Enquanto o `reset` moverá a branch para a qual o HEAD aponta, o `checkout` moverá o próprio HEAD para apontar para outra branch.

Por exemplo, digamos que temos as branches `master` e `develop` que apontam para commits diferentes, e nós estamos atualmente na `develop` (então o HEAD aponta para ela). Se rodarmos `git reset master`, a própria `develop` agora apontará para o mesmo commit que a `master`. Se, em vez disso, rodarmos `git checkout master`, a `develop` não se move, o HEAD sim. O HEAD agora apontará para `master`.

Então, em ambos os casos estamos movendo o HEAD para apontar para o commit A, mas *como* nós fazemos isso é muito diferente. O `reset` moverá a branch para a qual o HEAD aponta, o `checkout` move o próprio HEAD.

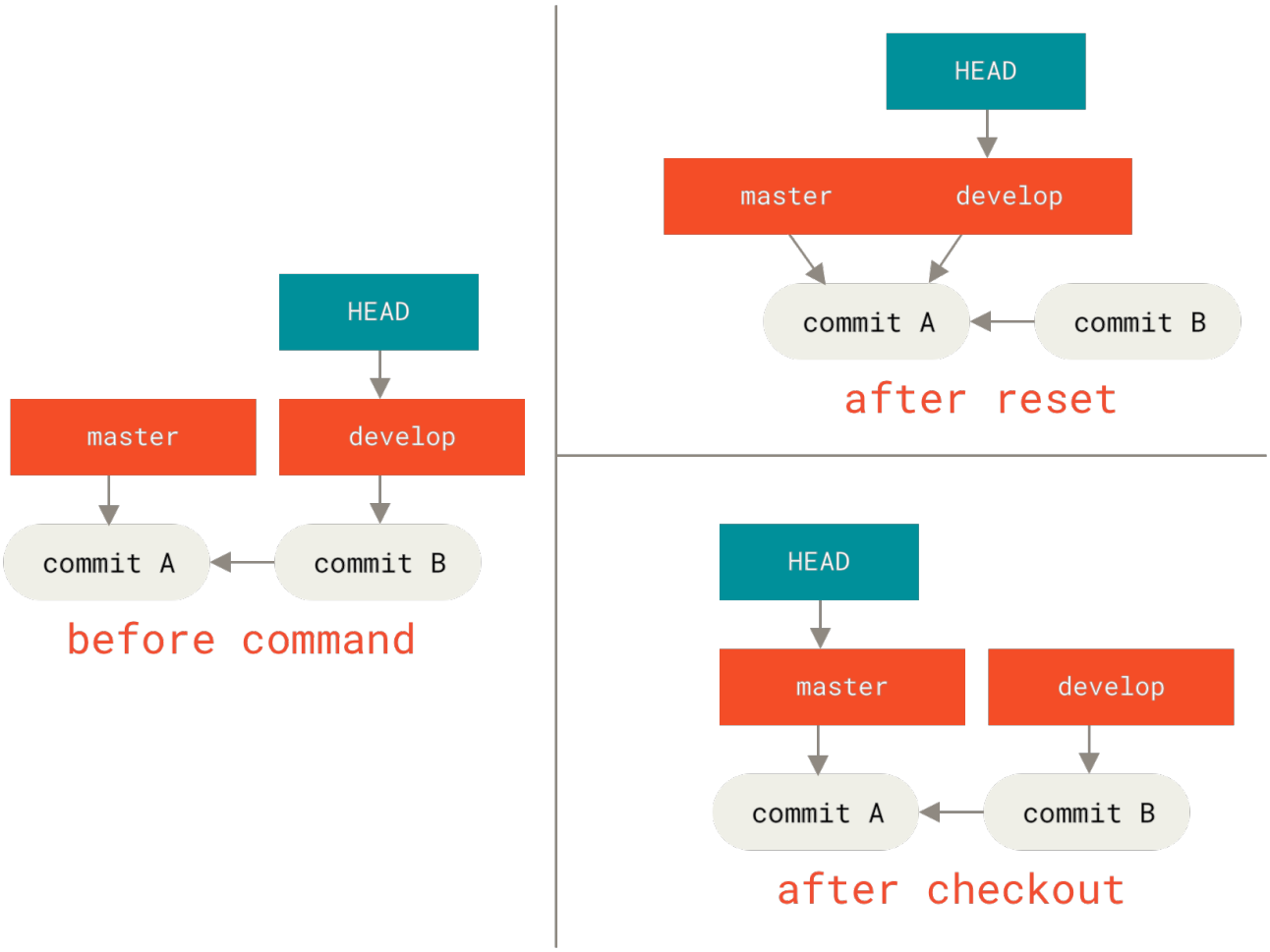


Figure 154. `git checkout` *e* `git reset`

Com Caminhos (Paths)

A outra maneira de rodar `checkout` é com um caminho de arquivo, que, assim como o `reset`, não move o HEAD. Ele é exatamente como `git reset [branch] file` no sentido de que ele atualiza o index com esse arquivo naquele commit, mas ele também sobrescreve o arquivo no diretório de trabalho. Seria exatamente como `git reset --hard [branch] file` (se o `reset` permitisse que você rodasse isso) — ele não é seguro para o diretório de trabalho e não move o HEAD.

Além disso, assim como `git reset` e `git add`, o `checkout` aceitará a opção `--patch` para permitir que você reverta o conteúdo de um arquivo seletivamente trecho por trecho (hunk-by-hunk).

Resumo

Esperamos que agora você entenda e se sinta mais confortável com o comando `reset`, mas provavelmente ainda esteja um pouco confuso sobre como exatamente ele difere do `checkout` e não consiga se lembrar de todas as regras das diferentes invocações.

Aqui está uma folha de dicas (cheat-sheet) sobre quais comandos afetam quais árvores. A coluna “HEAD” lê “REF” se o comando mover a referência (branch) para a qual o HEAD aponta, e “HEAD” se ele mover o próprio HEAD. Preste atenção especial à coluna 'WD Seguro?' — se disser **NÃO**, reserve um segundo para pensar antes de rodar o comando.

	HEAD	Index	Workdir	WD Seguro?
Nível do Commit				
`reset --soft [commit]`	REF	NÃO	NÃO	SIM
`reset [commit]`	REF	SIM	NÃO	SIM
`reset --hard [commit]`	REF	SIM	SIM	**NÃO**
`checkout <commit>`	HEAD	SIM	SIM	SIM
Nível do Arquivo				
`reset [commit] <paths>`	NÃO	SIM	NÃO	SIM
`checkout [commit] <paths>`	NÃO	SIM	SIM	**NÃO**

Merging Avançado

Fazer merge no Git é tipicamente bem fácil. Já que o Git facilita o merge de outra branch várias vezes, isso significa que você pode ter uma branch de vida muito longa, mas você pode mantê-la atualizada conforme avança, resolvendo pequenos conflitos frequentemente, em vez de ser surpreendido por um conflito enorme no final da série.

No entanto, às vezes ocorrem conflitos complicados. Ao contrário de alguns outros sistemas de controle de versão, o Git não tenta ser excessivamente esperto sobre a resolução de conflitos de merge. A filosofia do Git é ser inteligente para determinar quando a resolução de um merge é inequívoca, mas se houver um conflito, ele não tenta ser esperto para resolvê-lo automaticamente. Portanto, se você esperar muito tempo para fazer merge de duas branches que divergem rapidamente, você pode ter alguns problemas.

Nesta seção, abordaremos quais podem ser alguns desses problemas e quais ferramentas o Git fornece para ajudar a lidar com essas situações mais complicadas. Também abordaremos alguns dos diferentes tipos não padronizados de merges que você pode fazer, bem como veremos como desfazer merges que você já fez.

Conflitos de Merge

Embora tenhamos coberto alguns conceitos básicos sobre como resolver conflitos de merge em Conflitos Básicos de Mesclagem (Merge Conflicts), para conflitos mais complexos, o Git fornece algumas ferramentas para ajudá-lo a descobrir o que está acontecendo e como lidar melhor com o conflito.

Em primeiro lugar, se possível, tente certificar-se de que o seu diretório de trabalho está limpo antes de fazer um merge que possa ter conflitos. Se você tiver trabalho em andamento, ou comite-o em uma branch temporária ou faça o stash dele. Isso faz com que você possa desfazer **qualquer coisa** que tentar aqui. Se você tiver alterações não salvas no seu diretório de trabalho ao tentar um merge, algumas dessas dicas podem ajudá-lo a preservar esse trabalho.

Vamos analisar um exemplo muito simples. Temos um arquivo Ruby super simples que imprime 'hello world'.

```
#! /usr/bin/env ruby

def hello
  puts 'hello world'
end

hello()
```

No nosso repositório, nós criamos uma nova branch chamada `whitespace` e procedemos à alteração de todos os finais de linha do Unix para os finais de linha do DOS, alterando essencialmente todas as linhas do arquivo, mas apenas com espaços em branco (whitespace). Então nós alteramos a linha "hello world" para "hello mundo".

```
$ git checkout -b whitespace
Switched to a new branch 'whitespace'

$ unix2dos hello.rb
unix2dos: converting file hello.rb to DOS format ...
$ git commit -am 'Convert hello.rb to DOS'
[whitespace 3270f76] Convert hello.rb to DOS
 1 file changed, 7 insertions(+), 7 deletions(-)

$ vim hello.rb
$ git diff -b
diff --git a/hello.rb b/hello.rb
index ac51efd..e85207e 100755
--- a/hello.rb
+++ b/hello.rb
@@ -1,7 +1,7 @@
 #! /usr/bin/env ruby

 def hello
-  puts 'hello world'
+  puts 'hello mundo'^M
 end

 hello()

$ git commit -am 'Use Spanish instead of English'
[whitespace 6d338d2] Use Spanish instead of English
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Agora nós mudamos de volta para a nossa branch `master` e adicionamos alguma documentação para a função.

```
$ git checkout master
Switched to branch 'master'
```

```
$ vim hello.rb
$ git diff
diff --git a/hello.rb b/hello.rb
index ac51efd..36c06c8 100755
--- a/hello.rb
+++ b/hello.rb
@@ -1,5 +1,6 @@
 #! /usr/bin/env ruby

+# prints out a greeting
 def hello
   puts 'hello world'
 end

$ git commit -am 'Add comment documenting the function'
[master bec6336] Add comment documenting the function
 1 file changed, 1 insertion(+)
```

Agora tentamos fazer o merge da nossa branch `whitespace` e teremos conflitos por causa das alterações de espaço em branco.

```
$ git merge whitespace
Auto-merging hello.rb
CONFLICT (content): Merge conflict in hello.rb
Automatic merge failed; fix conflicts and then commit the result.
```

Abortando um Merge

Nós agora temos algumas opções. Primeiro, vamos cobrir como sair desta situação. Se talvez você não estivesse esperando conflitos e não quer lidar bem com a situação ainda, você pode simplesmente cancelar o merge com `git merge --abort`.

```
$ git status -sb
## master
UU hello.rb

$ git merge --abort

$ git status -sb
## master
```

A opção `git merge --abort` tenta reverter de volta para o seu estado antes de você rodar o merge. Os únicos casos onde ele pode não conseguir fazer isso perfeitamente seria se você tivesse alterações não comitadas e não stashadas no seu diretório de trabalho quando você o rodou, caso contrário, deve funcionar bem.

Se por alguma razão você só quer começar de novo, você também pode rodar `git reset --hard HEAD`, e seu repositório voltará para o último estado comitado. Lembre-se que qualquer trabalho não

comitado será perdido, então certifique-se de que você não quer nenhuma das suas alterações.

Ignorando Espaços em Branco (Whitespace)

Neste caso específico, os conflitos estão relacionados a espaços em branco. Nós sabemos isso porque o caso é simples, mas também é bem fácil dizer em casos reais ao olhar para o conflito porque toda linha é removida de um lado e adicionada de novo no outro. Por padrão, o Git vê todas estas linhas como tendo sido alteradas, então ele não consegue fazer o merge dos arquivos.

No entanto, a estratégia de merge padrão pode receber argumentos, e alguns deles são sobre ignorar adequadamente alterações de espaços em branco. Se você ver que tem muitos problemas de espaços em branco em um merge, você pode simplesmente abortá-lo e fazê-lo novamente, desta vez com `-Xignore-all-space` ou `-Xignore-space-change`. A primeira opção ignora o espaço em branco **completamente** ao comparar linhas, a segunda trata sequências de um ou mais caracteres de espaço em branco como equivalentes.

```
$ git merge -Xignore-space-change whitespace
Auto-merging hello.rb
Merge made by the 'recursive' strategy.
 hello.rb | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Já que neste caso, as alterações reais no arquivo não eram conflitantes, uma vez que ignoramos as alterações de espaços em branco, tudo faz merge perfeitamente.

Isso é um salva-vidas se você tem alguém na sua equipe que gosta de ocasionalmente reformatar tudo de espaços para tabs ou vice-versa.

Re-merging Manual de Arquivos

Embora o Git lide muito bem com o pré-processamento de espaços em branco, existem outros tipos de alterações que talvez o Git não consiga lidar automaticamente, mas são correções scriptáveis. Como exemplo, vamos fingir que o Git não conseguiu lidar com a alteração de espaços em branco e nós precisamos fazer isso à mão.

O que realmente precisamos fazer é rodar o arquivo que estamos tentando fazer merge através de um programa `dos2unix` antes de tentar o merge real do arquivo. Então, como faríamos isso?

Primeiro, nós entramos no estado de conflito de merge. Então nós queremos obter cópias da nossa versão do arquivo, a versão deles (da branch que estamos fazendo merge) e a versão comum (de onde ambos os lados ramificaram). Em seguida, queremos corrigir o lado deles ou o nosso e tentar novamente o merge apenas para este único arquivo.

Obter as três versões do arquivo é na verdade bem fácil. O Git armazena todas estas versões no index sob “estágios” (stages) que cada um tem números associados a eles. O Estágio 1 é o ancestral comum, o estágio 2 é a sua versão e o estágio 3 é do `MERGE_HEAD`, a versão que você está fazendo merge (“theirs”).

Você pode extrair uma cópia de cada uma dessas versões do arquivo conflitante com o comando `git`

`show` e uma sintaxe especial.

```
$ git show :1:hello.rb > hello.common.rb
$ git show :2:hello.rb > hello.ours.rb
$ git show :3:hello.rb > hello.theirs.rb
```

Se você quiser ser um pouco mais hardcore, você também pode usar o comando de encanamento (plumbing) `ls-files -u` para obter os SHA-1 reais dos blobs do Git para cada um desses arquivos.

```
$ git ls-files -u
100755 ac51efdc3df4f4fd328d1a02ad05331d8e2c9111 1   hello.rb
100755 36c06c8752c78d2aff89571132f3bf7841a7b5c3 2   hello.rb
100755 e85207e04dfdd5eb0a1e9febbc67fd837c44a1cd 3   hello.rb
```

O `:1:hello.rb` é apenas um atalho (shorthand) para procurar aquele SHA-1 do blob.

Agora que temos o conteúdo de todos os três estágios em nosso diretório de trabalho, podemos consertar manualmente a versão deles para resolver a questão do espaço em branco e fazer re-merge do arquivo com o pouco conhecido comando `git merge-file` que faz exatamente isso.

```
$ dos2unix hello.theirs.rb
dos2unix: converting file hello.theirs.rb to Unix format ...

$ git merge-file -p \
    hello.ours.rb hello.common.rb hello.theirs.rb > hello.rb

$ git diff -b
diff --cc hello.rb
index 36c06c8,e85207e..0000000
--- a/hello.rb
+++ b/hello.rb
@@@ -1,8 -1,7 +1,8 @@@
  #! /usr/bin/env ruby

 +# prints out a greeting
  def hello
-   puts 'hello world'
+   puts 'hello mundo'
  end

  hello()
```

Neste ponto, fizemos o merge do arquivo perfeitamente. Na verdade, isso funciona melhor do que a opção `ignore-space-change` porque isso realmente conserta as alterações de espaço em branco antes do merge, em vez de simplesmente ignorá-las. No merge com `ignore-space-change`, nós na verdade terminamos com algumas linhas com finais de linha do DOS, deixando as coisas misturadas.

Se você quiser ter uma ideia, antes de finalizar este commit, sobre o que foi realmente alterado entre um lado e outro, você pode pedir ao `git diff` para comparar o que está no seu diretório de trabalho que você está prestes a comitar como resultado do merge para qualquer um desses estágios. Vamos analisar todos eles.

Para comparar o seu resultado com o que você tinha na sua branch antes do merge, ou seja, para ver o que o merge introduziu, você pode rodar `git diff --ours`:

```
$ git diff --ours
* Unmerged path hello.rb
diff --git a/hello.rb b/hello.rb
index 36c06c8..44d0a25 100755
--- a/hello.rb
+++ b/hello.rb
@@ -2,7 +2,7 @@

 # prints out a greeting
 def hello
-  puts 'hello world'
+  puts 'hello mundo'
 end

 hello()
```

Então, aqui podemos ver facilmente que o que aconteceu na nossa branch, o que estamos realmente introduzindo neste arquivo com este merge, é a alteração daquela única linha.

Se quisermos ver como o resultado do merge diferiu do que estava no lado deles, você pode rodar `git diff --theirs`. Neste e no exemplo seguinte, nós temos que usar `-b` para retirar os espaços em branco porque estamos comparando com o que está no Git, não o nosso arquivo limpo `hello.theirs.rb`.

```
$ git diff --theirs -b
* Unmerged path hello.rb
diff --git a/hello.rb b/hello.rb
index e85207e..44d0a25 100755
--- a/hello.rb
+++ b/hello.rb
@@ -1,5 +1,6 @@
 #! /usr/bin/env ruby

+# prints out a greeting
 def hello
   puts 'hello mundo'
 end
```

Finalmente, você pode ver como o arquivo mudou a partir de ambos os lados com `git diff --base`.

```
$ git diff --base -b
* Unmerged path hello.rb
diff --git a/hello.rb b/hello.rb
index ac51efd..44d0a25 100755
--- a/hello.rb
+++ b/hello.rb
@@ -1,7 +1,8 @@
 #! /usr/bin/env ruby

+# prints out a greeting
 def hello
-  puts 'hello world'
+  puts 'hello mundo'
 end

 hello()
```

Neste ponto, podemos usar o comando `git clean` para limpar os arquivos extras que criamos para fazer o merge manual mas que não precisamos mais.

```
$ git clean -f
Removing hello.common.rb
Removing hello.ours.rb
Removing hello.theirs.rb
```

Fazendo Checkout de Conflitos

Talvez nós não estejamos felizes com a resolução neste ponto por algum motivo, ou talvez editar manualmente um ou ambos os lados ainda não tenha funcionado bem e precisamos de mais contexto.

Vamos mudar o exemplo um pouco. Para este exemplo, temos duas branches de vida mais longa que cada uma tem alguns commits nelas mas criam um conflito de conteúdo legítimo quando é feito o merge.

```
$ git log --graph --oneline --decorate --all
* f1270f7 (HEAD, master) Update README
* 9af9d3b Create README
* 694971d Update phrase to 'hola world'
| * e3eb223 (mundo) Add more tests
| * 7cff591 Create initial testing script
| * c3ffff1 Change text to 'hello mundo'
|/
* b7dcc89 Initial hello world code
```

Nós agora temos três commits únicos que vivem apenas na branch `master` e outros três que vivem na branch `mundo`. Se tentarmos fazer o merge da branch `mundo`, nós teremos um conflito.

```
$ git merge mundo
Auto-merging hello.rb
CONFLICT (content): Merge conflict in hello.rb
Automatic merge failed; fix conflicts and then commit the result.
```

Gostaríamos de ver qual é o conflito do merge. Se abrirmos o arquivo, veremos algo assim:

```
#! /usr/bin/env ruby

def hello
<<<<<<< HEAD
  puts 'hola world'
=======
  puts 'hello mundo'
>>>>>>> mundo
end

hello()
```

Ambos os lados do merge adicionaram conteúdo a este arquivo, mas alguns dos commits modificaram o arquivo no mesmo lugar que causou esse conflito.

Vamos explorar um par de ferramentas que você tem agora à sua disposição para determinar como este conflito ocorreu. Talvez não seja óbvio como exatamente você deveria consertar este conflito. Você precisa de mais contexto.

Uma ferramenta útil é o `git checkout` com a opção `--conflict`. Isso fará o re-checkout do arquivo novamente e substituirá os marcadores de conflito do merge. Isso pode ser útil se você quiser resetar os marcadores e tentar resolvê-los novamente.

Você pode passar para `--conflict` o valor `diff3` ou `merge` (que é o padrão). Se você passar `diff3`, o Git usará uma versão ligeiramente diferente de marcadores de conflito, não apenas dando a você as versões “ours” e “theirs”, mas também a versão “base” inline para lhe dar mais contexto.

```
$ git checkout --conflict=diff3 hello.rb
```

Uma vez que rodarmos isso, o arquivo ficará assim em vez disso:

```
#! /usr/bin/env ruby

def hello
<<<<<<< ours
  puts 'hola world'
||||||| base
  puts 'hello world'
=======
  puts 'hello mundo'
```

```
>>>>>>> theirs
end

hello()
```

Se você gostar desse formato, você pode defini-lo como padrão para conflitos de merge futuros, ajustando a configuração `merge.conflictstyle` para `diff3`.

```
$ git config --global merge.conflictstyle diff3
```

O comando `git checkout` também pode receber as opções `--ours` e `--theirs`, o que pode ser uma maneira muito rápida de simplesmente escolher um lado ou o outro sem fazer merge de nada.

Isso pode ser particularmente útil para conflitos de arquivos binários onde você pode simplesmente escolher um lado, ou onde você só quer fazer merge de certos arquivos de outra branch — você pode fazer o merge e então fazer checkout de certos arquivos de um lado ou do outro antes de comitar.

Log de Merge (Merge Log)

Outra ferramenta útil ao resolver conflitos de merge é o `git log`. Isso pode ajudá-lo a obter contexto sobre o que pode ter contribuído para os conflitos. Revisar um pouco do histórico para lembrar o porquê de duas linhas de desenvolvimento estarem mexendo na mesma área de código pode ser muito útil às vezes.

Para obter uma lista completa de todos os commits únicos que foram incluídos em qualquer uma das branches envolvidas neste merge, nós podemos usar a sintaxe de “três pontos” (triple dot) que aprendemos em Ponto Triplo (Triple Dot).

```
$ git log --oneline --left-right HEAD...MERGE_HEAD
< f1270f7 Update README
< 9af9d3b Create README
< 694971d Update phrase to 'hola world'
> e3eb223 Add more tests
> 7cff591 Create initial testing script
> c3ffff1 Change text to 'hello mundo'
```

Essa é uma boa lista dos seis commits totais envolvidos, bem como em qual linha de desenvolvimento cada commit estava.

No entanto, podemos simplificar ainda mais isso para nos dar um contexto muito mais específico. Se adicionarmos a opção `--merge` ao `git log`, ele mostrará apenas os commits de ambos os lados do merge que tocam em um arquivo que está atualmente em conflito.

```
$ git log --oneline --left-right --merge
< 694971d Update phrase to 'hola world'
```

```
> c3ffff1 Change text to 'hello mundo'
```

Se você rodar isso com a opção `-p` em vez disso, você obtém apenas os diffs do arquivo que acabou em conflito. Isso pode ser **muito** útil para dar rapidamente a você o contexto necessário para ajudar a entender o porquê algo entrou em conflito e como resolvê-lo de forma mais inteligente.

Formato Combinado de Diff (Combined Diff Format)

Como o Git faz stage de quaisquer resultados de merge que sejam bem-sucedidos, quando você roda `git diff` enquanto está em um estado de merge em conflito, você só obtém o que atualmente ainda está em conflito. Isso pode ser útil para ver o que você ainda tem que resolver.

Quando você roda `git diff` logo após um conflito de merge, ele lhe dará informações em um formato de saída de diff bastante único.

```
$ git diff
diff --cc hello.rb
index 0399cd5,59727f0..0000000
--- a/hello.rb
+++ b/hello.rb
@@@ -1,7 -1,7 +1,11 @@@
  #! /usr/bin/env ruby

  def hello
++<<<<<<< HEAD
 +  puts 'hola world'
++=======
+   puts 'hello mundo'
++>>>>>>> mundo
  end

  hello()
```

O formato é chamado “Combined Diff” (Diff Combinado) e lhe dá duas colunas de dados ao lado de cada linha. A primeira coluna mostra se aquela linha é diferente (adicionada ou removida) entre a branch “ours” e o arquivo no seu diretório de trabalho e a segunda coluna faz o mesmo entre a branch “theirs” e a sua cópia do diretório de trabalho.

Então, nesse exemplo você pode ver que as linhas `<<<<<<<` e `>>>>>>>` estão na cópia de trabalho, mas não estavam em nenhum dos lados do merge. Isso faz sentido porque a ferramenta de merge as colocou lá para nosso contexto, mas espera-se que nós as removamos.

Se resolvermos o conflito e rodarmos `git diff` novamente, veremos a mesma coisa, mas é um pouco mais útil.

```
$ vim hello.rb
$ git diff
diff --cc hello.rb
```

```
index 0399cd5,59727f0..0000000
--- a/hello.rb
+++ b/hello.rb
@@ -1,7 -1,7 +1,7 @@
  #! /usr/bin/env ruby

  def hello
-   puts 'hola world'
 -  puts 'hello mundo'
++  puts 'hola mundo'
  end

  hello()
```

Isso nos mostra que “hola world” estava no nosso lado mas não na cópia de trabalho, que “hello mundo” estava no lado deles mas não na cópia de trabalho e finalmente que “hola mundo” não estava em nenhum dos lados mas está agora na cópia de trabalho. Isso pode ser útil para revisar antes de comitar a resolução.

Você também pode obter isso a partir do `git log` para qualquer merge para ver como algo foi resolvido depois do fato. O Git enviará este formato se você rodar `git show` em um commit de merge, ou se você adicionar uma opção `--cc` a um `git log -p` (que por padrão só mostra patches para commits que não são de merge).

```
$ git log --cc -p -1
commit 14f41939956d80b9e17bb8721354c33f8d5b5a79
Merge: f1270f7 e3eb223
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri Sep 19 18:14:49 2014 +0200

    Merge branch 'mundo'

    Conflicts:
        hello.rb

diff --cc hello.rb
index 0399cd5,59727f0..e1d0799
--- a/hello.rb
+++ b/hello.rb
@@ -1,7 -1,7 +1,7 @@
  #! /usr/bin/env ruby

  def hello
-   puts 'hola world'
 -  puts 'hello mundo'
++  puts 'hola mundo'
  end

  hello()
```

Desfazendo Merges

Agora que você sabe como criar um commit de merge, provavelmente você fará alguns por engano. Uma das grandes coisas sobre trabalhar com o Git é que não há problema em cometer erros, porque é possível (e em muitos casos fácil) corrigi-los.

Commits de merge não são diferentes. Digamos que você começou a trabalhar em uma branch de tópico, acidentalmente fez o merge dela na `master`, e agora o seu histórico de commits está assim:

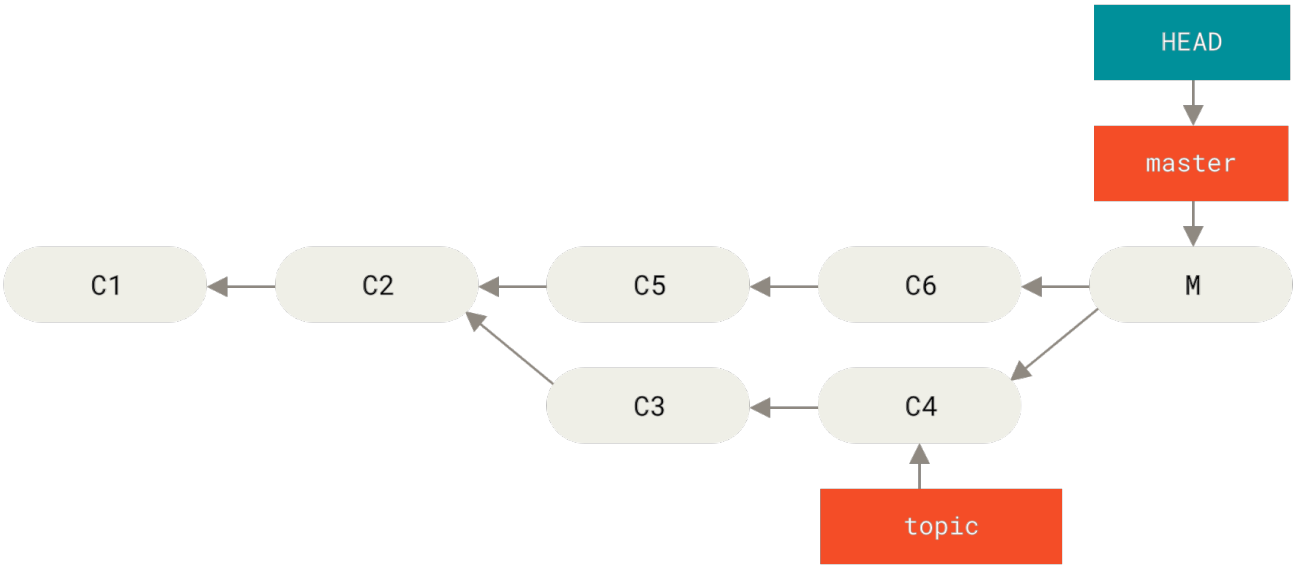


Figure 155. Commit de merge acidental

Existem duas maneiras de abordar este problema, dependendo de qual é o seu resultado desejado.

Consertar as referências

Se o commit de merge indesejado existe apenas no seu repositório local, a solução mais fácil e melhor é mover as branches para que elas apontem para onde você quer que elas apontem. Na maioria dos casos, se você seguir o `git merge` errôneo com `git reset --hard HEAD~`, isso resetará os ponteiros da branch para que eles fiquem assim:

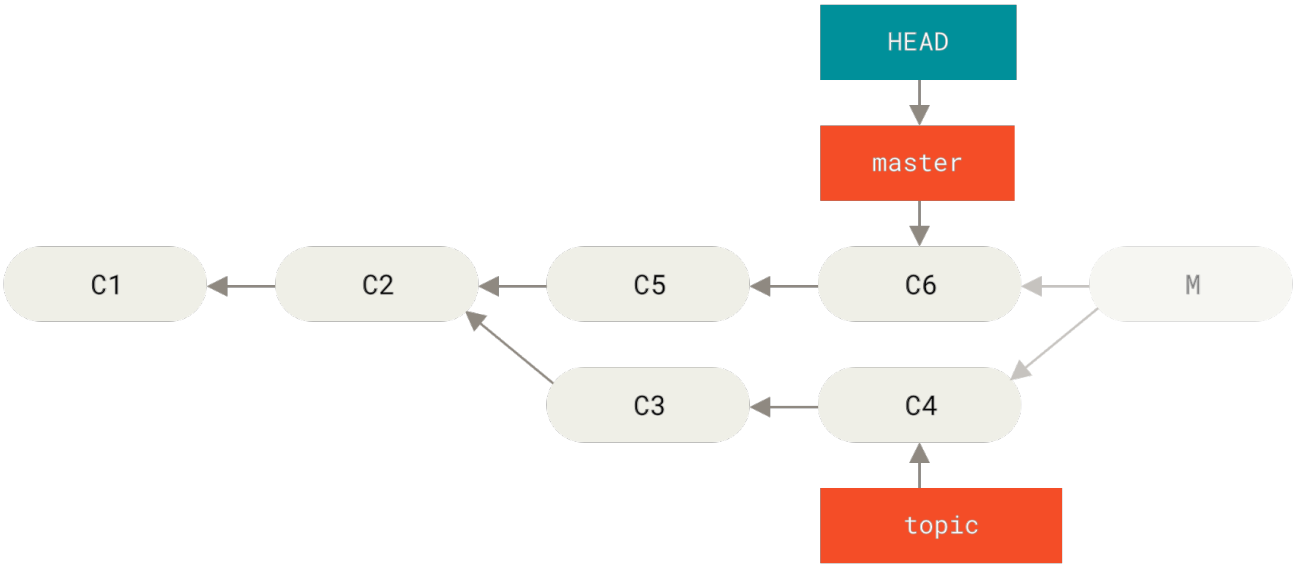


Figure 156. Histórico após `git reset --hard HEAD~`

Nós cobrimos o `reset` anteriormente em Reset Desmistificado, então não deve ser muito difícil descobrir o que está acontecendo aqui. Aqui está uma rápida recapitulação: `reset --hard` geralmente passa por três passos:

1. Move a branch para a qual o HEAD aponta. Neste caso, queremos mover a `master` para onde ela estava antes do commit de merge (`C6`).
2. Faz o index ficar igual ao HEAD.
3. Faz o diretório de trabalho ficar igual ao index.

A desvantagem dessa abordagem é que ela está reescrevendo o histórico, o que pode ser problemático com um repositório compartilhado. Dê uma olhada em Os Perigos do Rebase (Perils of Rebasing) para mais informações sobre o que pode acontecer; a versão curta é que se outras pessoas têm os commits que você está reescrevendo, você provavelmente deve evitar o `reset`. Esta abordagem também não funcionará se quaisquer outros commits foram criados desde o merge; mover as referências efetivamente perderia essas alterações.

Reverter o commit

Se mover os ponteiros das branches não vai funcionar para você, o Git dá a opção de fazer um novo commit que desfaz todas as alterações de um existente. O Git chama esta operação de “revert” (reverter), e neste cenário em particular, você a invocaria assim:

```
$ git revert -m 1 HEAD
[master b1d8379] Revert "Merge branch 'topic'"
```

A flag `-m 1` indica qual pai é a linha principal (“mainline”) e deve ser mantido. Quando você invoca um merge no `HEAD` (`git merge topic`), o novo commit tem dois pais: o primeiro é o `HEAD` (`C6`), e o segundo é a ponta (tip) da branch sendo mesclada (`C4`). Neste caso, queremos desfazer todas as alterações introduzidas ao mesclar o pai #2 (`C4`), enquanto mantemos todo o conteúdo do pai #1 (`C6`).

O histórico com o commit revertido fica assim:

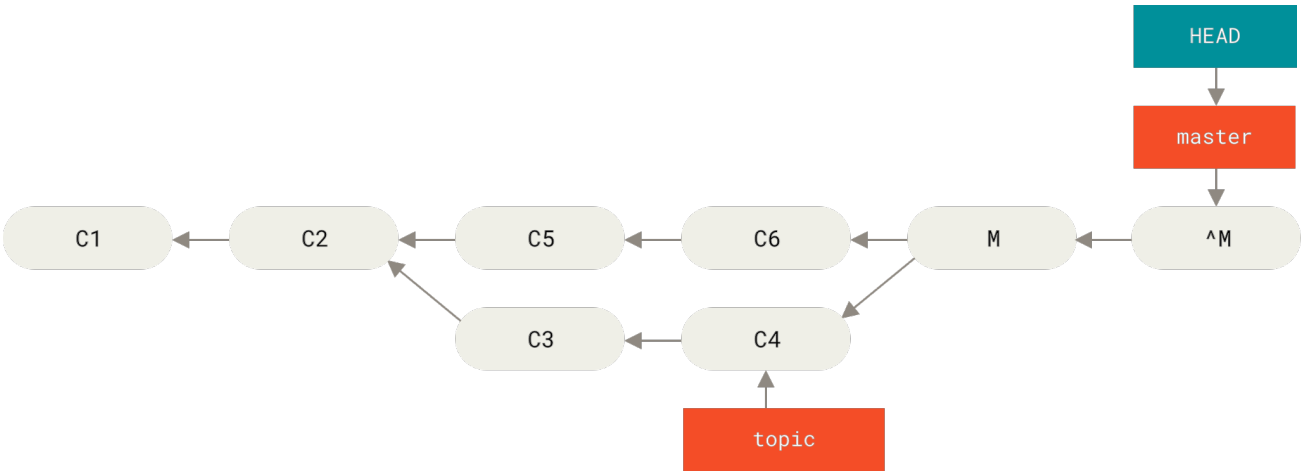


Figure 157. Histórico após `git revert -m 1`

O novo commit `^M` tem exatamente o mesmo conteúdo que `C6`, então começando daqui é como se o merge nunca tivesse acontecido, exceto que os commits agora não mesclados ainda estão no histórico do `HEAD`. O Git ficará confuso se você tentar fazer o merge de `topic` em `master` novamente:

```
$ git merge topic
Already up-to-date.
```

Não há nada em `topic` que já não seja alcançável a partir de `master`. O que é pior, se você adicionar trabalho ao `topic` e fizer merge novamente, o Git apenas trará as alterações *desde* o merge revertido:

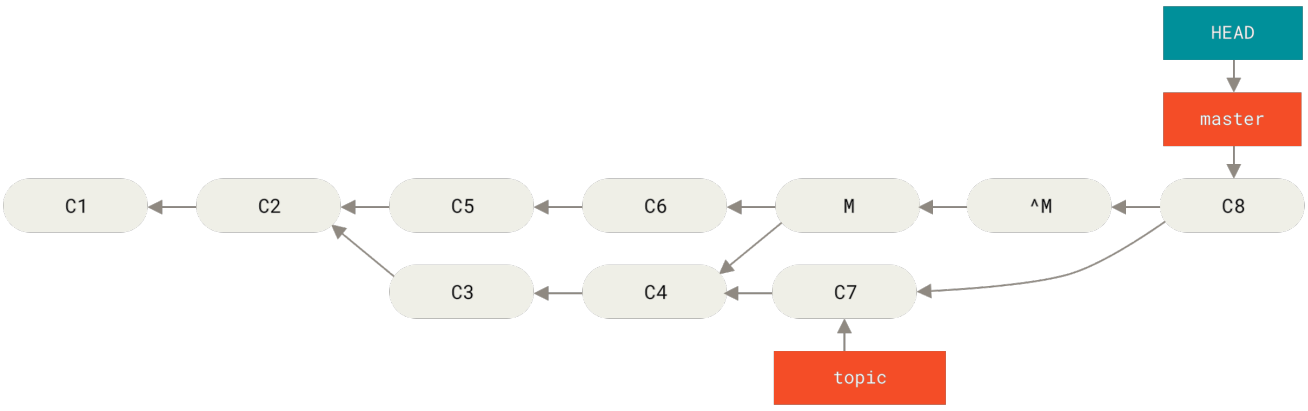


Figure 158. Histórico com um merge ruim

A melhor maneira de contornar isso é desfazer o reverso (un-revert) do merge original, já que agora você quer trazer as alterações que foram revertidas, **depois** criar um novo commit de merge:

```
$ git revert ^M
[master 09f0126] Revert "Revert "Merge branch 'topic'""
$ git merge topic
```

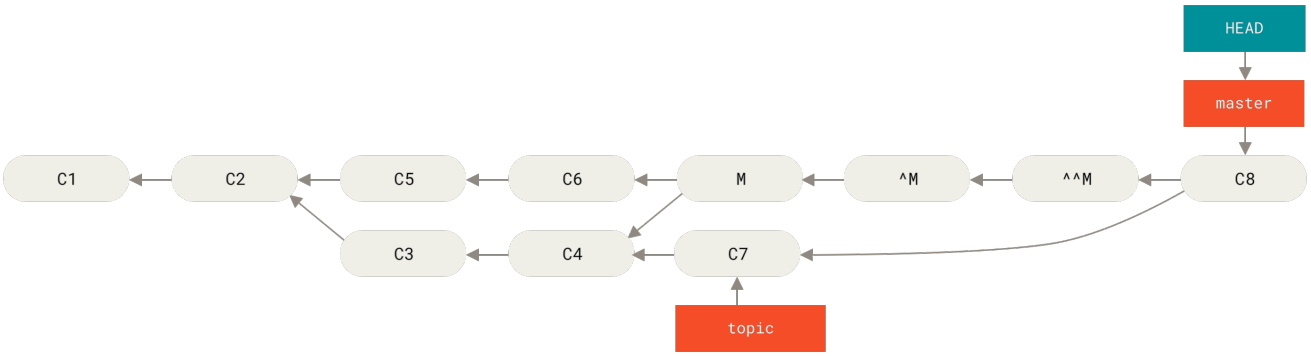


Figure 159. Histórico após re-merging de um merge revertido

Neste exemplo, `M` e `^M` se cancelam. `^^M` efetivamente faz o merge das alterações de `C3` e `C4`, e `C8` faz o merge das alterações de `C7`, então agora `topic` está totalmente mesclado.

Outros Tipos de Merges

Até agora cobrimos o merge normal de duas branches, normalmente lidado com o que é chamado de estratégia “recursive” (recursiva) de merge. No entanto, existem outras maneiras de fazer merge de branches. Vamos cobrir algumas delas rapidamente.

Preferência por Our ou Theirs

Antes de tudo, há outra coisa útil que podemos fazer com o modo “recursive” normal de merge. Nós já vimos as opções `ignore-all-space` e `ignore-space-change` que são passadas com um `-X`, mas também podemos dizer ao Git para favorecer um lado ou outro quando ele vê um conflito.

Por padrão, quando o Git vê um conflito entre duas branches sofrendo merge, ele adicionará marcadores de conflito de merge ao seu código e marcará o arquivo como em conflito e deixará que você o resolva. Se você preferir que o Git simplesmente escolha um lado específico e ignore o outro lado em vez de permitir que você resolva manualmente o conflito, você pode passar ao comando `merge` um `-Xours` ou `-Xtheirs`.

Se o Git vir isso, ele não adicionará marcadores de conflito. Quaisquer diferenças que sejam mescláveis (mergeable), ele fará o merge. Quaisquer diferenças que conflitem, ele simplesmente escolherá o lado que você especificar por inteiro, incluindo arquivos binários.

Se voltarmos ao exemplo do “hello world” que estávamos usando antes, podemos ver que fazer o merge da nossa branch causa conflitos.

```
$ git merge mundo
Auto-merging hello.rb
CONFLICT (content): Merge conflict in hello.rb
Resolved 'hello.rb' using previous resolution.
Automatic merge failed; fix conflicts and then commit the result.
```

No entanto, se rodarmos isso com `-Xours` ou `-Xtheirs`, ele não causa.

```
$ git merge -Xours mundo
```

```
Auto-merging hello.rb
Merge made by the 'recursive' strategy.
 hello.rb | 2 +-
 test.sh  | 2 ++
 2 files changed, 3 insertions(+), 1 deletion(-)
 create mode 100644 test.sh
```

Nesse caso, em vez de obter marcadores de conflito no arquivo com “hello mundo” de um lado e “hola world” do outro, ele simplesmente escolherá “hola world”. No entanto, todas as outras alterações não conflitantes naquela branch são mescladas com sucesso.

Esta opção também pode ser passada para o comando `git merge-file` que vimos anteriormente rodando algo como `git merge-file --ours` para merges de arquivos individuais.

Se você quer fazer algo parecido mas nem que o Git tente fazer o merge de alterações do outro lado, há uma opção mais draconiana, que é a *estratégia* de merge “ours”. Isso é diferente da *opção* recursiva de merge “ours”.

Isso basicamente fará um merge falso. Ele gravará um novo commit de merge com ambas as branches como pais, mas nem olhará para a branch que você está fazendo merge. Ele simplesmente gravará como o resultado do merge o código exato da sua branch atual.

```
$ git merge -s ours mundo
Merge made by the 'ours' strategy.
$ git diff HEAD HEAD~
$
```

Você pode ver que não há diferença entre a branch em que estávamos e o resultado do merge.

Isso muitas vezes pode ser útil para basicamente enganar o Git a pensar que uma branch já foi mesclada ao fazer um merge mais tarde. Por exemplo, digamos que você ramificou de uma branch `release` e fez algum trabalho nela que você vai querer fazer o merge de volta na sua branch `master` em algum momento. Enquanto isso, algum bugfix na `master` precisa ser feito backport para a sua branch `release`. Você pode fazer o merge da branch do bugfix na branch `release` e também `merge -s ours` na mesma branch para a sua branch `master` (embora a correção já esteja lá) de modo que quando mais tarde você fizer o merge da branch `release` de novo, não haja conflitos do bugfix.

Merging de Sub-árvore (Subtree Merging)

A ideia do merge de sub-árvore (subtree merge) é que você tem dois projetos, e um dos projetos mapeia para um subdiretório do outro. Quando você especifica um merge de sub-árvore, o Git muitas vezes é esperto o suficiente para descobrir que um é uma sub-árvore do outro e faz o merge apropriadamente.

Nós veremos um exemplo de adição de um projeto separado em um projeto existente e então o merge do código do segundo em um subdiretório do primeiro.

Primeiro, adicionaremos a aplicação Rack ao nosso projeto. Adicionaremos o projeto Rack como uma referência remota em nosso próprio projeto e então faremos o checkout dele em sua própria

branch:

```
$ git remote add rack_remote https://github.com/rack/rack
$ git fetch rack_remote --no-tags
warning: no common commits
remote: Counting objects: 3184, done.
remote: Compressing objects: 100% (1465/1465), done.
remote: Total 3184 (delta 1952), reused 2770 (delta 1675)
Receiving objects: 100% (3184/3184), 677.42 KiB | 4 KiB/s, done.
Resolving deltas: 100% (1952/1952), done.
From https://github.com/rack/rack
 * [new branch]      build      -> rack_remote/build
 * [new branch]      master     -> rack_remote/master
 * [new branch]      rack-0.4   -> rack_remote/rack-0.4
 * [new branch]      rack-0.9   -> rack_remote/rack-0.9
$ git checkout -b rack_branch rack_remote/master
Branch rack_branch set up to track remote branch refs/remotes/rack_remote/master.
Switched to a new branch "rack_branch"
```

Agora nós temos a raiz do projeto Rack na nossa branch `rack_branch` e nosso próprio projeto na branch `master`. Se você fizer checkout de uma e depois da outra, você pode ver que eles têm raízes de projeto diferentes:

```
$ ls
AUTHORS         KNOWN-ISSUES   Rakefile      contrib         lib
COPYING         README         bin           example         test
$ git checkout master
Switched to branch "master"
$ ls
README
```

Este é um conceito um tanto estranho. Nem todas as branches no seu repositório têm que ser branches do mesmo projeto na verdade. Isso não é comum, porque raramente é útil, mas é bem fácil ter branches que contenham históricos completamente diferentes.

Neste caso, nós queremos puxar (pull) o projeto Rack para o nosso projeto `master` como um subdiretório. Nós podemos fazer isso no Git com `git read-tree`. Você vai aprender mais sobre o `read-tree` e seus amigos em Git Internals (Por Dentro do Git), mas por agora saiba que ele lê a árvore (tree) raiz de uma branch para a sua área de stage atual e diretório de trabalho. Nós acabamos de mudar de volta para a nossa branch `master`, e nós puxamos (pull) a branch `rack_branch` para o subdiretório `rack` da nossa branch `master` do nosso projeto principal:

```
$ git read-tree --prefix=rack/ -u rack_branch
```

Quando nós comitamos, parece que nós temos todos os arquivos Rack sob aquele subdiretório — como se nós os tivéssemos copiado de um tarball. O que fica interessante é que nós podemos fazer merge de alterações de uma branch para a outra de forma bem fácil. Então, se o

projeto Rack atualiza, nós podemos puxar alterações do upstream mudando para aquela branch e dando pull:

```
$ git checkout rack_branch
$ git pull
```

Então, podemos fazer o merge dessas alterações de volta para a nossa branch `master`. Para puxar as alterações e pré-popular a mensagem de commit, use a opção `--squash`, bem como a opção `-Xsubtree` da estratégia de merge recursiva. A estratégia recursiva é o padrão aqui, mas a incluímos para maior clareza.

```
$ git checkout master
$ git merge --squash -s recursive -Xsubtree=rack rack_branch
Squash commit -- not updating HEAD
Automatic merge went well; stopped before committing as requested
```

Todas as alterações do projeto Rack são mescladas (merged in) e prontas para serem comitadas localmente. Você também pode fazer o oposto — fazer alterações no subdiretório `rack` da sua branch `master` e então fazer merge delas na sua branch `rack_branch` mais tarde para enviá-las aos mantenedores ou fazer o push delas para o upstream.

Isso nos dá uma forma de ter um fluxo de trabalho (workflow) um tanto similar ao fluxo de submodules sem usar submodules (que cobriremos em Submódulos). Nós podemos manter branches com outros projetos relacionados no nosso repositório e fazer o merge de sub-árvore deles no nosso projeto ocasionalmente. É bacana de algumas formas, por exemplo todo o código é comitado em um único lugar. No entanto, tem outras desvantagens em que é um pouco mais complexo e mais fácil cometer erros ao reintegrar as alterações ou acidentalmente dar push em uma branch em um repositório não relacionado.

Outra coisa um pouco estranha é que para obter um diff entre o que você tem no seu subdiretório `rack` e o código na sua branch `rack_branch` — para ver se você precisa fazer o merge deles — você não pode usar o comando `diff` normal. Em vez disso, você deve rodar `git diff-tree` com a branch com a qual você deseja comparar:

```
$ git diff-tree -p rack_branch
```

Ou, para comparar o que está no seu subdiretório `rack` com o que a branch `master` no servidor era da última vez que você deu fetch, você pode rodar:

```
$ git diff-tree -p rack_remote/master
```

Rerere

A funcionalidade `git rerere` é um pouco uma funcionalidade oculta. O nome significa “reutilizar a

resolução gravada" (reuse recorded resolution) e, como o nome indica, ele permite que você peça ao Git para lembrar como você resolveu um conflito de hunk para que da próxima vez que ele vir o mesmo conflito, o Git possa resolvê-lo para você automaticamente.

Há uma série de cenários em que esta funcionalidade pode ser muito útil. Um dos exemplos mencionados na documentação é quando você deseja garantir que uma branch de tópico de longa duração, no final das contas, faça o merge perfeitamente, mas você não quer ter um monte de commits de merge intermediários sobrecarregando seu histórico de commits. Com o `rerere` habilitado, você pode tentar o merge ocasional, resolver os conflitos, depois recuar o merge. Se você fizer isso de forma contínua, o merge final deve ser fácil porque o `rerere` pode simplesmente fazer tudo para você automaticamente.

Esta mesma tática pode ser usada se você quiser manter uma branch rebased, assim não terá que lidar com os mesmos conflitos de rebase a cada vez que o fizer. Ou se você quiser pegar uma branch que você fez o merge e consertou um monte de conflitos e então decidir fazer rebase em vez disso — você provavelmente não terá que fazer todos os mesmos conflitos novamente.

Outra aplicação de `rerere` é quando você faz o merge ocasional de um bando de branches de tópicos em evolução, para formar uma head de testes, como o próprio projeto Git faz frequentemente. Se os testes falharem, você pode rebobinar os merges e refazê-los sem a branch de tópico que fez os testes falharem, sem precisar re-resolver os conflitos novamente.

Para habilitar a funcionalidade `rerere`, você simplesmente tem que rodar essa configuração:

```
$ git config --global rerere.enabled true
```

Você também pode ativá-lo criando o diretório `.git/rr-cache` em um repositório específico, mas a configuração (config) é mais clara e permite ativar este recurso globalmente para você.

Agora, vamos ver um simples exemplo, semelhante ao nosso anterior. Digamos que temos um arquivo chamado `hello.rb` parecido com isso:

```
#! /usr/bin/env ruby

def hello
  puts 'hello world'
end
```

Em uma branch alteramos a palavra "hello" para "hola", e na outra alteramos "world" para "mundo", como antes.

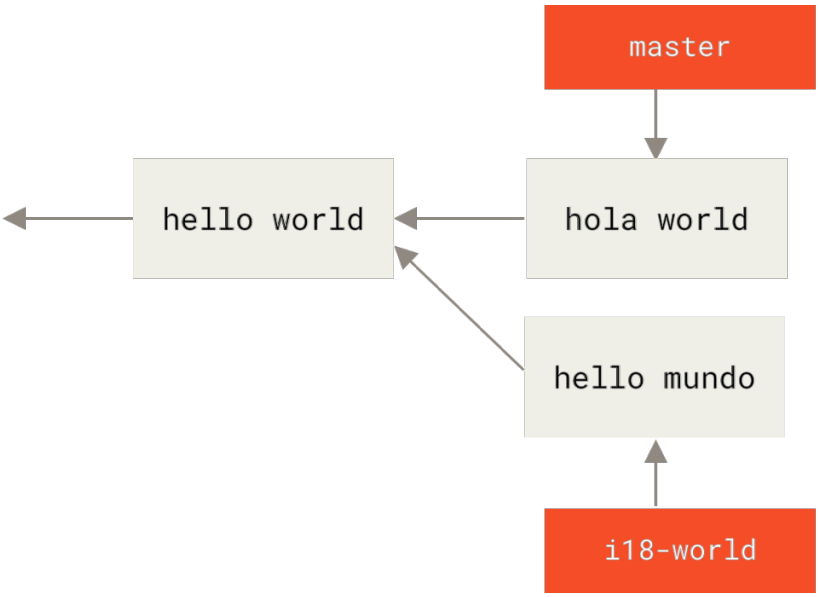


Figure 160. Duas branches mudando diferentemente a mesma parte do mesmo arquivo

Quando nós fizermos o merge em ambas branches, obteremos um conflito de merge:

```
$ git merge i18n-world
Auto-merging hello.rb
CONFLICT (content): Merge conflict in hello.rb
Recorded preimage for 'hello.rb'
Automatic merge failed; fix conflicts and then commit the result.
```

Você deve ter notado a nova linha `Recorded preimage for FILE` aí. Sem isso, ela se pareceria exatamente com um normal conflito de merge. Neste ponto, o `rerere` pode nos dizer algumas coisas. Normalmente, você pode executar `git status` neste momento para ver quais deram conflito:

```
$ git status
# On branch master
# Unmerged paths:
#   (use "git reset HEAD <file>..." to unstage)
#   (use "git add <file>..." to mark resolution)
#
#   both modified:      hello.rb
#
```

Entretanto, `git rerere` vai te dizer para qual estado o pre-merge foi gravado, por meio de `git rerere status`:

```
$ git rerere status
hello.rb
```

E o `git rerere diff` irá exibir a situação atual da resolução — o que você começou para resolver e o

que você solucionou.

```
$ git rerere diff
--- a/hello.rb
+++ b/hello.rb
@@ -1,11 +1,11 @@
 #! /usr/bin/env ruby

 def hello
-<<<<<<<
-  puts 'hello mundo'
-=======
+<<<<<<< HEAD
   puts 'hola world'
->>>>>>>
+=======
+  puts 'hello mundo'
+>>>>>>> i18n-world
 end
```

Também (e isto não está realmente relacionado com o `rerere`), você pode usar o `git ls-files -u` para ver os arquivos em conflito e as versões anterior, esquerda e direita:

```
$ git ls-files -u
100644 39804c942a9c1f2c03dc7c5ebcd7f3e3a6b97519 1   hello.rb
100644 a440db6e8d1fd76ad438a49025a9ad9ce746f581 2   hello.rb
100644 54336ba847c3758ab604876419607e9443848474 3   hello.rb
```

Agora você pode resolve-lo para ser simplesmente `puts 'hola mundo'` e executar o comando `git rerere diff` novamente, para ver o que o rerere vai se lembrar:

```
$ git rerere diff
--- a/hello.rb
+++ b/hello.rb
@@ -1,11 +1,7 @@
 #! /usr/bin/env ruby

 def hello
-<<<<<<<
-  puts 'hello mundo'
-=======
-  puts 'hola world'
->>>>>>>
+  puts 'hola mundo'
 end
```

Sendo assim, bascicamente ele diz, quando o Git enxergar um conflito de hunk em um arquivo

`hello.rb` que contenha “hello mundo” de um lado e “hola world” do outro, ele o resolverá como “hola mundo”.

Agora podemos marcá-lo como resolvido e realizar o commit dele:

```
$ git add hello.rb
$ git commit
Recorded resolution for 'hello.rb'.
[master 68e16e5] Merge branch 'i18n'
```

Você pode ver que ele gravou a resolução para o ARQUIVO ("Recorded resolution for FILE").

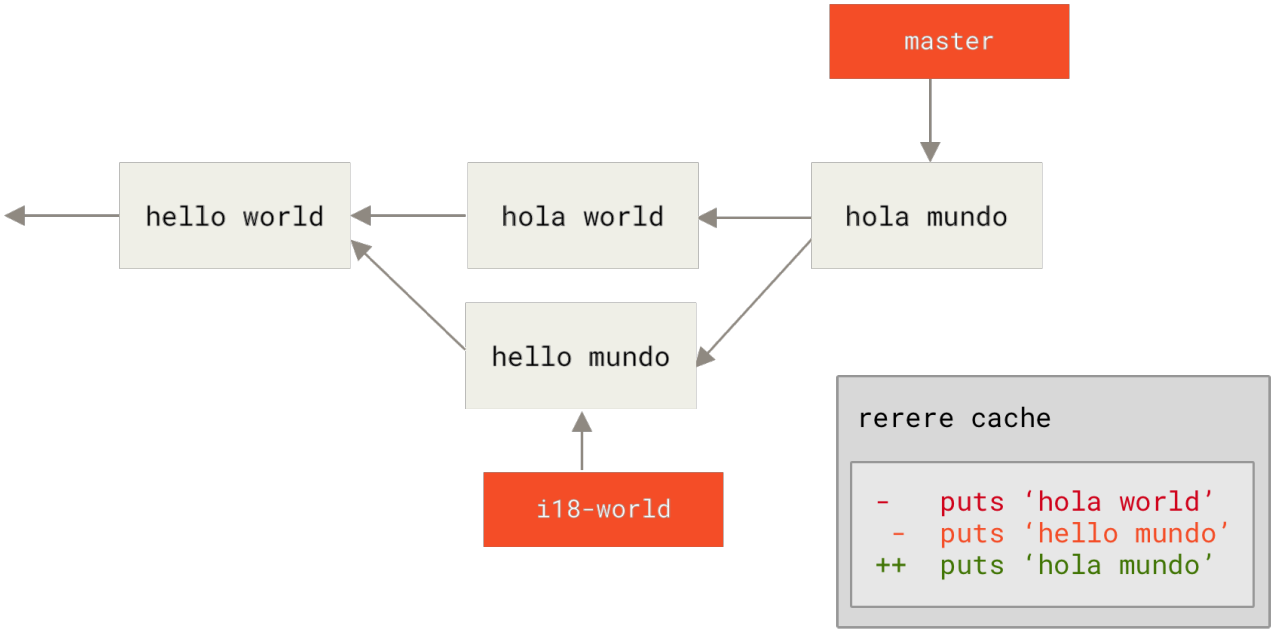


Figure 161. Resolução gravada para o ARQUIVO (Recorded resolution for FILE)

Agora, vamos desfazer aquele merge e em seguida dar rebase em nossa branch `master`. Para isso, devemos recuar nossa branch por meio de `git reset` conforme vimos em Reset Desmistificado.

```
$ git reset --hard HEAD^
HEAD is now at ad63f15 i18n the hello
```

Nosso merge está desfeito. Agora vamos fazer o rebase da branch de tópico.

```
$ git checkout i18n-world
Switched to branch 'i18n-world'

$ git rebase master
First, rewinding head to replay your work on top of it...
Applying: i18n one word
Using index info to reconstruct a base tree...
```

```
Falling back to patching base and 3-way merge...
Auto-merging hello.rb
CONFLICT (content): Merge conflict in hello.rb
Resolved 'hello.rb' using previous resolution.
Failed to merge in the changes.
Patch failed at 0001 i18n one word
```

Pronto, nós tivemos o mesmo conflito de merge conforme esperávamos, mas dê uma olhada na linha `Resolved FILE using previous resolution`. Ao olhar o arquivo, nós veremos que ele já foi resolvido, pois não existem marcadores de conflitos nele.

```
#! /usr/bin/env ruby

def hello
  puts 'hola mundo'
end
```

Além disso, `git diff` mostrará a você como ele foi automaticamente resolvido de novo (re-resolved):

```
$ git diff
diff --cc hello.rb
index a440db6,54336ba..0000000
--- a/hello.rb
+++ b/hello.rb
@@@ -1,7 -1,7 +1,7 @@@
  #! /usr/bin/env ruby

  def hello
-   puts 'hola world'
 -  puts 'hello mundo'
++  puts 'hola mundo'
  end
```

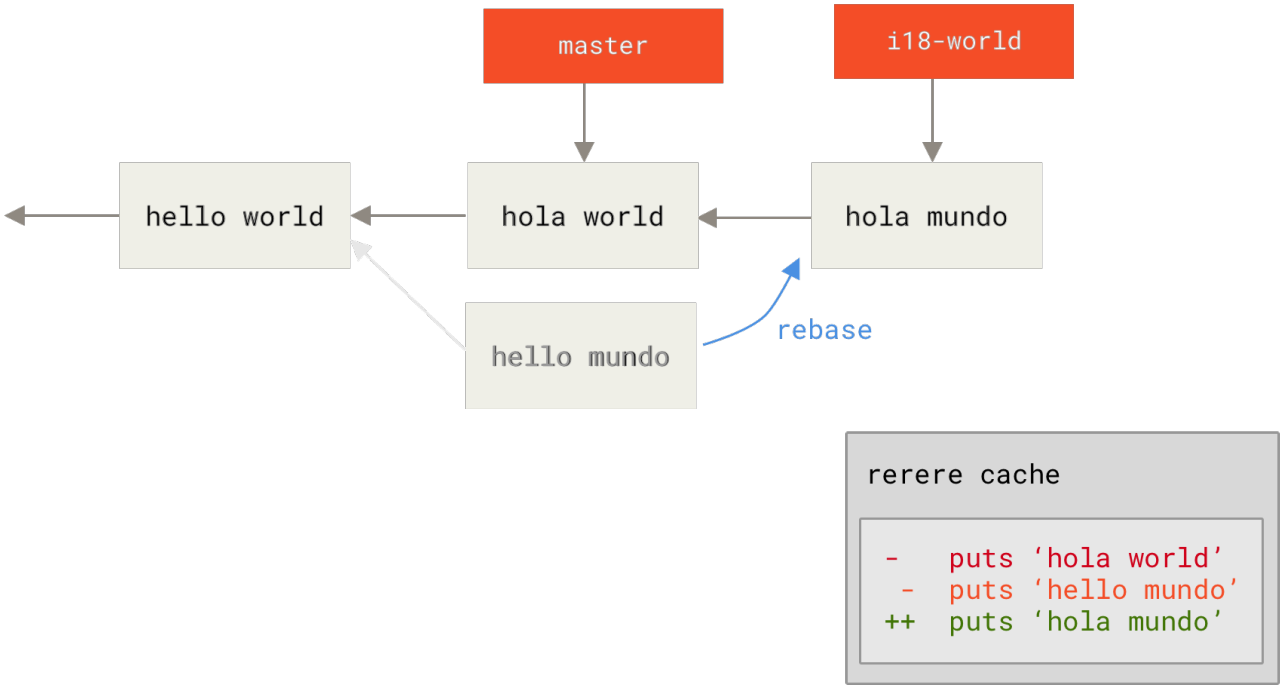


Figure 162. Conflito de merge resolvido automaticamente usando a resolução anterior

Nós podemos recriar o arquivo num estado de conflito com o `git checkout`:

```
$ git checkout --conflict=merge hello.rb
$ cat hello.rb
#! /usr/bin/env ruby

def hello
<<<<<<< ours
  puts 'hola world'
=======
  puts 'hello mundo'
>>>>>>> theirs
end
```

Nós já vimos um exemplo disto em Merging Avançado. Mas agora, vamos re-resolvê-lo executando novamente o `git rerere`:

```
$ git rerere
Resolved 'hello.rb' using previous resolution.
$ cat hello.rb
#! /usr/bin/env ruby

def hello
  puts 'hola mundo'
end
```

Nós re-resolvemos o arquivo automaticamente com a ajuda do armazenamento cache de soluções

do `rerere`. Agora você pode adicionar e continuar o rebase para completá-lo.

```
$ git add hello.rb
$ git rebase --continue
Applying: i18n one word
```

Portanto, se você fizer vários re-merges, ou quer manter as suas branch de tópicos atualizadas com o seu branch principal sem um monte de merges, ou você faz rebase com frequência, você pode ligar o `rerere` para fazer com que a sua vida fique um pouco mais fácil.

Depurando com Git

Além de ser principalmente para controle de versão, o Git também fornece alguns comandos para ajudá-lo a depurar seus projetos de código-fonte. Como o Git foi projetado para lidar com quase qualquer tipo de conteúdo, essas ferramentas são bastante genéricas, mas muitas vezes podem ajudá-lo a caçar um bug ou culpado quando as coisas dão errado.

Anotação de Arquivo

Se você rastrear um bug no seu código e quiser saber quando ele foi introduzido e por que, a anotação de arquivo (file annotation) é frequentemente sua melhor ferramenta. Ela mostra qual commit foi o último a modificar cada linha de qualquer arquivo. Então, se você ver que um método no seu código está com bugs, você pode anotar o arquivo com `git blame` para determinar qual commit foi responsável pela introdução daquela linha.

O exemplo seguinte usa `git blame` para determinar qual commit e committer foi responsável pelas linhas no `Makefile` de nível superior do kernel do Linux e, além disso, usa a opção `-L` para restringir a saída da anotação às linhas 69 a 82 daquele arquivo:

```
$ git blame -L 69,82 Makefile
b8b0618cf6fab (Cheng Renquan  2009-05-26 16:03:07 +0800 69) ifeq ("$(origin V)",
"command line")
b8b0618cf6fab (Cheng Renquan  2009-05-26 16:03:07 +0800 70)   KBUILD_VERBOSE = $(V)
^1da177e4c3f4 (Linus Torvalds 2005-04-16 15:20:36 -0700 71) endif
^1da177e4c3f4 (Linus Torvalds 2005-04-16 15:20:36 -0700 72) ifndef KBUILD_VERBOSE
^1da177e4c3f4 (Linus Torvalds 2005-04-16 15:20:36 -0700 73)   KBUILD_VERBOSE = 0
^1da177e4c3f4 (Linus Torvalds 2005-04-16 15:20:36 -0700 74) endif
^1da177e4c3f4 (Linus Torvalds 2005-04-16 15:20:36 -0700 75)
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 76) ifeq ($(KBUILD_VERBOSE),1)
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 77)   quiet =
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 78)   Q =
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 79) else
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 80)   quiet=quiet_
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 81)   Q = @
066b7ed955808 (Michal Marek   2014-07-04 14:29:30 +0200 82) endif
```

Note que o primeiro campo é o SHA-1 parcial do commit que modificou aquela linha por último. Os

próximos dois campos são valores extraídos daquele commit — o nome do autor e a data de autoria daquele commit — para que você possa ver facilmente quem modificou aquela linha e quando. Depois disso vêm o número da linha e o conteúdo do arquivo. Note também as linhas de commit `^1da177e4c3f4`, onde o prefixo `^` designa linhas que foram introduzidas no commit inicial do repositório e permaneceram inalteradas desde então. Isso é um pouco confuso, porque agora você já viu pelo menos três maneiras diferentes que o Git usa o `^` para modificar um SHA-1 de commit, mas é isso o que ele significa aqui.

Outra coisa legal sobre o Git é que ele não rastreia renomeações de arquivos explicitamente. Ele grava os snapshots e depois tenta descobrir o que foi renomeado implicitamente, após o fato. Uma das características interessantes disso é que você também pode pedir a ele para descobrir todos os tipos de movimento de código. Se você passar `-C` para o `git blame`, o Git analisa o arquivo que você está anotando e tenta descobrir de onde os trechos de código nele originalmente vieram, caso tenham sido copiados de outro lugar. Por exemplo, digamos que você está refatorando um arquivo chamado `GITServerHandler.m` em vários arquivos, um dos quais é `GITPackUpload.m`. Anotando o `GITPackUpload.m` com a opção `-C`, você pode ver de onde originalmente vieram seções do código:

```
$ git blame -C -L 141,153 GITPackUpload.m
f344f58d GITServerHandler.m (Scott 2009-01-04 141)
f344f58d GITServerHandler.m (Scott 2009-01-04 142) - (void) gatherObjectShasFromC
f344f58d GITServerHandler.m (Scott 2009-01-04 143) {
70befddd GITServerHandler.m (Scott 2009-03-22 144)         //NSLog(@"GATHER COMMI
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 145)
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 146)         NSString *parentSha;
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 147)         GITCommit *commit = [g
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 148)
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 149)         //NSLog(@"GATHER COMMI
ad11ac80 GITPackUpload.m    (Scott 2009-03-24 150)
56ef2caf GITServerHandler.m (Scott 2009-01-05 151)         if(commit) {
56ef2caf GITServerHandler.m (Scott 2009-01-05 152)                 [refDict setOb
56ef2caf GITServerHandler.m (Scott 2009-01-05 153)
```

Isso é realmente útil. Normalmente, você obtém como o commit original o commit onde você copiou o código para lá, porque essa é a primeira vez que você tocou nessas linhas neste arquivo. O Git diz qual foi o commit original em que você escreveu essas linhas, mesmo que isso tenha sido em outro arquivo.

Pesquisa Binária (Binary Search)

Anotar um arquivo ajuda se você sabe, de início, onde o problema está. Se você não sabe o que está quebrando, e houveram dezenas ou centenas de commits desde o último estado em que você sabe que o código funcionou, você provavelmente vai recorrer ao `git bisect` por ajuda. O comando `bisect` faz uma pesquisa binária por todo o seu histórico de commits para ajudá-lo a identificar, o mais rápido possível, qual commit introduziu um problema.

Digamos que você acabou de dar push de uma release do seu código para um ambiente de produção, e você está recebendo relatos de bugs sobre algo que não estava acontecendo no seu ambiente de desenvolvimento, e você não consegue imaginar por que o código está fazendo isso.

Você volta para o seu código e descobre que consegue reproduzir o problema, mas não consegue descobrir o que está dando errado. Você pode fazer um *bisect* do código para descobrir. Primeiro você roda `git bisect start` para dar o pontapé inicial, e depois você usa `git bisect bad` para dizer ao sistema que o commit atual no qual você está se encontra quebrado. Então, você tem que informar ao bisect qual foi o último estado sabidamente bom, usando `git bisect good <commit_bom>`:

```
$ git bisect start
$ git bisect bad
$ git bisect good v1.0
Bisecting: 6 revisions left to test after this
[ecb6e1bc347ccecc5f9350d878ce677feb13d3b2] Error handling on repo
```

O Git descobriu que houve cerca de 12 commits entre o commit que você marcou como o último commit bom (v1.0) e a atual versão ruim, e fez checkout daquele bem no meio para você. A esta altura, você pode rodar o seu teste para ver se o problema existe na época deste commit. Se ele existir, então ele foi introduzido em algum momento antes deste commit intermediário; se ele não existir, então o problema foi introduzido em algum momento após o commit intermediário. Vê-se que não há nenhum problema aqui, e você conta isso para o Git digitando `git bisect good` e prossegue na sua jornada:

```
$ git bisect good
Bisecting: 3 revisions left to test after this
[b047b02ea83310a70fd603dc8cd7a6cd13d15c04] Secure this thing
```

Agora você está em um outro commit, bem no meio do caminho entre o que você testou por último e o seu commit ruim. Você roda o teste novamente e constata que este commit está quebrado, então você informa isto ao Git usando `git bisect bad`:

```
$ git bisect bad
Bisecting: 1 revisions left to test after this
[f71ce38690acf49c1f3c9bea38e09d82a5ce6014] Drop exceptions table
```

Este commit está bom, e agora o Git tem toda a informação que ele precisa para determinar onde o problema foi introduzido. Ele te dá o SHA-1 do primeiro commit ruim (bad commit) e mostra algumas das informações do commit e quais arquivos foram modificados nesse commit, para que você possa descobrir o que aconteceu que possivelmente causou este bug:

```
$ git bisect good
b047b02ea83310a70fd603dc8cd7a6cd13d15c04 is first bad commit
commit b047b02ea83310a70fd603dc8cd7a6cd13d15c04
Author: PJ Hyett <pjhyett@example.com>
Date:   Tue Jan 27 14:48:32 2009 -0800

    Secure this thing
```

```
:040000 040000 40ee3e7821b895e52c1695092db9bdc4c61d1730
f24d3c6ebcfc639b1a3814550e62d60b8e68a8e4 M  config
```

Quando você terminar, você deve rodar `git bisect reset` para resetar seu HEAD de volta para onde você estava antes de você ter começado, ou você vai terminar em um estado estranho:

```
$ git bisect reset
```

Esta é uma ferramenta poderosa que pode ajudar você a checar centenas de commits à procura de um bug introduzido, em minutos. De fato, se você tiver um script que vai dar exit com 0 caso o projeto seja bom ou não-0 caso o projeto seja ruim, você consegue automatizar o `git bisect` completamente. Primeiro, mais uma vez você tem que determinar o escopo do bisect informando os commits conhecidos bom (good) e ruim (bad). Você pode fazer isso listando-os com o comando `bisect start` se quiser, primeiro o commit ruim e depois o bom:

```
$ git bisect start HEAD v1.0
$ git bisect run test-error.sh
```

Ao fazer isso o Git executa automaticamente o script `test-error.sh` a cada commit extraído até ele encontrar o primeiro que está com problema. Você pode ainda, usar comandos, tais como, `make` ou `make tests` ou até mesmo um script qualquer para a execução automatizada dos testes.

Submódulos

Costuma acontecer de, enquanto você trabalha em um projeto, você precise utilizar um outro projeto dentro dele. Pode ser uma biblioteca que um terceiro desenvolveu ou que você esteja desenvolvendo separadamente e usando em múltiplos projetos-pai. Um problema comum surge nesses cenários: você quer ser capaz de tratar os dois projetos como sendo separados e ainda assim poder utilizar um dentro do outro.

Aqui vai um exemplo. Suponha que você está desenvolvendo um website e criando feeds Atom. Em vez de escrever o seu próprio código gerador de Atom, você decide utilizar uma biblioteca. Você provavelmente terá que ou incluir esse código de uma biblioteca compartilhada como uma instalação CPAN ou Ruby gem, ou copiar o código-fonte para dentro da árvore do seu próprio projeto. O problema de incluir a biblioteca é que é difícil customizá-la de alguma forma e frequentemente mais difícil de fazer o deploy dela, porque você precisa garantir que cada cliente tenha aquela biblioteca disponível. O problema de copiar o código para o seu próprio projeto é que quaisquer customizações que você faça são difíceis de fazer merge quando alterações do upstream ficarem disponíveis.

O Git resolve esse problema com submódulos. Submódulos permitem que você mantenha um repositório Git como um subdiretório de um outro repositório Git. Isso permite a você clonar um outro repositório para o seu projeto e manter os seus commits separados.

Começando com Submódulos

Nós caminharemos pelo desenvolvimento de um projeto simples que foi dividido em um projeto principal e alguns sub-projetos.

Vamos começar adicionando um repositório Git existente como um submódulo do repositório no qual estamos trabalhando. Para adicionar um novo submódulo você usa o comando `git submodule add` com a URL absoluta ou relativa do projeto que você gostaria de começar a rastrear. Neste exemplo, nós adicionaremos uma biblioteca chamada "DbConnector".

```
$ git submodule add https://github.com/chaconinc/DbConnector
Cloning into 'DbConnector'...
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 0), reused 11 (delta 0)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done.
```

Por padrão, submódulos irão adicionar o subprojeto em um diretório de mesmo nome que o repositório, neste caso "DbConnector". Você pode adicionar um caminho diferente no final do comando se você quiser que ele vá para outro lugar.

Se você rodar `git status` a este ponto, você vai notar algumas coisas.

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   .gitmodules
    new file:   DbConnector
```

Primeiro você deve notar o novo arquivo `.gitmodules`. Esse é um arquivo de configuração que armazena o mapeamento entre a URL do projeto e o subdiretório local para o qual você fez o pull dele:

```
[submodule "DbConnector"]
    path = DbConnector
    url = https://github.com/chaconinc/DbConnector
```

Se você tiver múltiplos submódulos, você terá múltiplas entradas neste arquivo. É importante notar que este arquivo é controlado por versão junto com seus outros arquivos, como o seu arquivo `.gitignore`. Ele sofre push e pull com o resto do seu projeto. É assim que outras pessoas que clonarem este projeto vão saber de onde baixar os projetos do submódulo.



Como a URL no arquivo .gitmodules é a partir de onde outras pessoas tentarão dar clone/fetch primeiro, certifique-se de usar uma URL que elas consigam acessar, se possível. Por exemplo, se você usa uma URL para push diferente daquela que os outros utilizariam para pull, use aquela que os outros têm acesso. Você pode sobrescrever esse valor localmente com `git config submodule.DbConnector.url PRIVATE_URL` para o seu próprio uso. Quando aplicável, uma URL relativa pode ser útil.

A outra listagem na saída do `git status` é a entrada da pasta do projeto. Se você rodar `git diff` nisso, você vê algo interessante:

```
$ git diff --cached DbConnector
diff --git a/DbConnector b/DbConnector
new file mode 160000
index 0000000..c3f01dc
--- /dev/null
+++ b/DbConnector
@@ -0,0 +1 @@
+Subproject commit c3f01dc8862123d317dd46284b05b6892c7b29bc
```

Embora `DbConnector` seja um subdiretório no seu diretório de trabalho, o Git o enxerga como um submódulo e não rastreia o seu conteúdo quando você não está naquele diretório. Em vez disso, o Git o vê como um commit em particular daquele repositório.

Se você quiser uma saída de diff um pouco mais amigável, você pode passar a opção `--submodule` para o `git diff`.

```
$ git diff --cached --submodule
diff --git a/.gitmodules b/.gitmodules
new file mode 100644
index 0000000..71fc376
--- /dev/null
+++ b/.gitmodules
@@ -0,0 +1,3 @@
+[submodule "DbConnector"]
+       path = DbConnector
+       url = https://github.com/chaconinc/DbConnector
Submodule DbConnector 0000000...c3f01dc (new submodule)
```

Quando você comita, você vê algo assim:

```
$ git commit -am 'Add DbConnector module'
[master fb9093c] Add DbConnector module
 2 files changed, 4 insertions(+)
 create mode 100644 .gitmodules
 create mode 160000 DbConnector
```

Note o modo `160000` para a entrada `DbConnector`. Esse é um modo especial no Git que basicamente significa que você está gravando um commit como uma entrada de diretório em vez de um subdiretório ou um arquivo.

Por fim, dê push nessas alterações:

```
$ git push origin master
```

Clonando um Projeto com Submódulos

Aqui nós vamos clonar um projeto com um submódulo dentro dele. Quando você clona tal projeto, por padrão você obtém os diretórios que contêm submódulos, mas nenhum dos arquivos dentro deles ainda:

```
$ git clone https://github.com/chaconinc/MainProject
Cloning into 'MainProject'...
remote: Counting objects: 14, done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 14 (delta 1), reused 13 (delta 0)
Unpacking objects: 100% (14/14), done.
Checking connectivity... done.
$ cd MainProject
$ ls -la
total 16
drwxr-xr-x   9 schacon  staff  306 Sep 17 15:21 .
drwxr-xr-x   7 schacon  staff  238 Sep 17 15:21 ..
drwxr-xr-x  13 schacon  staff  442 Sep 17 15:21 .git
-rw-r--r--   1 schacon  staff   92 Sep 17 15:21 .gitmodules
drwxr-xr-x   2 schacon  staff   68 Sep 17 15:21 DbConnector
-rw-r--r--   1 schacon  staff  756 Sep 17 15:21 Makefile
drwxr-xr-x   3 schacon  staff  102 Sep 17 15:21 includes
drwxr-xr-x   4 schacon  staff  136 Sep 17 15:21 scripts
drwxr-xr-x   4 schacon  staff  136 Sep 17 15:21 src
$ cd DbConnector/
$ ls
$
```

O diretório `DbConnector` está lá, mas vazio. Você tem que rodar dois comandos a partir do projeto principal: `git submodule init` para inicializar o seu arquivo local de configurações, e `git submodule update` para buscar (fetch) todos os dados daquele projeto e fazer o checkout do commit apropriado listado no seu superprojeto (projeto-pai):

```
$ git submodule init
Submodule 'DbConnector' (https://github.com/chaconinc/DbConnector) registered for path
'DbConnector'
$ git submodule update
Cloning into 'DbConnector'...
```

```
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 0), reused 11 (delta 0)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done.
Submodule path 'DbConnector': checked out 'c3f01dc8862123d317dd46284b05b6892c7b29bc'
```

Agora o seu subdiretório `DbConnector` está no estado exato em que ele estava quando você comitou mais cedo.

No entanto, existe uma outra forma de fazer isso que é um pouco mais simples. Se você passar `--recurse-submodules` para o comando `git clone`, ele vai automaticamente inicializar e atualizar cada submódulo no repositório, incluindo submódulos aninhados se algum dos submódulos no repositório tiver submódulos também.

```
$ git clone --recurse-submodules https://github.com/chaconinc/MainProject
Cloning into 'MainProject'...
remote: Counting objects: 14, done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 14 (delta 1), reused 13 (delta 0)
Unpacking objects: 100% (14/14), done.
Checking connectivity... done.
Submodule 'DbConnector' (https://github.com/chaconinc/DbConnector) registered for path
'DbConnector'
Cloning into 'DbConnector'...
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 0), reused 11 (delta 0)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done.
Submodule path 'DbConnector': checked out 'c3f01dc8862123d317dd46284b05b6892c7b29bc'
```

Se você já clonou o projeto e esqueceu de `--recurse-submodules`, você pode combinar os passos do `git submodule init` e `git submodule update` rodando `git submodule update --init`. Para inicializar, fazer fetch e checkout também de quaisquer submódulos aninhados, você pode usar o infalível `git submodule update --init --recursive`.

Trabalhando em um Projeto com Submódulos

Agora nós temos uma cópia de um projeto com submódulos e iremos colaborar com nossos colegas de equipe tanto no projeto principal quanto no projeto do submódulo.

Puxando (Pulling) Alterações do Upstream a partir do Remoto do Submódulo

O modelo mais simples de se usar submódulos em um projeto seria se você simplesmente estivesse consumindo um subprojeto e quisesse obter atualizações dele de vez em quando mas não estivesse na verdade modificando nada no seu checkout. Vamos passar por um exemplo simples disso.

Se você quiser checar se há novos trabalhos em um submódulo, você pode entrar no diretório e

rodar `git fetch` e fazer `git merge` da branch de upstream para atualizar o código local.

```
$ git fetch
From https://github.com/chaconinc/DbConnector
   c3f01dc..d0354fc  master     -> origin/master
$ git merge origin/master
Updating c3f01dc..d0354fc
Fast-forward
 scripts/connect.sh | 1 +
 src/db.c           | 1 +
 2 files changed, 2 insertions(+)
```

Agora, se você voltar ao projeto principal e rodar `git diff --submodule`, você pode ver que o submódulo foi atualizado e obter uma lista de commits que foram adicionados a ele. Se você não quer digitar `--submodule` toda vez que você roda `git diff`, você pode defini-lo como formato padrão setando o valor de config `diff.submodule` como "log".

```
$ git config --global diff.submodule log
$ git diff
Submodule DbConnector c3f01dc..d0354fc:
  > more efficient db routine
  > better connection routine
```

Se você comitar nesse ponto então você vai travar o submódulo para ter o novo código quando as outras pessoas atualizarem.

Existe também uma maneira mais fácil de fazer isso, caso você prefira não fazer o fetch e o merge manualmente no subdiretório. Se você rodar `git submodule update --remote`, o Git vai entrar nos seus submódulos e fazer fetch e update para você.

```
$ git submodule update --remote DbConnector
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 2), reused 4 (delta 2)
Unpacking objects: 100% (4/4), done.
From https://github.com/chaconinc/DbConnector
   3f19983..d0354fc  master     -> origin/master
Submodule path 'DbConnector': checked out 'd0354fc054692d3906c85c3af05ddce39a1c0644'
```

Este comando vai por padrão assumir que você quer atualizar o checkout para a branch padrão do repositório do submódulo remoto (aquela apontada pelo `HEAD` no remoto). Você pode, no entanto, mudar isso para algo diferente se você quiser. Por exemplo, se você quer que o submódulo `DbConnector` rastreie a branch "stable" daquele repositório, você pode configurá-lo tanto no seu arquivo `.gitmodules` (para que todos os outros também o rastreiem), quanto apenas no seu arquivo local `.git/config`. Vamos defini-lo no arquivo `.gitmodules`:

```
$ git config -f .gitmodules submodule.DbConnector.branch stable

$ git submodule update --remote
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 2), reused 4 (delta 2)
Unpacking objects: 100% (4/4), done.
From https://github.com/chaconinc/DbConnector
   27cf5d3..c87d55d  stable -> origin/stable
Submodule path 'DbConnector': checked out 'c87d55d4c6d4b05ee34fbc8cb6f7bf4585ae6687'
```

Se você omitir o `-f .gitmodules` ele fará a alteração apenas para você, mas provavelmente faz mais sentido rastrear essa informação junto ao repositório para que todos os outros também a tenham.

Quando rodamos `git status` neste ponto, o Git nos mostrará que temos "new commits" no submódulo.

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

  modified:   .gitmodules
  modified:   DbConnector (new commits)

no changes added to commit (use "git add" and/or "git commit -a")
```

Se você definir a configuração `status.submodulesummary`, o Git também mostrará a você um curto sumário das alterações aos seus submódulos:

```
$ git config status.submodulesummary 1

$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   .gitmodules
    modified:   DbConnector (new commits)

Submodules changed but not updated:
```

```
* DbConnector c3f01dc...c87d55d (4):
  > catch non-null terminated lines
```

A este ponto se você rodar `git diff` nós podemos ver tanto que nós modificamos nosso arquivo `.gitmodules` quanto que há uma quantidade de commits que nós baixamos e que estão prontos para o commit em nosso projeto submódulo.

```
$ git diff
diff --git a/.gitmodules b/.gitmodules
index 6fc0b3d..fd1cc29 100644
--- a/.gitmodules
+++ b/.gitmodules
@@ -1,3 +1,4 @@
 [submodule "DbConnector"]
        path = DbConnector
        url = https://github.com/chaconinc/DbConnector
+       branch = stable
 Submodule DbConnector c3f01dc..c87d55d:
  > catch non-null terminated lines
  > more robust error handling
  > more efficient db routine
  > better connection routine
```

Isso é bem bacana pois nós podemos ver, na prática, o log dos commits que estamos prestes a comitar no nosso submódulo. Uma vez comitado, você pode ver esta informação depois do fato também quando você roda `git log -p`.

```
$ git log -p --submodule
commit 0a24cfc121a8a3c118e0105ae4ae4c00281cf7ae
Author: Scott Chacon <schacon@gmail.com>
Date:   Wed Sep 17 16:37:02 2014 +0200

    updating DbConnector for bug fixes

diff --git a/.gitmodules b/.gitmodules
index 6fc0b3d..fd1cc29 100644
--- a/.gitmodules
+++ b/.gitmodules
@@ -1,3 +1,4 @@
 [submodule "DbConnector"]
        path = DbConnector
        url = https://github.com/chaconinc/DbConnector
+       branch = stable
Submodule DbConnector c3f01dc..c87d55d:
  > catch non-null terminated lines
  > more robust error handling
  > more efficient db routine
  > better connection routine
```

O Git tentará, por padrão, atualizar **todos** os seus submódulos quando você rodar `git submodule update --remote`. Se você tiver muitos deles, você pode querer passar o nome de apenas o submódulo que você quer tentar atualizar.

Puxando (Pulling) Alterações do Upstream a partir do Remoto do Projeto

Vamos agora nos colocar no lugar do seu colaborador, que tem seu próprio clone local do repositório MainProject. Simplesmente executar `git pull` para pegar suas alterações recém-comitadas não é o bastante:

```
$ git pull
From https://github.com/chaconinc/MainProject
   fb9093c..0a24cfc  master     -> origin/master
Fetching submodule DbConnector
From https://github.com/chaconinc/DbConnector
   c3f01dc..c87d55d  stable     -> origin/stable
Updating fb9093c..0a24cfc
Fast-forward
 .gitmodules         | 2 +-
 DbConnector         | 2 +-
 2 files changed, 2 insertions(+), 2 deletions(-)

$ git status
 On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

    modified:   DbConnector (new commits)

Submodules changed but not updated:

* DbConnector c87d55d...c3f01dc (4):
  < catch non-null terminated lines
  < more robust error handling
  < more efficient db routine
  < better connection routine

no changes added to commit (use "git add" and/or "git commit -a")
```

Por padrão, o comando `git pull` faz o fetch recursivo de alterações de submódulos, conforme podemos ver na saída do primeiro comando acima. Entretanto, ele não **atualiza** (update) os submódulos. Isso é mostrado pela saída do comando `git status`, a qual mostra que o submódulo está modificado (“modified”), e que possui novos commits (“new commits”). Além do mais, os parênteses exibindo os novos commits apontam para a esquerda (<), indicando que estes commits estão registrados no MainProject mas não estão presentes no checkout local do `DbConnector`. Para finalizar a atualização, você precisa rodar o `git submodule update`:

```
$ git submodule update --init --recursive
Submodule path 'vendor/plugins/demo': checked out
'48679c6302815f6c76f1fe30625d795d9e55fc56'

$ git status
 On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working tree clean
```

Note que, para estar mais seguro, você deveria rodar `git submodule update` com a flag `--init` caso os commits do MainProject de que você acabou de dar pull tenham adicionado novos submódulos, e com a flag `--recursive` caso algum submódulo tenha submódulos aninhados.

Se você quiser automatizar esse processo, você pode adicionar a flag `--recurse-submodules` ao comando `git pull` (desde o Git 2.14). Isto fará com que o Git rode `git submodule update` logo após o pull, colocando os submódulos no estado correto. Ainda mais, se você deseja fazer com que o Git sempre dê pull com `--recurse-submodules`, você pode configurar a opção `submodule.recurse` como `true` (isso funciona para o `git pull` a partir do Git 2.15). Esta opção vai fazer o Git usar a flag `--recurse-submodules` para todos os comandos que lhe derem suporte (exceto `clone`).

Existe uma situação especial que pode acontecer ao se puxar atualizações do superprojeto: poderia ser que o repositório upstream mudou a URL do submódulo no arquivo `.gitmodules` em um dos commits de que você deu pull. Isso pode ocorrer por exemplo se o projeto do submódulo mudar sua plataforma de hospedagem. Nesse caso, é possível para o `git pull --recurse-submodules`, ou `git submodule update`, falhar caso o superprojeto faça referência a um commit do submódulo não encontrado no remoto do submódulo configurado localmente em seu repositório. A fim de remediar essa situação, o comando `git submodule sync` se faz necessário:

```
# copy the new URL to your local config
$ git submodule sync --recursive
# update the submodule from the new URL
$ git submodule update --init --recursive
```

Trabalhando em um Submódulo

É bem provável que se você está usando submódulos, você esteja fazendo isso porque você realmente quer trabalhar no código dentro do submódulo ao mesmo tempo em que está trabalhando no código dentro do projeto principal (ou ao longo de vários submódulos). De outro modo, você estaria provavelmente, ao invés disso, usando um sistema de gerenciamento de dependências mais simples (como Maven ou Rubygems).

Então agora vamos prosseguir com um exemplo de fazer alterações no submódulo ao mesmo tempo que no projeto principal e comitar e publicar (publish) essas alterações todas juntas.

Até agora, quando rodamos o comando `git submodule update` para fazer o fetch das alterações a partir dos repositórios dos submódulos, o Git traria as alterações e atualizaria os arquivos no subdiretório mas deixaria o sub-repositório no que se chama de estado de “detached HEAD” (HEAD

separado). Isto significa que não há nenhuma branch local de trabalho (como `master`, por exemplo) rastreando alterações. Sem uma branch de trabalho rastreando alterações, isso significa que mesmo se você comitar alterações para o submódulo, as ditas alterações serão bem possivelmente perdidas da próxima vez que você rodar um `git submodule update`. Você tem que executar alguns passos extras se quiser que alterações num submódulo sejam rastreadas.

A fim de configurar o seu submódulo para ser mais fácil de se entrar e codar nele, você precisa fazer duas coisas. Você precisa entrar em cada submódulo e fazer o checkout de uma branch para se trabalhar. Aí você precisa contar ao Git o que fazer se você tiver feito alterações e mais tarde `git submodule update --remote` puxar um novo trabalho do upstream. As opções são de que você pode fazer um merge delas no seu trabalho local, ou você pode tentar fazer o rebase do seu trabalho local em cima das novas alterações.

Primeiro de tudo, vamos entrar no nosso diretório de submódulo e fazer checkout de uma branch.

```
$ cd DbConnector/
$ git checkout stable
Switched to branch 'stable'
```

Vamos tentar atualizar o nosso submódulo com a opção de merge (opção “merge”). Para especificá-la de forma manual, podemos apenas adicionar a opção `--merge` à nossa chamada do comando `update`. Aqui veremos que houve uma alteração no servidor para este submódulo e ela sofre merge.

```
$ cd ..
$ git submodule update --remote --merge
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 2), reused 4 (delta 2)
Unpacking objects: 100% (4/4), done.
From https://github.com/chaconinc/DbConnector
   c87d55d..92c7337  stable     -> origin/stable
Updating c87d55d..92c7337
Fast-forward
 src/main.c | 1 +
 1 file changed, 1 insertion(+)
Submodule path 'DbConnector': merged in '92c7337b30ef9e0893e758dac2459d07362ab5ea'
```

Se formos para dentro do diretório `DbConnector`, teremos as novas alterações já mescladas (merged) em nossa branch `stable` local. Agora vamos ver o que acontece quando fazemos nossa própria alteração local para a biblioteca e alguma outra pessoa faz push de uma outra mudança para o upstream, tudo ao mesmo tempo.

```
$ cd DbConnector/
$ vim src/db.c
$ git commit -am 'Unicode support'
[stable f906e16] Unicode support
```

```
 1 file changed, 1 insertion(+)
```

Agora, se atualizarmos nosso submódulo, podemos ver o que acontece quando tivermos feito uma alteração local e o upstream também tiver uma mudança que precisamos incorporar.

```
$ cd ..
$ git submodule update --remote --rebase
First, rewinding head to replay your work on top of it...
Applying: Unicode support
Submodule path 'DbConnector': rebased into '5d60ef9bbebf5a0c1c1050f242ceeb54ad58da94'
```

Se você se esquecer de `--rebase` ou `--merge`, o Git vai só atualizar o submódulo para aquilo que se encontre no servidor e resetar seu projeto para o estado *detached HEAD*.

```
$ git submodule update --remote
Submodule path 'DbConnector': checked out '5d60ef9bbebf5a0c1c1050f242ceeb54ad58da94'
```

Caso isso aconteça, não se preocupe, você pode simplesmente retornar ao diretório e dar checkout da sua branch de novo (a qual continuará contendo o seu trabalho) e aplicar um merge ou um rebase com `origin/stable` (ou qualquer outro branch remoto que você quiser) manualmente.

Se você não tiver comitado suas modificações em seu submódulo e você rodar um `submodule update` que possa causar problemas, o Git irá buscar (fetch) as modificações, contudo, sem sobrescrever qualquer trabalho que não tenha sido salvo, dentro do diretório de seu submódulo.

```
$ git submodule update --remote
remote: Counting objects: 4, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 4 (delta 0)
Unpacking objects: 100% (4/4), done.
From https://github.com/chaconinc/DbConnector
   5d60ef9..c75e92a  stable     -> origin/stable
error: Your local changes to the following files would be overwritten by checkout:
    scripts/setup.sh
Please, commit your changes or stash them before you can switch branches.
Aborting
Unable to checkout 'c75e92a2b3855c9e5b66f915308390d9db204aca' in submodule path
'DbConnector'
```

Se as alterações que você realizou conflitarem com algo que mudou no upstream, o Git irá informá-lo quando você rodar o update.

```
$ git submodule update --remote --merge
Auto-merging scripts/setup.sh
CONFLICT (content): Merge conflict in scripts/setup.sh
Recorded preimage for 'scripts/setup.sh'
```

```
Automatic merge failed; fix conflicts and then commit the result.
Unable to merge 'c75e92a2b3855c9e5b66f915308390d9db204aca' in submodule path
'DbConnector'
```

Você pode ir para o diretório de submódulo e resolver o conflito, exatamente como você faria normalmente.

Publicando Modificações do Submódulo

Agora, nós temos algumas modificações no nosso diretório de submódulo. Algumas delas foram trazidas desde o upstream através de nossos updates, já outras foram criadas localmente e ainda não estão à disposição de qualquer outra pessoa, uma vez que ainda não realizamos o push.

```
$ git diff
Submodule DbConnector c87d55d..82d2ad3:
  > Merge from origin/stable
  > Update setup script
  > Unicode support
  > Remove unnecessary method
  > Add new option for conn pooling
```

Se nós dermos um commit no projeto principal e aplicarmos o push nele sem aplicar o push às modificações do submódulo juntas, as outras pessoas, ao tentarem realizar check out em nossas alterações, vão ter problemas, visto que elas não possuirão meio de obter aquelas atualizações do submódulo, das quais elas dependem. Aquelas alterações, as do submódulo, só existirão na nossa cópia local.

Para ter a certeza que isto não ocorra, você tem de pedir que o Git verifique, previamente à submissão do projeto principal, se em todos os seus submódulos foi aplicado o push corretamente. O comando `git push` pode ser acompanhado pelo argumento `--recurse-submodules` o qual pode assumir valores como: “check” ou “on-demand”. A opção “check” fará `push` simplesmente falhar se alguma alteração commitada no submódulo ainda não tiver sido enviada por push.

```
$ git push --recurse-submodules=check
The following submodule paths contain changes that can
not be found on any remote:
  DbConnector

Please try

    git push --recurse-submodules=on-demand

or cd to the path and use

    git push

to push them to a remote.
```

Como você pode ver, ele também oferece orientações úteis sobre o que fazer em seguida. A opção simples consiste em adentrar cada submódulo e, manualmente, realizar push nos remotes para nos certificar que tais submódulos estarão disponíveis de maneira externa e, então, tentar efetuar esse push novamente. Se você desejar que o comportamento verificado em "check" suceda em todo e qualquer push, você pode deixá-lo como o comportamento padrão mediante este comando `git config push.recurseSubmodules check`.

A outra opção compreende a utilização do valor "on-demand", que tentará executar as ações anteriores por você.

```
$ git push --recurse-submodules=on-demand
Pushing submodule 'DbConnector'
Counting objects: 9, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 917 bytes | 0 bytes/s, done.
Total 9 (delta 3), reused 0 (delta 0)
To https://github.com/chaconinc/DbConnector
   c75e92a..82d2ad3  stable -> stable
Counting objects: 2, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 266 bytes | 0 bytes/s, done.
Total 2 (delta 1), reused 0 (delta 0)
To https://github.com/chaconinc/MainProject
   3d6d338..9a377d1  master -> master
```

Dali, pelo que você foi capaz de ver, o Git foi ao módulo `DbConnector` e ali aplicou o push antes de dar push no projeto central. Se esse push no submódulo vier a falhar em razão de algum motivo, o push do projeto pai vai também falhar. É possível deixar esse procedimento de modo default, fazendo `git config push.recurseSubmodules on-demand`.

Modificações de Submódulo por Merging

Se você alterar uma referência de submódulo ao mesmo tempo que outra pessoa, poderá encontrar alguns problemas. Isso quer dizer: se as histórias do submódulo divergiram e os commits acompanharam essas ramificações, que divergem, em superprojetos; vai ser preciso um pouco de trabalho, de sua parte, com vistas a reparar isto.

Se um dos commits for ancestral direto do outro (um merge fast-forward), o Git simplesmente escolherá o segundo para o merge, e tudo funcionará bem.

Com isso, o Git não intentará mesmo num tipo banal de merging a seu dispor. Se os commits no submódulo divergirem e precisarem que lhes faça um merge, você terá algo semelhante a isto, como resposta:

```
$ git pull
remote: Counting objects: 2, done.
remote: Compressing objects: 100% (1/1), done.
```

```
remote: Total 2 (delta 1), reused 2 (delta 1)
Unpacking objects: 100% (2/2), done.
From https://github.com/chaconinc/MainProject
   9a377d1..eb974f8  master     -> origin/master
Fetching submodule DbConnector
warning: Failed to merge submodule DbConnector (merge following commits not found)
Auto-merging DbConnector
CONFLICT (submodule): Merge conflict in DbConnector
Automatic merge failed; fix conflicts and then commit the result.
```

Em suma: o que aconteceu, aqui, é que o Git imaginou que ambas the branches registraram pontos da história do submódulo que são incompatíveis e cujas pontas hão de ser amarradas, via um *merge*. Ele explica o problema como “merge following commits not found”, o que é confuso, mas logo explicaremos o motivo.

De modo a solucionar esse imbróglio, você terá de descobrir em qual estado o seu submódulo deveria estar. Estranhamente, o Git não fornece muitas informações úteis aqui, nem mesmo os SHA-1s dos commits dos dois lados do histórico. Felizmente, é simples descobri-los. Se você submeter o comando `git diff`, poderá extrair os SHA-1s, dos commits anotados nos dois branches nos quais tem buscado realizar a fusão (merge).

```
$ git diff
diff --cc DbConnector
index eb41d76,c771610..0000000
--- a/DbConnector
+++ b/DbConnector
```

Em sendo assim, neste caso, `eb41d76` é o commit em nosso submódulo que **nós** possuíamos e `c771610` é aquele commit que o upstream tinha. Se entrarmos no diretório do submódulo, ele já deverá estar em `eb41d76`, pois o merge não o teria alterado. Por outro lado, caso não seja, você poderá apenas criar um branch, apontando em direção dele e então submeter um checkout logo a seguir.

O que importa é o que se tira do SHA-1 de autoria do commit que vêm da parte que nos é oposta. É isso que você precisará mesclar e resolver. Você pode simplesmente tentar o merge diretamente com o SHA-1 ou criar um branch para ele e então tentar mesclá-lo. Sugerimos a segunda opção, ainda que seja apenas para produzir uma mensagem de commit de merge melhor.

Assim, entraremos no diretório do submódulo, criaremos um branch chamado “try-merge” com base no segundo SHA-1 de `git diff` e faremos o merge manualmente.

```
$ cd DbConnector

$ git rev-parse HEAD
eb41d764bccf88be77aced643c13a7fa86714135

$ git branch try-merge c771610

$ git merge try-merge
```

```
Auto-merging src/main.c
CONFLICT (content): Merge conflict in src/main.c
Recorded preimage for 'src/main.c'
Automatic merge failed; fix conflicts and then commit the result.
```

Encontramos um conflito de merge real; se o resolvermos e fizermos seu commit, poderemos simplesmente atualizar o projeto principal com o resultado.

```
$ vim src/main.c ①
$ git add src/main.c
$ git commit -am 'merged our changes'
Recorded resolution for 'src/main.c'.
[master 9fd905e] merged our changes

$ cd .. ②
$ git diff ③
diff --cc DbConnector
index eb41d76,c771610..0000000
--- a/DbConnector
+++ b/DbConnector
@@@ -1,1 -1,1 +1,1 @@@
- Subproject commit eb41d764bccf88be77aced643c13a7fa86714135
 -Subproject commit c77161012afbbe1f58b5053316ead08f4b7e6d1d
++Subproject commit 9fd905e5d7f45a0d4cbc43d1ee550f16a30e825a
$ git add DbConnector ④

$ git commit -m "Merge Tom's Changes" ⑤
[master 10d2c60] Merge Tom's Changes
```

① Primeiro, nós resolvemos o conflito.

② Depois nós retornamos ao diretório do projeto pai.

③ Podemos realizar uma nova checagem dos SHA-1s.

④ Tratar a entrada do submódulo que gerou o conflito.

⑤ Faça commit do nosso merge.

Isso pode parecer um pouco confuso, mas na verdade não é muito difícil.

Curiosamente, há outro caso que o Git consegue tratar. Se houver no diretório do submódulo um commit de merge cujo histórico contenha **ambos** os commits, o Git o sugerirá como possível solução. Ele percebe que, em algum ponto do projeto do submódulo, alguém mesclou branches que continham esses dois commits; talvez seja esse o resultado que você queira.

É por isso que a mensagem de erro anterior era “merge following commits not found”: o Git não conseguiu fazer **isso**. É confuso, pois quem esperaria que ele sequer **tentasse** fazer isso?

Se ele encontrar um único commit de merge aceitável, você verá algo assim:

```
$ git merge origin/master
warning: Failed to merge submodule DbConnector (not fast-forward)
Found a possible merge resolution for the submodule:
 9fd905e5d7f45a0d4cbc43d1ee550f16a30e825a: > merged our changes
If this is correct simply add it to the index for example
by using:

  git update-index --cacheinfo 160000 9fd905e5d7f45a0d4cbc43d1ee550f16a30e825a
"DbConnector"

which will accept this suggestion.
Auto-merging DbConnector
CONFLICT (submodule): Merge conflict in DbConnector
Automatic merge failed; fix conflicts and then commit the result.
```

O comando sugerido pelo Git atualizará o index como se você tivesse executado `git add` (o que elimina o conflito) e então fará o commit. No entanto, provavelmente você não deveria fazer isso. É igualmente fácil entrar no diretório do submódulo, examinar a diferença, fazer fast-forward para esse commit, testá-lo corretamente e então fazer o commit.

```
$ cd DbConnector/
$ git merge 9fd905e
Updating eb41d76..9fd905e
Fast-forward

$ cd ..
$ git add DbConnector
$ git commit -am 'Fast forward to a common submodule child'
```

Isso alcança o mesmo resultado, mas dessa forma você ao menos pode verificar que tudo funciona e, ao terminar, terá o código no diretório do submódulo.

Dicas de Submódulos

Há algumas coisas que você pode fazer para tornar o trabalho com submódulos um pouco mais fácil.

Foreach de Submódulo

Há um comando de submódulo `foreach` para rodar algum comando arbitrário em cada submódulo. Isso pode ser realmente útil se você tem um número de submódulos no mesmo projeto.

Por exemplo, digamos que nós queiramos iniciar um novo recurso ou fazer uma correção de bug e nós temos trabalhos ocorrendo em vários submódulos. Podemos facilmente fazer stash de todo o trabalho em todos os submódulos.

```
$ git submodule foreach 'git stash'
```

```
Entering 'CryptoLibrary'
No local changes to save
Entering 'DbConnector'
Saved working directory and index state WIP on stable: 82d2ad3 Merge from
origin/stable
HEAD is now at 82d2ad3 Merge from origin/stable
```

Então nós podemos criar uma nova branch e mudar para ela em todos os nossos submódulos.

```
$ git submodule foreach 'git checkout -b featureA'
Entering 'CryptoLibrary'
Switched to a new branch 'featureA'
Entering 'DbConnector'
Switched to a new branch 'featureA'
```

Você entendeu a ideia. Uma coisa realmente útil que você pode fazer é produzir um diff unificado do que foi alterado em seu projeto principal e todos os seus subprojetos também.

```
$ git diff; git submodule foreach 'git diff'
Submodule DbConnector contains modified content
diff --git a/src/main.c b/src/main.c
index 210f1ae..1f0acdc 100644
--- a/src/main.c
+++ b/src/main.c
@@ -245,6 +245,8 @@ static int handle_alias(int *argcp, const char ***argv)

      commit_pager_choice();

+     url = url_decode(url_orig);
+
      /* build alias_argv */
      alias_argv = xmalloc(sizeof(*alias_argv) * (argc + 1));
      alias_argv[0] = alias_string + 1;
Entering 'DbConnector'
diff --git a/src/db.c b/src/db.c
index 1aaefb6..5297645 100644
--- a/src/db.c
+++ b/src/db.c
@@ -93,6 +93,11 @@ char *url_decode_mem(const char *url, int len)
        return url_decode_internal(&url, len, NULL, &out, 0);
 }

+char *url_decode(const char *url)
+{
+       return url_decode_mem(url, strlen(url));
+}
+
 char *url_decode_parameter_name(const char **query)
 {
```

```
        struct strbuf out = STRBUF_INIT;
```

Aqui nós podemos ver que estamos definindo uma função num submódulo e a chamando no projeto principal. Isso é, obviamente, um exemplo simplificado, mas esperançosamente te dá uma ideia de como isso pode ser útil.

Aliases Úteis

Você pode querer configurar alguns apelidos (aliases) para alguns destes comandos dado que eles podem ser muito grandes e não é possível configurar opções para a maioria deles a fim de os tornarem padrões. Abordamos a configuração de aliases do Git em Aliases (Apelidos) no Git, mas aqui está um exemplo do que você talvez queira configurar se pretende trabalhar muito com submódulos no Git.

```
$ git config alias.sdiff '!'"git diff && git submodule foreach 'git diff'"
$ git config alias.spush 'push --recurse-submodules=on-demand'
$ git config alias.supdate 'submodule update --remote --merge'
```

Dessa forma, você pode simplesmente executar `git supdate` quando quiser atualizar seus submódulos ou `git spush` para fazer push com verificação das dependências de submódulos.

Problemas com Submódulos

No entanto, o uso de submódulos não é isento de contratempos.

Mudando de ramos (branches)

Por exemplo, alternar entre branches que contêm submódulos também pode ser complicado em versões do Git anteriores à 2.13. Se você criar um novo branch, adicionar um submódulo nele e então voltar para um branch sem esse submódulo, o diretório do submódulo permanecerá como um diretório não rastreado:

```
$ git --version
git version 2.12.2

$ git checkout -b add-crypto
Switched to a new branch 'add-crypto'

$ git submodule add https://github.com/chaconinc/CryptoLibrary
Cloning into 'CryptoLibrary'...
...

$ git commit -am 'Add crypto library'
[add-crypto 4445836] Add crypto library
 2 files changed, 4 insertions(+)
 create mode 160000 CryptoLibrary

$ git checkout master
```

```
warning: unable to rmdir CryptoLibrary: Directory not empty
Switched to branch 'master'
Your branch is up-to-date with 'origin/master'.

$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    CryptoLibrary/

nothing added to commit but untracked files present (use "git add" to track)
```

Remover o diretório não é difícil, mas pode ser um pouco confuso vê-lo ali. Se você o remover e depois voltar para o branch que contém o submódulo, precisará executar `submodule update --init` para preenchê-lo novamente.

```
$ git clean -ffdx
Removing CryptoLibrary/

$ git checkout add-crypto
Switched to branch 'add-crypto'

$ ls CryptoLibrary/

$ git submodule update --init
Submodule path 'CryptoLibrary': checked out 'b8dda6aa182ea4464f3f3264b11e0268545172af'

$ ls CryptoLibrary/
Makefile    includes    scripts     src
```

Novamente, não é realmente difícil, mas pode ser um pouco confuso.

Nas versões contemporâneas da ferramenta (Git >= 2.13), toda esta gama de ações encontrou simplificação diante do acréscimo de `--recurse-submodules`, como flag a acompanhar o comando `git checkout`; a incumbência dessa se dá por repousar os submódulos ao estado ideal àquele correspondente no ramo que então tencionamos.

```
$ git --version
git version 2.13.3

$ git checkout -b add-crypto
Switched to a new branch 'add-crypto'

$ git submodule add https://github.com/chaconinc/CryptoLibrary
Cloning into 'CryptoLibrary'...
```

```
...

$ git commit -am 'Add crypto library'
[add-crypto 4445836] Add crypto library
 2 files changed, 4 insertions(+)
 create mode 160000 CryptoLibrary

$ git checkout --recurse-submodules master
Switched to branch 'master'
Your branch is up-to-date with 'origin/master'.

$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

nothing to commit, working tree clean
```

Usar a flag `--recurse-submodules` de `git checkout` também pode ser útil quando você trabalha em vários branches do superprojeto e, em cada um deles, o submódulo aponta para commits diferentes. De fato, se você alternar entre branches que registram o submódulo em commits diferentes, ao executar `git status` o submódulo aparecerá como “modified” e indicará “new commits”. Atribui-se isso ao fato de, como padrão, o submódulo ser preterido quanto a acompanhamento de trânsito ao flutuarmos nestes ramos.

Isso pode ser muito confuso, portanto é uma boa ideia sempre usar `git checkout --recurse-submodules` quando o projeto contém submódulos. Em versões antigas do Git que não possuem a flag `--recurse-submodules`, você pode usar `git submodule update --init --recursive` depois do checkout para colocar os submódulos no estado correto.

Felizmente, você pode instruir o Git (>=2.14) a sempre usar a flag `--recurse-submodules` definindo a opção de configuração `submodule.recurse`: `git config submodule.recurse true`. Como observado acima, isso também fará o Git percorrer recursivamente os submódulos em todo comando que possua a opção `--recurse-submodules` (exceto `git clone`).

Modificando de Subdiretórios para Submódulos

Outra ressalva importante que muitas pessoas encontram envolve a mudança de subdiretórios para submódulos. Se você rastreava arquivos no projeto e quer movê-los para um submódulo, deve ter cuidado, ou o Git ficará irritado com você. Suponha que existam arquivos em um subdiretório do projeto e que você queira transformá-lo em um submódulo. Se excluir o subdiretório e então executar `submodule add`, o Git reclamará:

```
$ rm -Rf CryptoLibrary/
$ git submodule add https://github.com/chaconinc/CryptoLibrary
'CryptoLibrary' already exists in the index
```

Primeiro, você precisa retirar o diretório `CryptoLibrary` da área de preparação. Depois, poderá adicionar o submódulo:

```
$ git rm -r CryptoLibrary
$ git submodule add https://github.com/chaconinc/CryptoLibrary
Cloning into 'CryptoLibrary'...
remote: Counting objects: 11, done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 0), reused 11 (delta 0)
Unpacking objects: 100% (11/11), done.
Checking connectivity... done.
```

Agora suponha que você tenha feito isso em um branch. Se tentar voltar para um branch em que esses arquivos ainda estejam na árvore real, e não em um submódulo, receberá este erro:

```
$ git checkout master
error: The following untracked working tree files would be overwritten by checkout:
  CryptoLibrary/Makefile
  CryptoLibrary/includes/crypto.h
  ...
Please move or remove them before you can switch branches.
Aborting
```

Você pode forçar a mudança com `checkout -f`, mas tome cuidado para não ter alterações não salvas ali, pois esse comando poderá sobrescrevê-las.

```
$ git checkout -f master
warning: unable to rmdir CryptoLibrary: Directory not empty
Switched to branch 'master'
```

Então, ao voltar, por algum motivo você encontrará um diretório `CryptoLibrary` vazio, e talvez nem `git submodule update` consiga corrigi-lo. Pode ser necessário entrar no diretório do submódulo e executar `git checkout .` para recuperar todos os arquivos. Você pode executar isso em um script `submodule foreach` para aplicá-lo a vários submódulos.

É importante observar que, hoje em dia, os submódulos mantêm todos os dados Git no diretório `.git` do projeto principal; portanto, ao contrário de versões muito antigas do Git, destruir o diretório de um submódulo não fará você perder commits ou branches.

Com essas ferramentas, os submódulos podem ser um método bastante simples e eficaz para desenvolver simultaneamente vários projetos relacionados, mas ainda separados.

Empacotando (Bundling)

Apesar de termos abordado as maneiras comuns de transferir dados do Git em uma rede (HTTP, SSH, etc), na verdade há uma maneira de mais de fazê-lo que não é muito usada mas que pode ser bastante útil.

O Git é capaz de “empacotar” (bundle) os seus dados em um único arquivo. Isso pode ser útil em

vários cenários. Talvez a sua rede caiu e você quer enviar alterações para os seus colegas. Pode ser que você esteja trabalhando externo em algum lugar e não tem acesso à rede local por motivos de segurança. Talvez sua placa de rede wireless/ethernet simplesmente quebrou. Talvez você não tenha acesso a um servidor compartilhado no momento, quer mandar atualizações por email para alguém e não quer ter que transferir 40 commits por `format-patch`.

É aí que o comando `git bundle` pode ser útil. O comando `bundle` empacotará tudo que seria normalmente mandado (pushed) pela rede com um comando `git push` para dentro de um arquivo binário que você pode mandar por email para alguém ou pôr em um pen drive, para depois desempacotar (unbundle) em um outro repositório.

Vejamos um exemplo simples. Digamos que você tem um repositório com dois commits:

```
$ git log
commit 9a466c572fe88b195efd356c3f2bbeccdb504102
Author: Scott Chacon <schacon@gmail.com>
Date:   Wed Mar 10 07:34:10 2010 -0800

    Second commit

commit b1ec3248f39900d2a406049d762aa68e9641be25
Author: Scott Chacon <schacon@gmail.com>
Date:   Wed Mar 10 07:34:01 2010 -0800

    First commit
```

Se você quiser enviar aquele repositório para alguém e você não tiver acesso a um repositório para o qual fazer o push, ou simplesmente não quiser configurar um, você pode empacotá-lo com `git bundle create`.

```
$ git bundle create repo.bundle HEAD master
Counting objects: 6, done.
Delta compression using up to 2 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 441 bytes, done.
Total 6 (delta 0), reused 0 (delta 0)
```

Agora você tem um arquivo chamado `repo.bundle` que tem todos os dados necessários para recriar a branch `master` do repositório. Com o comando `bundle` você precisa listar todas as referências ou uma gama específica de commits que você quer que sejam incluídos. Se você pretende que isto seja clonado em algum lugar, você deveria adicionar o HEAD como referência, bem como nós fizemos aqui.

Você pode mandar este arquivo `repo.bundle` por email para alguém, ou pô-lo em um pendrive e levá-lo pessoalmente.

Na outra ponta, digamos que te enviem este arquivo `repo.bundle` e você queira trabalhar no projeto. Você pode clonar a partir do arquivo binário para um diretório, mais ou menos igual a quando você

faz a partir de uma URL.

```
$ git clone repo.bundle repo
Cloning into 'repo'...
...
$ cd repo
$ git log --oneline
9a466c5 Second commit
b1ec324 First commit
```

Se você não incluir o HEAD nas referências, você tem também que especificar `-b master` ou seja qual for a branch incluída porque senão ele não saberá qual branch utilizar no check out.

Agora digamos que você faça três commits nisso e queira mandar os novos commits de volta através de um bundle em um pen drive ou por email.

```
$ git log --oneline
71b84da Last commit - second repo
c99cf5b Fourth commit - second repo
7011d3d Third commit - second repo
9a466c5 Second commit
b1ec324 First commit
```

Primeiro nós precisamos determinar a gama de commits que queremos incluir no bundle. Diferente dos protocolos de rede, os quais calculam por nós qual é o conjunto mínimo de dados a se transferir pela rede, nós teremos de calcular isso manualmente. Você pode apenas fazer a mesma coisa e empacotar (bundle) o repositório inteiro, o que vai funcionar, mas é melhor empacotar apenas a diferença - somente os três commits que acabamos de fazer localmente.

A fim de fazer isso, você terá que calcular a diferença. Conforme descrevemos em Intervalos (Ranges) de Commits, você pode especificar uma gama de commits de uma série de formas. Para obtermos os três commits que temos na nossa branch `master` e que não estavam na branch que clonamos originalmente, nós podemos usar algo como `origin/master..master` ou `master ^origin/master`. Você pode testar isso com o comando `log`.

```
$ git log --oneline master ^origin/master
71b84da Last commit - second repo
c99cf5b Fourth commit - second repo
7011d3d Third commit - second repo
```

Então, agora que temos a lista de commits que queremos incluir no bundle, vamos empacotá-los. Nós fazemos isso com o comando `git bundle create`, dando a ele um nome de arquivo que queremos que seja o nosso bundle e a gama de commits que queremos por ali dentro.

```
$ git bundle create commits.bundle master ^9a466c5
Counting objects: 11, done.
```

```
Delta compression using up to 2 threads.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (9/9), 775 bytes, done.
Total 9 (delta 0), reused 0 (delta 0)
```

Agora nós temos um arquivo `commits.bundle` no nosso diretório. Se pegarmos isso e mandarmos para a nossa colega, ela então poderá importar para o repositório original, mesmo se mais trabalho tiver sido feito por lá nesse ínterim.

Quando ela receber o bundle, ela pode inspecioná-lo para ver o que ele contém antes que ela o importe para o repositório dela. O primeiro comando é o comando `bundle verify` que irá confirmar se o arquivo é realmente um bundle válido do Git e se você tem todos os ancestrais necessários para reconstituí-lo de forma correta.

```
$ git bundle verify ../commits.bundle
The bundle contains 1 ref
71b84daaf49abed142a373b6e5c59a22dc6560dc refs/heads/master
The bundle requires these 1 ref
9a466c572fe88b195efd356c3f2bbeccdb504102 second commit
../commits.bundle is okay
```

Se a pessoa tivesse criado um bundle apenas com os dois últimos commits que ela tinha feito, ao invés de todos os três, o repositório original não seria capaz de importá-lo, posto que estaria faltando histórico que lhe era requisito. O comando `verify` ter-se-ia parecido mais com isso, pelo contrário:

```
$ git bundle verify ../commits-bad.bundle
error: Repository lacks these prerequisite commits:
error: 7011d3d8fc200abe0ad561c011c3852a4b7bbe95 Third commit - second repo
```

Entretanto, nosso primeiro bundle é válido, de modo que nós podemos fazer o fetch dos commits dali. Se você quer ver quais branches no bundle podem ser importadas, tem também um comando só pra listar os heads:

```
$ git bundle list-heads ../commits.bundle
71b84daaf49abed142a373b6e5c59a22dc6560dc refs/heads/master
```

O subcomando `verify` também te contará os heads. O ponto é ver o que pode ser sofrer um pull (pulled in), de forma que você possa usar os comandos `fetch` ou `pull` para importar commits deste bundle. Aqui nós faremos o fetch da branch `master` do bundle, destinando a uma branch chamada `other-master` em nosso repositório:

```
$ git fetch ../commits.bundle master:other-master
From ../commits.bundle
 * [new branch]      master     -> other-master
```

Agora podemos ver que nós temos os commits importados na branch `other-master` assim como quaisquer commits que tenhamos feito nesse meio tempo em nossa própria branch `master`.

```
$ git log --oneline --decorate --graph --all
* 8255d41 (HEAD, master) Third commit - first repo
| * 71b84da (other-master) Last commit - second repo
| * c99cf5b Fourth commit - second repo
| * 7011d3d Third commit - second repo
|/
* 9a466c5 Second commit
* b1ec324 First commit
```

Sendo assim, o `git bundle` pode ser bastante útil para se compartilhar ou executar operações do tipo-rede quando você não dispõe da rede adequada ou de um repositório compartilhado para fazê-lo.

Substituir (Replace)

Como já enfatizamos anteriormente, os objetos no banco de dados de objetos do Git são imutáveis, mas o Git fornece uma maneira interessante de *fingir* substituir objetos em seu banco de dados por outros objetos.

O comando `replace` permite especificar um objeto no Git e dizer "toda vez que você se referir a *este* objeto, finja que é um objeto *diferente*". Isto é mais comumente útil para substituir um commit no seu histórico por outro, sem ter que reconstruir todo o histórico com, digamos, o `git filter-branch`.

Por exemplo, digamos que você tenha um enorme histórico de código e deseje dividir o seu repositório em um histórico curto para novos desenvolvedores e um histórico muito maior e mais longo para pessoas interessadas em garimpar dados. Você pode enxertar um histórico no outro "substituindo" o primeiro commit da linha nova pelo último commit da antiga. Isto é ótimo porque significa que você não precisa realmente reescrever todos os commits do histórico novo, como você normalmente teria que fazer para juntá-los (já que a descendência afeta os SHA-1s).

Vamos tentar isso. Vamos pegar um repositório existente, dividi-lo em dois repositórios, um recente e um histórico, e então veremos como nós podemos recombiná-los sem modificar os valores de SHA-1 do repositório recente usando o `replace`.

Nós vamos usar um repositório simples com cinco commits simples:

```
$ git log --oneline
ef989d8 Fifth commit
c6e1e95 Fourth commit
9c68fdc Third commit
945704c Second commit
c1822cf First commit
```

Nós queremos quebrar isso em duas linhas de histórico. A primeira linha vai do commit um ao

commit quatro - esta será a histórica. A segunda linha será composta apenas pelos commits quatro e cinco - esta será a de histórico recente.

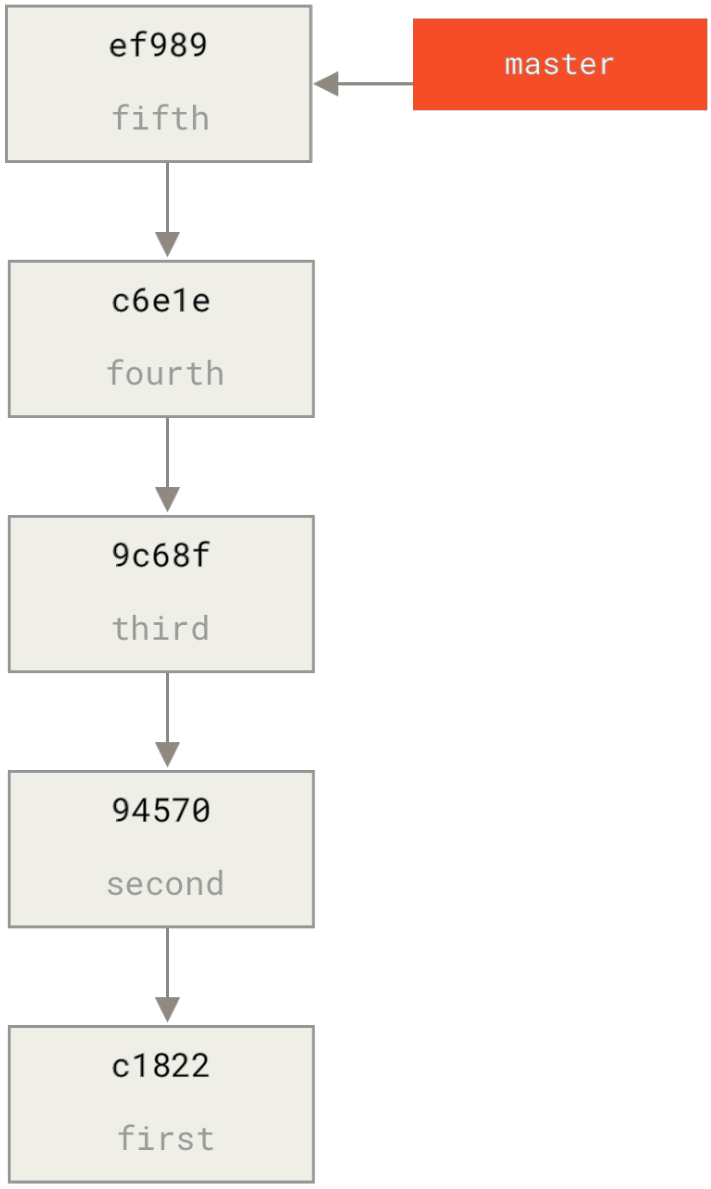


Figure 163. Exemplo de histórico do Git

Bom, criar o histórico histórico é fácil, nós podemos apenas colocar uma branch no histórico e depois dar um push desta branch para a branch `master` de um novo repositório remoto.

```
$ git branch history c6e1e95
$ git log --oneline --decorate
ef989d8 (HEAD, master) Fifth commit
c6e1e95 (history) Fourth commit
9c68fdc Third commit
945704c Second commit
c1822cf First commit
```

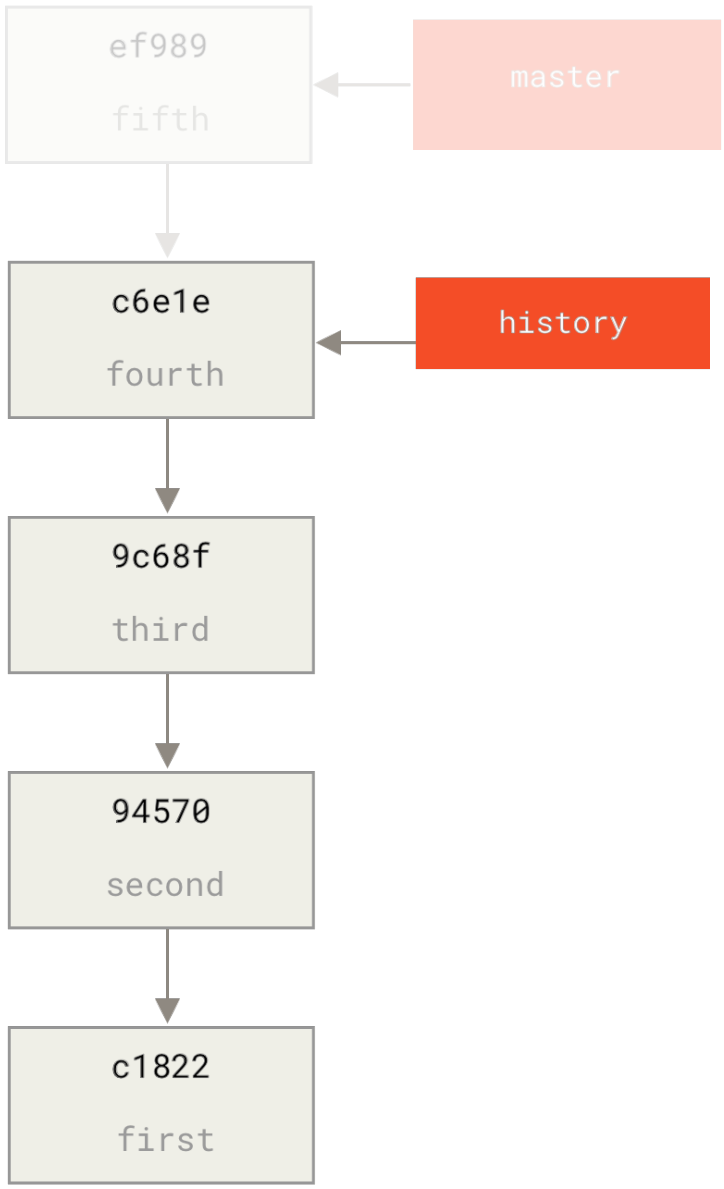


Figure 164. Criando uma nova branch `history`

Agora nós podemos dar um push da nova branch `history` para a branch `master` do nosso novo repositório:

```
$ git remote add project-history https://github.com/schacon/project-history
$ git push project-history history:master
Counting objects: 12, done.
Delta compression using up to 2 threads.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (12/12), 907 bytes, done.
Total 12 (delta 0), reused 0 (delta 0)
Unpacking objects: 100% (12/12), done.
To git@github.com:schacon/project-history.git
 * [new branch]      history -> master
```

OK, nosso histórico está publicado. Agora a parte mais difícil é truncar (encurtar) nosso histórico recente para que ele fique menor. Nós precisamos de uma sobreposição para que possamos substituir um commit em um deles por um commit equivalente no outro, então nós vamos truncar isto deixando somente os commits quatro e cinco (fazendo com que o commit quatro se sobreponha).

```
$ git log --oneline --decorate
ef989d8 (HEAD, master) Fifth commit
c6e1e95 (history) Fourth commit
9c68fdc Third commit
945704c Second commit
c1822cf First commit
```

É útil neste caso criar um commit base que tenha instruções de como expandir o histórico, assim os outros desenvolvedores saberão o que fazer se baterem no primeiro commit do histórico truncado e precisarem de mais. Então, o que vamos fazer é criar um objeto commit inicial, sendo o nosso ponto base com instruções, e então realizar um rebase nos commits remanescentes (quatro e cinco) em cima dele.

Para se fazer isso, precisamos escolher um ponto em que faremos a divisão, que para nós é o terceiro commit, que é o `9c68fdc` no jargão do SHA. Assim, nosso commit base será baseado nesta árvore (tree). Nós podemos criar nosso commit base utilizando o comando `commit-tree`, o qual simplesmente apanha a tree (árvore) e nos devolve um brand new SHA-1 de objeto de commit e que não tem ascendência (parentless).

```
$ echo 'Get history from blah blah blah' | git commit-tree 9c68fdc^{tree}
622e88e9cbfbacfb75b5279245b9fb38dfea10cf
```



O comando `commit-tree` faz parte de um conjunto conhecido normalmente como comandos de encanamento (plumbing). Em geral, esses comandos não se destinam ao uso direto; em vez disso, são usados por **outros** comandos do Git para realizar tarefas menores. Quando fazemos coisas mais incomuns como esta, eles permitem operações de nível realmente baixo, mas não foram feitos para o uso cotidiano. Você pode ler mais sobre comandos de encanamento em Encanamento e Porcelana (Plumbing and Porcelain).

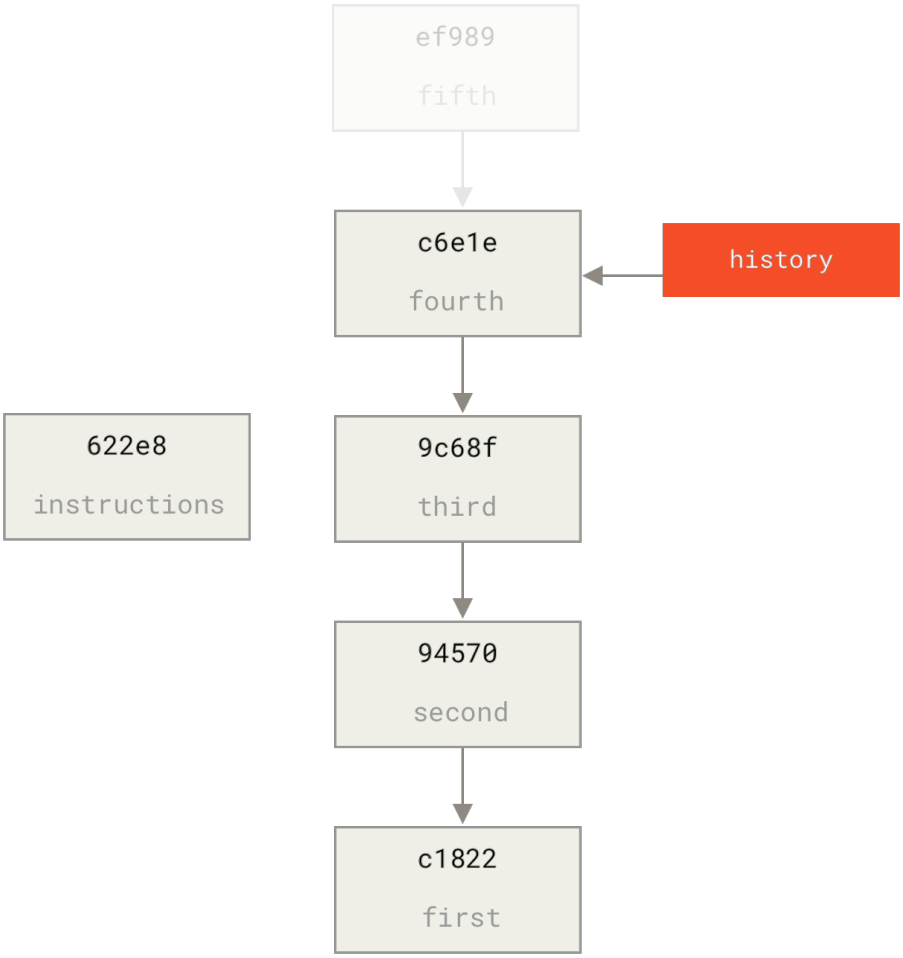


Figure 165. Criando um commit-base com `commit-tree`

Agora que temos um commit-base, podemos fazer rebase do restante do histórico sobre ele com `git rebase --onto`. O argumento `--onto` será o SHA-1 que acabamos de receber de `commit-tree`, e o ponto do rebase será o terceiro commit (o pai do primeiro commit que queremos manter, `9c68fdc`):

```
$ git rebase --onto 622e88 9c68fdc
First, rewinding head to replay your work on top of it...
Applying: fourth commit
Applying: fifth commit
```

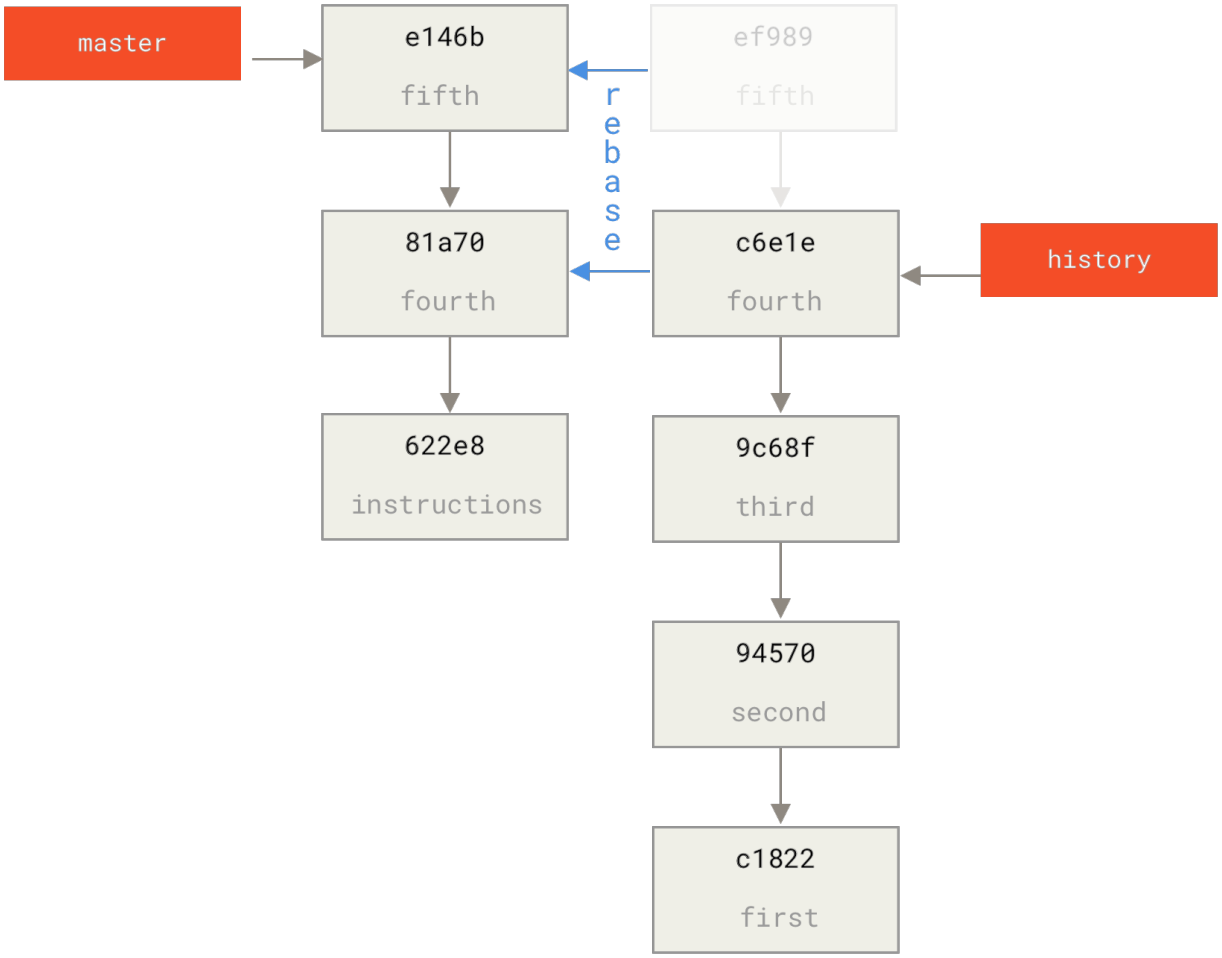


Figure 166. Fazendo rebase do histórico sobre o commit-base

Muito bem, agora reescrevemos nosso histórico recente sobre um commit-base descartável que contém instruções para reconstituir todo o histórico, caso quiséssemos. Podemos fazer push desse novo histórico para um novo projeto; quando alguém clonar esse repositório, verá apenas os dois commits mais recentes e, em seguida, um commit-base com instruções.

Agora vamos assumir o papel de alguém que clona o projeto pela primeira vez e quer obter todo o histórico. Para obter os dados do histórico depois de clonar esse repositório truncado, essa pessoa teria de adicionar um segundo remote para o repositório histórico e fazer fetch:

```
$ git clone https://github.com/schacon/project
$ cd project

$ git log --oneline master
e146b5f Fifth commit
81a708d Fourth commit
622e88e Get history from blah blah blah

$ git remote add project-history https://github.com/schacon/project-history
$ git fetch project-history
From https://github.com/schacon/project-history
```

```
 * [new branch]      master     -> project-history/master
```

Sendo desta forma o companheiro estaria retendo num braço de código dito `master` suas mais novas submissões (*commits*), mantendo as de enredo dantescas nesse que é o ramo nomeado de `project-history/master`.

```
$ git log --oneline master
e146b5f Fifth commit
81a708d Fourth commit
622e88e Get history from blah blah blah

$ git log --oneline project-history/master
c6e1e95 Fourth commit
9c68fdc Third commit
945704c Second commit
c1822cf First commit
```

A fim da união recíproca dos ditos arranjos, chame o `git replace` conjugando aquele *commit* tencionado em ser substituído com a peça do que em seu lugar fará a devida permuta. No presente ensaio o alvo de toda essa ação é aquele *quarto commit* abrigado na envergadura atrelada à branch `master`, que sairá afim do *commit quarto* advindo de `project-history/master`:

```
$ git replace 81a708d c6e1e95
```

Agora, se você examinar o histórico do branch `master`, ele terá esta aparência:

```
$ git log --oneline master
e146b5f Fifth commit
81a708d Fourth commit
9c68fdc Third commit
945704c Second commit
c1822cf First commit
```

Legal, não é? Sem precisar alterar todos os SHA-1s do upstream, conseguimos substituir um commit do histórico por outro completamente diferente, e todas as ferramentas comuns (`bisect`, `blame` etc.) funcionarão como esperado.

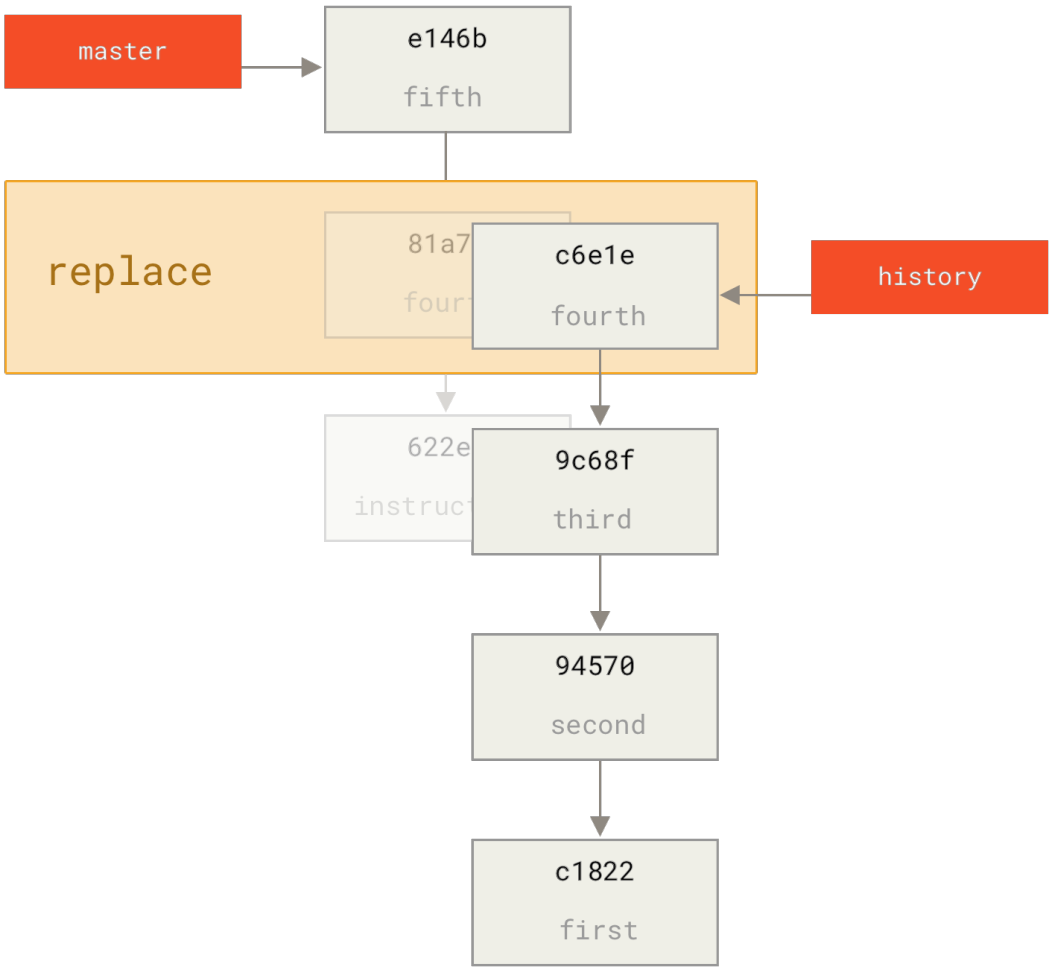


Figure 167. Combinando os commits com `git replace`

Curiosamente, ele ainda mostra `81a708d` como o SHA-1, embora esteja usando os dados do commit `c6e1e95` pelo qual o substituímos. Mesmo que você execute um comando como `cat-file`, ele mostrará os dados substitutos:

```
$ git cat-file -p 81a708d
tree 7bc544cf438903b65ca9104a1e30345eee6c083d
parent 9c68fdceee073230f19ebb8b5e7fc71b479c0252
author Scott Chacon <schacon@gmail.com> 1268712581 -0700
committer Scott Chacon <schacon@gmail.com> 1268712581 -0700

fourth commit
```

Lembre-se de que o pai real de `81a708d` era nosso commit substituto (`622e88e`), e não `9c68fdce`, como aparece aqui.

Outro aspecto interessante é que esses dados ficam armazenados em nossas referências:

```
$ git for-each-ref
e146b5f14e79d4935160c0e83fb9ebe526b8da0d commit refs/heads/master
```

```
c6e1e95051d41771a649f3145423f8809d1a74d4 commit refs/remotes/history/master
e146b5f14e79d4935160c0e83fb9ebe526b8da0d commit refs/remotes/origin/HEAD
e146b5f14e79d4935160c0e83fb9ebe526b8da0d commit refs/remotes/origin/master
c6e1e95051d41771a649f3145423f8809d1a74d4 commit
refs/replace/81a708dd0e167a3f691541c7a6463343bc457040
```

Isso significa que é fácil compartilhar nossa substituição com outras pessoas, pois podemos fazer push dela para o servidor e elas podem baixá-la facilmente. Isso não é tão útil no cenário de enxerto de histórico que examinamos aqui (afinal, todos baixariam ambos os históricos; então, por que separá-los?), mas pode ser útil em outras circunstâncias.

Armazenamento de Credenciais

Se você usa o transporte SSH para se conectar a remotes, pode ter uma chave sem frase secreta, o que permite transferir dados com segurança sem digitar seu nome de usuário e sua senha. No entanto, isso não é possível com os protocolos HTTP – toda conexão exige um nome de usuário e uma senha. Isso se torna ainda mais difícil em sistemas com autenticação de dois fatores, nos quais o token usado como senha é gerado aleatoriamente e é impronunciável.

Felizmente, o Git possui um sistema de credenciais que pode ajudar com isso. O Git oferece algumas opções prontas para uso:

- O padrão é não armazenar nada em cache. Toda conexão solicitará seu nome de usuário e sua senha.
- O modo “cache” mantém as credenciais na memória por determinado período. Nenhuma senha é armazenada em disco, e elas são removidas do cache depois de 15 minutos.
- O modo “store” salva as credenciais em um arquivo de texto puro no disco, e elas nunca expiram. Isso significa que, enquanto você não alterar sua senha no host Git, nunca mais precisará digitar suas credenciais. A desvantagem dessa abordagem é que suas senhas ficam armazenadas sem criptografia em um arquivo de texto puro no seu diretório pessoal.
- Se você usa macOS, o Git inclui um modo “osxkeychain”, que armazena as credenciais no chaveiro seguro associado à sua conta do sistema. Esse método armazena as credenciais em disco, e elas nunca expiram, mas são criptografadas pelo mesmo sistema que armazena certificados HTTPS e dados de preenchimento automático do Safari.
- Se você usa Windows, pode habilitar o recurso **Git Credential Manager** ao instalar o Git for Windows ou instalar separadamente a versão mais recente do GCM como um serviço independente. Ele é semelhante ao helper “osxkeychain” descrito acima, mas usa o Gerenciador de Credenciais do Windows para controlar informações confidenciais. Ele também pode fornecer credenciais ao WSL1 ou WSL2. Consulte as instruções de instalação do GCM para obter mais informações.

Você pode escolher um desses métodos definindo um valor de configuração do Git:

```
$ git config --global credential.helper cache
```

Alguns desses helpers possuem opções. O helper “store” aceita um argumento `--file <path>`, que personaliza o local em que o arquivo de texto puro é salvo (o padrão é `~/.git-credentials`). O helper “cache” aceita a opção `--timeout <seconds>`, que altera o tempo durante o qual seu daemon permanece em execução (o padrão é “900”, ou 15 minutos). Este é um exemplo de como configurar o helper “store” com um nome de arquivo personalizado:

```
$ git config --global credential.helper 'store --file ~/.my-credentials'
```

O Git permite até mesmo configurar vários helpers. Ao procurar credenciais para um host específico, o Git consultará os helpers em ordem e parará assim que receber a primeira resposta. Ao salvar credenciais, o Git enviará o nome de usuário e a senha para **todos** os helpers da lista, e cada um poderá decidir o que fazer com eles. Este é o aspecto de um arquivo `.gitconfig` caso você tivesse um arquivo de credenciais em um pen drive, mas quisesse usar o cache em memória para evitar digitação quando o dispositivo não estivesse conectado:

```
[credential]
    helper = store --file /mnt/thumbdrive/.git-credentials
    helper = cache --timeout 30000
```

Por Baixo dos Panos

Como tudo isso funciona? O comando principal do Git para o sistema de helpers de credenciais é `git credential`, que recebe um comando como argumento e mais dados pela entrada padrão.

Talvez seja mais fácil entender com um exemplo. Suponha que um helper de credenciais tenha sido configurado e armazenado credenciais para `mygithost`. Esta sessão usa o comando “fill”, invocado quando o Git tenta encontrar credenciais para um host:

```
$ git credential fill ①
protocol=https ②
host=mygithost
③
protocol=https ④
host=mygithost
username=bob
password=s3cre7
$ git credential fill ⑤
protocol=https
host=unknownhost

Username for 'https://unknownhost': bob
Password for 'https://bob@unknownhost':
protocol=https
host=unknownhost
username=bob
password=s3cre7
```

① Esta é a linha de comando que inicia a interação.

② Em seguida, Git-credential aguarda dados na entrada padrão. Fornecemos as informações que conhecemos: o protocolo e o nome do host.

③ Uma linha em branco indica que a entrada terminou e que o sistema de credenciais deve responder com o que sabe.

④ Então, Git-credential assume o controle e grava na saída padrão as informações que encontrou.

⑤ Se nenhuma credencial for encontrada, o Git solicita ao usuário o nome de usuário e a senha e os devolve na saída padrão do processo que o invocou (aqui, ela está associada ao mesmo console).

Na verdade, o sistema de credenciais invoca um programa separado do próprio Git; qual programa e de que maneira dependem do valor de configuração `credential.helper`. Ele pode assumir várias formas:

Valor de Configuração	Comportamento
`foo`	Executa `git-credential-foo`
`foo -a --opt=bcd`	Executa `git-credential-foo -a --opt=bcd`
`/absolute/path/foo -xyz`	Executa `/absolute/path/foo -xyz`
`!f() { echo "password=s3cre7"; }; f`	O código depois de `!` é avaliado no shell

Portanto, os helpers descritos acima na verdade se chamam `git-credential-cache`, `git-credential-store` e assim por diante, e podemos configurá-los para receber argumentos de linha de comando. A forma geral é “git-credential-foo [args] <action>.” O protocolo de entrada e saída padrão é o mesmo de git-credential, mas esses helpers usam um conjunto de ações ligeiramente diferente:

- `get` é uma solicitação de um par de nome de usuário e senha.
- `store` é uma solicitação para salvar um conjunto de credenciais na memória desse helper.
- `erase` remove da memória desse helper as credenciais correspondentes às propriedades fornecidas.

Para as ações `store` e `erase`, nenhuma resposta é necessária (o Git a ignora de qualquer forma). Para a ação `get`, porém, o Git se interessa muito pelo que o helper tem a dizer. Se o helper não souber nada útil, poderá simplesmente encerrar sem produzir saída; se souber, deverá complementar as informações fornecidas com as que armazenou. A saída é tratada como uma série de atribuições; tudo o que for fornecido substituirá o que o Git já sabe.

Este é o mesmo exemplo anterior, mas sem passar por `git-credential` e usando diretamente `git-credential-store`:

```
$ git credential-store --file ~/git.store store ①
protocol=https
host=mygithost
username=bob
password=s3cre7
$ git credential-store --file ~/git.store get ②
```

```
protocol=https
host=mygithost

username=bob ③
password=s3cre7
```

① Aqui instruímos `git-credential-store` a salvar algumas credenciais: o nome de usuário “bob” e a senha “s3cre7” devem ser usados quando `https://mygithost` for acessado.

② Agora recuperaremos essas credenciais. Fornecemos as partes da conexão que já conhecemos (`https://mygithost`) e uma linha em branco.

③ `git-credential-store` responde com o nome de usuário e a senha que armazenamos acima.

Este é o conteúdo do arquivo `~/git.store`:

```
https://bob:s3cre7@mygithost
```

Ele é apenas uma série de linhas, cada uma contendo uma URL acompanhada de credenciais. Os helpers `osxkeychain` e `wincred` usam o formato nativo dos armazenamentos nos quais se apoiam, enquanto `cache` usa seu próprio formato em memória (que nenhum outro processo consegue ler).

Um Cache de Credenciais Personalizado

Como `git-credential-store` e programas semelhantes são separados do Git, não é difícil perceber que *qualquer* programa pode ser um helper de credenciais do Git. Os helpers fornecidos pelo Git atendem a muitos casos de uso comuns, mas não a todos. Por exemplo, suponha que sua equipe tenha credenciais compartilhadas por todos, talvez para fazer deploy. Elas ficam armazenadas em um diretório compartilhado, mas você não quer copiá-las para seu próprio armazenamento de credenciais, pois elas mudam com frequência. Nenhum dos helpers existentes atende a esse caso; vejamos o que seria necessário para criar o nosso. Esse programa precisa ter várias características importantes:

1. A única ação à qual precisamos prestar atenção é `get`; `store` e `erase` são operações de escrita, portanto simplesmente encerraremos sem erros ao recebê-las.
2. O formato do arquivo de credenciais compartilhadas é o mesmo usado por `git-credential-store`.
3. A localização desse arquivo é bastante padronizada, mas devemos permitir que o usuário passe um caminho personalizado por precaução.

Mais uma vez, escreveremos essa extensão em Ruby, mas qualquer linguagem funcionará, desde que o Git possa executar o produto final. Este é o código-fonte completo do nosso novo helper de credenciais:

```
#!/usr/bin/env ruby

require 'optparse'
```

```
path = File.expand_path '~/.git-credentials' ①
OptionParser.new do |opts|
    opts.banner = 'USAGE: git-credential-read-only [options] <action>'
    opts.on('-f', '--file PATH', 'Specify path for backing store') do |argpath|
        path = File.expand_path argpath
    end
end.parse!

exit(0) unless ARGV[0].downcase == 'get' ②
exit(0) unless File.exist? path

known = {} ③
while line = STDIN.gets
    break if line.strip == ''
    k,v = line.strip.split '=', 2
    known[k] = v
end

File.readlines(path).each do |fileline| ④
    prot,user,pass,host = fileline.scan(/^(.*?):\/\/(.*?):(.*?)@(.*)$/).first
    if prot == known['protocol'] and host == known['host'] and user ==
known['username'] then
        puts "protocol=#{prot}"
        puts "host=#{host}"
        puts "username=#{user}"
        puts "password=#{pass}"
        exit(0)
    end
end
```

① Aqui analisamos as opções da linha de comando, permitindo que o usuário especifique o arquivo de entrada. O padrão é `~/.git-credentials`.

② Esse programa só responde se a ação for `get` e o arquivo de armazenamento existir.

③ Esse laço lê a entrada padrão até encontrar a primeira linha em branco. As entradas são armazenadas no hash `known` para consulta posterior.

④ Esse laço lê o conteúdo do arquivo de armazenamento em busca de correspondências. Se o protocolo, o host e o nome de usuário de `known` corresponderem à linha, o programa imprimirá os resultados na saída padrão e encerrará.

Salvaremos nosso helper como `git-credential-read-only`, colocaremos o arquivo em algum local do `PATH` e o marcaremos como executável. Esta é a aparência de uma sessão interativa:

```
$ git credential-read-only --file=/mnt/shared/creds get
protocol=https
host=mygithost
username=bob

protocol=https
host=mygithost
```

```
username=bob
password=s3cre7
```

Como seu nome começa com “git-”, podemos usar a sintaxe simples no valor de configuração:

```
$ git config --global credential.helper 'read-only --file /mnt/shared/creds'
```

Como você pode ver, estender esse sistema é bastante simples e pode resolver alguns problemas comuns para você e sua equipe.

Resumo

Você viu várias ferramentas avançadas que permitem manipular seus commits e área de stage com mais precisão. Quando notar problemas, você deverá ser capaz de descobrir facilmente qual commit os introduziu, quando e por quem. Se você quiser usar subprojetos no seu projeto, você aprendeu como acomodar essas necessidades. Neste ponto, você deverá ser capaz de fazer a maioria das coisas no Git que precisará na linha de comando no dia a dia e se sentir confortável ao fazê-lo.

Customizando o Git

Até agora, cobrimos os conceitos básicos de como o Git funciona e como usá-lo, e introduzimos uma série de ferramentas que o Git fornece para ajudá-lo a usá-lo de forma fácil e eficiente. Neste capítulo, veremos como você pode fazer o Git operar de uma maneira mais personalizada, introduzindo várias definições de configuração importantes e o sistema de ganchos (hooks). Com essas ferramentas, é fácil fazer com que o Git funcione exatamente da maneira que você, sua empresa ou seu grupo precisa.

Configuração do Git

Como você viu rapidamente no Primeiros Passos, é possível ajustar opções da configuração do Git através do comando `git config`. E que uma das primeiras coisas que você fez com ele foi configurar o seu nome e endereço de email:

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

Neste momento você vai ficar sabendo de mais algumas opções bem interessantes e que também são passíveis de ajuste por essa via com vistas a customizar sua lida junto a esse Git.

Mas antes, uma rápida recapitulação: O Git lança mão de um conjunto de arquivos em que ficam retidas configurações a orientá-lo sobre perfis de comportamento para além do formato padrão, a fim de deixá-lo no jeito que você possa querer. O primeiro lugar em que o Git procura esses valores é o arquivo `[path]/etc/gitconfig` do sistema, que contém configurações aplicadas a todos os usuários do sistema e a todos os seus repositórios. Havendo que passar a opção `--system` ao `git config`, ele foca a ler e bem com na escrita com estrita destinação nas guisas deste arquivo em específico.

Em seguida, o Git procura o arquivo `~/.gitconfig` (ou `~/.config/git/config`), específico de cada usuário. Você dita ao Git para ler e ali despontar na lide sobre ele com escritas por ali, de molde através das passagens concernentes à via d'opção pautada por `--global`.

Por fim, o Git procura valores no arquivo de configuração do diretório Git (`.git/config`) do repositório que você está usando no momento. Esses valores são específicos desse único repositório e correspondem a passar a opção `--local` para `git config`. Se você não especificar o nível com o qual deseja trabalhar, esse será o padrão.

Cada um desses "níveis" (sistema, global e local) sobrescreve valores do nível anterior; assim, por exemplo, os valores em `.git/config` prevalecem sobre os de `[path]/etc/gitconfig`.



Os arquivos de configuração do Git são texto puro, portanto você também pode definir esses valores editando o arquivo manualmente e inserindo a sintaxe correta. No entanto, em geral é mais fácil executar o comando `git config`.

Configurações Básicas de Cliente

As opções de configuração reconhecidas pelo Git se dividem em duas categorias: as do lado do cliente e as do lado do servidor. A maioria das opções fica no lado do cliente e configura suas preferências pessoais de trabalho. O Git oferece *muitas* opções de configuração, mas grande parte delas só é útil em casos extremos específicos; abordaremos aqui apenas as opções mais comuns e úteis. Para ver uma lista de todas as opções reconhecidas pela sua versão do Git, execute:

```
$ man git-config
```

Esse comando lista todas as opções disponíveis com bastante detalhe. Você também encontra esse material de referência em https://git-scm.com/docs/git-config.



Para casos de uso avançados, consulte "Conditional includes" na documentação mencionada acima.

`core.editor`

Por padrão, o Git usa o editor de texto que você definiu por meio de uma das variáveis de ambiente do shell, `VISUAL` ou `EDITOR`; se nenhuma estiver definida, usa o editor `vi` para criar e editar suas mensagens de commit e tag. Para trocar esse padrão por outro editor, use a configuração `core.editor`:

```
$ git config --global core.editor emacs
```

Agora, independentemente do editor padrão definido no shell, o Git iniciará o Emacs para editar as mensagens.

`commit.template`

Se você definir esse valor como o caminho de um arquivo no sistema, o Git usará esse arquivo como mensagem inicial padrão ao criar um commit. A vantagem de criar um modelo de commit personalizado é poder usá-lo para lembrar a si mesmo (ou a outras pessoas) do formato e estilo adequados ao escrever uma mensagem de commit.

Por exemplo, considere um arquivo de modelo em `~/.gitmessage.txt` com este conteúdo:

```
Subject line (try to keep under 50 characters)

Multi-line description of commit,
feel free to be detailed.

[Ticket: X]
```

Observe como esse modelo lembra o autor do commit de manter a linha de assunto curta (para beneficiar a saída de `git log --oneline`), acrescentar mais detalhes abaixo e mencionar o número de uma issue ou de um ticket do rastreador de bugs, caso exista.

Para instruir o Git a usá-lo como a mensagem padrão exibida no editor ao executar `git commit`, defina o valor de configuração `commit.template`:

```
$ git config --global commit.template ~/.gitmessage.txt
$ git commit
```

Então, ao fazer um commit, seu editor será aberto com uma mensagem provisória semelhante a esta:

```
Subject line (try to keep under 50 characters)

Multi-line description of commit,
feel free to be detailed.

[Ticket: X]
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# modified:   lib/test.rb
#
~
~
".git/COMMIT_EDITMSG" 14L, 297C
```

Se sua equipe tem uma política para mensagens de commit, colocar no sistema um modelo dessa política e configurar o Git para usá-lo por padrão pode aumentar a probabilidade de ela ser seguida regularmente.

core.pager

Essa configuração determina qual paginador é usado quando o Git pagina a saída de comandos como `log` e `diff`. Você pode defini-la como `more` ou como seu paginador preferido (o padrão é `less`), ou pode desativá-la definindo uma string vazia:

```
$ git config --global core.pager ''
```

Se você executar esse comando, o Git imprimirá toda a saída de todos os comandos, independentemente do comprimento.

user.signingkey

Se você cria tags anotadas e assinadas (como discutido em Assinando Seu Trabalho), definir sua chave de assinatura GPG na configuração simplifica o processo. Defina o ID da sua chave assim:

```
$ git config --global user.signingkey <gpg-key-id>
```

Agora você pode assinar tags sem precisar especificar sua chave toda vez que usar o comando `git tag`:

```
$ git tag -s <tag-name>
```

core.excludesfile

Você pode colocar padrões no arquivo `.gitignore` do projeto para que o Git não os veja como arquivos não rastreados nem tente adicioná-los ao stage ao executar `git add`, como discutido em Ignorando Arquivos.

No entanto, às vezes você deseja ignorar certos arquivos em todos os repositórios com os quais trabalha. Se seu computador executa macOS, você provavelmente conhece os arquivos `.DS_Store`. Se seu editor preferido é Emacs ou Vim, conhece nomes de arquivo que terminam em `~` ou `.swp`.

Essa configuração permite criar uma espécie de arquivo `.gitignore` global. Se você criar um arquivo `~/.gitignore_global` com este conteúdo:

```
*~
.*.swp
.DS_Store
```

...e executar `git config --global core.excludesfile ~/.gitignore_global`, o Git nunca mais o incomodará com esses arquivos.

help.autocorrect

Se você digitar um comando incorretamente, ele mostrará algo semelhante a isto:

```
$ git chekcout master
git: 'chekcout' is not a git command. See 'git --help'.

The most similar command is
    checkout
```

O Git tenta, de maneira prestativa, descobrir o que você quis dizer, mas ainda se recusa a executar o comando. Se você definir `help.autocorrect` como 1, o Git realmente executará o comando para você:

```
$ git chekcout master
WARNING: You called a Git command named 'chekcout', which does not exist.
Continuing under the assumption that you meant 'checkout'
in 0.1 seconds automatically...
```

Observe a menção a “0.1 seconds”. Na verdade, `help.autocorrect` é um número inteiro que representa décimos de segundo. Portanto, se você defini-lo como 50, o Git lhe dará 5 segundos para mudar de ideia antes de executar o comando corrigido automaticamente.

Cores no Git

O Git oferece suporte completo a saída colorida no terminal, o que facilita muito a análise visual rápida da saída dos comandos. Várias opções permitem ajustar as cores às suas preferências.

`color.ui`

O Git colore automaticamente a maior parte da saída, mas há uma configuração geral caso você não goste desse comportamento. Para desativar toda a saída colorida do Git no terminal, faça isto:

```
$ git config --global color.ui false
```

A configuração padrão é `auto`, que colore a saída quando ela vai diretamente para um terminal, mas omite os códigos de controle de cor quando a saída é redirecionada para um pipe ou arquivo.

Você também pode defini-la como `always` para ignorar a diferença entre terminais e pipes. Raramente isso será desejável; na maioria dos cenários, se quiser códigos de cor na saída redirecionada, você pode passar a flag `--color` ao comando Git para forçar seu uso. A configuração padrão quase sempre é a melhor opção.

`color.*`

Se quiser especificar com mais detalhes quais comandos recebem cores e de que maneira, o Git oferece configurações de cor específicas para cada comando. Cada uma delas pode ser definida como `true`, `false` ou `always`:

```
color.branch
color.diff
color.interactive
color.status
```

Além disso, cada uma possui subconfigurações que permitem definir cores específicas para partes da saída, caso você queira substituir cada cor. Por exemplo, para configurar as meta informações na sua saída do diff para o primeiro plano azul, fundo preto e texto em negrito, você pode executar:

```
$ git config --global color.diff.meta "blue black bold"
```

Você pode configurar a cor para qualquer um dos seguintes valores: `normal`, `black`, `red`, `green`, `yellow`, `blue`, `magenta`, `cyan` ou `white`. Se você quiser um atributo como negrito no exemplo anterior, pode escolher entre `bold`, `dim`, `ul` (sublinhado), `blink` e `reverse` (troca o primeiro plano e o fundo).

Ferramentas Externas de Merge e Diff

Embora o Git tenha uma implementação interna de diff, que é o que temos mostrado neste livro, você pode configurar uma ferramenta externa em vez disso. Você também pode configurar uma ferramenta gráfica de resolução de conflito de merge em vez de ter que resolver conflitos manualmente. Vamos demonstrar a configuração da Perforce Visual Merge Tool (P4Merge) para fazer seus diffs e resoluções de merge, porque é uma ferramenta gráfica agradável e é gratuita.

Se você quiser experimentar isso, o P4Merge funciona em todas as principais plataformas, então você deve ser capaz de fazer isso. Usaremos nomes de caminho nos exemplos que funcionam em sistemas macOS e Linux; para Windows, você terá que alterar `/usr/local/bin` para um caminho executável em seu ambiente.

Para começar, baixe o P4Merge da Perforce. Em seguida, você configurará scripts wrapper externos para executar seus comandos. Usaremos o caminho do macOS para o executável; em outros sistemas, será onde o seu binário `p4merge` está instalado. Configure um script wrapper de merge chamado `extMerge` que chama o seu binário com todos os argumentos fornecidos:

```
$ cat /usr/local/bin/extMerge
#!/bin/sh
/Applications/p4merge.app/Contents/MacOS/p4merge $*
```

O wrapper diff verifica se sete argumentos são fornecidos e passa dois deles para o seu script de merge. Por padrão, o Git passa os seguintes argumentos para o programa diff:

```
path old-file old-hex old-mode new-file new-hex new-mode
```

Como você só deseja os argumentos `old-file` e `new-file`, você usa o script wrapper para passar os que precisa.

```
$ cat /usr/local/bin/extDiff
#!/bin/sh
[ $# -eq 7 ] && /usr/local/bin/extMerge "$2" "$5"
```

Você também precisa se certificar de que essas ferramentas são executáveis:

```
$ sudo chmod +x /usr/local/bin/extMerge
$ sudo chmod +x /usr/local/bin/extDiff
```

Agora você pode configurar seu arquivo de configuração para usar suas ferramentas personalizadas de resolução de merge e diff. Isso exige várias configurações personalizadas: `merge.tool` para dizer ao Git qual estratégia usar, `mergetool.<tool>.cmd` para especificar como executar o comando, `mergetool.<tool>.trustExitCode` para dizer ao Git se o código de saída desse programa indica uma resolução de merge bem-sucedida ou não, e `diff.external` para dizer ao Git qual comando executar para diffs. Então, você pode executar quatro comandos de configuração:

```
$ git config --global merge.tool extMerge
$ git config --global mergetool.extMerge.cmd \
  'extMerge "$BASE" "$LOCAL" "$REMOTE" "$MERGED"'
$ git config --global mergetool.extMerge.trustExitCode false
$ git config --global diff.external extDiff
```

ou você pode editar o seu arquivo `~/.gitconfig` para adicionar estas linhas:

```
[merge]
  tool = extMerge
[mergetool "extMerge"]
  cmd = extMerge "$BASE" "$LOCAL" "$REMOTE" "$MERGED"
  trustExitCode = false
[diff]
  external = extDiff
```

Depois que tudo isso estiver configurado, se você executar comandos de diff como este:

```
$ git diff 32d1776b1^ 32d1776b1
```

Em vez de obter a saída do diff na linha de comando, o Git inicia o P4Merge, que se parece com isto:

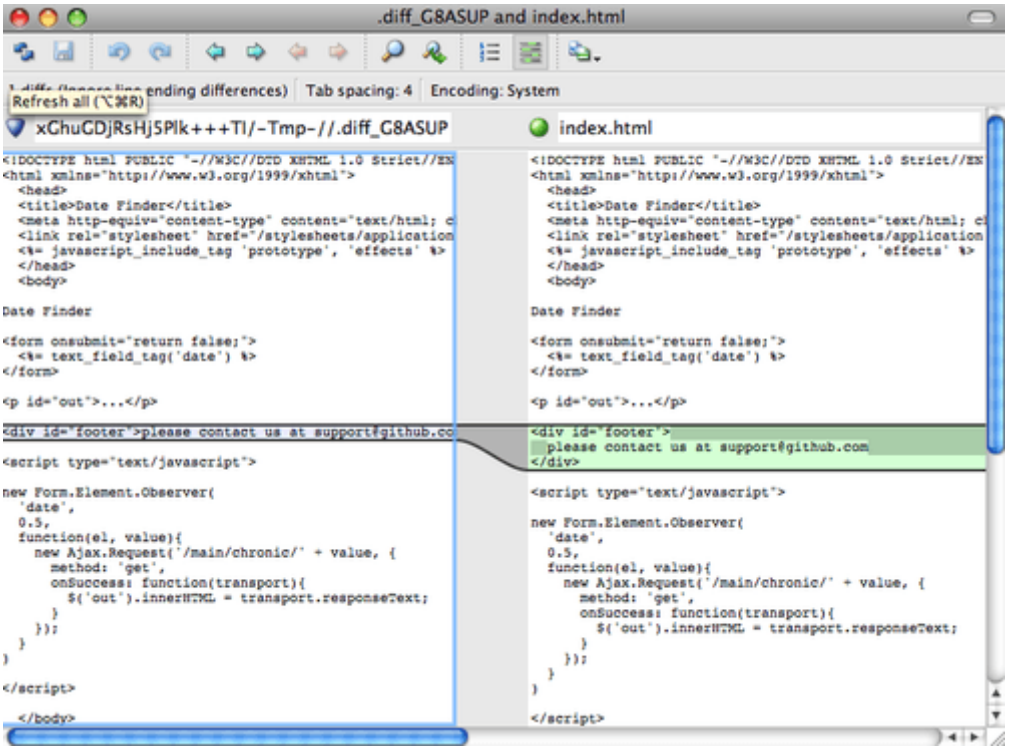


Figure 168. P4Merge

Se você tentar fazer o merge de duas branches e posteriormente tiver conflitos de merge, você pode executar o comando `git mergetool`; ele inicia o P4Merge para permitir que você resolva os conflitos através dessa ferramenta GUI.

A coisa boa sobre essa configuração de wrapper é que você pode alterar suas ferramentas de diff e merge facilmente. Por exemplo, para alterar suas ferramentas `extDiff` e `extMerge` para executarem a ferramenta KDiff3 em vez disso, tudo o que você tem que fazer é editar o seu arquivo `extMerge`:

```
$ cat /usr/local/bin/extMerge
#!/bin/sh
/Applications/kdiff3.app/Contents/MacOS/kdiff3 $*
```

Agora, o Git usará a ferramenta KDiff3 para a visualização de diffs e resolução de conflitos de merge.

O Git vem pré-configurado para usar várias outras ferramentas de resolução de conflitos sem que você precise configurar a configuração cmd. Para ver uma lista das ferramentas que ele suporta, tente o seguinte:

```
$ git mergetool --tool-help
'git mergetool --tool=<tool>' may be set to one of the following:
        emerge
        gvimdiff
        gvimdiff2
        opendiff
        p4merge
        vimdiff
        vimdiff2

The following tools are valid, but not currently available:
        araxis
        bc3
        codecompare
        deltawalker
        diffmerge
        diffuse
        ecmerge
        kdiff3
        meld
        tkdiff
        tortoisemerge
        xxdiff

Some of the tools listed above only work in a windowed
environment. If run in a terminal-only session, they will fail.
```

Se você não estiver interessado em usar o KDiff3 para diff, mas sim usá-lo apenas para a resolução de conflitos de merge, e o comando kdiff3 estiver no seu caminho, então você pode executar:

```
$ git config --global merge.tool kdiff3
```

Se você executar isso em vez de configurar os arquivos `extMerge` e `extDiff`, o Git usará o KDiff3 para resolução de conflitos de merge e a ferramenta diff normal do Git para diffs.

Formatação e Espaço em Branco

Problemas de formatação e espaço em branco são alguns dos problemas mais frustrantes e sutis que muitos desenvolvedores enfrentam ao colaborar, especialmente em plataformas cruzadas. É muito fácil que patches ou outro trabalho colaborativo introduzam alterações sutis de espaço em branco porque os editores as introduzem silenciosamente, e se os seus arquivos tocarem em um sistema Windows, suas terminações de linha podem ser substituídas. O Git tem algumas opções de configuração para ajudar com esses problemas.

`core.autocrlf`

Se você estiver programando no Windows e trabalhando com pessoas que não estão (ou vice-versa), provavelmente enfrentará problemas de terminação de linha em algum momento. Isso ocorre porque o Windows usa um caractere de retorno de carro e um caractere de alimentação de linha para novas linhas em seus arquivos, enquanto os sistemas macOS e Linux usam apenas o caractere de alimentação de linha. Este é um fato sutil, mas incrivelmente irritante do trabalho multiplataforma; muitos editores no Windows substituem silenciosamente as terminações de linha estilo LF existentes por CRLF, ou inserem ambos os caracteres de terminação de linha quando o usuário pressiona a tecla Enter.

O Git pode lidar com isso convertendo automaticamente as terminações de linha CRLF em LF quando você adiciona um arquivo ao índice, e vice-versa quando ele faz o checkout do código no seu sistema de arquivos. Você pode ativar essa funcionalidade com a configuração `core.autocrlf`. Se você estiver em uma máquina Windows, defina-a como `true` — isso converte as terminações LF em CRLF quando você faz o checkout do código:

```
$ git config --global core.autocrlf true
```

Se você estiver em um sistema Linux ou macOS que usa terminações de linha LF, você não vai querer que o Git as converta automaticamente quando você fizer o checkout de arquivos; no entanto, se um arquivo com terminações CRLF for introduzido acidentalmente, você pode querer que o Git o conserte. Você pode dizer ao Git para converter CRLF em LF no commit, mas não o contrário, configurando `core.autocrlf` como `input`:

```
$ git config --global core.autocrlf input
```

Essa configuração deve deixá-lo com terminações CRLF nos checkouts do Windows, mas terminações LF nos sistemas macOS e Linux e no repositório.

Se você for um programador Windows fazendo um projeto exclusivo para Windows, poderá desativar essa funcionalidade, registrando os retornos de carro no repositório, configurando o valor da configuração para `false`:

```
$ git config --global core.autocrlf false
```

core.whitespace

O Git vem pré-configurado para detectar e corrigir alguns problemas de espaço em branco. Ele pode procurar seis problemas principais de espaço em branco — três estão habilitados por padrão e podem ser desativados, e três estão desabilitados por padrão, mas podem ser ativados.

Os três que estão ativados por padrão são `blank-at-eol`, que procura espaços no final de uma linha; `blank-at-eof`, que detecta linhas em branco no final de um arquivo; e `space-before-tab`, que procura espaços antes de tabulações no início de uma linha.

Os três que estão desativados por padrão, mas podem ser ativados, são `indent-with-non-tab`, que procura linhas iniciadas por espaços em vez de tabulações (e é controlado pela opção `tabwidth`); `tab-in-indent`, que procura tabulações na parte de indentação de uma linha; e `cr-at-eol`, que informa ao Git que retornos de carro no final das linhas são aceitáveis.

Você pode dizer ao Git quais deles você deseja habilitar configurando `core.whitespace` para os valores que deseja ativar ou desativar, separados por vírgulas. Você pode desativar uma opção adicionando um `-` na frente do seu nome, ou usar o valor padrão, deixando-o fora da string de configuração por completo. Por exemplo, se você quiser que todos, exceto o `space-before-tab`, sejam definidos, você pode fazer isso (sendo `trailing-space` um atalho para cobrir tanto o `blank-at-eol` quanto o `blank-at-eof`):

```
$ git config --global core.whitespace \
    trailing-space,-space-before-tab,indent-with-non-tab,tab-in-indent,cr-at-eol
```

Ou você pode especificar apenas a parte de customização:

```
$ git config --global core.whitespace \
    -space-before-tab,indent-with-non-tab,tab-in-indent,cr-at-eol
```

O Git detectará esses problemas quando você executar um comando `git diff` e tentará colori-los para que você possa corrigi-los antes de fazer o commit. Ele também usará esses valores para ajudá-lo quando você aplicar patches com `git apply`. Quando estiver aplicando patches, você pode pedir ao Git para avisá-lo se ele estiver aplicando patches com os problemas de espaço em branco especificados:

```
$ git apply --whitespace=warn <patch>
```

Ou você pode fazer com que o Git tente corrigir o problema automaticamente antes de aplicar o patch:

```
$ git apply --whitespace=fix <patch>
```

Essas opções se aplicam ao comando `git rebase` também. Se você fez commit de problemas de espaço em branco, mas ainda não fez push para o upstream, pode executar `git rebase --whitespace=fix` para que o Git corrija automaticamente os problemas de espaço em branco enquanto reescreve os patches.

Configuração de Servidor

Não há tantas opções de configuração disponíveis para o lado do servidor do Git, mas há algumas interessantes das quais você pode querer tomar nota.

`receive.fsckObjects`

O Git é capaz de se certificar de que todo objeto recebido durante um push ainda corresponde ao seu checksum SHA-1 e aponta para objetos válidos. No entanto, ele não faz isso por padrão; é uma operação bastante custosa, e pode tornar a operação mais lenta, especialmente em repositórios ou pushes grandes. Se você quiser que o Git verifique a consistência do objeto em cada push, você pode forçá-lo a fazê-lo definindo `receive.fsckObjects` como verdadeiro:

```
$ git config --system receive.fsckObjects true
```

Agora, o Git verificará a integridade do seu repositório antes de cada push ser aceito, a fim de garantir que clientes defeituosos (ou maliciosos) não estejam introduzindo dados corrompidos.

`receive.denyNonFastForwards`

Se você fizer rebase de commits que já enviou e tentar fazer push novamente, ou tentar enviar para uma branch remota um commit que não contenha aquele para o qual a branch remota aponta atualmente, o envio será negado. Geralmente, essa é uma boa política; mas, no caso do rebase, você pode chegar à conclusão de que sabe o que está fazendo e forçar a atualização da branch remota utilizando a flag `-f` no seu comando de push.

Para dizer ao Git que recuse os pushes forçados, configure o `receive.denyNonFastForwards`:

```
$ git config --system receive.denyNonFastForwards true
```

A outra maneira de fazer isso é por meio de hooks de recebimento no lado do servidor, que abordaremos em breve. Essa abordagem permite que você faça coisas mais complexas, como negar os não-fast-forwards para um determinado subconjunto de usuários.

`receive.denyDeletes`

Uma forma de contornar a política `denyNonFastForwards` é o usuário excluir a branch e depois enviá-la novamente com a nova referência. Para evitar isso, defina `receive.denyDeletes` como true:

```
$ git config --system receive.denyDeletes true
```

Isto nega qualquer deleção de branches ou tags — nenhum usuário poderá fazê-lo. Para remover

branches remotas, você precisa excluir manualmente os arquivos de referência no servidor. Também há maneiras mais interessantes de fazer isso por usuário, por meio de ACLs, como você aprenderá em Uma Política de Exemplo Imposta pelo Git.

Atributos do Git

Algumas dessas configurações também podem ser especificadas para um caminho, para que o Git aplique essas configurações apenas para um subdiretório ou subconjunto de arquivos. Essas configurações específicas de caminho são chamadas de atributos do Git e são definidas em um arquivo `.gitattributes` em um dos seus diretórios (normalmente a raiz do projeto) ou no arquivo `.git/info/attributes`, caso você não queira incluir o arquivo de atributos nos commits do projeto.

Com os atributos, você pode, por exemplo, especificar estratégias de merge diferentes para arquivos ou diretórios individuais do projeto, dizer ao Git como calcular o diff de arquivos não textuais ou fazer o Git filtrar o conteúdo antes de armazená-lo ou extraí-lo. Nesta seção, você vai aprender sobre alguns dos atributos que você pode configurar nos caminhos do seu projeto Git e ver alguns exemplos do uso desse recurso na prática.

Arquivos Binários

Um truque legal para o qual você pode usar os atributos do Git é dizer a ele quais arquivos são binários (em casos onde ele não seria capaz de descobrir de outra forma) e dar a ele instruções especiais sobre como lidar com esses arquivos. Por exemplo, alguns arquivos de texto podem ser gerados por máquina e não sujeitos a diff, enquanto alguns arquivos binários podem passar por um diff. Você verá como dizer ao Git o que é o que.

Identificando Arquivos Binários

Alguns arquivos parecem com arquivos de texto, mas para todos os propósitos devem ser tratados como dados binários. Por exemplo, os projetos do Xcode no macOS contêm um arquivo terminado em `.pbxproj`, que é basicamente um conjunto de dados JSON (formato de dados JavaScript em texto puro) gravado em disco pelo IDE, que registra as suas configurações de build e assim por diante. Apesar de tecnicamente ser um arquivo de texto (por ser todo em UTF-8), você não vai querer tratá-lo como tal porque é, na verdade, um banco de dados leve – você não pode fazer um merge no conteúdo dele se duas pessoas o alterarem, e usar o diff geralmente não será de ajuda. O arquivo deve ser consumido por uma máquina. Em suma, você quer tratá-lo como um arquivo binário.

Para dizer ao Git para tratar todos os arquivos `pbxproj` como dados binários, adicione a seguinte linha ao seu arquivo `.gitattributes`:

```
*.pbxproj binary
```

Agora, o Git não tentará converter ou corrigir problemas de CRLF; nem tentará computar ou imprimir um diff para alterações nesse arquivo ao executar um `git show` ou um `git diff` no seu projeto.

Fazendo o Diff de Arquivos Binários

Você também pode usar a funcionalidade de atributos do Git para fazer o diff de arquivos binários com eficácia. Você faz isso dizendo ao Git como converter seus dados binários num formato de texto que pode ser comparado por meio do diff normal.

Primeiro, você usará essa técnica para resolver um dos problemas mais irritantes conhecidos pela humanidade: os documentos de controle de versão do Microsoft Word. Se você deseja fazer controle de versão em documentos do Word, você pode colocá-los em um repositório Git e fazer o commit de vez em quando; mas de que serve isso? Se você executar `git diff` normalmente, você verá apenas algo como isto:

```
$ git diff
diff --git a/chapter1.docx b/chapter1.docx
index 88839c4..4afcb7c 100644
Binary files a/chapter1.docx and b/chapter1.docx differ
```

Você não pode comparar as duas versões diretamente, a menos que as verifique e faça uma varredura manualmente, certo? Acontece que você pode fazer isso muito bem utilizando os atributos do Git. Coloque a linha a seguir no seu arquivo `.gitattributes`:

```
*.docx diff=word
```

Isso informa ao Git que qualquer arquivo correspondente a esse padrão (`.docx`) deve usar o filtro "word" quando você tentar visualizar um diff que contenha alterações. O que é o filtro "word"? Você terá que configurá-lo. Aqui você configurará o Git para usar o programa `docx2txt` para converter documentos do Word em arquivos de texto legíveis, nos quais ele fará o diff da maneira correta.

Primeiro, você vai ter que instalar o `docx2txt`; é possível baixá-lo a partir de https://sourceforge.net/projects/docx2txt. Siga as instruções presentes no arquivo `INSTALL` para colocá-lo em algum lugar que o seu shell possa achá-lo. No próximo passo, você deve criar um script wrapper para converter a saída para o formato que o Git espera. Crie um arquivo chamado `docx2txt` num local que fique visível no seu caminho, e adicione estes conteúdos:

```
#!/bin/bash
docx2txt.pl "$1" -
```

Não se esqueça de usar o comando `chmod a+x` no arquivo. Finalmente, você pode configurar o Git para usar este script:

```
$ git config diff.word.textconv docx2txt
```

Agora o Git sabe que, ao calcular um diff entre dois snapshots, deve passar todos os arquivos terminados em `.docx` pelo filtro "word", definido como o programa `docx2txt`. Isto faz um trabalho

eficaz elaborando ótimas versões em formato de texto para os seus arquivos do Word antes de tentar fazer um diff deles.

Aqui vai um exemplo: O Capítulo 1 deste livro foi convertido para o formato do Word e teve um commit num repositório Git. Depois um novo parágrafo foi adicionado. Veja o que o `git diff` irá mostrar:

```
$ git diff
diff --git a/chapter1.docx b/chapter1.docx
index 0b013ca..ba25db5 100644
--- a/chapter1.docx
+++ b/chapter1.docx
@@ -2,6 +2,7 @@
 This chapter will be about getting started with Git. We will begin at the beginning
by explaining some background on version control tools, then move on to how to get Git
running on your system and finally how to get it setup to start working with. At the
end of this chapter you should understand why Git is around, why you should use it and
you should be all setup to do so.
 1.1. About Version Control
 What is "version control", and why should you care? Version control is a system that
records changes to a file or set of files over time so that you can recall specific
versions later. For the examples in this book you will use software source code as the
files being version controlled, though in reality you can do this with nearly any type
of file on a computer.
+Testing: 1, 2, 3.
 If you are a graphic or web designer and want to keep every version of an image or
layout (which you would most certainly want to), a Version Control System (VCS) is a
very wise thing to use. It allows you to revert files back to a previous state, revert
the entire project back to a previous state, compare changes over time, see who last
modified something that might be causing a problem, who introduced an issue and when,
and more. Using a VCS also generally means that if you screw things up or lose files,
you can easily recover. In addition, you get all this for very little overhead.
 1.1.1. Local Version Control Systems
 Many people's version-control method of choice is to copy files into another
directory (perhaps a time-stamped directory, if they're clever). This approach is very
common because it is so simple, but it is also incredibly error prone. It is easy to
forget which directory you're in and accidentally write to the wrong file or copy over
files you don't mean to.
```

O Git nos diz, com sucesso e de forma sucinta, que nós adicionamos a string “Testing: 1, 2, 3.”, o que está correto. Não é algo perfeito – as mudanças de formatação não seriam apresentadas aqui – mas certamente isso funciona.

Outro problema interessante que você pode resolver dessa forma envolve comparar as diferenças entre arquivos de imagem. Uma maneira de fazer isso é passar os arquivos de imagem por um filtro que extraia suas informações EXIF – metadados registrados na maioria dos formatos de imagem. Se você baixar e instalar o programa `exiftool`, poderá usá-lo para converter suas imagens em texto sobre os metadados, de modo que pelo menos o diff mostre uma representação textual das alterações ocorridas. Adicione a seguinte linha ao seu arquivo `.gitattributes`:

```
*.png diff=exif
```

Configure o Git para usar esta ferramenta:

```
$ git config diff.exif.textconv exiftool
```

Caso você substitua uma imagem no seu projeto e execute o `git diff`, você verá algo como isto:

```
diff --git a/image.png b/image.png
index 88839c4..4afcb7c 100644
--- a/image.png
+++ b/image.png
@@ -1,12 +1,12 @@
 ExifTool Version Number         : 7.74
-File Size                       : 70 kB
-File Modification Date/Time     : 2009:04:21 07:02:45-07:00
+File Size                       : 94 kB
+File Modification Date/Time     : 2009:04:21 07:02:43-07:00
 File Type                       : PNG
 MIME Type                       : image/png
-Image Width                     : 1058
-Image Height                    : 889
+Image Width                     : 1056
+Image Height                    : 827
 Bit Depth                       : 8
 Color Type                      : RGB with Alpha
```

É possível que você veja facilmente que o tamanho do arquivo e as dimensões da imagem foram alterados.

Expansão de Palavras-Chave

A expansão de palavras-chave no estilo do SVN ou CVS costuma ser solicitada por desenvolvedores habituados a esses sistemas. O principal problema disso no Git é que você não pode modificar um arquivo com informações sobre o commit depois de criá-lo, pois o Git calcula primeiro o checksum do arquivo. No entanto, você pode inserir texto em um arquivo quando ele passa por checkout e removê-lo novamente antes que seja adicionado a um commit. Os atributos do Git lhe oferecem duas maneiras para se fazer isso.

Primeiro, você pode inserir automaticamente o checksum SHA-1 de um blob em um campo `$Id$` do arquivo. Se você definir esse atributo para um arquivo ou conjunto de arquivos, na próxima vez que fizer checkout dessa branch o Git substituirá esse campo pelo SHA-1 do blob. É importante observar que esse não é o SHA-1 do commit, mas o do próprio blob. Coloque a linha a seguir no seu arquivo `.gitattributes`:

```
*.txt ident
```

Acrescente uma referência `$Id$` a um arquivo de teste:

```
$ echo '$Id$' > test.txt
```

Da próxima vez que você fizer checkout nesse arquivo, o Git irá injetar o SHA-1 do blob:

```
$ rm test.txt
$ git checkout -- test.txt
$ cat test.txt
$Id: 42812b7653c7b88933f8a9d6cad0ca16714b9bb3 $
```

No entanto, esse resultado tem utilidade limitada. Se você já usou a substituição de palavras-chave no CVS ou no Subversion, sabe que é possível incluir uma marca de data – o SHA-1 não ajuda muito, pois é relativamente aleatório e, apenas olhando, não é possível dizer se um SHA-1 é mais antigo ou mais recente que outro.

Você pode criar seus próprios filtros para fazer substituições em arquivos durante o commit e o checkout. Eles são os chamados filtros do tipo “clean” e “smudge”. No arquivo `.gitattributes`, você pode definir um filtro para caminhos específicos e depois configurar scripts que processem os arquivos imediatamente antes do checkout (“smudge”, veja O filtro “smudge” é executado no checkout) e imediatamente antes de serem adicionados ao stage (“clean”, veja O filtro “clean” vai rodar na hora em que os arquivos forem pro stage). Esses filtros podem ser configurados para fazer todo tipo de coisa interessante.

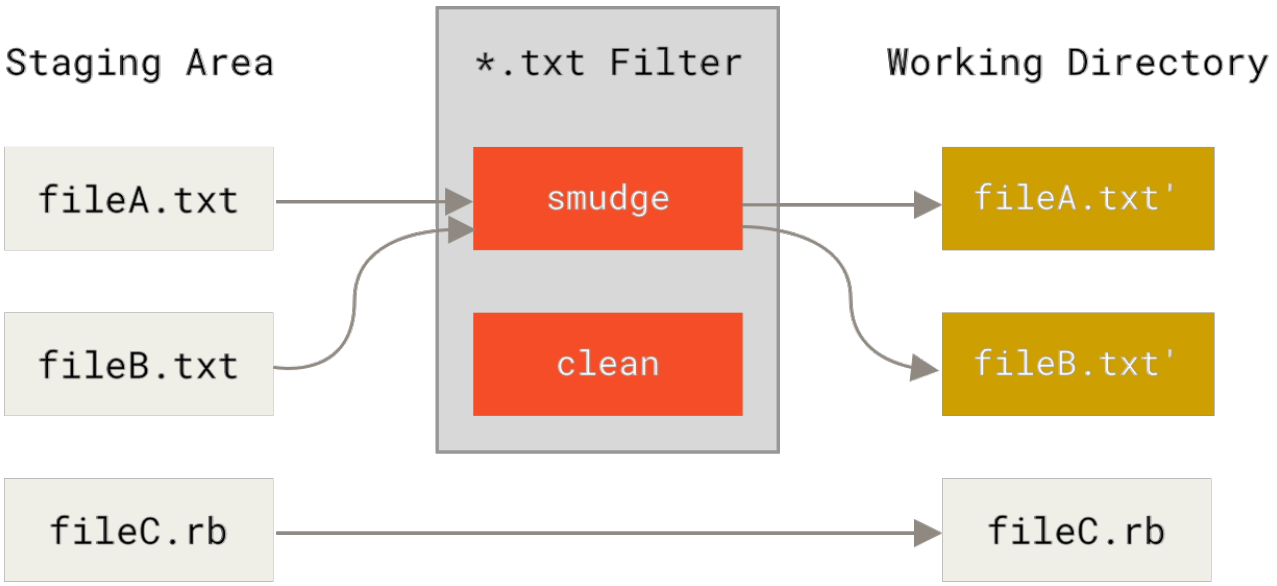


Figure 169. O filtro “smudge” é executado no checkout

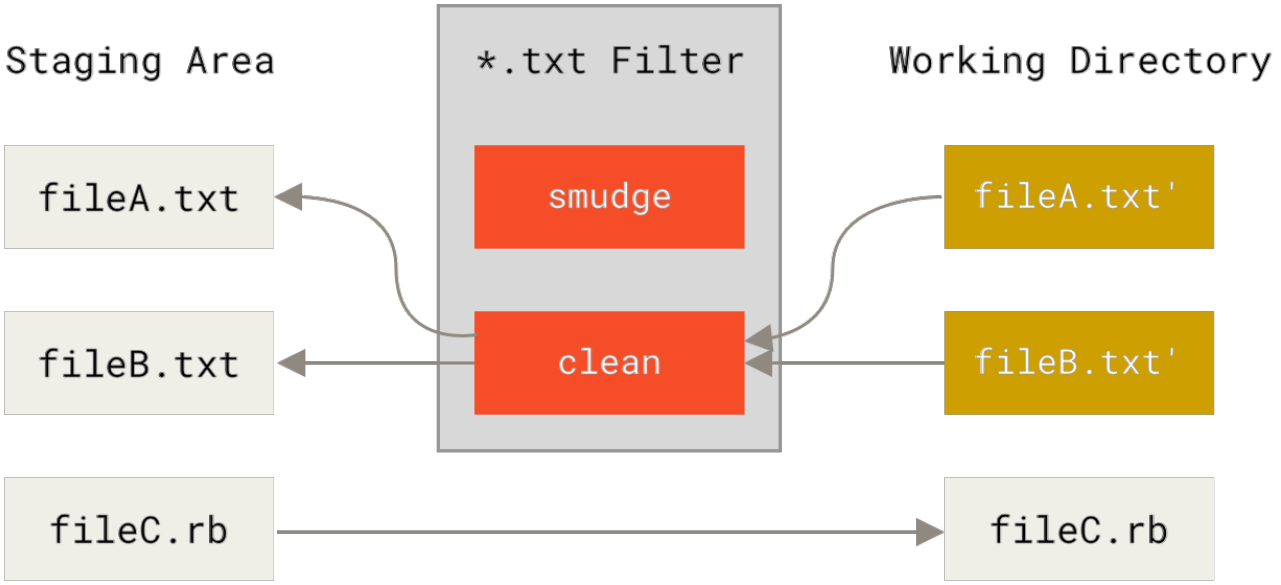


Figure 170. O filtro "clean" vai rodar na hora em que os arquivos forem pro stage

A mensagem de commit original desse recurso apresenta um exemplo simples que passa todo o código-fonte C pelo programa `indent` antes do commit. Para configurá-lo, defina no arquivo `.gitattributes` o atributo filter dos arquivos `\*.c` como o filtro "indent":

```
*.c filter=indent
```

Em seguida, informe ao Git o que o filtro "indent" deve fazer nas etapas smudge e clean:

```
$ git config --global filter.indent.clean indent
$ git config --global filter.indent.smudge cat
```

Nesse caso, quando você fizer commit de arquivos que correspondam a `*.c`, o Git os passará pelo programa `indent` antes de adicioná-los ao stage e depois pelo programa `cat` antes de colocá-los novamente no disco. O programa `cat` essencialmente não faz nada: ele devolve os mesmos dados que recebe. Essa combinação efetivamente filtra todos os arquivos de código-fonte C por meio do `indent` antes do commit.

Outro exemplo interessante implementa a expansão da palavra-chave `$Date$`, no estilo do RCS. Para fazer isso corretamente, você precisa de um pequeno script que receba o nome de um arquivo, descubra a data do último commit desse projeto e insira essa data no arquivo. Este pequeno script Ruby faz isso:

```
#! /usr/bin/env ruby
data = STDIN.read
last_date = `git log --pretty=format:"%ad" -1`
puts data.gsub('$Date$', '$Date: ' + last_date.to_s + '$')
```

Tudo o que o script faz é obter a data do commit mais recente com o comando `git log`, inserir essa

data em todas as strings `$Date$` que encontrar na entrada padrão e imprimir os resultados – deve ser simples fazer isso na linguagem com a qual você se sente mais à vontade. Você pode nomear esse arquivo como `expand_date` e colocá-lo no seu path. Agora, você precisa configurar um filtro no Git (chame-o de `dater`) e instruí-lo a usar seu filtro `expand_date` para processar os arquivos no checkout. Você usará uma expressão Perl para desfazer essa alteração no commit:

```
$ git config filter.dater.smudge expand_date
$ git config filter.dater.clean 'perl -pe "s/\\\$Date[^\\\$]*\\\$/\\\$Date\\\$/"'
```

Esse trecho de Perl remove tudo o que encontrar em uma string `$Date$`, retornando-a ao estado inicial. Agora que seu filtro está pronto, você pode testá-lo configurando um atributo do Git para esse arquivo, que aciona o novo filtro, e criando um arquivo com a palavra-chave `$Date$`:

```
date*.txt filter=dater
```

```
$ echo '# $Date$' > date_test.txt
```

Se você fizer commit dessas alterações e executar o checkout do arquivo novamente, verá a palavra-chave devidamente substituída:

```
$ git add date_test.txt .gitattributes
$ git commit -m "Test date expansion in Git"
$ rm date_test.txt
$ git checkout date_test.txt
$ cat date_test.txt
# $Date: Tue Apr 21 07:26:52 2009 -0700$
```

Você pode perceber como essa técnica pode ser poderosa em aplicações personalizadas. Você precisa ter cuidado, no entanto, porque o arquivo `.gitattributes` é commitado e repassado junto com o projeto, mas o driver (neste caso, `dater`) não é, então ele não funcionará em todos os lugares. Ao projetar esses filtros, eles devem ser capazes de falhar com elegância e fazer com que o projeto ainda funcione corretamente.

Exportando Seu Repositório

Os dados de atributos do Git também permitem que você faça algumas coisas interessantes ao exportar um arquivo do seu projeto.

`export-ignore`

Você pode dizer ao Git para não exportar certos arquivos ou diretórios ao gerar um arquivo. Se houver um subdiretório ou arquivo que você não queira incluir no arquivo compactado, mas queira manter no projeto, poderá identificá-lo com o atributo `export-ignore`.

Por exemplo, digamos que você tenha alguns arquivos de teste em um subdiretório `test/`, e não

faça sentido incluí-los na exportação tarball do seu projeto. Você pode adicionar a seguinte linha ao seu arquivo de atributos do Git:

```
test/ export-ignore
```

Agora, quando você executar o `git archive` para criar um tarball do seu projeto, esse diretório não será incluído no arquivo.

`export-subst`

Ao exportar arquivos para implantação, você pode aplicar a formatação e o processamento de expansão de palavra-chave do `git log` a partes selecionadas dos arquivos marcados com o atributo `export-subst`.

Por exemplo, se você quiser incluir um arquivo chamado `LAST_COMMIT` no seu projeto, e ter os metadados sobre o último commit injetados automaticamente nele quando o `git archive` for executado, você pode, por exemplo, configurar os seus arquivos `.gitattributes` e `LAST_COMMIT` assim:

```
LAST_COMMIT export-subst
```

```
$ echo 'Last commit date: $Format:%cd by %aN$' > LAST_COMMIT
$ git add LAST_COMMIT .gitattributes
$ git commit -am 'adding LAST_COMMIT file for archives'
```

Quando você executar o `git archive`, o conteúdo do arquivo arquivado ficará assim:

```
$ git archive HEAD | tar xCf ../deployment-testing -
$ cat ../deployment-testing/LAST_COMMIT
Last commit date: Tue Apr 21 08:38:48 2009 -0700 by Scott Chacon
```

As substituições podem incluir, por exemplo, a mensagem de commit e quaisquer `git notes`, e o `git log` pode fazer uma quebra de palavras simples:

```
$ echo '$Format:Last commit: %h by %aN at %cd%n%+w(76,6,9)%B$' > LAST_COMMIT
$ git commit -am 'export-subst uses git log'\''s custom formatter

git archive uses git log'\''s `pretty=format:` processor
directly, and strips the surrounding `$Format:` and `$`
markup from the output.
'
$ git archive @ | tar xfO - LAST_COMMIT
Last commit: 312ccc8 by Jim Hill at Fri May 8 09:14:04 2015 -0700
       export-subst uses git log's custom formatter

         git archive uses git log's `pretty=format:` processor directly, and
```

```
        strips the surrounding `$Format:` and `$` markup from the output.
```

O arquivo resultante é adequado para trabalho de implantação, mas como qualquer arquivo exportado, ele não é adequado para um trabalho de desenvolvimento posterior.

Estratégias de Merge

Você também pode usar os atributos do Git para dizer ao Git para usar estratégias de merge diferentes para arquivos específicos no seu projeto. Uma opção muito útil é dizer ao Git para não tentar fazer o merge de arquivos específicos quando eles tiverem conflitos, mas sim usar o seu lado do merge em vez do de outra pessoa.

Isso é útil se uma branch no seu projeto tiver divergido ou for especializada, mas você quiser ser capaz de fazer o merge de alterações de volta dela, e você quiser ignorar certos arquivos. Digamos que você tenha um arquivo de configurações de banco de dados chamado `database.xml` que é diferente em duas branches, e você deseja fazer o merge na sua outra branch sem bagunçar o arquivo do banco de dados. Você pode configurar um atributo como este:

```
database.xml merge=ours
```

Em seguida, defina uma estratégia de merge `ours` fictícia com:

```
$ git config --global merge.ours.driver true
```

Se você fizer o merge na outra branch, em vez de ter conflitos de merge com o arquivo `database.xml`, você verá algo como isto:

```
$ git merge topic
Auto-merging database.xml
Merge made by recursive.
```

Neste caso, o `database.xml` permanece na versão que você tinha originalmente.

Hooks do Git

Como muitos outros Sistemas de Controle de Versão, o Git tem uma maneira de disparar scripts customizados quando certas ações importantes ocorrem. Existem dois grupos desses hooks: os do lado do cliente (client-side) e os do lado do servidor (server-side). Hooks do lado do cliente são acionados por operações como fazer commits e merges, enquanto os hooks do lado do servidor rodam em operações de rede, como o recebimento de commits que sofreram push. Você pode usar esses hooks por toda a sorte de motivos.

Instalando um Hook

Todos os hooks são armazenados no subdiretório `hooks` do diretório Git. Na maior parte dos

projetos, trata-se do `.git/hooks`. Quando você inicializa um novo repositório com `git init`, o Git popula o diretório hooks com um monte de scripts de exemplo, muitos dos quais chegam a ser úteis por si mesmos; mas eles também documentam os valores de entrada de cada script em si. Todos os exemplos são escritos como shell scripts, com algum Perl inserido, mas quaisquer scripts executáveis desde que devidamente nomeados funcionarão muito bem – você pode programá-los em Ruby ou Python ou em qualquer linguagem com a qual tenha familiaridade. Se você desejar utilizar os scripts de hook empacotados, você terá que renomeá-los; os nomes de arquivo de todos eles terminam com `.sample`.

Para habilitar um script de hook, coloque um arquivo no subdiretório `hooks` do seu diretório `.git` que seja nomeado adequadamente (sem nenhuma extensão) e seja executável. Desse ponto em diante, ele deverá ser chamado. Abordaremos a maioria dos nomes de arquivo dos principais hooks aqui.

Hooks do Lado do Cliente (Client-Side)

Há muitos hooks do lado do cliente. Esta seção os divide em hooks do fluxo de trabalho de commit, scripts do fluxo de trabalho de e-mail e tudo o mais.



É importante notar que os hooks do lado do cliente **não** são copiados quando você clona um repositório. Se a sua intenção com esses scripts é impor uma política, você provavelmente vai querer fazer isso no lado do servidor; veja o exemplo em Uma Política de Exemplo Imposta pelo Git.

Hooks de Fluxo de Trabalho de Commit

Os primeiros quatro hooks têm a ver com o processo de commit.

O hook `pre-commit` é executado primeiro, antes mesmo de você digitar uma mensagem de commit. Ele é usado para inspecionar o snapshot que está prestes a sofrer o commit, para ver se você esqueceu algo, para ter a certeza de que os testes rodam ou para examinar o que quer que você precise inspecionar no código. Se você sair com um status diferente de zero (non-zero) deste hook, aborta-se o commit, embora você possa contorná-lo com `git commit --no-verify`. Você pode fazer coisas como checar o estilo do código (rodar o `lint` ou algo equivalente), checar espaços em branco à direita (trailing whitespace - o hook padrão faz exatamente isso) ou verificar se há uma documentação apropriada nos novos métodos.

O hook `prepare-commit-msg` é rodado antes que o editor de mensagem de commit seja iniciado, mas depois que a mensagem padrão é criada. Ele permite que você edite a mensagem padrão antes que o autor do commit a veja. Esse hook aceita alguns parâmetros: o caminho para o arquivo que contém a mensagem de commit até o momento, o tipo de commit e o SHA-1 do commit, se este for um commit alterado (amended commit). Esse hook geralmente não é útil para commits normais; em vez disso, é bom para commits onde a mensagem padrão é gerada automaticamente, como em mensagens de commit de templates, commits de merge, commits esmagados (squashed commits) e commits alterados (amended commits). Você pode usá-lo em conjunto com um template de commit para inserir informações programaticamente.

O hook `commit-msg` aceita um parâmetro, que é novamente o caminho para um arquivo temporário que contém a mensagem de commit escrita pelo desenvolvedor. Se esse script sair com status não-

zero (non-zero), o Git aborta o processo de commit, portanto, você pode usá-lo para validar o estado do seu projeto ou a mensagem de commit antes de permitir que um commit seja concluído. Na última seção deste capítulo, nós demonstraremos o uso deste hook para checar se a sua mensagem de commit está em conformidade com um padrão exigido.

Depois que todo o processo de commit é completado, o hook `post-commit` roda. Ele não recebe nenhum parâmetro, mas você pode facilmente obter o último commit rodando `git log -1 HEAD`. Geralmente, esse script é usado para notificação ou algo parecido.

Hooks do Fluxo de Trabalho de E-mail

Você pode configurar três hooks do lado do cliente para um fluxo de trabalho baseado em e-mail. Todos eles são invocados pelo comando `git am`, portanto, se você não está usando esse comando no seu fluxo de trabalho, você pode pular tranquilamente para a próxima seção. Se você estiver pegando patches por e-mail preparados por `git format-patch`, então alguns destes podem lhe ser úteis.

O primeiro hook que é rodado é o `applypatch-msg`. Ele recebe um único argumento: o nome do arquivo temporário que contém a mensagem de commit proposta. O Git aborta o patch se este script sair com status diferente de zero (non-zero). Você pode usá-lo para ter a certeza de que uma mensagem de commit está devidamente formatada ou para normalizar a mensagem fazendo com que o script a edite ali no local (in place).

O próximo hook a rodar quando se aplica patches via `git am` é o `pre-applypatch`. De uma forma um tanto confusa, ele roda *após* o patch ser aplicado, mas antes que um commit seja feito, então você pode usá-lo para inspecionar o snapshot antes de fazer o commit. Você pode rodar testes ou de alguma outra forma inspecionar a árvore de trabalho com este script. Se algo estiver faltando ou se os testes não passarem, a saída num não-zero aborta o script do `git am` sem fazer o commit do patch.

O último hook a rodar durante uma operação `git am` é o `post-applypatch`, que roda após o commit ser feito. Você pode usá-lo para notificar um grupo ou o autor do patch do qual você fez pull de que você fez isso. Você não tem como parar o processo de patch com este script.

Outros Hooks de Cliente

O hook `pre-rebase` roda antes de você fazer o rebase de qualquer coisa e pode interromper o processo saindo com um não-zero (non-zero). Você pode usar este hook para não permitir o rebase de quaisquer commits que já tenham sido alvo de push. O hook `pre-rebase` de exemplo que o Git instala faz isso, embora ele faça algumas suposições que podem não bater (match) com o seu fluxo de trabalho.

O hook `post-rewrite` é rodado por comandos que substituem commits, como `git commit --amend` e `git rebase` (embora não pelo `git filter-branch`). Seu argumento único é qual comando engatilhou (triggered) a reescrita, e ele recebe uma lista de reescritas no `stdin`. Este hook tem muitos dos mesmos usos que os hooks `post-checkout` e `post-merge`.

Após você rodar um `git checkout` com sucesso, o hook `post-checkout` roda; você pode usá-lo para configurar apropriadamente seu diretório de trabalho para o ambiente do seu projeto. Isso pode significar mover arquivos binários grandes que você não quer que passem por controle de versão

no código (source controlled), documentação gerada automaticamente ou algo nessa linha.

O hook `post-merge` roda após um comando `merge` bem-sucedido. Você pode usá-lo para restaurar dados na árvore de trabalho que o Git não pode rastrear, tais como dados de permissões. Este hook também (likewise) pode validar a presença de arquivos externos ao controle do Git que você possa querer copiar quando a árvore de trabalho mudar.

O hook `pre-push` roda durante o `git push`, após as refs do remoto terem sido atualizadas, mas antes de quaisquer objetos terem sido transferidos. Ele recebe o nome e o local do remoto como parâmetros e uma lista de refs que estão para serem atualizadas através da `stdin`. Você pode usá-lo para validar um conjunto de atualizações de ref antes que um push ocorra (um código de saída num despontar não-zero abortará o push).

O Git ocasionalmente faz coleta de lixo (garbage collection) como parte do seu funcionamento normal, invocando `git gc --auto`. O hook `pre-auto-gc` é invocado logo antes que a coleta de lixo ocorra, e pode ser usado para notificá-lo de que isso está acontecendo, ou para abortar a coleta se agora não for um bom momento.

Hooks do Lado do Servidor

Além dos hooks do lado do cliente, você pode usar dois (a couple of) hooks importantes do lado do servidor como um administrador de sistema para impor (enforce) quase qualquer tipo de política no seu projeto. Esses scripts rodam antes e depois dos pushes para o servidor. Os hooks de pré (pre hooks) podem sair num despontar não-zero (exit non-zero) a qualquer momento para rejeitar o push, bem como imprimir uma mensagem de erro de volta para o cliente; você pode configurar uma política de push que seja tão complexa quanto desejar.

`pre-receive`

O primeiro script a rodar ao lidar com um push de um cliente é o `pre-receive`. Ele pega (takes) uma lista das referências que estão sofrendo push vindas da stdin; se ele sair com não-zero (non-zero), nenhuma delas será aceita. Você pode usar este hook para fazer coisas como garantir que nenhuma das referências atualizadas seja non-fast-forward, ou para fazer controle de acesso para todas as refs e arquivos que eles estejam a modificar com o push.

`update`

O script `update` é muito semelhante ao script `pre-receive`, exceto que ele é rodado uma vez para cada branch que o encarregado pelo push (pusher) esteja tentando atualizar. Se o pusher estiver tentando fazer o push para várias branches, o `pre-receive` roda somente uma vez, enquanto que o `update` roda uma vez por branch para a qual eles estejam fazendo o push. Em vez de ler do stdin, este script aceita (takes) três argumentos: o nome da referência (branch), o SHA-1 para onde a referência apontava antes do push, e o SHA-1 que o usuário está tentando fazer o push. Se o script `update` sair com não-zero, somente aquela referência é rejeitada; as outras referências ainda podem ser atualizadas.

`post-receive`

O hook `post-receive` roda depois que todo o processo for concluído e pode ser usado para atualizar outros serviços ou notificar os usuários. Ele aceita os mesmos dados pelo stdin que o hook `pre-`

`receive`. Exemplos disso incluem mandar um e-mail para uma lista, notificar um servidor de integração contínua (continuous integration) ou atualizar um sistema de rastreamento de tickets (ticket-tracking system) – você pode até mesmo analisar (parse) as mensagens de commit para ver se quaisquer tickets precisam vir a ser abertos, modificados ou fechados. Este script não tem como parar o processo de push, mas o cliente não é desconectado até que ele tenha sido concluído; portanto, tenha cuidado se você tentar fazer qualquer coisa que possa tomar muito tempo.



Se você estiver escrevendo um script/hook que outros precisarão ler, dê preferência às versões mais longas das flags de linha de comando; daqui a seis meses você irá nos agradecer por isso.

Uma Política de Exemplo Imposta pelo Git

Nesta seção, você usará o que aprendeu para estabelecer um fluxo de trabalho do Git que verifica um formato de mensagem de commit personalizado e permite que apenas determinados usuários modifiquem determinados subdiretórios em um projeto. Você construirá scripts do cliente que ajudam o desenvolvedor a saber se seu push será rejeitado e scripts do servidor que de fato impõem as políticas.

Os scripts que mostraremos são escritos em Ruby; em parte por causa de nossa inércia intelectual, mas também porque o Ruby é fácil de ler, mesmo que você não saiba necessariamente escrevê-lo. No entanto, qualquer linguagem funcionará – todos os scripts de gancho de amostra distribuídos com o Git estão em Perl ou Bash; portanto, você também pode ver muitos exemplos de ganchos nessas linguagens examinando as amostras.

Gancho no Lado do Servidor

Todo o trabalho no lado do servidor irá para o arquivo `update` no seu diretório `hooks`. O gancho `update` é executado uma vez por branch que sofre push e recebe três argumentos:

- O nome da referência que sofre push
- A revisão antiga de onde esse branch estava
- A nova revisão sendo enviada (push)

Você também tem acesso ao usuário que faz o push se o push estiver sendo executado via SSH. Se você permitiu que todos se conectassem a um único usuário (como “git”) por meio de autenticação de chave pública, pode ser necessário fornecer a esse usuário um invólucro do shell (shell wrapper) que determine qual usuário está se conectando com base na chave pública e definir uma variável de ambiente conforme necessário. Aqui vamos assumir que o usuário conectado está na variável de ambiente `$USER`; portanto, o script de atualização (update) começa reunindo todas as informações necessárias:

```
#!/usr/bin/env ruby

$refname = ARGV[0]
$oldrev  = ARGV[1]
$newrev  = ARGV[2]
```

```
$user    = ENV['USER']

puts "Enforcing Policies..."
puts "(#{$refname}) (#{$oldrev[0,6]}) (#{$newrev[0,6]})"
```

Sim, essas são variáveis globais. Não julgue – é mais fácil demonstrar dessa maneira.

Impondo um Formato de Mensagem de Commit Específico

Seu primeiro desafio é impor que cada mensagem de commit siga a um formato específico. Apenas para ter um objetivo, assuma que cada mensagem deve incluir uma string que se pareça com "ref: 1234" porque você deseja que cada commit seja vinculado a um item de trabalho em seu sistema de bilhetagem (ticketing). Você deve examinar cada commit sendo enviado via push, ver se essa string está na mensagem de commit e, se a string estiver ausente de algum dos commits, sair sem zero, para que o push seja rejeitado.

Você pode obter uma lista dos valores de SHA-1 de todos os commits que sofrem push obtendo os valores `$newrev` e `$oldrev` e passando-os a um comando de encanamento do Git chamado `git rev-list`. Este é basicamente o comando `git log`, mas por padrão ele imprime apenas os valores do SHA-1 e nenhuma outra informação. Portanto, para obter uma lista de todos os SHA-1 de commit introduzidos entre um SHA-1 de commit e outro, você pode executar algo como isto:

```
$ git rev-list 538c33..d14fc7
d14fc7c847ab946ec39590d87783c69b031bdfb7
9f585da4401b0a3999e84113824d15245c13f0be
234071a1be950e2a8d078e6141f5cd20c1e61ad3
dfa04c9ef3d5197182f13fb5b9b1fb7717d2222a
17716ec0f1ff5c77eff40b7fe912f9f6cfd0e475
```

Você pode obter essa saída, percorrer cada um desses SHA-1s de commit, pegar a mensagem para ele e testar essa mensagem contra uma expressão regular que procura por um padrão.

Você precisa descobrir como obter a mensagem de commit de cada um desses commits para testar. Para obter os dados brutos de commit, você pode usar outro comando de encanamento chamado `git cat-file`. Analisaremos todos esses comandos de encanamento em detalhes em Git Internals (Por Dentro do Git); mas, por enquanto, eis o que esse comando fornece a você:

```
$ git cat-file commit ca82a6
tree cfda3bf379e4f8dba8717dee55aab78aef7f4daf
parent 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
author Scott Chacon <schacon@gmail.com> 1205815931 -0700
committer Scott Chacon <schacon@gmail.com> 1240030591 -0700

Change the version number
```

Uma maneira simples de obter a mensagem de commit de um commit quando você tem o valor de SHA-1 é ir para a primeira linha em branco e levar tudo a partir de lá. Você pode fazer isso com o

comando `sed` nos sistemas Unix:

```
$ git cat-file commit ca82a6 | sed '1,/^$/d'
Change the version number
```

Você pode usar essa técnica para pegar a mensagem de commit de cada commit que estiver tentando fazer push e sair se você ver algo que não corresponde. Para sair do script e rejeitar o push, saia sem zero. O método inteiro se parece com isso:

```
$regex = /\[ref: (\d+)\]/

# enforced custom commit message format
def check_message_format
  missed_revs = `git rev-list #{$oldrev}..#{$newrev}`.split("\n")
  missed_revs.each do |rev|
    message = `git cat-file commit #{rev} | sed '1,/^$/d'`
    if !$regex.match(message)
      puts "[POLICY] Your message is not formatted correctly"
      exit 1
    end
  end
end
check_message_format
```

Colocar isso em seu script `update` rejeitará atualizações que contenham commits que possuam mensagens que não estejam de acordo com sua regra.

Impondo um Sistema ACL Baseado em Usuário

Suponha que você queira adicionar um mecanismo que use uma lista de controle de acesso (ACL - Access Control List) que especifica quais usuários têm permissão para fazer push das alterações para quais partes de seus projetos. Algumas pessoas têm acesso total e outras podem fazer push das alterações apenas a determinados subdiretórios ou arquivos específicos. Para impor isso, você gravará essas regras em um arquivo chamado `acl` que reside no seu repositório Git bare no servidor. Você fará com que o gancho `update` analise essas regras, veja quais arquivos estão sendo introduzidos para todos os commits sendo submetidos a um push e determine se o usuário que realiza o push tem acesso para atualizar todos esses arquivos.

A primeira coisa que você fará será escrever sua ACL. Aqui você usará um formato muito parecido com o mecanismo de ACL do CVS: ele usa uma série de linhas, onde o primeiro campo é `avail` (disponível) ou `unavail` (indisponível), o próximo campo é uma lista delimitada por vírgula dos usuários aos quais a regra se aplica, e o último campo é o caminho ao qual a regra se aplica (em branco significa acesso aberto). Todos esses campos são delimitados por um caractere pipe (`|`).

Nesse caso, você tem alguns administradores, alguns escritores de documentação com acesso ao diretório `doc` e um desenvolvedor que só tem acesso aos diretórios `lib` e `tests`, e seu arquivo ACL se parece com este:

```
avail|nickh,pjhyett,defunkt,tpw
avail|usinclair,cdickens,ebronte|doc
avail|schacon|lib
avail|schacon|tests
```

Você começa com a leitura desses dados em uma estrutura que você pode usar. Nesse caso, para manter o exemplo simples, você só executará as diretivas `avail`. Aqui está um método que fornece um array associativo em que a chave é o nome de usuário e o valor é um array de caminhos para o qual o usuário tem acesso de gravação:

```
def get_acl_access_data(acl_file)
  # read in ACL data
  acl_file = File.read(acl_file).split("\n").reject { |line| line == '' }
  access = {}
  acl_file.each do |line|
    avail, users, path = line.split('|')
    next unless avail == 'avail'
    users.split(',').each do |user|
      access[user] ||= []
      access[user] << path
    end
  end
  access
end
```

No arquivo ACL examinado anteriormente, esse método `get_acl_access_data` retorna uma estrutura de dados semelhante a esta:

```
{"defunkt"=>[nil],
 "tpw"=>[nil],
 "nickh"=>[nil],
 "pjhyett"=>[nil],
 "schacon"=>["lib", "tests"],
 "cdickens"=>["doc"],
 "usinclair"=>["doc"],
 "ebronte"=>["doc"]}
```

Agora que você classificou as permissões, você precisa determinar quais caminhos os commits submetidos a push foram modificados, para garantir que o usuário que estiver fazendo o push tenha acesso a todos eles.

Você pode ver facilmente quais arquivos foram modificados em um único commit com a opção `--name-only` do comando `git log` (mencionado brevemente em Fundamentos do Git):

```
$ git log -1 --name-only --pretty=format:'' 9f585d
```

```
README
lib/test.rb
```

Se você usar a estrutura da ACL retornada pelo método `get_acl_access_data` e a verificar com os arquivos listados em cada um dos commits, poderá determinar se o usuário tem acesso a todos os seus commits de push:

```
# only allows certain users to modify certain subdirectories in a project
def check_directory_perms
  access = get_acl_access_data('acl')

  # see if anyone is trying to push something they can't
  new_commits = `git rev-list #{$oldrev}..#{$newrev}`.split("\n")
  new_commits.each do |rev|
    files_modified = `git log -1 --name-only --pretty=format:'' #{rev}`.split("\n")
    files_modified.each do |path|
      next if path.size == 0
      has_file_access = false
      access[$user].each do |access_path|
        if !access_path  # user has access to everything
           || (path.start_with? access_path) # access to this path
          has_file_access = true
        end
      end
      if !has_file_access
        puts "[POLICY] You do not have access to push to #{path}"
        exit 1
      end
    end
  end
end

check_directory_perms
```

Você obtém uma lista de novos commits sendo enviados para o servidor com `git rev-list`. Em seguida, para cada um desses commits, você descobre quais arquivos foram modificados e verifica se o usuário que está fazendo push tem acesso a todos os caminhos sendo modificados.

Agora, seus usuários não podem fazer um push para qualquer commit com mensagens mal formadas ou arquivos modificados fora de seus caminhos designados.

Testando

Se você executar `chmod u+x .git/hooks/update`, que é o arquivo no qual você deve ter colocado todo esse código, e tentar dar um push de um commit com uma mensagem não compatível, você obterá algo parecido com isto:

```
$ git push -f origin master
```

```
Counting objects: 5, done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 323 bytes, done.
Total 3 (delta 1), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
Enforcing Policies...
(refs/heads/master) (8338c5) (c5b616)
[POLICY] Your message is not formatted correctly
error: hooks/update exited with error code 1
error: hook declined to update refs/heads/master
To git@gitserver:project.git
 ! [remote rejected] master -> master (hook declined)
error: failed to push some refs to 'git@gitserver:project.git'
```

Existem algumas coisas interessantes aqui. Primeiro, você vê onde o gancho começa a rodar.

```
Enforcing Policies...
(refs/heads/master) (fb8c72) (c56860)
```

Lembre-se de que você o imprimiu no início do script de atualização (update script). Qualquer eco de script para `stdout` será transferido para o cliente.

A próxima coisa que você notará é a mensagem de erro.

```
[POLICY] Your message is not formatted correctly
error: hooks/update exited with error code 1
error: hook declined to update refs/heads/master
```

A primeira linha foi impressa por você, as outras duas era o Git dizendo a você que o script de atualização (update script) saiu sem ser zero e foi isso que negou o seu push. Por fim, você tem isso:

```
To git@gitserver:project.git
 ! [remote rejected] master -> master (hook declined)
error: failed to push some refs to 'git@gitserver:project.git'
```

Você verá uma mensagem rejeitada remota (remote rejected) para cada referência que o seu gancho recusou e diz a você que ela foi recusada especificamente por causa de uma falha de gancho.

Além disso, se alguém tenta editar um arquivo no qual eles não têm acesso e dar um push de um commit contendo ele, eles verão algo parecido. Por exemplo, se um autor da documentação tenta dar um push em um commit modificando algo no diretório `lib`, ele vê:

```
[POLICY] You do not have access to push to lib/test.rb
```

A partir de agora, contanto que o script de atualização (update script) esteja lá e que seja executável, seu repositório nunca terá uma mensagem de commit que não contenha o seu padrão, e seus usuários ficarão contidos.

Ganchos do Lado do Cliente

A desvantagem dessa abordagem são as reclamações que inevitavelmente resultarão quando os commits de push de seus usuários forem rejeitados. Ter seu trabalho cuidadosamente elaborado e ser rejeitado no último minuto pode ser extremamente frustrante e confuso; e, além disso, eles terão que editar seu histórico para corrigi-lo, o que nem sempre é para os fracos de coração.

A resposta para esse dilema é fornecer alguns ganchos do lado do cliente (client-side hooks) que os usuários podem executar para notificá-los quando estão fazendo algo que o servidor provavelmente rejeitará. Dessa forma, eles podem corrigir quaisquer problemas antes de comitar e antes que esses problemas se tornem mais difíceis de resolver. Como os ganchos não são transferidos com o clone de um projeto, você deve distribuir esses scripts de outra forma e, em seguida, pedir aos usuários que os copiem para seu diretório `.git/hooks` e os tornem executáveis. Você pode distribuir esses ganchos dentro do projeto ou em um projeto separado, mas o Git não os configurará automaticamente.

Para começar, você deve verificar sua mensagem de commit logo antes de cada commit ser gravado, para saber que o servidor não rejeitará suas alterações devido a mensagens de commit mal formatadas. Para fazer isso, você pode adicionar o gancho `commit-msg`. Se você fizer com que ele leia a mensagem do arquivo passado como o primeiro argumento e os comparar com o padrão, poderá forçar o Git a abortar o commit, se não houver correspondência:

```
#!/usr/bin/env ruby
message_file = ARGV[0]
message = File.read(message_file)

$regex = /\[ref: (\d+)\]/

if !$regex.match(message)
  puts "[POLICY] Your message is not formatted correctly"
  exit 1
end
```

Se o script estiver em vigor (em `.git/hooks/commit-msg`) e ser um executável, e você fizer um commit com uma mensagem que não esteja formatada corretamente, você verá isto:

```
$ git commit -am 'Test'
[POLICY] Your message is not formatted correctly
```

Nenhum commit foi concluído nessa instância. No entanto, se sua mensagem contém o padrão adequado, o Git permite que você comite:

```
$ git commit -am 'Test [ref: 132]'
```

```
[master e05c914] Test [ref: 132]
 1 file changed, 1 insertions(+), 0 deletions(-)
```

Em seguida, você deseja certificar-se de que não está modificando arquivos que estejam fora do escopo da sua ACL. Se o diretório `.git` do seu projeto contiver uma cópia do arquivo ACL usado anteriormente, o script de `pre-commit` subsequente imporá essas restrições para você:

```
#!/usr/bin/env ruby

$user    = ENV['USER']

# [ insert acl_access_data method from above ]

# only allows certain users to modify certain subdirectories in a project
def check_directory_perms
  access = get_acl_access_data('.git/acl')

  files_modified = `git diff-index --cached --name-only HEAD`.split("\n")
  files_modified.each do |path|
    next if path.size == 0
    has_file_access = false
    access[$user].each do |access_path|
    if !access_path || (path.index(access_path) == 0)
      has_file_access = true
    end
    if !has_file_access
      puts "[POLICY] You do not have access to push to #{path}"
      exit 1
    end
  end
end

check_directory_perms
```

Isto é, grosseiramente, o mesmo script do que a parte do lado do servidor (server-side), mas com duas diferenças importantes. Primeiro, o arquivo ACL fica em um lugar diferente, porque este script roda a partir do seu diretório de trabalho, não do seu diretório `.git`. Você tem que mudar o caminho para o arquivo ACL disso:

```
access = get_acl_access_data('acl')
```

para isto:

```
access = get_acl_access_data('.git/acl')
```

A outra diferença importante é a maneira de obter uma lista dos arquivos que foram alterados.

Como o método do servidor (server-side) examina o log de commits e, nesse ponto, o commit ainda não foi gravado, você deve obter sua lista de arquivos da área de preparação (staging area). Em vez de:

```
files_modified = `git log -1 --name-only --pretty=format:'' #{ref}`
```

você tem de usar:

```
files_modified = `git diff-index --cached --name-only HEAD`
```

Mas essas são as duas únicas diferenças – por outro lado, o script funciona da mesma forma. Uma advertência é que ele espera que você esteja executando localmente como o mesmo usuário com o qual realiza o push para a máquina remota. Se for diferente, você deve definir a variável `$user` manualmente.

Outra coisa que podemos fazer aqui é ter a certeza de que o usuário não faça um push nas referências non-fast-forwarded. Para obter uma referência que não seja um avanço rápido (fast-forward), é necessário rebasear um commit no qual você já enviou via push ou tentar colocar em push um branch local diferente para o mesmo branch remoto.

Presumivelmente, o servidor já está configurado com `receive.denyDeletes` e `receive.denyNonFastForwards` para impor esta política, de modo que a única coisa acidental que você pode tentar buscar é rebasear (rebasing) commits que já foram passados por um push.

Aqui está um exemplo de script pré-rebase que verifica isso. Ele recebe uma lista de todos os commits que você está prestes a reescrever e verifica se eles existem em alguma de suas referências remotas. Se ele vir um que esteja acessível em uma das suas referências remotas, ele aborta a refazagem (rebase).

```
#!/usr/bin/env ruby

base_branch = ARGV[0]
if ARGV[1]
  topic_branch = ARGV[1]
else
  topic_branch = "HEAD"
end

target_shas = `git rev-list #{base_branch}..#{topic_branch}`.split("\n")
remote_refs = `git branch -r`.split("\n").map { |r| r.strip }

target_shas.each do |sha|
  remote_refs.each do |remote_ref|
    shas_pushed = `git rev-list ^#{sha}^@ refs/remotes/#{remote_ref}`
    if shas_pushed.split("\n").include?(sha)
      puts "[POLICY] Commit #{sha} has already been pushed to #{remote_ref}"
      exit 1
```

```
    end
  end
end
```

Este script usa uma sintaxe que não foi coberta em Seleção de Revisão. Você obtém uma lista dos commits que já sofreram o push executando isso:

```
`git rev-list ^#{sha}^@ refs/remotes/#{remote_ref}`
```

A sintaxe `SHA^@` resolve a todos os pais do commit em questão. Você está procurando por qualquer commit que seja acessível a partir do último commit no controle remoto e que não seja acessível a partir de nenhum pai de nenhum dos SHA-1s em que você está tentando fazer push - ou seja, é um avanço rápido (fast-forward).

A principal desvantagem dessa abordagem é que ela pode ser muito lenta e geralmente é desnecessária - se você não tentar forçar o push com `-f`, o servidor o avisará e não aceitará o push. No entanto, é um exercício interessante e pode, em teoria, ajudá-lo a evitar um rebase no qual você pode ter que voltar e consertar posteriormente.

Resumo

Abordamos a maioria das principais maneiras de personalizar seu cliente e servidor Git para melhor se adequar ao seu fluxo de trabalho e projetos. Você aprendeu sobre todos os tipos de definições de configuração, atributos baseados em arquivo e ganchos (hooks) de evento, e construiu um exemplo de servidor de aplicação de políticas. Agora você deve ser capaz de fazer o Git se adequar a quase qualquer fluxo de trabalho que você possa imaginar.

Git e Outros Sistemas

O mundo não é perfeito. Normalmente, você não pode mudar imediatamente todos os projetos com os quais entra em contato para o Git. Às vezes, você está preso a um projeto usando outro VCS e gostaria que fosse o Git. Passaremos a primeira parte deste capítulo aprendendo sobre as maneiras de usar o Git como cliente quando o projeto no qual você está trabalhando estiver hospedado em um sistema diferente.

Em algum momento, você pode querer converter seu projeto existente para o Git. A segunda parte deste capítulo aborda como migrar seu projeto para o Git a partir de vários sistemas específicos, bem como um método que funcionará se não houver nenhuma ferramenta de importação pré-criada.

Git como Cliente

O Git oferece uma experiência tão agradável para os desenvolvedores que muitas pessoas descobriram como usá-lo em sua estação de trabalho, mesmo que o resto da equipe esteja usando um VCS totalmente diferente. Há vários desses adaptadores, chamados "pontes" (`bridges`), disponíveis. Aqui, abordaremos aqueles que você tem maior probabilidade de encontrar no mundo real.

Git e Subversion

Uma grande fração de projetos de desenvolvimento de código aberto e um bom número de projetos corporativos usam o Subversion para gerenciar seu código-fonte. Ele existe há mais de uma década e, na maior parte desse tempo, foi a escolha VCS *de fato* para projetos de código aberto. Também é muito parecido em muitos aspectos com o CVS, que era o garoto grande do mundo do controle de fonte antes disso.

Um dos ótimos recursos do Git é uma ponte bidirecional para o Subversion, chamada `git svn`. Essa ferramenta permite que você use o Git como um cliente válido para um servidor Subversion, para que você possa usar todos os recursos locais do Git e, em seguida, faça um push para um servidor Subversion, como se você estivesse usando o Subversion localmente. Isso significa que você pode fazer ramificações e mesclagens locais, usar a área de preparação, usar rebasing (refazagem) e cherry-picking e assim por diante, enquanto seus colaboradores continuam a trabalhar de maneira escura e antiga. É uma boa maneira de introduzir o Git no ambiente corporativo e ajudar seus colegas desenvolvedores a se tornarem mais eficientes enquanto você faz lobby para obter a alteração da infraestrutura para suportar o Git completamente. A ponte Subversion é a porta de entrada para o mundo do DVCS.

`git svn`

O comando base no Git para todos os comandos de ponte do Subversion é `git svn`. Ele recebe vários comandos, portanto, mostraremos o mais comum ao passar por alguns fluxos de trabalho simples.

É importante observar que, quando você usa o `git svn`, interage com o Subversion, um sistema que funciona de maneira muito diferente do Git. Embora você **possa** fazer uma ramificação e mesclagem local, geralmente é melhor manter seu histórico da forma mais linear possível ao

refazer seu trabalho (rebase) e evitar interagir simultaneamente com um repositório remoto do Git.

Não reescreva o seu histórico e tente empurrá-lo novamente, e não o empurre para um repositório git paralelo para colaborar com outros desenvolvedores do Git ao mesmo tempo. O Subversion pode ter apenas um histórico linear, e confundi-lo é muito fácil. Se você estiver trabalhando com uma equipe, e alguns usarem SVN e outros usarem Git, certifique-se de que todos usem o servidor SVN para colaborar - fazer isso facilitará sua vida.

Configurando

Para demonstrar essa funcionalidade, você precisa de um repositório SVN típico em que tem acesso de gravação. Se você deseja copiar esses exemplos, precisará fazer uma cópia gravável de um repositório de teste SVN. Para fazer isso facilmente, você pode usar uma ferramenta chamada `svnsync` que vem com o Subversion.

Para acompanhar, você deve primeiro criar um novo repositório local do Subversion:

```
$ mkdir /tmp/test-svn
$ svnadmin create /tmp/test-svn
```

Em seguida, ative todos os usuários a alterarem os revprops – a maneira mais fácil é adicionar um script `pre-revprop-change` que sempre sai (exit) de zero:

```
$ cat /tmp/test-svn/hooks/pre-revprop-change
#!/bin/sh
exit 0;
$ chmod +x /tmp/test-svn/hooks/pre-revprop-change
```

Agora você pode sincronizar este projeto com a sua máquina local chamando `svnsync init` a partir de repositórios e para eles.

```
$ svnsync init file:///tmp/test-svn \
  http://your-svn-server.example.org/svn/
```

Isso define as propriedades para executar a sincronização. Você pode clonar o código executando:

```
$ svnsync sync file:///tmp/test-svn
Committed revision 1.
Copied properties for revision 1.
Transmitting file data .............................[...]
Committed revision 2.
Copied properties for revision 2.
[…]
```

Embora essa operação possa levar apenas alguns minutos, se você tentar copiar o repositório original para outro repositório remoto em vez de um local, o processo levará quase uma hora,

mesmo que haja menos de 100 commits. O Subversion deve clonar uma revisão por vez e depois inseri-lo em outro repositório - é ridiculamente ineficiente, mas é a única maneira fácil de fazer isso.

Primeiros Passos

Agora que você tem um repositório Subversion ao qual tem acesso de gravação, pode passar por um fluxo de trabalho típico. Você começará com o comando `git svn clone`, que importa um repositório inteiro do Subversion para um repositório local do Git. Lembre-se de que, se você estiver importando de um repositório do Subversion hospedado de verdade, precisará substituir o arquivo `file:///tmp/test-svn` aqui pela URL do seu repositório do Subversion:

```
$ git svn clone file:///tmp/test-svn -T trunk -b branches -t tags
Initialized empty Git repository in /private/tmp/progit/test-svn/.git/
r1 = dcbfb5891860124cc2e8cc616cded42624897125 (refs/remotes/origin/trunk)
    A   m4/acx_pthread.m4
    A   m4/stl_hash.m4
    A   java/src/test/java/com/google/protobuf/UnknownFieldSetTest.java
    A   java/src/test/java/com/google/protobuf/WireFormatTest.java
...
r75 = 556a3e1e7ad1fde0a32823fc7e4d046bcfd86dae (refs/remotes/origin/trunk)
Found possible branch point: file:///tmp/test-svn/trunk => file:///tmp/test-
svn/branches/my-calc-branch, 75
Found branch parent: (refs/remotes/origin/my-calc-branch)
556a3e1e7ad1fde0a32823fc7e4d046bcfd86dae
Following parent with do_switch
Successfully followed parent
r76 = 0fb585761df569eaecd8146c71e58d70147460a2 (refs/remotes/origin/my-calc-branch)
Checked out HEAD:
  file:///tmp/test-svn/trunk r75
```

Isso executa o equivalente a dois comandos – `git svn init` seguido de `git svn fetch` – na URL fornecida. Isso pode demorar um pouco. Se, por exemplo, o projeto de teste tiver apenas cerca de 75 commits e a base de código não for tão grande, o Git ainda deve verificar cada versão, uma por vez, e comitá-las individualmente. Para um projeto com centenas ou milhares de commits, isso pode literalmente levar horas ou mesmo dias para terminar.

A parte `-T trunk -b branches -t tags` diz ao Git que esse repositório do Subversion segue as convenções básicas de ramificação e marcação. Se você nomear o seu tronco (trunk), ramificações (branches) ou etiquetas (tags) de maneira diferente, poderá alterar essas opções. Como isso é tão comum, você pode substituir toda a parte por `-s`, que significa layout padrão e implica todas essas opções. O seguinte comando é equivalente:

```
$ git svn clone file:///tmp/test-svn -s
```

Neste ponto, você deve ter um repositório Git válido que importou suas ramificações (branches) e tags:

```
$ git branch -a
* master
  remotes/origin/my-calc-branch
  remotes/origin/tags/2.0.2
  remotes/origin/tags/release-2.0.1
  remotes/origin/tags/release-2.0.2
  remotes/origin/tags/release-2.0.2rc1
  remotes/origin/trunk
```

Observe como essa ferramenta gerencia as tags do Subversion como referências remotas (remote refs). Vamos dar uma olhada no comando de encanamento do Git `show-ref`:

```
$ git show-ref
556a3e1e7ad1fde0a32823fc7e4d046bcfd86dae refs/heads/master
0fb585761df569eaecd8146c71e58d70147460a2 refs/remotes/origin/my-calc-branch
bfd2d79303166789fc73af4046651a4b35c12f0b refs/remotes/origin/tags/2.0.2
285c2b2e36e467dd4d91c8e3c0c0e1750b3fe8ca refs/remotes/origin/tags/release-2.0.1
cbda99cb45d9abcb9793db1d4f70ae562a969f1e refs/remotes/origin/tags/release-2.0.2
a9f074aa89e826d6f9d30808ce5ae3ffe711feda refs/remotes/origin/tags/release-2.0.2rc1
556a3e1e7ad1fde0a32823fc7e4d046bcfd86dae refs/remotes/origin/trunk
```

O Git não faz isso quando é clonado de um servidor Git; aqui está a aparência de um repositório com tags após um clone novo:

```
$ git show-ref
c3dcbe8488c6240392e8a5d7553bbffcb0f94ef0 refs/remotes/origin/master
32ef1d1c7cc8c603ab78416262cc421b80a8c2df refs/remotes/origin/branch-1
75f703a3580a9b81ead89fe1138e6da858c5ba18 refs/remotes/origin/branch-2
23f8588dde934e8f33c263c6d8359b2ae095f863 refs/tags/v0.1.0
7064938bd5e7ef47bfd79a685a62c1e2649e2ce7 refs/tags/v0.2.0
6dcb09b5b57875f334f61aebed695e2e4193db5e refs/tags/v1.0.0
```

O Git busca (fetch) as tags diretamente nas `refs/tags`, em vez de tratá-las como branches remotos.

Comitando Novamente no Subversion

Agora que você tem um diretório de trabalho, pode trabalhar um pouco no projeto e fazer com que os seus commits voltem a ser a montante (upstream), usando o Git efetivamente como um cliente SVN. Se você editar um dos arquivos e comitá-lo, terá um commit que existe no Git localmente que não existe no servidor Subversion:

```
$ git commit -am 'Adding git-svn instructions to the README'
[master 4af61fd] Adding git-svn instructions to the README
 1 file changed, 5 insertions(+)
```

Em seguida, você precisa efetuar o push de sua mudança para montante (upstream). Observe como

isso muda a maneira como você trabalha com o Subversion – você pode fazer vários commits offline e, em seguida, fazer push em todos os servidores do Subversion de uma só vez. Para fazer um push em um servidor Subversion, execute o comando `git svn dcommit`:

```
$ git svn dcommit
Committing to file:///tmp/test-svn/trunk ...
    M   README.txt
Committed r77
    M   README.txt
r77 = 95e0222ba6399739834380eb10afcd73e0670bc5 (refs/remotes/origin/trunk)
No changes between 4af61fd05045e07598c553167e0f31c84fd6ffe1 and
refs/remotes/origin/trunk
Resetting to the latest refs/remotes/origin/trunk
```

Isso pega todos os commits que você fez no código do servidor do Subversion, faz um commit no Subversion para cada um e, em seguida, reescreve o commit do Git local para incluir um identificador exclusivo. Isso é importante porque significa que todas as somas de verificação (checksums) do SHA-1 de seus commits mudam. Em parte por esse motivo, não é uma boa ideia trabalhar com versões remotas de seus projetos com base no Git simultaneamente a um servidor Subversion. Se você observar o último commit, poderá ver o novo `git-svn-id` que foi adicionado:

```
$ git log -1
commit 95e0222ba6399739834380eb10afcd73e0670bc5
Author: ben <ben@0b684db3-b064-4277-89d1-21af03df0a68>
Date:   Thu Jul 24 03:08:36 2014 +0000

    Adding git-svn instructions to the README

    git-svn-id: file:///tmp/test-svn/trunk@77 0b684db3-b064-4277-89d1-21af03df0a68
```

Observe que a soma de verificação (checksum) SHA-1 que originalmente começava com `4af61fd` quando você o comitou, agora começa com `95e0222`. Se você deseja fazer push no servidor Git e em um servidor Subversion, você tem que dar o push (`dcommit`) no servidor Subversion primeiro, porque essa ação altera os dados de seu commit.

Extraindo as Novas Mudanças (Pulling)

Se você estiver trabalhando com outros desenvolvedores, em algum momento um de vocês dará um push e, em seguida, o outro tentará enviar um conflito na mudança. Essa alteração será rejeitada até que você junte os trabalhos. Em `git svn`, isso se parece com isso:

```
$ git svn dcommit
Committing to file:///tmp/test-svn/trunk ...

ERROR from SVN:
Transaction is out of date: File '/trunk/README.txt' is out of date
W: d5837c4b461b7c0e018b49d12398769d2bfc240a and refs/remotes/origin/trunk differ,
```

```
using rebase:
:100644 100644 f414c433af0fd6734428cf9d2a9fd8ba00ada145
c80b6127dd04f5fcda218730ddf3a2da4eb39138 M  README.txt
Current branch master is up to date.
ERROR: Not all changes have been committed into SVN, however the committed
ones (if any) seem to be successfully integrated into the working tree.
Please see the above messages for details.
```

Para resolver essa situação, você pode executar o `git svn rebase`, que extrai todas as alterações no servidor que você ainda não possui e recria o trabalho que você tem sobre o que está no servidor:

```
$ git svn rebase
Committing to file:///tmp/test-svn/trunk ...

ERROR from SVN:
Transaction is out of date: File '/trunk/README.txt' is out of date
W: eaa029d99f87c5c822c5c29039d19111ff32ef46 and refs/remotes/origin/trunk differ,
using rebase:
:100644 100644 65536c6e30d263495c17d781962cfff12422693a
b34372b25ccf4945fe5658fa381b075045e7702a M  README.txt
First, rewinding head to replay your work on top of it...
Applying: update foo
Using index info to reconstruct a base tree...
M   README.txt
Falling back to patching base and 3-way merge...
Auto-merging README.txt
ERROR: Not all changes have been committed into SVN, however the committed
ones (if any) seem to be successfully integrated into the working tree.
Please see the above messages for details.
```

Agora, todo o seu trabalho está em cima do que há no servidor Subversion, para que você possa `dcommit` com êxito:

```
$ git svn dcommit
Committing to file:///tmp/test-svn/trunk ...
    M   README.txt
Committed r85
    M   README.txt
r85 = 9c29704cc0bbbed7bd58160cfb66cb9191835cd8 (refs/remotes/origin/trunk)
No changes between 5762f56732a958d6cfda681b661d2a239cc53ef5 and
refs/remotes/origin/trunk
Resetting to the latest refs/remotes/origin/trunk
```

Observe que, ao contrário do Git, que exige a mesclagem de trabalho a montante que você ainda não possui localmente antes de poder realizar push, `git svn` faz isso apenas se as alterações entrarem em conflito (muito parecido com a forma como o Subversion funciona). Se outra pessoa der push de uma mudança num arquivo e, em seguida, você der push em uma mudança de um outro arquivo, seu `dcommit` funcionará perfeitamente:

```
$ git svn dcommit
Committing to file:///tmp/test-svn/trunk ...
    M   configure.ac
Committed r87
    M   autogen.sh
r86 = d8450bab8a77228a644b7dc0e95977ffc61adff7 (refs/remotes/origin/trunk)
    M   configure.ac
r87 = f3653ea40cb4e26b6281cec102e35dcba1fe17c4 (refs/remotes/origin/trunk)
W: a0253d06732169107aa020390d9fefd2b1d92806 and refs/remotes/origin/trunk differ,
using rebase:
:100755 100755 efa5a59965fbbb5b2b0a12890f1b351bb5493c18
e757b59a9439312d80d5d43bb65d4a7d0389ed6d M  autogen.sh
First, rewinding head to replay your work on top of it...
```

Isso é importante de se lembrar, porque o resultado é um estado de projeto que não existia em nenhum de seus computadores quando você os empurrou (push). Se as alterações forem incompatíveis, mas não entrarem em conflito, você poderá ter problemas difíceis de diagnosticar. Isso é diferente de usar um servidor Git – no Git, você pode testar completamente o estado do sistema cliente antes de publicá-lo, enquanto no SVN você nunca pode ter certeza de que os estados imediatamente antes e depois do commit são idênticos.

Você também deve executar este comando para efetuar pull nas alterações do servidor Subversion, mesmo que não esteja pronto para se comprometer (comitar). Você pode executar `git svn fetch` para obter os novos dados, mas `git svn rebase` busca (fetch) e atualiza seus commits locais.

```
$ git svn rebase
    M   autogen.sh
r88 = c9c5f83c64bd755368784b444bc7a0216cc1e17b (refs/remotes/origin/trunk)
First, rewinding head to replay your work on top of it...
Fast-forwarded master to refs/remotes/origin/trunk.
```

A execução de `git svn rebase` de vez em quando garante que seu código esteja sempre atualizado. Você deve ter certeza de que seu diretório de trabalho esteja limpo ao executar isso. Se você tiver alterações locais, deve fazer stash do trabalho ou criar um commit temporário antes de executar `git svn rebase` – caso contrário, o comando será interrompido se constatar que o rebase resultará em um conflito de merge.

Problemas do Branching do Git

Quando você se sentir confortável com um fluxo de trabalho do Git, provavelmente criará ramificações (branches) de tópicos, fará o trabalho e as mesclará (merge) nelas. Se você estiver empurrando (pushing) para um servidor Subversion via `git svn`, convém refazer (rebase) seu trabalho em uma única filial de cada vez, em vez de mesclar as ramificações (branches). O motivo para preferir o rebasing é que o Subversion tem um histórico linear e não lida com mesclagens como o Git, portanto o `git svn` segue apenas o primeiro pai ao converter os instantâneos (snapshots) em commits do Subversion.

Suponha que seu histórico seja o seguinte: você criou uma ramificação `experiment`, fez dois commits

e depois mesclou-os no `master`. Quando você usar `dcommit`, você verá uma saída parecida com esta:

```
$ git svn dcommit
Committing to file:///tmp/test-svn/trunk ...
    M   CHANGES.txt
Committed r89
    M   CHANGES.txt
r89 = 89d492c884ea7c834353563d5d913c6adf933981 (refs/remotes/origin/trunk)
    M   COPYING.txt
    M   INSTALL.txt
Committed r90
    M   INSTALL.txt
    M   COPYING.txt
r90 = cb522197870e61467473391799148f6721bcf9a0 (refs/remotes/origin/trunk)
No changes between 71af502c214ba13123992338569f4669877f55fd and
refs/remotes/origin/trunk
Resetting to the latest refs/remotes/origin/trunk
```

A execução do `dcommit` em um branch com histórico mesclado (merge history) funciona bem, exceto quando você analisa o histórico do projeto Git. Ele não reescreveu nenhum dos commits efetuados no branch `experiment` – em vez disso, todas essas alterações aparecem na versão SVN do commit de consolidação (merge commit) único.

Quando outra pessoa clona esse trabalho, ela vê apenas o commit de consolidação (merge commit) com todo o trabalho esmagado, como se você executasse `git merge --squash`; eles não veem os dados de confirmação sobre de onde eles vieram ou quando foram confirmados.

Ramificação (Branching) do Subversion

Branching no Subversion não é o mesmo que branching no Git; se você puder evitar o uso, provavelmente é o melhor. No entanto, você pode criar e comitar em branches no Subversion usando o `git svn`.

Criando um Novo Branch do SVN

Para criar um novo branch no Subversion, você roda o `git svn branch [new-branch]`:

```
$ git svn branch opera
Copying file:///tmp/test-svn/trunk at r90 to file:///tmp/test-svn/branches/opera...
Found possible branch point: file:///tmp/test-svn/trunk => file:///tmp/test-
svn/branches/opera, 90
Found branch parent: (refs/remotes/origin/opera)
cb522197870e61467473391799148f6721bcf9a0
Following parent with do_switch
Successfully followed parent
r91 = f1b64a3855d3c8dd84ee0ef10fa89d27f1584302 (refs/remotes/origin/opera)
```

Isso faz o equivalente ao comando `svn copy trunk branches/opera` no Subversion e opera no

servidor Subversion. É importante observar que não faz uma verificação nesse branch; se você comitar nesse ponto, esse commit irá para o `trunk` no servidor, não no `opera`.

Mudando os Branches Ativos

O Git descobre em que ramificação as suas comissões ocorrem, procurando a ponta (tip) de qualquer uma de suas filiais (branches) do Subversion em seu histórico – você deve ter apenas um e deve ser o último com um `git-svn-id` no histórico da sua ramificação atual.

Se você deseja trabalhar em mais de um branch simultaneamente, pode configurar branches locais no `dcommit` para branches específicos do Subversion, iniciando-os no commit de Subversion importado para essa branch. Se você deseja uma filial (branch) `opera` na qual possa trabalhar separadamente, execute:

```
$ git branch opera remotes/origin/opera
```

Agora, se você deseja mesclar (merge) o seu branch `opera` no `trunk` (seu branch `master`), é possível fazê-lo com um `git merge` normal. Mas você precisa fornecer uma mensagem descritiva de commit (via `-m`), ou a consolidação (merge) dirá "Merge branch opera" em vez de algo útil.

Lembre-se de que, embora você esteja usando o `git merge` para executar esta operação, o merge provavelmente será muito mais fácil do que no Subversion (porque o Git detectará automaticamente a base de consolidação (merge) apropriada para você), esse não é um normal commit do Git normal. Você tem que levar (push) esses dados de volta a um servidor Subversion que não consegue lidar com um commit que rastreia mais de um pai; logo, depois de enviá-lo (push up), parecerá um único commit no qual esmaga todo o trabalho de outro branch sob um único commit. Depois de mesclar (merge) um branch com outro, não é possível voltar com facilidade e continuar trabalhando nela, como é normalmente possível no Git. O comando `dcommit` que você executa apaga qualquer informação que diga qual branch foi mesclado (merged), para que os cálculos base subsequentes da base de consolidação fiquem errados – o `dcommit` faz o seu resultado do `git merge` parecer como se tivesse rodado o comando `git merge --squash`. Infelizmente, não há uma boa maneira de evitar essa situação – o Subversion não pode armazenar essas informações, portanto, você sempre será prejudicado por suas limitações enquanto o usar como servidor. Para evitar problemas, exclua o branch local (neste caso, `opera`) depois de mesclá-lo ao trunk.

Comandos do Subversion

O conjunto de ferramentas `git svn` fornece vários comandos para ajudar a facilitar a transição para o Git, fornecendo algumas funcionalidades semelhantes às do Subversion. Aqui estão alguns comandos que fornecem o que o Subversion costumava fazer.

Histórico em Estilo SVN

Se você está acostumado ao Subversion e deseja ver o seu histórico no estilo de saída SVN, execute `git svn log` para visualizar o seu histórico de commits na formatação SVN:

```
$ git svn log
------------------------------------------------------------------------
```

```
r87 | schacon | 2014-05-02 16:07:37 -0700 (Sat, 02 May 2014) | 2 lines

autogen change

------------------------------------------------------------------------
r86 | schacon | 2014-05-02 16:00:21 -0700 (Sat, 02 May 2014) | 2 lines

Merge branch 'experiment'

------------------------------------------------------------------------
r85 | schacon | 2014-05-02 16:00:09 -0700 (Sat, 02 May 2014) | 2 lines

updated the changelog
```

Você deve saber duas coisas importantes sobre o `git svn log`. Primeiro, ele funciona offline, diferentemente do comando real `svn log`, que solicita os dados ao servidor Subversion. Segundo, ele mostra apenas confirmações (commits) que foram submetidas ao servidor Subversion. Os commits do Git local com as quais você não se comitou não aparecem; tampouco as comissões que as pessoas fizeram no servidor Subversion nesse meio tempo. É mais como o último estado conhecido dos commits no servidor Subversion.

Anotações (Annotations) do SVN

Assim como o comando `git svn log` simula o comando `svn log` offline, você pode obter o equivalente de `svn annotate` ao executar `git svn blame [FILE]`. A saída se parece com esta:

```
$ git svn blame README.txt
 2   temporal Protocol Buffers - Google's data interchange format
 2   temporal Copyright 2008 Google Inc.
 2   temporal http://code.google.com/apis/protocolbuffers/
 2   temporal
22   temporal C++ Installation - Unix
22   temporal =======================
 2   temporal
79    schacon Committing in git-svn.
78    schacon
 2   temporal To build and install the C++ Protocol Buffer runtime and the Protocol
 2   temporal Buffer compiler (protoc) execute the following:
 2   temporal
```

Mais uma vez, ele não mostra os commits que você fez localmente no Git ou que foram empurrados (pushed) para o Subversion durante esse período.

Informações do Servidor SVN

Você também pode obter o mesmo tipo de informação que a opção `svn info` fornece ao executar a opção `git svn info`:

```
$ git svn info
```

```
Path: .
URL: https://schacon-test.googlecode.com/svn/trunk
Repository Root: https://schacon-test.googlecode.com/svn
Repository UUID: 4c93b258-373f-11de-be05-5f7a86268029
Revision: 87
Node Kind: directory
Schedule: normal
Last Changed Author: schacon
Last Changed Rev: 87
Last Changed Date: 2009-05-02 16:07:37 -0700 (Sat, 02 May 2009)
```

É como no `blame` e no `log`, ele fica offline e está atualizado apenas na última vez em que você se comunicou com o servidor Subversion.

Ignorando O Que O Subversion Ignora

Se você clonar um repositório Subversion que possua propriedades `svn:ignore` configuradas em qualquer lugar, provavelmente você desejará configurar os arquivos `.gitignore` correspondentes para não comprometer acidentalmente arquivos que não deveriam. O `git svn` possui dois comandos para ajudar com esse problema. O primeiro é o `git svn create-ignore`, que cria automaticamente arquivos `.gitignore` correspondentes para você, para que o seu próximo commit possa incluí-los.

O segundo comando é `git svn show-ignore`, que imprime no stdout as linhas que você precisa colocar em um arquivo `.gitignore` para redirecionar a saída para o arquivo de exclusão (exclude) de projeto:

```
$ git svn show-ignore > .git/info/exclude
```

Dessa forma, você não joga lixo no projeto com arquivos `.gitignore`. Esta é uma boa opção se você for o único usuário do Git em uma equipe de Subversion e os seus colegas de equipe não quiserem arquivos `.gitignore` no projeto.

Resumo do Git-Svn

As ferramentas `git svn` são úteis se você estiver preso a um servidor Subversion ou em um ambiente de desenvolvimento que exija a execução de um servidor Subversion. Você deve considerá-lo como um Git prejudicado, no entanto, ou você terá problemas na tradução que podem confundir você e seus colaboradores. Para evitar problemas, tente seguir estas diretrizes:

- Mantenha um histórico linear do Git que não contenha os commits consolidados (merge) gerados pelo `git merge`. Refaça (rebase) todo o trabalho que você faz fora do seu branch principal; não os consolide (merge).
- Não configure e nem colabore em um servidor Git separado. Possivelmente possua um para acelerar clones de novos desenvolvedores, mas não o empurre (push) para nada que não tenha uma entrada `git-svn-id`. Você pode até adicionar um gancho (hook) de `pre-receive` que verifica cada mensagem de commit em busca de um `git-svn-id` e rejeita push que contenha commits sem ela.

Se você seguir essas diretrizes, trabalhar com um servidor Subversion pode ser mais suportável. No entanto, se for possível mover para um servidor Git real, isso poderá ganhar muito mais para sua equipe.

Git e Mercurial

O universo DVCS é maior que apenas o Git. De fato, existem muitos outros sistemas nesse espaço, cada um com seu próprio ângulo sobre como fazer corretamente o controle de versão distribuído. Além do Git, o mais popular é o Mercurial, e os dois são muito semelhantes em muitos aspectos.

A boa notícia, se você prefere o comportamento do lado do cliente (client-side) do Git, mas está trabalhando com um projeto cujo código-fonte é controlado pelo Mercurial, é que há uma maneira de usar o Git como cliente de um repositório hospedado no Mercurial. Como a maneira pela qual o Git conversa com os repositórios do servidor se dá através de remotos, não deve surpreender o fato de que esta ponte (bridge) seja implementada como um ajudante remoto (remote helper). O nome do projeto é git-remote-hg e pode ser encontrado em https://github.com/felipec/git-remote-hg.

git-remote-hg

Primeiro, você deve instalar o git-remote-hg. Isso implica basicamente soltar o arquivo em algum lugar no seu caminho (path), da seguinte forma:

```
$ curl -o ~/bin/git-remote-hg \
  https://raw.githubusercontent.com/felipec/git-remote-hg/master/git-remote-hg
$ chmod +x ~/bin/git-remote-hg
```

i. assumindo que `~/bin` está no seu `$PATH`. O Git-remote-hg possui uma outra dependência: a biblioteca `mercurial` do Python. Se você possui o Python instalado, isso é tão simples quanto:

```
$ pip install mercurial
```

Se você não tem o Python instalado, acesse https://www.python.org/ e faça a obtenção dele antes.

A última coisa que você precisará é do cliente Mercurial. Vá para https://www.mercurial-scm.org/ e o instale caso ainda não o tenha feito.

Agora você está pronto para agitar. Tudo que você precisa é de um repositório Mercurial no qual possa empurrar (fazer o push). Felizmente, todos os repositórios Mercurial podem agir dessa maneira, portanto, usaremos o repositório "hello world" que todo mundo usa para aprender o Mercurial:

```
$ hg clone http://selenic.com/repo/hello /tmp/hello
```

Primeiros Passos

Agora que temos um repositório "lado do servidor (server-side)" adequado, podemos seguir um

fluxo de trabalho típico. Como você verá, esses dois sistemas são bastante semelhantes e não há muito atrito.

Como sempre no Git, primeiramente nós o clonamos:

```
$ git clone hg::/tmp/hello /tmp/hello-git
$ cd /tmp/hello-git
$ git log --oneline --graph --decorate
* ac7955c (HEAD, origin/master, origin/branches/default, origin/HEAD,
refs/hg/origin/branches/default, refs/hg/origin/bookmarks/master, master) Create a
makefile
* 65bb417 Create a standard 'hello, world' program
```

Você notará que trabalhar com um repositório Mercurial usa o comando padrão `git clone`. Isso ocorre porque o git-remote-hg está trabalhando em um nível bastante baixo, usando um mecanismo semelhante a como o protocolo HTTP/S do Git é implementado (helpers remotos). Como o Git e o Mercurial são projetados para que todos os clientes tenham uma cópia completa do histórico do repositório, este comando faz um clone completo, incluindo todo o histórico do projeto, e o faz com bastante rapidez.

O comando `log` mostra dois commits, os mais recentes sendo apontados por um monte de referências. Acontece que alguns deles não estão realmente lá. Vamos dar uma olhada no que realmente está no diretório `.git`:

```
$ tree .git/refs
.git/refs
├── heads
│   └── master
├── hg
│   └── origin
│       ├── bookmarks
│       │   └── master
│       └── branches
│           └── default
├── notes
│   └── hg
├── remotes
│   └── origin
│       └── HEAD
└── tags

9 directories, 5 files
```

O Git-remote-hg está tentando tornar as coisas mais idiomaticamente no formato Git-esque, mas sob o capô está gerenciando o mapeamento conceitual entre dois sistemas ligeiramente diferentes. O diretório `refs/hg` é onde as referências remotas (remote refs) reais são armazenadas. Por exemplo, o arquivo `refs/hg/origin/branches/default` é um arquivo de ref do Git que contém o SHA-1 que começa com “ac7955c”, que é o commit no qual a `master` aponta. Portanto, o diretório `refs/hg`

é como um falso `refs/remotes/origin`, mas tem a distinção adicional entre favoritos (bookmarks) e ramificações (branches).

O arquivo `notes/hg` é o ponto de partida de como o git-remote-hg mapeia hashes de commit do Git para IDs de conjunto de alterações do Mercurial. Vamos explorar um pouco:

```
$ cat notes/hg
d4c10386...

$ git cat-file -p d4c10386...
tree 1781c96...
author remote-hg <> 1408066400 -0800
committer remote-hg <> 1408066400 -0800

Notes for master

$ git ls-tree 1781c96...
100644 blob ac9117f...  65bb417...
100644 blob 485e178...  ac7955c...

$ git cat-file -p ac9117f
0a04b987be5ae354b710cefeba0e2d9de7ad41a9
```

Portanto, `refs/notes/hg` aponta para uma árvore, que no banco de dados de objetos do Git é uma lista de outros objetos com nomes. O `git ls-tree` produz o modo, tipo, hash do objeto e o nome de arquivo para os itens em uma árvore. Depois de cavar em um dos itens da árvore, descobrimos que, dentro dele, há um blob chamado "ac9117f" (o hash SHA-1 do commit apontado por `master`), com o conteúdo "0a04b98" (que é o ID do changeset do Mercurial na ponta (tip) do branch `default`).

A boa notícia é que não precisamos nos preocupar com tudo isso a maior parte do tempo. O fluxo de trabalho típico não será muito diferente do trabalho com um controle remoto no Git.

Há mais uma coisa que devemos atender antes de continuarmos: ignorações. O Mercurial e o Git usam um mecanismo muito semelhante para isso, mas é provável que você não queira confirmar (comitar) um arquivo `.gitignore` em um repositório Mercurial. Felizmente, o Git possui uma maneira de ignorar arquivos que são locais a um repositório no disco e o formato Mercurial é compatível com o Git; portanto, basta copiá-lo para:

```
$ cp .hgignore .git/info/exclude
```

O arquivo `.git/info/exclude` age como um `.gitignore`, mas não é incluído nos commits.

Fluxo de Trabalho

Vamos supor que fizemos algum trabalho e fizemos alguns commits no branch `master` e você está pronto para enviá-lo (push) ao repositório remoto. Aqui está a aparência do nosso repositório no momento:

```
$ git log --oneline --graph --decorate
* ba04a2a (HEAD, master) Update makefile
* d25d16f Goodbye
* ac7955c (origin/master, origin/branches/default, origin/HEAD,
refs/hg/origin/branches/default, refs/hg/origin/bookmarks/master) Create a makefile
* 65bb417 Create a standard 'hello, world' program
```

Nosso branch `master` tem dois commits à frente da `origin/master`, mas esses dois commits existem apenas na nossa máquina local. Vamos ver se alguém fez um trabalho importante ao mesmo tempo:

```
$ git fetch
From hg::/tmp/hello
   ac7955c..df85e87  master     -> origin/master
   ac7955c..df85e87  branches/default -> origin/branches/default
$ git log --oneline --graph --decorate --all
* 7b07969 (refs/notes/hg) Notes for default
* d4c1038 Notes for master
* df85e87 (origin/master, origin/branches/default, origin/HEAD,
refs/hg/origin/branches/default, refs/hg/origin/bookmarks/master) Add some
documentation
| * ba04a2a (HEAD, master) Update makefile
| * d25d16f Goodbye
|/
* ac7955c Create a makefile
* 65bb417 Create a standard 'hello, world' program
```

Como usamos o sinalizador (flag) `--all`, vemos as refs de “notes” usadas internamente pelo git-remote-hg, mas podemos ignorá-las. O resto é o que esperávamos; `origin/master` avançou em um commit, e nosso histórico agora divergiu. Ao contrário dos outros sistemas com os quais trabalhamos neste capítulo, o Mercurial é capaz de lidar com merges, portanto não faremos nada de extraordinário.

```
$ git merge origin/master
Auto-merging hello.c
Merge made by the 'recursive' strategy.
 hello.c | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --decorate
*   0c64627 (HEAD, master) Merge remote-tracking branch 'origin/master'
|\
| * df85e87 (origin/master, origin/branches/default, origin/HEAD,
refs/hg/origin/branches/default, refs/hg/origin/bookmarks/master) Add some
documentation
* | ba04a2a Update makefile
* | d25d16f Goodbye
|/
* ac7955c Create a makefile
```

```
* 65bb417 Create a standard 'hello, world' program
```

Perfeito. Executamos os testes e tudo passa; portanto, estamos prontos para compartilhar o nosso trabalho com o restante da equipe:

```
$ git push
To hg::/tmp/hello
   df85e87..0c64627  master -> master
```

Isso é tudo! Se você der uma olhada no repositório do Mercurial, verá que isso fez o que esperávamos:

```
$ hg log -G --style compact
o    5[tip]:4,2   dc8fa4f932b8   2014-08-14 19:33 -0700   ben
|\     Merge remote-tracking branch 'origin/master'
| |
| o  4   64f27bcefc35   2014-08-14 19:27 -0700   ben
| |    Update makefile
| |
| o  3:1   4256fc29598f   2014-08-14 19:27 -0700   ben
| |    Goodbye
| |
@ |  2   7db0b4848b3c   2014-08-14 19:30 -0700   ben
|/     Add some documentation
|
o  1   82e55d328c8c   2005-08-26 01:21 -0700   mpm
|    Create a makefile
|
o  0   0a04b987be5a   2005-08-26 01:20 -0700   mpm
     Create a standard 'hello, world' program
```

O conjunto de alterações (changeset) numerado *2* foi criado por Mercurial, e os conjuntos de alterações numerados *3* e *4* foram feitos por git-remote-hg, promovendo os commits efetuados pelo Git (push).

Ramificações e Favoritos (Branches e Bookmarks)

O Git possui apenas um tipo de ramificação (branch): uma referência que se move quando são feitos commits. No Mercurial, esse tipo de referência é chamado de "favorito (bookmark)" e se comporta da mesma maneira que uma filial (branch) do Git.

O conceito do Mercurial de uma "ramificação (`branch`)" é mais pesado. A ramificação na qual um conjunto de alterações (changeset) é criado é gravada *juntamente com o conjunto de alterações*, o que significa que ele sempre estará no histórico do repositório. Aqui está um exemplo de uma comissão efetuada no branch `develop`:

```
$ hg log -l 1
```

```
changeset:   6:8f65e5e02793
branch:      develop
tag:         tip
user:        Ben Straub <ben@straub.cc>
date:        Thu Aug 14 20:06:38 2014 -0700
summary:     More documentation
```

Observe a linha que começa com “branch”. O Git não pode realmente replicar isso (e não é necessário; os dois tipos de ramificações (branch) podem ser representados como uma referência (ref) do Git), mas o git-remote-hg precisa entender a diferença, porque o Mercurial se importa.

Criar favoritos no Mercurial é tão fácil quanto criar ramificações (branches) do Git. Do lado do Git:

```
$ git checkout -b featureA
Switched to a new branch 'featureA'
$ git push origin featureA
To hg::/tmp/hello
 * [new branch]      featureA -> featureA
```

Isto é tudo que há nisto. Do lado do Mercurial, a aparência é a seguinte:

```
$ hg bookmarks
   featureA                  5:bd5ac26f11f9
$ hg log --style compact -G
@  6[tip]   8f65e5e02793   2014-08-14 20:06 -0700   ben
|    More documentation
|
o    5[featureA]:4,2   bd5ac26f11f9   2014-08-14 20:02 -0700   ben
|\     Merge remote-tracking branch 'origin/master'
| |
| o  4   0434aaa6b91f   2014-08-14 20:01 -0700   ben
| |    update makefile
| |
| o  3:1   318914536c86   2014-08-14 20:00 -0700   ben
| |    goodbye
| |
o |  2   f098c7f45c4f   2014-08-14 20:01 -0700   ben
|/     Add some documentation
|
o  1   82e55d328c8c   2005-08-26 01:21 -0700   mpm
|    Create a makefile
|
o  0   0a04b987be5a   2005-08-26 01:20 -0700   mpm
     Create a standard 'hello, world' program
```

Observe a nova tag `[featureA]` na revisão 5. Eles agem exatamente como os branches do Git no lado do Git, com uma exceção: você não pode excluir um favorito (bookmark) do lado do Git (esta é uma limitação dos ajudantes remotos).

Você também pode trabalhar em um branch "pesado (heavyweight)" do Mercurial: basta colocar um branch no espaço de nome das branches:

```
$ git checkout -b branches/permanent
Switched to a new branch 'branches/permanent'
$ vi Makefile
$ git commit -am 'A permanent change'
$ git push origin branches/permanent
To hg::/tmp/hello
 * [new branch]      branches/permanent -> branches/permanent
```

Veja a aparência do lado do Mercurial:

```
$ hg branches
permanent                      7:a4529d07aad4
develop                        6:8f65e5e02793
default                        5:bd5ac26f11f9 (inactive)
$ hg log -G
o  changeset:   7:a4529d07aad4
|  branch:      permanent
|  tag:         tip
|  parent:      5:bd5ac26f11f9
|  user:        Ben Straub <ben@straub.cc>
|  date:        Thu Aug 14 20:21:09 2014 -0700
|  summary:     A permanent change
|
| @  changeset:   6:8f65e5e02793
|/   branch:      develop
|    user:        Ben Straub <ben@straub.cc>
|    date:        Thu Aug 14 20:06:38 2014 -0700
|    summary:     More documentation
|
o    changeset:   5:bd5ac26f11f9
|\   bookmark:    featureA
| |  parent:      4:0434aaa6b91f
| |  parent:      2:f098c7f45c4f
| |  user:        Ben Straub <ben@straub.cc>
| |  date:        Thu Aug 14 20:02:21 2014 -0700
| |  summary:     Merge remote-tracking branch 'origin/master'
[...]
```

O nome da ramificação (branch name) “permanent” foi gravada no conjunto de alterações (changeset) marcado *7*.

Do ponto de vista do Git, trabalhar com qualquer um desses estilos de ramificação (branch styles) é a mesma coisa: basta fazer o checkout, comitar, fazer fetch, merge, puxar (pull) e dar push, como faria normalmente. Uma coisa que você deve saber é que o Mercurial não suporta a reescrita da história, apenas as acrescenta. Veja como é o nosso repositório Mercurial após um rebase interativo

e um force-push:

```
$ hg log --style compact -G
o  10[tip]   99611176cbc9   2014-08-14 20:21 -0700   ben
|    A permanent change
|
o  9   f23e12f939c3   2014-08-14 20:01 -0700   ben
|    Add some documentation
|
o  8:1   c16971d33922   2014-08-14 20:00 -0700   ben
|    goodbye
|
| o  7:5   a4529d07aad4   2014-08-14 20:21 -0700   ben
| |    A permanent change
| |
| | @  6   8f65e5e02793   2014-08-14 20:06 -0700   ben
| |/     More documentation
| |
| o    5[featureA]:4,2   bd5ac26f11f9   2014-08-14 20:02 -0700   ben
| |\     Merge remote-tracking branch 'origin/master'
| | |
| | o  4   0434aaa6b91f   2014-08-14 20:01 -0700   ben
| | |    update makefile
| | |
+---o  3:1   318914536c86   2014-08-14 20:00 -0700   ben
| |      goodbye
| |
| o  2   f098c7f45c4f   2014-08-14 20:01 -0700   ben
|/     Add some documentation
|
o  1   82e55d328c8c   2005-08-26 01:21 -0700   mpm
|    Create a makefile
|
o  0   0a04b987be5a   2005-08-26 01:20 -0700   mpm
     Create a standard "hello, world" program
```

Os conjuntos de alterações *8*, *9* e *10* foram criados e pertencem à ramificação (branch) `permanent`, mas os antigos conjuntos de alterações ainda estão lá. Isso pode ser **muito** confuso para seus colegas de equipe que usam o Mercurial; tente evitá-lo.

Resumo do Mercurial

O Git e o Mercurial são semelhantes o suficiente para que o trabalho além da fronteira seja indolor. Se você evita mudar o histórico ao ter saído da sua máquina (como geralmente é recomendado), talvez nem saiba que o outro lado é um Mercurial.

Git e Perforce

O Perforce é um sistema de controle de versão muito popular em ambientes corporativos. Existe

desde 1995, o que o torna o sistema mais antigo abordado neste capítulo. Como tal, foi desenvolvido com as restrições de sua época; assume que você esteja sempre conectado a um único servidor central e apenas uma versão seja mantida no disco local. Certamente, seus recursos e restrições são adequados a vários problemas específicos, mas há muitos projetos que usam o Perforce, onde o Git funcionaria melhor.

Existem duas opções se você deseja misturar o uso do Perforce com o do Git. A primeira que abordaremos é a ponte "Git Fusion" (fusão com o Git) dos fabricantes do Perforce, que permite expor subárvores do seu depósito do Perforce como repositórios Git de leitura e gravação. O segundo é o git-p4, uma ponte do lado do cliente (client-side bridge) que permite usar o Git como cliente Perforce, sem exigir nenhuma reconfiguração do servidor Perforce.

Git Fusion

O Perforce fornece um produto chamado Git Fusion (disponível em https://www.perforce.com/manuals/git-fusion/), que sincroniza um servidor Perforce com repositórios Git no lado do servidor.

Configurando

Em nossos exemplos, usaremos o método de instalação mais fácil para o Git Fusion, que é o download de uma máquina virtual que executa o daemon do Perforce e o Git Fusion. Você pode obter a imagem da máquina virtual em https://www.perforce.com/downloads e, assim que o download for concluído, importe-o para o seu software de virtualização favorito (usaremos o VirtualBox).

Ao iniciar a máquina pela primeira vez, ele solicita que você personalize a senha de três usuários do Linux (`root`, `perforce` e `git`) e forneça um nome de instância, que pode ser usado para distinguir esta instalação de outras na mesma rede. Quando tudo estiver concluído, você verá isto:

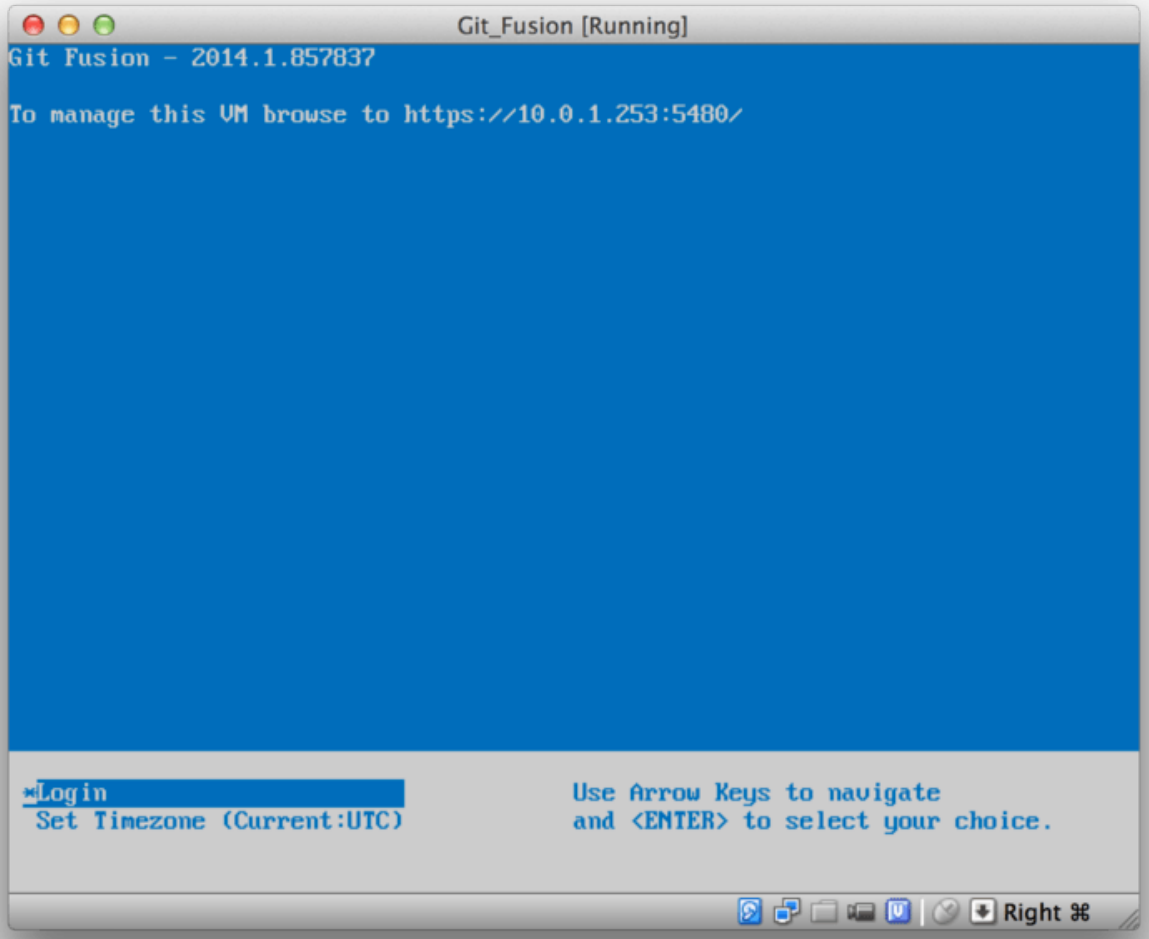


Figure 171. A tela de inicialização da máquina virtual do Git Fusion

Você deve tomar nota do endereço IP mostrado aqui, para utilizarmos posteriormente. Em seguida, criaremos um usuário do Perforce. Selecione a opção “Login” na parte inferior e pressione Enter (ou digite um SSH na máquina) e faça o login como `root`. Então, use esses comandos para criar um usuário:

```
$ p4 -p localhost:1666 -u super user -f john
$ p4 -p localhost:1666 -u john passwd
$ exit
```

O primeiro abrirá um editor VI para personalizar o usuário, mas você pode aceitar os padrões digitando `:wq` e pressionando Enter. O segundo solicitará que você digite uma senha duas vezes. Isso é tudo o que precisamos fazer com um prompt do shell (shell prompt), portanto, saia da sessão.

A próxima coisa que você precisa fazer para acompanhar é dizer ao Git para não verificar os certificados SSL. A imagem do Git Fusion vem com um certificado, mas é de um domínio que não corresponde ao endereço IP da sua máquina virtual, de modo que o Git rejeitará a conexão HTTPS. Se for uma instalação permanente, consulte o manual do Perforce Git Fusion para instalar um certificado diferente; para nossos propósitos de exemplo, isso será suficiente:

```
$ export GIT_SSL_NO_VERIFY=true
```

Agora podemos testar se tudo está funcionando.

```
$ git clone https://10.0.1.254/Talkhouse
Cloning into 'Talkhouse'...
Username for 'https://10.0.1.254': john
Password for 'https://john@10.0.1.254':
remote: Counting objects: 630, done.
remote: Compressing objects: 100% (581/581), done.
remote: Total 630 (delta 172), reused 0 (delta 0)
Receiving objects: 100% (630/630), 1.22 MiB | 0 bytes/s, done.
Resolving deltas: 100% (172/172), done.
Checking connectivity... done.
```

A imagem da máquina virtual vem equipada com um projeto de amostra que você pode clonar. Aqui estamos fazendo um clone por HTTPS, com o usuário `john` que criamos acima; o Git solicita as credenciais para essa conexão, mas o cache de credenciais nos permitirá ignorar essa etapa em todas as solicitações subsequentes.

Configuração do Fusion

Depois que você tiver o Git Fusion instalado, precisará ajustar a configuração. Isso é muito fácil de fazer usando o seu cliente (client) favorito do Perforce; basta mapear o diretório `//.git-fusion` no servidor Perforce para a sua área de trabalho. A estrutura do arquivo se parece com isto:

```
$ tree
.
├── objects
│   ├── repos
│   │   └── [...]
│   └── trees
│       └── [...]
│
├── p4gf_config
├── repos
│   └── Talkhouse
│       └── p4gf_config
└── users
    └── p4gf_usermap

498 directories, 287 files
```

O diretório de `objects` é usado internamente pelo Git Fusion para mapear objetos do Perforce para o Git e vice-versa, para que você não precise mexer com nada lá dentro. Existe um arquivo `p4gf_config` global neste diretório, bem como um para cada repositório - esses são os arquivos de configuração que determinam como o Git Fusion se comporta. Vamos dar uma olhada no arquivo

na raiz:

```
[repo-creation]
charset = utf8

[git-to-perforce]
change-owner = author
enable-git-branch-creation = yes
enable-swarm-reviews = yes
enable-git-merge-commits = yes
enable-git-submodules = yes
preflight-commit = none
ignore-author-permissions = no
read-permission-check = none
git-merge-avoidance-after-change-num = 12107

[perforce-to-git]
http-url = none
ssh-url = none

[@features]
imports = False
chunked-push = False
matrix2 = False
parallel-push = False

[authentication]
email-case-sensitivity = no
```

Não entraremos no significado dessas bandeiras (flags) aqui, mas observe que este é apenas um arquivo de texto no formato INI, muito parecido com o qual o Git usa para configuração. Esse arquivo especifica as opções globais, que podem ser substituídas por arquivos de configuração específicos do repositório, como `repos/Talkhouse/p4gf_config`. Se você abrir este arquivo, verá a seção `[@repo]` com algumas configurações diferentes dos padrões globais. Você também verá seções parecidas com esta:

```
[Talkhouse-master]
git-branch-name = master
view = //depot/Talkhouse/main-dev/... ...
```

Esse é um mapeamento entre um branch do Perforce e um branch do Git. A seção pode ser nomeada com o que você preferir, desde que o nome seja único. `git-branch-name` permite que você converta um caminho (path) de depósito que seria incômodo no Git para um nome mais amigável. A configuração `view` controla como os arquivos Perforce são mapeados no repositório do Git, usando a sintaxe padrão do view mapping. Mais de um mapeamento pode ser especificado, como neste exemplo:

```
[multi-project-mapping]
git-branch-name = master
view = //depot/project1/main/... project1/...
       //depot/project2/mainline/... project2/...
```

Dessa forma, se o seu mapeamento normal da área de trabalho incluir alterações na estrutura dos diretórios, você poderá replicá-lo com um repositório Git.

O último arquivo que discutiremos é o `users/p4gf_usermap`, que mapeia os usuários do Perforce para usuários do Git, no qual você pode nem precisar dele. Ao converter de um conjunto de alterações do Perforce (Perforce changeset) em um commit do Git, o comportamento padrão do Git Fusion é procurar o usuário do Perforce e usar o endereço de e-mail e o nome completo armazenado nele para o campo autor/commiter do Git. Ao se converter da outra forma, o padrão é pesquisar no usuário do Perforce com o endereço de e-mail armazenado no campo do autor (author field) do commit do Git e enviar o conjunto de alterações (changeset) como esse usuário (com a aplicação das permissões). Na maioria dos casos, esse comportamento servirá perfeitamente, mas considere o seguinte arquivo de mapeamento:

```
john john@example.com "John Doe"
john johnny@appleseed.net "John Doe"
bob employeeX@example.com "Anon X. Mouse"
joe employeeY@example.com "Anon Y. Mouse"
```

Cada linha está no formato `<user> <email> "<full name>"` e cria um mapeamento de usuário único. As duas primeiras linhas mapeiam dois endereços de email distintos na mesma conta de usuário do Perforce. Isso é útil se você criou commits do Git sob vários endereços de email diferentes (ou se alterou os endereços de email), mas deseja que eles sejam mapeados para o mesmo usuário do Perforce. Ao criar um commit do Git a partir de um conjunto de alterações do Perforce, a primeira linha correspondente ao usuário do Perforce é usada para obter informações da autoria no Git.

As duas últimas linhas mascaram os nomes e os endereços de e-mail de Bob e Joe para os commits do Git que são criados. Isso é bom se você quiser abrir o código de um projeto interno, mas não quiser publicar seu diretório de funcionários em todo o mundo. Observe que os endereços de e-mail e os nomes completos devem ser únicos, a menos que você queira que todos os commits do Git sejam atribuídas a um único autor fictício.

Fluxo de Trabalho

O Perforce Git Fusion é uma ponte de mão dupla (two-way bridge) entre o Perforce e o controle de versão do Git. Vamos dar uma olhada em como é trabalhar no lado do Git. Vamos supor que tenhamos feito o mapeamento no projeto “Jam” usando um arquivo de configuração como o mostrado acima, que podemos fazer um clone assim:

```
$ git clone https://10.0.1.254/Jam
Cloning into 'Jam'...
Username for 'https://10.0.1.254': john
Password for 'https://john@10.0.1.254':
```

```
remote: Counting objects: 2070, done.
remote: Compressing objects: 100% (1704/1704), done.
Receiving objects: 100% (2070/2070), 1.21 MiB | 0 bytes/s, done.
remote: Total 2070 (delta 1242), reused 0 (delta 0)
Resolving deltas: 100% (1242/1242), done.
Checking connectivity... done.
$ git branch -a
* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/master
  remotes/origin/rel2.1
$ git log --oneline --decorate --graph --all
* 0a38c33 (origin/rel2.1) Create Jam 2.1 release branch.
| * d254865 (HEAD, origin/master, origin/HEAD, master) Upgrade to latest metrowerks on
Beos -- the Intel one.
| * bd2f54a Put in fix for jam's NT handle leak.
| * c0f29e7 Fix URL in a jam doc
| * cc644ac Radstone's lynx port.
[...]
```

Da primeira vez em que você fizer isso, pode demorar algum tempo. O que está acontecendo é que o Git Fusion está convertendo todos os conjuntos de alterações aplicáveis no histórico do Perforce em commits do Git. Isso acontece localmente no servidor, portanto, é relativamente rápido, mas, se você tem muito histórico, ainda pode demorar algum tempo. As buscas subsequentes fazem a conversão em incrementos; por isso, se sentirá mais parecida com a velocidade nativa do Git.

Como você pode ver, o nosso repositório se parece exatamente com qualquer outro repositório Git com o qual você possa trabalhar. Existem três filiais e o Git criou utilmente uma branch `master` local que rastreia a `origin/master`. Vamos trabalhar um pouco e criar alguns novos commits:

```
# ...
$ git log --oneline --decorate --graph --all
* cfd46ab (HEAD, master) Add documentation for new feature
* a730d77 Whitespace
* d254865 (origin/master, origin/HEAD) Upgrade to latest metrowerks on Beos -- the
Intel one.
* bd2f54a Put in fix for jam's NT handle leak.
[...]
```

Temos duas comissões novas. Agora vamos verificar se alguém está trabalhando:

```
$ git fetch
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From https://10.0.1.254/Jam
   d254865..6afeb15  master     -> origin/master
```

```
$ git log --oneline --decorate --graph --all
* 6afeb15 (origin/master, origin/HEAD) Update copyright
| * cfd46ab (HEAD, master) Add documentation for new feature
| * a730d77 Whitespace
|/
* d254865 Upgrade to latest metrowerks on Beos -- the Intel one.
* bd2f54a Put in fix for jam's NT handle leak.
[...]
```

Parece que alguém sim! Você não saberia disso a partir desta visão, mas o commit `6afeb15` foi de fato criado utilizando um cliente Perforce. Ele parece apenas mais um commit sob o ponto de vista do Git, que é exatamente o ponto em questão. Vamos ver como o servidor Perforce lida com um merge de commit:

```
$ git merge origin/master
Auto-merging README
Merge made by the 'recursive' strategy.
 README | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
Counting objects: 9, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 917 bytes | 0 bytes/s, done.
Total 9 (delta 6), reused 0 (delta 0)
remote: Perforce: 100% (3/3) Loading commit tree into memory...
remote: Perforce: 100% (5/5) Finding child commits...
remote: Perforce: Running git fast-export...
remote: Perforce: 100% (3/3) Checking commits...
remote: Processing will continue even if connection is closed.
remote: Perforce: 100% (3/3) Copying changelists...
remote: Perforce: Submitting new Git commit objects to Perforce: 4
To https://10.0.1.254/Jam
   6afeb15..89cba2b  master -> master
```

O Git acha que deu certo. Vamos dar uma olhada no histórico do arquivo `README` sob o ponto de vista do Perforce, usando o recurso de gráfico de revisões (revision graph) do `p4v`:

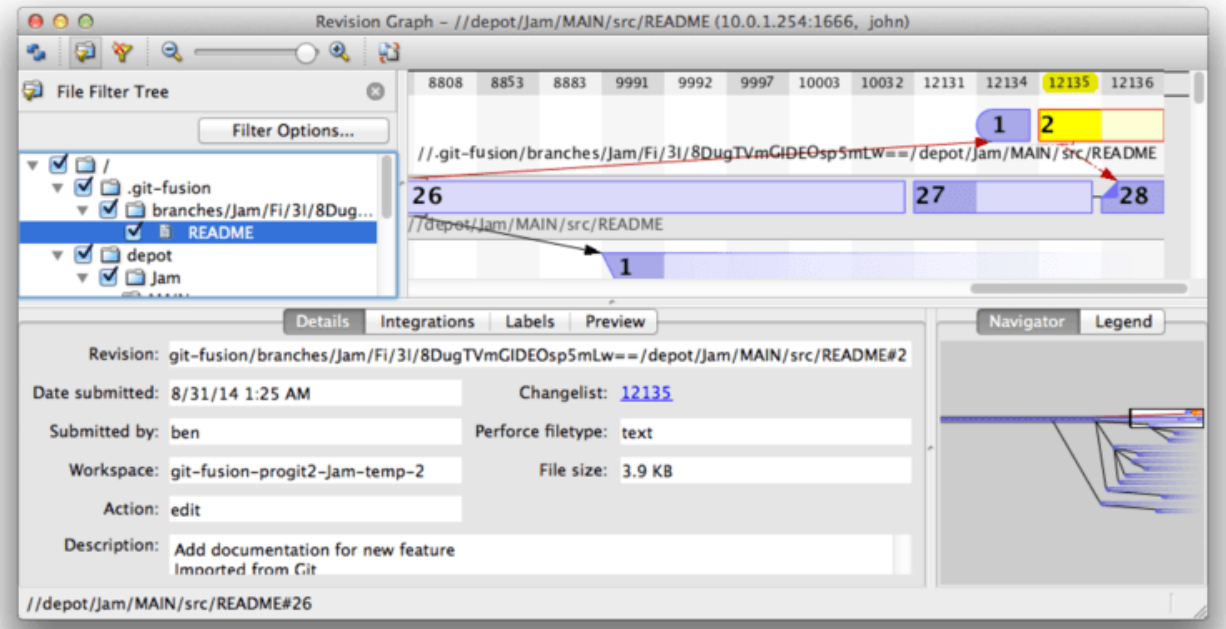


Figure 172. O gráfico de revisão do Perforce resultante da execução do push do Git

Se você nunca viu essa visão antes, pode parecer confuso, mas ela mostra os mesmos conceitos que em um visualizador gráfico (graphical viewer) em relação ao histórico do Git. Estamos examinando o histórico do arquivo `README`, portanto, a árvore de diretórios no canto superior esquerdo mostra apenas esse arquivo à medida que ele surge em várias ramificações (branches). No canto superior direito, temos um gráfico visual de como as diferentes revisões do arquivo estão relacionadas, e a visão geral deste gráfico está no canto inferior direito. O resto da exibição (view) é entregue à exibição de detalhes para a revisão selecionada (`2` neste caso).

Uma coisa a se notar é que o gráfico é exatamente como o do histórico do Git. O Perforce não possuía um branch nomeado para armazenar os commits `1` e `2`, portanto, ele fez um branch "anonymous" no diretório `.git-fusion` para retê-lo. Isso também acontecerá nos branches do Git nomeados que não correspondam a um branch do Perforce nomeada (e você pode mapeá-los para um branch do Perforce usando o arquivo de configuração posteriormente).

A maior parte disso acontece nos bastidores, mas o resultado final é que uma pessoa na equipe pode estar usando o Git, outra pode usar o Perforce e nenhum deles saberá sobre a escolha do outro.

Resumo do Git-Fusion

Se você tem (ou consegue obter) acesso ao seu servidor Perforce, o Git Fusion é uma ótima maneira de fazer o Git e o Perforce conversarem um com o outro. Há um pouco de configuração envolvida, mas a curva de aprendizado não é muito acentuada. Esta é uma das poucas seções deste capítulo nas quais os avisos sobre o uso de todo o poder do Git não serão exibidas. Isso não quer dizer que o Perforce ficará satisfeito com tudo o que você jogar sobre ele - se você tentar reescrever o histórico que já foi enviado (pushed), o Git Fusion irá rejeitá-lo - mas o Git Fusion se esforça muito para se parecer com algo nativo. Você pode até usar submódulos Git (embora pareçam estranhos para os usuários do Perforce) e consolidar os branches (merge) (isso será gravado como uma integração no lado do Perforce).

Se você não conseguir convencer o administrador do seu servidor a configurar o Git Fusion, ainda há uma maneira de usar essas ferramentas juntas.

Git-p4

O Git-p4 é uma ponte de mão dupla entre Git e o Perforce. Ele é executado inteiramente no seu repositório Git, portanto você não precisará de nenhum tipo de acesso ao servidor Perforce (além das credenciais do usuário, é claro). O Git-p4 não é uma solução tão flexível ou completa quanto o Git Fusion, mas permite que você faça a maior parte do que deseja sem ser invasivo no ambiente do servidor.



Você precisará da ferramenta `p4` em algum lugar no seu `PATH` para trabalhar com o git-p4. No momento em que este documento foi escrito, ele está disponível gratuitamente em https://www.perforce.com/downloads/helix-command-line-client-p4.

Configurando

Por exemplo, estaremos executando o servidor Perforce do OVA Git Fusion como mostrado acima, mas contornaremos (bypass) o servidor Git Fusion e acessaremos diretamente o controle de versão do Perforce.

Para usar o cliente de linha de comando `p4` (do qual o git-p4 depende), você precisará definir algumas variáveis de ambiente:

```
$ export P4PORT=10.0.1.254:1666
$ export P4USER=john
```

Primeiros Passos

Como qualquer coisa no Git, o primeiro comando é o clone:

```
$ git p4 clone //depot/www/live www-shallow
Importing from //depot/www/live into www-shallow
Initialized empty Git repository in /private/tmp/www-shallow/.git/
Doing initial import of //depot/www/live/ from revision #head into
refs/remotes/p4/master
```

Isso cria o que em termos do Git é um clone "raso (`shallow`)"; apenas a última revisão do Perforce é importada para o Git; Lembre-se, o Perforce não foi projetado para fornecer todas as revisões a todos os usuários. Isso é suficiente para usar o Git como um cliente do Perforce, mas, para outros propósitos, não é.

Quando terminar, temos um repositório Git totalmente funcional:

```
$ cd myproject
$ git log --oneline --all --graph --decorate
```

```
* 70eaf78 (HEAD, p4/master, p4/HEAD, master) Initial import of //depot/www/live/ from
the state at revision #head
```

Observe como há um remoto (remote) “p4” para o servidor Perforce, mas todo o resto parece um clone padrão. Na verdade, isso é um pouco enganoso; na verdade, não há nada remoto (remote) lá.

```
$ git remote -v
```

Não há remotos neste repositório. O Git-p4 criou algumas referências (refs) para representar o estado do servidor e elas parecem com referências remotas do `git log`, mas não são gerenciadas pelo próprio Git e não podem sofrer o comando push.

Fluxo de Trabalho

Tudo bem, vamos trabalhar um pouco. Vamos supor que você tenha feito algum progresso em um recurso (feature) muito importante e esteja pronto para exibi-lo ao resto de sua equipe.

```
$ git log --oneline --all --graph --decorate
* 018467c (HEAD, master) Change page title
* c0fb617 Update link
* 70eaf78 (p4/master, p4/HEAD) Initial import of //depot/www/live/ from the state at
revision #head
```

Realizamos duas novas comissões e estamos prontos para enviar ao servidor Perforce. Vamos verificar se alguém estava trabalhando hoje:

```
$ git p4 sync
git p4 sync
Performing incremental import into refs/remotes/p4/master git branch
Depot paths: //depot/www/live/
Import destination: refs/remotes/p4/master
Importing revision 12142 (100%)
$ git log --oneline --all --graph --decorate
* 75cd059 (p4/master, p4/HEAD) Update copyright
| * 018467c (HEAD, master) Change page title
| * c0fb617 Update link
|/
* 70eaf78 Initial import of //depot/www/live/ from the state at revision #head
```

Parece que estavam, e a `master` e a `p4/master` divergiram. O sistema de ramificações (branching system) do Perforce não é *nada* como o do Git, portanto o envio de um commit consolidado (merge commits) não faz nenhum sentido. O Git-p4 recomenda que você refaça os (rebase) seus commits, e até acompanha um atalho para fazer isso:

```
$ git p4 rebase
Performing incremental import into refs/remotes/p4/master git branch
```

```
Depot paths: //depot/www/live/
No changes to import!
Rebasing the current branch onto remotes/p4/master
First, rewinding head to replay your work on top of it...
Applying: Update link
Applying: Change page title
 index.html | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Você provavelmente pode dizer pela saída, mas o `git p4 rebase` é um atalho para o `git p4 sync` seguido do `git rebase p4/master`. É um pouco mais inteligente que isso, especialmente ao trabalhar com várias filiais (branches), mas essa é uma boa aproximação.

Agora, nossa história é linear novamente e estamos prontos para contribuir com nossas alterações de volta no Perforce. O comando `git p4 submit` tentará criar uma nova revisão no Perforce para cada commit do Git que esteja entre a `p4/master` e a `master`. Sua execução nos leva ao nosso editor favorito, e o conteúdo do arquivo se parece com isto:

```
# A Perforce Change Specification.
#
#  Change:      The change number. 'new' on a new changelist.
#  Date:        The date this specification was last modified.
#  Client:      The client on which the changelist was created.  Read-only.
#  User:        The user who created the changelist.
#  Status:      Either 'pending' or 'submitted'. Read-only.
#  Type:        Either 'public' or 'restricted'. Default is 'public'.
#  Description: Comments about the changelist.  Required.
#  Jobs:        What opened jobs are to be closed by this changelist.
#               You may delete jobs from this list.  (New changelists only.)
#  Files:       What opened files from the default changelist are to be added
#               to this changelist.  You may delete files from this list.
#               (New changelists only.)

Change:  new

Client:  john_bens-mbp_8487

User: john

Status:  new

Description:
   Update link

Files:
   //depot/www/live/index.html   # edit


######## git author ben@straub.cc does not match your p4 account.
######## Use option --preserve-user to modify authorship.
```

```
######## Variable git-p4.skipUserNameCheck hides this message.
######## everything below this line is just the diff #######
--- //depot/www/live/index.html  2014-08-31 18:26:05.000000000 0000
+++ /Users/ben/john_bens-mbp_8487/john_bens-mbp_8487/depot/www/live/index.html   2014-
08-31 18:26:05.000000000 0000
@@ -60,7 +60,7 @@
 </td>
 <td valign=top>
 Source and documentation for
-<a href="http://www.perforce.com/jam/jam.html">
+<a href="jam.html">
 Jam/MR</a>,
 a software build tool.
 </td>
```

Este é basicamente o mesmo conteúdo que você veria na execução da opção `p4 submit`, exceto o material no final onde o git-p4 o incluiu de forma prestativa. O Git-p4 tenta honrar com as suas configurações do Git e do Perforce individualmente, quando for necessário fornecer um nome para o commit ou o seu changeset (conjunto de alterações), mas em alguns casos, será preciso substituí-lo. Por exemplo, se a confirmação (commit) do Git importada foi escrita por um contribuinte que não tem uma conta de usuário no Perforce, você ainda pode querer que o changeset pareça que ele que o escreveu (e não você).

O Git-p4 importou generosamente a mensagem do commit do Git como o conteúdo desse changeset do Perforce; portanto, tudo o que precisamos fazer é salvar e sair duas vezes (uma para cada commit). A saída (output) resultante do shell será parecida com isto:

```
$ git p4 submit
Perforce checkout for depot path //depot/www/live/ located at /Users/ben/john_bens-
mbp_8487/john_bens-mbp_8487/depot/www/live/
Synchronizing p4 checkout...
... - file(s) up-to-date.
Applying dbac45b Update link
//depot/www/live/index.html#4 - opened for edit
Change 12143 created with 1 open file(s).
Submitting change 12143.
Locking 1 files ...
edit //depot/www/live/index.html#5
Change 12143 submitted.
Applying 905ec6a Change page title
//depot/www/live/index.html#5 - opened for edit
Change 12144 created with 1 open file(s).
Submitting change 12144.
Locking 1 files ...
edit //depot/www/live/index.html#6
Change 12144 submitted.
All commits applied!
Performing incremental import into refs/remotes/p4/master git branch
Depot paths: //depot/www/live/
```

```
Import destination: refs/remotes/p4/master
Importing revision 12144 (100%)
Rebasing the current branch onto remotes/p4/master
First, rewinding head to replay your work on top of it...
$ git log --oneline --all --graph --decorate
* 775a46f (HEAD, p4/master, p4/HEAD, master) Change page title
* 05f1ade Update link
* 75cd059 Update copyright
* 70eaf78 Initial import of //depot/www/live/ from the state at revision #head
```

O resultado é como se tivéssemos feito um `git push`, que é a analogia mais próxima ao que realmente aconteceu.

Note que durante esse processo todo commit no Git se transforma em um changeset no Perforce; se você quiser esmagá-los em um único changeset, poderá fazê-lo com um rebase interativo antes de executar a opção `git p4 submit`. Note também que as hashes de SHA-1 em todos os commits enviados (submitted) como changesets foram alteradas; isto é devido a uma linha a mais ao final de cada commit no qual o git-p4 foi convertido:

```
$ git log -1
commit 775a46f630d8b46535fc9983cf3ebe6b9aa53145
Author: John Doe <john@example.com>
Date:   Sun Aug 31 10:31:44 2014 -0800

    Change page title

    [git-p4: depot-paths = "//depot/www/live/": change = 12144]
```

O que acontecerá se você tentar enviar um commit consolidado (merge commit)? Vamos tentar. Eis a situação na qual nos encontramos:

```
$ git log --oneline --all --graph --decorate
* 3be6fd8 (HEAD, master) Correct email address
*   1dcbf21 Merge remote-tracking branch 'p4/master'
|\
| * c4689fc (p4/master, p4/HEAD) Grammar fix
* | cbacd0a Table borders: yes please
* | b4959b6 Trademark
|/
* 775a46f Change page title
* 05f1ade Update link
* 75cd059 Update copyright
* 70eaf78 Initial import of //depot/www/live/ from the state at revision #head
```

O histórico no Git e no Perforce diverge após a `775a46f`. O lado do Git possui duas comissões, logo em seguida, um merge de commits com o head no Perforce e então um outro commit. Vamos tentar enviar isso no topo de um changeset único do lado do Perforce. Vejamos o que acontece se

tentarmos os enviar agora:

```
$ git p4 submit -n
Perforce checkout for depot path //depot/www/live/ located at /Users/ben/john_bens-
mbp_8487/john_bens-mbp_8487/depot/www/live/
Would synchronize p4 checkout in /Users/ben/john_bens-mbp_8487/john_bens-
mbp_8487/depot/www/live/
Would apply
  b4959b6 Trademark
  cbacd0a Table borders: yes please
  3be6fd8 Correct email address
```

A bandeira (flag) `-n` é um atalho para a `--dry-run`, a qual tenta informar o que ocorreria se o comando enviar (submit) rodasse em caráter definitivo. Neste caso, parece que estaríamos criando três changesets do Perforce, as quais correspondem a três commits de não-mesclagem que ainda não existem no servidor do Perforce. Parece exatamente o que procuramos, vejamos como ele se tornará:

```
$ git p4 submit
[…]
$ git log --oneline --all --graph --decorate
* dadbd89 (HEAD, p4/master, p4/HEAD, master) Correct email address
* 1b79a80 Table borders: yes please
* 0097235 Trademark
* c4689fc Grammar fix
* 775a46f Change page title
* 05f1ade Update link
* 75cd059 Update copyright
* 70eaf78 Initial import of //depot/www/live/ from the state at revision #head
```

Nossa história se tornou linear, exatamente como se a tivéssemos refeito a base (rebased) antes de submeter (que é de fato e exatamente o que aconteceu). Isso significa que você pode estar livre para criar, trabalhar, descartar e unificar os branches do lado do Git, sem receios de que a sua história se torne de alguma maneira, incompatível com o Perforce. Se você puder fazer isso (rebase), poderá contribuir para um servidor do Perforce.

Ramificação (Branching)

Se o seu projeto no Perforce tem múltiplas branches (ramificações), a sua sorte não esgotou; o git-p4 pode os administrar a fazer a impressão de um modo no qual se perceba como o Git. Digamos que o seu depósito do Perforce seja exposto da seguinte forma:

```
//depot
   └── project
       ├── main
       └── dev
```

E vamos dizer que você possua um branch `dev`, no qual tenha uma especificação de visão que se pareça a esta:

```
//depot/project/main/... //depot/project/dev/...
```

O Git-p4 pode detectar automaticamente a situação e fazer a coisa certa:

```
$ git p4 clone --detect-branches //depot/project@all
Importing from //depot/project@all into project
Initialized empty Git repository in /private/tmp/project/.git/
Importing revision 20 (50%)
    Importing new branch project/dev

    Resuming with change 20
Importing revision 22 (100%)
Updated branches: main dev
$ cd project; git log --oneline --all --graph --decorate
* eae77ae (HEAD, p4/master, p4/HEAD, master) main
| * 10d55fb (p4/project/dev) dev
| * a43cfae Populate //depot/project/main/... //depot/project/dev/....
|/
* 2b83451 Project init
```

Observe o especificador “@all” no caminho (path) do depósito; que diz a git-p4 para clonar não apenas o changeset mais recente nessa sub-árvore (subtree), como a todos os changesets que em algum momento já mexeram naqueles caminhos. Isto se aproxima mais aos conceitos do Git em um clone, mas, se você estiver trabalhando num projeto de longo histórico, pode levar algum tempo.

A bandeira (flag) `--detect-branches` diz ao git-p4 que utilize as especificações dos branches do Perforce a fim de mapear as refs de Git nas branches. Se os mapeamentos não estiverem presentes no servidor Perforce (onde sua utilidade no Perforce é de um modo perfeitamente válido), você pode dizer ao git-p4 do que os mapeamentos das branches se tratam e que você obterá o mesmo resultado:

```
$ git init project
Initialized empty Git repository in /tmp/project/.git/
$ cd project
$ git config git-p4.branchList main:dev
$ git clone --detect-branches //depot/project@all .
```

Ao setar a variável de configuração da `git-p4.branchList` em relação a `main:dev` informará a git-p4 que ambas as ramificações (branches) “main” e “dev” se tratam da mesma, onde a segunda em questão seja a filha da primeira.

Se nós usarmos `git checkout -b dev p4/project/dev` no momento e fizermos algumas comissões, o git-p4 será esperto o suficiente a destinar o branch correto, quando estivermos com o `git p4 submit`. Infelizmente, o git-p4 não pode misturar clones rasos (shallow clones) com as múltiplas

ramificações (branches); caso você tenha um enorme projeto e deseja trabalhar com mais de um branch, você deverá usar a função `git p4 clone` uma vez a cada branch ao qual você quer destinar as suas submissões.

Ao se criarem as branches de integração, você terá de fazer o uso de um cliente Perforce (Perforce client). O git-p4 só pode sincronizar e se destinar aos branches os quais estão em existência, e a cada changeset em nível linear a seu tempo. Se você unir (merge) dois branches no Git, e tentar enviar o novo changeset, tudo o que for estarão gravadas é um punhado de alterações nos arquivos; os metadados acerca do qual the sobre os branches que se envolvam nas perdas destas integrações.

Resumo do Git e do Perforce

O Git-p4 possibilita ao uso com o workflow do Git acompanhado de um servidor Perforce, e ele o faz muito bem. Porém, é importante recordar que o Perforce se responsabiliza da fonte de seus projetos, e que você apenas usa o Git de maneira local. Apenas tenha muito cuidado ao dividir as suas comissões (commits) no Git; caso você possua um acesso remoto ao qual as outras pessoas usam, não submeta (push) nenhum commit onde os mesmos não tenham sido enviados previamente a um servidor do Perforce.

Se você desejar que eles se unam (mix) de forma livre no controle das fontes (source control) de clientes (clients) entre o Perforce e o Git, e se for possível o convencimento do seu administrador de servidores à instalação dele, o Git Fusion faz do uso do Git num controle de versão clientelar da mais alta linha (first-class), em favor de um servidor de Perforce.

Migrando para o Git

Se você possui uma base de código existente em outro VCS, mas decidiu começar a usar o Git, deve migrar seu projeto de uma maneira ou de outra. Esta seção aborda alguns importadores para sistemas comuns e demonstra como desenvolver seu próprio importador personalizado. Você aprenderá como importar dados de vários dos maiores sistemas SCM usados profissionalmente, porque eles compõem a maioria dos usuários que estão mudando e porque é fácil encontrar ferramentas de alta qualidade para eles.

Subversion

Se você leu a seção anterior sobre o uso do `git svn`, poderá facilmente usar essas instruções para fazer o `git svn clone` de um repositório; em seguida, pare de usar o servidor Subversion, dê um push para um novo servidor Git e comece a usá-lo. Se você quiser o histórico, você pode fazer isso o mais rápido que puder extraindo os dados do servidor Subversion (o que pode demorar um pouco).

No entanto, a importação não é perfeita; e como demorará muito, você pode fazer isso da maneira correta. O primeiro problema é a informação do autor. No Subversion, cada pessoa submetida tem um usuário no sistema que é gravado nas informações de commit. Os exemplos na seção anterior mostram `schacon` em alguns lugares, como a saída `blame` e o `git svn log`. Se você deseja mapear isso para obter melhores dados de autor do Git, é necessário um mapeamento dos usuários do Subversion para os autores do Git. Crie um arquivo chamado `users.txt` que possua esse mapeamento em um formato como este:

```
schacon = Scott Chacon <schacon@geemail.com>
selse = Someo Nelse <selse@geemail.com>
```

Para obter uma lista dos nomes de autores que o SVN usa, você pode executar o seguinte:

```
$ svn log --xml --quiet | grep author | sort -u | \
  perl -pe 's/.*>(.*?)<.*/$1 = /'
```

Isso gera a saída do log no formato XML, mantém apenas as linhas com informações do autor, descarta duplicatas e retira as tags XML. Obviamente, isso só funciona em uma máquina com o `grep`, `sort` e `perl` instalados. Em seguida, redirecione a saída para o arquivo `users.txt`, para que você possa adicionar os dados equivalentes do usuário do Git ao lado de cada entrada.



Se você está tentando fazer isso em uma máquina Windows, este é o ponto em que terá problemas. A Microsoft forneceu bons conselhos e exemplos em https://learn.microsoft.com/en-us/azure/devops/repos/git/perform-migration-from-svn-to-git.

Você pode fornecer esse arquivo ao `git svn` para ajudá-lo a mapear os dados do autor com mais precisão. Você também pode informar o `git svn` a não incluir os metadados que o Subversion importa normalmente, passando o `--no-metadata` para o comando `clone` ou `init`. Os metadados incluem um `git-svn-id` dentro de cada mensagem de commit que o Git irá gerar durante a importação. Isso pode inchar o seu log do Git e torná-lo um pouco incerto.



Você precisa manter os metadados quando quiser espelhar commits feitos no repositório do Git de volta no repositório original do SVN. Se você não deseja que a sincronização em seu log de commit ocorra, fique à vontade para omitir o parâmetro `--no-metadata`.

Isso faz o seu comando de `import` parecer com isto:

```
$ git svn clone http://my-project.googlecode.com/svn/ \
      --authors-file=users.txt --no-metadata --prefix "" -s my_project
$ cd my_project
```

Agora você deve ter uma importação (import) do Subversion melhor em seu diretório `my_project`. Em vez dos commits que se parecem com isto:

```
commit 37efa680e8473b615de980fa935944215428a35a
Author: schacon <schacon@4c93b258-373f-11de-be05-5f7a86268029>
Date:   Sun May 3 00:12:22 2009 +0000

    fixed install - go to trunk

    git-svn-id: https://my-project.googlecode.com/svn/trunk@94 4c93b258-373f-11de-
```

```
    be05-5f7a86268029
```

eles se parecem assim:

```
commit 03a8785f44c8ea5cdb0e8834b7c8e6c469be2ff2
Author: Scott Chacon <schacon@geemail.com>
Date:   Sun May 3 00:12:22 2009 +0000

    fixed install - go to trunk
```

Não apenas o campo Autor (Author) parece muito melhor, mas o `git-svn-id` não está mais lá também.

Você também deve fazer um pouco de limpeza pós-importação (post-import cleanup). Por um lado, você deve limpar as referências estranhas que o `git svn` configurou. Primeiro, você moverá as tags para que sejam de fato tags, em vez de ramificações (branches) remotas estranhas e, em seguida, moverá o restante das filiais para que sejam locais.

Para mover as tags para se tornarem em tags adequadas do Git, execute:

```
$ for t in $(git for-each-ref --format='%(refname:short)' refs/remotes/tags); do git
tag ${t/tags\//} $t && git branch -D -r $t; done
```

Isso utiliza as referências (refs) que eram filiais (branches) remotas que começavam com `refs/remotes/tags/` e as torna verdadeiras tags (mais leves).

Em seguida, mova o restante das referências sob o `refs/remotes` para que se tornem em ramificações (branches) locais:

```
$ for b in $(git for-each-ref --format='%(refname:short)' refs/remotes); do git branch
$b refs/remotes/$b && git branch -D -r $b; done
```

Pode acontecer que você veja alguns branches extras com o sufixo `@xxx` (onde xxx é um número), enquanto no Subversion você vê apenas uma filial. Na verdade, trata-se de um recurso do Subversion chamado “peg-revisions” (revisões vinculadas), que é algo pelo qual o Git simplesmente não possui contrapartida sintática. Por isso, o `git svn` simplesmente adiciona o número da versão SVN ao nome da branch, da mesma forma que você o escreveria no SVN para abordar a peg-revision daquele branch. Se você não se importa mais com as peg-revisions, simplesmente as remova:

```
$ for p in $(git for-each-ref --format='%(refname:short)' | grep @); do git branch -D
$p; done
```

Agora, todos os branches antigos são realmente branches do Git e todas as tags antigas são as do Git.

Há uma última coisa para limpar. Infelizmente, o `git svn` cria um branch extra chamado `trunk`, no qual mapeia para a branch padrão do Subversion, mas o `trunk` aponta ao mesmo lugar que o `master`. Desde que o `master` é mais idomaticamente Git, aqui está o modo de como se remover o branch a mais:

```
$ git branch -d trunk
```

A última coisa a fazer é adicionar o seu novo servidor Git como um arquivo remoto e fazer um push a ele. Aqui está um exemplo sobre como adicionar seu servidor como um controle remoto:

```
$ git remote add origin git@my-git-server:myrepository.git
```

Como você deseja que todas os seus branches e tags se levantem, agora você pode executar isso:

```
$ git push origin --all
$ git push origin --tags
```

Todos as suas filiais (branches) e tags devem estar no seu novo servidor Git em uma importação limpa e agradável.

Mercurial

Uma vez que o Mercurial e o Git têm modelos bastante parecidos para a representação de versões, e visto que o Git é um pouco mais flexível, converter de um repositório no Mercurial para o Git é bem simples, utilizando-se a ferramenta chamada "hg-fast-export", da qual você precisará uma cópia:

```
$ git clone https://github.com/frej/fast-export.git
```

O primeiro passo para a conversão é efetuar um clone por completo do repositório no Mercurial a que se queira converter:

```
$ hg clone <remote repo URL> /tmp/hg-repo
```

O próximo passo é a criação de um arquivo para o mapeamento dos autores. O Mercurial é mais tolerante do que o Git em virtude do que colocará no campo dos autores em relação aos conjuntos de alterações (changesets); sendo assim, essa é a hora de limpar a casa. A geração dele é na base de um comando de linha única no shell do `bash`:

```
$ cd /tmp/hg-repo
$ hg log | grep user: | sort | uniq | sed 's/user: *//' > ../authors
```

Isto deve levar alguns segundos, a depender em quão longo seja a história do seu projeto, e logo após, o arquivo `/tmp/authors` se parecerá com algo do tipo:

```
bob
bob@localhost
bob <bob@company.com>
bob jones <bob <AT> company <DOT> com>
Bob Jones <bob@company.com>
Joe Smith <joe@company.com>
```

Nesse exemplo, a mesma pessoa (Bob) criou os changesets com quatro nomes diferentes, com o qual somente um deles pareça correto, e onde um deles seja completamente inválido por um commit no Git. A hg-fast-export nos permite consertar a fazer com que cada linha se torne uma regra: `"<input>"="<output>"`, ao mapear um `<input>` em favor de um `<output>`. Dentro das sequências de strings (strings) `<input>` e `<output>`, todas as saídas das sequências codificadas do `string_escape` no Python são suportadas. Se o arquivo de mapeamento do autor não conter uma correspondência com o `<input>`, o mesmo autor será enviado ao Git sem sofrer as alterações. Se todos os nomes dos usuários parecem bem, não será necessário a sua presença num arquivo para eles em momento nenhum. No exemplo, nós queremos que o nosso arquivo pareça assim:

```
"bob"="Bob Jones <bob@company.com>"
"bob@localhost"="Bob Jones <bob@company.com>"
"bob <bob@company.com>"="Bob Jones <bob@company.com>"
"bob jones <bob <AT> company <DOT> com>"="Bob Jones <bob@company.com>"
```

O mesmo tipo do arquivo de mapeamento pode ser usado na renomeação dos branches e das tags quando o nome no Mercurial não for permitido pelo Git.

O próximo passo é criar o nosso novo repositório no Git e rodar o script de exportação:

```
$ git init /tmp/converted
$ cd /tmp/converted
$ /tmp/fast-export/hg-fast-export.sh -r /tmp/hg-repo -A /tmp/authors
```

A bandeira (flag) `-r` diz ao hg-fast-export onde encontrar o repositório do Mercurial que desejamos converter, e a bandeira `-A` nos diz onde encontrar o arquivo para os mapeamentos de autoria (os arquivos de mapeamentos das branches e das tags são especificados com as flags `-B` e `-T`, respectivamente). O script faz a análise sintática (parses) aos changesets do Mercurial e os converte em um script voltado para o recurso do Git, "fast-import" (do qual discutiremos detalhadamente mais à frente). Isto toma um pouco do seu tempo (apesar de ser *muito* mais rápido do que numa base em rede (network)), e a saída (output) costuma ser muito prolixa:

```
$ /tmp/fast-export/hg-fast-export.sh -r /tmp/hg-repo -A /tmp/authors
Loaded 4 authors
master: Exporting full revision 1/22208 with 13/0/0 added/changed/removed files
master: Exporting simple delta revision 2/22208 with 1/1/0 added/changed/removed files
master: Exporting simple delta revision 3/22208 with 0/1/0 added/changed/removed files
[…]
master: Exporting simple delta revision 22206/22208 with 0/4/0 added/changed/removed
```

```
files
master: Exporting simple delta revision 22207/22208 with 0/2/0 added/changed/removed
files
master: Exporting thorough delta revision 22208/22208 with 3/213/0
added/changed/removed files
Exporting tag [0.4c] at [hg r9] [git :10]
Exporting tag [0.4d] at [hg r16] [git :17]
[…]
Exporting tag [3.1-rc] at [hg r21926] [git :21927]
Exporting tag [3.1] at [hg r21973] [git :21974]
Issued 22315 commands
git-fast-import statistics:
---------------------------------------------------------------------
Alloc'd objects:     120000
Total objects:       115032 (    208171 duplicates                  )
      blobs  :        40504 (    205320 duplicates      26117 deltas of      39602
attempts)
      trees  :        52320 (      2851 duplicates      47467 deltas of      47599
attempts)
      commits:        22208 (         0 duplicates          0 deltas of          0
attempts)
      tags   :            0 (         0 duplicates          0 deltas of          0
attempts)
Total branches:         109 (         2 loads     )
      marks:        1048576 (     22208 unique    )
      atoms:           1952
Memory total:          7860 KiB
       pools:          2235 KiB
     objects:          5625 KiB
---------------------------------------------------------------------
pack_report: getpagesize()            =       4096
pack_report: core.packedGitWindowSize = 1073741824
pack_report: core.packedGitLimit      = 8589934592
pack_report: pack_used_ctr            =      90430
pack_report: pack_mmap_calls          =      46771
pack_report: pack_open_windows        =          1 /          1
pack_report: pack_mapped              =  340852700 /  340852700
---------------------------------------------------------------------

$ git shortlog -sn
   369  Bob Jones
   365  Joe Smith
```

É quase tudo o que há nisto. Todas as tags do Mercurial foram convertidas para as tags do Git, e as filiais (branches) e os favoritos (bookmarks) do Mercurial foram convertidos para as branches do Git. Agora você se encontra pronto para subir o seu repositório num push no lado dos servidores da sua casa (server-side home):

```
$ git remote add origin git@my-git-server:myrepository.git
```

```
$ git push origin --all
```

Perforce

O próximo sistema ao qual você abordará para importações é o Perforce. Conforme o que foi discutido anteriormente, existem duas maneiras das quais se possa fazer com que o Git e o Perforce conversem um com o outro: git-p4 e o Perforce Git Fusion.

Perforce Git Fusion

O Git Fusion torna esse processo quase indolor. Apenas configure as propriedades, mapeamento dos usuários, e os branches do seu projeto utilizando-se de um arquivo de configuração (conforme já foi discutido em Git Fusion), e efetue o clone do repositório. O Git Fusion fará com que pareça que você tem um repositório nativo do Git, do qual então, já se encontra pronto a ser empurrado (push) rumo ao seu host (hospedeiro) nativo no Git, se você assim quiser. Você poderá inclusive, fazer com que o Perforce se torne o seu host (hospedeiro) no Git se desejar.

Git-p4

O Git-p4 também pode atuar como uma ferramenta à importação. Como exemplo disto, iremos importar o projeto Jam a partir de um Depósito Público do Perforce (Perforce Public Depot). A fim de configurar o seu cliente, você deverá exportar a variável do ambiente P4PORT à apontar no depósito do Perforce:

```
$ export P4PORT=public.perforce.com:1666
```



Para que se possa prosseguir, você necessitará de um depósito Perforce para se conectar com ele. Usaremos o depósito público no site public.perforce.com como nosso exemplo, mas você também pode usar qualquer um ao qual você possua acesso.

Rode o comando `git p4 clone` na intenção de importar o projeto Jam do servidor Perforce, ao suprir o caminho (path) ao depósito e projeto e no caminho onde se tem o interesse em importar o seu projeto:

```
$ git-p4 clone //guest/perforce_software/jam@all p4import
Importing from //guest/perforce_software/jam@all into p4import
Initialized empty Git repository in /private/tmp/p4import/.git/
Import destination: refs/remotes/p4/master
Importing revision 9957 (100%)
```

Este projeto em particular só possui um branch, mas se você conter branches configurados em visualizações nos branches (ou somente numa configuração com diretórios), você poderá utilizar o marcador (flag) `--detect-branches` à função `git p4 clone` para importar a todas os branches deste projeto também. Verifique Ramificação (Branching) para um pouco mais de detalhes acerca disto.

Neste ponto em questão, você quase finalizou. Se você for ao diretório `p4import` e rodar a opção `git log`, você poderá ver o seu trabalho que foi importado:

```
$ git log -2
commit e5da1c909e5db3036475419f6379f2c73710c4e6
Author: giles <giles@giles@perforce.com>
Date:   Wed Feb 8 03:13:27 2012 -0800

    Correction to line 355; change </UL> to </OL>.

    [git-p4: depot-paths = "//public/jam/src/": change = 8068]

commit aa21359a0a135dda85c50a7f7cf249e4f7b8fd98
Author: kwirth <kwirth@perforce.com>
Date:   Tue Jul 7 01:35:51 2009 -0800

    Fix spelling error on Jam doc page (cummulative -> cumulative).

    [git-p4: depot-paths = "//public/jam/src/": change = 7304]
```

Você poderá ver que o `git-p4` deixou um identificador em cada mensagem de commit. Estará perfeitamente bem ao manter aquele identificador lá, para caso em que você necessite dar referências no número da mudança do Perforce de forma posterior. Ainda assim, se você desejar remover aquele identificador, a hora certa de fazê-lo é agora – antes que você comece com o seu trabalho no novo repositório. Você pode usar `git filter-branch` a remover o identificador de sequências num formato em massa (en masse):

```
$ git filter-branch --msg-filter 'sed -e "/^\[git-p4:/d"'
Rewrite e5da1c909e5db3036475419f6379f2c73710c4e6 (125/125)
Ref 'refs/heads/master' was rewritten
```

Se você rodar a opção `git log`, poderá ver que todos as somas de verificação (checksums) na forma SHA-1 destinadas as suas comissões (commits), houveram mudanças, mas que as sequências na `git-p4` não se encontram mais no formato de mensagens às comissões:

```
$ git log -2
commit b17341801ed838d97f7800a54a6f9b95750839b7
Author: giles <giles@giles@perforce.com>
Date:   Wed Feb 8 03:13:27 2012 -0800

    Correction to line 355; change </UL> to </OL>.

commit 3e68c2e26cd89cb983eb52c024ecdfba1d6b3fff
Author: kwirth <kwirth@perforce.com>
Date:   Tue Jul 7 01:35:51 2009 -0800

    Fix spelling error on Jam doc page (cummulative -> cumulative).
```

O seu documento importado se encontra apto para ser lançado (push up) de encontro ao seu novo servidor Git.

Um Importador Personalizado

Se o seu sistema não estiver entre nenhum dos listados acima, você deve procurar por um importador de forma online - os importadores de qualidade estão disponíveis em muitos outros sistemas, incluindo os de CVS, Clear Case, Visual Source Safe e até mesmo o diretório de arquivos. Se nenhuma dessas ferramentas funcionar para você, se você tiver uma ferramenta mais obscura ou se precisar de um processo de importação mais personalizado, use o `git fast-import`. O comando em questão ler as simples instruções do stdin ao escrever na especificação de dados do Git. Nesta maneira, criar objetos no Git é muito mais fácil do que se você estivesse rodando os comandos no Git em formatos crus, ou no caso da tentativa em redigir aos objetos neste mesmo formato (confira o Git Internals (Por Dentro do Git) para mais informações). Neste sentido, você pode escrever o script de um importador que ler as informações necessárias do lado de fora do sistema no qual você fará a importação, e fará a impressão sem rodeios das informações para o stdout. Em seguida, você pode executar esse programa e encaminhar sua saída para o `git fast-import` por meio de um pipe.

Para demonstrarmos mais rápido, você redigirá um importador simples. Suponha que você trabalhe em `current`, faça backups ocasionais do projeto copiando o diretório para um diretório de backup com data no formato `back_YYYY_MM_DD` e queira importar esse conteúdo para o Git. A estrutura do seu diretório se assemelha com isto:

```
$ ls /opt/import_from
back_2014_01_02
back_2014_01_04
back_2014_01_14
back_2014_02_03
current
```

Ao se importar um diretório no Git, você terá de fazer as revisões na maneira de como o Git guarda a base dos seus dados. Como você deve se lembrar, o Git é fundamentalmente uma lista encadeada de objetos commit que apontam para um snapshot do conteúdo. Tudo o que se deve fazer é contar ao `fast-import` que tipo de conteúdo os snapshots são, quais dados de commits lhes direcionam, e na ordem aos quais neles entram. Sua estratégia deve ser seguir pelos snapshots a um nível de cada vez, e criar os seus commits pelo conteúdo em cada um de seus diretórios, se ligando em todo e cada submetimento de volta ao de modo antecedente.

Assim como fizemos no Uma Política de Exemplo Imposta pelo Git, nós redigiremos isso em Ruby, já que este é pelo qual trabalhamos no modo geral, bem como tem a tendência em possuir uma leitura mais fácil. Você pode redigir a este exemplo de uma maneira muito simples sobre qualquer coisa na qual possua maior familiaridade – isto só necessita de sua impressão das devidas informações em prol do `stdout`. Se estiver usando o Windows, isso significa que você deverá tomar um cuidado especial para não inserir retornos de carro no final das linhas – o `git fast-import` exige apenas avanços de linha (LF), e não a combinação de retorno de carro e avanço de linha (CRLF) usada pelo Windows.

Para começar, entre no diretório de destino e identifique cada subdiretório; cada um deles é um snapshot que você deseja importar como um commit. Você fará a troca por entre cada um dos subdiretórios e imprimir os comandos que sejam necessários à finalidade de os exportar. O laço principal básico será semelhante a este:

```
last_mark = nil

# loop through the directories
Dir.chdir(ARGV[0]) do
  Dir.glob("*").each do |dir|
    next if File.file?(dir)

    # move into the target directory
    Dir.chdir(dir) do
      last_mark = print_export(dir, last_mark)
    end
  end
end
```

Você executa `print_export` dentro de cada diretório; o método recebe o manifesto e a marca do snapshot anterior e retorna o manifesto e a marca do snapshot atual, permitindo encadeá-los corretamente. “Mark” é o termo usado pelo `fast-import` para um identificador atribuído a um commit; conforme cria os commits, você atribui uma marca a cada um deles para poder referenciá-lo a partir de outros commits. Assim, a primeira coisa que deva ser feita no seu método de `print_export` é gerar uma marca advinda do nome de seus diretórios:

```
mark = convert_dir_to_mark(dir)
```

Você fará isso criando um array de diretórios e usando o valor do índice como marca, pois uma marca precisa ser um número inteiro. Seu método tem a aparência de:

```
$marks = []
def convert_dir_to_mark(dir)
  if !$marks.include?(dir)
    $marks << dir
  end
  ($marks.index(dir) + 1).to_s
end
```

Agora que você tem uma representação inteira do commit, precisa de uma data para os metadados dele. Como a data está expressa no nome do diretório, você fará sua análise sintática. A linha seguinte no seu arquivo em `print_export` será:

```
date = convert_dir_to_date(dir)
```

onde `convert_dir_to_date` (converter diretório para data) se defina a partir de:

```
def convert_dir_to_date(dir)
  if dir == 'current'
    return Time.now().to_i
  else
    dir = dir.gsub('back_', '')
    (year, month, day) = dir.split('_')
    return Time.local(year, month, day).to_i
  end
end
```

Isso retorna um valor inteiro para a data de cada diretório. A última metainformação necessária para cada commit são os dados do autor do commit, definidos diretamente em uma variável global:

```
$author = 'John Doe <john@example.com>'
```

Agora você está pronto para começar a imprimir os dados de commit para o importador. As informações iniciais indicam que você está definindo um objeto commit e em qual branch ele se encontra, seguidas pela marca gerada, pelos dados e pela mensagem do autor do commit e, por fim, pelo commit anterior, se houver. Os códigos tendem a ter essa visualização:

```
# print the import information
puts 'commit refs/heads/master'
puts 'mark :' + mark
puts "committer #{$author} #{date} -0700"
export_data('imported from ' + dir)
puts 'from :' + last_mark if last_mark
```

Você define diretamente o fuso horário (-0700), pois isso é simples. Se estiver importando de outro sistema, deverá especificar o fuso horário como um deslocamento. As mensagens dos envios têm de se expressar por um formato em especial:

```
data (size)\n(contents)
```

O formato consiste na palavra data, no tamanho dos dados que serão lidos, em uma quebra de linha e, por fim, nos dados. Como mais adiante você precisará usar o mesmo formato para especificar o conteúdo dos arquivos, crie um método auxiliar, `export_data`:

```
def export_data(string)
  print "data #{string.size}\n#{string}"
end
```

Resta apenas especificar o conteúdo dos arquivos de cada snapshot. Isso é fácil, pois cada snapshot

está em um diretório – você pode imprimir o comando `deleteall`, seguido pelo conteúdo de cada arquivo do diretório. O Git logo fará as gravações de cada snapshot da maneira em que se julgar como sendo a mais apropriada:

```
puts 'deleteall'
Dir.glob("**/*").each do |file|
  next if !File.file?(file)
  inline_data(file)
end
```

Observação: Como muitos sistemas tratam suas revisões como alterações de um commit para outro, o fast-import também aceita, em cada commit, comandos que especificam quais arquivos foram adicionados, removidos ou modificados e qual é o novo conteúdo. Você poderia calcular as diferenças entre os snapshots e fornecer apenas esses dados, mas isso é mais complexo – é mais simples fornecer todos os dados ao Git e deixar que ele descubra as diferenças. Se essa abordagem for mais adequada aos seus dados, consulte a página de manual do `fast-import` para saber como fornecê-los dessa maneira.

O formato para listar o conteúdo de um novo arquivo ou especificar um arquivo modificado com seu novo conteúdo é o seguinte:

```
M 644 inline path/to/file
data (size)
(file contents)
```

Aqui, 644 é o modo (se houver arquivos executáveis, você precisará detectá-los e especificar 755), e inline indica que o conteúdo será listado imediatamente depois dessa linha. O seu método em conformidade do `inline_data` deve remeter num teor destas bases:

```
def inline_data(file, code = 'M', mode = '644')
  content = File.read(file)
  puts "#{code} #{mode} inline #{file}"
  export_data(content)
end
```

Você reutiliza o método `export_data` definido anteriormente, pois o formato é o mesmo usado para especificar os dados da mensagem de commit.

Por último, você precisa retornar a marca atual para que ela possa ser passada à próxima iteração:

```
return mark
```



Se estiver usando o Windows, você precisará acrescentar uma etapa. Como mencionado anteriormente, o Windows usa CRLF para quebras de linha, enquanto o `git fast-import` aceita somente LF. Para contornar esse problema e fazer o `git`

`fast-import` funcionar, você precisa instruir o Ruby a usar LF em vez de CRLF:

```
$stdout.binmode
```

Isto é tudo. Este é o script completo:

```
#!/usr/bin/env ruby

$stdout.binmode
$author = "John Doe <john@example.com>"

$marks = []
def convert_dir_to_mark(dir)
    if !$marks.include?(dir)
        $marks << dir
    end
    ($marks.index(dir)+1).to_s
end

def convert_dir_to_date(dir)
    if dir == 'current'
        return Time.now().to_i
    else
        dir = dir.gsub('back_', '')
        (year, month, day) = dir.split('_')
        return Time.local(year, month, day).to_i
    end
end

def export_data(string)
    print "data #{string.size}\n#{string}"
end

def inline_data(file, code='M', mode='644')
    content = File.read(file)
    puts "#{code} #{mode} inline #{file}"
    export_data(content)
end

def print_export(dir, last_mark)
    date = convert_dir_to_date(dir)
    mark = convert_dir_to_mark(dir)

    puts 'commit refs/heads/master'
    puts "mark :#{mark}"
    puts "committer #{$author} #{date} -0700"
    export_data("imported from #{dir}")
    puts "from :#{last_mark}" if last_mark
```

```
    puts 'deleteall'
    Dir.glob("**/*").each do |file|
        next if !File.file?(file)
        inline_data(file)
    end
    mark
end

# Loop through the directories
last_mark = nil
Dir.chdir(ARGV[0]) do
    Dir.glob("*").each do |dir|
        next if File.file?(dir)

        # move into the target directory
        Dir.chdir(dir) do
            last_mark = print_export(dir, last_mark)
        end
    end
end
```

Se você executar esse script, obterá um conteúdo semelhante a este:

```
$ ruby import.rb /opt/import_from
commit refs/heads/master
mark :1
committer John Doe <john@example.com> 1388649600 -0700
data 29
imported from back_2014_01_02deleteall
M 644 inline README.md
data 28
# Hello

This is my readme.
commit refs/heads/master
mark :2
committer John Doe <john@example.com> 1388822400 -0700
data 29
imported from back_2014_01_04from :1
deleteall
M 644 inline main.rb
data 34
#!/bin/env ruby

puts "Hey there"
M 644 inline README.md
(...)
```

Para executar o importador, estando no diretório Git para o qual você deseja importar, encaminhe

essa saída para o `git fast-import` por meio de um pipe. Como ponto de partida, você pode criar um diretório, executar `git init` nele e depois executar seu script:

```
$ git init
Initialized empty Git repository in /opt/import_to/.git/
$ ruby import.rb /opt/import_from | git fast-import
git-fast-import statistics:
---------------------------------------------------------------------
Alloc'd objects:       5000
Total objects:           13 (         6 duplicates                  )
      blobs  :            5 (         4 duplicates          3 deltas of          5
attempts)
      trees  :            4 (         1 duplicates          0 deltas of          4
attempts)
      commits:            4 (         1 duplicates          0 deltas of          0
attempts)
      tags   :            0 (         0 duplicates          0 deltas of          0
attempts)
Total branches:           1 (         1 loads     )
      marks:           1024 (         5 unique    )
      atoms:              2
Memory total:          2344 KiB
       pools:          2110 KiB
     objects:           234 KiB
---------------------------------------------------------------------
pack_report: getpagesize()            =       4096
pack_report: core.packedGitWindowSize = 1073741824
pack_report: core.packedGitLimit      = 8589934592
pack_report: pack_used_ctr            =         10
pack_report: pack_mmap_calls          =          5
pack_report: pack_open_windows        =          2 /          2
pack_report: pack_mapped              =       1457 /       1457
---------------------------------------------------------------------
```

Como você pode ver, quando a execução é concluída com sucesso, o programa apresenta várias estatísticas sobre o que realizou. Nesse caso, você importou um total de 13 objetos de 4 commits para 1 branch. Agora, execute `git log` para ver seu novo histórico:

```
$ git log -2
commit 3caa046d4aac682a55867132ccdfbe0d3fdee498
Author: John Doe <john@example.com>
Date:   Tue Jul 29 19:39:04 2014 -0700

    imported from current

commit 4afc2b945d0d3c8cd00556fbe2e8224569dc9def
Author: John Doe <john@example.com>
Date:   Mon Feb 3 01:00:00 2014 -0700
```

```
    imported from back_2014_02_03
```

Pronto: um repositório Git organizado e limpo. É importante observar que nenhum checkout foi feito – inicialmente, não há nenhum arquivo no seu diretório de trabalho. De se obtê-las, a obrigação será de fato às restaurações da forma contida no branch da onde a `master` se situa neste respectivo momento:

```
$ ls
$ git reset --hard master
HEAD is now at 3caa046 imported from current
$ ls
README.md main.rb
```

Você pode fazer muito mais com a ferramenta `fast-import`: lidar com modos diferentes, dados binários, várias branches e merges, tags, indicadores de progresso e muito mais. Há diversos exemplos de cenários mais complexos no diretório `contrib/fast-import` do código-fonte do Git.

Resumo

Você deve se sentir confortável usando o Git como cliente para outros sistemas de controle de versão, ou importando quase qualquer repositório existente para o Git sem perder dados. No próximo capítulo, abordaremos os detalhes brutos do Git para que você possa criar cada byte, se necessário.

Git Internals (Por Dentro do Git)

Você pode ter pulado para este capítulo a partir de um capítulo anterior ou pode ter chegado aqui depois de ler todo o livro de forma sequencial até este ponto - em ambos os casos, é aqui onde abordaremos o funcionamento interno e a implementação do Git. Descobrimos que a compreensão dessa informação foi de fundamental importância para apreciar a utilidade e o poder do Git, mas outras pessoas nos argumentaram que ela pode ser confusa e desnecessariamente complexa para iniciantes. Portanto, fizemos desta discussão o último capítulo do livro, para que você possa lê-la mais cedo ou mais tarde, dependendo do seu processo de aprendizagem. Deixamos a seu critério essa decisão.

Agora que você chegou aqui, vamos começar. Primeiro, se ainda não estiver claro, o Git é, fundamentalmente, um sistema de arquivos endereçável por conteúdo (content-addressable filesystem) com uma interface de usuário VCS (sistema de controle de versão) baseada nela. Você aprenderá mais sobre o que isso significa em breve.

No começo do Git (principalmente antes da versão 1.5), a interface do usuário era muito mais complexa, pois enfatizava esse sistema de arquivos em vez de um VCS polido. Nos últimos anos, a interface do usuário (UI) foi refinada até ser tão limpa e fácil de usar quanto qualquer sistema disponível no mercado; contudo, ainda perdura o estereótipo sobre a interface inicial do Git que, naquela época, era complexa e difícil de aprender.

A camada do sistema de arquivos endereçável por conteúdo é incrivelmente interessante, portanto nós a abordaremos primeiro neste capítulo; então, você aprenderá sobre os mecanismos de transporte e as tarefas de manutenção de repositório com as quais possivelmente você terá de lidar com elas.

Encanamento e Porcelana (Plumbing and Porcelain)

Este livro aborda principalmente como usar o Git com cerca de 30 subcomandos, como `checkout`, `branch`, `remote` e assim por diante. No entanto, como o Git foi inicialmente um kit de ferramentas para um sistema de controle de versão, e não um VCS completo e fácil de usar, ele possui vários subcomandos que realizam operações de baixo nível e foram projetados para ser encadeados no estilo UNIX ou chamados a partir de scripts. Esses comandos são geralmente chamados de comandos de “plumbing” do Git, enquanto os comandos mais amigáveis ao usuário são chamados de comandos de “porcelain”.

Como você já deve ter percebido, os nove primeiros capítulos deste livro tratam quase exclusivamente dos comandos de porcelana. Neste capítulo, porém, você lidará principalmente com os comandos de encanamento de nível mais baixo, pois eles dão acesso ao funcionamento interno do Git e ajudam a demonstrar como e por que o Git faz o que faz. Muitos desses comandos não foram feitos para uso manual na linha de comando, mas para servir como blocos de construção de novas ferramentas e scripts personalizados.

Quando você executa `git init` em um diretório novo ou existente, o Git cria o diretório `.git`, onde fica quase tudo o que ele armazena e manipula. Se quiser fazer backup ou clone do repositório, copiar esse único diretório para outro local fornecerá praticamente tudo de que você precisa. Basicamente, todo este capítulo trata do que você pode ver nesse diretório. Eis a aparência típica de

um diretório `.git` recém-inicializado:

```
$ ls -F1
config
description
HEAD
hooks/
info/
objects/
refs/
```

Dependendo da sua versão do Git, talvez você veja algum conteúdo adicional ali, mas este é um repositório recém-criado com `git init` — é o que aparece por padrão. O arquivo `description` é usado apenas pelo programa GitWeb, portanto não se preocupe com ele. O arquivo `config` contém as opções de configuração específicas do projeto, e o diretório `info` mantém um arquivo de exclusão global para os padrões ignorados que você não quer registrar em um arquivo `.gitignore`. O diretório `hooks` contém os scripts de hook do lado do cliente ou do servidor, discutidos em detalhes em Hooks do Git.

Restam quatro entradas importantes: os arquivos `HEAD` e `index` (este ainda não foi criado) e os diretórios `objects` e `refs`. Essas são as partes centrais do Git. O diretório `objects` armazena todo o conteúdo do banco de dados; o diretório `refs` armazena ponteiros para objetos de commit nesses dados (branches, tags, remotes e outros); o arquivo `HEAD` aponta para o branch no qual você fez checkout; e o arquivo `index` é onde o Git armazena as informações da área de preparação. Agora examinaremos cada uma dessas seções em detalhes para entender como o Git funciona.

Objetos do Git

O Git é um sistema a caráter num arquivo endereçável por conteúdos da sua base. Ótimo. O que isto significa afinal? Quer dizer o que em decorrência da forma central do Git que lhe baseia consiste nas instâncias num simples valor chave pelas armazenagens aos dados. Isso significa que você pode inserir qualquer conteúdo em um repositório Git, e o Git devolverá uma chave exclusiva para recuperá-lo mais tarde.

Para demonstrar, examinemos `git hash-object`, que recebe dados, armazena-os em `.git/objects` (o *banco de dados de objetos*) e devolve a chave que referencia esse objeto.

Primeiro, inicialize um repositório Git e verifique se não há nada em `objects`:

```
$ git init test
Initialized empty Git repository in /tmp/test/.git/
$ cd test
$ find .git/objects
.git/objects
.git/objects/info
.git/objects/pack
$ find .git/objects -type f
```

O Git inicializou o diretório `objects` diretamente e criou os subdiretórios `pack` e `info` dentro dele, mas não há nenhum arquivo regular. Agora, use `git hash-object` para criar e armazenar um novo objeto:

```
$ echo 'test content' | git hash-object -w --stdin
d670460b4b4aece5915caf5c68d12f560a9fe3e4
```

Em sua forma mais simples, `git hash-object` recebe o conteúdo e retorna a chave que *seria* usada para armazená-lo. A opção `-w` instrui o comando a também gravar o objeto no banco de dados. `--stdin` instrui `git hash-object` a obter o conteúdo da entrada padrão; sem ela, o comando esperaria um nome de arquivo.

A saída é um hash de checksum com 40 caracteres. Esse é o hash SHA-1 – um checksum do conteúdo e de um cabeçalho, sobre o qual você aprenderá em breve. Agora você pode ver como o Git armazenou seus dados:

```
$ find .git/objects -type f
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4
```

Ao examinar novamente `objects`, você verá um arquivo para o novo conteúdo. O Git armazena inicialmente cada conteúdo em um arquivo nomeado com o checksum SHA-1 do conteúdo e de seu cabeçalho. O subdiretório é nomeado com os dois primeiros caracteres do SHA-1, e o nome do arquivo são os 38 caracteres restantes.

Você pode examinar o banco de dados de objetos com `git cat-file`. Esse comando é um canivete suíço para a inspeção de objetos do Git. A opção `-p` faz `cat-file` descobrir o tipo do conteúdo e exibi-lo adequadamente:

```
$ git cat-file -p d670460b4b4aece5915caf5c68d12f560a9fe3e4
test content
```

Agora, você pode adicionar conteúdo para o Git e recuperá-lo de volta. Você também pode fazer isso para conteúdos em arquivos. Por exemplo, você pode fazer um controle de versão simples em um arquivo. Primeiro, crie um arquivo e salve seus conteúdos em seu banco de dados:

```
$ echo 'version 1' > test.txt
$ git hash-object -w test.txt
83baae61804e65cc73a7201a7252750c76066a30
```

Depois, escreva alguns conteúdos novos nesse arquivo, e salve-o novamente:

```
$ echo 'version 2' > test.txt
$ git hash-object -w test.txt
1f7a7a472abf3dd9643fd615f6da379c4acb3e3a
```

O banco de dados agora contém as duas versões desse arquivo e o primeiro conteúdo:

```
$ find .git/objects -type f
.git/objects/1f/7a7a472abf3dd9643fd615f6da379c4acb3e3a
.git/objects/83/baae61804e65cc73a7201a7252750c76066a30
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4
```

Agora exclua a cópia local de `test.txt` e recupere a primeira versão do banco de dados:

```
$ git cat-file -p 83baae61804e65cc73a7201a7252750c76066a30 > test.txt
$ cat test.txt
version 1
```

ou à segunda versão:

```
$ git cat-file -p 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a > test.txt
$ cat test.txt
version 2
```

Memorizar o SHA-1 de cada versão não é prático; além disso, você armazena apenas o conteúdo, não o nome do arquivo. Esse tipo de objeto é chamado de *blob*. Dado o SHA-1 de um objeto, obtenha seu tipo com `git cat-file -t`:

```
$ git cat-file -t 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a
blob
```

Objetos Tree

O próximo tipo é a *tree*, que armazena nomes e permite reunir arquivos. O Git armazena o conteúdo em uma maneira similar a um sistema de arquivos UNIX, porém um pouco simplificado. Todo o conteúdo é armazenado como objetos *tree* e *blob*, com as *trees* correspondendo a entradas de um diretório UNIX e *blobs* correspondendo mais ou menos a *inodes* ou conteúdos de arquivos. Uma tree contém entradas com o SHA-1 de um blob ou subtree, seu modo, tipo e nome. Suponha que a tree mais recente seja semelhante a esta:

```
$ git cat-file -p master^{tree}
100644 blob a906cb2a4a904a152e80877d4088654daad0c859      README
100644 blob 8f94139338f9404f26296befa88755fc2598c289      Rakefile
040000 tree 99f1a6d12cb4b6f19c8655fca46c3ecf317074e0      lib
```

A sintaxe `master^{tree}` especifica o objeto *tree* que é apontado pelo último *commit* em sua *branch* `master` . Note que o subdiretório `lib` não é um *blob*, mas uma referência para outra *tree*:

```
$ git cat-file -p 99f1a6d12cb4b6f19c8655fca46c3ecf317074e0
```

```
100644 blob 47c6340d6459e05787f644c2447d2595f5d3a54b      simplegit.rb
```



Dependendo do shell, `master^{tree}` pode causar erros.

No CMD, duplique o caractere de escape `^`: `git cat-file -p master^^{tree}`. No PowerShell, coloque parâmetros com `{}` entre aspas: `git cat-file -p 'master^{tree}'`.

No ZSH, coloque a expressão entre aspas: `git cat-file -p "master^{tree}"`.

Conceitualmente, os dados armazenados se parecem com isto:

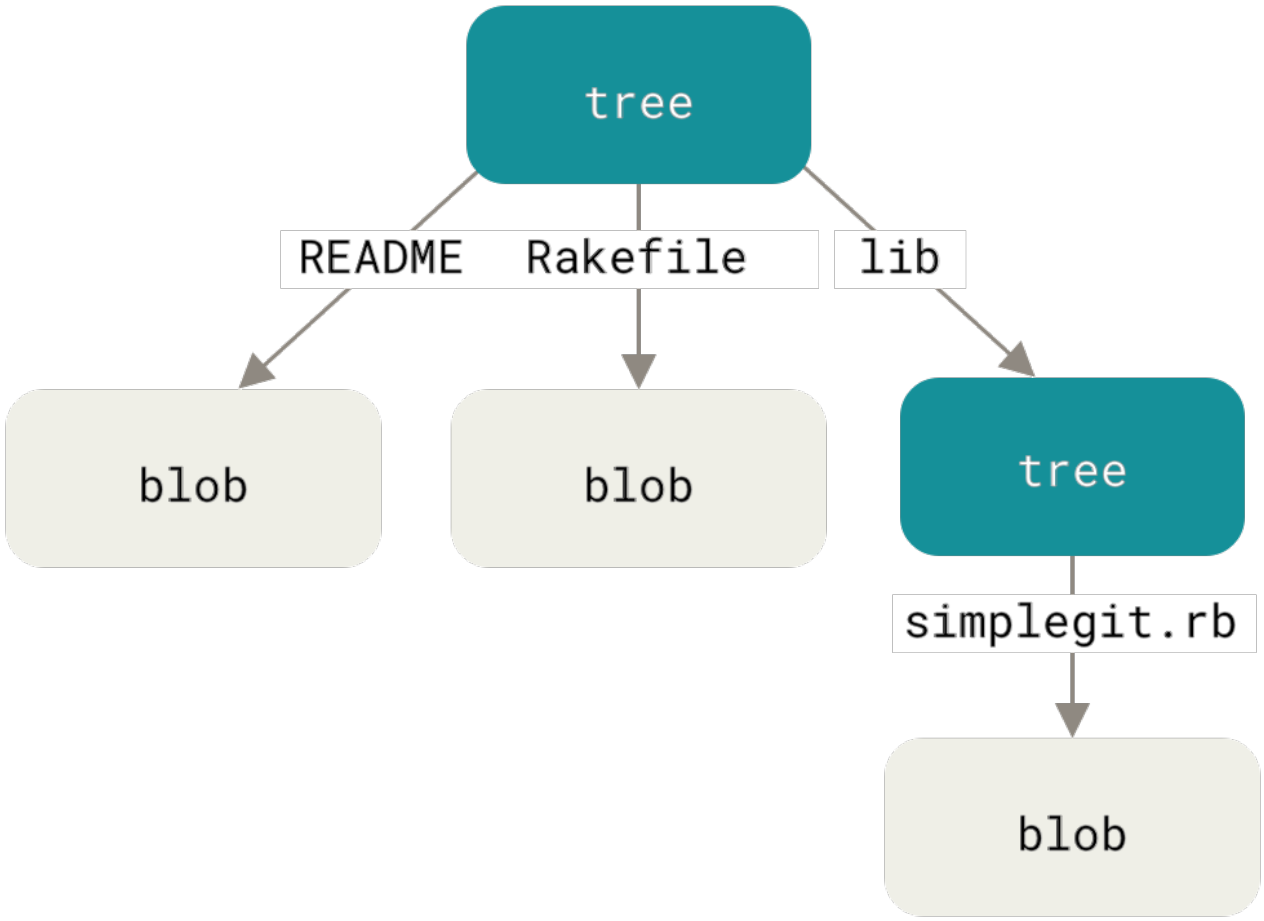


Figure 173. Simple version of the Git data model

Você pode criar facilmente a sua própria *tree*. O Git normalmente cria uma *tree* a partir do estado da sua área de *stage* ou *index* e escrevendo uma série de objetos *tree* a partir dela. Então, para criar um objeto *tree*, você primeiro precisa popular um *index* adicionando alguns arquivos. Para criar um index com a primeira versão de `test.txt`, use `git update-index`. Você usa esse comando para adicionar artificialmente a versão anterior do arquivo `test.txt` à nova área de *stage*. Você precisa passar a ele a opção `--add` porque o arquivo ainda não existe em sua área de *stage* (você nem precisa ter uma área de *stage* ainda) e a opção `--cacheinfo` porque o arquivo que você está adicionando não está em seu diretório mas está no seu banco de dados. Depois você especifica o modo, o SHA-1 e o nome do arquivo:

```
$ git update-index --add --cacheinfo 100644 \
  83baae61804e65cc73a7201a7252750c76066a30 test.txt
```

Neste caso, você está especificando um modo `100644`, o que significa se trata de um arquivo normal. Outras opções são `100755`, o que significa que é um arquivo executável; e `120000`, que especifica um link simbólico. O modo vem dos modos UNIX, mas esses três são os únicos válidos para arquivos (blobs) no Git; outros são usados para diretórios e submódulos.

Use `git write-tree` para gravar a área de stage em uma tree. `-w` não é necessário – o comando cria uma tree a partir do index caso ela ainda não exista:

```
$ git write-tree
d8329fc1cc938780ffdd9f94e0d364e0ea74f579
$ git cat-file -p d8329fc1cc938780ffdd9f94e0d364e0ea74f579
100644 blob 83baae61804e65cc73a7201a7252750c76066a30      test.txt
```

Confirme que esse é um objeto tree usando `git cat-file`:

```
$ git cat-file -t d8329fc1cc938780ffdd9f94e0d364e0ea74f579
tree
```

Você pode criar um novo arquivo *tree* com a segunda versão de `test.txt`, além de um novo arquivo:

```
$ echo 'new file' > new.txt
$ git update-index --cacheinfo 100644 \
  1f7a7a472abf3dd9643fd615f6da379c4acb3e3a test.txt
$ git update-index --add new.txt
```

Sua área de *stage* agora tem a nova versão de `test.txt`, bem como o novo arquivo `new.txt`. Escreva essa *tree* (grave o estado da área de *stage* ou *index* para um objeto) e veja como ela se parece:

```
$ git write-tree
0155eb4229851634a0f03eb265b69f5a2d56f341
$ git cat-file -p 0155eb4229851634a0f03eb265b69f5a2d56f341
100644 blob fa49b077972391ad58037050f2a75f74e3671e92      new.txt
100644 blob 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a      test.txt
```

Essa tree contém duas entradas, e o SHA-1 de `test.txt` é o da "versão 2" (`1f7a7a`). Apenas por diversão, adicione a primeira *tree* como um subdiretório neste aqui. Leia trees para a área de stage com `git read-tree`. Use `--prefix` para ler uma tree como subtree:

```
$ git read-tree --prefix=bak d8329fc1cc938780ffdd9f94e0d364e0ea74f579
$ git write-tree
3c4e9cd789d88d8d89c1073707c3585e41b0e614
```

```
$ git cat-file -p 3c4e9cd789d88d8d89c1073707c3585e41b0e614
040000 tree d8329fc1cc938780ffdd9f94e0d364e0ea74f579      bak
100644 blob fa49b077972391ad58037050f2a75f74e3671e92      new.txt
100644 blob 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a      test.txt
```

Se você criasse um diretório de trabalho a partir da nova *tree* que você criou, você teria os dois arquivos no nível mais alto do diretório de trabalho e um subdiretório chamado `bak` que conteria a primeira versão do arquivo `test.txt`. Você pode pensar nos dados que o Git armazena para essas estruturas como sendo algo assim:

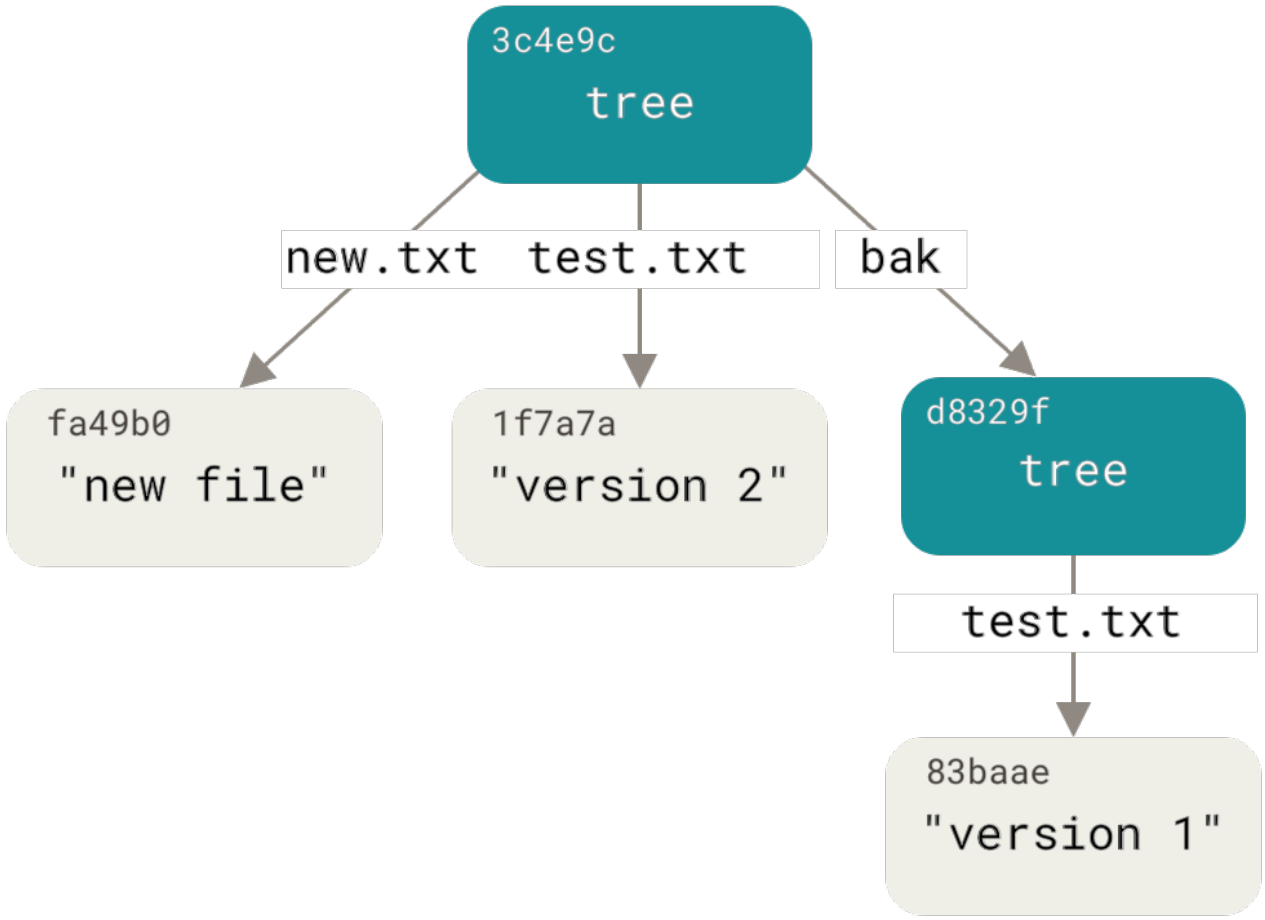


Figure 174. A estrutura de conteúdo dos dados atuais do Git

Objetos de Commit (Commit Objects)

Agora há três trees para os snapshots, mas é preciso lembrar seus três SHA-1 para recuperá-los. Você também não tem nenhuma informação sobre quem salvou os snapshots, quando eles foram salvos, ou porque eles foram salvos. Essas são informações básicas que o objeto *commit* armazena para você.

Para criar um objeto *commit*, você chama `commit-tree` e especifica o SHA-1 de uma única *tree* e quais objetos *commit* precedem diretamente ele, se houver. Começando com a primeira *tree* que você escreveu:

```
$ echo 'First commit' | git commit-tree d8329f
```

```
fdf4fc3344e67ab068f836878b6c4951e3b15f3d
```



Você irá obter um valor diferente para o *hash* por causa das diferentes hora de criação e dados do autor. Embora um commit possa ser reproduzido a partir desses dados, detalhes históricos do livro fazem os hashes impressos talvez não corresponderem aos commits apresentados. Substitua os *hashes* de *commit* e *tag* pelos seus próprios *checksums* posteriormente neste capítulo.

Examine o novo objeto commit com `git cat-file`:

```
$ git cat-file -p fdf4fc3
tree d8329fc1cc938780ffdd9f94e0d364e0ea74f579
author Scott Chacon <schacon@gmail.com> 1243040974 -0700
committer Scott Chacon <schacon@gmail.com> 1243040974 -0700

First commit
```

Um commit especifica a tree do snapshot; os pais; autor e committer; uma linha em branco; e a mensagem.

A seguir, você irá escrever outros dois objetos *commit*, cada um referenciando o *commit* que veio diretamente antes dele:

```
$ echo 'Second commit' | git commit-tree 0155eb -p fdf4fc3
cac0cab538b970a37ea1e769cbbde608743bc96d
$ echo 'Third commit'  | git commit-tree 3c4e9c -p cac0cab
1a410efbd13591db07496601ebc7a059dd55cfe9
```

Cada um dos três objetos *commit* aponta para uma das três *trees* de *snapshot* que você criou. Curiosamente, você tem agora um histórico do Git real que você pode ver com o comando `git log`, se você executá-lo no SHA-1 do último *commit*:

```
$ git log --stat 1a410e
commit 1a410efbd13591db07496601ebc7a059dd55cfe9
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri May 22 18:15:24 2009 -0700

    Third commit

 bak/test.txt | 1 +
 1 file changed, 1 insertion(+)

commit cac0cab538b970a37ea1e769cbbde608743bc96d
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri May 22 18:14:29 2009 -0700

    Second commit
```

```
 new.txt  | 1 +
 test.txt | 2 +-
 2 files changed, 2 insertions(+), 1 deletion(-)

commit fdf4fc3344e67ab068f836878b6c4951e3b15f3d
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri May 22 18:09:34 2009 -0700

    First commit

 test.txt | 1 +
 1 file changed, 1 insertion(+)
```

Incrível. Você acabou de fazer operações de baixo nível para criar um histórico do Git sem usar nenhum dos comandos de *front-end*. É isso que `git add` e `git commit` fazem: armazenam blobs, atualizam o index e gravam trees e commits que referenciam seus antecessores. Blob, tree e commit são armazenados inicialmente como arquivos separados em `.git/objects`. Estes são todos os objetos no diretório de exemplo, comentados com o que eles armazenam:

```
$ find .git/objects -type f
.git/objects/01/55eb4229851634a0f03eb265b69f5a2d56f341 # tree 2
.git/objects/1a/410efbd13591db07496601ebc7a059dd55cfe9 # commit 3
.git/objects/1f/7a7a472abf3dd9643fd615f6da379c4acb3e3a # test.txt v2
.git/objects/3c/4e9cd789d88d8d89c1073707c3585e41b0e614 # tree 3
.git/objects/83/baae61804e65cc73a7201a7252750c76066a30 # test.txt v1
.git/objects/ca/c0cab538b970a37ea1e769cbbde608743bc96d # commit 2
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4 # 'test content'
.git/objects/d8/329fc1cc938780ffdd9f94e0d364e0ea74f579 # tree 1
.git/objects/fa/49b077972391ad58037050f2a75f74e3671e92 # new.txt
.git/objects/fd/f4fc3344e67ab068f836878b6c4951e3b15f3d # commit 1
```

Se você seguir as referências internas, você obterá um grafo de objetos mais ou menos como este:

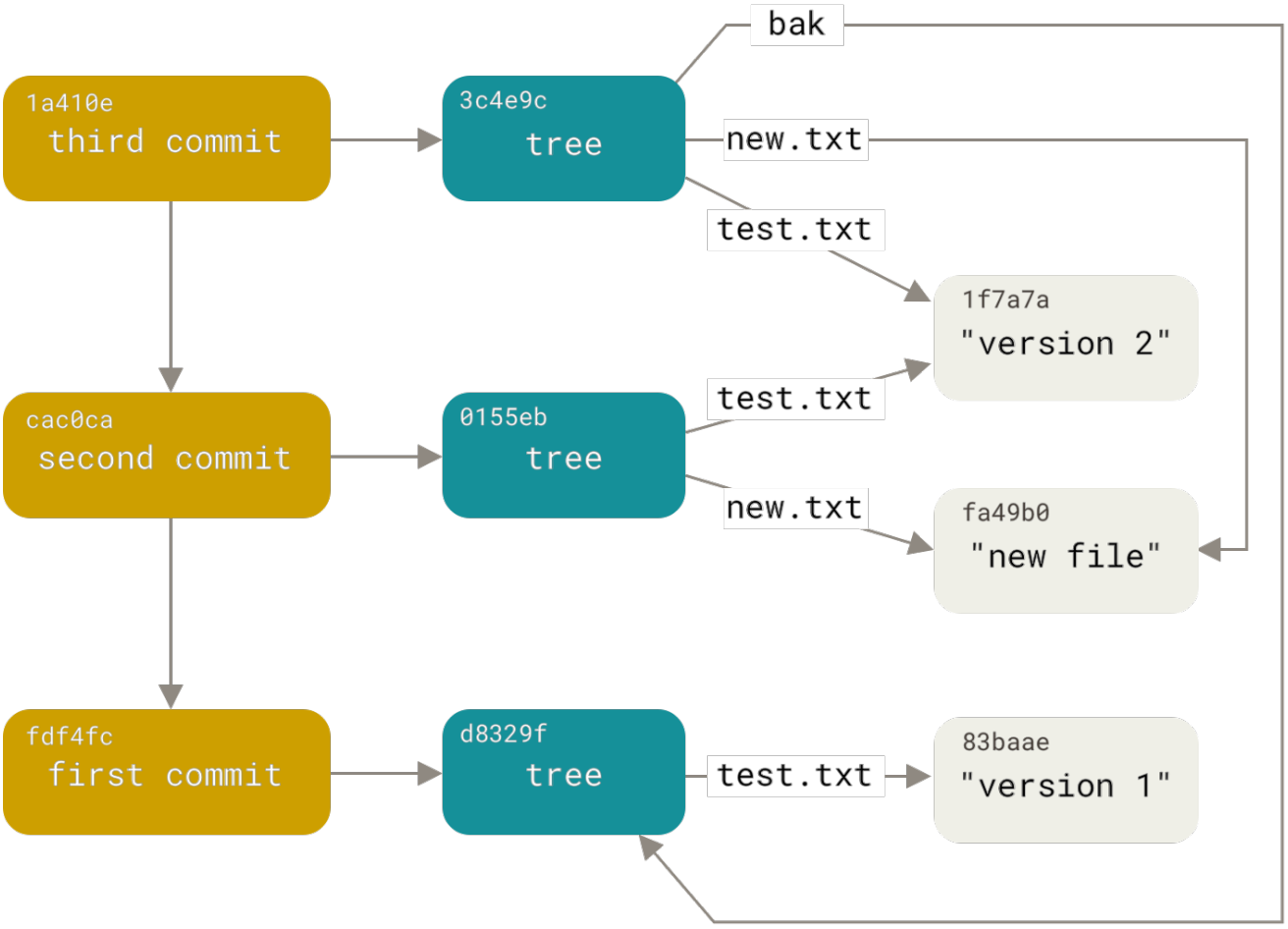


Figure 175. Todos os objetos alcançáveis no seu diretório Git

Armazenamento de Objetos (Object Storage)

Um cabeçalho é armazenado com cada objeto enviado ao banco de dados. Vamos examinar como o Git armazena objetos. Você armazenará interativamente em Ruby um blob com a string “what is up, doc?”.

Você pode iniciar o modo interativo do Ruby com o comando `irb`:

```
$ irb
>> content = "what is up, doc?"
=> "what is up, doc?"
```

Primeiro, o Git constrói um cabeçalho que identifica o tipo – nesse caso, blob. Ele acrescenta um espaço, o tamanho em bytes e um byte nulo:

```
>> header = "blob #{content.bytesize}\0"
=> "blob 16\u0000"
```

O Git concatena o cabeçalho e o conteúdo original e então calcula o *checksum* SHA-1 do novo conteúdo. Você pode calcular o valor SHA-1 de uma *string* em Ruby incluindo a biblioteca SHA1 digest com o comando `require` e então chamando `Digest::SHA1.hexdigest()` com a *string*:

```
>> store = header + content
=> "blob 16\u0000what is up, doc?"
>> require 'digest/sha1'
=> true
>> sha1 = Digest::SHA1.hexdigest(store)
=> "bd9dbf5aae1a3862dd1526723246b20206e5fc37"
```

Compare com a saída de `git hash-object`. Usamos `echo -n` para não acrescentar uma quebra de linha.

```
$ echo -n "what is up, doc?" | git hash-object --stdin
bd9dbf5aae1a3862dd1526723246b20206e5fc37
```

O Git comprime o novo conteúdo com zlib, o que você pode fazer em Ruby com a biblioteca zlib. Primeiro, você precisa incluir a biblioteca e então executar `Zlib::Deflate.deflate()` no conteúdo:

```
>> require 'zlib'
=> true
>> zlib_content = Zlib::Deflate.deflate(store)
=> "x\x9CK\xCA\xC9OR04c(\xCFH,Q\xC8,V(-\xD0QH\xC9O\xB6\a\x00_\x1C\a\x9D"
```

Por último, você irá salvar o seu conteúdo comprimido com zlib em um objeto no disco. Você irá determinar o caminho do objeto que você quer escrever (sendo os dois primeiros caracteres do valor do SHA-1 o nome do subdiretório, e os últimos 38 caracteres sendo o nome do arquivo dentro desse diretório). ==== Objetos de Commit (Commit Objects) Em Ruby, você pode usar a função `FileUtils.mkdir_p()` para criar o subdiretório se ele não existir. Depois, abra o arquivo com `File.open()` e escreva o conteúdo previamente comprimido com zlib no arquivo chamando `write()` no *file handle* resultante:

```
>> path = '.git/objects/' + sha1[0,2] + '/' + sha1[2,38]
=> ".git/objects/bd/9dbf5aae1a3862dd1526723246b20206e5fc37"
>> require 'fileutils'
=> true
>> FileUtils.mkdir_p(File.dirname(path))
=> ".git/objects/bd"
>> File.open(path, 'w') { |f| f.write zlib_content }
=> 32
```

Verifique o conteúdo com `git cat-file`:

```
---
$ git cat-file -p bd9dbf5aae1a3862dd1526723246b20206e5fc37
what is up, doc?
---
```

É isso - você criou um objeto *blob* do Git válido.

Todos os objetos são armazenados do mesmo jeito, apenas com diferentes tipos - em vez da *string blob*, o cabeçalho começará com *commit* ou *tree*. Além disso, embora o conteúdo de um blob possa ser quase qualquer coisa, os conteúdos de um commit ou tree estão formatados de uma forma muito específica.

Referências do Git

Se você estivesse interessado em ver o histórico do seu repositório a partir de um commit, por exemplo, `1a410e`, você poderia executar algo como `git log 1a410e` para exibir esse histórico, mas você ainda precisaria se lembrar de que `1a410e` é o commit que você quer usar como o ponto de partida para esse histórico. Em vez disso, seria mais fácil se você tivesse um arquivo onde pudesse armazenar esse valor SHA-1 sob um nome simples, de forma que pudesse usar esse nome simples em vez do valor SHA-1 bruto.

No Git, esses nomes simples são chamados de “referências” (references) ou “refs”; você pode encontrar os arquivos que contêm esses valores SHA-1 no diretório `.git/refs`. No projeto atual, esse diretório não contém arquivos, mas contém uma estrutura simples:

```
$ find .git/refs
.git/refs
.git/refs/heads
.git/refs/tags
$ find .git/refs -type f
```

Para criar uma nova referência que ajudará você a lembrar onde está seu último commit, você pode tecnicamente fazer algo tão simples quanto isto:

```
$ echo 1a410efbd13591db07496601ebc7a059dd55cfe9 > .git/refs/heads/master
```

Agora, você pode usar a referência head que você acabou de criar em vez do valor SHA-1 em seus comandos do Git:

```
$ git log --pretty=oneline master
1a410efbd13591db07496601ebc7a059dd55cfe9 Third commit
cac0cab538b970a37ea1e769cbbde608743bc96d Second commit
fdf4fc3344e67ab068f836878b6c4951e3b15f3d First commit
```

Não é recomendável editar diretamente os arquivos de referência; em vez disso, o Git fornece o comando mais seguro `git update-ref` para fazer isso se você quiser atualizar uma referência:

```
$ git update-ref refs/heads/master 1a410efbd13591db07496601ebc7a059dd55cfe9
```

Isso é basicamente o que um branch no Git é: um simples ponteiro ou referência para a head de

uma linha de trabalho. Para criar um branch de volta no segundo commit, você pode fazer isto:

```
$ git update-ref refs/heads/test cac0ca
```

O seu branch conterá apenas o trabalho a partir desse commit para baixo:

```
$ git log --pretty=oneline test
cac0cab538b970a37ea1e769cbbde608743bc96d Second commit
fdf4fc3344e67ab068f836878b6c4951e3b15f3d First commit
```

Agora, o seu banco de dados do Git conceitualmente se parece com algo assim:

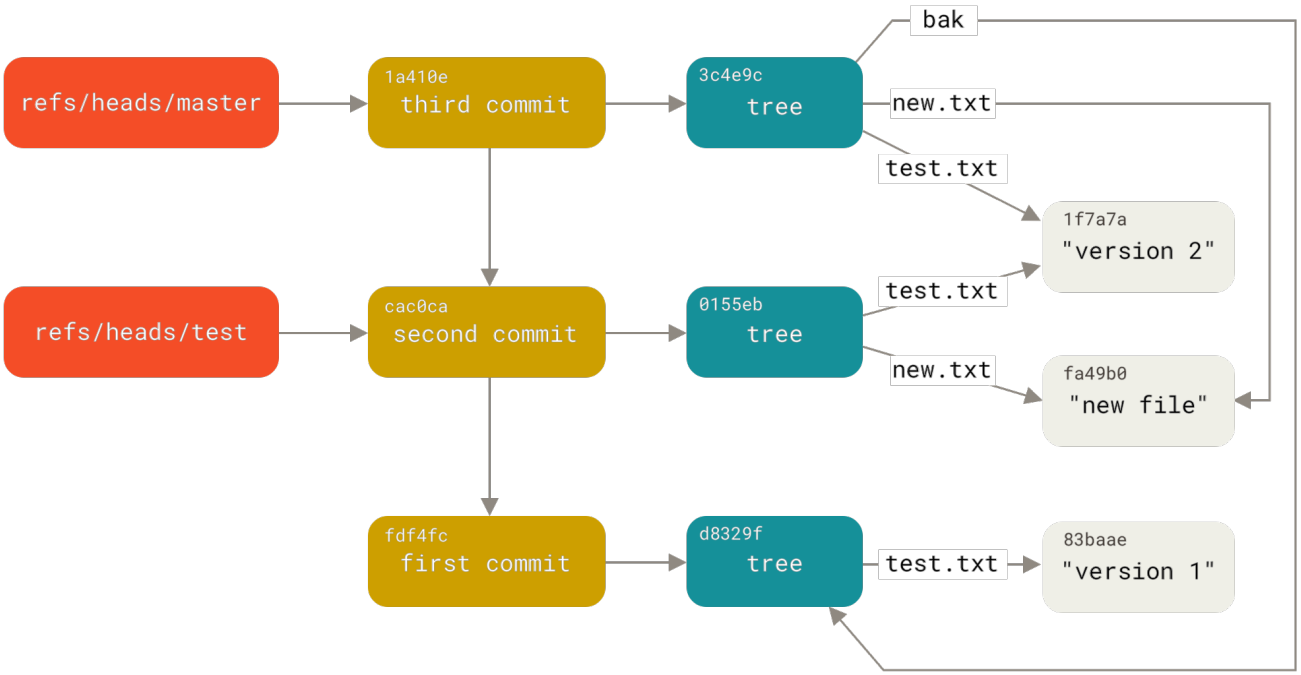


Figure 176. Objetos do diretório do Git com referências à head de branch incluídas

Quando você executa comandos como `git branch <branch>`, o Git basicamente executa o comando `update-ref` para adicionar o SHA-1 do último commit do branch em que você está, para qualquer nova referência que você deseje criar.

A HEAD

A pergunta agora é: quando você executa o comando `git branch <branch>`, como o Git fica sabendo do SHA-1 do último commit? A resposta é o arquivo HEAD.

Normalmente, o arquivo HEAD é uma referência simbólica ao branch em que você está no momento. Por referência simbólica, queremos dizer que, diferente de uma referência normal, ela contém um ponteiro para outra referência.

Entretanto, em alguns casos raros, o arquivo HEAD pode conter o valor SHA-1 de um objeto do Git. Isso ocorre quando você faz o checkout de uma tag, commit ou branch remoto, o que coloca seu repositório no estado "detached HEAD" (HEAD desanexada).

Se você olhar para o arquivo, normalmente você verá algo parecido com isto:

```
$ cat .git/HEAD
ref: refs/heads/master
```

Se você executar `git checkout test`, o Git atualizará o arquivo para que ele se pareça com isto:

```
$ cat .git/HEAD
ref: refs/heads/test
```

Quando você executa `git commit`, ele cria o objeto commit, especificando o pai desse objeto de commit para ser qualquer valor SHA-1 que a referência em HEAD aponte.

Você também pode editar esse arquivo manualmente, mas, mais uma vez, existe um comando mais seguro para se fazer isso: `git symbolic-ref`. Você pode ler o valor do seu HEAD através deste comando:

```
$ git symbolic-ref HEAD
refs/heads/master
```

Você também pode definir o valor da HEAD usando o mesmo comando:

```
$ git symbolic-ref HEAD refs/heads/test
$ cat .git/HEAD
ref: refs/heads/test
```

Você não pode definir uma referência simbólica de fora do estilo das referências (refs style):

```
$ git symbolic-ref HEAD test
fatal: Refusing to point HEAD outside of refs/
```

Tags

Acabamos de discutir os três principais tipos de objetos do Git (*blobs*, *trees* e *commits*), mas existe um quarto. O objeto *tag* é muito parecido com um objeto de commit — ele contém um tagger, uma data, uma mensagem e um ponteiro. A diferença principal é que um objeto tag normalmente aponta para um commit em vez de uma tree (árvore). Ele se parece como uma referência a um branch, mas nunca se move — ele sempre aponta para o mesmo commit, mas atribuindo um nome mais amigável a ele.

Conforme discutimos no Fundamentos do Git, existem dois tipos de tags: anotadas (annotated) e leves (lightweight). Você pode criar uma tag leve executando algo como isto:

```
$ git update-ref refs/tags/v1.0 cac0cab538b970a37ea1e769cbbde608743bc96d
```

Isto é tudo o que uma tag leve se consiste — uma referência que nunca se move. No entanto, uma tag anotada é mais complexa. Se você criar uma tag anotada, o Git criará um objeto tag e depois escreverá uma referência que aponta para ela em vez de diretamente para o commit. Você pode ver isso ao se criar uma tag anotada (usando a opção `-a`):

```
$ git tag -a v1.1 1a410efbd13591db07496601ebc7a059dd55cfe9 -m 'Test tag'
```

Aqui está o valor SHA-1 do objeto que ele criou:

```
$ cat .git/refs/tags/v1.1
9585191f37f7b0fb9444f35a9bf50de191beadc2
```

Agora, rode o comando `git cat-file -p` naquele valor do SHA-1:

```
$ git cat-file -p 9585191f37f7b0fb9444f35a9bf50de191beadc2
object 1a410efbd13591db07496601ebc7a059dd55cfe9
type commit
tag v1.1
tagger Scott Chacon <schacon@gmail.com> Sat May 23 16:48:58 2009 -0700

Test tag
```

Observe que a entrada do objeto aponta para o valor SHA-1 do commit que você rotulou (tagged). Observe também que ele não precisa apontar a um commit; você pode rotular qualquer tipo de objeto no Git. No código-fonte do Git, por exemplo, o mantenedor adicionou a sua chave pública GPG na forma de um objeto blob e depois o rotulou em uma tag. Você pode visualizar essa chave pública rodando isso em um clone no repositório do Git:

```
$ git cat-file blob junio-gpg-pub
```

O repositório do kernel do Linux também tem um objeto tag que não aponta a um commit — a primeira tag criada que aponta à inicial da árvore de importação do código-fonte.

Remotos (Remotes)

O terceiro tipo de referência que você verá é uma referência remota (remote reference). Se você adicionar um remote e fizer push para ele, o Git armazenará no diretório `refs/remotes`, para cada branch, o valor enviado pela última vez a esse remote. Por exemplo, você pode adicionar um remote chamado `origin` e fazer push do seu branch `master` para ele:

```
$ git remote add origin git@github.com:schacon/simplegit-progit.git
```

```
$ git push origin master
Counting objects: 11, done.
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 716 bytes, done.
Total 7 (delta 2), reused 4 (delta 1)
To git@github.com:schacon/simplegit-progit.git
  a11bef0..ca82a6d  master -> master
```

Em seguida, consultando o arquivo `refs/remotes/origin/master`, você pode ver onde estava o branch `master` do remote `origin` na última vez em que se comunicou com o servidor:

```
$ cat .git/refs/remotes/origin/master
ca82a6dff817ec66f44342007202690a93763949
```

Referências remotas diferem das branches (nas referências a `refs/heads`) principalmente porque elas são consideradas do tipo somente de leitura (read-only). Você poderá fazer o `git checkout` com um, mas o Git nunca referenciará a HEAD a esse referido, simbolicamente; portanto, você jamais fará as atualizações nele usando um comando de `commit`. O Git as gerencia como marcadores do último estado conhecido desses branches nos respectivos servidores.

Arquivos de pacote (Packfiles)

Se você seguiu todas as instruções de exemplo da seção anterior, agora você deve ter um repositório Git de teste com 11 objetos — quatro blobs, três trees (árvores), três commits e uma tag:

```
$ find .git/objects -type f
.git/objects/01/55eb4229851634a0f03eb265b69f5a2d56f341 # tree 2
.git/objects/1a/410efbd13591db07496601ebc7a059dd55cfe9 # commit 3
.git/objects/1f/7a7a472abf3dd9643fd615f6da379c4acb3e3a # test.txt v2
.git/objects/3c/4e9cd789d88d8d89c1073707c3585e41b0e614 # tree 3
.git/objects/83/baae61804e65cc73a7201a7252750c76066a30 # test.txt v1
.git/objects/95/85191f37f7b0fb9444f35a9bf50de191beadc2 # tag
.git/objects/ca/c0cab538b970a37ea1e769cbbde608743bc96d # commit 2
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4 # 'test content'
.git/objects/d8/329fc1cc938780ffdd9f94e0d364e0ea74f579 # tree 1
.git/objects/fa/49b077972391ad58037050f2a75f74e3671e92 # new.txt
.git/objects/fd/f4fc3344e67ab068f836878b6c4951e3b15f3d # commit 1
```

O Git comprime o conteúdo desses arquivos com zlib e, como você não está armazenando muita coisa, todos esses arquivos juntos ocupam apenas 925 bytes. Agora você vai adicionar mais conteúdo de tamanho razoável ao repositório para demonstrar uma funcionalidade interessante do Git. Para demonstrar, nós adicionaremos o arquivo `repo.rb` da biblioteca do Grit — que é um arquivo de código-fonte de aproximadamente 22K:

```
$ curl https://raw.githubusercontent.com/mojombo/grit/master/lib/grit/repo.rb >
repo.rb
```

```
$ git checkout master
$ git add repo.rb
$ git commit -m 'Create repo.rb'
[master 484a592] Create repo.rb
 3 files changed, 709 insertions(+), 2 deletions(-)
 delete mode 100644 bak/test.txt
 create mode 100644 repo.rb
 rewrite test.txt (100%)
```

Se você olhar para a árvore resultante, você poderá ver o valor SHA-1 que foi calculado para o seu novo objeto blob `repo.rb`:

```
$ git cat-file -p master^{tree}
100644 blob fa49b077972391ad58037050f2a75f74e3671e92      new.txt
100644 blob 033b4468fa6b2a9547a70d88d1bbe8bf3f9ed0d5      repo.rb
100644 blob e3f094f522629ae358806b17daf78246c27c007b      test.txt
```

Você pode então usar o `git cat-file` para ver quão grande é esse objeto:

```
$ git cat-file -s 033b4468fa6b2a9547a70d88d1bbe8bf3f9ed0d5
22044
```

Neste momento, faça uma pequena modificação nesse arquivo e veja o que acontece:

```
$ echo '# testing' >> repo.rb
$ git commit -am 'Modify repo.rb a bit'
[master 2431da6] Modify repo.rb a bit
 1 file changed, 1 insertion(+)
```

Ao conferir a árvore criada por esse último commit, você verá algo interessante:

```
$ git cat-file -p master^{tree}
100644 blob fa49b077972391ad58037050f2a75f74e3671e92      new.txt
100644 blob b042a60ef7dff760008df33cee372b945b6e884e      repo.rb
100644 blob e3f094f522629ae358806b17daf78246c27c007b      test.txt
```

O blob agora é um blob diferente, o que significa que, embora você tenha adicionado apenas uma única linha ao final de um arquivo de 400 linhas, o Git armazenou esse novo conteúdo como um objeto completamente novo:

```
$ git cat-file -s b042a60ef7dff760008df33cee372b945b6e884e
22054
```

Você tem dois objetos de 22K quase idênticos em seu disco (cada um comprimido para

aproximadamente 7K). Não seria legal se o Git pudesse armazenar um deles na íntegra, mas então para o segundo objeto o Git armazenasse apenas o delta entre ele e o primeiro?

Acontece que ele pode. O formato inicial em que o Git salva objetos no disco é chamado de formato de objeto "loose" (solto). No entanto, ocasionalmente, o Git empacota (packs) vários desses objetos num único arquivo binário chamado de "packfile" (arquivo de pacote) a fim de economizar espaço e ser mais eficiente. O Git faz isso se você tiver muitos objetos loose soltos, se você rodar o comando `git gc` manualmente, ou se você os submeter (push) ao servidor remoto. Para ver o que acontece, você pode pedir ao Git para empacotar os objetos chamando o comando `git gc`:

```
$ git gc
Counting objects: 18, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (14/14), done.
Writing objects: 100% (18/18), done.
Total 18 (delta 3), reused 0 (delta 0)
```

Se você olhar em seu diretório `objects`, você notará que a maioria de seus objetos sumiu e um novo par de arquivos apareceu:

```
$ find .git/objects -type f
.git/objects/bd/9dbf5aae1a3862dd1526723246b20206e5fc37
.git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4
.git/objects/info/packs
.git/objects/pack/pack-978e03944f5c581011e6998cd0e9e30000905586.idx
.git/objects/pack/pack-978e03944f5c581011e6998cd0e9e30000905586.pack
```

Os objetos que permanecem são os blobs que não são apontados por nenhum commit — neste caso, o exemplo "what is up, doc?" e os blobs de exemplo "test content" que você criou anteriormente. Como você nunca os adicionou a nenhum commit, eles são considerados dangling (soltos/perdidos) e não são empacotados no seu novo packfile.

Os outros arquivos são o seu novo arquivo packfile e um índice. O packfile é um único arquivo que abrange o conteúdo de todos os objetos que foram removidos do seu sistema de arquivos. O índice é um arquivo que contém os offsets (deslocamentos) para que você possa buscar e encontrar de forma veloz para algum objeto mais específico. O que é legal é que, embora os objetos no disco antes de você executar o comando `gc` tivessem juntos em média 15K de tamanho, o novo arquivo de pacote (packfile) tem apenas 7K. Você cortou pela metade a utilização do seu disco em virtude pelo empacotamento de seus objetos.

Como o Git faz isso? Quando o Git empacota objetos, ele procura por arquivos que tenham nomes e tamanhos semelhantes e armazena apenas os deltas (as diferenças) de uma versão do arquivo para a próxima. Você pode olhar no arquivo packfile e ver o que o Git fez para poupar mais espaço. O comando plumbing `git verify-pack` permite que você veja tudo o que foi empacotado:

```
$ git verify-pack -v .git/objects/pack/pack-
978e03944f5c581011e6998cd0e9e30000905586.idx
```

```
2431da676938450a4d72e260db3bf7b0f587bbc1 commit 223 155 12
69bcdaff5328278ab1c0812ce0e07fa7d26a96d7 commit 214 152 167
80d02664cb23ed55b226516648c7ad5d0a3deb90 commit 214 145 319
43168a18b7613d1281e5560855a83eb8fde3d687 commit 213 146 464
092917823486a802e94d727c820a9024e14a1fc2 commit 214 146 610
702470739ce72005e2edff522fde85d52a65df9b commit 165 118 756
d368d0ac0678cbe6cce505be58126d3526706e54 tag    130 122 874
fe879577cb8cffcdf25441725141e310dd7d239b tree   136 136 996
d8329fc1cc938780ffdd9f94e0d364e0ea74f579 tree   36 46 1132
deef2e1b793907545e50a2ea2ddb5ba6c58c4506 tree   136 136 1178
d982c7cb2c2a972ee391a85da481fc1f9127a01d tree   6 17 1314 1 \
  deef2e1b793907545e50a2ea2ddb5ba6c58c4506
3c4e9cd789d88d8d89c1073707c3585e41b0e614 tree   8 19 1331 1 \
  deef2e1b793907545e50a2ea2ddb5ba6c58c4506
0155eb4229851634a0f03eb265b69f5a2d56f341 tree   71 76 1350
83baae61804e65cc73a7201a7252750c76066a30 blob   10 19 1426
fa49b077972391ad58037050f2a75f74e3671e92 blob   9 18 1445
b042a60ef7dff760008df33cee372b945b6e884e blob   22054 5799 1463
033b4468fa6b2a9547a70d88d1bbe8bf3f9ed0d5 blob   9 20 7262 1 \
  b042a60ef7dff760008df33cee372b945b6e884e
1f7a7a472abf3dd9643fd615f6da379c4acb3e3a blob   10 19 7282
non delta: 15 objects
chain length = 1: 3 objects
.git/objects/pack/pack-978e03944f5c581011e6998cd0e9e30000905586.pack: ok
```

Aqui, o blob `033b4`, que se você bem se lembrar era a primeira versão do seu arquivo `repo.rb`, está fazendo a referência ao blob `b042a`, que se tratava da segunda versão do arquivo. A terceira coluna na saída é o tamanho do objeto no pack (pacote), de modo que você possa ver que `b042a` consome 22K do arquivo, mas que `033b4` leva apenas 9 bytes. O que também é interessante é que a segunda versão do arquivo é a única que é armazenada de forma intacta, enquanto a versão original é armazenada como delta — isso ocorre porque é mais provável que você precise de um acesso mais rápido à versão mais recente do arquivo.

A parte muito interessante em tudo isso é que ela pode ser repacotada a qualquer momento. Ocasionalmente o Git reempacota o seu banco de dados automaticamente, sempre visando tentar economizar mais espaço, mas você pode também o reempacotar a qualquer momento que quiser ao se rodar com a manualidade através do `git gc` pelas suas próprias mãos.

Refspec

Ao longo deste livro, usamos mapeamentos simples de branches remotos para referências locais, mas eles podem ser mais complexos. Suponha que você estava acompanhando as últimas seções e criou um pequeno repositório Git local, e agora queria adicionar um *remoto* (remote) a ele:

```
$ git remote add origin https://github.com/schacon/simplegit-progit
```

Executar o comando acima adiciona uma seção ao arquivo `.git/config` do seu repositório, especificando o nome do remoto (`origin`), a URL do repositório remoto e a *refspec* a ser usada para a

busca (fetching):

```
[remote "origin"]
    url = https://github.com/schacon/simplegit-progit
    fetch = +refs/heads/*:refs/remotes/origin/*
```

O formato do refspec é, primeiro, um `+` opcional, seguido por `<src>:<dst>`, onde `<src>` é o padrão para referências no lado remoto e `<dst>` é onde essas referências serão rastreadas localmente. O `+` diz ao Git para atualizar a referência, mesmo se não for um fast-forward.

No caso padrão que é automaticamente escrito por um comando `git remote add origin`, o Git busca todas as referências sob `refs/heads/` no servidor e as escreve para `refs/remotes/origin/` localmente. Então, se houver um branch `master` no servidor, você pode acessar o log daquele branch localmente por meio de qualquer um dos seguintes:

```
$ git log origin/master
$ git log remotes/origin/master
$ git log refs/remotes/origin/master
```

Eles são todos equivalentes, porque o Git expande cada um deles para `refs/remotes/origin/master`.

Se, em vez disso, você quiser que o Git faça o pull apenas do branch `master` a cada vez, e não de todos os outros branches no servidor remoto, você pode alterar a linha de fetch para se referir apenas àquele branch:

```
fetch = +refs/heads/master:refs/remotes/origin/master
```

Esse é apenas o refspec padrão para o comando `git fetch` para aquele remoto. Se você quiser fazer uma busca apenas uma vez, você também pode especificar o refspec específico na linha de comando. Para fazer pull do branch `master` do remoto para `origin/mymaster` localmente, você pode executar:

```
$ git fetch origin master:refs/remotes/origin/mymaster
```

Você também pode especificar múltiplos refspecs. Na linha de comando, você pode fazer pull de vários branches assim:

```
$ git fetch origin master:refs/remotes/origin/mymaster \
     topic:refs/remotes/origin/topic
From git@github.com:schacon/simplegit
 ! [rejected]        master     -> origin/mymaster  (non fast forward)
 * [new branch]      topic      -> origin/topic
```

Neste caso, o pull do branch `master` foi rejeitado porque não foi listado como uma referência fast-forward. Você pode anular isso especificando o `+` na frente do refspec.

Você também pode especificar múltiplos refspecs para buscar no seu arquivo de configuração. Se você quiser buscar sempre os branches `master` e `experiment` a partir do remote `origin`, adicione duas linhas:

```
[remote "origin"]
    url = https://github.com/schacon/simplegit-progit
    fetch = +refs/heads/master:refs/remotes/origin/master
    fetch = +refs/heads/experiment:refs/remotes/origin/experiment
```

Desde o Git 2.6.0, você pode usar globs parciais no padrão para combinar múltiplos branches, então isso funciona:

```
fetch = +refs/heads/qa*:refs/remotes/origin/qa*
```

Ainda melhor, você pode usar namespaces (ou diretórios) para realizar o mesmo com mais estrutura. Se você tem uma equipe de controle de qualidade (QA) que envia uma série de branches, e você quer obter o branch `master` e qualquer um dos branches da equipe de QA, mas nada mais, você pode usar uma seção de configuração assim:

```
[remote "origin"]
    url = https://github.com/schacon/simplegit-progit
    fetch = +refs/heads/master:refs/remotes/origin/master
    fetch = +refs/heads/qa/*:refs/remotes/origin/qa/*
```

Se você tiver um processo de fluxo de trabalho complexo que tem uma equipe de controle de qualidade (QA) submetendo branches (pushing), desenvolvedores submetendo branches e equipes de integração submetendo e colaborando em branches remotas, será possível classificá-las em um namespace com facilidade dessa forma.

Submetendo Refspecs (Pushing)

É bom que você possa buscar referências no namespace dessa forma, mas como a equipe de QA consegue colocar seus branches em um namespace `qa/` em primeiro lugar? Você consegue isso usando refspecs para fazer o push.

Se a equipe de QA quiser enviar seu branch `master` para `qa/master` no servidor remoto, ela pode executar:

```
$ git push origin master:refs/heads/qa/master
```

Se eles quiserem que o Git faça isso automaticamente toda vez que executarem `git push origin`, eles podem adicionar um valor `push` ao seu arquivo de configuração:

```
[remote "origin"]
    url = https://github.com/schacon/simplegit-progit
```

```
    fetch = +refs/heads/*:refs/remotes/origin/*
    push = refs/heads/master:refs/heads/qa/master
```

Mais uma vez, isso fará com que um `git push origin` envie o branch `master` local para o branch remoto `qa/master` por padrão.



Não é possível usar a especificação de ref para buscar em um repositório e enviar para outro. Para obter um exemplo de como fazer isso, consulte Mantenha o seu repositório público do GitHub atualizado.

Excluindo Referências

Você também pode usar o refspec para excluir referências do servidor remoto, executando algo assim:

```
$ git push origin :topic
```

Como o refspec é `<src>:<dst>`, ao deixar de fora a parte `<src>`, isso basicamente diz para transformar o branch `topic` no remoto em nada, o que a exclui.

Ou você pode usar a sintaxe mais nova (disponível desde o Git v1.7.0):

```
$ git push origin --delete topic
```

Protocolos de Transferência

O Git pode transferir dados entre dois repositórios de duas maneiras principais: pelo protocolo “dumb” e pelo protocolo “smart”. Esta seção cobrirá rapidamente como esses dois protocolos principais operam.

O Protocolo Dumb

Se você estiver configurando um repositório para ser servido somente para leitura via HTTP, provavelmente será usado o protocolo dumb. Esse protocolo é chamado de “dumb” porque não exige nenhum código específico do Git no lado do servidor durante o processo de transporte; o processo de fetch é uma série de requisições HTTP `GET`, nas quais o cliente pode presumir a organização do repositório Git no servidor.



Hoje em dia o protocolo burro é muito raramente usado. É difícil o tornar seguro ou torná-lo privado, de forma em que a maioria dos hosts Git (ambos em servidores baseados em nuvem e os locais/físicos/on-premises) se recusarão a usá-lo. Em geral, recomenda-se usar o protocolo smart, que descreveremos um pouco mais adiante.

Vamos acompanhar o processo de `http-fetch` para a biblioteca do simplegit:

```
$ git clone http://server/simplegit-progit.git
```

A primeira coisa que esse comando faz é baixar o arquivo `info/refs`. Esse arquivo é escrito pelo comando `update-server-info`, e é por isso que você precisa habilitar isso como um hook `post-receive` para que o transporte por intermédio HTTP funcione adequadamente:

```
=> GET info/refs
ca82a6dff817ec66f44342007202690a93763949     refs/heads/master
```

Agora você tem uma lista das referências remotas e seus SHA-1s. Depois, você procura pelo que é a referência da HEAD, para que você saiba do que deve fazer o checkout quando você terminar:

```
=> GET HEAD
ref: refs/heads/master
```

Você precisará fazer o checkout do branch `master` quando você tiver concluído o processo. Neste ponto, você está pronto para iniciar o processo da caminhada. Como seu ponto de partida é o objeto commit `ca82a6` que você viu no arquivo `info/refs`, comece buscando-o:

```
=> GET objects/ca/82a6dff817ec66f44342007202690a93763949
(179 bytes of binary data)
```

Você recebe um objeto de volta – ele está em formato loose no servidor e foi obtido por uma requisição HTTP GET estática. Você pode usar a descompressão dele via zlib, remover o cabeçalho e dar uma olhada ao conteúdo do commit:

```
$ git cat-file -p ca82a6dff817ec66f44342007202690a93763949
tree cfda3bf379e4f8dba8717dee55aab78aef7f4daf
parent 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
author Scott Chacon <schacon@gmail.com> 1205815931 -0700
committer Scott Chacon <schacon@gmail.com> 1240030591 -0700

Change version number
```

Em seguida, você tem mais dois objetos para recuperar – `cfda3b`, a árvore de conteúdo para a qual aponta o commit que acabamos de obter, e `085bb3`, o commit pai:

```
=> GET objects/08/5bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
(179 bytes of data)
```

Isso te devolve ao seu próximo objeto de commit. Pegue o objeto da tree (árvore):

```
=> GET objects/cf/da3bf379e4f8dba8717dee55aab78aef7f4daf
```

```
(404 - Not Found)
```

Opa – parece que esse objeto árvore não está em formato loose no servidor, portanto você recebe uma resposta 404. Há alguns motivos para isso – o objeto pode estar em um repositório alternativo ou em um packfile desse repositório. O Git verifica por qualquer alternativo que primeiro for listado:

```
=> GET objects/info/http-alternates
(empty file)
```

Se isso retornar uma lista de URLs alternativas, o Git procurará ali arquivos loose e packfiles – esse é um bom mecanismo para que projetos derivados uns dos outros compartilhem objetos no disco. Contudo, como nenhum alternativo é listado neste caso, seu objeto deve estar num arquivo de packfile. Para ver quais packfiles estão disponíveis nesse servidor, você precisa obter o arquivo `objects/info/packs`, que contém uma lista deles (também gerada por `update-server-info`):

```
=> GET objects/info/packs
P pack-816a9b2334da9953e530f27bcac22082a9f5b835.pack
```

Há apenas um arquivo de packfile no servidor, então seu objeto está obviamente nele, mas você checará o arquivo de índice para ter certeza disso. Isso também é útil quando há vários packfiles no servidor, pois permite descobrir qual deles contém o objeto necessário:

```
=> GET objects/pack/pack-816a9b2334da9953e530f27bcac22082a9f5b835.idx
(4k of binary data)
```

Agora que você tem o índice do packfile, pode verificar se o objeto está nele – o índice lista os SHA-1 dos objetos contidos no packfile e os deslocamentos até esses objetos. O seu objeto se localiza lá, sendo assim então vá em frente e obtenha o arquivo de packfile inteiramente:

```
=> GET objects/pack/pack-816a9b2334da9953e530f27bcac22082a9f5b835.pack
(13k of binary data)
```

Você detém de um objeto à sua árvore, então, assim, continue a seguir seu trajeto caminhando aos seus referidos commits. Eles todos também estão dentro do packfile em que você acabou de baixar, então você não precisa fazer mais requisições para o seu servidor. O Git faz o checkout de uma cópia de trabalho do branch `master` que foi apontado pela referência da HEAD que você baixou logo mais ao seu início.

O Protocolo Smart

O protocolo dumb é simples, mas um pouco ineficiente, e não permite gravar dados do cliente no servidor. O protocolo smart é o método mais comum de transferência de dados, mas exige na extremidade remota um processo que compreenda o Git – ele pode ler os dados locais, descobrir o

que o cliente possui e necessita e gerar um packfile personalizado. Há dois conjuntos de processos para transferir dados: um par para enviar dados e outro para baixá-los.

Enviando Dados

Para enviar dados para um processo remoto, o Git usa os processos `send-pack` e `receive-pack`. O processo `send-pack` é executado no cliente e se conecta a um processo `receive-pack` no lado remoto.

SSH

Por exemplo, digamos que você execute `git push origin master` em seu projeto, e a `origin` é definida como uma URL que usa o protocolo de SSH. O Git aciona o processo `send-pack`, que inicia uma conexão via SSH com seu servidor. Ele tenta executar um comando no servidor remoto por meio de uma chamada num processo provindo pelo respectivo SSH de modo em que se pareça com algo mais como desta exata assimilação disposta que venha logo a seguir logo abaixo, em seguida:

```
$ ssh -x git@server "git-receive-pack 'simplegit-progit.git'"
00a5ca82a6dff817ec66f4437202690a93763949 refs/heads/master□report-status \
    delete-refs side-band-64k quiet ofs-delta \
    agent=git/2:2.1.1+github-607-gfba4028 delete-refs
0000
```

O comando `git-receive-pack` responde imediatamente com uma linha para cada referência que possui naquele momento – nesse caso, apenas a branch `master` e seu SHA-1. A primeira linha também contém uma lista das capacidades do servidor (aqui, `report-status`, `delete-refs`, e algumas outras mais à sua exatidão, incluindo nelas até mesmo o identificador do cliente).

Os dados são transmitidos em blocos. Cada bloco começa com um valor hexadecimal de 4 caracteres que especifica seu comprimento (incluindo os 4 bytes do próprio tamanho). Os blocos geralmente contêm uma única linha de dados e um avanço de linha ao final. Seu primeiro bloco começa com 00a5, que é 165 em hexadecimal, indicando que o bloco tem 165 bytes. O próximo trecho é 0000, o que significa que o servidor concluiu a listagem de suas próprias referências.

Agora que ele conhece o estado do servidor, seu processo de `send-pack` determina quais são os commits que ele tem e por fim quais são eles os quais o servidor não detém consigo em partes correspondentes nos referidos termos presentes nas suas referenciadas partes da parte provinda dele próprio, que se restam ali, com as faltas destes. Para cada referência que o push atualizará, o processo `send-pack` transmite essa informação ao processo `receive-pack`. Por exemplo, se você estiver atualizando o branch `master` e adicionando um branch `experiment`, a resposta `send-pack` pode ficar parecida com isso:

```
0076ca82a6dff817ec66f44342007202690a93763949 15027957951b64cf874c3557a0f3547bd83b3ff6
\
    refs/heads/master report-status
006c0000000000000000000000000000000000000000 cdfdb42577e2506715f8cfeacdbabc092bf63e8d
\
    refs/heads/experiment
0000
```

O Git envia uma linha para cada referência que você está atualizando, com o comprimento da linha, o SHA-1 antigo, o novo SHA-1 e a referência que está sendo atualizada. A primeira linha também contém as capacidades do cliente. O valor SHA-1 formados de todos os termos contendo apenas os números constituídos em '0's significa que não havia nada ali antes – porque você está adicionando a referência de `experiment`. Se você estivesse excluindo uma referência, veria o oposto: apenas '0's do lado direito.

Em seguida, o cliente envia um packfile com todos os objetos que o servidor ainda não possui. Por fim, o servidor responde com uma indicação de sucesso (ou falha):

```
000eunpack ok
```

HTTP(S)

Esse processo é praticamente o mesmo via HTTP, embora a negociação inicial seja um pouco diferente. A conexão é iniciada com esta requisição:

```
=> GET http://server/simplegit-progit.git/info/refs?service=git-receive-pack
001f# service=git-receive-pack
00ab6c5f0e45abd7832bf23074a333f739977c9e8188 refs/heads/master□report-status \
    delete-refs side-band-64k quiet ofs-delta \
    agent=git/2:2.1.1~vmg-bitmaps-bugaloo-608-g116744e
0000
```

Esse é o fim da primeira troca de informações entre cliente e servidor. O cliente então faz outra requisição, dessa vez um `POST`, com os dados fornecidos pelo `send-pack`.

```
=> POST http://server/simplegit-progit.git/git-receive-pack
```

A requisição `POST` inclui a saída do `send-pack` e o packfile como sua carga útil. O servidor então indica sucesso ou falha com a sua correspondente resposta HTTP que a responde.

Lembre-se de que o protocolo HTTP ainda pode encapsular esses dados em uma codificação de transferência em blocos.

Baixando (Transferindo/Downloading) Dados

Quando você baixa dados, os processos `fetch-pack` e `upload-pack` entram em ação. O cliente inicia um processo `fetch-pack`, que se conecta a um processo `upload-pack` no lado remoto para negociar quais dados serão transferidos.

SSH

Se você está fazendo o fetch via SSH, o `fetch-pack` roda algo assim:

```
$ ssh -x git@server "git-upload-pack 'simplegit-progit.git'"
```

Depois que o `fetch-pack` se conecta, o `upload-pack` devolve algo semelhante a isto:

```
00dfca82a6dff817ec66f44342007202690a93763949 HEAD□multi_ack thin-pack \
    side-band side-band-64k ofs-delta shallow no-progress include-tag \
    multi_ack_detailed symref=HEAD:refs/heads/master \
    agent=git/2:2.1.1+github-607-gfba4028
003fe2409a098dc3e53539a9028a94b6224db9d6a6b6 refs/heads/master
0000
```

Isso é muito semelhante à resposta do `receive-pack`, mas as capacidades são diferentes. Além disso, ele informa para onde HEAD aponta (`symref=HEAD:refs/heads/master`), permitindo ao cliente saber o que deve colocar em checkout caso se trate de um clone.

Nesse ponto, o processo `fetch-pack` examina os objetos que possui e responde com os objetos de que precisa, enviando "want" seguido pelo SHA-1 desejado. Ele envia todos os objetos que já possui com "have", seguido pelo SHA-1. Ao final da lista, escreve "done" para fazer o processo `upload-pack` começar a enviar o packfile com os dados necessários:

```
003cwant ca82a6dff817ec66f44342007202690a93763949 ofs-delta
0032have 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7
0009done
0000
```

HTTP(S)

A negociação inicial de uma operação de fetch requer duas requisições HTTP. A primeira é uma requisição `GET` para o mesmo endpoint usado no protocolo burro:

```
=> GET $GIT_URL/info/refs?service=git-upload-pack
001e# service=git-upload-pack
00e7ca82a6dff817ec66f44342007202690a93763949 HEAD□multi_ack thin-pack \
    side-band side-band-64k ofs-delta shallow no-progress include-tag \
    multi_ack_detailed no-done symref=HEAD:refs/heads/master \
    agent=git/2:2.1.1+github-607-gfba4028
003fca82a6dff817ec66f44342007202690a93763949 refs/heads/master
0000
```

Isso é muito semelhante a invocar `git-upload-pack` por uma conexão SSH, mas a segunda troca é realizada como uma requisição separada:

```
=> POST $GIT_URL/git-upload-pack HTTP/1.0
0032want 0a53e9ddeaddad63ad106860237bbf53411d11a7
0032have 441b40d833fdfa93eb2908e52742248faf0ee993
0000
```

Novamente, este é o exato e mesmo formato que o disposto e baseador correspondido logo acima, e

anteriormente. A resposta a essa requisição indica sucesso ou falha e inclui o packfile.

Resumo dos Protocolos

Esta seção contém uma visão geral muito básica dos protocolos de transferência. O protocolo inclui muitos outros recursos, como as capacidades `multi_ack` ou `side-band`, mas abordá-las está fora do escopo deste livro. Tentamos dar uma ideia do fluxo geral entre cliente e servidor; se precisar de mais detalhes, provavelmente será melhor consultar o código-fonte do Git.

Manutenção e Recuperação de Dados

Eventualmente você precisará fazer alguma limpeza – tornar o repositório mais compacto, limpar um repositório que foi importado, ou recuperar trabalho perdido. Essa seção cobrirá alguns desses cenários.

Manutenção

Ocasionalmente, o Git automaticamente executa um comando chamado “auto gc”. Na maior parte do tempo, este comando não faz nada. No entanto, se há muitos objetos loose (objetos soltos que não estão dentro de um packfile) ou muitos packfiles (arquivos de pacotes), o Git inicializa uma versão do comando `git gc` de pleno direito. O “gc” significa garbage collect (coleta de lixo), e o comando faz algumas coisas: ele junta todos os objetos soltos e os coloca em um packfile, consolida os packfiles em um grande packfile, e apaga os objetos que não são alcançáveis a partir de nenhum commit e que têm alguns meses de idade.

Você pode rodar um `auto gc` manualmente da seguinte maneira:

```
$ git gc --auto
```

Novamente, geralmente isso não faz nada. Você deve ter mais ou menos uns 7.000 objetos do tipo loose (objetos soltos) ou mais de 50 do tipo packfiles para que o Git dispare um comando `gc` de fato. Você pode modificar esses limites com as definições de configuração `gc.auto` e `gc.autopacklimit`, respectivamente.

A outra coisa que o `gc` fará é empacotar as suas referências para dentro de um único arquivo. Suponha que o seu repositório contenha os seguintes branches e tags:

```
$ find .git/refs -type f
.git/refs/heads/experiment
.git/refs/heads/master
.git/refs/tags/v1.0
.git/refs/tags/v1.1
```

Se você rodar o `git gc`, você não terá mais esses arquivos no diretório `refs`. O Git vai movê-los visando a eficiência para um arquivo chamado `.git/packed-refs` que se parece com isso:

```
$ cat .git/packed-refs
# pack-refs with: peeled fully-peeled
cac0cab538b970a37ea1e769cbbde608743bc96d refs/heads/experiment
ab1afef80fac8e34258ff41fc1b867c702daa24b refs/heads/master
cac0cab538b970a37ea1e769cbbde608743bc96d refs/tags/v1.0
9585191f37f7b0fb9444f35a9bf50de191beadc2 refs/tags/v1.1
^1a410efbd13591db07496601ebc7a059dd55cfe9
```

Se você atualizar alguma referência, o Git não editará esse arquivo, em vez disso, ele criará um novo arquivo em `refs/heads`. Para obter o SHA-1 apropriado para uma determinada referência, o Git verifica por essa referência no diretório `refs` e então verifica o arquivo `packed-refs` como alternativa. Então se você não conseguir achar a referência no diretório `refs`, ela provavelmente estará no seu arquivo `packed-refs`.

Note a última linha do arquivo, que começa com um `^`. Isto significa que a tag diretamente acima é uma tag anotada (annotated tag) e que a linha é o commit para o qual a tag apontada está orientada.

Recuperação de Dados

Em algum ponto na sua jornada com o Git, você pode perder um commit acidentalmente. Geralmente, isto acontece porque você forçou a deleção de um branch que tinha algum trabalho, e acontece que você afinal de contas o queria; ou você fez um hard-reset num branch, consequentemente abandonando commits que você desejava manter. Assumindo que isto aconteça, como você pode recuperar os seus commits de volta?

Aqui está um exemplo que faz um hard-reset no branch `master` do seu repositório de testes para um commit mais antigo e em seguida recupera os commits perdidos. Primeiro, vamos verificar onde o seu repositório está neste exato ponto:

```
$ git log --pretty=oneline
ab1afef80fac8e34258ff41fc1b867c702daa24b Modify repo.rb a bit
484a59275031909e19aadb7c92262719cfcdf19a Create repo.rb
1a410efbd13591db07496601ebc7a059dd55cfe9 Third commit
cac0cab538b970a37ea1e769cbbde608743bc96d Second commit
fdf4fc3344e67ab068f836878b6c4951e3b15f3d First commit
```

Agora, mova o branch `master` de volta para o commit do meio:

```
$ git reset --hard 1a410efbd13591db07496601ebc7a059dd55cfe9
HEAD is now at 1a410ef Third commit
$ git log --pretty=oneline
1a410efbd13591db07496601ebc7a059dd55cfe9 Third commit
cac0cab538b970a37ea1e769cbbde608743bc96d Second commit
fdf4fc3344e67ab068f836878b6c4951e3b15f3d First commit
```

Você efetivamente perdeu os dois commits superiores – você não tem nenhum branch a partir do qual aqueles commits sejam alcançáveis. Você precisa encontrar o SHA-1 do último commit e então adicionar um branch que aponte para ele. O truque é encontrar aquele último commit SHA-1 – não é como se você o tivesse memorizado, certo?

Muitas vezes, a maneira mais rápida é usar uma ferramenta chamada `git reflog`. Conforme você está trabalhando, o Git grava silenciosamente qual é o seu HEAD toda vez que você o altera. A cada vez que você comita ou muda de branch, o reflog é atualizado. O reflog também é atualizado pelo comando `git update-ref`, o que é outro motivo para usá-lo em vez de simplesmente escrever o valor do SHA-1 nos seus arquivos de ref, como vimos em Referências do Git. Você pode ver por onde você esteve a qualquer momento, executando `git reflog`:

```
$ git reflog
1a410ef HEAD@{0}: reset: moving to 1a410ef
ab1afef HEAD@{1}: commit: Modify repo.rb a bit
484a592 HEAD@{2}: commit: Create repo.rb
```

Aqui podemos ver os dois commits dos quais fizemos checkout, entretanto não há muita informação aqui. Para ver a mesma informação de uma forma muito mais útil, nós podemos executar `git log -g`, o qual dará a você uma saída de log normal para o seu reflog.

```
$ git log -g
commit 1a410efbd13591db07496601ebc7a059dd55cfe9
Reflog: HEAD@{0} (Scott Chacon <schacon@gmail.com>)
Reflog message: updating HEAD
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri May 22 18:22:37 2009 -0700

        Third commit

commit ab1afef80fac8e34258ff41fc1b867c702daa24b
Reflog: HEAD@{1} (Scott Chacon <schacon@gmail.com>)
Reflog message: updating HEAD
Author: Scott Chacon <schacon@gmail.com>
Date:   Fri May 22 18:15:24 2009 -0700

       Modify repo.rb a bit
```

Parece que o commit mais abaixo é o que você perdeu, então você pode recuperá-lo criando um novo branch naquele commit. Por exemplo, você pode criar um branch chamado `recover-branch` a partir daquele commit (ab1afef):

```
$ git branch recover-branch ab1afef
$ git log --pretty=oneline recover-branch
ab1afef80fac8e34258ff41fc1b867c702daa24b Modify repo.rb a bit
484a59275031909e19aadb7c92262719cfcdf19a Create repo.rb
1a410efbd13591db07496601ebc7a059dd55cfe9 Third commit
```

```
cac0cab538b970a37ea1e769cbbde608743bc96d Second commit
fdf4fc3344e67ab068f836878b6c4951e3b15f3d First commit
```

Legal – agora você tem um branch chamado `recover-branch` onde o seu branch `master` costumava estar, fazendo com que os dois primeiros commits sejam alcançáveis de novo. A seguir, suponha que a sua perda por alguma razão não estava no reflog – você pode simular isto removendo a `recover-branch` e apagando o reflog. Agora os dois primeiros commits não são alcançáveis por mais nada:

```
$ git branch -D recover-branch
$ rm -Rf .git/logs/
```

Pelo fato de os dados do reflog serem mantidos no diretório `.git/logs/`, você efetivamente não tem nenhum reflog. Como você consegue recuperar esse commit a esta altura? Uma forma é usar o utilitário `git fsck`, o qual verifica a integridade do seu banco de dados. Se você executá-lo com a opção `--full`, ele mostra a você todos os objetos que não são apontados por outro objeto:

```
$ git fsck --full
Checking object directories: 100% (256/256), done.
Checking objects: 100% (18/18), done.
dangling blob d670460b4b4aece5915caf5c68d12f560a9fe3e4
dangling commit ab1afef80fac8e34258ff41fc1b867c702daa24b
dangling tree aea790b9a58f6cf6f2804eeac9f0abbe9631e4c9
dangling blob 7108f7ecb345ee9d0084193f147cdad4d2998293
```

Neste caso, você consegue ver o seu commit faltante depois do texto “dangling commit” (commit solto/perdido). Você pode recuperá-lo da mesma forma, adicionando um branch que aponte para aquele SHA-1.

Removendo Objetos

Há muitas coisas ótimas sobre o Git, mas um recurso que pode causar problemas é o fato de que um `git clone` baixa o histórico inteiro do projeto, incluindo cada versão de cada arquivo. Isto é bom se o projeto inteiro se trata de código-fonte, pois o Git é altamente otimizado para comprimir esses dados de maneira eficiente. Porém, se alguém em algum momento no histórico do seu projeto adicionou um único arquivo enorme, todo clone para todo o sempre será forçado a baixar aquele arquivo grande, mesmo que ele tenha sido removido do projeto no commit logo em seguida. Como ele é alcançável a partir do histórico, ele sempre estará lá.

Isto pode ser um enorme problema quando você estiver convertendo repositórios do Subversion ou Perforce para o Git. Como você não baixa o histórico inteiro naqueles sistemas, este tipo de adição traz poucas consequências. Se você fez uma importação de outro sistema ou descobriu de alguma forma que o seu repositório está bem maior do que deveria ser, veja aqui como você pode encontrar e remover grandes objetos.

Fique avisado: esta técnica é destrutiva para o histórico dos seus commits. Ela reescreve todo objeto de commit a partir da tree mais antiga que você precisa modificar para remover uma

referência de arquivo grande. Se você fizer isto imediatamente após uma importação, antes de alguém ter começado a basear seu trabalho no commit, você não terá problemas – do contrário, você terá de notificar todos os contribuidores que eles devem fazer o rebase de seus trabalhos para os seus novos commits.

Para demonstrar, você vai adicionar um arquivo grande no seu repositório de testes, removê-lo no commit seguinte, encontrá-lo, e removê-lo permanentemente do repositório. Primeiro, adicione um objeto grande ao seu histórico:

```
$ curl -L https://www.kernel.org/pub/software/scm/git/git-2.1.0.tar.gz > git.tgz
$ git add git.tgz
$ git commit -m 'Add git tarball'
[master 7b30847] Add git tarball
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 git.tgz
```

Ops – você não queria adicionar um enorme tarball ao seu projeto. Melhor se livrar disto:

```
$ git rm git.tgz
rm 'git.tgz'
$ git commit -m 'Oops - remove large tarball'
[master dadf725] Oops - remove large tarball
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 git.tgz
```

Agora, rode o `gc` no seu banco de dados e veja quanto espaço você está usando:

```
$ git gc
Counting objects: 17, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (13/13), done.
Writing objects: 100% (17/17), done.
Total 17 (delta 1), reused 10 (delta 0)
```

Você pode executar o comando `count-objects` para ver rapidamente a quantidade de espaço que você está consumindo:

```
$ git count-objects -v
count: 7
size: 32
in-pack: 17
packs: 1
size-pack: 4868
prune-packable: 0
garbage: 0
size-garbage: 0
```

A entrada `size-pack` é o tamanho dos seus packfiles em kilobytes, então você está utilizando quase 5MB. Antes do último commit, você estava usando algo mais perto de 2K – claramente, remover o arquivo do commit anterior não o removeu do seu histórico. Toda vez que alguém clonar este repositório, terá que baixar todos os 5MB apenas para obter este projeto pequeno, porque você acidentalmente adicionou um arquivo grande. Vamos nos livrar disto.

Primeiro você tem que encontrá-lo. Neste caso, você já sabe de que arquivo se trata. Mas suponha que você não soubesse; como você identificaria qual ou quais arquivos estariam tomando tanto espaço? Se você rodar `git gc`, todos os objetos estarão em um packfile; você pode identificar os grandes objetos executando um outro comando plumbing chamado `git verify-pack` e ordenando pelo terceiro campo da saída, que é o tamanho do arquivo. Você também pode repassá-lo pelo pipe usando o comando `tail` porque você está interessado somente nos últimos poucos maiores arquivos:

```
$ git verify-pack -v .git/objects/pack/pack-29…69.idx \
  | sort -k 3 -n \
  | tail -3
dadf7258d699da2c8d89b09ef6670edb7d5f91b4 commit 229 159 12
033b4468fa6b2a9547a70d88d1bbe8bf3f9ed0d5 blob   22044 5792 4977696
82c99a3e86bb1267b236a4b6eff7868d97489af1 blob   4975916 4976258 1438
```

O objeto grande está no final: 5MB. Para descobrir que arquivo é este, você usará o comando `rev-list`, que você usou rapidamente em Impondo um Formato de Mensagem de Commit Específico. Se você passar a flag `--objects` para o `rev-list`, ele lista todos os SHA-1s de commits e também os SHA-1s de blobs com os caminhos de arquivos associados a eles. Você pode usar isso para encontrar o nome do seu blob:

```
$ git rev-list --objects --all | grep 82c99a3
82c99a3e86bb1267b236a4b6eff7868d97489af1 git.tgz
```

Agora, você precisa remover este arquivo de todas as trees no seu passado. Você pode ver com facilidade quais commits modificaram esse arquivo:

```
$ git log --oneline --branches -- git.tgz
dadf725 Oops - remove large tarball
7b30847 Add git tarball
```

Você precisa reescrever todos os commits seguintes a `7b30847` para remover totalmente este arquivo do seu histórico no Git. Para fazer isso, você usa o `filter-branch`, que você usou em Reescrevendo o Histórico:

```
$ git filter-branch --index-filter \
  'git rm --ignore-unmatch --cached git.tgz' -- 7b30847^..
Rewrite 7b30847d080183a1ab7d18fb202473b3096e9f34 (1/2)rm 'git.tgz'
Rewrite dadf7258d699da2c8d89b09ef6670edb7d5f91b4 (2/2)
```

```
Ref 'refs/heads/master' was rewritten
```

A opção `--index-filter` é similar à opção `--tree-filter` usada em Reescrevendo o Histórico, exceto que, em vez de passar um comando que modifica arquivos checados no disco, você estará modificando sua staging area (área de preparação) ou o index em cada vez.

Em vez de remover um arquivo específico com algo como `rm file`, você terá de removê-lo com `git rm --cached` – você precisa removê-lo a partir do índice, não a partir do disco. A razão para se fazer desta forma é a velocidade – porque o Git não tem que efetuar checkout de cada revisão pro disco antes de rodar o seu filtro, o processo pode ser muito, muito mais veloz. Você pode realizar a mesma tarefa usando `--tree-filter` caso queira. A opção `--ignore-unmatch` para o `git rm` diz a ele para não falhar caso o padrão que você está tentando remover não estiver ali. Por fim, você pede para o `filter-branch` reescrever o seu histórico apenas a partir do commit `7b30847` em diante, pois você sabe que é de onde o problema se originou. Do contrário, ele irá começar a partir do início e vai, de maneira desnecessária, levar mais tempo.

Seu histórico não contém mais nenhuma referência a esse arquivo. Entretanto, o seu reflog e um novo conjunto de refs que o Git adicionou sob `.git/refs/original` quando você rodou o `filter-branch` ainda contêm, então você terá que removê-los para então reempacotar o banco de dados. Você precisa se livrar de qualquer coisa que tenha um ponteiro para esses antigos commits antes de reempacotar:

```
$ rm -Rf .git/refs/original
$ rm -Rf .git/logs/
$ git gc
Counting objects: 15, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (11/11), done.
Writing objects: 100% (15/15), done.
Total 15 (delta 1), reused 12 (delta 0)
```

Vamos ver quanto espaço você salvou.

```
$ git count-objects -v
count: 11
size: 4904
in-pack: 15
packs: 1
size-pack: 8
prune-packable: 0
garbage: 0
size-garbage: 0
```

O tamanho do repositório empacotado baixou para 8K, o que é bem melhor que 5MB. Você pode ver pelo valor do tamanho que o grande objeto ainda está nos seus loose objects, portanto não se foi; mas ele não será transferido num push ou num clone subsequente, que é o mais importante. Se você quisesse muito, você poderia remover o objeto por completo executando o `git prune` com a

opção `--expire`:

```
$ git prune --expire now
$ git count-objects -v
count: 0
size: 0
in-pack: 15
packs: 1
size-pack: 8
prune-packable: 0
garbage: 0
size-garbage: 0
```

Variáveis de Ambiente

O Git sempre roda dentro de um shell `bash` e utiliza uma série de variáveis de ambiente do shell para determinar como ele se comporta. Ocasionalmente, é útil saber quais são essas variáveis e como elas podem ser usadas para fazer com que o Git se comporte da maneira que você deseja. Esta não é uma lista exaustiva de todas as variáveis de ambiente às quais o Git presta atenção, mas nós cobriremos as mais úteis.

Comportamento Global

Algumas partes do comportamento geral do Git enquanto programa de computador dependem das variáveis de ambiente.

`GIT_EXEC_PATH` determina onde o Git procura pelos seus subprogramas (como `git-commit`, `git-diff`, e outros). Você pode verificar a configuração atual rodando `git --exec-path`.

`HOME` não é usualmente considerada como customizável (muitas outras coisas dependem dela), mas é onde o Git procura pelo arquivo global de configurações. Se você quiser uma instalação do Git verdadeiramente portátil, completa com as configurações globais, você pode sobrescrever a `HOME` no perfil de shell do Git portátil.

`PREFIX` é similar, mas serve para as configurações de todo o sistema (system-wide). O Git procura por este arquivo no caminho `$PREFIX/etc/gitconfig`.

`GIT_CONFIG_NOSYSTEM`, se definido, desabilita o uso do arquivo de configurações de todo o sistema. Isto é útil se a configuração do seu sistema estiver interferindo com os seus comandos, mas você não tem o acesso para alterá-la ou removê-la.

`GIT_PAGER` controla o programa utilizado para exibir a saída de múltiplas páginas na linha de comando. Se isto não estiver definido, a `PAGER` será utilizada como alternativa (fallback).

`GIT_EDITOR` é o editor que o Git iniciará quando o usuário precisar editar algum texto (uma mensagem de commit, por exemplo). Se não for definido, o `EDITOR` será usado.

Localizações de Repositórios

O Git utiliza várias variáveis de ambiente para determinar como ele irá se comunicar com o repositório atual.

`GIT_DIR` é a localização da pasta `.git`. Se isto não for especificado, o Git irá subir pela árvore de diretórios até atingir o `~` ou `/`, procurando por um diretório `.git` em cada passo.

`GIT_CEILING_DIRECTORIES` controla o comportamento de busca por um diretório `.git`. Se você acessar diretórios que demoram a carregar (tais como aqueles em drives de fita, ou através de uma conexão de rede lenta), você talvez queira que o Git pare de tentar mais cedo do que ele tentaria normalmente, especialmente se o Git for invocado ao carregar o prompt do seu shell.

`GIT_WORK_TREE` é a localização da raiz do diretório de trabalho de um repositório non-bare. Se `--git-dir` ou `GIT_DIR` for especificado, mas nenhum entre `--work-tree`, `GIT_WORK_TREE` ou `core.worktree` for especificado, o diretório de trabalho atual será considerado como o nível principal da sua árvore de trabalho.

`GIT_INDEX_FILE` é o caminho apontado para o arquivo de index (somente repositórios non-bare).

`GIT_OBJECT_DIRECTORY` pode ser utilizado para especificar a localização do diretório que usualmente reside em `.git/objects`.

`GIT_ALTERNATE_OBJECT_DIRECTORIES` é uma lista separada por dois-pontos (formatada como `/dir/one:/dir/two:…`) a qual diz ao Git onde ele deve procurar por objetos caso não os encontre no `GIT_OBJECT_DIRECTORY`. Se você por acaso tiver muitos projetos com arquivos grandes que contenham exatamente os mesmos conteúdos, esta variável poderá ser usada para evitar o armazenamento de muitas cópias deles.

Pathspecs

Um "pathspec" refere-se à forma como você especifica caminhos para as coisas no Git, incluindo o uso de wildcards (caracteres curinga). Eles são utilizados no arquivo `.gitignore`, mas também na linha de comando (como em `git add *.c`).

`GIT_GLOB_PATHSPECS` e `GIT_NOGLOB_PATHSPECS` controlam o comportamento padrão dos curingas nos pathspecs. Se `GIT_GLOB_PATHSPECS` for definido como 1, os caracteres curinga agirão como curingas (o que é o padrão); se `GIT_NOGLOB_PATHSPECS` for definido como 1, os caracteres curinga apenas corresponderão a eles mesmos, o que significa que algo como `*.c` corresponderá apenas a um arquivo *chamado* "*.c", ao invés de qualquer arquivo cujo nome termine em `.c`. Você pode sobrescrever isso em casos individuais iniciando o pathspec com `:(glob)` ou `:(literal)`, como em `:(glob)\*.c`.

`GIT_LITERAL_PATHSPECS` desabilita ambos os comportamentos acima; nenhum caractere curinga funcionará, e os prefixos de sobrescrita também serão desabilitados.

`GIT_ICASE_PATHSPECS` define todos os pathspecs para funcionarem de maneira insensível a maiúsculas e minúsculas (case-insensitive).

Criando Commits (Committing)

A criação final de um objeto commit no Git é normalmente feita por `git-commit-tree`, que utiliza essas variáveis de ambiente como sua fonte primária de informações, recorrendo aos valores de configuração somente se elas não estiverem presentes.

`GIT_AUTHOR_NAME` é o nome legível por humanos no campo “author” (autor).

`GIT_AUTHOR_EMAIL` é o e-mail para o campo “author”.

`GIT_AUTHOR_DATE` é a marcação de data/hora (timestamp) usada para o campo “author”.

`GIT_COMMITTER_NAME` define o nome legível para o campo “committer”.

`GIT_COMMITTER_EMAIL` é o endereço de e-mail para o campo “committer”.

`GIT_COMMITTER_DATE` é usado para a marcação de data/hora no campo “committer”.

`EMAIL` é o endereço de e-mail de alternativa (fallback) caso o valor da configuração `user.email` não esteja definido. Se *isto* não estiver definido, o Git recorre aos nomes de usuário do sistema e do host.

Operações de Rede

O Git usa a biblioteca `curl` para realizar operações de rede via HTTP, então `GIT_CURL_VERBOSE` diz ao Git para emitir todas as mensagens geradas por essa biblioteca. Isso é parecido com a execução de `curl -v` na linha de comando.

`GIT_SSL_NO_VERIFY` diz ao Git para não verificar os certificados SSL. Isto pode às vezes ser necessário se você estiver usando um certificado auto-assinado (self-signed) para hospedar repositórios Git via HTTPS, ou se você estiver no meio da configuração de um servidor Git mas ainda não instalou um certificado completo.

Se a taxa de dados de uma operação HTTP for menor do que `GIT_HTTP_LOW_SPEED_LIMIT` bytes por segundo por mais do que `GIT_HTTP_LOW_SPEED_TIME` segundos, o Git irá abortar aquela operação. Esses valores sobrescrevem os valores das configurações de `http.lowSpeedLimit` e `http.lowSpeedTime`.

`GIT_HTTP_USER_AGENT` define a string user-agent utilizada pelo Git ao se comunicar via HTTP. O padrão é um valor parecido com `git/2.0.0`.

Fazendo Diffs e Merges

`GIT_DIFF_OPTS` é um nome um pouco inadequado (misnomer). Os únicos valores válidos são `-u<n>` ou `--unified=<n>`, os quais controlam o número de linhas de contexto mostradas em um comando `git diff`.

`GIT_EXTERNAL_DIFF` é usado para sobrescrever o valor de configuração `diff.external`. Se ele for definido, o Git invocará este programa quando um `git diff` for invocado.

`GIT_DIFF_PATH_COUNTER` e `GIT_DIFF_PATH_TOTAL` são úteis por dentro do programa especificado por `GIT_EXTERNAL_DIFF` ou `diff.external`. O primeiro representa qual arquivo numa série está sofrendo o

diff (começando por 1), e o último (latter) é o número total de arquivos no lote (batch).

`GIT_MERGE_VERBOSITY` controla a saída para a estratégia de merge do tipo recursive. Os valores aceitáveis são como a seguir:

- 0 não tem saída alguma, com a possível exceção de uma única mensagem de erro.
- 1 mostra somente os conflitos.
- 2 também exibe alterações de arquivos.
- 3 mostra quando arquivos foram pulados porque não haviam sofrido alterações.
- 4 mostra todos os caminhos à medida que são processados.
- 5 ou mais mostra informações detalhadas de depuração.

O valor padrão é 2.

Depuração (Debugging)

Quer *realmente* saber o que o Git está fazendo? O Git possui um conjunto bastante completo de logs de rastreio (traces) embutido, e a única coisa de que você precisa é ligá-lo. Os possíveis valores para essas variáveis são listados a seguir:

- "true", "1", ou "2" – a categoria de rastro é escrita no stderr.
- Um caminho absoluto começando com `/` – a saída do rastreamento será escrita naquele arquivo.

`GIT_TRACE` controla os rastros (traces) gerais, que não se encaixam em nenhuma categoria específica. Isto inclui a expansão de aliases, e também a delegação para outros subprogramas.

```
$ GIT_TRACE=true git lga
20:12:49.877982 git.c:554               trace: exec: 'git-lga'
20:12:49.878369 run-command.c:341       trace: run_command: 'git-lga'
20:12:49.879529 git.c:282               trace: alias expansion: lga => 'log' '--graph'
'--pretty=oneline' '--abbrev-commit' '--decorate' '--all'
20:12:49.879885 git.c:349               trace: built-in: git 'log' '--graph' '--
pretty=oneline' '--abbrev-commit' '--decorate' '--all'
20:12:49.899217 run-command.c:341       trace: run_command: 'less'
20:12:49.899675 run-command.c:192       trace: exec: 'less'
```

`GIT_TRACE_PACK_ACCESS` controla o rastreio das formas de acesso aos packfiles. O primeiro campo é o packfile sendo acessado, o segundo é o offset por dentro daquele arquivo:

```
$ GIT_TRACE_PACK_ACCESS=true git status
20:10:12.081397 sha1_file.c:2088        .git/objects/pack/pack-c3fa...291e.pack 12
20:10:12.081886 sha1_file.c:2088        .git/objects/pack/pack-c3fa...291e.pack 34662
20:10:12.082115 sha1_file.c:2088        .git/objects/pack/pack-c3fa...291e.pack 35175
# […]
20:10:12.087398 sha1_file.c:2088        .git/objects/pack/pack-e80e...e3d2.pack
56914983
```

```
20:10:12.087419 sha1_file.c:2088        .git/objects/pack/pack-e80e...e3d2.pack
14303666
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working directory clean
```

`GIT_TRACE_PACKET` habilita rastreios a nível de pacotes (packet-level tracing) para operações de rede.

```
$ GIT_TRACE_PACKET=true git ls-remote origin
20:15:14.867043 pkt-line.c:46           packet:          git< # service=git-upload-
pack
20:15:14.867071 pkt-line.c:46           packet:          git< 0000
20:15:14.867079 pkt-line.c:46           packet:          git<
97b8860c071898d9e162678ea1035a8ced2f8b1f HEAD\0multi_ack thin-pack side-band side-
band-64k ofs-delta shallow no-progress include-tag multi_ack_detailed no-done
symref=HEAD:refs/heads/master agent=git/2.0.4
20:15:14.867088 pkt-line.c:46           packet:          git<
0f20ae29889d61f2e93ae00fd34f1cdb53285702 refs/heads/ab/add-interactive-show-diff-func-
name
20:15:14.867094 pkt-line.c:46           packet:          git<
36dc827bc9d17f80ed4f326de21247a5d1341fbc refs/heads/ah/doc-gitk-config
# […]
```

`GIT_TRACE_PERFORMANCE` controla o registro de dados relacionados à performance. A saída exibirá quanto tempo em duração demorará a cada invocação específica feita para o `git`.

```
$ GIT_TRACE_PERFORMANCE=true git gc
20:18:19.499676 trace.c:414             performance: 0.374835000 s: git command: 'git'
'pack-refs' '--all' '--prune'
20:18:19.845585 trace.c:414             performance: 0.343020000 s: git command: 'git'
'reflog' 'expire' '--all'
Counting objects: 170994, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (43413/43413), done.
Writing objects: 100% (170994/170994), done.
Total 170994 (delta 126176), reused 170524 (delta 125706)
20:18:23.567927 trace.c:414             performance: 3.715349000 s: git command: 'git'
'pack-objects' '--keep-true-parents' '--honor-pack-keep' '--non-empty' '--all' '--
reflog' '--unpack-unreachable=2.weeks.ago' '--local' '--delta-base-offset'
'.git/objects/pack/.tmp-49190-pack'
20:18:23.584728 trace.c:414             performance: 0.000910000 s: git command: 'git'
'prune-packed'
20:18:23.605218 trace.c:414             performance: 0.017972000 s: git command: 'git'
'update-server-info'
20:18:23.606342 trace.c:414             performance: 3.756312000 s: git command: 'git'
'repack' '-d' '-l' '-A' '--unpack-unreachable=2.weeks.ago'
Checking connectivity: 170994, done.
20:18:25.225424 trace.c:414             performance: 1.616423000 s: git command: 'git'
'prune' '--expire' '2.weeks.ago'
```

```
20:18:25.232403 trace.c:414            performance: 0.001051000 s: git command: 'git'
'rerere' 'gc'
20:18:25.233159 trace.c:414            performance: 6.112217000 s: git command: 'git'
'gc'
```

`GIT_TRACE_SETUP` mostra as informações a respeito do que o Git está descobrindo em base no repositório e sobre o ambiente com que ele está interagindo.

```
$ GIT_TRACE_SETUP=true git status
20:19:47.086765 trace.c:315            setup: git_dir: .git
20:19:47.087184 trace.c:316            setup: worktree: /Users/ben/src/git
20:19:47.087191 trace.c:317            setup: cwd: /Users/ben/src/git
20:19:47.087194 trace.c:318            setup: prefix: (null)
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working directory clean
```

Diversos

`GIT_SSH`, se for especificado, é o programa que será invocado no lugar de `ssh` quando o Git tentar se conectar em um host SSH. Ele é invocado de forma parecida a `$GIT_SSH [username@]host [-p <port>] <command>`. Note que isto não constitui a forma mais fácil para se customizar o jeito que o `ssh` é invocado; ele não suportará parâmetros extras de linha de comando. Para que possa suportar parâmetros extras na linha de comando, você pode usar o `GIT_SSH_COMMAND`, ou criar um script de wrapper (wrapper script) e definir `GIT_SSH` de modo que aponte a ele, ou usar o arquivo `~/.ssh/config`.

`GIT_SSH_COMMAND` define o comando SSH que é utilizado quando o Git tentar se conectar a um host SSH. O comando é interpretado pelo próprio shell, e argumentos extras de linha de comando podem ser utilizados com `ssh`, tal como com `GIT_SSH_COMMAND="ssh -i ~/.ssh/my_key" git clone git@example.com:my/repo`.

`GIT_ASKPASS` substitui o valor de configuração `core.askpass`. Esse programa é invocado sempre que o Git precisa solicitar credenciais ao usuário; ele recebe um prompt de texto como argumento de linha de comando e deve retornar a resposta em `stdout` (consulte Armazenamento de Credenciais para saber mais sobre esse subsistema).

`GIT_NAMESPACE` controla o acesso a refs com namespace e equivale à flag `--namespace`. Isso é útil principalmente no lado do servidor, onde talvez você queira armazenar vários forks de um único repositório em um só repositório, mantendo apenas as refs separadas.

`GIT_FLUSH` pode ser usada para forçar o Git a usar E/S sem buffer ao escrever incrementalmente em stdout. O valor 1 faz o Git descarregar o buffer com mais frequência; o valor 0 faz com que toda a saída seja armazenada em buffer. O valor padrão (quando essa variável não está definida) seleciona um esquema de buffering adequado de acordo com a atividade e o modo de saída.

`GIT_REFLOG_ACTION` permite especificar o texto descritivo gravado no reflog. Eis um exemplo:

```
$ GIT_REFLOG_ACTION="my action" git commit --allow-empty -m 'My message'
[master 9e3d55a] My message
$ git reflog -1
9e3d55a HEAD@{0}: my action: My message
```

Resumo

Neste ponto, você deve ter um bom entendimento sobre o que o Git faz em plano de fundo e, até certo ponto, a forma como foi implementado. Este capítulo abordou vários comandos de encanamento (plumbing) - comandos que possuem um nível mais baixo e mais simples do que aos da categoria de porcelana (porcelain), dos quais você aprendeu por conta do resto do livro. Ao entender o modo sobre como o Git atua por um nível mais baixo, deverá facilitar no compreendimento à razão por estar fazendo o que faz e também te possibilita criar suas próprias ferramentas e os scripts de suporte, com o intuito de fazer com que o seu específico fluxo de trabalho venha a atuar em seu benefício próprio.

O Git, à base de um formato num arquivo de um sistema por abordagens no conteúdo, reflete a algo em que a sua poderosa atuação possa vir de se fazer facilmente como muito além do que apenas numa base de VCS. Almejamos para que este recentíssimo descobrimento aos seus novos conhecimentos pelas internas da formatação no Git, seja o responsável à implantação a sua própria agradável aplicação destas tecnologias e que sinta de forma mais confortável ao usar o Git sob um trajeto nos caminhos avançados.

Appendix A: O Git em Outros Ambientes

Se você leu o livro inteiro, aprendeu muito sobre como usar o Git na linha de comando. Você pode trabalhar com arquivos locais, conectar seu repositório a outros em uma rede e trabalhar eficientemente com os outros. Mas a história não termina aí; O Git geralmente é usado como parte de um ecossistema maior, e o terminal nem sempre é a melhor maneira de trabalhar com ele. Agora veremos alguns dos outros tipos de ambientes nos quais o Git pode ser útil e como outros aplicativos (incluindo o seu) funcionam em conjunto com o Git.

Interfaces Gráficas

O ambiente nativo do Git é no terminal. Novos recursos aparecem lá primeiro, e apenas na linha de comando todo o poder do Git está completamente a sua disposição. Mas texto simples não é a melhor escolha para todas as tarefas; às vezes uma representação visual é o que você precisa, e alguns usuários são muito mais confortáveis com uma interface de apontar e clicar.

É importante notar que diferentes interfaces são feitas sob medida para diferentes fluxos de trabalho. Alguns clientes expõem apenas um subconjunto de funções do Git cuidadosamente escolhido, a fim de dar suporte a uma forma de trabalho específica que o autor considera eficaz. Visto por esse ângulo, nenhuma dessas ferramentas pode ser considerada “melhor” do que as outras, elas são simplesmente mais adequadas ao propósito a que se destinam. Note também que não há nada que estes clientes gráficos possam fazer que o cliente de linha de comando não possa; a linha de comando ainda é onde você terá o maior poder e controle ao trabalhar com os seus repositórios.

`gitk` e `git-gui`

Quando você instala o Git, você também adquire as suas ferramentas visuais, `gitk` e `git-gui`.

O `gitk` é um visualizador gráfico de histórico. Pense nele como uma poderosa interface gráfica sobre `git log` e `git grep`. Esta é a ferramenta a ser usada quando você está tentando encontrar algo que aconteceu no passado, ou visualizar o histórico do seu projeto.

O Gitk é mais facilmente invocado a partir da linha de comando. Basta entrar com `cd` em um repositório Git e digitar:

```
$ gitk [git log options]
```

O Gitk aceita muitas opções de linha de comando, a maioria das quais é repassada para a ação subjacente do `git log`. Provavelmente uma das mais úteis é a flag `--all`, que diz ao `gitk` para mostrar os commits alcançáveis a partir de *qualquer* referência, e não apenas do HEAD. A interface do Gitk parece com isto:

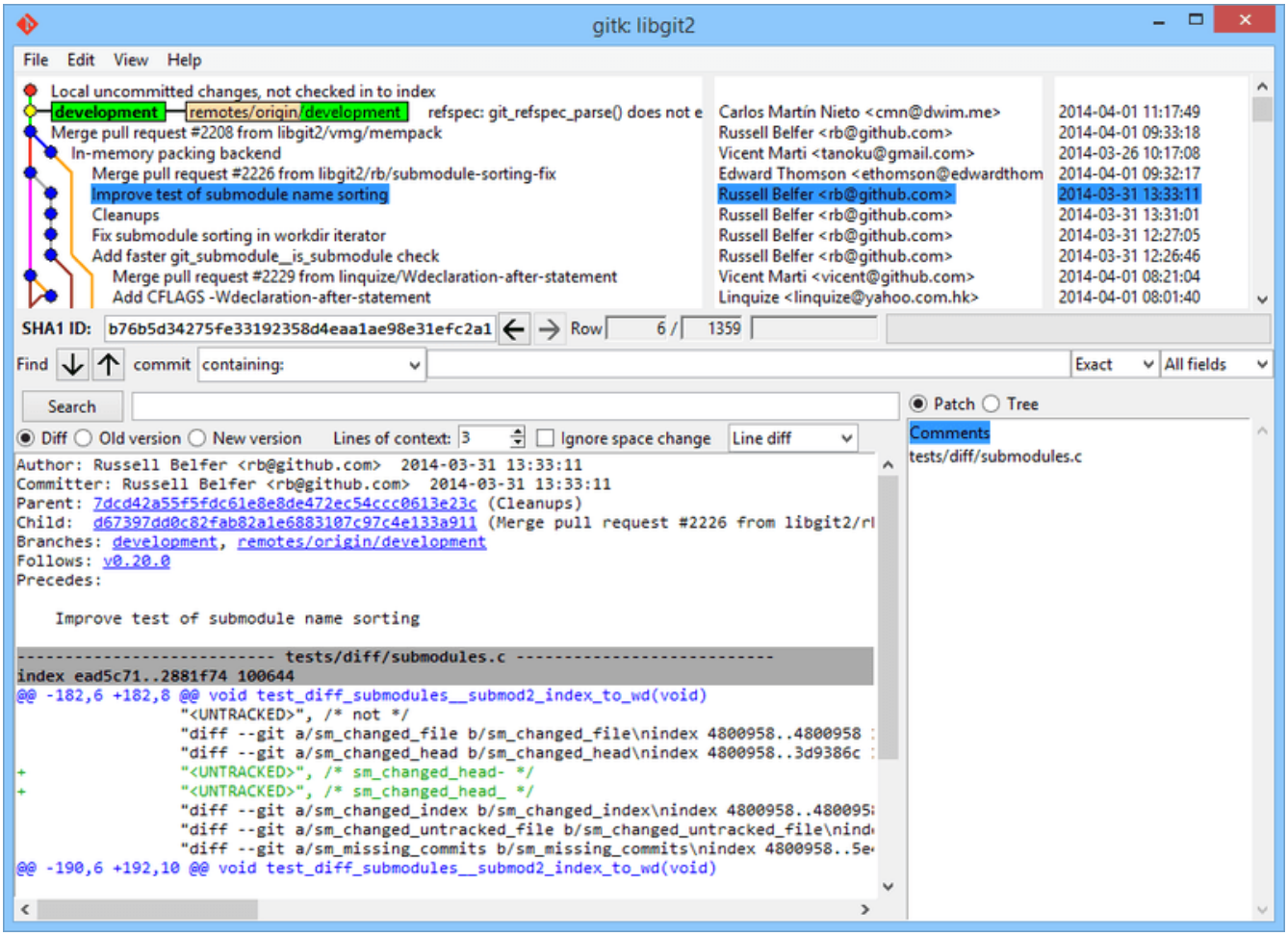


Figure 177. O visualizador de histórico `gitk`

No topo fica algo que se parece um pouco com a saída do `git log --graph`; cada ponto representa um commit, as linhas representam relacionamentos com commits pais, e as refs são mostradas como caixas coloridas. O ponto amarelo representa o HEAD, e o ponto vermelho representa as alterações que ainda não se tornaram um commit. Na parte de baixo fica uma visualização do commit selecionado; os comentários e as mudanças (patch) na esquerda, e uma visão de resumo na direita. No meio há um conjunto de controles utilizados para pesquisar o histórico.

O `git-gui`, por outro lado, é principalmente uma ferramenta para forjar commits. Ele, também, é mais facilmente invocado a partir da linha de comando:

```
$ git gui
```

E ele se parece com isto:

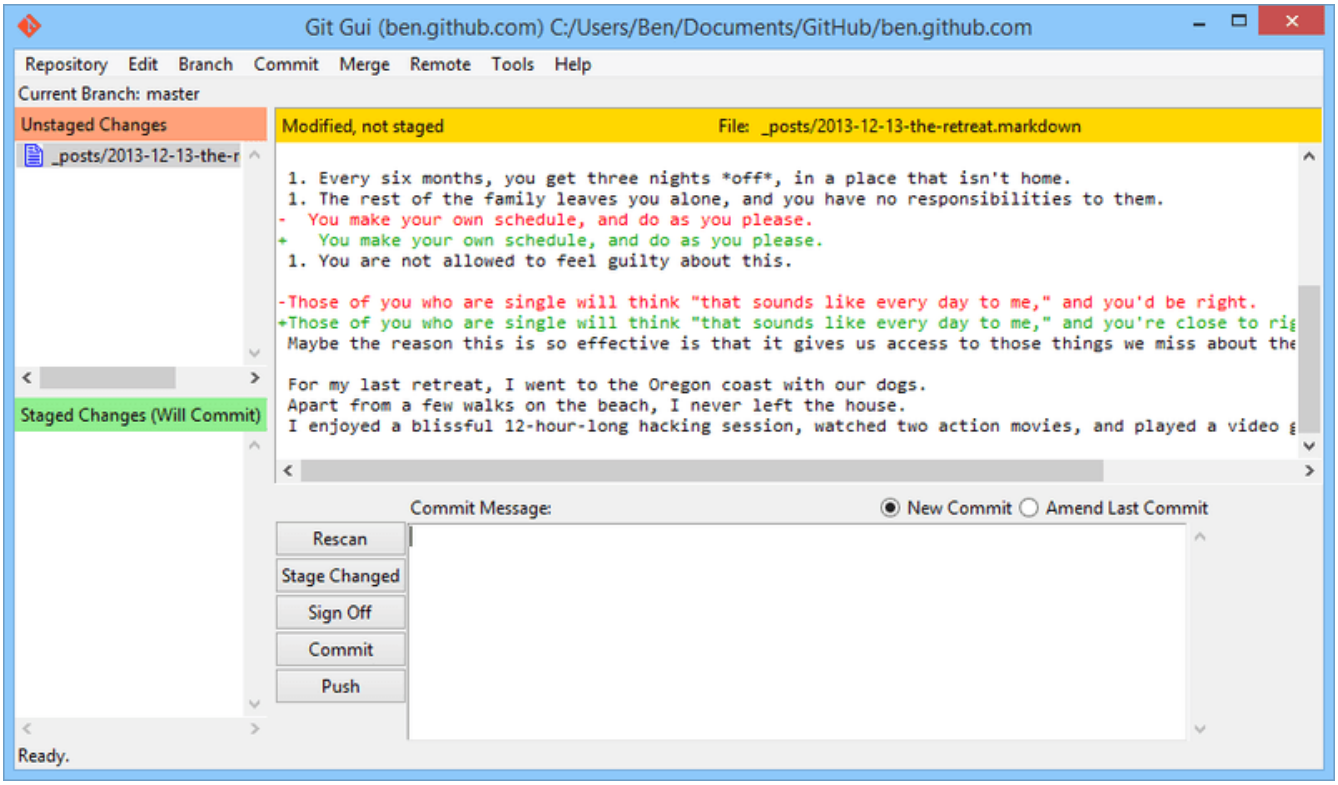


Figure 178. A ferramenta de commits `git-gui`

Na esquerda fica o index; alterações não preparadas (unstaged changes) ficam no topo, alterações preparadas (staged changes) na parte inferior. Você pode mover arquivos inteiros entre os dois estados clicando nos ícones deles, ou você pode selecionar um arquivo para visualização clicando no nome dele.

No topo à direita fica a visualização de diff, que mostra as alterações para o arquivo selecionado no momento. Você pode preparar blocos (hunks) individuais (ou linhas individuais) clicando com o botão direito nesta área.

Na parte inferior direita fica a área de mensagens e ações. Digite sua mensagem na caixa de texto e clique em “Commit” para fazer algo similar a `git commit`. Você também pode escolher consertar o último commit marcando a opção do botão de seleção “Amend”, o que atualizará a área de “Staged Changes” com os conteúdos do último commit. Assim você poderá simplesmente preparar ou não preparar algumas alterações, alterar a mensagem de commit, e clicar em “Commit” novamente para substituir o commit antigo por um novo.

O `gitk` e o `git-gui` são exemplos de ferramentas orientadas a tarefas. Cada uma delas é adaptada para um propósito específico (visualizar histórico e criar commits, respectivamente), e omite os recursos não necessários para essa tarefa.

GitHub para macOS e Windows

O GitHub criou dois clientes Git orientados a fluxos de trabalho: um para Windows e um para macOS. Esses clientes são bons exemplos de ferramentas orientadas a fluxos de trabalho – em vez de expor *toda* a funcionalidade do Git, eles se concentram em um conjunto selecionado de recursos comumente usados que funcionam bem juntos. Eles se parecem com isto:

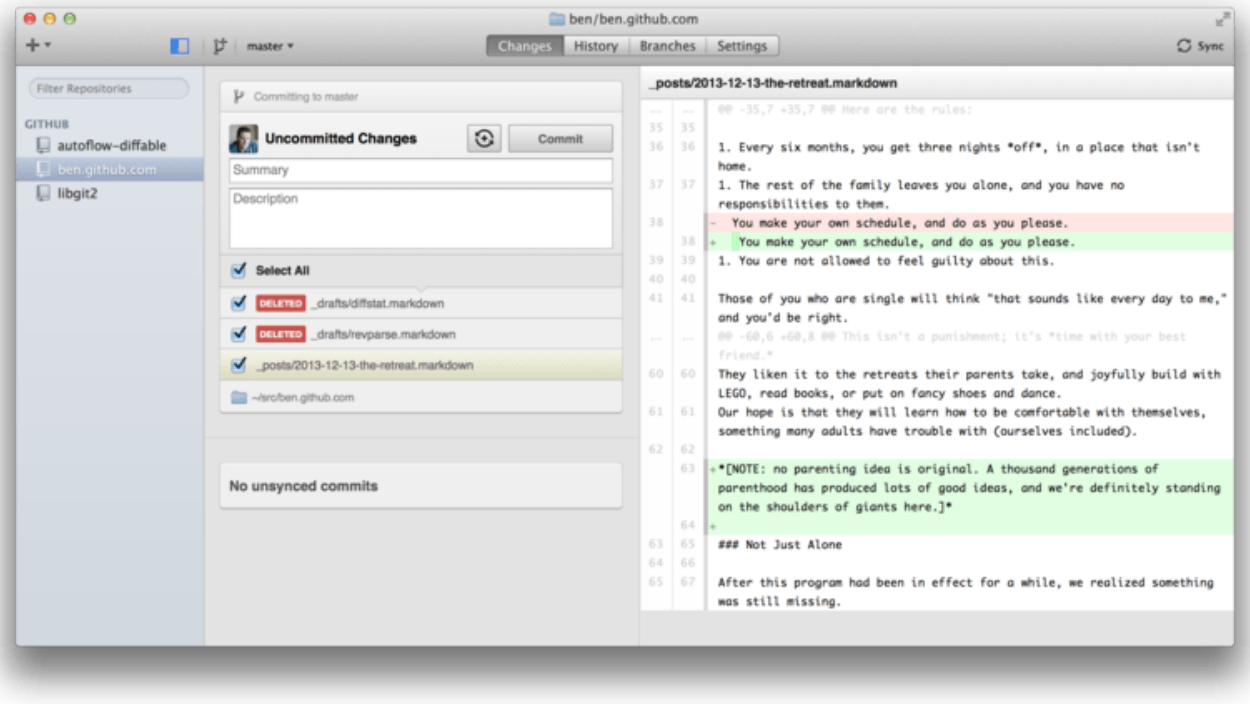


Figure 179. GitHub para macOS

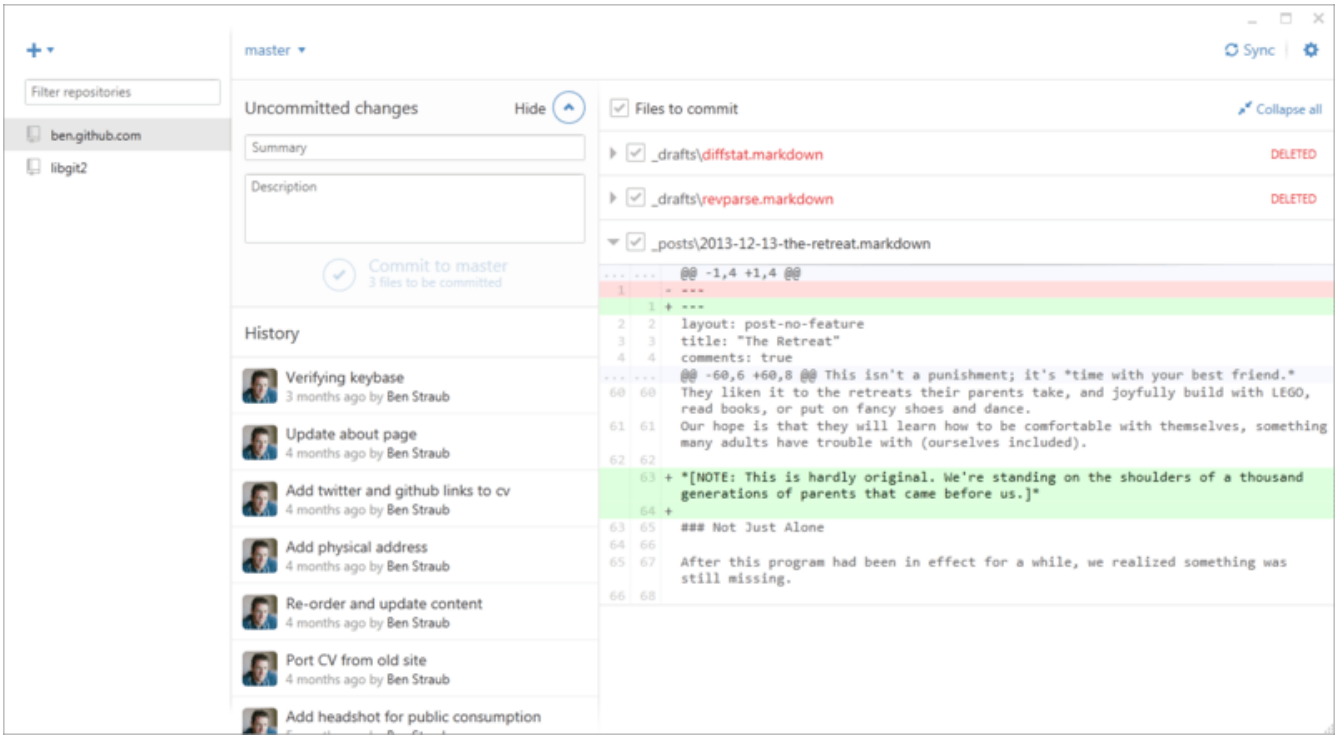


Figure 180. GitHub para Windows

Eles foram desenvolvidos para terem uma aparência e funcionamento muito parecidos, portanto, vamos tratá-los como um único produto neste capítulo. Não faremos um detalhamento dessas ferramentas (elas possuem sua própria documentação), mas um tour rápido da visualização “changes” (que é onde você passará a maior parte do seu tempo) se faz necessário.

- À esquerda fica a lista de repositórios que o cliente está rastreando; você pode adicionar um repositório (seja clonando-o ou anexando-o localmente) clicando no ícone “+” no topo desta área.

- No centro fica a área de entrada de commit, que permite digitar uma mensagem de commit e selecionar quais arquivos devem ser incluídos. No Windows, o histórico de commit é exibido diretamente abaixo; no macOS, fica em uma aba separada.
- À direita fica a visualização do diff, que mostra o que foi alterado em seu diretório de trabalho, ou quais alterações foram incluídas no commit selecionado.
- A última coisa a notar é o botão “Sync” no canto superior direito, que é a principal forma como você interage pela rede.



Você não precisa de uma conta do GitHub para usar essas ferramentas. Embora sejam projetados para destacar o serviço e fluxo de trabalho recomendados do GitHub, eles funcionarão de bom grado com qualquer repositório e executarão operações de rede com qualquer host Git.

Instalação

O GitHub para Windows e macOS pode ser baixado em https://desktop.github.com/. Quando os aplicativos são executados pela primeira vez, eles guiam você por toda a configuração inicial do Git, como configurar seu nome e endereço de e-mail, e ambos definem padrões coerentes para muitas opções de configuração comuns, como caches de credenciais e o comportamento de CRLF.

Ambos são aplicativos do tipo “evergreen” – as atualizações são baixadas e instaladas em segundo plano enquanto os aplicativos estão abertos. Isso inclui utilmente uma versão embutida (bundled) do Git, o que significa que você provavelmente não terá que se preocupar em atualizá-lo manualmente novamente. No Windows, o cliente inclui um atalho para abrir o PowerShell com o Posh-git, sobre o qual falaremos mais tarde neste capítulo.

O próximo passo é fornecer à ferramenta alguns repositórios para ela trabalhar. O cliente mostra para você uma lista contendo os repositórios aos quais você tem acesso no GitHub, e permite clonálos em um único passo. Se você já tiver um repositório local, basta arrastar seu diretório a partir do Finder ou Windows Explorer para dentro da janela do cliente do GitHub, e ele será incluído na lista de repositórios à esquerda.

Fluxo de Trabalho Recomendado

Uma vez que esteja instalado e configurado, você pode usar o cliente do GitHub para muitas das tarefas comuns do Git. O fluxo de trabalho proposto para essa ferramenta é algumas vezes chamado de “GitHub Flow.” Nós cobrimos isso com mais detalhes em O Fluxo do GitHub (GitHub Flow), mas a ideia geral é que (a) você vai estar realizando commits em um branch, e (b) você estará sincronizando com um repositório remoto bem regularmente.

O gerenciamento de branches é uma das áreas onde as duas ferramentas divergem. No macOS, há um botão no topo da janela para a criação de um novo branch:

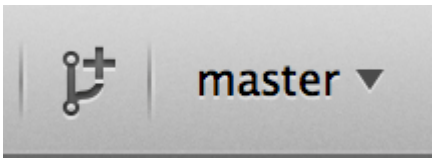


Figure 181. Botão de “Create Branch” no macOS

No Windows, isto é feito ao digitar o nome do novo branch no componente de alternância de branch (branch-switching widget):

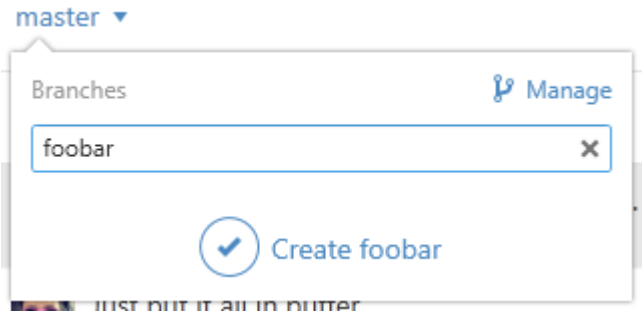


Figure 182. Criando um branch no Windows

Uma vez que o seu branch foi criado, a criação de novos commits é algo bastante simples. Faça algumas modificações no seu diretório de trabalho, e quando você voltar para a janela do cliente do GitHub, ela irá lhe mostrar quais arquivos sofreram modificações. Digite a sua mensagem do commit, selecione os arquivos que você queira incluir, e então clique no botão “Commit” (ctrl-enter ou ⌘-enter).

A principal maneira de você interagir com outros repositórios por meio da rede é através da função de “Sync”. O Git possui internamente operações separadas para empurrar (pushing), buscar (fetching), unificar (merging), e de rebasing, no entanto os clientes do GitHub transformam (collapse) todos estes procedimentos dentro de uma funcionalidade única que ocorre em múltiplos passos. Eis o que acontece quando você clica no botão de Sync (Sincronizar):

1. `git pull --rebase`. Se isso falhar devido a um conflito de merge, o passo recorre alternativamente ao `git pull --no-rebase`.
2. `git push`.

Esta é a sequência mais comum de comandos de rede ao se trabalhar usando esse estilo, e portanto, o ato de esmagá-los (squashing them) dessa forma num comando apenas fará com que poupe muito tempo.

Resumo

Estas ferramentas são muito bem adequadas para o fluxo de trabalho (workflow) ao qual elas foram projetadas. Os desenvolvedores e não-desenvolvedores conseguem estar de mesma forma colaborando em um projeto numa questão de apenas minutos, e diversas dentre as melhores práticas para esse tipo de fluxo de trabalho estão cozidas e preparadas (baked into) diretamente para dentro das ferramentas em si. Entretanto, se o seu fluxo de trabalho for diferente, ou se você desejar maior controle sobre a forma que se dá de como e quando que as operações de rede são feitas, recomendamos a você pela utilização de algum outro cliente de Git ou da mesma forma o uso por via da linha de comandos.

Outras GUIs

Há um número de outros clientes de Git na categoria gráfica, e eles permeiam pela gama indo desde as ferramentas especializadas, com propósito único, e indo a todo caminho que dispõe até aplicações que tentam expor de tudo que o Git puder fazer. O site oficial do Git apresenta a você

contida em seu dispor uma lista providenciada de forma sendo ela uma curada lista dos clientes mais populares no https://git-scm.com/downloads/guis. Uma lista mais compreensível, ampla e geral, está disposta de disponível lá no site da wiki do Git, pelo caminho no link no qual de https://archive.kernel.org/oldwiki/git.wiki.kernel.org/index.php/Interfaces,_frontends,_and_tools.html#Graphical_Interfaces.

Git no Visual Studio

O Visual Studio tem ferramentas para o Git embutidas diretamente na IDE, começando a partir do Visual Studio 2019 na versão 16.8.

As ferramentas dão suporte para as seguintes funcionalidades do Git:

- Criar ou clonar um repositório.
- Abrir e navegar no histórico de um repositório.
- Criar e realizar o checkout de branches e de tags.
- Fazer stash, preparar no stage, e criar commit das alterações.
- Fazer fetch, pull, push, ou realizar um sync (sincronizar) dos commits.
- Fazer merge e rebase de branches.
- Resolver conflitos de merges.
- Visualizar diffs.
- ... e mais!

Leia a documentação oficial para aprender mais.

Git no Visual Studio Code

O Visual Studio Code possui suporte integrado ao Git. Você precisa ter o Git versão 2.0.0 (ou mais recente) instalado.

Os principais recursos são:

- Ver o diff do arquivo que você está editando na margem do editor.
- A Barra de Status do Git (no canto inferior esquerdo) mostra o branch atual, indicadores de alterações pendentes e commits recebidos e enviados.
- Você pode realizar as operações mais comuns do Git no próprio editor:
  - Inicializar um repositório.
  - Clonar um repositório.
  - Criar branches e tags.
  - Preparar e fazer commit das alterações.
  - Fazer push/pull/sync com um branch remoto.
  - Resolver conflitos de merge.

    - Visualizar diffs.
- Com uma extensão, você também pode gerenciar Pull Requests do GitHub: https://marketplace.visualstudio.com/items?itemName=GitHub.vscode-pull-request-github.

A documentação oficial está disponível em https://code.visualstudio.com/docs/sourcecontrol/overview.

Git no IntelliJ / PyCharm / WebStorm / PhpStorm / RubyMine

As IDEs da JetBrains (tais como o IntelliJ IDEA, PyCharm, WebStorm, PhpStorm, RubyMine, dentre outros) acompanham (ship with) um plugin de Integração com o Git. Isso providencia uma visualização dedicada dentro da IDE para lidar trabalhando junto do Git e dos Pull Requests de GitHub.

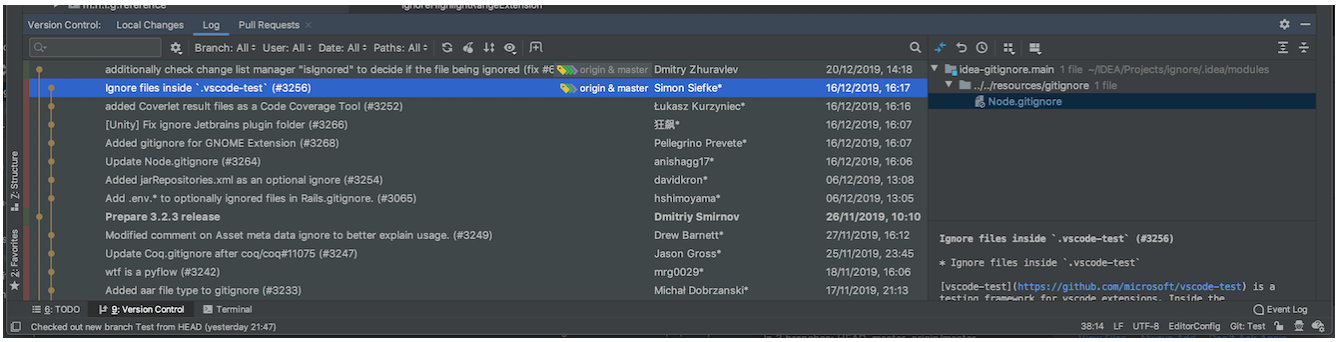


Figure 183. Versão de Controle da Janela de Ferramenta (ToolWindow) nas IDEs da JetBrains

A integração baseia-se em (relies on) dispor do cliente de linha de comandos em Git, requerendo por sua conta que se haja um devidamente instalado. A documentação na base de fonte oficial conta com a disponibilidade através de acessos ao https://www.jetbrains.com/help/idea/using-git-integration.html.

Git no Sublime Text

A partir da versão 3.2 em diante, o Sublime Text possui integração com o Git no editor.

Os recursos (features) são:

- A barra lateral exibirá o `git status` dos arquivos e diretórios através de um pequeno ícone/símbolo (badge/icon).
- Arquivos e pastas que estejam dentro de seu arquivo `.gitignore` serão mostrados esmaecidos na barra lateral.
- Na barra de status, você pode observar o seu branch atual em Git e ver a quantidade de alterações em modificações que você efetuou.
- Todas as mudanças a um arquivo constam agora num dispor visíveis se através dos marcadores da medianiz nela (gutter).
- Você pode usar parte das funcionalidades contidas no cliente em Git do Sublime Merge por intermédio e através de dentro do Sublime Text. Para isso, é um requisito que o Sublime Merge

esteja instalado. Acesse: https://www.sublimemerge.com/.

A documentação correspondente ao foco na oficialidade pelo Sublime Text poderá ser encontrada bem por aqui, no dispor nela ao contido em: https://www.sublimetext.com/docs/git_integration.html.

Git no Bash

Se você é um usuário do Bash, pode aproveitar alguns dos recursos do seu shell para tornar a sua experiência com o Git muito mais amigável. O Git na verdade é distribuído junto com plugins para vários shells, mas eles não vêm ativados por padrão.

Primeiro, você precisa conseguir uma cópia do arquivo de autocompletar a partir do código-fonte da versão (release) do Git que você está usando. Verifique a sua versão ao digitar `git version`, e então use `git checkout tags/vX.Y.Z`, onde `vX.Y.Z` deverá corresponder à versão do Git que você estiver usando. Copie o arquivo `contrib/completion/git-completion.bash` para um local de fácil acesso, como o seu diretório home (home directory), e adicione isto ao seu `.bashrc`:

```
. ~/git-completion.bash
```

Uma vez feito isto, mude o seu diretório para um repositório no Git e digite:

```
$ git chec<tab>
```

...e o Bash fará a ação de autocompletar para `git checkout`. Isso funciona para todos os subcomandos do Git, os parâmetros de linha de comando, e também origens remotas e nomes de refs, onde for apropriado.

Também é muito prático customizar o seu prompt para mostrar informações sobre o repositório Git do diretório atual. Isso pode ser tão simples ou complexo quanto você quiser, mas em geral existem poucas e essenciais (key) partes de informações que a maior parte das pessoas gostaria de dispor, como o branch atual, e o status do diretório de trabalho. Para adicionar isto ao seu prompt, basta copiar o arquivo `contrib/completion/git-prompt.sh` do repositório fonte do Git para o seu diretório home, e então adicionar algo assim no seu `.bashrc`:

```
. ~/git-prompt.sh
export GIT_PS1_SHOWDIRTYSTATE=1
export PS1='\w$(__git_ps1 " (%s)")\$ '
```

O `\w` significa para imprimir (print) o diretório atual de trabalho, e o `\$` imprimirá a parte do `$` ali no prompt, e o `__git_ps1 " (%s)"` chamará a função provida pelo `git-prompt.sh` com o argumento ali na sua formatação. Agora o seu prompt de bash parecerá assim sempre que você estiver em qualquer lugar dentro de um projeto controlado pelo Git:



Figure 184. Prompt de `bash` customizado

Ambos destes scripts já vêm com documentações muito úteis; dê uma olhada sobre aos e sob nos conteúdos lá para o `git-completion.bash` e ao `git-prompt.sh` visando angariar um pouco mais de informações.

Git no Zsh

O Zsh também é distribuído com uma biblioteca de autocompletar (tab-completion) para o Git. Para usá-la, basta executar `autoload -Uz compinit && compinit` no seu `.zshrc`. A interface do Zsh é um pouco mais poderosa que a do Bash:

```
$ git che<tab>
check-attr        -- display gitattributes information
check-ref-format  -- ensure that a reference name is well formed
checkout          -- checkout branch or paths to working tree
checkout-index    -- copy files from index to working directory
cherry            -- find commits not merged upstream
cherry-pick       -- apply changes introduced by some existing commits
```

As autocompletações ambíguas não são apenas listadas; elas têm descrições úteis, e você pode navegar graficamente pela lista apertando a tecla tab repetidamente. Isso funciona com comandos do Git, seus argumentos, e nomes de coisas dentro do repositório (como refs e remotes), bem como nomes de arquivos e todas as outras coisas que o Zsh sabe autocompletar.

O Zsh é distribuído com um framework para obter informações de sistemas de controle de versão, chamado `vcs_info`. Para incluir o nome do branch no prompt do lado direito, adicione estas linhas ao seu arquivo `~/.zshrc`:

```
autoload -Uz vcs_info
precmd_vcs_info() { vcs_info }
precmd_functions+=( precmd_vcs_info )
setopt prompt_subst
RPROMPT='${vcs_info_msg_0_}'
# PROMPT='${vcs_info_msg_0_}%# '
zstyle ':vcs_info:git:*' formats '%b'
```

Isso resulta na exibição do branch atual no lado direito da janela do terminal, sempre que o seu shell estiver dentro de um repositório Git. O lado esquerdo também é suportado, obviamente; apenas descomente a atribuição a `PROMPT`. Fica um pouco parecido com isto:



Figure 185. Prompt de `zsh` customizado

Para mais informações sobre o `vcs_info`, confira a sua documentação na página de manual do `zshcontrib(1)`, ou online em https://zsh.sourceforge.io/Doc/Release/User-Contributions.html#Version-Control-Information.

Em vez do `vcs_info`, você pode preferir o script de customização de prompt que é distribuído junto com o Git, chamado `git-prompt.sh`; veja https://github.com/git/git/blob/master/contrib/completion/git-prompt.sh para detalhes. O `git-prompt.sh` é compatível tanto com o Bash quanto com o Zsh.

O Zsh é poderoso o suficiente para que existam frameworks inteiros dedicados a torná-lo melhor. Um deles é chamado "oh-my-zsh", e pode ser encontrado em https://github.com/ohmyzsh/ohmyzsh. O sistema de plugins do oh-my-zsh vem com um autocompletar poderoso para o Git, e ele possui uma variedade de "temas" para o prompt, muitos dos quais exibem dados de controle de versão. Um exemplo de um tema do oh-my-zsh é apenas um exemplo do que pode ser feito com esse sistema.

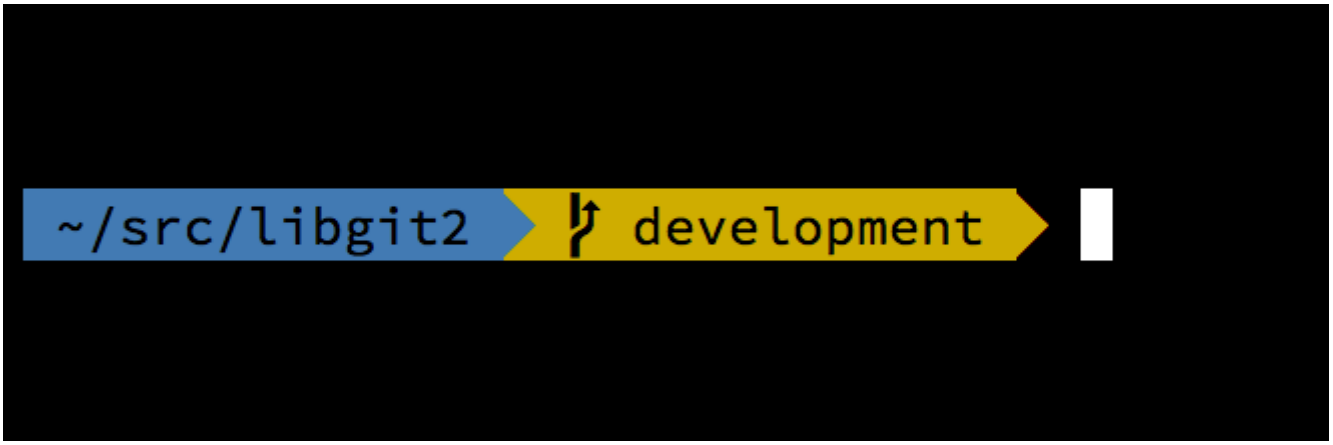


Figure 186. Um exemplo de um tema do oh-my-zsh

Git no PowerShell

O terminal de linha de comando legado no Windows (`cmd.exe`) não é realmente capaz de prover uma experiência customizada com o Git, mas se você estiver usando o PowerShell, você está com sorte. Isto também funciona se você estiver rodando o PowerShell Core no Linux ou macOS. Um pacote chamado posh-git (https://github.com/dahlbyk/posh-git) fornece facilidades poderosas de autocompletar com tabulação, bem como um prompt melhorado para te ajudar a ficar por dentro do status do seu repositório. Fica parecido com isto:



Figure 187. PowerShell com o Posh-git

Instalação

Pré-requisitos (apenas Windows)

Antes que você consiga rodar scripts do PowerShell na sua máquina, você precisa definir a sua `ExecutionPolicy` local para `RemoteSigned` (basicamente, qualquer coisa exceto `Undefined` e `Restricted`). Se você escolher `AllSigned` ao invés de `RemoteSigned`, também os scripts locais (os seus próprios) precisarão ser assinados digitalmente para poderem ser executados. Com o `RemoteSigned`, apenas os scripts que tenham o `ZoneIdentifier` definido como `Internet` (foram baixados da web) precisam ser assinados, outros não. Se você for um administrador e quiser definir isto para todos os usuários daquela máquina, use `-Scope LocalMachine`. Se você for um usuário normal, sem direitos administrativos, pode usar `-Scope CurrentUser` para definir isto apenas para você.

Mais sobre Escopos (Scopes) do PowerShell: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_scopes.

Mais sobre Política de Execução (ExecutionPolicy) do PowerShell: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/set-executionpolicy.

Para definir o valor de `ExecutionPolicy` para `RemoteSigned` em relação a todos os usuários, utilize o seguinte comando:

```
> Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy RemoteSigned -Force
```

PowerShell Gallery

Se você tiver no mínimo o PowerShell 5 ou o PowerShell 4 com o PackageManagement instalado, você pode usar o gerenciador de pacotes para instalar o posh-git para você.

Mais informações sobre a Galeria do PowerShell (PowerShell Gallery): https://learn.microsoft.com/en-us/powershell/scripting/gallery/overview.

```
> Install-Module posh-git -Scope CurrentUser -Force
> Install-Module posh-git -Scope CurrentUser -AllowPrerelease -Force # Newer beta
version with PowerShell Core support
```

Se você quiser instalar o posh-git para todos os usuários, use `-Scope AllUsers` em vez do anterior, e execute o comando a partir de um console do PowerShell elevado (com permissões de administrador). Se o segundo comando falhar com um erro como `Module 'PowerShellGet' was not installed by using Install-Module`, você precisará rodar um outro comando antes:

```
> Install-Module PowerShellGet -Force -SkipPublisherCheck
```

Depois você pode voltar e tentar novamente. Isto acontece porque os módulos que são enviados (ship with) junto ao Windows PowerShell são assinados com um certificado de publicação diferente.

Atualizar o Prompt do PowerShell

Para incluir informações do Git no seu prompt, o módulo posh-git precisa ser importado. Para ter o posh-git importado sempre que o PowerShell for iniciado, execute o comando `Add-PoshGitToProfile`, o qual vai adicionar a declaração de importação no seu script `$profile`. Este script é executado toda vez que você abre um novo console do PowerShell. Tenha em mente que existem múltiplos scripts do tipo `$profile`. Por exemplo, um para o console e um outro separado para o ISE.

```
> Import-Module posh-git
> Add-PoshGitToProfile -AllHosts
```

A partir do código-fonte (From Source)

Basta baixar um release do posh-git em https://github.com/dahlbyk/posh-git/releases, e descompactá-lo. Então importe o módulo usando o caminho completo (full path) para o arquivo `posh-git.psd1`:

```
> Import-Module <path-to-uncompress-folder>\src\posh-git.psd1
> Add-PoshGitToProfile -AllHosts
```

Isto irá adicionar a linha correta ao seu arquivo `profile.ps1`, e o posh-git estará ativo na próxima vez que você abrir o PowerShell.

Para uma descrição sobre as informações do sumário de status do Git (Git status summary) exibidas no prompt, veja: https://github.com/dahlbyk/posh-git/blob/master/README.md#git-status-summary-information Para mais detalhes em como customizar o seu prompt do posh-git, veja: https://github.com/dahlbyk/posh-git/blob/master/README.md#customization-variables.

Resumo

Você aprendeu a aproveitar o poder do Git usando as ferramentas que você usa durante o trabalho diário e também a acessar repositórios Git a partir dos seus próprios programas.

Appendix B: Incorporando o Git nos seus Aplicativos

Se o seu aplicativo é feito para desenvolvedores, as chances são altas de que ele poderia se beneficiar de alguma integração com o controle de versão de fonte. Mesmo aplicativos que não sejam para desenvolvedores, tais como os editores de documentos, podem potencialmente se beneficiar de funções em controle de versão, e o modelo do Git funciona muito bem em relação a muitos cenários distintos.

Se você precisar de integrar o Git junto com seu aplicativo, você tem basicamente duas opções: executar um shell e chamar pelo programa do `git` a partir da sua linha de comandos, ou ainda, no caso incorporar uma biblioteca do Git dentro de seu próprio aplicativo. Neste ponto abordaremos sobre integração com a linha de comandos e em diversas das bibliotecas incorporáveis para o Git que dispõem da maior popularidade.

Linha de Comando do Git

Uma opção é iniciar um processo de shell e usar a ferramenta de linha de comando do Git para realizar o trabalho. Essa abordagem tem a vantagem de ser canônica e oferecer suporte a todos os recursos do Git. Ela também é bastante fácil, pois a maioria dos ambientes de execução oferece um mecanismo relativamente simples para iniciar um processo com argumentos de linha de comando. No entanto, essa abordagem tem algumas desvantagens.

Uma delas é que toda a saída está em texto simples. Isso significa que você terá de interpretar o formato de saída do Git, que muda ocasionalmente, para ler informações de andamento e resultado, o que pode ser ineficiente e propenso a erros.

Outra é a falta de recuperação de erros. Se um repositório estiver corrompido de alguma forma ou o usuário tiver um valor de configuração malformado, o Git simplesmente se recusará a executar muitas operações.

Outra ainda é o gerenciamento de processos. O Git exige que você mantenha um ambiente de shell em um processo separado, o que pode acrescentar uma complexidade indesejada. Coordenar muitos desses processos (especialmente quando vários deles podem acessar o mesmo repositório) pode ser um grande desafio.

Libgit2

Outra opção à sua disposição é usar a Libgit2. A Libgit2 é uma implementação do Git sem dependências, com foco em oferecer uma boa API para uso dentro de outros programas. Você pode encontrá-la em https://libgit2.org.

Primeiro, vejamos como é a API em C. Eis um tour relâmpago:

```
// Open a repository
git_repository *repo;
```

```
int error = git_repository_open(&repo, "/path/to/repository");

// Dereference HEAD to a commit
git_object *head_commit;
error = git_revparse_single(&head_commit, repo, "HEAD^{commit}");
git_commit *commit = (git_commit*)head_commit;

// Print some of the commit's properties
printf("%s", git_commit_message(commit));
const git_signature *author = git_commit_author(commit);
printf("%s <%s>\n", author->name, author->email);
const git_oid *tree_id = git_commit_tree_id(commit);

// Cleanup
git_commit_free(commit);
git_repository_free(repo);
```

As primeiras linhas abrem um repositório Git. O tipo `git_repository` representa um handle para um repositório com cache em memória. Esse é o método mais simples, usado quando você conhece o caminho exato para o diretório de trabalho ou a pasta `.git` de um repositório. Há também `git_repository_open_ext`, que inclui opções de busca; `git_clone` e funções relacionadas, para criar um clone local de um repositório remoto; e `git_repository_init`, para criar um repositório inteiramente novo.

O segundo trecho de código usa a sintaxe rev-parse (consulte Referências de Branch para saber mais) para obter o commit para o qual HEAD acaba apontando. O tipo retornado é um ponteiro `git_object`, que representa algo existente no banco de dados de objetos Git de um repositório. Na verdade, `git_object` é um tipo "parent" de vários tipos diferentes de objetos; o layout de memória de cada tipo "child" é igual ao de `git_object`, portanto você pode convertê-lo com segurança para o tipo correto. Neste caso, `git_object_type(commit)` retornaria `GIT_OBJ_COMMIT`, portanto é seguro convertê-lo para um ponteiro `git_commit`.

O próximo pedaço de código mostra como acessar as propriedades do commit. A última linha usa o tipo `git_oid`, que é a representação de um hash SHA-1 na Libgit2.

A partir deste exemplo, alguns padrões começam a surgir:

- Se você declarar um ponteiro e passar uma referência a ele para uma chamada da Libgit2, essa chamada provavelmente retornará um código de erro inteiro. O valor `0` indica sucesso; qualquer valor menor indica erro.
- Se a Libgit2 preencher um ponteiro para você, será sua responsabilidade liberá-lo.
- Se a Libgit2 retornar um ponteiro `const` de uma chamada, você não precisará liberá-lo, mas ele se tornará inválido quando o objeto ao qual pertence for liberado.
- Escrever em C é um pouco doloroso.

Esse último ponto significa que é pouco provável que você escreva em C ao usar a Libgit2. Felizmente, há vários bindings específicos de linguagem que facilitam bastante o trabalho com repositórios Git a partir da sua linguagem e do seu ambiente. Vejamos o exemplo anterior escrito

com os bindings Ruby da Libgit2, chamados Rugged e disponíveis em https://github.com/libgit2/rugged.

```
repo = Rugged::Repository.new('path/to/repository')
commit = repo.head.target
puts commit.message
puts "#{commit.author[:name]} <#{commit.author[:email]}>"
tree = commit.tree
```

Como você pode ver, o código é muito menos carregado. Primeiro, o Rugged usa exceções; ele pode lançar erros como `ConfigError` ou `ObjectError` para sinalizar condições de erro. Segundo, não há liberação explícita de recursos, pois o Ruby possui coleta de lixo. Vejamos um exemplo um pouco mais complexo: criar um commit do zero

```
blob_id = repo.write("Blob contents", :blob) ①

index = repo.index
index.read_tree(repo.head.target.tree)
index.add(:path => 'newfile.txt', :oid => blob_id) ②

sig = {
    :email => "bob@example.com",
    :name => "Bob User",
    :time => Time.now,
}

commit_id = Rugged::Commit.create(repo,
    :tree => index.write_tree(repo), ③
    :author => sig,
    :committer => sig, ④
    :message => "Add newfile.txt", ⑤
    :parents => repo.empty? ? [] : [ repo.head.target ].compact, ⑥
    :update_ref => 'HEAD', ⑦
)
commit = repo.lookup(commit_id) ⑧
```

① Crie um novo blob que contenha o conteúdo de um novo arquivo.

② Preencha o index com a árvore do commit apontado por head e adicione o novo arquivo no caminho `newfile.txt`.

③ Isso cria uma nova árvore no ODB e a usa no novo commit.

④ Usamos a mesma assinatura nos campos de autor e committer.

⑤ A mensagem de commit.

⑥ Ao criar um commit, você precisa especificar os pais do novo commit. Isso usa a ponta de HEAD como o único pai.

⑦ O Rugged (e a Libgit2) pode, opcionalmente, atualizar uma referência ao criar um commit.

⑧ O valor de retorno é o hash SHA-1 de um novo objeto de commit, o qual você pode então usar para obter um objeto `Commit`.

O código Ruby é simples e limpo, mas, como a Libgit2 faz o trabalho pesado, ele também será executado com bastante rapidez. Se você não programa em Ruby, abordamos outros bindings em Outros Bindings.

Funcionalidades Avançadas

O Libgit2 tem algumas capacidades que estão fora do escopo do núcleo do Git. Um exemplo é a plugabilidade: o Libgit2 permite que você providencie "backends" customizados para diversos tipos de operações, de modo que você possa armazenar as coisas de uma forma diferente daquela que o Git padrão faz. O Libgit2 permite backends customizados para a configuração, o armazenamento de referências, e a base de dados de objetos, entre outras coisas.

Vamos dar uma olhada em como isso funciona. O código abaixo foi emprestado a partir do conjunto de exemplos para backend providos pelo próprio time de atuantes no Libgit2 (os quais ali constam que podem ser achados em https://github.com/libgit2/libgit2-backends). Aqui está como um backend customizado para o banco de dados de objetos é configurado:

```
git_odb *odb;
int error = git_odb_new(&odb); ①

git_odb_backend *my_backend;
error = git_odb_backend_mine(&my_backend, /*…*/); ②

error = git_odb_add_backend(odb, my_backend, 1); ③

git_repository *repo;
error = git_repository_open(&repo, "some-path");
error = git_repository_set_odb(repo, odb); ④
```

Note que os erros são capturados, mas não tratados. Esperamos que o seu código seja melhor que o nosso.

① Inicializa um "frontend," vazio do banco de dados de objetos (ODB), que atuará como contêiner dos "backends" responsáveis pelo trabalho real.

② Inicializa um backend de ODB customizado.

③ Adiciona o backend ao frontend.

④ Abre um repositório, e o configura para usar o nosso ODB para procurar por objetos.

But what is this `git_odb_backend_mine` thing? Bem, esse é o construtor da sua própria implementação de ODB, e você pode fazer o que quiser nele, desde que preencha corretamente a estrutura `git_odb_backend`. Esta é uma possível implementação:

```
typedef struct {
    git_odb_backend parent;
```

```
    // Some other stuff
    void *custom_context;
} my_backend_struct;

int git_odb_backend_mine(git_odb_backend **backend_out, /*…*/)
{
    my_backend_struct *backend;

    backend = calloc(1, sizeof (my_backend_struct));

    backend->custom_context = …;

    backend->parent.read = &my_backend__read;
    backend->parent.read_prefix = &my_backend__read_prefix;
    backend->parent.read_header = &my_backend__read_header;
    // …

    *backend_out = (git_odb_backend *) backend;

    return GIT_SUCCESS;
}
```

A restrição mais sutil é que o primeiro membro de `my_backend_struct` deve ser uma estrutura `git_odb_backend`; isso garante que a disposição da memória seja a esperada pelo código da Libgit2. O restante é arbitrário; essa estrutura pode ser tão grande ou pequena quanto necessário.

A função de inicialização aloca um pouco de memória para a estrutura, configura o contexto customizado, e então preenche os membros da estrutura `parent` que ela suporta. Dê uma olhada no arquivo `include/git2/sys/odb_backend.h` no código-fonte da Libgit2 para um conjunto completo de assinaturas de chamadas; seu caso de uso em particular ajudará a determinar quais dessas você irá querer suportar.

Outros Bindings

O Libgit2 possui bindings para muitas linguagens. Aqui mostramos um pequeno exemplo usando alguns dos pacotes de bindings mais completos na data em que isto é escrito; bibliotecas existem para muitas outras linguagens, incluindo C++, Go, Node.js, Erlang, e a JVM, todas em vários estágios de maturidade. A coleção oficial de bindings pode ser encontrada navegando pelos repositórios em https://github.com/libgit2. O código que nós iremos escrever vai retornar a mensagem de commit a partir do commit que eventualmente seja apontado por HEAD (mais ou menos como no `git log -1`).

LibGit2Sharp

Se você estiver escrevendo um aplicativo de .NET ou Mono, LibGit2Sharp (https://github.com/libgit2/libgit2sharp) é o que você está procurando. Os bindings são escritos em C#, e foi tomado um grande cuidado para envolver (wrap) as chamadas brutas (raw) do Libgit2 com APIs de CLR com uma pegada bem nativa (native-feeling). Aqui temos como nosso programa de exemplo se parece:

```
new Repository(@"C:\path\to\repo").Head.Tip.Message;
```

Para aplicativos de Windows no desktop, existe até mesmo um pacote no NuGet que te ajudará a iniciar nisto tudo muito rapidamente.

objective-git

Se o seu aplicativo está rodando numa plataforma da Apple, é muito provável que você esteja usando o Objective-C enquanto sendo sua linguagem para implementação. Objective-Git (https://github.com/libgit2/objective-git) é o nome para os bindings da Libgit2 para este respectivo ambiente. O programa de exemplo se parece com isto:

```
GTRepository *repo =
    [[GTRepository alloc] initWithURL:[NSURL fileURLWithPath: @"/path/to/repo"]
error:NULL];
NSString *msg = [[[repo headReferenceWithError:NULL] resolvedTarget] message];
```

O Objective-git é totalmente interoperável com Swift, portanto não se preocupe se você já deixou o Objective-C para trás.

pygit2

Os bindings Python da Libgit2 são chamados Pygit2 e estão disponíveis em https://www.pygit2.org. Nosso programa de exemplo:

```
pygit2.Repository("/path/to/repo") # open repository
    .head                          # get the current branch
    .peel(pygit2.Commit)           # walk down to the commit
    .message                       # read the message
```

Leituras Adicionais

Naturalmente, uma abordagem completa dos recursos da Libgit2 está fora do escopo deste livro. Se quiser mais informações sobre a própria Libgit2, há documentação da API em https://libgit2.github.com/libgit2 e um conjunto de guias em https://libgit2.github.com/docs. Para os outros bindings, consulte o README e os testes incluídos; muitas vezes há pequenos tutoriais e indicações de leituras adicionais nesses arquivos.

JGit

Se você quiser usar o Git dentro de um programa Java, há uma biblioteca Git completa chamada JGit. O JGit é uma implementação relativamente completa do Git, escrita nativamente em Java e amplamente usada pela comunidade Java. O projeto JGit está sob o guarda-chuva do Eclipse, e sua página pode ser encontrada em https://projects.eclipse.org/projects/technology.jgit.

Configuração

Há várias maneiras de integrar o JGit ao seu projeto e começar a escrever código com ele. Provavelmente, a mais simples é usar o Maven – a integração é feita adicionando o trecho a seguir à tag `<dependencies>` do seu arquivo `pom.xml`:

```
<dependency>
    <groupId>org.eclipse.jgit</groupId>
    <artifactId>org.eclipse.jgit</artifactId>
    <version>3.5.0.201409260305-r</version>
</dependency>
```

A `version` provavelmente já terá avançado quando você ler isto; consulte https://mvnrepository.com/artifact/org.eclipse.jgit/org.eclipse.jgit para obter informações atualizadas sobre o repositório. Depois dessa etapa, o Maven obterá e usará automaticamente as bibliotecas do JGit de que você precisa.

Se preferir gerenciar as dependências binárias por conta própria, há binários pré-compilados do JGit disponíveis em https://projects.eclipse.org/projects/technology.jgit/downloads. Você pode incorporá-los ao projeto executando um comando como este:

```
javac -cp .:org.eclipse.jgit-3.5.0.201409260305-r.jar App.java
java -cp .:org.eclipse.jgit-3.5.0.201409260305-r.jar App
```

Encanamento (Plumbing)

O JGit tem dois níveis básicos de API: encanamento (plumbing) e porcelana (porcelain). Essa terminologia vem do próprio Git, e o JGit se divide aproximadamente nos mesmos tipos de área: as APIs de porcelana (porcelain) são um front-end amigável para ações comuns no nível do usuário (o tipo de coisa para a qual um usuário comum utilizaria a ferramenta de linha de comando do Git), enquanto as APIs de encanamento (plumbing) permitem interagir diretamente com objetos de baixo nível do repositório.

O ponto de partida para a maioria das sessões com o JGit é a classe `Repository`, e a primeira coisa a fazer é criar uma instância dela. Para um repositório baseado em sistema de arquivos (sim, o JGit permite outros modelos de armazenamento), isso é feito com `FileRepositoryBuilder`:

```
// Create a new repository
Repository newlyCreatedRepo = FileRepositoryBuilder.create(
    new File("/tmp/new_repo/.git"));
newlyCreatedRepo.create();

// Open an existing repository
Repository existingRepo = new FileRepositoryBuilder()
    .setGitDir(new File("my_repo/.git"))
    .build();
```

O builder oferece uma API fluente para fornecer tudo de que precisa para encontrar um repositório Git, independentemente de o programa saber exatamente onde ele está. Ele pode usar variáveis de ambiente (`.readEnvironment()`), partir de um local no diretório de trabalho e pesquisar (`.setWorkTree(…).findGitDir()`), ou simplesmente abrir um diretório `.git` conhecido, como acima.

Depois de obter uma instância de `Repository`, você pode fazer todo tipo de coisa com ela. Aqui está uma pequena amostra:

```
// Get a reference
Ref master = repo.getRef("master");

// Get the object the reference points to
ObjectId masterTip = master.getObjectId();

// Rev-parse
ObjectId obj = repo.resolve("HEAD^{tree}");

// Load raw object contents
ObjectLoader loader = repo.open(masterTip);
loader.copyTo(System.out);

// Create a branch
RefUpdate createBranch1 = repo.updateRef("refs/heads/branch1");
createBranch1.setNewObjectId(masterTip);
createBranch1.update();

// Delete a branch
RefUpdate deleteBranch1 = repo.updateRef("refs/heads/branch1");
deleteBranch1.setForceUpdate(true);
deleteBranch1.delete();

// Config
Config cfg = repo.getConfig();
String name = cfg.getString("user", null, "name");
```

Há bastante coisa acontecendo aqui; vamos examinar uma seção de cada vez.

A primeira linha obtém um ponteiro para a referência `master`. O JGit obtém automaticamente a referência `master` *real*, que reside em `refs/heads/master`, e retorna um objeto que permite consultar informações sobre a referência. Você pode obter o nome (`.getName()`), bem como o objeto de destino de uma referência direta (`.getObjectId()`) ou a referência apontada por uma referência simbólica (`.getTarget()`). Os objetos Ref também são usados para representar referências e objetos de tag, portanto você pode verificar se a tag está “peeled,”, o que significa que ela aponta para o destino final de uma sequência (possivelmente longa) de objetos de tag.

A segunda linha obtém o destino da referência `master`, que é retornado como uma instância de ObjectId. ObjectId representa o hash SHA-1 de um objeto, que pode existir ou não no banco de dados de objetos do Git. A terceira linha é semelhante, mas mostra como o JGit lida com a sintaxe rev-parse (para saber mais, consulte Referências de Branch); você pode passar qualquer

especificador de objeto que o Git reconheça, e o JGit retornará um ObjectId válido para esse objeto ou `null`.

As duas linhas seguintes mostram como carregar o conteúdo bruto de um objeto. Neste exemplo, chamamos `ObjectLoader.copyTo()` para transmitir o conteúdo do objeto diretamente para stdout, mas ObjectLoader também tem métodos para ler o tipo e o tamanho de um objeto, além de retorná-lo como um array de bytes. Para objetos grandes (nos quais `.isLarge()` retorna `true`), você pode chamar `.openStream()` para obter um objeto semelhante a InputStream, capaz de ler os dados brutos do objeto sem carregá-los todos na memória de uma só vez.

As linhas seguintes mostram o que é necessário para criar um novo branch. Criamos uma instância de RefUpdate, configuramos alguns parâmetros e chamamos `.update()` para efetuar a alteração. Logo depois vem o código que exclui esse mesmo branch. Observe que `.setForceUpdate(true)` é necessário para isso funcionar; caso contrário, a chamada `.delete()` retornará `REJECTED` e nada acontecerá.

O último exemplo mostra como obter o valor de `user.name` dos arquivos de configuração do Git. Essa instância de Config usa o repositório que abrimos anteriormente para a configuração local, mas também detecta automaticamente os arquivos de configuração global e de sistema e lê seus valores.

Esta é apenas uma pequena amostra da API de encanamento completa; há muitos outros métodos e classes disponíveis. Também não mostramos aqui como o JGit trata erros: por meio de exceções. Às vezes, as APIs do JGit lançam exceções Java padrão (como `IOException`), mas também há vários tipos de exceção específicos do JGit (como `NoRemoteRepositoryException`, `CorruptObjectException` e `NoMergeBaseException`).

Porcelana (Porcelain)

As APIs de encanamento (plumbing) são bastante completas, mas pode ser trabalhoso combiná-las para alcançar objetivos comuns, como adicionar um arquivo ao index ou criar um novo commit. Para facilitar essas tarefas, o JGit oferece um conjunto de APIs de nível mais alto, cujo ponto de entrada é a classe `Git`:

```
Repository repo;
// construct repo...
Git git = new Git(repo);
```

A classe Git oferece um bom conjunto de métodos de alto nível no estilo *builder*, que podem ser usados para construir comportamentos bastante complexos. Vejamos um exemplo — fazer algo semelhante a `git ls-remote`:

```
CredentialsProvider cp = new UsernamePasswordCredentialsProvider("username",
"p4ssw0rd");
Collection<Ref> remoteRefs = git.lsRemote()
    .setCredentialsProvider(cp)
    .setRemote("origin")
    .setTags(true)
    .setHeads(false)
```

```
    .call();
for (Ref ref : remoteRefs) {
    System.out.println(ref.getName() + " -> " + ref.getObjectId().name());
}
```

Esse é um padrão comum da classe Git; os métodos retornam um objeto de comando que permite encadear chamadas de método para definir parâmetros, e a execução ocorre quando você chama `.call()`. Neste caso, estamos solicitando ao remote `origin` as tags, mas não os heads. Observe também o uso de um objeto `CredentialsProvider` para autenticação.

Muitos outros comandos estão disponíveis por meio da classe Git, incluindo, entre outros, `add`, `blame`, `commit`, `clean`, `push`, `rebase`, `revert` e `reset`.

Leituras Adicionais

Esta é apenas uma pequena amostra de tudo o que o JGit é capaz de fazer. Se tiver interesse e quiser saber mais, veja onde procurar informações e inspiração:

- A documentação oficial da API do JGit pode ser encontrada em https://help.eclipse.org/latest/topic/org.eclipse.egit.doc/help/JGit/User_Guide/User-Guide.html. Ela está no formato Javadoc padrão, portanto sua IDE preferida para JVM também poderá instalá-la localmente.
- O JGit Cookbook, em https://github.com/centic9/jgit-cookbook, contém muitos exemplos de como realizar tarefas específicas com o JGit.

go-git

Caso queira integrar o Git a um serviço escrito em Golang, também há uma implementação de biblioteca inteiramente em Go. Essa implementação não possui dependências nativas e, portanto, não está sujeita a erros de gerenciamento manual de memória. Ela também pode ser analisada normalmente pelas ferramentas padrão de desempenho do Golang, como os profilers de CPU e memória e o detector de condições de corrida.

O go-git se concentra em extensibilidade e compatibilidade e oferece suporte à maioria das APIs de encanamento, conforme documentado em https://github.com/go-git/go-git/blob/master/COMPATIBILITY.md.

Eis um exemplo básico de uso das APIs Go:

```
import "github.com/go-git/go-git/v5"

r, err := git.PlainClone("/tmp/foo", false, &git.CloneOptions{
    URL:      "https://github.com/go-git/go-git",
    Progress: os.Stdout,
})
```

Assim que tiver uma instância de `Repository`, você poderá acessar informações e fazer alterações nela:

```
// retrieves the branch pointed by HEAD
ref, err := r.Head()

// get the commit object, pointed by ref
commit, err := r.CommitObject(ref.Hash())

// retrieves the commit history
history, err := commit.History()

// iterates over the commits and print each
for _, c := range history {
    fmt.Println(c)
}
```

Funcionalidades Avançadas

O go-git tem alguns recursos avançados dignos de nota; um deles é um sistema de armazenamento plugável, semelhante aos backends da Libgit2. A implementação padrão usa armazenamento em memória, que é muito rápido.

```
r, err := git.Clone(memory.NewStorage(), nil, &git.CloneOptions{
    URL: "https://github.com/go-git/go-git",
})
```

O armazenamento plugável oferece muitas opções interessantes. Por exemplo, https://github.com/go-git/go-git/tree/master/_examples/storage permite armazenar referências, objetos e configurações em um banco de dados Aerospike.

Outro recurso é uma abstração flexível do sistema de arquivos. Com https://pkg.go.dev/github.com/go-git/go-billy/v5?tab=doc#Filesystem, é fácil armazenar todos os arquivos de maneiras diferentes; por exemplo, empacotando-os em um único arquivo no disco ou mantendo-os todos na memória.

Outro caso de uso avançado envolve um cliente HTTP altamente configurável, como o encontrado em https://github.com/go-git/go-git/blob/master/_examples/custom_http/main.go.

```
customClient := &http.Client{
    Transport: &http.Transport{ // accept any certificate (might be useful for
testing)
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    },
    Timeout: 15 * time.Second,  // 15 second timeout
        CheckRedirect: func(req *http.Request, via []*http.Request) error {
        return http.ErrUseLastResponse // don't follow redirect
    },
}

// Override http(s) default protocol to use our custom client
```

```
client.InstallProtocol("https", githttp.NewClient(customClient))

// Clone repository using the new client if the protocol is https://
r, err := git.Clone(memory.NewStorage(), nil, &git.CloneOptions{URL: url})
```

Leituras Adicionais

Uma abordagem completa dos recursos do go-git está fora do escopo deste livro. Se quiser mais informações sobre o go-git, há documentação da API em https://pkg.go.dev/github.com/go-git/go-git/v5 e um conjunto de exemplos de uso em https://github.com/go-git/go-git/tree/master/_examples.

Dulwich

Há também uma implementação do Git escrita inteiramente em Python: Dulwich. O projeto está hospedado em https://www.dulwich.io/. Seu objetivo é oferecer uma interface para repositórios Git (tanto locais quanto remotos) que não invoque o Git diretamente, mas use apenas Python. No entanto, ela dispõe de extensões C opcionais que melhoram significativamente o desempenho.

O Dulwich segue o design do Git e separa a API em dois níveis básicos: encanamento (plumbing) e porcelana (porcelain).

Eis um exemplo de uso da API de baixo nível para acessar a mensagem do commit mais recente:

```
from dulwich.repo import Repo
r = Repo('.')
r.head()
# '57fbe010446356833a6ad1600059d80b1e731e15'

c = r[r.head()]
c
# <Commit 015fc1267258458901a94d228e39f0a378370466>

c.message
# 'Add note about encoding.\n'
```

Para exibir um log de commits usando a API de porcelana de alto nível, você pode usar:

```
from dulwich import porcelain
porcelain.log('.', max_entries=1)

#commit: 57fbe010446356833a6ad1600059d80b1e731e15
#Author: Jelmer Vernooĳ <jelmer@jelmer.uk>
#Date:   Sat Apr 29 2017 23:57:34 +0000
```

Leituras Adicionais

A documentação da API, o tutorial e muitos exemplos de como realizar tarefas específicas com o Dulwich estão disponíveis no site oficial https://www.dulwich.io.

Appendix C: Comandos do Git

Ao longo do livro, apresentamos dezenas de comandos do Git e nos esforçamos para introduzi-los dentro de uma espécie de narrativa, acrescentando novos comandos à história aos poucos. No entanto, isso deixa os exemplos de uso dos comandos um tanto dispersos por todo o livro.

Neste apêndice, percorreremos todos os comandos do Git abordados ao longo do livro, agrupados aproximadamente de acordo com sua finalidade. Falaremos, em linhas gerais, sobre o que cada comando faz e indicaremos em que parte do livro você pode encontrar exemplos de seu uso.



Você pode abreviar opções longas. Por exemplo, você pode digitar `git commit --a`, que funciona como se tivesse digitado `git commit --amend`. Isso só funciona quando as letras após `--` identificam uma única opção. Use a opção completa ao escrever scripts.

Configuração e Ajustes

Há dois comandos usados com muita frequência, desde as primeiras invocações do Git até ajustes e consultas comuns do dia a dia: `config` e `help`.

git config

O Git tem uma forma padrão de realizar centenas de tarefas. Para muitas delas, você pode instruir o Git a adotar outra forma como padrão ou definir suas preferências. Isso abrange desde informar seu nome ao Git até preferências específicas de cores no terminal ou do editor que você usa. Esse comando lê e grava em vários arquivos, permitindo definir valores globalmente ou para repositórios específicos.

O comando `git config` foi usado em quase todos os capítulos do livro.

Em Configuração inicial do Git, nós o usamos para especificar nosso nome, endereço de e-mail e editor preferido antes mesmo de começar a usar o Git.

Em Aliases (Apelidos) no Git, mostramos como usá-lo para criar atalhos de comandos que se expandem em longas sequências de opções, evitando que você precise digitá-las todas as vezes.

Em Rebase (Rebasing), nós o usamos para tornar `--rebase` o padrão ao executar `git pull`.

Em Armazenamento de Credenciais, nós o usamos para configurar um armazenamento padrão para suas senhas HTTP.

Em Expansão de Palavras-Chave, mostramos como configurar filtros smudge e clean no conteúdo que entra e sai do Git.

Por fim, praticamente toda a seção Configuração do Git é dedicada ao comando.

Comandos git config core.editor

Como complemento às instruções de configuração em Seu Editor, muitos editores podem ser configurados da seguinte maneira:

Table 4. Lista completa de comandos de configuração de `core.editor`

Editor	Comando de configuração
Atom	`git config --global core.editor "atom --wait"`
BBEdit (macOS, com ferramentas de linha de comando)	`git config --global core.editor "bbedit -w"`
Emacs	`git config --global core.editor emacs`
Gedit (Linux)	`git config --global core.editor "gedit --wait --new-window"`
Gvim (Windows de 64 bits)	`git config --global core.editor "'C:\Program Files\Vim\vim72\gvim.exe' --nofork '%*'"` (Veja também a nota abaixo)
Helix	`git config --global core.editor "hx"`
Kate (Linux)	`git config --global core.editor "kate --block"`
nano	`git config --global core.editor "nano -w"`
Notepad (Windows de 64 bits)	`git config core.editor notepad`
Notepad++ (Windows de 64 bits)	`git config --global core.editor "'C:\Program Files\Notepad++\notepad++.exe' -multiInst -notabbar -nosession -noPlugin"` (Veja também a nota abaixo)
Scratch (Linux)	`git config --global core.editor "scratch-text-editor"`
Sublime Text (macOS)	`git config --global core.editor "/Applications/Sublime\ Text.app/Contents/SharedSupport/bin/subl --new-window --wait"`
Sublime Text (Windows de 64 bits)	`git config --global core.editor "'C:\Program Files\Sublime Text 3\sublime_text.exe' -w"` (Veja também a nota abaixo)
TextEdit (macOS)	`git config --global core.editor "open --wait-apps --new -e"`
Textmate	`git config --global core.editor "mate -w"`
Textpad (Windows de 64 bits)	`git config --global core.editor "'C:\Program Files\TextPad 5\TextPad.exe' -m"` (Veja também a nota abaixo)
UltraEdit (Windows de 64 bits)	`git config --global core.editor Uedit32`
Vim	`git config --global core.editor "vim --nofork"`
Visual Studio Code	`git config --global core.editor "code --wait"`
VSCodium (binários de software livre/de código aberto do VSCode)	`git config --global core.editor "codium --wait"`
WordPad	`git config --global core.editor "'C:\Program Files\Windows NT\Accessories\wordpad.exe'"`

Editor	Comando de configuração
Xi	`git config --global core.editor "xi --wait"`



Se você usa um editor de 32 bits em um sistema Windows de 64 bits, o programa será instalado em `C:\Program Files (x86)\`, e não em `C:\Program Files\` como na tabela acima.

git help

O comando `git help` é usado para exibir toda a documentação distribuída com o Git sobre qualquer comando. Embora este apêndice ofereça uma visão geral aproximada da maioria dos comandos mais populares, você sempre pode executar `git help <command>` para obter uma lista completa de todas as opções e flags possíveis de cada comando.

Apresentamos o comando `git help` em Obtendo Ajuda e mostramos como usá-lo para encontrar mais informações sobre o `git shell` em Configurando o Servidor.

Obtendo e Criando Projetos

Há duas maneiras de obter um repositório Git. Uma é copiá-lo de um repositório existente na rede ou em outro local; a outra é criar um novo em um diretório existente.

git init

Para transformar um diretório em um novo repositório Git e começar a controlar suas versões, basta executar `git init`.

Apresentamos esse comando pela primeira vez em Obtendo um Repositório Git, onde mostramos como criar um repositório inteiramente novo para começar a trabalhar.

Em Branches Remotos (Remote Branches), falamos brevemente sobre como alterar o nome padrão do branch, deixando de usar “master”.

Em Colocando o Repositório Nu (Bare) em um Servidor, usamos esse comando para criar um repositório bare vazio para um servidor.

Por fim, em Encanamento e Porcelana (Plumbing and Porcelain), examinamos alguns detalhes do que ele realmente faz nos bastidores.

git clone

O comando `git clone` funciona, na verdade, como uma espécie de wrapper para vários outros comandos. Ele cria um novo diretório, entra nele e executa `git init` para transformá-lo em um repositório Git vazio, adiciona um remote (`git remote add`) para a URL fornecida (chamado `origin` por padrão), executa `git fetch` desse repositório remoto e então faz checkout do commit mais recente no seu diretório de trabalho com `git checkout`.

O comando `git clone` é usado em dezenas de pontos ao longo do livro, mas listaremos apenas

alguns dos mais interessantes.

Ele é apresentado e explicado em Clonando um Repositório Existente, onde examinamos alguns exemplos.

Em Colocando o Git em um Servidor, vemos como usar a opção `--bare` para criar uma cópia de um repositório Git sem diretório de trabalho.

Em Empacotando (Bundling), nós o usamos para desempacotar um repositório Git armazenado em um bundle.

Por fim, em Clonando um Projeto com Submódulos, conhecemos a opção `--recurse-submodules`, que simplifica um pouco a clonagem de um repositório com submódulos.

Embora seja usado em muitos outros pontos do livro, esses são os casos mais singulares ou aqueles em que ele é usado de maneiras um pouco diferentes.

Snapshots Básicos

Para o fluxo básico de preparar conteúdo e fazer commit dele no histórico, são necessários apenas alguns comandos básicos.

git add

O comando `git add` adiciona conteúdo do diretório de trabalho à área de preparação (ou "index") para o próximo commit. Por padrão, quando o comando `git commit` é executado, ele considera apenas essa área de preparação; portanto, `git add` é usado para definir exatamente como deve ser o snapshot do próximo commit.

Esse é um comando extremamente importante no Git, mencionado ou usado dezenas de vezes neste livro. Examinaremos rapidamente alguns de seus usos específicos.

Apresentamos e explicamos `git add` em detalhes pela primeira vez em Rastreando Novos Arquivos.

Em Conflitos Básicos de Mesclagem (Merge Conflicts), mencionamos como usá-lo para resolver conflitos de merge.

Em Área de Stage Interativa, vemos como usá-lo para preparar interativamente apenas partes específicas de um arquivo modificado.

Por fim, em Objetos Tree, nós o reproduzimos em baixo nível para que você tenha uma ideia do que ele faz nos bastidores.

git status

O comando `git status` mostra os diferentes estados dos arquivos no diretório de trabalho e na área de preparação. Quais arquivos foram modificados e não estão preparados e quais estão preparados, mas ainda não foram commitados. Em sua forma normal, ele também mostra algumas orientações básicas sobre como mover arquivos entre esses estados.

Abordamos `status` pela primeira vez em Verificando o status dos seus arquivos, tanto em sua forma básica quanto na simplificada. Embora seja usado ao longo de todo o livro, praticamente tudo o que você pode fazer com o comando `git status` é tratado ali.

git diff

O comando `git diff` é usado quando você quer ver as diferenças entre duas árvores quaisquer. Pode ser a diferença entre seu ambiente de trabalho e a área de preparação (apenas `git diff`), entre a área de preparação e o último commit (`git diff --staged`) ou entre dois commits (`git diff master branchB`).

Vemos os usos básicos de `git diff` pela primeira vez em Visualizando suas Alterações Preparadas e Não Preparadas, onde mostramos como identificar quais alterações estão preparadas e quais ainda não estão.

Em Diretrizes de Commit, nós o usamos com a opção `--check` para procurar possíveis problemas de espaço em branco antes do commit.

Em Determinando o Que é Introduzido, vemos como conferir as diferenças entre branches com mais eficiência usando a sintaxe `git diff A…B`.

Em Merging Avançado, nós o usamos para ignorar diferenças de espaço em branco com `-b` e mostramos como comparar diferentes estágios de arquivos em conflito com `--theirs`, `--ours` e `--base`.

Por fim, em Começando com Submódulos, nós o usamos com `--submodule` para comparar alterações em submódulos de maneira eficaz.

git difftool

O comando `git difftool` simplesmente inicia uma ferramenta externa para mostrar a diferença entre duas árvores, caso você queira usar algo diferente do comando `git diff` integrado.

Nós o mencionamos apenas brevemente em Visualizando suas Alterações Preparadas e Não Preparadas.

git commit

O comando `git commit` pega todo o conteúdo de arquivos preparado com `git add`, registra um novo snapshot permanente no banco de dados e então move até ele o ponteiro do branch atual.

Abordamos os fundamentos de fazer commits pela primeira vez em Fazendo o Commit de suas Alterações. Ali também demonstramos como usar a flag `-a` para pular a etapa de `git add` nos fluxos de trabalho cotidianos e como usar a flag `-m` para fornecer uma mensagem de commit na linha de comando em vez de abrir um editor.

Em Desfazendo Coisas, abordamos o uso da opção `--amend` para refazer o commit mais recente.

Em Branches em Poucas Palavras, entramos em muito mais detalhes sobre o que `git commit` faz e por que funciona dessa maneira.

Em Assinando Commits, vimos como assinar commits criptograficamente com a flag `-S`.

Por fim, em Objetos de Commit (Commit Objects), examinamos o que o comando `git commit` faz em segundo plano e como ele realmente é implementado.

git reset

O comando `git reset` é usado principalmente para desfazer coisas, como o verbo talvez já indique. Ele move o ponteiro `HEAD`, opcionalmente altera o `index` ou a área de preparação e também pode alterar o diretório de trabalho se você usar `--hard`. Essa última opção faz com que o comando possa eliminar seu trabalho se for usado incorretamente; portanto, certifique-se de entendê-lo antes de usá-lo.

Abordamos pela primeira vez o uso mais simples de `git reset` em Retirando a Preparação de um Arquivo Preparado (Unstaging), onde o usamos para retirar da área de preparação um arquivo no qual havíamos executado `git add`.

Depois, abordamos o comando em bastante detalhe em Reset Desmistificado, seção inteiramente dedicada a explicá-lo.

Em Abortando um Merge, usamos `git reset --hard` para abortar um merge; ali também usamos `git merge --abort`, que funciona como uma espécie de wrapper para o comando `git reset`.

git rm

O comando `git rm` é usado para remover arquivos da área de preparação e do diretório de trabalho do Git. Ele é semelhante a `git add`, pois prepara a remoção de um arquivo para o próximo commit.

Abordamos o comando `git rm` em algum detalhe em Removendo Arquivos, incluindo a remoção recursiva de arquivos e, com `--cached`, a remoção apenas da área de preparação, mantendo-os no diretório de trabalho.

O único outro uso diferente de `git rm` no livro aparece em Removendo Objetos, onde usamos e explicamos brevemente `--ignore-unmatch` ao executar `git filter-branch`; essa opção simplesmente evita um erro quando o arquivo que tentamos remover não existe. Isso pode ser útil em scripts.

git mv

O comando `git mv` é um pequeno atalho conveniente para mover um arquivo e então executar `git add` no novo arquivo e `git rm` no antigo.

Mencionamos esse comando apenas brevemente em Movendo Arquivos.

git clean

O comando `git clean` é usado para remover arquivos indesejados do diretório de trabalho. Isso pode incluir artefatos temporários de build ou arquivos de conflito de merge.

Em Limpando Seu Diretório de Trabalho, abordamos muitas das opções e situações em que você pode usar o comando clean.

Branches e Merges

Poucos comandos implementam a maior parte das funcionalidades de branch e merge do Git.

git branch

O comando `git branch` funciona, na verdade, como uma ferramenta de gerenciamento de branches. Ele pode listar seus branches, criar um novo branch, excluir branches e renomeá-los.

A maior parte de Ramificação (Branching) no Git é dedicada ao comando `branch`, que é usado ao longo de todo o capítulo. Nós o apresentamos pela primeira vez em Criando um Novo Branch e examinamos a maioria de seus outros recursos (listagem e exclusão) em Gerenciamento de Branches.

Em Branches de Rastreamento (Tracking Branches), usamos a opção `git branch -u` para configurar um branch de rastreamento.

Por fim, em Referências do Git, examinamos parte do que ele faz em segundo plano.

git checkout

O comando `git checkout` é usado para alternar entre branches e fazer checkout de conteúdo no diretório de trabalho.

Encontramos o comando pela primeira vez em Mudando de Branches, junto com o comando `git branch`.

Em Branches de Rastreamento (Tracking Branches), vemos como usá-lo para começar a rastrear branches com a flag `--track`.

Em Fazendo Checkout de Conflitos, nós o usamos com `--conflict=diff3` para reintroduzir conflitos em arquivos.

Em Reset Desmistificado, examinamos mais detalhadamente sua relação com `git reset`.

Por fim, em A HEAD, abordamos alguns detalhes de implementação.

git merge

A ferramenta `git merge` é usada para mesclar um ou mais branches no branch em que você fez checkout. Em seguida, ela avança o branch atual até o resultado do merge.

O comando `git merge` foi apresentado pela primeira vez em Ramificação Básica. Embora seja usado em vários pontos do livro, há poucas variações do comando `merge` — em geral, apenas `git merge <branch>` com o nome do único branch que você quer mesclar.

No final de Projeto Público Bifurcado (Forked), abordamos como fazer um merge squash (no qual o Git mescla o trabalho, mas o trata como um único novo commit, sem registrar o histórico do branch que está sendo mesclado).

Em Merging Avançado, examinamos muitos aspectos do processo e do comando de merge, incluindo o comando `-Xignore-space-change` e a flag `--abort` para abortar um merge problemático.

Em Assinando Commits, aprendemos a verificar assinaturas antes do merge quando o projeto usa assinaturas GPG.

Por fim, em Merging de Sub-árvore (Subtree Merging), aprendemos sobre merges de subárvores.

git mergetool

O comando `git mergetool` simplesmente inicia um auxiliar externo de merge caso você tenha problemas com um merge no Git.

Nós o mencionamos rapidamente em Conflitos Básicos de Mesclagem (Merge Conflicts) e detalhamos como implementar sua própria ferramenta externa de merge em Ferramentas Externas de Merge e Diff.

git log

O comando `git log` é usado para mostrar o histórico registrado e alcançável de um projeto, retrocedendo a partir do snapshot do commit mais recente. Por padrão, ele mostra apenas o histórico do branch atual, mas você pode fornecer heads ou branches diferentes, inclusive vários deles, como pontos de partida para percorrer o histórico. Ele também é usado com frequência para mostrar diferenças entre dois ou mais branches no nível dos commits.

Esse comando é usado em quase todos os capítulos do livro para demonstrar o histórico de um projeto.

Apresentamos o comando e o abordamos com alguma profundidade em Visualizando o Histórico de Commits. Ali examinamos as opções `-p` e `--stat` para ter uma ideia do que foi introduzido em cada commit e as opções `--pretty` e `--oneline` para visualizar o histórico de forma mais concisa, além de algumas opções simples de filtragem por data e autor.

Em Criando um Novo Branch, nós o usamos com a opção `--decorate` para visualizar facilmente a localização dos ponteiros dos branches e também com a opção `--graph` para ver como são históricos divergentes.

Em Equipe Pequena Privada e Intervalos (Ranges) de Commits, abordamos a sintaxe `branchA..branchB` para usar o comando `git log` e ver quais commits são exclusivos de um branch em relação a outro. Em Intervalos (Ranges) de Commits, examinamos isso em bastante detalhe.

Em Log de Merge (Merge Log) e Ponto Triplo (Triple Dot), abordamos o uso do formato `branchA...branchB` e da sintaxe `--left-right` para ver o que está em um branch ou no outro, mas não em ambos. Em Log de Merge (Merge Log), também vemos como usar a opção `--merge` para ajudar a depurar conflitos de merge, bem como a opção `--cc` para examinar conflitos de commits de merge no histórico.

Em Nomes Curtos do RefLog, usamos a opção `-g` para visualizar o reflog do Git por meio dessa ferramenta, em vez de percorrer branches.

Em Pesquisando (Searching), examinamos o uso das opções `-S` e `-L` para realizar buscas bastante sofisticadas por algo que ocorreu no histórico do código, como visualizar o histórico de uma função.

Em Assinando Commits, vemos como usar `--show-signature` para adicionar uma informação de validação a cada commit na saída de `git log`, conforme ele tenha ou não uma assinatura válida.

git stash

O comando `git stash` é usado para armazenar temporariamente trabalho não commitado, permitindo limpar o diretório de trabalho sem precisar fazer commit de trabalho inacabado em um branch.

Isso é abordado praticamente por completo em Fazendo Stash e Limpando.

git tag

O comando `git tag` é usado para criar um marcador permanente em um ponto específico do histórico do código. Em geral, isso é usado para itens como releases.

Esse comando é apresentado e abordado em detalhes em Criando Tags, e nós o usamos na prática em Adicionando Tags aos Seus Lançamentos.

Em Assinando Seu Trabalho, também abordamos como criar uma tag assinada com GPG usando a flag `-s` e como verificá-la com a flag `-v`.

Compartilhando e Atualizando Projetos

Não há muitos comandos do Git que acessam a rede; quase todos operam no banco de dados local. Quando você estiver pronto para compartilhar seu trabalho ou obter alterações de outro local, há alguns comandos que lidam com repositórios remotos.

git fetch

O comando `git fetch` se comunica com um repositório remoto, busca todas as informações presentes nele que ainda não estão no repositório atual e as armazena no banco de dados local.

Vemos esse comando pela primeira vez em Buscando (fetching) e Extraindo (pulling) de Seus Remotos e encontramos outros exemplos de seu uso em Branches Remotos (Remote Branches).

Também o usamos em vários exemplos de Contribuindo com um Projeto.

Em Refs de Pull Request, nós o usamos para buscar uma única referência específica fora do namespace padrão; em Empacotando (Bundling), vemos como fazer fetch a partir de um bundle.

Em Refspec, configuramos refspecs altamente personalizados para fazer `git fetch` se comportar de maneira um pouco diferente do padrão.

git pull

O comando `git pull` é basicamente uma combinação de `git fetch` e `git merge`: o Git faz fetch do

remote especificado e tenta mesclar imediatamente o resultado no branch atual.

Nós o apresentamos rapidamente em Buscando (fetching) e Extraindo (pulling) de Seus Remotos e mostramos como ver o que será mesclado ao executá-lo em Inspecionando um Remoto.

Em Rebase Quando Você Fizer Rebase, também vemos como usá-lo para ajudar com dificuldades de rebase.

Em Fazendo Checkout de Branches Remotos, mostramos como usá-lo com uma URL para obter alterações de forma pontual.

Por fim, em Assinando Commits, mencionamos brevemente que você pode usar a opção `--verify-signatures` para verificar se os commits obtidos por pull foram assinados com GPG.

git push

O comando `git push` é usado para se comunicar com outro repositório, calcular o que existe no banco de dados local e não existe no remoto e então enviar a diferença ao outro repositório. Ele exige acesso de gravação ao outro repositório e, por isso, normalmente requer algum tipo de autenticação.

Vemos o comando `git push` pela primeira vez em Enviando (pushing) para seus Remotos. Ali abordamos os fundamentos de fazer push de um branch para um repositório remoto. Em Enviando (Pushing), aprofundamos um pouco o push de branches específicos e, em Branches de Rastreamento (Tracking Branches), vemos como configurar branches de rastreamento para fazer push automaticamente. Em Excluindo Branches Remotos, usamos a flag `--delete` para excluir um branch no servidor com `git push`.

Ao longo de Contribuindo com um Projeto, vemos vários exemplos de uso de `git push` para compartilhar trabalho em branches por meio de vários remotes.

Em Compartilhando Tags, vemos como usá-lo com a opção `--tags` para compartilhar as tags que você criou.

Em Publicando Modificações do Submódulo, usamos a opção `--recurse-submodules` para verificar se todo o trabalho dos submódulos foi publicado antes de fazer push do superprojeto, o que pode ser muito útil ao usar submódulos.

Em Outros Hooks de Cliente, falamos brevemente sobre o hook `pre-push`, um script que podemos configurar para ser executado antes da conclusão de um push e verificar se ele deve ser permitido.

Por fim, em Submetendo Refspecs (Pushing), vemos como fazer push com um refspec completo em vez dos atalhos gerais usados normalmente. Isso permite especificar com muita precisão qual trabalho você deseja compartilhar.

git remote

O comando `git remote` é uma ferramenta para gerenciar seu registro de repositórios remotos. Ele permite salvar URLs longas com nomes curtos, como “origin”, para que você não precise digitá-las o tempo todo. Você pode ter vários desses registros, e o comando `git remote` é usado para adicioná-

los, alterá-los e excluí-los.

Esse comando é abordado em detalhes em Trabalhando com Repositórios Remotos, incluindo como listar, adicionar, remover e renomear remotes.

Ele também é usado em quase todos os capítulos seguintes do livro, mas sempre no formato padrão `git remote add <name> <url>`.

git archive

O comando `git archive` é usado para criar um arquivo contendo um snapshot específico do projeto.

Em Preparando um Lançamento (Release), usamos `git archive` para criar um tarball de um projeto a ser compartilhado.

git submodule

O comando `git submodule` é usado para gerenciar repositórios externos dentro de um repositório comum. Eles podem conter bibliotecas ou outros tipos de recursos compartilhados. O comando `submodule` possui vários subcomandos (`add`, `update`, `sync` etc.) para gerenciar esses recursos.

Esse comando só é mencionado e inteiramente abordado em Submódulos.

Inspeção e Comparação

git show

O comando `git show` pode exibir um objeto Git de maneira simples e legível para pessoas. Normalmente, você o usaria para mostrar informações sobre uma tag ou um commit.

Nós o usamos pela primeira vez em Tags Anotadas (Annotated Tags) para exibir informações de tags anotadas.

Mais adiante, nós o usamos bastante em Seleção de Revisão para mostrar os commits aos quais nossas várias seleções de revisão são resolvidas.

Um dos usos mais interessantes de `git show` aparece em Re-merging Manual de Arquivos, onde extraímos conteúdo específico de arquivos em vários estágios durante um conflito de merge.

git shortlog

O comando `git shortlog` é usado para resumir a saída de `git log`. Ele aceita muitas das mesmas opções que o comando `git log`, mas, em vez de listar todos os commits, apresenta um resumo deles agrupado por autor.

Em O Shortlog, mostramos como usá-lo para criar um bom changelog.

git describe

O comando `git describe` recebe qualquer coisa que possa ser resolvida em um commit e produz uma string razoavelmente legível que não mudará. É uma maneira de obter uma descrição do commit tão inequívoca quanto seu SHA-1, mas mais compreensível.

Em Gerando um Número de Build e Preparando um Lançamento (Release), usamos `git describe` para obter uma string com a qual nomear nosso arquivo de lançamento.

Depuração

O Git possui alguns comandos que ajudam a depurar um problema no código. Eles permitem desde descobrir onde algo foi introduzido até identificar quem o introduziu.

git bisect

A ferramenta `git bisect` é extremamente útil para depuração e permite descobrir qual commit específico introduziu primeiro um bug ou problema por meio de uma busca binária automática.

Ela é inteiramente abordada em Pesquisa Binária (Binary Search) e só é mencionada nessa seção.

git blame

O comando `git blame` anota cada linha de um arquivo com o commit que a alterou por último e com a pessoa que criou esse commit. Isso ajuda a encontrar a pessoa a quem pedir mais informações sobre uma seção específica do código.

Ele é abordado em Anotação de Arquivo e só é mencionado nessa seção.

git grep

O comando `git grep` pode ajudar a encontrar qualquer string ou expressão regular em qualquer arquivo do código-fonte, até mesmo em versões antigas do projeto.

Ele é abordado em Git Grep e só é mencionado nessa seção.

Patches

Alguns comandos do Git se concentram no conceito de considerar os commits pelas alterações que introduzem, como se a sequência de commits fosse uma série de patches. Esses comandos ajudam a gerenciar seus branches dessa maneira.

git cherry-pick

O comando `git cherry-pick` é usado para pegar a alteração introduzida em um único commit do Git e tentar reintroduzi-la como um novo commit no branch atual. Isso pode ser útil para trazer individualmente apenas um ou dois commits de um branch, em vez de mesclar o branch e trazer todas as alterações.

O cherry-pick é descrito e demonstrado em Fluxos de Trabalho de Rebase e Cherry-Pick.

git rebase

O comando `git rebase` é basicamente um `cherry-pick` automatizado. Ele determina uma série de commits e então aplica cherry-pick a cada um deles, na mesma ordem, em outro local.

O rebase é abordado em detalhes em Rebase (Rebasing), incluindo os problemas de colaboração envolvidos ao fazer rebase de branches que já são públicos.

Em Substituir (Replace), nós o usamos na prática em um exemplo que divide o histórico em dois repositórios separados, usando também a flag `--onto`.

Em Rerere, examinamos como lidar com um conflito de merge durante um rebase.

Em Alterando Várias Mensagens de Commit, também o usamos em um modo interativo para scripts com a opção `-i`.

git revert

O comando `git revert` é essencialmente um `git cherry-pick` ao contrário. Ele cria um novo commit que aplica exatamente o oposto da alteração introduzida no commit selecionado, na prática desfazendo ou revertendo essa alteração.

Em Reverter o commit, usamos esse comando para desfazer um commit de merge.

E-mail

Muitos projetos Git, incluindo o próprio Git, são mantidos inteiramente por meio de listas de discussão. O Git possui várias ferramentas integradas que facilitam esse processo, desde a geração de patches que você pode enviar facilmente por e-mail até a aplicação desses patches a partir de uma caixa de correio.

git apply

O comando `git apply` aplica um patch criado com o comando `git diff` ou até mesmo com o GNU diff. Ele é semelhante ao que o comando `patch` faria, com algumas pequenas diferenças.

Em Aplicando Patches a partir de E-mail, demonstramos seu uso e as circunstâncias em que ele pode ser apropriado.

git am

O comando `git am` é usado para aplicar patches provenientes de uma caixa de entrada de e-mail, especificamente uma no formato mbox. Isso é útil para receber patches por e-mail e aplicá-los facilmente ao projeto.

Em Aplicando um Patch com `am`, abordamos o uso e o fluxo de trabalho de `git am`, incluindo as opções `--resolved`, `-i` e `-3`.

Também há vários hooks que você pode usar para ajudar no fluxo de trabalho de `git am`, todos abordados em Hooks do Fluxo de Trabalho de E-mail.

Em Notificações por E-mail, também o usamos para aplicar alterações de um Pull Request do GitHub formatadas como patch.

git format-patch

O comando `git format-patch` é usado para gerar uma série de patches no formato mbox, devidamente formatados para envio a uma lista de discussão.

Em Projeto Público Grande, examinamos um exemplo de contribuição para um projeto usando a ferramenta `git format-patch`.

git imap-send

O comando `git imap-send` envia uma caixa de correio gerada com `git format-patch` para uma pasta de rascunhos IMAP.

Em Projeto Público Grande, examinamos um exemplo de contribuição para um projeto por meio do envio de patches com a ferramenta `git imap-send`.

git send-email

O comando `git send-email` é usado para enviar por e-mail patches gerados com `git format-patch`.

Em Projeto Público Grande, examinamos um exemplo de contribuição para um projeto por meio do envio de patches com a ferramenta `git send-email`.

git request-pull

O comando `git request-pull` é usado simplesmente para gerar um modelo de corpo de mensagem a ser enviado por e-mail a alguém. Se você tem um branch em um servidor público e quer informar a alguém como integrar essas alterações sem enviar os patches por e-mail, pode executar esse comando e enviar a saída para a pessoa que deverá incorporar as alterações.

Em Projeto Público Bifurcado (Forked), demonstramos como usar `git request-pull` para gerar uma mensagem de pull.

Sistemas Externos

O Git inclui alguns comandos para integração com outros sistemas de controle de versão.

git svn

O comando `git svn` é usado para se comunicar, como cliente, com o sistema de controle de versão Subversion. Isso significa que você pode usar o Git para fazer checkout a partir de um servidor Subversion e enviar commits para ele.

Esse comando é abordado em profundidade em Git e Subversion.

git fast-import

Para outros sistemas de controle de versão ou para importar de praticamente qualquer formato, você pode usar `git fast-import` para mapear rapidamente o outro formato para algo que o Git possa registrar com facilidade.

Esse comando é abordado em profundidade em Um Importador Personalizado.

Administração

Se você administra um repositório Git ou precisa fazer um reparo de grande porte, o Git oferece vários comandos administrativos para ajudar.

git gc

O comando `git gc` executa a "garbage collection" no repositório, removendo arquivos desnecessários do banco de dados e empacotando os arquivos restantes em um formato mais eficiente.

Normalmente, esse comando é executado em segundo plano, embora você possa executá-lo manualmente se quiser. Examinamos alguns exemplos disso em Manutenção.

git fsck

O comando `git fsck` é usado para verificar se há problemas ou inconsistências no banco de dados interno.

Nós o usamos apenas uma vez, brevemente, em Recuperação de Dados para procurar objetos dangling.

git reflog

O comando `git reflog` percorre um log das posições ocupadas pelos heads dos seus branches durante o trabalho para encontrar commits que você possa ter perdido ao reescrever históricos.

Abordamos esse comando principalmente em Nomes Curtos do RefLog, onde mostramos seu uso normal e como usar `git log -g` para ver as mesmas informações no formato de saída de `git log`.

Em Recuperação de Dados, também examinamos um exemplo prático de recuperação de um branch perdido dessa maneira.

git filter-branch

O comando `git filter-branch` é usado para reescrever grandes quantidades de commits de acordo com determinados padrões, como remover um arquivo de todos eles ou filtrar todo o repositório até restar um único subdiretório para extrair um projeto.

Em Removendo um Arquivo de Todos os Commits, explicamos o comando e exploramos várias opções, como `--commit-filter`, `--subdirectory-filter` e `--tree-filter`.

Em Git-p4, nós o usamos para corrigir repositórios externos importados.

Comandos de Encanamento

Também encontramos no livro vários comandos de encanamento de baixo nível.

O primeiro deles é `ls-remote`, em Refs de Pull Request, que usamos para examinar as referências brutas no servidor.

Usamos `ls-files` em Re-merging Manual de Arquivos, Rerere e O Index para ter uma visão mais bruta da área de preparação.

Também mencionamos `rev-parse` em Referências de Branch, que recebe praticamente qualquer string e a transforma no SHA-1 de um objeto.

No entanto, a maioria dos comandos de encanamento de baixo nível que abordamos está em Git Internals (Por Dentro do Git), que é, em linhas gerais, o foco daquele capítulo. Procuramos evitar seu uso na maior parte do restante do livro.

Índice

@

$EDITOR, 350
$VISUAL
  see $EDITOR, 350
.gitignore, 352
.NET, 491
@{upstream}, 96
@{u}, 96

A

aliases, 62
Apache, 121
Apple, 492
archiving, 366
attributes, 360
autocorrect, 352

B

bash, 481
binary files, 360
BitKeeper, 13
branches, 64
  basic workflow, 71
  creating, 66
  deleting remote, 97
  diffing, 161
  long-running, 84
  managing, 80
  merging, 76
  remote, 87, 160
  switching, 67
  topic, 85, 157
  tracking, 95
  upstream, 95
build numbers, 170

C

C, 487
C#, 491
Cocoa, 492
color, 353
commit templates, 350
contributing, 133
  private managed team, 143
  private small team, 136
  public large project, 153
  public small project, 149
credentials, 343
crlf, 357
CVS, 10

D

difftool, 354
distributed git, 130
Dulwich, 498

E

editor
  changing default, 37
email, 155
  applying patches from, 157
excludes, 352, 433

F

files
  moving, 40
  removing, 38
forking, 132, 178

G

Git as a client, 382
git commands
  add, 29, 30, 31
  am, 158
  apply, 157
  archive, 170
  branch, 66, 80
  checkout, 67
  cherry-pick, 167
  clone, 27
    bare, 114
  commit, 36, 64
  config, 21, 23, 37, 62, 155, 349
  credential, 343
  daemon, 120
  describe, 170
  diff, 34
    check, 134
  fast-import, 424
  fetch, 54

fetch-pack, 457
filter-branch, 423
format-patch, 154
gitk, 473
gui, 473
help, 24, 120
http-backend, 121
init, 27, 30
bare, 115, 118
instaweb, 124
log, 41
merge, 74
squash, 153
mergetool, 79
p4, 409, 422
pull, 54
push, 54, 60, 93
rebase, 98
receive-pack, 456
remote, 52, 53, 55, 56
request-pull, 150
rerere, 168
send-pack, 456
shortlog, 171
show, 59
show-ref, 385
status, 28, 36
svn, 382
tag, 57, 58, 59
upload-pack, 457
git-svn, 382
GitHub, 172
API, 221
Flow, 178
organizations, 213
pull requests, 182
user accounts, 172
GitHub for macOS, 475
GitHub for Windows, 475
gitk, 473
GitLab, 125
GitWeb, 123
Go, 496
go-git, 496
GPG, 351
Graphical tools, 473
GUIs, 473

H

hooks, 368
post-update, 111

I

ignoring files, 32
Importing
from Mercurial, 419
from others, 424
from Perforce, 422
from Subversion, 416
integrating work, 163
Interoperation with other VCSs
Mercurial, 393
Perforce, 400
Subversion, 382
IRC, 24

J

Java, 492
JetBrains, 480
jgit, 492

K

keyword expansion, 363

L

libgit2, 487
line endings, 357
Linus Torvalds, 13
Linux, 13
installing, 18
log filtering, 46
log formatting, 43

M

macOS
installing, 18
maintaining a project, 157
master, 65
Mercurial, 393, 419
mergetool, 354
merging, 76
conflicts, 77
strategies, 368
vs. rebasing, 106
Migrating to Git, 416
Mono, 491

O

Objective-C, 492
origin, 88

P

pager, 351
Perforce, 10, 13, 400, 422
  Git Fusion, 401
policy example, 372
posh-git, 484
PowerShell, 484
protocols
  dumb HTTP, 110
  git, 112
  local, 108
  smart HTTP, 110
  SSH, 112
pulling, 96
pushing, 93
Python, 492, 498

R

rebasing, 97
  perils of, 102
  vs. merging, 106
references
  remote, 87
releasing, 170
rerere, 168
Ruby, 488

S

serving repositories, 108
  git protocol, 120
  GitLab, 125
  GitWeb, 123
  HTTP, 121
  SSH, 115
SHA-1, 15
shell prompts
  bash, 481
  PowerShell, 484
  zsh, 482
SSH keys, 116
  with GitHub, 173
staging area
  skipping, 38
Sublime Text, 480
Subversion, 10, 13, 131, 382, 416

T

tab completion
  bash, 481
  PowerShell, 484
  zsh, 482
tags, 57, 169
  annotated, 58
  lightweight, 58
  signing, 169

V

version control, 9
  centralized, 10
  distributed, 11
  local, 9
Visual Studio, 479
Visual Studio Code, 479

W

whitespace, 357
Windows
  installing, 19
workflows, 130
  centralized, 130
  dictator and lieutenants, 132
  integration manager, 131
  merging, 163
  merging (large), 165
  rebasing and cherry-picking, 167

X

Xcode, 18

Z

zsh, 482